

Programmerbar logik

Programmerbar logik

*Digital konstruktion i VHDL, från ett
grindnät till en CAN-nod i en FPGA*

Erik Pihl

Programmerbar logik

Copyright © 2026 QRTECH Academy.

Boken är satt ur kursens eget material: tjugo föreläsningar med appendix, deras övningar, de genomarbetade exemplen, grupprojektets specifikation och de två dokument som utgör kontraktet mot drivrutinskursen. Koden i boken, och all kod i det tillhörande kursrepot, får användas under MIT-licensen, som är repots licens:

<https://opensource.org/licenses/MIT>

Kursrepot håller föreläsningsmaterialet, de genomarbetade exemplen, samtliga utdelade testbänkar, byggskripten och Pythonkällorna till figurerna. Varje sökväg boken hänvisar till, till exempel `lectures/L06/serial_rx8/serial_rx8.vhd`, är en sökväg i det repot.

VHDL-koden i boken är VHDL-93, simulerad med GHDL och syntetiserad med Quartus Prime Lite. C++-koden i avsnitt [10.1](#) är C++17, kompilerad med GCC. DE0-CV, Cyclone V, Quartus, USB-Blaster, AVR och AVR32DB28 är varumärken som tillhör respektive ägare. CAN är standardiserat i ISO 11898; den kontroller boken bygger är en medvetet förenklad undervisningskonstruktion och ingen certifierbar implementation, och avsnitt [19.7](#) listar skillnaderna.

Text och figurer är på svenska. All kod, alla kommentarer i koden och all utskrift från verktyg och testbänkar är på engelska, precis som i kursmaterialet.

Satt i TeX Gyre Pagella och DejaVu Sans Mono med Lua^ATeX.

Denna utgåva: 13 september 2026.

Innehåll

Förord

xiii

I Digital konstruktion i VHDL

1	Kombinatorik och första VHDL	3
1.1	Digitala signaler och logiska grindar	3
1.2	Boolesk algebra som notation	4
1.3	Att bygga och simulera en krets i CircuitVerse	5
1.4	Precis så mycket VHDL att du kan läsa och skriva ett grindnät	6
1.5	Den kompletta modulen: or_gate	9
1.6	En bromsassistent, från krav till krets	10
1.7	Sammanfattning.	12
1.8	Övningar	13
2	Större nät, multiplexrar och submoduler	21
2.1	Karnaughdiagram: från sanningstabell till minimerat grindnät.	21
2.2	Att översätta Karnaughdiagrammets nät till VHDL.	25
2.3	std_logic_vector: mer än en ledning.	26
2.4	Multiplexrar	28
2.5	Multiplexrar i VHDL: process och case	30
2.6	Att bygga en design av submoduler	33
2.7	Att verifiera din VHDL med en testbänk	34
2.8	Sammanfattning.	38
2.9	Övningar	39
3	Sekvensnät	51
3.1	Från kombinatorik till sekvensnät	51
3.2	D-låset	52
3.3	D-vippan	53
3.4	Klockning	55

3.5	Register: vippor parallellt	56
3.6	Den synkrona processmallen i VHDL	56
3.7	Flankdetektering	58
3.8	Genomarbetat exempel: en D-vippa i VHDL	59
3.9	Genomarbetat exempel: flankdetekterad lysdiodsväxling	60
3.10	Sammanfattning.	62
3.11	Övningar	63
4	Metastabilitet och synkronisering	69
4.1	Problemet: en knapp är inte en testbänkssignal	69
4.2	Metastabilitet: vad som faktiskt händer inuti vippan	70
4.3	Dubbelvippsynkroniseraren	71
4.4	Intuition för tidsförhållandena: varför "nästan säkert" duger	73
4.5	Att synkronisera en resetsignal: aktivera asynkront, släpp synkront	73
4.6	Att återanvända kedjan för flankdetektering, och studsfiltrering i förbigående	74
4.7	Vad en riktig studsfiltrering i hårdvara tillför	75
4.8	Generics: en modul, flera storlekar	76
4.9	Genomarbetat exempel: led_toggle_sync	77
4.10	Sammanfattning.	80
4.11	Övningar	82
5	Variabler och hårdvaran under	93
5.1	Vad som återstår	93
5.2	variable: processlokal lagring som uppdateras direkt.	94
5.3	Vad FPGA:n faktiskt består av	97
5.4	Varför en klocka har en hastighetsgräns.	98
5.5	Låset du får av misstag.	99
5.6	Sammanfattning.	99
5.7	Övningar	100
6	Räknare och skiftregister	109
6.1	Från register till räknare	109
6.2	Skiftregister	112
6.3	Serie in, parallell ut (SIPO)	113
6.4	Parallell in, serie ut (PISO)	114
6.5	Genomarbetat exempel: en 8-bitars seriemottagare	114
6.6	På kortet: att se en räknare slå över	118
6.7	På kortet: att skifta in en byte för hand	119
6.8	Sammanfattning.	120

6.9	Övningar	121
7	Timers	129
7.1	Timers: en räknare med ett målvärde	129
7.2	Att bygga en timer för hand i CircuitVerse	130
7.3	Modulen timer i VHDL	132
7.4	Att komponera en komplett timerkrets	133
7.5	walking_led: ett skiftregister med vandrande lysdiod	136
7.6	Sammanfattning.	138
7.7	Övningar	139
8	Tillståndsmaskiner	149
8.1	Vad en ändlig tillståndsmaskin är	149
8.2	Att konstruera en Mooremaskin för hand.	150
8.3	Från grindnät till CircuitVerse	153
8.4	Samma maskin i VHDL	154
8.5	Mealymaskiner: skillnaden på en klockcykel.	160
8.6	På kortet	163
8.7	Sammanfattning.	164
8.8	Övningar	166
9	Praktisk tentamen 1: sekvensnät	183
9.1	Upplägg	183
9.2	Hjälpmedel.	184
9.3	Före tentamen	184
9.4	Det här mäter tentamen	184
9.5	Efter tentamen.	185
II	Grupprojektet: en CAN-nod	
10	Projektstart: CAN-bussen, ramen och can_def	189
10.1	Ramar och serialisering	190
10.2	Introduktion till CAN	196
10.3	Broadcast, inte adressering	198
10.4	CAN-ramformatet	199
10.5	Bitstoppning	201
10.6	Arbitrering, genomräknad i sin helhet	202
10.7	Bittajming och sampelpunkten	202
10.8	Paketet can_def	203
10.9	Sammanfattning.	205

10.10	Övningar	206
11	Arkitektur, toppnivå och registerkartan	215
11.1	Kontrollerns arkitektur.	215
11.2	Att följa en ram genom blocken.	217
11.3	Kontrollerns gränssnitt.	219
11.4	Att skriva <code>can_controller.vhd</code>	221
11.5	Vad kontrollern synkroniserar, och varför det är kontrollerns jobb. . .	223
11.6	En första titt på registerkartan	224
11.7	Sammanfattning.	224
11.8	Övningar	225
12	Synkronisering och bittimern	229
12.1	Att konstruera <code>meta_prev</code>	229
12.2	Att konstruera <code>bit_timer</code>	232
12.3	Sammanfattning.	235
12.4	Övningar	236
13	CRC-15-motorn	241
13.1	Gränssnitt	241
13.2	Rekursionen	242
13.3	En motor, två jobb.	243
13.4	Ett genomräknat exempel, för hand	244
13.5	Vad testbänken låser fast.	244
13.6	Att lägga in den i <code>can_controller</code>	245
13.7	Sammanfattning.	245
13.8	Övningar	246
14	Sändningsskiftregister och bitstoppning	249
14.1	Gränssnitt	249
14.2	Vad modulen måste minnas.	251
14.3	Processen, uppifrån och ned	252
14.4	Utgångarnas tajming, den del testbänken låser exakt.	256
14.5	En grupp, presentation för presentation	256
14.6	Vad testbänken låser fast, och att lägga in modulen.	258
14.7	Sammanfattning.	258
14.8	Övningar	259
15	Mottagningsskiftregister och avstoppning	263
15.1	Gränssnitt	263
15.2	Vad modulen måste minnas.	265

15.3	Processen, uppifrån och ned	266
15.4	En grupp, sampel för sampel	269
15.5	Vad testbänken låser fast.	270
15.6	Att lägga in den i <code>can_controller</code>	271
15.7	Sammanfattning.	271
15.8	Övningar	273
16	CAN-kontrollern I: tillståndsmaskinen och sändvägen	275
16.1	En nod, två roller	276
16.2	Ramen, fält för fält	278
16.3	Vad du lägger till i arkitekturen	280
16.4	Den kombinatoriska processen	282
16.5	Den sekventiella processen	285
16.6	Att fånga CRC:n	290
16.7	Att mata <code>crc15</code>	291
16.8	Att nollställa <code>crc15</code>	291
16.9	Sammanfattning.	292
16.10	Övningar	293
17	CAN-kontrollern II: mottagning och arbitrering	297
17.1	Att läsa tillbaka bussen.	297
17.2	Mottagarvägen, tillstånd för tillstånd	298
17.3	Ett rent bord mellan ramar	300
17.4	Att mata <code>crc15</code> på mottagarsidan	301
17.5	Väntan på ett extra sampel: den avgörande tajminglärdomen	302
17.6	Detektering av förlorad arbitrering	303
17.7	Att färdigställa processerna	304
17.8	Att verifiera den: <code>can_controller_tb</code>	311
17.9	Sammanfattning.	311
17.10	Övningar	312
18	Registerbanken och SPI från vågformen	315
18.1	Varför det här lagret finns	315
18.2	Gränssnittet	316
18.3	De tre STATUS-bitarna: en låsning var	317
18.4	<code>TX_SEND</code> : en skrivning blir en puls	318
18.5	<code>TX_ABORT</code> : nödutgången	318
18.6	<code>RX</code> -sidan: infångning vid pulsen	319
18.7	Maskning vid skrivning	319
18.8	Att skriva <code>register_bank.vhd</code>	320

18.9	SPI från vågformen och uppåt	321
18.10	Sammanfattning.	326
18.11	Övningar	327
19	SPI-bryggan, can_spi_node och bring-up	331
19.1	Att verifiera hela systemet.	332
19.2	Transaktionsbryggan spi_reg_bridge.	334
19.3	Att konstruera can_spi_node	336
19.4	Att få ut bussen på en ledning	338
19.5	Valfritt: en wrapper på kortnivå för felsökning	338
19.6	Bring-up	340
19.7	Hur produktionsklara CAN-kontrollrar går längre	342
19.8	Konstruktionen, sedd i backspegeln	343
19.9	Sammanfattning.	345
19.10	Övningar	346
20	Praktisk tentamen 2: CAN-moduler	353
20.1	Varför en individuell tentamen på ett grupparbete	353
20.2	Upplägg	354
20.3	Hjälpmedel.	355
20.4	Före tentamen	355
20.5	Det här mäter tentamen	355
20.6	Efter kursen	356
A	Simulering med GHDL	357
A.1	Varför simulera	357
A.2	Att installera GHDL	357
A.3	Var filerna bor	358
A.4	De tre GHDL-stegen	358
A.5	Att bygga allt på en gång	360
A.6	Att städa upp	361
B	Quartus Prime Lite och DE0-CV	363
B.1	Vad som behövs	363
B.2	Att installera Quartus Prime Lite	364
B.3	Att skapa ett nytt projekt	365
B.4	Att lägga till sin VHDL.	365
B.5	Att tilldela pinnar i Pin Planner.	366
B.6	Att kompilera konstruktionen.	367
B.7	Att programmera DE0-CV.	367

B.8	Felsökning	368
B.9	Att programmera från Linux och från WSL.	368
C	Gruppprojektet	371
C.1	Arbetsform.	371
C.2	Så här drivs projektet.	372
C.3	Projektstruktur	373
C.4	Namnkonventioner och kodstil.	373
C.5	Det delade paketet <code>can_def</code>	374
C.6	Modulerna i <code>controller/</code>	374
C.7	Modulerna i <code>bridge/</code>	378
C.8	Testplan.	379
C.9	Examination och betyg.	381
C.10	Praktiska tips	383
D	Registerkartan och SPI-protokollet	385
D.1	Registren	386
D.2	<code>STATUS</code>	386
D.3	Semantik, register för register.	387
D.4	Databyteordning	388
D.5	Motsvarigheter i hårdvaran	388
D.6	Elektriska parametrar och ramning	389
D.7	Transaktioner	390
D.8	Tidskrav	391
D.9	Genomarbetat exempel.	392
E	Vidare läsning	395
E.1	Grunderna	395
E.2	Protokollreferensen	396
E.3	Standarder och specifikationer	397
E.4	Verktygen.	397
E.5	Vidare i digital konstruktion	397

Förord

Den här boken tar dig från ett grindnät på ett papper till en CAN-nod i en FPGA som en mikrokontroller kan prata med över SPI.

Första halvan bygger upp VHDL och synkron konstruktion från grunden: kombinatoriska nät, sekvensnät, metastabilitet och synkronisering, räknare, skiftregister, timers och tillståndsmaskiner, var och en först resonerad om som en *krets* och först därefter uttryckt i språket. Andra halvan är ett grupprojekt där den kunskapen används till något större: en förenklad CAN-kontroller, konstruerad modul för modul, och de två lager som gör den nåbar utifrån — en registerbank och en SPI-brygga.

Ämnet är **VHDL och digital konstruktion i FPGA, inte digitalteknikens grunder**. Grindar, sanningstabeller, boolesk algebra och Karnaughdiagram repeteras snarare än lärs ut, och de repeteras där språket kräver det. Vad boken lär ut från noll är språket och verktygskedjan.

Det här förutsätts du ha med dig

- Logiska grindar och sanningstabeller, och att läsa en summa-av-produkter ur en tabell.
- Boolesk algebra: de grundläggande identiteterna och De Morgans lagar.
- Karnaughdiagram, för funktioner av tre eller fyra variabler.
- Register, D-vippor, och vad en klockflank gör.
- Tillståndsmaskiner som begrepp: tillstånd, övergångar, och skillnaden mellan Moore och Mealy.

Behöver något av det fräschas upp står allt i *Digital Electronics* (<https://github.com/qrotech-academy/digital-electronics>), en fritt tillgänglig bok på engelska som går från en enda grind till en tillståndsmaskin: grindar och boolesk algebra i kapitel 1 och 2, Karnaughdiagram i kapitel 3, vippor och register i kapitel 5 och tillståndsmaskiner i kapitel 6. Kapitlen i den här boken hänvisar dit där de repeterar grunderna, och avsnitt E.1 har hela kartan.

Inget programmeringsspråk utöver allmän mjukvaruvana förutsätts. Vana vid C eller något liknande hjälper, eftersom flera förklaringar ställer VHDL:s semantik mot den, och det enda avsnitt som faktiskt innehåller C++ (avsnitt 10.1) är en kort inledning till ramning snarare än ett moment i språket.

Till grupprojektet tillkommer **Git och GitHub** på grundnivå: klona, branch, commit, push och Pull Request.

Det här ska du kunna efteråt

- Realisera en boolesk funktion som ett grindnät, och simulera den för hand.
- Läsa och skriva kombinatorisk och sekventiell VHDL: entiteter, arkitekturer, signaler, variabler och processer.
- Förklara metastabilitet och tillämpa dubbelvippsynkroniseraren på varje asynkron ingång.
- Konstruera räknare, skiftregister, timers och tillståndsmaskiner, och komponera en design av moduler du redan byggt i stället för att skriva om deras logik.
- Konstruera ett kommunikationsprotokolls hårdvara: bittajming, checksummor, bitstoppning, ramsekvensering och arbitrerung.
- Konstruera gränssnittet mellan hårdvara och mjukvara: en registerbank och en SPI-transport, mot ett kontrakt någon annan skriver mot.
- Verifiera en design genom att köra en självkontrollerande testbänk under GHDL, och spåra ett fel tillbaka till koden som orsakade det.
- Arbeta i en grupp med feature-branchar, Pull Requests och kodgranskning.

Hur boken är upplagd

Boken är satt ur en kurs om tjugo pass, och **varje kapitel är ett pass**. Kapitlets avsnitt är passets appendix, i samma ordning och med samma underrubriker, så att ett kapitel här och en föreläsningskatalog i kursrepot går att läsa sida vid sida. De två delarna motsvarar kursens två halvår:

Kapitel	Pass	Ämne
<i>Del I: digital konstruktion i VHDL</i>		
1	L01	Kombinatorik och första VHDL
2	L02	Större nät, multiplexrar och submoduler
3	L03	Sekvensnät
4	L04	Metastabilitet och synkronisering
5	L05	Variabler och hårdvaran under
6	L06	Räknare och skiftregister
7	L07	Timers
8	L08	Tillståndsmaskiner
9	L09	Praktisk tentamen 1: sekvensnät
<i>Del II: gruppprojektet, en CAN-nod</i>		
10	L10	Projektstart: CAN-bussen, ramen och can_def
11	L11	Arkitektur, toppnivån och registerkartan
12	L12	Synkronisering och bittimern
13	L13	CRC-15-motorn
14	L14	Sändningsskiftregister och bitstoppning
15	L15	Mottagningsskiftregister och avstoppning
16	L16	CAN-kontrollern I: tillståndsmaskinen och sändvägen
17	L17	CAN-kontrollern II: mottagning och arbitrering
18	L18	Registerbanken och SPI från vågformen
19	L19	SPI-bryggan, can_spi_node och bring-up
20	L20	Praktisk tentamen 2: CAN-moduler

Varje kapitel öppnar med vad som finns i det, arbetar sig igenom materialet, avslutas med en genomgång av vad du ska kunna förklara efteråt, och slutar med sina övningar. Bilagorna håller verktygsreferenserna, projektspecifikationen, det delade kontraktet mot drivrutinskursen, och en avslutande not om var man läser vidare.

Kapitel 9 och 20 är de två examinationstillfällena. De innehåller ingen ny teori; de säger vad tentamen mäter, vilken form uppgifterna har och hur man förbereder sig, och de är värda att läsa i förväg snarare än dagen innan.

En bok till hör till del II. Den här boken beskriver hur man *bygger* en CAN-nod; protokollet självt beskrivs i *CAN: bussen, framen och kontrollern*, en kort bok skriven för båda halvorna av ett CAN-system, på svenska och på engelska med samma kapitelnumrering. Den är protokollreferensen genom hela andra halvan: dess kapitel 1–6 är kravspecifikationen för det som byggs här, bit för bit, och dess kapitel 7 beskriver precis den förenklade kontroller del II landar i. Varje kapitel som lägger till en protokollregel säger var i den boken regeln står, och avsnitt E.2 har hela kapitelkartan mellan de två. Läs gärna dess kapitel 1 och 2 innan kapitel 10; resten fungerar bäst som uppslagsverk.

Tre lager av verifiering

Kursen kontrollerar sitt arbete på tre nivåer, och att hålla isär dem är en poäng i sig:

Lager	Vad det kontrollerar
För hand	I CircuitVerse, innan någon VHDL finns. Du härleder vad kretsen ska göra, bygger den, och simulerar den mot din egen förutsägelse. Ingenting automatiskt kontrollerar det steget, medvetet: att förutsäga ett beteende och sedan bekräfta det är den färdighet resten av boken vilar på.
I övningarna	Du skriver VHDL:en och kör en utdelad självkontrollerande testbänk under GHDL. Det behövs inget FPGA-kort, och nästan allt går att verifiera på en vanlig laptop.
På kortet	Designen syntetiseras i Quartus Prime Lite och körs på ett Terasic DE0-CV. Under den första halvan demonstreras det steget; i grupprojektets sista pass gör du det själv.

Avsnitt 2.7 är den fullständiga introduktionen till testbänkar, och bilaga A är referensen du kommer tillbaka till. Bilaga B är Quartus-flödet.

Vad övningarna ser ut som

Övningarna kommer efter varje kapitel, numrerade inom kapitlet, och var och en är märkt med sin sort:

Sort	Vad den ber dig om
KRETS	Rita, bygg och simulera en krets för hand i CircuitVerse, och stäm av den mot din egen sanningstabell eller ditt eget tidsdiagram.
VHDL	Skriv en modul, med det entitetsnamn och den portordning övningen anger, och kör dess utdelade testbänk.
SIMULERING	Ändra något och ta reda på vad GHDL rapporterar. Flera av dem ber dig först <i>förutsäga</i> utfallet och sedan köra.
RESONEMANG	Svara i löpande text eller i siffror: vad koden gör, varför en konstruktion är bättre än en annan, eller vad ett tal blir.
C++	Kursens enda C++-moment, i kapitel 10.

Det finns inga tryckta lösningar, och det är avsiktligt. Varje övning som ber dig skriva en VHDL-modul har en utdelad självkontrollerande testbänk, och den är facit: den driver din modul genom varje fall och säger exakt vilken kontroll som fallerade. Övningarna med penna och papper kontrollerar du mot den sanningstabell eller det tidsdiagram du själv härledde, vilket är hela poängen med dem. En lista med svar skulle ta bort just det steg övningen finns till för.

Portordningen är kontraktet. Varje utdelad testbänk binder till modulen den driver **positionellt**, och det gör varje instansiering i kursen också. Ingenting binder en port vid namn. Kontraktet är alltså *ordningen* och *typerna* på portarna, exakt som varje övnings gränssnittstabell listar dem, uppifrån och ned. Portarnas *namn* är dina.

Den affären är värd att förstå snarare än bara följa. Positionell bindning är kortare att skriva och att läsa, och den gör gränssnittstabellen till enda sanningskälla. Det den ger upp är kompilatorns hjälp: byt plats på två portar av samma typ och ingenting klagar vid analys, utan designen beter sig helt enkelt fel i simuleringen i stället. Boken behåller ändå sina portnamn, så att texten och din kod talar om samma signaler.

Att sätta upp verktygen

Allt i den här boken körs i en Linuxterminal: på Linux, eller på Windows genom WSL. Tre saker räcker för allt som inte är ett kort:

- **GHDL**, den öppna VHDL-analysatorn och -simulatorn. Det är verktyget kursen körs på: varje modul verifieras mot en testbänk i GHDL, och ingenting du ombeds producera behöver något annat.
- **CircuitVerse**, en gratis logiksimulator som körs i webbläsaren (<https://circuitverse.org/simulator>). Inget att installera.
- **Git**, till gruppprojektet, plus g++ till kapitel 10:s ramningsövning.

På WSL eller Ubuntu:

```
sudo apt -y update
sudo apt -y install git make ghdl g++
ghdl --version          # 3.x eller senare
```

Kursrepot har testramverket som en submodule, så kлона det med submodulerna:

```
git clone --recurse-submodules <repo-url>
git submodule update --init      # om det redan är klonat utan dem
```

Två kommandon i repots rot täcker det mesta boken ber om:

```
make build                # Bygg och simulera allt som finns: exempel, övningar och-
└─ projekt.
make lint                  # Markdownlänkar, dubblettkontroll, inga radslut med └─
└─ blanksteg.
```

`make build-project` rapporterar varje projekttestbänk som överhoppad i ett färskt utcheckat repo. Det är det förväntade tillståndet: `controller/` och `bridge/` innehåller bara de utdelade filerna, och modulerna är gruppens att skriva i sitt eget repo. En testbänk vars moduler saknas hoppas över, aldrig underkänns.

Quartus Prime Lite och ett DE0-CV-kort är rekommenderade men inte nödvändiga. Ingen övning och inget bedömt moment kräver dem; bilaga B beskriver flödet för den som

vill, och för gruppprojektets sista pass, där varje grupp tar upp sin egen nod på utdelad hårdvara.

Var man läser vidare

Den nod som byggs i del [II](#) är ena halvan av ett system. Den andra halvan, C++-koden som talar med den över SPI, skrivs i en annan kurs, mot samma registerkarta och samma protokollspecifikation — och mot samma protokollbok, som är gemensam för de två. Bilaga [E](#) beskriver den, standarderna konstruktionen förhåller sig till, och fyra riktningar vidare i ämnet.

Del I

Digital konstruktion i VHDL

Kombinatorik och första VHDL

Från en sanningstabell till ett fungerande grindnät, i CircuitVerse och i VHDL.

I DET HÄR KAPITLET

- Logiska grindar och sanningstabeller, och varför digital logik tål brus som analog inte gör.
- Boolesk algebra: att läsa en summa-av-produkter direkt ur en sanningstabell.
- Att bygga och simulera ett grindnät för hand i CircuitVerse.
- VHDL redan från första passet: `entity`, `architecture`, `std_logic` och den konkurrenta tilldelningen.
- En komplett modul, `or_gate`, från sanningstabell till syntetiserbar fil.
- En bromsassistent, härledd ur ett krav och demonstrerad på ett DE0-CV-kort.

Det här kapitlet går den väg varje krets i boken går: från ett krav, via en sanningstabell och en boolesk ekvation, till ett grindnät som byggs och simuleras för hand, och först därefter till VHDL. Ordningen är inte en pedagogisk omväg. Den är den arbetsgång resten av boken vilar på, och den är lättast att se på en krets som är liten nog att överblicka i sin helhet.

Arbeta kapitlet i ordning. Avsnitt 1.1 till 1.3 är kretsen; avsnitt 1.4 och 1.5 är precis så mycket VHDL att du kan läsa och skriva ett grindnät; avsnitt 1.6 är kravet passet bygger live, och det är värt att komma dit med ett eget svar att jämföra mot. Övningarna i avsnitt 1.8 är där du skriver koden själv.

Repetition: Grindarna, sanningstabellerna och den booleska algebran har du med dig från digitaltekniken. Behöver de fräschas upp står de, med övningar och svar, i avsnitt 1.1–1.4 och 2.1–2.7 i *Digital Electronics*, och dess bilaga B beskriver CircuitVerse.

1.1 Digitala signaler och logiska grindar

En digital signal antar ett av två värden, 0 eller 1, i stället för det kontinuerliga intervall en analog signal använder. Allt som ligger tillräckligt nära 0 eller 1 läses som exakt 0 eller 1, så små mängder elektriskt brus ändrar ingenting. Den brusimmuniteten är anledningen till att nästan all modern beräkning, från mikrokontroller till FPGA:er, är digital.

En **logisk grind** har en eller flera enbitarsingångar, en enbitarsutgång och en fast boolesk funktion mellan dem.

Kombinatorik, det här kapitlets ämne, producerar utgångar enbart utifrån de aktuella ingångarna, utan minne. **Sekvensnät** (kapitel 3) beror på de aktuella ingångarna *och* på tillstånd lagrat i vippror.

Varje grundläggande grind med två ingångar, plus den unära NOT:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

A	NOT A
0	1
1	0

Värt att lägga på minnet direkt ur tabellerna:

- NAND, NOR och XNOR är inverserna av AND, OR och XOR.
- XOR är 1 när ingångarna skiljer sig åt. Bortom två ingångar generaliseras det till **udda paritet**: 1 när ett udda antal ingångar är 1 (Övning 1.3).
- Varje boolesk funktion kan byggas av AND, OR och NOT, och även av enbart NAND eller enbart NOR.

1.2 Boolesk algebra som notation

Tre operatorer, som avbildas rakt av på grindarna ovan:

Grind	Notation i boolesk algebra
AND	multiplikation: A AND B skrivs AB
OR	addition: A OR B skrivs $A + B$
NOT	prim: NOT A skrivs A'

En funktion skriven som en OR av AND-termer, till exempel $X = AB' + CD$, står på **summa-av-produkter**-form.

Varje sanningstabell kan läsas av direkt som en summa av produkter:

1. Leta upp varje rad där utgången är 1.
2. Skriv för var och en en AND-term som innehåller samtliga ingångar: direkt om ingången är 1 på den raden, primmad om den är 0.

3. OR:a ihop de termerna.

Till exempel:

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Raden $AB = 01$ bidrar med $A'B$ och raden $AB = 10$ med AB' , vilket ger $X = A'B + AB'$. Stäm av mot tabellen: det är XOR, så samma nät är en enda XOR-grind i stället för två AND-grindar, två NOT-grindar och en OR.

Lagarna, som en påminnelse

Du har mött dem förut. De står här för att boken ska ha en enda notation för dem och för att senare kapitel ska kunna hänvisa till dem, inte för att de behöver läras ut:

Lag	Med OR	Med AND
Identitet	$A + 0 = A$	$A \cdot 1 = A$
Dominans	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotens	$A + A = A$	$A \cdot A = A$
Komplement	$A + A' = 1$	$A \cdot A' = 0$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan	$(A + B)' = A'B'$	$(AB)' = A' + B'$

De Morgan är den lag boken lutar sig mot, mindre för minimering än som en regel för hur en inversion flyttas genom en grind: en OR med båda ingångarna inverterade *är* en NAND, och en AND med båda ingångarna inverterade *är* en NOR. Det är därför en NAND-grind kan bygga vad som helst, vilket Övning 1.6 låter dig skriva i VHDL, och det kommer tillbaka varje gång en aktivt låg signal möter logik skriven för aktivt hög. I en kurs där varje reset och varje knapp är aktivt låg är det ofta.

Glappet mellan en rakt avläst summa av produkter och det minsta nätet är vad ett **Karnaughdiagram** sluter, och avsnitt 2.1 går igenom ett från början till slut. Inte för att tekniken är ny för dig, utan för att det är den minimerade ekvationen som skrivs i VHDL, och kapitel 8 härleder sin tillståndsmaskins nästa-tillstånd-logik på precis det sättet.

1.3 Att bygga och simulera en krets i CircuitVerse

CircuitVerse (<https://circuitverse.org/simulator>) är en gratis, webbläsarbaserad logiksimulator. Varje grindnät i den här boken byggs och testas där för hand innan det skrivs i VHDL:

1. Öppna simulatören och starta ett nytt projekt.
2. Placera ett **input**-element per insignal och ett **output**-element per utsignal, omdöpta så att de matchar dina variabler.
3. Placera de grindar din ekvation kräver, från panelen *Gates*.
4. Koppla ihop nätet en grind i taget, enligt din härledda ekvation. Du ska redan veta exakt vilka grindar du behöver innan du öppnar simulatören, vilket är därför avsnitt 1.2 kommer först.
5. Slå om varje ingång och kontrollera att utgången stämmer med din sanningstabell på varje rad.
6. Spara via Project -> Save Online, eller Project -> Download as image/data för en återimporterbar .cv-fil.

Steg 5 är inte valfritt, och ingenting annat gör det åt dig. Tre regler gör det till en kontroll snarare än en formalitet:

- **Uttömmande, inte stickprov.** Två ingångar är fyra rader, tre är åtta. Vid de storlekarna finns ingen ursäkt för att sampla. För sekvensnäten längre fram blir det här "stega klockan och kontrollera vid varje flank", samma disciplin tillämpad på tid.
- **Härled först, simulera sedan.** Rita kretsen och bestäm *sedan* vad den ska göra, så kommer du att läsa av dina egna förväntningar ur simuleringen, vilket inte bevisar något.
- **En avvikelse är information.** Den säger att ritningen och ekvationen är oense, och att en av dem har fel.

Varje hårdvarubugg du kommer att jaga hittas genom att förutsäga ett beteende och sedan kontrollera det. Att göra det för hand på en krets med fyra grindar är hur du lär dig göra det på en du inte kan överblicka på en gång.

När konstruktionen väl är VHDL tar en självkontrollerande testbänk över det jobbet. Varje genomarbetat exempel och varje övning har en utdelad; du kör dem snarare än skriver dem, vilket avsnitt 2.7 går igenom.

1.4 Precis så mycket VHDL att du kan läsa och skriva ett grindnät

VHDL beskriver hårdvara, inte en sekvens av instruktioner. Varje sats nedan körs konkurrent och kontinuerligt, precis som grindarna den beskriver. Språket kommer allteftersom det behövs; det här avsnittet räcker för ett enkelt kombinatoriskt nät.

Varje VHDL-modul har två delar:

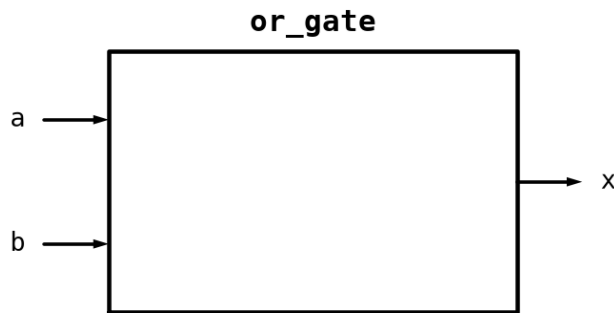
- En **entitet**, vyn utifrån: portarna, och ingenting om vad som händer inuti.
- En **arkitektur**, implementationen bakom den.

Den uppdelningen är ett kontrakt, inte ceremoni. Entiteten är allt en annan konstruktion behöver för att kunna använda modulen, vilket är det som låter en konstruktion instansiera

en annan utan att läsa den, och som låter dig skriva om en implementation utan att röra det som beror på den. Formen är en header vid sidan av en källfil, eller ett gränssnitt vid sidan av sin klass; skillnaden är att VHDL kräver den för varje modul, utan något sätt att hoppa över gränssnittet eller läcka implementationen genom det.

Ta `or_gate`: ingångarna `a`, `b`, utgången `x`, och $x = a + b$.

```
entity or_gate is
    port(a, b: in std_logic;
          x : out std_logic);
end entity;
```



Figur 1.1 Entiteten `or_gate`: vyn utifrån, och ingenting om vad som händer inuti.

`std_logic` är signaltypen för en enstaka bit. Den är inte inbyggd i VHDL utan kommer från paketet `std_logic_1164`, vilket är därför varje fil i den här boken inleds med:

```
library ieee;
use ieee.std_logic_1164.all;
```

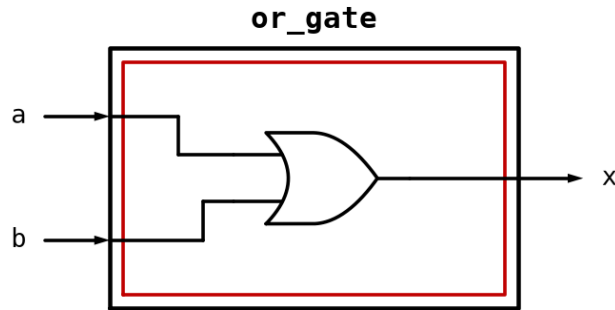
Den har nio värden i stället för två, eftersom verklig hårdvara behöver mer än sant och falskt:

- '0' och '1': driven låg och driven hög. De enda två du kommer att skriva i den här boken.
- 'Z': högimpedans, ingenting driver ledningen alls.
- 'U': oinitierad, det en signal håller innan något har tilldelat den ett värde.
- 'X': okänd, det en simulator visar när två saker driver samma signal och är oense.
- '-', 'W', 'L', 'H': don't-care-värden och svagt drivande värden, sällan skrivna för hand.

Att känna igen de sju du inte kommer att skriva spelar ändå roll: de är hur en simulator talar om för dig att något är fel. En signal som ligger kvar på 'U' eller 'X' är en bugg, inte ett värde.

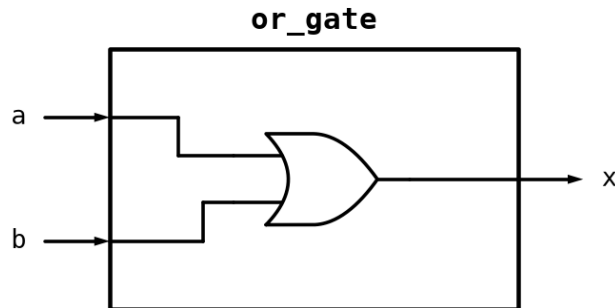
Arkitekturen fyller i lådan:

```
architecture behaviour of or_gate is
begin
    x <= a or b;
end architecture;
```



Figur 1.2 Arkitekturen or_gate: implementationen bakom entiteten.

Tillsammans utgör de den kompletta modulen:



Figur 1.3 Modulen or_gate: entitet och arkitektur tillsammans.

$x \leq a \text{ or } b$; är avsnitt 1.2:s booleska algebra nästan ordagrant: VHDL:s and, or, not och xor arbetar direkt på `std_logic`, så varje ekvation du härlett för hand blir VHDL med knappt någon översättning alls.

Tilldelningen är **konkurrent**. Den körs inte en gång; den gäller kontinuerligt, som en ledning mellan verkliga grindar.

Nyckelordet `signal` deklarerar en intern ledning inuti en arkitektur, till skillnad från en utifrån synlig port. Du använder en i Övning 1.7 för att namnge ett mellanresultat i stället för att nästla in det på plats, och kapitel 2 lutar sig mot samma idé rakt igenom.

1.5 Den kompletta modulen: or_gate

Avsnitt 1.4:s delar tillsammans blir en komplett, syntetiserbar fil. Det här är hela `or_gate.vhd`, och det är en riktig konstruktion som syntetiseras och körs på DE0-CV:

```
lectures/L01/or_gate/or_gate.vhd

-----
-- Two-input OR gate: x = a or b.
--
-- Inputs:
--     - a, b: OR inputs.
--
-- Outputs:
--     - x: OR output.
-----

library ieee;
use ieee.std_logic_1164.all;

entity or_gate is
    port(a, b: in std_logic;
         x : out std_logic);
end entity;

architecture behaviour of or_gate is
begin
    x <= a or b;
end architecture;
```

- `library/use`-satserna drar in `std_logic`.
- `entity` deklarerar utsidan: två ingångar, en utgång.
- `architecture` implementerar insidan, som en enda konkurrent tilldelning.

Det är den minsta kompletta VHDL-konstruktion som finns, och formen ändras aldrig. Varje modul i den här boken, ända upp till CAN-kontrollern i kapitel 17, är samma `library/entity/architecture`-skelett. Det som växer är arkitekturrampen, aldrig ramen.

Att kontrollera ditt arbete. `or_gate` har en utdelad `or_gate_tb.vhd`, som driver alla fyra ingångskombinationerna och kontrollerar utgången:

```
cd lectures/L01/or_gate
ghdl -a --std=93 or_gate.vhd or_gate_tb.vhd
ghdl -e --std=93 or_gate_tb
ghdl -r --std=93 or_gate_tb --assert-level=error --stop-time=10ms
```

Ingen utskrift utöver en avslutande notering betyder att varje kontroll gick igenom. Testbänkar behandlas ordentligt i avsnitt 2.7; tills vidare, betrakta de här tre kommandona som sättet du bekräftar att en modul fungerar på.

1.6 En bromsassistent, från krav till krets

Passet bygger en annan krets än `or_gate`, live, så att du får se samma väg gås två gånger: en gång här i skrift, och en gång på något du inte redan läst svaret på.

Kravet: **bilen bromsar om föraren bromsar, eller om assistanssystemet bestämmer sig för det**. Assistanssystemet bestämmer sig för det när endera detektorn ser något, om det inte samtidigt rapporterar ett fel, i vilket fall det inte anförtros att bromsa alls.

Port	Riktning	Betydelse
<code>driver_brake</code>	in	Föraren står på bromspedalen.
<code>sensor</code>	in	Avståndssensorn ser ett hinder.
<code>radar</code>	in	Radarn ser ett hinder.
<code>error</code>	in	Assistanssystemet rapporterar ett fel.
<code>engine_brake</code>	out	Lägg an bromsarna.

Tre satser avgör tillsammans hela strukturen:

- endera detektorn räcker på egen hand, och ingen av dem väger tyngre än den andra.
- ett fel utlöser inget larm; det *hindrar* assistanssystemet från att kunna bromsa.
- föraren kan alltid bromsa, även när assistanssystemet är trasigt och inte gör någonting.

Den sista är ett säkerhetskrav, och det är den att hålla fast vid när du härleder nätet: pedalen är inte en ingång till assistanslogiken, den går förbi den. Räkna ut vad som ändras om pedalen i stället matas genom fellogiken, så har du hittat buggen som strukturen finns till för att förhindra.

Sanningstabellen

Sexton rader, i den ordning en fyrabitars räknare skulle producera dem. `adas_brake` är ingen port; den är assistanssystemets eget beslut, visad som en mellankolumn så att du kan se var strukturen kommer ifrån.

driver_brake	sensor	radar	error	adas_brake	engine_brake
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	0	0
1	0	0	0	0	1
1	0	0	1	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	1	0	1	1
1	1	1	1	0	1

Den nedre halvan är genomgående 1: så snart `driver_brake` är satt ändrar ingenting annat svaret. Det är hur "åsidosätter allt" ser ut i en tabell.

Fyra svar att ta med sig till passet

Allt ovan är specifikationen. Härledningen, grindnätet och VHDL-koden är det som byggs live, och det är värt att komma dit med ett eget svar att jämföra mot snarare än ett du redan läst. Så, utifrån tabellen ovan:

- Läs av en summa-av-produkter-ekvation för `engine_brake` direkt ur den, på samma sätt som avsnitt 1.2 gjorde.
- Förenkla den för hand. Kravet är tre satser långt, så det minimala nätet är avsevärt mindre än vad den radvisa avläsningen ger dig. Räkna ut hur mycket mindre innan du får det berättat för dig.
- Räkna grindarna din förenklade ekvation behöver, och skriv ner antalet.
- Skissa nätet, och avgör vilken enda grind säkerhetsargumentet ovan vilar på.

Ta med de fyra svaren till passet. Att ha fel om något av dem är värt mer än att inte ha gissat, eftersom du då vet exakt vilket steg du ska granska om när den live-gjorda härledningen når fram till det. Det är avsnitt 1.3:s "härled först, simulera sedan" tillämpat på en krets du inte byggt ännu, och det är den vana resten av boken är byggd på. Efteråt skriver du VHDL-koden själv, i Övning 1.7, och en testbänk avgör om dina ekvationer och sanningstabellen ovan är överens.

Varför det finns en testbänk för den

Sexton kombinationer är få nog att kontrollera för hand en gång. Det är inte få nog att kontrollera varje gång någon redigerar filen, vilket är den situation varje modul i den här boken befinner sig i härifrån och framåt.

En **testbänk** är en andra VHDL-fil som gör kontrollen. Den är inte en del av konstruktionen och når aldrig FPGA:n: den driver ingångarna genom varje fall, räknar ut vad utgången borde ha varit, jämför och rapporterar. För adas, som skrivs vid sidan av den under passet, är det en loop över de sexton kombinationerna som återger bromsregeln oberoende.

Oberoende är det viktiga ordet. En testbänk som läste sitt förväntade svar ur konstruktionen skulle stämma med den per konstruktion, även när konstruktionen har fel. Att återge kravet separat är det som gör det till en kontroll.

Det är därför nästan varje katalog i kursrepot har en `_tb.vhd` vid sidan av modulen: det är så du kan få en modul i handen, ändra den, och inom en sekund veta om du gjort sönder den. Avsnitt 2.7 går igenom hur de körs; du ombeds inte skriva någon någonstans i den här boken.

1.7 Sammanfattning

Det här ska du kunna nu

- Härleda en summa-av-produkter ur en sanningstabell, och sedan bygga och simulera nätet i CircuitVerse.
- Tillämpa De Morgan för att flytta en inversion genom en grind, och använda det för att realisera en funktion i en grinduppsättning som saknar operatoren du utgick från.
- Läs och skriva en kombinatorisk VHDL-modul, en entity med `std_logic`-portar och en architecture som driver dess utgång, och säga varför VHDL skiljer de två åt.
- Förklara varför en konkurrent tilldelning beskriver en ledning snarare än en sats som körs en gång, och varför en signal som ligger kvar på 'U' eller 'X' är ett fel och inte ett värde.
- Köra en utdelad testbänk och avgöra om din modul klarade den.

Frågor att testa dig själv med

1. Vad är skillnaden mellan en entity och en architecture, och varför skiljer VHDL på dem?
2. Varför är `x <= a or b`; inte "en sats som körs en gång"? Vad beskriver den i stället?
3. I bromsassistenten läggs förarens pedal in med OR sist, i stället för att gå genom felhanteringslogiken. Vad går sönder om du kopplar den tvärtom?

Blicken framåt

Kapitel 2 går vidare med:

- Karnaughdiagram: att hitta det *minsta* grindnätet, inte bara ett korrekt.
- `std_logic_vector`, `process` och `case`.
- Submoduler, demonstrerade på en tvåsiffrig hexadecimal display driven av åtta strömbrytare.
- Självkontrollerande testbänkar: att verifiera din egen VHDL utan kort.

1.8 Övningar

Så kontrollerar du ditt arbete. Övning 1.1 är en kort uppvärmning med papper och penna på den notation boken använder rakt igenom; den kräver varken VHDL, GHDL eller kort, och anger vad ett bra svar täcker.

Från Övning 1.2 och framåt har varje övning som ber dig skriva en VHDL-modul en utdelad självkontrollerande testbänk under `lectures/L01/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7 för de tre kommandona.

Inget FPGA-kort behövs till någon övning. Stegen med Quartus-syntes och kortprogrammering demonstreras under passet; ditt jobb efteråt är att få VHDL-koden rätt, och testbänken är hur du bekräftar det.

ÖVNINGSGRUPP 1

Sanningstabeller och grindar

Övning 1.1 En funktion som beror på mindre än den ser ut att göra KRETS

En funktion av två ingångar A, B och en utgång X har sanningstabellen nedan:

A	B	X
0	0	1
0	1	0
1	0	1
1	1	0

a) Härled summa-av-produkter-ekvationen för X direkt ur sanningstabellen (avsnitt 1.2). Två rader har $X = 1$, så du får två AND-termer.

b) Förenkla den algebraiskt. Bryt ut det som de två termerna delar, och tillämpa $A + A' = 1$ och $1 \cdot B' = B'$ från avsnitt 1.2. Du ska hamna i en enda literal.

- c) Ditt svar på b) säger något om A. Vad? Vilken enda grind ur tabellen i avsnitt 1.1 beräknar funktionen, och vad händer med ingången A när du ritar den?
- d) Bygg den enkelgrindsversionen i CircuitVerse och bekräfta att den återger varje rad i sanningstabellen ovan. Lämna ingången A okopplad, eller utelämna den helt enkelt ur ritningen; b) är motiveringen till att göra så.

ÖVNINGSGRUPP 2

Entiteter och arkitekturer

Övning 1.2 and_gate och nand_gate

VHDL

Skriv två små grindar som kompletta VHDL-moduler, `and_gate` och `nand_gate`.

Ingen av grindarna är poängen, och du ska inte förvänta dig att någon av dem är svår. Poängen är att det är här en entitet och en arkitektur slutar vara något du läser om i avsnitt 1.4 och blir något du skriver, och att de två halvorna är värda att skriva i den ordningen.

Båda entiteterna har samma portar:

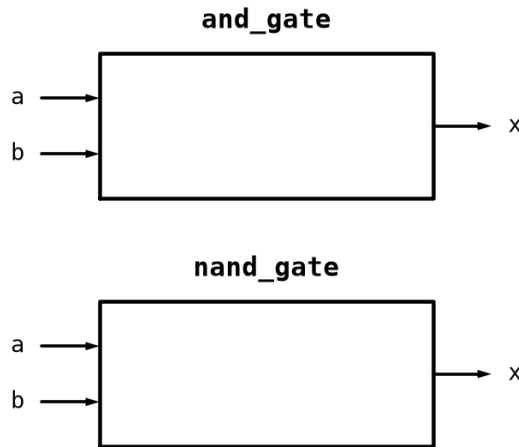
Port	Riktning	Typ
a	in	std_logic
b	in	std_logic
x	out	std_logic

a) Skriv entity-deklarationen för `and_gate`, och ingenting annat. I VHDL är en moduls gränssnitt något du kan skriva ner och resonera om innan någon implementation alls finns, vilket är den vana den här deluppgiften finns till för att bygga. Identifiera vilka portar som krävs, riktningen på var och en, och typen på var och en.

b) Lägg till en architecture som implementerar AND-grinden med en enda konkurrent signaltilldelning.

c) Skriv nu `nand_gate`: samma gränssnitt med inverterad utgång, med VHDL:s `nand`-operator direkt. Skriv **inte** `not` (a and b).

De två är likvärdiga som boolesk algebra, och på en FPGA gör de ingen skillnad alls, eftersom båda hamnar i samma lookup-tabell. Det spelar roll på den sortens hårdvara som Övning 1.6 nedan handlar om, där NAND är den grind du fysiskt har och allt annat måste byggas av den. Att skriva vad du menar, och låta verktygen avgöra vad det kostar, är vanan; att sträcka sig förbi en operator språket redan ger dig är det som ska undvikas.



Självkontroll. Döp dina entiteter till `and_gate` och `nand_gate`, var och en med portarna *a*, *b* (in) och *x* (out), deklarerade i den ordningen; deras testbänkar ligger i `exercises/and_gate/` och `exercises/nand_gate/`, och var och en kontrollerar alla fyra raderna i motsvarande sanningstabell.

Övning 1.3 xor3: udda paritet med tre ingångar

VHDL

En XOR-grind har tre ingångar, *a*, *b*, *c*, och en utgång, *x*. Utgången är 1 när ett udda antal av ingångarna är 1 (XOR med tre ingångar, eller paritetsfunktionen).

- Skriv sanningstabellen för *x* över alla åtta kombinationer av *a*, *b*, *c*, och bekräfta att den stämmer med beskrivningen "ett udda antal 1:or".
- Implementera grinden i VHDL, som en egen modul med en entitet vars portar är tre `std_logic`-ingångar och en `std_logic`-utgång, och en arkitektur som driver *x* med en enda konkurrent tilldelning. Avsnitt 1.4 visar mönstret entitet/arkitektur och noterar att VHDL:s xor-operator arbetar direkt på `std_logic`; avsnitt 1.5 visar en komplett modul från början till slut.
- Bygg samma grind i CircuitVerse och bekräfta att den återger varje rad i din sanningstabell.

Tips: xor kedjar från vänster till höger, så hela arkitekturkroppen är en enda rad, `x <= a xor b xor c;`, utan behov av vare sig intern signal eller mellanliggande grindar.



Självkontroll. Döp din VHDL-entitet till `xor3`, med portarna `a`, `b`, `c` (in) och `x` (out), deklarerade i den ordningen; dess testbänk ligger i `exercises/xor3/`, och avsnitt 1.5 listar de tre kommandon som kör den.

Övning 1.4 majority3: en trippelredundant omröstning

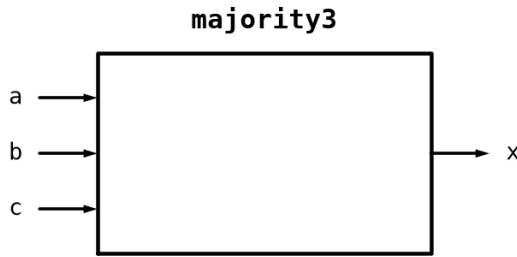
VHDL

En **majoritetsgrind** har tre ingångar, `a`, `b`, `c`, och en utgång, `x`. Utgången är 1 när *minst två* av de tre ingångarna är 1.

Det här är kretsen bakom en trippelredundant omröstning: tre sensorer rapporterar samma mätvärde, och systemet agerar på det svar två av dem är överens om, så att en enskild trasig sensor inte kan bestämma något på egen hand.

- a) Skriv sanningstabellen för `x` över alla åtta kombinationer av `a`, `b`, `c`.
 - b) Härled summa-av-produkter-ekvationen för `x` direkt ur sanningstabellen (avsnitt 1.2):
 - Fyra rader har $x = 1$, så du får fyra AND-termer.
 - Varje term innehåller alla tre ingångarna, primmade där ingången är 0 på sin rad.
 - c) Implementera grinden i VHDL, som en egen modul:
 - En entitet med tre `std_logic`-ingångar och en `std_logic`-utgång.
 - En arkitektur som driver `x` med en enda konkurrent tilldelning, och som översätter dina fyra AND-termer rakt av (avsnitt 1.4).
 - d) Bygg samma nät i CircuitVerse och bekräfta att det återger varje rad i din sanningstabell.
- Tips:** Skriv ut ekvationen på papper innan du öppnar vare sig editorn eller CircuitVerse. Fyra AND-termer med tre ingångar plus en OR-grind med fyra ingångar är mycket ledningsdragning att rätta i efterhand.

Blick framåt: Räkna grindarna din ekvation kräver, och övertyga dig sedan genom inspektion om att $x = ab + ac + bc$ beräknar samma funktion med tre AND-grindar med två ingångar i stället för fyra med tre. Summa-av-produkter gav dig ett *korrekt* nät, inte det *minsta*; att hitta det mindre systematiskt, snarare än genom att stirra på det, är vad kapitel 2 är till för.



Självkontroll. Döp din VHDL-entitet till `majority3`, med portarna `a`, `b`, `c` (in) och `x` (out), deklarerade i den ordningen; dess testbänk ligger i `exercises/majority3/` och kontrollerar alla åtta raderna. Vilken som helst av ekvationerna går igenom — testbänken kontrollerar funktionen, inte vilket nät du valde.

Övning 1.5 `half_adder`: den första modulen med två utgångar

VHDL

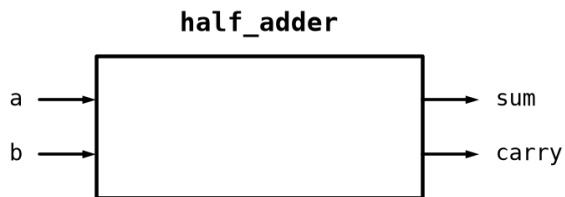
Skriv en **halvadderare**: `sum` och `carry` ur två enskilda bitar. Du kan kretsen; skälet till att den står här är att den är bokens första modul med **två utgångar**, och det är en VHDL-fråga snarare än en logikfråga.

a) Implementera den, med två konkurrenta tilldelningar och utan intern signal.

Två utgångar betyder inte två entiteter, två arkitekturer eller en process. Varje utgångsport drivs av sin egen konkurrenta tilldelning, och de två är inte steg som sker i en ordning: de beskriver två separata ledningsstycken, båda aktiva hela tiden. Om du kommer på dig själv med att sträcka dig efter en process för att "returnera" två värden är den instinkten mjukvaruinstinkten, och avsnitt 1.4:s "en konkurrent tilldelning beskriver en ledning" är rättelsen.

b) Bygg den i CircuitVerse och bekräfta båda utgångarna, så att du har sett formen med två utgångar som en ritning innan du möter den som en entitet med flera out-portar i kapitel 2.

Blick framåt: Det här är cellen som räknaren i kapitel 6 är byggd av. Adderaren med fyra bitar där är fyra sådana kedjade carry-till-carry, och räknaren är den adderaren med sin utgång återkopplad till sin ingång, vilket är därför överslaget inte behöver någon logik alls.



Självkontroll. Döp din entitet till `half_adder`, med portarna `a`, `b` (in) och `sum`, `carry` (out), deklarerade i den ordningen; dess testbänk ligger i `exercises/half_adder/` och kontrollerar `sum` och `carry` var för sig, så att en konstruktion som har den ena rätt och den andra fel får veta vilken som är vilken.

ÖVNINGSGRUPP 3

Universella grindar

Övning 1.6 `or_from_nand`

VHDL

Avsnitt 1.1 slår fast att varje boolesk funktion kan byggas med enbart NAND-grindar. Den här övningen får dig att bevisa det för dig själv för de tre operatorer du har använt.

a) Härled OR-fallet på papper innan du bygger något, eftersom De Morgan (avsnitt 1.2) ger dig det direkt. Utgå från $(A'B')'$ och tillämpa lagen; du ska komma fram till $A + B$. Läs nu tillbaka det uttrycket som en krets: vad är $A'B'$ med en inversion på sin utgång, och hur många NAND-grindar är alltihop?

b) Bygg i CircuitVerse var och en av följande av enbart NAND-grindar, och stäm av var och en mot dess sanningstabell i avsnitt 1.1 efter hand:

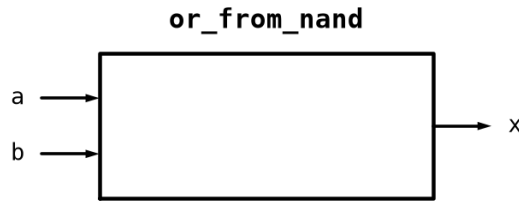
- NOT `a`: en NAND-grind räcker, om du matar samma signal till båda dess ingångar.
- `a AND b`: en NAND-grind, följt av den NOT du just byggde.
- `a OR b`: nätet du härledde i a). Stäm av ditt grindantal mot vad du förutsade.

c) Skriv OR-versionen i VHDL, som en egen modul:

- En entitet med två `std_logic`-ingångar och en `std_logic`-utgång.
- En arkitektur som driver `x` med en enda konkurrent tilldelning.
- Använd enbart VHDL:s `nand`-operator: ingen `or`, ingen `and`, ingen `not` någonstans i uttrycket.

d) Förklara varför det här spelar roll i praktiken. Tänk på att en chiptillverkningsprocess fysiskt måste implementera varje grindtyp den erbjuder, och att NAND är den billigaste grinden att bygga i CMOS.

Tips: VHDL:s `nand`-operator kedjar inte så som `and` och `or` gör: `a nand b nand c` är inte giltigt. Parentessätt varje NAND explicit, vilket ändå är vad du vill här, eftersom varje parentespar är en fysisk grind i din CircuitVerse-ritning.



Självkontroll. Döp din VHDL-entitet till `or_from_nand`, med portarna `a`, `b` (in) och `x` (out), deklarerade i den ordningen; dess testbänk ligger i `exercises/or_from_nand/` och kontrollerar den mot alla fyra raderna i OR-sanningstabellen.

ÖVNINGSGRUPP 4

Att sätta ihop det hela

Övning 1.7 Bromsassistenten i VHDL

VHDL

Skriv bromsassistenten från avsnitt 1.6 i VHDL.

Avsnitt 1.6 lät dig härleda dess ekvationer, räkna dess grindar och skissa dess nät *före* passet, och passet byggde den sedan live. Det är här du skriver den själv, och där en testbänk avgör om din läsning av kravet stämmer med den i sanningstabellen.

Entiteten måste ha:

Port	Riktn.	Typ	Beskrivning
<code>driver_brake</code>	in	<code>std_logic</code>	Föraren står på bromspedalen.
<code>sensor</code>	in	<code>std_logic</code>	Avståndssensorn ser ett hinder.
<code>radar</code>	in	<code>std_logic</code>	Radarn ser ett hinder.
<code>error</code>	in	<code>std_logic</code>	Assistanssystemet rapporterar ett fel.
<code>engine_brake</code>	out	<code>std_logic</code>	Lägg an bromsarna.

a) Implementera den utifrån **dina egna** ekvationer snarare än utifrån det som stod på skärmen. Om de två är oense är den oenigheten det mest användbara den här övningen kan ge dig, och testbänken talar om vilken av dem sanningstabellen håller med.

Använd två konkurrenta tilldelningar och en intern signal:

- Signalen håller assistanssystemets eget beslut, innan förarens pedal vägs in.
- Det här är bokens första övning med en intern signal, och det är anledningen till att nyckelordet finns: ett namn på ett mellanresultat, så att varje rad fortfarande läser sig som den sats i kravet den kom ifrån.

b) Gör nu sönder den avsiktligt, på det specifika sätt avsnitt 1.6 varnar för. Mata `driver_brake` genom fellogiken i stället för att låta den gå förbi:

```
engine_brake <= (driver_brake or sensor or radar) and (not error);
```

Det är en grind enklare, och det ser rimligt ut. Kör testbänken mot den och svara:

- Vilken ingångskombination faller den på först, och vad händer fysiskt i bilen i det ögonblicket?
- Hur många av de sexton raderna får den fel, och vad har de alla gemensamt?
- De rader som fallerar är precis de som säkerhetskravet skrevs för. Vad säger det dig om värdet av en sanningstabell du härlett ur ett krav, till skillnad från en du läst av ur en krets du redan byggt?



Självkontroll. Döp din entitet till *adas*, med ingångarna *driver_brake*, *sensor*, *radar*, *error* och utgången *engine_brake*, deklarerade i den ordningen; dess testbänk ligger i *exercises/adas/* och kontrollerar alla sexton raderna i avsnitt 1.6:s sanningstabell.

Större nät, multiplexrar och submoduler

Att minimera ett nät, de första konstruktionerna som kräver en process, och den första designen som består av mer än en entitet.

I DET HÄR KAPITLET

- Karnaughdiagram: varför en summa-av-produkter läst direkt ur sanningstabellen sällan ger det minsta nätet.
- `std_logic_vector`: att bunta ihop flera ledningar under ett namn, indexering och slicing.
- Multiplexrar, från 2-till-1 till 8-till-1.
- process och case: sekventiella satser inuti i övrigt parallell hårdvara.
- Submoduler: att skriva ett block en gång och instansiera det mer än en gång.
- Självkontrollerande testbänkar, och de tre GHDL-kommandon som kör en.

Kapitel 1 tog en krets hela vägen från ett krav till en fil. Det här kapitlet gör tre saker med den vägen: det stramar åt *härledningen* med Karnaughdiagram, det ger dig de VHDL-konstruktioner ett större nät behöver, och det gör veriferingen till något du kör i stället för något du gör med ögat.

Läs avsnitt 2.1 till 2.6 i ordning; de bygger på varandra. Avsnitt 2.7 är bokens guide till att köra testbänkar, och varje senare kapitels övningar hänvisar tillbaka hit.

Repetition: Karnaughdiagrammet har ett eget kapitel i *Digital Electronics*, kapitel 3, och multiplexern står i avsnitt 4.2 där.

2.1 Karnaughdiagram: från sanningstabell till minimerat grindnät

Att läsa av en summa-av-produkter-ekvation direkt ur en sanningstabell, så som avsnitt 1.2 gjorde, fungerar alltid men ger sällan det minsta nätet: varje 1-rad blir en egen AND-term, även när flera rader delar de flesta av sina ingångar och skulle kunna slås ihop.

Ett **Karnaughdiagram** (K-diagram) gör de gemensamma mönstren synliga för hand, för funktioner av 2 till 4 variabler, utan någon formell manipulation av boolesk algebra. Ett genomarbetat exempel följer; mer övning finns i avsnitt 2.9, och klassisk minimeringsteori (don't-cares, Quine-McCluskey) ligger utanför bokens ram.

Idén är att lägga ut sanningstabellen som ett rutnät i stället för som en lista, och ordna rubrikerna så att varje ruta skiljer sig från var och en av sina grannar i sidled och i höjddled i exakt ett ingångsvärde, även runt kanterna vänster/höger och upp/ner. Den ordningen är **Graykod**: för två bitar 00, 01, 11, 10, där bara en bit ändras i varje steg, inklusive från den sista posten tillbaka till den första.

Sedan:

1. Skriv en 1 i varje ruta där utgången är 1, och lämna resten tomma.
2. Rita rektangulära grupper runt 1:orna:
 - Varje grupp måste vara en rektangel vars bredd och höjd var för sig är en tvåpotens (1, 2 eller 4), så att också dess storlek är en tvåpotens.
 - en grupp om 4 är alltså antingen en 1x4-linje eller ett 2x2-block, och båda är tillåtna.
 - rutor som bara möts i ett hörn är ingen rektangel, så de kan inte grupperas.
 - Grupper får överlappa, och får gå runt en kant av rutnätet.
3. Gör varje grupp så stor som möjligt, och täck varje 1 minst en gång. En grupp om 1 behåller alla sina ingångar i termen, en grupp om 2 stryker en, en grupp om 4 stryker två, och så vidare.
 - Använd sedan så få grupper som räcker: stryk varje grupp vars rutor allihop redan täcks av en annan. Ingenting ovan förbjuder en överflödig grupp, och en överflödig grupp är en överflödig AND-term i svaret, korrekt men inte minimal.
4. Behåll för varje grupp bara den eller de ingångar som har samma värde i varje ruta i gruppen, stryk resten, och skriv det som en AND-term:
 - skriv en behållen ingång direkt om det konstanta värdet är 1, och primmad om det är 0.
 - en grupp som bara täcker rutor där $B = 0$ bidrar alltså med termen B' .
5. X är alla gruppernas termer OR:ade med varandra.

Genomarbetat exempel

Tre ingångar A, B, C, en utgång X:

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Lägg ut den med AB i Graykod nedåt i raderna och C i sidled i kolumnerna, och fyll i en 1 överallt där $X = 1$:

		C	
		0	1
AB	00		1
	01		1
	11	1	1
	10		1

Figur 2.1 Karnaughdiagram för X , ifyllt från sanningstabellen.

Hela $C = 1$ -kolumnen är fyra 1:or, en grupp om 4. Varje ruta delar $C = 1$ och inget annat är konstant (A och B varierar båda), så den reduceras till envariabeltermen C :

		C	
		0	1
AB	00		1
	01		1
	11	1	1
	10		1

Figur 2.2 Samma diagram med den fyra rutor stora $C = 1$ -gruppen markerad.

En 1 är fortfarande inte täckt, på raden $AB = 11$. Gruppera den med sin granne till höger, som C -gruppen redan täcker. Båda rutorna i paret delar $A = 1$ och $B = 1$, och C varierar, så det reduceras till AB :

		C	
		0	1
AB	00		1
	01		1
	11	1	1
	10		1

Figur 2.3 Samma diagram med den två rutor stora $AB = 11$ -gruppen tillagd.

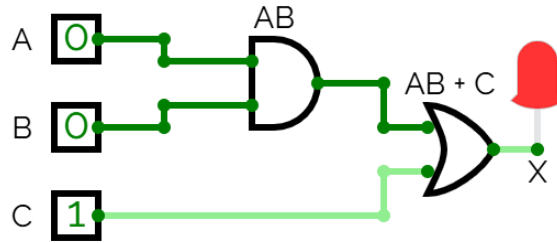
Varje 1 är nu täckt; rutan $AB = 11$, $C = 1$ ligger i båda grupperna, och överlapp är i sin ordning. Summeringen ger:

$$X = AB + C$$

Att läsa sanningstabellen direkt, på kapitel 1:s vis, hade gett fem AND-termer, en per 1-rad. K-diagrammet hittade en likvärdig ekvation med två termer, vilket är hela värdet av att göra det här för hand.

Att realisera nätet

$X = AB + C$ är två grindar: en AND som beräknar AB och matar en OR vars andra ingång är C .



Figur 2.4 Grindnät för det minimerade uttrycket.

Du bygger det här nätet för hand i CircuitVerse under passet, precis som du byggde `or_gates` nät i kapitel 1, och översätter det till VHDL i avsnitt 2.2.

2.2 Att översätta Karnaughdiagrammets nät till VHDL

$X = AB + C$ översätts rakt av till VHDL. Tre ingångar, en utgång:

```
library ieee;
use ieee.std_logic_1164.all;

entity gate_network is
    port(a, b, c: in std_logic;
         x      : out std_logic);
end entity;
```

Arkitekturen använder en intern signal `ab` för att hålla AND-grindens utgång, och speglar tvågrindsnätet ett till ett:

```
architecture behaviour of gate_network is
    signal ab: std_logic;
begin
    -- AND gate: AB.
    ab <= a and b;
```

```
-- OR gate: AB + C.
x <= ab or c;
end architecture;
```

Utan mellansignalen slås det ihop till $x \leq (a \text{ and } b) \text{ or } c$. Båda är lika giltiga och syntetiseras till samma hårdvara; använd den som är mest läsbar för nätet du har framför dig.

Den byggbara versionen använder den hopslagna formen, eftersom det vid två grindar inte finns något att vinna på att namnge mellansignalen:

```
lectures/L02/gate_network/gate_network.vhd

-----
-- Minimized gate network for the Appendix A Karnaugh example:  $x = ab + c$ .
--
-- Inputs:
--   - a, b, c: The three function inputs.
-- Outputs:
--   - x: Network output.
-----

library ieee;
use ieee.std_logic_1164.all;

entity gate_network is
    port(a, b, c: in std_logic;
         x      : out std_logic);
end entity;

architecture behaviour of gate_network is
begin
    x <= (a and b) or c;
end architecture;
```

Öppna den vid sidan av versionen ovan och övertyga dig om att de beskriver samma krets; det är påståendet det här avsnittet gör, och det kostar inget att kontrollera.

För att köra den på hårdvara, tilldela portarna till strömbrytare och en lysdiod med DE0-CV:s pin planner (bilaga B). Ingenting här beror på specifika pinnummer.

2.3 std_logic_vector: mer än en ledning

Allt hittills har varit `std_logic`, en ledning som bär en bit. Att bunta ihop flera ledningar under ett namn är vad `std_logic_vector` är till för.

```
signal count: std_logic_vector(3 downto 0);
```

Det deklarerar fyra ledningar: count(3) ner till count(0). Intervallet är 3 downto 0 snarare än 0 to 3 så att biten längst till vänster är den mest signifikanta, så som du skulle skriva talet på papper. Boken använder downto överallt, och det gör nästan all VHDL du kommer att möta.

En vektor är en porttyp som vilken annan som helst:

```
entity display is
    port(number: in std_logic_vector(3 downto 0);
          hex   : out std_logic_vector(6 downto 0));
end entity;
```

Värden

En enskild bit tar enkla citattecken, en vektor dubbla:

```
x      <= '1';           -- std_logic: one bit
count <= "1010";         -- std_logic_vector(3 downto 0): four bits
count <= (others => '0'); -- every bit '0', whatever the width
```

Den sista är ett *aggregat*: (others => '0') betyder "alla återstående bitar är '0'", och eftersom ingen namngavs individuellt är det allihop. Det fortsätter fungera när bredden ändras, och det dyker upp genom hela resten av boken.

Indexering och slicing

En bit ur en vektor är en std_logic; en sammanhängande följd är en *slice*, som själv är en vektor:

```
carry    <= count(3);
high_half <= input(7 downto 4);
```

Slicing är hur en bred port matar flera smalare, vilket är vad avsnitt 2.6:s tvåsiffriga display gör med sina åtta strömbrytaringångar.

En detalj är inte vad en mjukvaruutvecklare väntar sig. input(7 downto 4) har indexintervallet 7 downto 4, inte 3 downto 0. Kopplad till en port deklarerad std_logic_vector(3 downto 0) matchas de två **positionellt, från vänster till höger**, inte efter indexnummer: input(7) driver bit 3, input(6) driver bit 2, och så vidare. Längderna måste stämma överens. Numreringen behöver inte det.

En vektor är ett knippe bitar, inte ett tal

`count <= "1010"` gör inte `count` lika med tio. `std_logic_vector` säger ingenting om vad bitarna *betyder*, så den erbjuder bara de operationer som är meningsfulla för ett knippe ledningar: de logiska operatorerna tillämpade bit för bit (`a and b`, `not a`, `a xor b`), jämförelse (`count = "1010"`) och konkatenering ("`10`" & "`11`" är "`1011`").

Aritmetik står inte på den listan. `count + 1` går inte att kompilera, eftersom ingenting har sagt om de fyra ledningarna är ett teckenlöst tal, ett tecknat, eller fyra orelaterade signaler.

Boken går inte runt det med en numerisk vektortyp. Allt som räknas är ett vanligt heltal med ett intervall:

```
signal count: natural range 0 to 9;
```

som du kan addera till och jämföra direkt, och som syntesen gör om till exakt de vippor intervallet behöver. Vektorer förblir vad de är: knippen av ledningar, för saker som verkligen är knippen av ledningar, som åtta strömbrytare eller sju displaysegment.

De två världarna möts bara vid en gräns, och konverteringarna dyker upp där de behövs:

```
count_v <= std_logic_vector(to_unsigned(count, 4)); -- integer to vector
count    <= to_integer(unsigned(count_v));         -- vector to integer
```

Båda kommer från `ieee.numeric_std`, vilket är varför vissa filer lägger till en tredje `use-sats`.

2.4 Multiplexrar

En **multiplexer** ("mux") kopplar exakt en av flera dataingångar till en enda utgång, vald av en separat uppsättning **väljaringångar**. En N-till-1-mux har N dataingångar och $\lceil \log_2 N \rceil$ väljarbitar.

Det enklaste fallet, en 2-till-1-mux med ingångarna A, B, väljaren S och utgången X:

S	X
0	B
1	A

Som boolesk ekvation, $X = S'B + SA$: för varje kombination av ingångar är exakt en AND-term aktiv, och den termens dataingång släpps igenom.

Större muxar är samma mönster med fler väljarbitar. En 8-till-1-mux med dataingångarna A–H och väljaren `S[2:0]`:

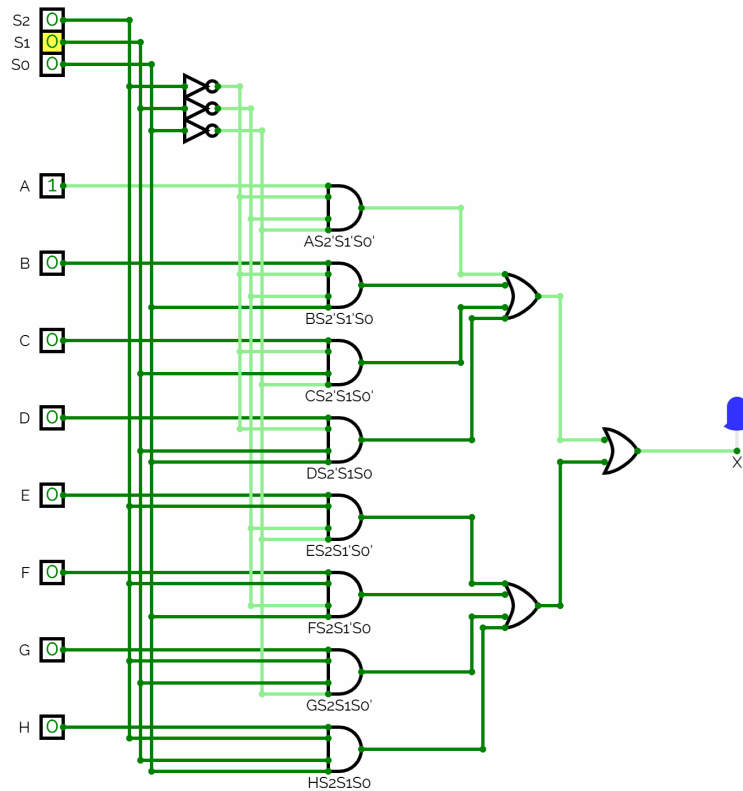
S[2:0]	X
000	A
001	B
010	C
011	D
100	E
101	F
110	G
111	H

Läst som en summa av produkter med en term per rad bidrar raden 000 med $A S'_2 S'_1 S'_0$ och resten följer samma form, där var och en grindar sin dataingång med den väljarkombination som väljer den:

$$X = A S'_2 S'_1 S'_0 + B S'_2 S'_1 S_0 + \dots + H S_2 S_1 S_0$$

Värt att skriva ut en gång för hand, inte värt att memorera. Mönstret är poängen, och det är detsamma i vilken storlek som helst.

Som grindar: en AND med fyra ingångar per dataingång (databiten plus alla tre väljarbitarna), med väljarbitarna inverterade efter behov, och alla åtta AND-utgångar summerade av ett träd av OR-grindar.



Figur 2.5 8-till-1-multiplexer som åtta AND-grindar som matar ett OR-träd.

Det här är en verklig byggsten. ATmega328P (chippet på en Arduino Uno och Nano) använder en internt så att dess analogkapabla pinnar, plus en intern bandgapsreferens och en temperatursensor, kan dela på en enda analog-till-digital-omvandlare, vald av de fyra MUX-bitarna i dess ADMUX-register.

2.5 Multiplexrar i VHDL: process och case

Varje sats hittills har varit **konkurrent**: en stående beskrivning av en ledning, alla aktiva samtidigt. En multiplexer är den första kretsen som är otymplig att skriva så, och den behöver två nya konstruktioner:

- en **process**, ett block vars innehåll körs sekventiellt, uppifrån och ned, som vanlig kod. Den körs om varje gång någon signal i dess **känslighetslista** ändras, och ur arkitekturens synvinkel är den fortfarande bara ännu en konkurrent sats. Varje process i boken har den

här formen; kapitel 3 återanvänder den för klockad logik, med klockan i känslighetslistan i stället för de ingångar som avkodas.

- en **case-sats**, VHDL:s switch. Precis som if och for är den en *sekventiell* sats, tillåten bara inuti en process, och varje värde väljaren kan anta måste täckas, vilket when others garanterar.

”Välj en av flera ingångar utifrån en väljare” är en case-sats, nästan ord för ord. 2-till-1-muxen från avsnitt 2.4:

```
MUX_PROCESS: process(a, b, sel) is
begin
    case (sel) is
        when '0'    => x <= b;
        when '1'    => x <= a;
        when others => x <= '0';
    end case;
end process;
```

Två saker att notera:

- Enbitsvärden tar enkla citattecken, '0'/'1', till skillnad från de dubbla citattecken som används för vektorer som "000".
- when others är obligatoriskt snarare än defensivt. VHDL kräver att ett case täcker varje värde av väljarens typ, och std_logic har nio (avsnitt 1.4), så '0' och '1' ensamma gör aldrig ett case uttömmande. Det är en språkregel, inte ett påstående om ledningen: ett syntetiserat nät bär en 0 eller en 1 och inget annat, och de andra sju värdena är simulatorns, där en signal som ligger kvar på 'U' eller 'X' är en bugg snarare än en nivå.

Genomarbetat exempel: 8-till-1-muxen

Samma form med en trebitars väljare och åtta grenar, en per rad i avsnitt 2.4:s sanningstabell. Dataingångarna A–H blir vektorn inputs(7 downto 0), med A som inputs(7) ner till H som inputs(0), så väljaren "000" kopplar fram inputs(7) och "111" kopplar fram inputs(0).

```
lectures/L02/mux_8to1/mux_8to1.vhd
-----
-- 8-to-1 multiplexer: routes one of inputs(7:0) to x, selected by sel.
--
-- Inputs:
--   - inputs: Data inputs; sel "000" routes inputs(7), "111" routes inputs(0).
--   - sel   : Selector.
-- Outputs:
--   - x: Selected input.
-----
```

```

library ieee;
use ieee.std_logic_1164.all;

entity mux_8to1 is
    port(inputs: in std_logic_vector(7 downto 0);
          sel   : in std_logic_vector(2 downto 0);
          x     : out std_logic);
end entity;

architecture behaviour of mux_8to1 is
begin
    MUX_PROCESS: process (inputs, sel) is
    begin
        -- Check the selector combination, update the output accordingly.
        -- Safety default: drive output low if an invalid selector occurs.
        case (sel) is
            when "000" =>
                x <= inputs(7);
            when "001" =>
                x <= inputs(6);
            when "010" =>
                x <= inputs(5);
            when "011" =>
                x <= inputs(4);
            when "100" =>
                x <= inputs(3);
            when "101" =>
                x <= inputs(2);
            when "110" =>
                x <= inputs(1);
            when "111" =>
                x <= inputs(0);
            when others =>
                x <= '0';
        end case;
    end process;
end architecture;

```

Jämför den med avsnitt 2.4:s ekvation med åtta termer för samma krets. Ekvationen beskriver grindarna; case-satsen beskriver *avsikten* och låter syntesverktyget ta fram grindarna. Båda är korrekta och båda syntetiseras till samma hårdvara. Det glappet, mellan att beskriva struktur och att beskriva beteende, är det mesta av vad ett hårdvarubeskrivande språk ger dig.

2.6 Att bygga en design av submoduler

Allt hittills har varit en entitet med en arkitektur. Det slutar räcka i samma stund en design behöver samma block två gånger.

DE0-CV har sex 7-segmentsdisplayer, och du vill driva två av dem, var och en visande en hexadecimal siffra. Logiken som gör ett fyrabitars tal till sju segmentsignaler är densamma för båda. Du skulle kunna skriva den två gånger inuti en arkitektur och ingenting skulle hindra dig, men den andra kopian är en belastning: varje rättning måste göras i båda, och den dag de glider isär är den dag du förlorar en kväll på att lista ut vilken av dem som är fel.

Skriv blocket en gång, som en egen entitet, och *instansiera* det två gånger.

Submodulen

En vanlig modul. Ingenting hos den säger "jag är en submodul":

```
entity display is
    port(number: in std_logic_vector(3 downto 0);
          hex   : out std_logic_vector(6 downto 0));
end entity;
```

Om den slutar som en hel design eller som en del av en större avgörs av den som instansierar den, inte av modulen.

Att instansiera den

Med entity work.<name>, inuti en annan arkitektur:

```
entity hex_display is
    port(input      : in std_logic_vector(7 downto 0);
          hex1, hex0: out std_logic_vector(6 downto 0));
end entity;

architecture structure of hex_display is
begin
    display1: entity work.display
        port map(input(7 downto 4), hex1);

    display0: entity work.display
        port map(input(3 downto 0), hex0);
end architecture;
```

- display1 och display0 är **instansetiketter**, som namnger de två kopiorna. De måste vara unika inom arkitekturen, och de är inte modulens namn: båda instanserna är display.

- `entity work.display` väljer modulen. `work` är det bibliotek dina analyserade filer hamnar i som förval, i både GHDL och Quartus.
- `port map(...)` kopplar instansens portar till signaler i den omslutande arkitekturen, i **den ordning entiteten deklarerar dem**. `display` deklarerar `number` och sedan `hex`, så det första objektet driver `number` och det andra drivs av `hex`.
- `input(7 downto 4)` är en **slice**, som instansen ser som en `std_logic_vector(3 downto 0)`.

Lägg märke till vad `hex_displays` arkitektur *inte* innehåller: någon logik alls. Den är ledningsdragning och inget annat. Varje grind i den färdiga designen kommer från de två `display`-instanserna.

Positionell och namngiven associering

`port map` ovan är **positionell** och matchar portar efter ordning. VHDL tillåter också **namngiven** associering, som du kommer att möta i andras kod:

```
display1: entity work.display
    port map(number => input(7 downto 4), hex => hex1);
```

De två är likvärdiga. Boken använder positionell överallt, vilket betyder att ordningen på en entitets portdeklARATIONER är något du måste få rätt. Det är ingen formalitet: det är precis vad de utdelade testbänkarna förlitar sig på, vilket är varför varje övnings självkontrollsruta stavar ut sina portar i *ordning*.

Vad det här kostar i hårdvara

Inget överraskande. Att instansiera en modul två gånger lägger två kopior av dess grindar på FPGA:n, precis som att skriva logiken två gånger skulle göra. Besparingen ligger i källkoden, inte i kislet: en beskrivning, ett ställe att rätta på, och en design du kan läsa.

2.7 Att verifiera din VHDL med en testbänk

Varje VHDL-modul du skriver till en övning kommer med en **självkontrollerande testbänk**: en liten VHDL-fil som driver dina ingångar, kontrollerar dina utgångar och talar om vilket fall som misslyckades. Kapitel 1:s övningar delade redan ut sådana, och avsnitt 1.5 gav dig de tre kommandon som kör en.

Du ombeds inte skriva den testbänk som verifierar en modul du bygger; varenda en av dem delas ut. Att köra dem räcker för att verifiera varje övning på din egen laptop, utan FPGA-kort. (Ett par projektövningar från kapitel 13 och framåt bjuder in till en kort egen testtrigg, men ingenting du bedöms på vilar på en testbänk du skrivit själv.)

Det här avsnittet är bokens introduktion, och varje senare kapitelns övningar hänvisar tillbaka hit. Bilaga A är den fylligare versionen, skriven för grupprojektet; kom tillbaka hit för introduktionen och gå dit för detaljerna.

2.7.1 Det här behöver du

GHDL, den öppna VHDL-analysatorn och -simulatorn. Kontrollera med `ghdl --version`; installera på WSL eller Ubuntu med `sudo apt -y install ghdl`.

Varje kommando nedan använder `--std=93`, den VHDL-standard boken riktar in sig på.

2.7.2 Vad en testbänk är

En testbänk är själv en VHDL-modul, men en speciell sådan:

- Dess entitet har **inga portar**: ingenting kopplas till omvärlden, eftersom den är simuleringens topp.
- Dess arkitektur **instansierar din konstruktion**, "device under test", märkt `dut`, och driver dess ingångar.
- Den **kontrollerar** varje utgång med ett assert. Om en utgång är fel utlöses assertionen med ett meddelande och simuleringen stannar.

Så i stället för att du läser av ett vågformsdiagram för att avgöra om utgången är rätt, avgör testbänken det, och säger bara ifrån när något är fel.

2.7.3 Att köra en testbänk: analysera, elaborera, köra

Tre steg. Skicka med `--std=93` till alla tre, och ge **testbänkens entitetsnamn**, inte ett filnamn, till de två sista:

```
ghdl -a --std=93 <module>.vhd <module>_tb.vhd      # analyze both files (module ↵
↵ first)
ghdl -e --std=93 <module>_tb                        # elaborate the testbench
ghdl -r --std=93 <module>_tb --assert-level=error --stop-time=10ms # run it
```

- `--assert-level=error` gör att en misslyckad kontroll stoppar körningen och returnerar en **slutkod skild från noll**, ett riktigt godkänt/underkänt som du kan kontrollera med `echo $?`. Testbänkarna i den här bokens första del reser fel med severity `failure`, vilket stoppar körningen av sig självt, så du får ett riktigt godkänt/underkänt även utan flaggan. Ha den på ändå: det är vanan som också fungerar med testbänkar som rapporterar på en lägre severity-nivå, och grupprojektets testbänkar är precis sådana (bilaga A).
- `--stop-time=10ms` sätter ett tak för hur länge simuleringen får köra. En testbänk väntar ofta på att din modul ska göra något, med en rad som `wait until tx = '0'`; . Om din modul aldrig gör det blir den väntan aldrig färdig, och utan en stopptid hänger simuleringen: ingen utskrift, inget fel, bara en terminal som står och väntar. 10 ms *simulerad* tid

är långt mer än någon testbänk här behöver, och tar en bra bit under en sekund i verklig tid.

- En godkänd körning skriver ut sin "all checks passed"-notering och avslutar med 0. Varje testbänk i den här boken slutar med den noteringen, så om du inte ser den blev körningen inte färdig: den antingen underkände en kontroll eller slog i stopptiden.

2.7.4 Att kontrollera en övning

Varje VHDL-övning har sin testbänk i sin egen katalog. Det här kapitlets Övning 2.3 har till exempel `lectures/L02/exercises/xyz_logic/xyz_logic_tb.vhd`. Skriv din modul i **samma katalog**, med exakt det namn övningen ber om, och kör sedan de tre kommandona därifrån:

```
cd lectures/L02/exercises/xyz_logic
ghdl -a --std=93 xyz_logic.vhd xyz_logic_tb.vhd
ghdl -e --std=93 xyz_logic_tb
ghdl -r --std=93 xyz_logic_tb --assert-level=error --stop-time=10ms
```

Varje testbänks filhuvud listar precis de här kommandona.

Testbänken kopplar till din modul **positionellt**, inte via namn, så din entitet måste deklarerar sina portar i **samma ordning** som övningen listar dem. Namnen är dina att välja, men använd de angivna ändå, så att din modul och testbänkens felmeddelanden talar om samma signaler.

En port som deklarerats i fel ordning misslyckas på ett av två sätt, varav inget säger "fel ordning":

- om de felplacerade portarna har **olika** typ eller bredd stannar analysen med `can't associate "sel" with port "sel"`, med en pekare på testbänkens port map-rad.
- om de har **samma** typ, säg flera `std_logic`-portar, analyseras och elaboreras den alldeles utmärkt, och faller sedan vid körning som ett vanligt assertion-fel, vilket läser sig precis som en logikbugg i din konstruktion.

Så när en testbänk faller på allra första fallet den kontrollerar: läs om din portordning innan du letar igenom din arkitektur. Detsamma gäller **generics**: en modul som bär på sådana måste deklarerar exakt de som övningen anger, i ordning.

2.7.5 När konstruktionen behöver mer än en fil

Vissa övningar bygger en modul av en annan, så `ghdl -a`-raden namnger flera filer i stället för två. Elaborerings- och körstegen är oförändrade: de tar fortfarande bara testbänkens **entitetsnamn**.

```
ghdl -a --std=93 <subcomponent>.vhd <module>.vhd <module>_tb.vhd
ghdl -e --std=93 <module>_tb
```

```
ghdl -r --std=93 <module>_tb --assert-level=error --stop-time=10ms
```

- **Ordningen spelar roll på analysraden:** namnge en modul före allt som instansierar den, så att GHDL redan har sett entiteten när det når det `entity work.<name>` som hänvisar till den. Subkomponenter först, din toppnivå därefter, testbänken sist.
- Ingen övningskatalog har någon utdelad subkomponent. Varje modul en konstruktion instansierar är en du skrev i en tidigare övning och kopierar in själv; övningen säger vilken, och varifrån.
- Övningarna som behöver det här är det här kapitlets `hex_display` (som instansierar din `display`), kapitel 4:s `led_toggle_sync2`, kapitel 7:s `blinker` och `walking_led`, samt alla kapitel 8:s, där mottagaren namnger fem filer och sändaren fyra.
- Varje testbänks filhuvud listar det exakta kommandot för sin egen övning.

2.7.6 Att läsa resultatet

En misslyckad kontroll ser ut så här:

```
xyz_logic_tb.vhd:38:13:@10ns:(assertion failure): xyz_logic: wrong X with a='0' b↵
↵ = '0' c='0' d='0', expected '0' but got '1'!
ghdl:error: assertion failed
```

Läs den i ordning: `file:line` är var assertionen står, `@time` är när den utlöstes, och därefter kommer meddelandet. Det är samma form som ett kompileringsfel från GHDL, så det avbildas på sådant du redan kan. Inget fel betyder att varje fall stämde.

2.7.7 Ett komplett exempel

En kombinatorisk testbänk i den form varje testbänk i den här boken använder, här för kapitel 1:s `or_gate`:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity or_gate_tb is
end entity;

architecture behaviour of or_gate_tb is
signal a, b, x: std_logic;
begin
    dut: entity work.or_gate
        port map(a, b, x); -- positional: same order as or_gate's port clause.
```

```

SIM_PROCESS: process is
begin
    for i in 0 to 3 loop
        (a, b) <= std_logic_vector(to_unsigned(i, 2)); -- apply inputs
        wait for 10 ns; -- let them settle
        assert x = (a or b) -- check AFTER the wait
        report "or_gate: wrong output with a=" & std_logic'image(a)
            & " b=" & std_logic'image(b)
            & ", expected " & std_logic'image(a or b)
            & " but got " & std_logic'image(x) & "!"
        severity failure;
    end loop;
    report "or_gate: all checks passed!" severity note;
    wait; -- end the simulation
end process;
end architecture;

```

Två regler håller en kombinatorisk testbänk korrekt:

- **Lägg på, vänta, kontrollera, i den ordningen.** En signaltilldelning *schemalägger* bara ett värde, så ingångarna och utgången uppdateras inte förrän vid `wait`. Att kontrollera före den testar föregående varvs värden.
- **Avsluta med `wait`;** så att stimulusprocessen suspenderas och simuleringen stannar av sig själv.

Klockade konstruktioner behöver lite mer, en klockgenererande process och ett sätt att stoppa den, vilket är precis det maskineri de utdelade testbänkarna redan innehåller. Till övningarna skriver du bara modulen.

Kör en nu. Mot det här kapitlets första genomarbetade exempel:

```

cd lectures/L02/gate_network
ghdl -a --std=93 gate_network.vhd gate_network_tb.vhd
ghdl -e --std=93 gate_network_tb
ghdl -r --std=93 gate_network_tb --assert-level=error --stop-time=10ms

```

2.8 Sammanfattning

Det här ska du kunna nu

- Härleda en minimerad ekvation ur ett Karnaughdiagram och göra en VHDL-arkitektur av den.
- Läs och skriva portar och signaler av typen `std_logic_vector`.
- Skriva en process med korrekt känslighetslista och ett case som täcker varje indatavärde.
- Bygga en design av mer än en entity med positionell port `map`, och säga varför två instanser av samma modul lägger två kopior av dess grindar på FPGA:n.
- Verifiera en modul genom att köra dess testbänk, och läsa ett misslyckat assert.

Frågor att testa dig själv med

1. Kan du, givet en sanningstabell, härleda dess ekvation både direkt och via ett Karnaughdiagram? Vad skiljer de två resultaten?
2. Varför kan ett case inte stå direkt i en arkitekturrupp?
3. Vad skyddar `when others` dig mot, när väljaren "bara kan" vara 0 eller 1?
4. I `display1: entity work.display`, vad namnger `display1`, och vad namnger `work.display`? Skulle en andra instans kunna återanvända samma etikett?
5. `hex_displays` arkitektur innehåller ingen logik alls. Var kommer grindarna i den färdiga designen ifrån?
6. Du instansierade `display` två gånger i stället för att skriva dess case två gånger. Får FPGA:n en eller två kopior av den logiken? Vad vann du alltså egentligen?
7. Boken skriver alltid port `map` positionellt. Vad går sönder om en entitets portar deklarerar i en annan ordning än den testbänken förväntar sig, och när märker du det?

Blicken framåt

Kapitel 3 går vidare med:

- Sekvensnät: kretsar som minns.
- D-låset, och D-vippan byggd av ett par av dem.
- Klockning, och när en signaltilldelning faktiskt får effekt.
- Flankdetektering: en knapptryckning som växlar en lysdiod.

2.9 Övningar

Så kontrollerar du ditt arbete. Varje övning nedan som ber dig skriva en VHDL-modul har en utdelad självkontrollerande testbänk under `lectures/L02/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7 för de tre kommandona.

Inget FPGA-kort behövs till någon övning. Stegen med Quartus-syntes och kortprogrammering demonstreras under passet; ditt jobb efteråt är att få VHDL-koden rätt, och testbänken är hur du bekräftar det.

ÖVNINGSGRUPP 1

Karnaughdiagram

Övning 2.1 Tre variabler

KRETS

Härled en minimerad ekvation för X ur Karnaughdiagrammet nedan, realisera sedan motsvarande grindnät och simulera det i CircuitVerse:

ABC	X
000	1
001	1
010	0
011	0
100	1
101	1
110	0
111	0

Tips: Lägg AB nedåt i raderna i Graykodsordning (00, 01, 11, 10) och C i sidled i kolumnerna, precis som i det genomarbetade exemplet i avsnitt 2.1, innan du börjar gruppera 1:or.

Övning 2.2 Fyra variabler

KRETS

Härled en minimerad ekvation för X ur Karnaughdiagrammet nedan, realisera sedan motsvarande grindnät och simulera det i CircuitVerse:

ABCD	X
0000	0
0001	1
0010	0
0011	1
0100	0
0101	0
0110	0
0111	0
1000	0
1001	1
1010	0
1011	1
1100	1
1101	0
1110	1
1111	0

Tips: Med fyra variabler lägger du två av dem (i Graykod) på varje axel, så att du får ett 4x4-rutnät. Grupper kan fortfarande gå runt varje kant av rutnätet, inte bara vänster-höger; kom ihåg att kontrollera överkant och underkant också.

ÖVNINGSGRUPP 2

Kretsar i VHDL

Övning 2.3 xyz_logic: tre utgångar ur fyra ingångar

VHDL

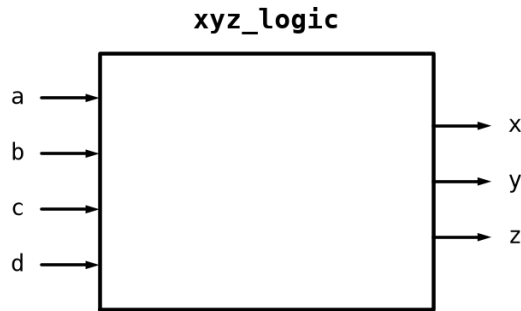
Ett grindnät har fyra ingångar, ABCD, och tre utgångar, XYZ. Det ges av sanningstabellen nedan:

ABCD	XYZ
0000	001
0001	010
0010	001
0011	011
0100	100
0101	110
0110	100
0111	111
1000	101
1001	110
1010	101
1011	111
1100	000
1101	010
1110	000
1111	011

- Behandla X, Y och Z som tre separata funktioner av ABCD med en utgång var, och härled en minimerad ekvation för var och en via ett Karnaughdiagram.
- Realisera det resulterande grindnätet och simulera det i CircuitVerse; stäm av alla tre utgångarna mot sanningstabellen ovan.
- Implementera konstruktionen i VHDL, i en egen modul med fyra std_logic-ingångar och tre std_logic-utgångar. Avsnitt 1.4 och avsnitt 2.2 visar mönstret för att göra konkurrenta VHDL-tilldelningar av en härledd ekvation.
- Kör testbänken och bekräfta att alla 16 raderna går igenom.
Om en rad fallerar namnger assertion-meddelandet ingångskombinationen:

- Gå tillbaka till Karnaughdiagrammet för den av X, Y, Z som är fel.
- En enda fel utgång på en enda rad är nästan alltid en felgrupperad 1 i ett diagram.

Tips: Två av de tre utgångarna här visar sig bero på bara en eller två av de fyra ingångarna när de väl minimerats. Anta inte att varje utgång behöver alla fyra.



Självkontroll. Döp din VHDL-entitet till `xyz_logic`, med portarna `a`, `b`, `c`, `d` (in) och `x`, `y`, `z` (out), deklarerade i den ordningen; dess testbänk ligger i `exercises/xyz_logic/` och kontrollerar alla 16 raderna i sanningstabellen ovan.

ÖVNINGSGRUPP 3

Multiplexrar

Övning 2.4 4-till-1-muxen för hand

KRETS

En 4-till-1-multiplexer har fyra dataingångar, A–D, två väljarbiter, $S[1:0]$, och en utgång, X.

Låt väljaren namnge sin dataingång direkt: $S[1:0] = "00"$ väljer A, $"01"$ väljer B, $"10"$ väljer C och $"11"$ väljer D. Det är den avbildning Övning 2.6 implementerar.

- Skriv ut multiplexerns sanningstabell (fyra rader, en per väljarkombination).
- Härled summa-av-produkter-ekvationen för X uttryckt i A–D och $S[1:0]$. Avsnitt 2.4 gör likadant för 8-till-1-fallet; halvera det här.
- Bygg och simulera nätet i CircuitVerse, och bekräfta att X följer varje dataingång i tur och ordning när du ändrar $S[1:0]$.

Spara ditt svar: Övning 2.6 nedan implementerar precis den här kretsen i VHDL, när avsnitt 2.5 väl har gett dig de process- och case-satser den behöver.

Övning 2.5 mux2 med if/else

VHDL

Avsnitt 2.5 skriver 2:1-multiplexern med en case-sats. Skriv samma krets en gång till med en if/else i stället, och övertyga dig själv om att de två beskriver identisk hårdvara.

Skriv en entitet med namnet mux2 med:

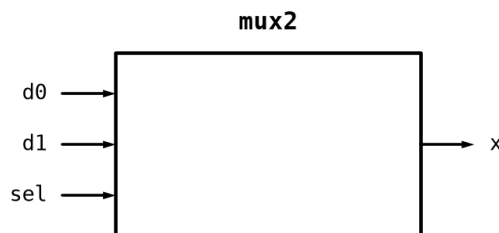
Port	Riktn.	Typ	Beskrivning
d0	in	std_logic	Dataingång.
d1	in	std_logic	Dataingång.
sel	in	std_logic	Väljare.
x	out	std_logic	Den valda dataingången.

Väljaren kopplar fram den dataingång vars nummer den namnger:

- sel = '0' väljer d0.
- sel = '1' väljer d1.
- Observera att det är omvänt mot case-exemplet i avsnitt 2.5, som väljer sin *andra* ingång b vid '0'. Vilken ingång ett väljarvärde namnger är en konvention, inte en regel, så läs av det ur specifikationen varje gång i stället för att anta.

Implementera multiplexern med en processbaserad arkitektur:

- Ta med d0, d1 och sel i känslighetslistan.
- Använd en if/else-sats på sel.
- Tilldela x i varje gren.
- Använd inte:
 - En villkorlig konkurrent tilldelning (x <= d1 when sel = '1' else d0;).
 - En with...select-sats: en konkurrent form av case som boken inte går igenom, nämnd bara för att du inte ska sträcka dig efter den om du mött den någon annanstans.



Självkontroll. Döp din entitet till mux2, med portarna d0, d1, sel (in) och x (out), deklarerade i den ordningen; dess testbänk ligger i exercises/mux2/ och kontrollerar alla åtta kombinationer av de tre ingångarna.

Övning 2.6 mux4

VHDL

Utöka Övning 2.5 till en 4:1-multiplexer med namnet mux4.

Det här är kretsen du härledde, ritade och simulerade för hand i Övning 2.4. Härled case-grenarna på nytt själv för fyra ingångar och två väljarbiter i stället för att kopiera 8-till-1-exemplet från avsnitt 2.5.

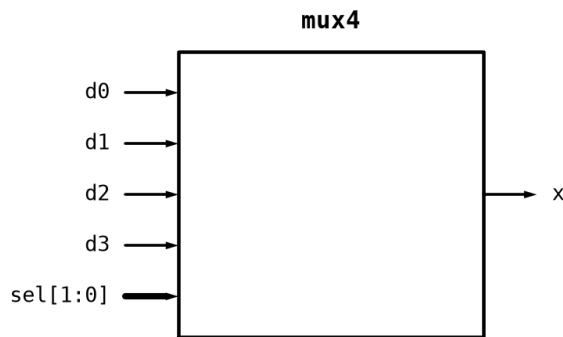
Entiteten måste ha:

Port	Riktn.	Typ	Beskrivning
d0	in	std_logic	Dataingång.
d1	in	std_logic	Dataingång.
d2	in	std_logic	Dataingång.
d3	in	std_logic	Dataingång.
sel	in	std_logic_vector(1 downto 0)	Väljare.
x	out	std_logic	Den valda dataingången.

Som i Övning 2.5 namnger väljaren sin dataingång direkt: "00" väljer d0, "01" väljer d1, "10" väljer d2 och "11" väljer d3. Återigen är det den omvända avbildningen mot avsnitt 2.5:s genomarbetade 8-till-1-exempel, där "000" kopplar fram inputs(7). Härled dina case-grenar ur de fyra raderna ovan, inte ur den listningen.

Implementera multiplexern med en process:

- Ta med alla dataingångar och sel i känslighetslistan.
- Använd en case-sats på sel, en gren per väljarvärde ovan.
- Ta med en when others-gren, och välj ett säkert utgångsvärde för väljaringångar som innehåller värden som 'X', 'U' eller 'Z'.
- Ersätt inte case-satsen med en if/elsif-kedja.



Självkontroll. Döp din entitet till mux4, med portarna d0, d1, d2, d3, sel (std_logic_vector(1 downto 0)) och x (out), deklarerade i den ordningen; dess testbänk ligger i exercises/mux4/ och sveper alla fyra väljarvärdena mot alla sexton dataingångskombinationerna.

ÖVNINGSGRUPP 4

Submoduler och 7-segmentsdisplayer

Övning 2.7 och 2.8 bygger en konstruktion i två halvor: en modul som driver en enda 7-segmentsdisplay, och en toppnivå som använder två kopior av den för att visa en hexadecimal siffra på var och en. Gör dem i ordning: Övning 2.8 instansierar det du skriver i Övning 2.7.

En 7-segmentsdisplay är sju lysdioder, namngivna a till g, arrangerade runt en åtta. Tänd rätt delmängd så får du en siffra. Det här är utseendet, och det är det du behöver för att kunna räkna ut koderna nedan:

```

      aaaa
    f    b
    f    b
      gggg
    e    c
    e    c
      dddd

```

Så 1 tänder b och c (de två till höger), och 7 tänder a, b och c. Att jämföra koderna som ges för de två siffrorna är ett snabbt sätt att bekräfta att du har utseendet rättvänt innan du börjar härleda A till F.

På DE0-CV är displayerna kopplade **aktivt låga**: att driva ett segment till '0' tänder det, och '1' släcker det. Det är därför koden för 8, som tänder varje segment, är "0000000", och varför en släckt display är "1111111".

Genomgående avbildas hex(6 downto 0) på segmenten g, f, e, d, c, b, a: bit 6 är g och bit 0 är a.

Övning 2.7 display

VHDL

Skriv en modul med namnet `display` som driver en 7-segmentsdisplay med en hexadecimal siffra.

Entiteten måste ha:

Port	Riktn.	Typ	Beskrivning
number	in	std_logic_vector(3 downto 0)	Värdet som ska visas, 0000 till och med 1111.
hex	out	std_logic_vector(6 downto 0)	Segmentkoden för det värdet.

Att mata number värdet 0111 måste alltså lägga koden för 7 på hex, och att mata den 1110 måste lägga koden för E på hex.

Implementera den med en process:

- Lägg number i känslighetslistan.
- Använd en case-sats på number, en gren per värde.
- Ta med en when others-gren, och släck displayen i den.

Du får koderna för 0 till 9. Räkna ut A till F själv, utifrån segmentlayouten ovan:

```
constant DISPLAY_0 : std_logic_vector(6 downto 0) := "1000000";
constant DISPLAY_1 : std_logic_vector(6 downto 0) := "1111001";
constant DISPLAY_2 : std_logic_vector(6 downto 0) := "0100100";
constant DISPLAY_3 : std_logic_vector(6 downto 0) := "0110000";
constant DISPLAY_4 : std_logic_vector(6 downto 0) := "0011001";
constant DISPLAY_5 : std_logic_vector(6 downto 0) := "0010010";
constant DISPLAY_6 : std_logic_vector(6 downto 0) := "0000010";
constant DISPLAY_7 : std_logic_vector(6 downto 0) := "1111000";
constant DISPLAY_8 : std_logic_vector(6 downto 0) := "0000000";
constant DISPLAY_9 : std_logic_vector(6 downto 0) := "0010000";
-- Todo: add DISPLAY_A through DISPLAY_F here.
constant DISPLAY_OFF: std_logic_vector(6 downto 0) := "1111111";
```

Två av de sex är gemener på en 7-segmentsdisplay, eftersom deras versalformer skulle vara omöjliga att skilja från en siffra: b skulle läsas som 8 och d skulle läsas som 0. A, C, E och F är versaler.

Alla sexton koderna sitter också i `display_tb.vhd`, eftersom en självkontrollerande testbänk inte kan kontrollera en utgång utan att veta vad den borde vara. Så det här är inget pussel du kan bli uteläst från. Härled de sex ändå: det tar ett par minuter med segmentdiagrammet, och att läsa av dem ur testbänken lär dig ingenting som case-satsen inte gör. Jobbet i den här övningen är case-satsen med sexton grenar, inte uppslagstabellen.



Självkontroll. Döp din entitet till `display`, med portarna `number` (`std_logic_vector(3 downto 0)`) och `hex` (`out, std_logic_vector(6 downto 0)`), deklarerade i den ordningen; dess testbänk ligger i `exercises/display/` och kontrollerar alla sexton värdena mot hela kodtabellen, och namnger siffran och båda koderna när en är fel.

Övning 2.8 hex_display

VHDL

Skriv en modul med namnet `hex_display` som visar ett tvåsiffrigt hexadecimalt tal på två displayer, med två instanser av din `display` från Övning 2.7.

Entiteten måste ha:

Port	Riktn.	Typ	Beskrivning
input	in	<code>std_logic_vector(7 downto 0)</code>	Åtta strömbrytare: de fyra översta är den vänstra siffran, de fyra nedersta den högra.
hex1	out	<code>std_logic_vector(6 downto 0)</code>	Segmentkod för den vänstra siffran.
hex0	out	<code>std_logic_vector(6 downto 0)</code>	Segmentkod för den högra siffran.

När den fungerar dyker talet på `input(7 downto 4)` upp på `hex1`, och talet på `input(3 downto 0)` på `hex0`.

Bygg den med submoduler, enligt avsnitt 2.6:

- Skapa två instanser av `display`, med etiketterna `display1` och `display0`.
- Koppla `input(7 downto 4)` och `hex1` till `display1`.
- Koppla `input(3 downto 0)` och `hex0` till `display0`.
- Använd positionell port `map`, som överallt annars i den här boken.
- Skriv ingen egen logik i `hex_display`. Om du kommer på dig själv med att skriva en `case-sats` här duplicerar du Övning 2.7 i stället för att återanvända den.

Kopiera in din `display.vhd` från Övning 2.7 i övningskatalogen, eftersom `hex_display` inte kan analyseras utan den. Det här är bokens första konstruktion byggd av mer än en fil, så namnge `display.vhd` först, före den fil som instansierar den:

```
ghdl -a --std=93 display.vhd hex_display.vhd hex_display_tb.vhd
ghdl -e --std=93 hex_display_tb
ghdl -r --std=93 hex_display_tb --assert-level=error --stop-time=10ms
```

Avsnitt 2.7.5 förklarar varför ordningen spelar roll.



Självkontroll. Döp din entitet till `hex_display`, med portarna `input` (`std_logic_vector(7 downto 0)`), `hex1` (out) och `hex0` (out, båda `std_logic_vector(6 downto 0)`), deklarerade i

den ordningen; dess testbänk ligger i `exercises/hex_display/` och sveper alla 256 ingångsvärdena, och kontrollerar varje halva oberoende så att en konstruktion som kastar om de två siffrorna fångas i stället för att gå igenom på de värden där båda halvorna råkar stämma överens.

Övning 2.9 En siffra som grindnät

KRETS

Om du har tid. Realisera en siffra som ett grindnät, så som kapitel 1 och avsnitt 2.1 lät dig göra:

- Ta de fyra ingångarna och ett segment, säg *a*, och härled den booleska ekvationen för det ur sanningstabellen med 16 rader. Ett Karnaughdiagram är värt att använda här: de flesta segment minimeras rejält.
- Upprepa för så många av de sex återstående segmenten som du har tålamod med, och bygg resultatet i CircuitVerse.
- Kopiera sedan hela grindnätet för att driva en andra display.

Den kopian är poängen med övningen. Att duplicera ett block grindar är precis vad det gjorde att instansiera `display` en andra gång i Övning 2.8, och det är värt att ha gjort en gång för hand för att se vad den enda raden VHDL står för.

ÖVNINGSGRUPP 5

Att namnge ett mellanresultat

Övning 2.10 `combo_logic`, skriven två gånger

VHDL

Skriv en entitet med namnet `combo_logic` som implementerar den booleska funktionen

$$x = ab + c'd$$

Avsnitt 2.2 hävdade att det inte kostar något och ger läsbarhet att namnge ett deluttryck med en intern signal. Den här övningen är där du bekräftar det snarare än tar på tro: du skriver funktionen två gånger, en gång som ett enda uttryck och en gång med termen $c'd$ utbruten i en intern signal med namnet *y*.

En signal deklarerad inuti en arkitektur är en ledning. Den är ingen variabel, den lagrar ingenting, och att namnge ett deluttryck med en sådan lägger varken till en grind, en fördröjning eller en vippa i kretsen. Den ger bara en del av nätet ett namn, så att du, när du läser det senare, kan se vad den delen är till för. Båda arkitekturerna nedan syntetiseras till samma grindar, och den utdelade testbänken går igenom mot vilken som helst av dem.

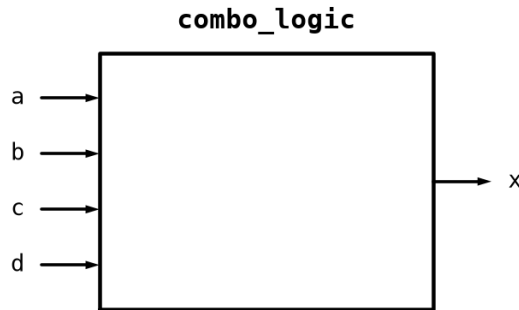
Entiteten måste ha portarna *a*, *b*, *c*, *d* (in) och *x* (out), alla `std_logic`.

a) Implementera hela funktionen med en enda konkurrent signaltilldelning: deklarera inga interna signaler, och översätt varje boolesk operation till motsvarande VHDL-operator.

b) Skriv om arkitekturen med en intern signal med namnet y , som representerar termen $c'd$, och tilldela x det slutliga uttrycket. Det här är samma form som `gate_network` i avsnitt 2.2, som namnger sin AND-term ab .

Bekräfta att de två arkitekturerna beskriver samma krets.

Tips: Två beskrivningar representerar samma kombinatoriska krets när de producerar samma utgång för varje möjlig ingångskombination. Med fyra ingångar finns 16 möjliga ingångskombinationer: skriv den fullständiga sanningstabellen och verifiera att båda arkitekturerna producerar samma värde på x på varje rad.



Självkontroll. Döp din entitet till `combo_logic`, med portarna `a`, `b`, `c`, `d` (in) och `x` (out), deklarerade i den ordningen; dess testbänk ligger i `exercises/combo_logic/` och kontrollerar alla 16 raderna. Den går igenom mot vilken som helst av arkitekturerna, vilket är poängen med jämförelsen ovan.

Sekvensnät

Att lagra tillstånd: D-låset, D-vippan, register och klockning.

I DET HÄR KAPITLET

- Varför kombinatorisk logik ensam inte kan minnas något, och vad "tillstånd" betyder i en krets.
- D-låset och D-vippan, och varför synkrona designers byggs av vippan.
- Klockning: period, frekvens, och stigande kontra fallande flank.
- Den synkrona processmallen i VHDL, och när en signaltilldelning faktiskt får effekt.
- Flankdetektering: att göra en nivå till en enda klockcykel lång puls.
- `led_toggle`: en knapptryckning som växlar en lysdiod och håller kvar den.

Två förutsägelser är värda att göra innan respektive simulering körs, och de är hela skälet till att kapitlet bygger låset innan det bygger vippan:

- för låset, vad Q gör när `enable` går låg medan D fortfarande ändrar sig.
- för vippan, vad Q gör när D ändrar sig *mellan* två klockflanker.

Repetition: Låset, vippan, registret och flankdetekteringen står i avsnitt 5.1–5.6 i *Digital Electronics*, om de behöver fräschas upp innan de skrivs i VHDL.

3.1 Från kombinatorik till sekvensnät

Varje krets i kapitel 1 och 2 var **kombinatorisk**: dess utgång är en ren funktion av dess nuvarande ingångar, så samma ingångar ger alltid samma utgång, och det finns ingen föreställning om "före" eller "efter" inuti kretsen.

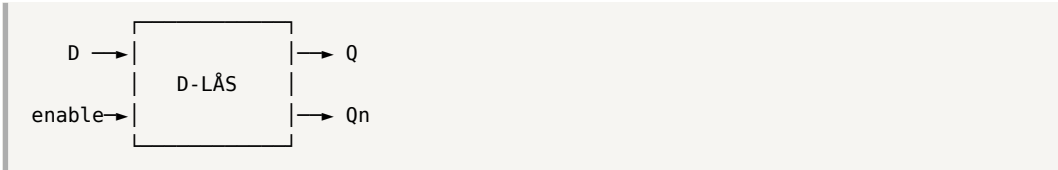
De flesta användbara system behöver mer. En räknare måste minnas sitt räknevärde. Ett register måste hålla kvar ett värde efter att signalen som producerade det har ändrats. En lysdiod som växlas av en knapp måste minnas om den är tänd, eftersom det inte alls ligger kodat i knappens ögonblickliga läge.

Alla dessa behöver **minne**: en utgång som beror på kretsens egen *föregående* utgång. Ett sekvensnät är kombinatorik med en del av utgången återkopplad till ingången, och den

återkopplingsslingan är vad som gör ett tillståndslöst grindnät till en krets med tillstånd. Det här kapitlet bygger den primitiv som gör kontrollerad återkoppling praktisk, D-vippan, och sedan de två saker du bygger direkt av den: register och flankdetektorer.

3.2 D-låset

Det enklaste lagringselementet är **D-låset**: två korskopplade grindar, grindade av en enable-ingång, som lagrar en enda bit.



D är värdet som ska lagras, enable styr om låset är *öppet* (genomsläppligt) eller *stängt* (hållande), Q är det lagrade värdet och Qn är alltid dess invers.

Varje utgång återkopplas in i den andras ekvation, vilket är precis avsnitt 3.1:s återkopplingsslinga:

$$Q = (D' \cdot \text{enable} + Q_n)'$$

$$Q_n = (D \cdot \text{enable} + Q)'$$

enable	Beteende	Resultat
1	låset är öppet (genomsläppligt)	$Q = D, Q_n = D'$; utgången följer ingången omedelbart
0	låset är stängt (låst)	Q och Qn håller kvar det de senast hade

Bygg det i CircuitVerse av en NOT-grind, två AND-grindar och två NOR-grindar (en NOR är en OR följd av en NOT). Korskoppla återkopplingen så att den NOR som producerar Q matar den NOR som ligger på Qn-sidan och tvärtom. Driv D och enable från ingångsströmbrytare och betrakta Q och Qn på lysdioder.

Testa det sedan:

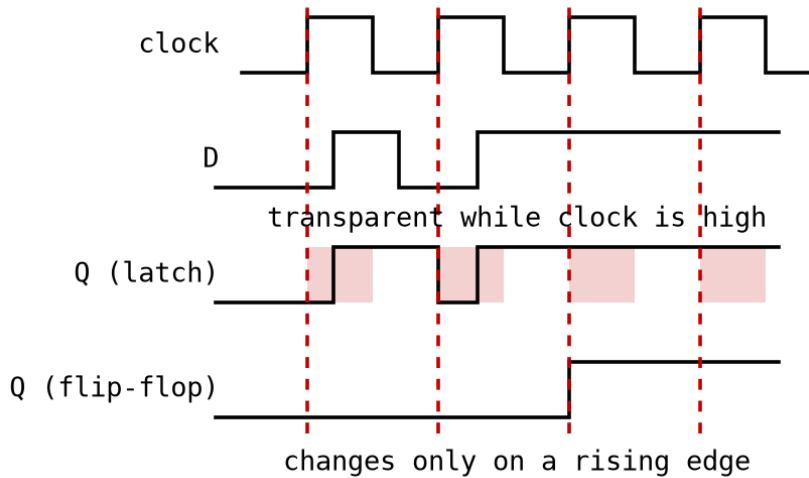
- Med enable = 1, ändra D och bekräfta att Q följer omedelbart. Låset är **genomsläppligt**.
- Med enable = 0, bekräfta att Q är oförändrad vad D än gör. Låset är **låst**.
- Växla D några gånger medan du slår om enable, och bekräfta att låset alltid fryser det Q höll i det ögonblick enable gick låg. Det är vad **nivåkänslig** betyder: genomsläpplig hela den tid enable är hög, inte bara i ett enda ögonblick.

Den sista punkten är också låsets grundläggande svaghet. Eftersom det är genomsläppligt hela den tid enable = 1 går varje glitch på D under det fönstret rakt igenom till Q. I ett stort synkront system är det långt svårare att garantera stabila data under en hel klockfas än att garantera dem kring ett enda ögonblick. Det är problemet D-vippan löser.

3.3 D-vippan

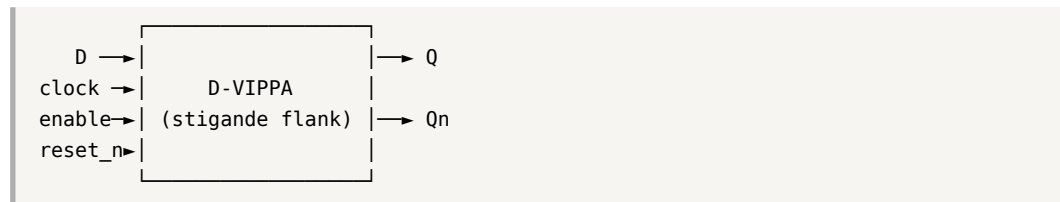
En **D-vippa** lagrar en bit på samma sätt som ett lås, men är **flanktriggad** i stället för nivå-känslig: den samplar D bara i det *ögonblick* klockan skiftar, inte under hela den tid klockan är hög eller låg.

Mata båda med samma D och samma klocka, så syns skillnaden direkt:



Figur 3.1 Ett lås följer D under varje fönster där klockan är hög, medan en vippa ändrar sig bara vid de stigande flankerna.

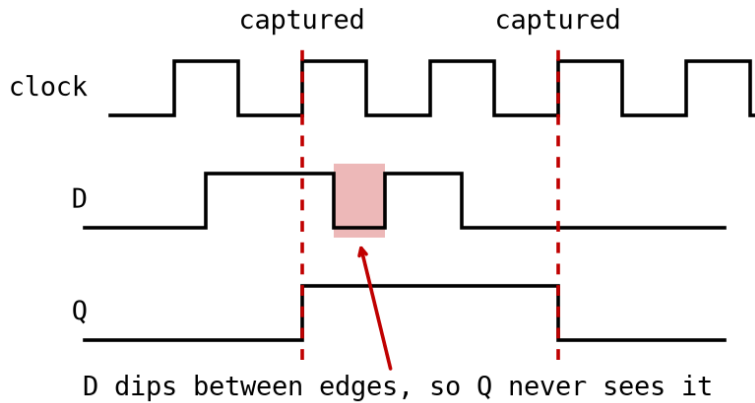
Låset ändrar sig tre gånger: två gånger för att D rörde sig medan klockan var hög, och en gång för att klockan gick hög mot ett D som redan hade ändrats. Vippan ändrar sig en gång, eftersom bara en av de fyra stigande flankerna fann D skilt från det värde Q redan höll. Båda betar sig korrekt; de svarar på olika frågor.



clock är tidsreferensen (avsnitt 3.4). enable grindar om vippan tar emot ett nytt värde *vid den triggande flanken*. reset_n är en asynkron, aktiv låg reset: närhelst den är 0 tvingas Q omedelbart till 0, oavsett klockan.

Villkor	Resultat
<code>reset_n = 0</code>	$Q = 0, Q_n = 1$, omedelbart (asynkront, ignorerar klockan helt)
<code>reset_n = 1</code> , stigande flank på <code>clock</code> , <code>enable = 1</code>	$Q = D, Q_n = D'$, samplat i det ögonblicket
<code>reset_n = 1</code> , stigande flank på <code>clock</code> , <code>enable = 0</code>	Q håller kvar sitt föregående värde
<code>reset_n = 1</code> , ingen stigande flank på <code>clock</code>	Q håller kvar sitt föregående värde, vad D än gör

Den sista raden är värd att se snarare än att läsa:



Figur 3.2 D går hög mitt i en cykel och fångas vid nästa stigande flank, dippar sedan och återhämtar sig helt mellan två flanker utan att Q någonsin rör sig.

D dippar låg och kommer tillbaka inom en enda klockperiod, och Q rör sig aldrig, eftersom ingen flank inträffade medan den var låg. Ett lås i samma läge hade följt med ner och upp igen.

Invärtas är en flanktriggad vippa två D-lås i serie, ett **master-slave**-par. Det ena är genomsläppligt medan klockan är låg och fångar D; det andra är genomsläppligt medan klockan är hög och för det fångade värdet vidare till Q. Nettoeffekten är att Q ändrar sig bara i det ögonblick klockan går från låg till hög, aldrig medan den ligger kvar på någondera nivå. Det yttre beteendet är vad VHDL:s `rising_edge()` modellerar (avsnitt 3.6).

Bygg den i CircuitVerse genom att bygga vidare på ditt lås från avsnitt 3.2:

- Duplicera låset så att två sitter i serie: *mastern* matar *slaven*, vars Q är vippans utgång.
- Lägg till `clock`, `reset_n`, D och `enable` som ingångar.
- Driv de två låsens genomsläpplighet från klockan och dess invers. För att hålla de två betydelserna isär, kalla den interna styrsignalen per lås för `gate`: *masterns* `gate` är `clock'` och *slavens* är `clock`. Vippans yttre `enable` är en annan signal och kopplas separat, nedan.

- Koppla det yttre enable som en 2-till-1-mux (avsnitt 2.4) på masterns dataingång: när enable = 1 släpper muxen igenom D, och när enable = 0 släpper den tillbaka in vippans eget Q, så att flanken fångar om det värde som redan är lagrat. Grinda *datat*, aldrig klockan.
- Lägg till resetten i **båda** låsen. Att tvinga Q låg kräver två ändringar i varje lås, inte en:
 - mata in ett aktivt högt reset (reset_n genom en NOT) i den NOR som producerar Q, vilket tvingar Q till 0.
 - AND:a in reset_n i termen D AND gate i samma lås, vilket frigör Qn att gå hög.
- Sätt klockperioden till något långsamt nog att titta på, t.ex. 1000 ms.

Med resetten inkopplad blir ekvationerna för varje lås:

$$Q = (D' \cdot gate + Q_n + reset)'$$

$$Q_n = (D \cdot gate \cdot reset_n + Q)'$$

En frestande genväg är att låta Qn-ekvationen vara och i stället bryta slingan, genom att AND:a reset_n på den Qn-ledning som matar Q:s NOR. Det räcker inte. Med gate = 1 och D = 1 lämnar det Qn på 0, vilket är det som håller Q på 1, så resetten gör i tyngst ingenting i precis det fall du är mest benägen att testa den i. Båda ändringarna, i båda låsen, eller ingendera.

Bekräfta att Q uppdateras bara i det ögonblick klockan går hög, och ligger stilla mellan flankerna även om D ändrar sig, och att en låg reset_n tvingar Q till 0 omedelbart, medan ett släpp av den inte ändrar något förrän vid nästa stigande flank.

3.4 Klockning

Klockan är en periodisk fyrkantvåg, som varje vippa i en synkron krets läser från samma källa. Två tal beskriver den: **perioden** T , tiden för en hel cykel, och **frekvensen** $f = 1/T$ i Hz.

Varje period har exakt två **flanker**: en **stigande flank** från 0 till 1, och en **fallande flank** från 1 till 0. En synkron design väljer en av dem, nästan alltid den stigande flanken, och uppdaterar varje vippa vid den flanken och bara den. Att använda en enda flank överallt är vad som gör tajmingen i en stor krets analyserbar: varje vippa ändrar sig vid samma väldefinierade ögonblick, så du kan resonera en klockcykel i taget i stället för att följa varje signal kontinuerligt.

Simulerade och verkliga klockor går i vitt skilda tidsskalor, och logiken är identisk i båda fallen:

- I CircuitVerse, sätt en period du hinner följa, t.ex. 1000 ms.
- På DE0-CV är systemklockan fasta 50 MHz, en period på 20 ns. Ingen uppfattar enskilda flanker i den takten, så du resonerar i termer av "vid nästa stigande flank", aldrig i verklig tid.

3.5 Register: vippor parallellt

Ett **register** lagrar mer än en bit: N D-vippor sida vid sida, en per bit, som alla delar samma clock, reset_n och enable.

```
D(0) → [D-FF] → Q(0)
D(1) → [D-FF] → Q(1)
D(2) → [D-FF] → Q(2)
    □      □
D(N-1) → [D-FF] → Q(N-1)
```

(clock, reset_n, enable delas av varje vipa ovan)

Det är hela idén: ett 8-bitars register är åtta D-vippor som uppdateras vid samma klockflank.

I VHDL instansierar du sällan N vippor explicit. Deklarera en `std_logic_vector` och tilldela hela den inuti en enda synkron process (avsnitt 3.6): varje bit syntetiseras till en egen vipa, och alla delar processens klocka och reset.

```
signal q: std_logic_vector(3 downto 0);
...
process (clock, reset_n) is
begin
    if (reset_n = '0') then
        q <= "0000";
    elsif (rising_edge(clock)) then
        if (enable = '1') then
            q <= d; -- d is a std_logic_vector(3 downto 0) input
        end if;
    end if;
end process;
```

Fyra bitar, en process, fyra vippor i hårdvara: i princip inte annorlunda än fyra `d_flip_flop`-instanser kopplade parallellt.

3.6 Den synkrona processmallen i VHDL

Varje synkront element i det här kapitlet, vippan, registret och flankdetektorn, använder samma form. Utan reset är en enda D-vipa:

```
process (clock) is
begin
    if (rising_edge(clock)) then
```

```

        q <= d;
    end if;
end process;

```

q tar d:s värde vid varje stigande flank och håller kvar det resten av tiden. Den enda raden inuti if-satsen räcker för att syntesen ska härleda en riktig D-vippa.

Regler

- **clock är den enda signalen i känslighetslistan**, om det inte finns en asynkron reset. Processen ska köra bara när klockan ändras; den har ingen anledning att reagera på något annat.
- En process reagerar på *varje* ändring hos en signal i känslighetslistan, inte bara stigande, så `rising_edge()` filtrerar specifikt ut den stigande flanken. All logik som uppdateras synkront ligger inuti det if:et.
- Varje signal som tilldelas inuti en sådan process syntetiseras till en vippa, en per bit.

Med en asynkron reset

En asynkron reset måste också stå i känslighetslistan, eftersom den kan aktiveras oberoende av klockan, och den har företräde när den är aktiv.

```

process (clock, reset_n) is
begin
    if (reset_n = '0') then
        q <= '0';
    elsif (rising_edge(clock)) then
        q <= d;
    end if;
end process;

```

Läs den i prioritetsordning: kontrollera resetten först, och leta efter en stigande flank bara om den är inaktiv. Om ingetdera gäller (en fallande flank, eller att `reset_n` går från 0 tillbaka till 1, vilket också triggar om processen) tilldelas ingenting och q behåller sitt föregående värde.

När tilldelningen faktiskt får effekt

En regel ligger under allt ovanstående, och det är det enda stället där `<=` betar sig olik tilldelning i alla språk du har skrivit mjukvara i.

En signaltilldelning får inte effekt när dess rad körs. Den **schemalägger** ett värde, som tillämpas först när processen avslutat det här varvet och suspenderar. Fram till dess ger varje läsning av den signalen det värde den höll *innan varvet började*, hur många tilldelningar som än redan har körts.

Det låter som en teknikalitet. Det är skälet till att mallen ovan är en vipa och inte en ledning:

```
process (clock) is
begin
  if (rising_edge(clock)) then
    b <= a;
    c <= b;
  end if;
end process;
```

Läst som mjukvara slutar både b och c med att hålla a. I hårdvara gör de inte det. Vid varje flank schemaläggs b att ta a:s värde, och c schemaläggs att ta det värde b höll *på väg in i den flanken*, inte det som just schemalagts på den. Resultatet är två vipor i en kedja: a når b efter en flank och c efter två. Byt plats på raderna och ingenting ändras, eftersom ingen av tilldelningarna kan observera den andras resultat.

Två konsekvenser värda att ta med sig:

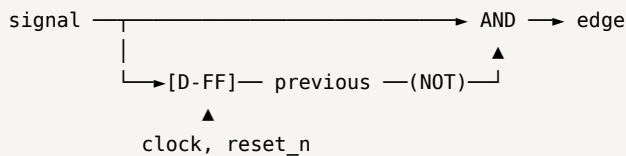
- **Ordningen spelar ingen roll** mellan tilldelningar till *olika* signaler i samma klockade process. Alla läser de gamla värdena och alla får effekt samtidigt. Det är motsatsen till den sekventiella läsning som ordet "process" inbjuder till, och det är därför en kedja av vipor kan skrivas i vilken ordning som helst.
- **Tilldela samma signal två gånger i ett varv och bara den sista tilldelningen överlever.** De tidigare kastas utan att någonsin nå signalen.

Avsnitt 3.7:s flankdetektor är precis tvåelements-kedjan ovan, och fungerar av precis det här skälet. Kapitel 5 återvänder till regeln från andra hållet, med en *variable*, som uppdateras omedelbart och därför beter sig så som den mjukvarumässiga läsningen väntar sig.

3.7 Flankdetektering

Vippans andra vardagsuppgift är att detektera det *ögonblick* en signal ändras, snarare än att läsa av dess nivå: lagra signalens föregående värde i en vipa och jämför det sedan med det nuvarande värdet med en grind.

- **Detektering av stigande flank** (0 till 1): $edge = current \cdot previous'$
- **Detektering av fallande flank** (1 till 0): $edge = current' \cdot previous$



edge är hög i exakt en klockcykel, den omedelbart efter övergången, eftersom previous kommer ikapp först vid *nästa* flank. Det är mekanismen bakom ”gör något en gång per knapptryckning” i stället för ”gör något varje klockcykel knappen hålls nere”, vilket är precis vad avsnitt 3.9 löser.

3.8 Genomarbetat exempel: en D-vippa i VHDL

d_flip_flop.vhd implementerar avsnitt 3.3 och 3.6 tillsammans: ett enable som grindar om en stigande flank fångar d, och en asynkron aktiv låg reset.

```
lectures/L03/d_flip_flop/d_flip_flop.vhd

-----
-- Rising-edge D flip-flop with asynchronous reset and enable.
--
-- Inputs:
--   - clock   : 50 MHz system clock.
--   - reset_n : Active-low asynchronous reset.
--   - d       : Data input.
--   - enable  : Capture enable ('1' = load d on the edge, '0' = hold).
-- Outputs:
--   - q       : Flip-flop output.
--   - q_n     : Inverse of q.
-----

library ieee;
use ieee.std_logic_1164.all;

entity d_flip_flop is
    port(clock, reset_n, d, enable: in std_logic;
         q, q_n                : out std_logic);
end entity;

architecture behaviour of d_flip_flop is
    -- Internal representation of q.
    signal q_s: std_logic;
begin
    q    <= q_s;
    q_n <= not q_s;

    -- Implement D flip flop logic.
    process (clock, reset_n) is
    begin
        if (reset_n = '0') then
            q_s <= '0';
```

```

        elsif (rising_edge(clock)) then
            if (enable = '1') then
                q_s <= d;
            end if;
        end if;
    end process;
end architecture;

```

Dess enda process är varianten med reset plus enable av avsnitt 3.6:s mall: resetten har företräde, sedan fångar en stigande flank *d* in i en intern signal *q_s* om *enable* = '1'. *q_s* finns eftersom en *utgångsport* inte kan läsas tillbaka inuti samma arkitektur; *q* och *q_n* drivs båda kombinatoriskt från den, utanför processen.

För att prova den på hårdvara, tilldelar *clock* till systemklockan på 50 MHz, *reset_n* till en tryckknapp, *d* och *enable* till två skjutströmbrytare, och *q/q_n* till två lysdioder. Slå sedan på *enable* och ändra *d*, så följer lysdioden med; slå av *enable* och ändra *d*, så gör den inte det.

Det finns ingen klockflank du kan uppfatta vid 50 MHz, så vad du egentligen bekräftar är att logiken fortfarande håller när du inte längre kan se de enskilda flankerna. Vippan ser ut att svara på din strömbrytare; i själva verket svarade den på en av femtio miljoner klockflanker den sekunden, och strömbrytaren avgjorde bara vilket värde den flanken fångade.

3.9 Genomarbetat exempel: flankdetekterad lysdiodsväxling

led_toggle.vhd kombinerar avsnitt 3.5, 3.6 och 3.7 till ett litet system: två oberoende tryckknappar, som var och en växlar sin egen lysdiod exakt en gång per tryck.

```

Lectures/L03/led_toggle/led_toggle.vhd
-----
-- Toggles led(i) on each falling edge of button_n(i); all LEDs off on reset.
--
-- Inputs:
--   - clock    : 50 MHz system clock.
--   - reset_n  : Active-low reset.
--   - button_n : Active-low push buttons.
-- Outputs:
--   - led: LED toggled by the matching button.
-----

library ieee;
use ieee.std_logic_1164.all;

entity led_toggle is
    port(clock, reset_n: in std_logic;
          button_n      : in std_logic_vector(1 downto 0));

```

```

        led          : out std_logic_vector(1 downto 0));
end entity;

architecture behaviour of led_toggle is
-- Previous button values (for edge detection).
signal button_prev_n: std_logic_vector(1 downto 0);

-- Button pressdown indicators (falling edge).
signal button_edge: std_logic_vector(1 downto 0);

-- LED states (necessary for toggling, as output signals cannot be read).
signal led_s: std_logic_vector(1 downto 0);
begin
    -- Detect a falling edge of button_n.
    button_edge <= (not button_n) and button_prev_n;
    led <= led_s;

    -- Update button_prev_n on system reset or on rising edge of the system clock.
    BUTTON_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            button_prev_n <= "11";
        elsif (rising_edge(clock)) then
            button_prev_n <= button_n;
        end if;
    end process;

    -- Update led_s on system reset or on rising edge of the system clock.
    LED_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            led_s <= "00";
        elsif (rising_edge(clock)) then
            for i in 0 to 1 loop
                if (button_edge(i) = '1') then
                    led_s(i) <= not led_s(i);
                end if;
            end loop;
        end if;
    end process;
end architecture;

```

Tre 2-bitars signaler, två hållna i klockade processer och en beräknad mellan dem:

- `button_prev_n` är ett 2-bitars register (avsnitt 3.5) som håller förra cykelns `button_n`, resatt till "11" eftersom knapparna är aktiva låga.
- `button_edge <= (not button_n) and button_prev_n` är avsnitt 3.7:s detektor för fallande flank tillämpad på båda bitarna samtidigt, hög i en cykel exakt när en knapp går från släppt till nedtryckt.
- `led_s` är ett andra 2-bitars register som, i stället för att fånga ett nytt värde varje cykel, **växlar** bit i bara när `button_edge(i) = '1'`. Utan den grindningen skulle det växla varje klockcykel i stället för en gång per tryck, vilket är hela poängen med att beräkna `button_edge`.

På kortet. Skapa Quartus Prime Lite-projektet mot DE0-CV:s 5CEBA4F23C7N precis som i kapitel 1, tilldela sedan `clock` till oscillatoren på 50 MHz, `reset_n` och `button_n(1 downto 0)` till tre tryckknappar, och `led(1 downto 0)` till två lysdioder. Varje tryck ska växla exakt en lysdiod, en gång, hur länge du än håller den nere.

En reservation, medvetet lämnad öppen. `button_n` är en rå signal rakt från en fysisk tryckknappspinne. I simulering är det en ren, ögonblicklig övergång; på riktig hårdvara är den varken ögonblicklig (kontakter studsar) eller synkroniserad till `clock`. Den här flankdetektorn är logiskt korrekt men *osäker* att koppla direkt till en riktig knapp på en FPGA utan mer arbete först, och det arbetet är precis vad kapitel 4 tar upp.

3.10 Sammanfattning

Det här ska du kunna nu

- Säga varför sekvensnät finns, förklara den funktionella skillnaden mellan ett D-lås och en D-vippa, och varför synkrona designer nästan uteslutande byggs av vippor.
- Läsa och skriva den synkrona processmallen, och känna igen ett register som flera vippor som delar klocka, reset och enable.
- Säga när en signaltilldelning får effekt: `<=` schemalägger snarare än tillämpar, så två tilldelningar i en klockad process bygger en kedja, och deras inbördes ordning spelar ingen roll.
- Bygga en flankdetektor av en vippa och en grind, och säga exakt vilken signal den producerar och hur länge.

Frågor att testa dig själv med

1. Vad är den funktionella skillnaden mellan ett D-lås och en D-vippa, och under vilket villkor ändrar var och en sin utgång?
2. Varför uppdaterar synkrona kretsar sitt tillstånd vid en enda klockflank i stället för att kontinuerligt reagera på sina ingångar?

3. Givet en signal och en vipa som håller dess värde från en cykel tidigare, hur kombinerar du dem till en puls som är hög i exakt en klockcykel när signalen går hög?
4. Varför måste `clock` vara den enda signalen i en vippprocess känslighetslista, om inte designen har en asynkron reset?
5. En klockad process innehåller `b <= a`; och därefter `c <= b`; . Vad håller `c` efter en klock-flank, och varför är det inte `a`? Vad ändras om du byter plats på raderna, och varför?

Blicken framåt

Kapitel 4 går vidare med:

- Varför den råa knappen som matar det här kapitlets flankdetektor kan driva en vipa in i ett odefinierat, instabilt tillstånd, och varför det är ett annat sorts problem än något här.
- Dubbelvippssynkroniseraren, och vad den gör och inte gör åt en studsande knapp.
- Samma lysdiodsväxling, ombyggd så att den faktiskt är säker på riktig hårdvara.

3.11 Övningar

Så kontrollerar du ditt arbete. Varje övning nedan som ber dig skriva en VHDL-modul har en utdelad självkontrollerande testbänk under `lectures/L03/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7 för de tre kommandona. Det här är det första kapitlet vars övningar behöver det för en klockad design, så det är värt att läsa om innan du börjar med `register4`.

ÖVNINGSGRUPP 1

D-låset

Övning 3.1 Den första kretsen som minns

KRETS

Bygg den första kretsen i den här boken som kan *minnas* något.

Allt du har byggt hittills har varit kombinatoriskt: utgångarna följer ingångarna, och i samma ögonblick som ingångarna ändras ändras utgångarna med dem. Ett lås är det minsta möjliga avsteget från det. Det har en dataingång `D` och en styringång `enable`, och det har två utgångar, `Q` och dess komplement `Qn`. Så länge `enable` är hög är låset **genomsläppligt**: `Q` följer helt enkelt `D`, och det ser ut som en ledning. Det intressanta är vad som händer när `enable` går låg. Låset **låser**, och håller kvar det `D` var i det ögonblicket, och från och med då kan `D` göra vad det vill utan att `Q` märker det. Det hållna värdet är en bit minne, och det är byggt av ingenting annat än de grindar du redan känner till, kopplade så att utgången återkopplas till ingången.

Q_n är ingen dekoration. Återkopplingslingan är det som får kretsen att minnas, och de två utgångarna är det som matar varandra: Q beräknas ur Q_n och Q_n ur Q . Bygg det så ser du slingan; det är hela poängen med att rita det för hand innan du ens möter VHDL:en.

Använd ekvationerna från avsnitt 3.2:

$$Q = (D' \cdot enable + Q_n)' \quad Q_n = (D \cdot enable + Q)'$$

- Realisera motsvarande grindnät: rita nätet för hand, och återskapa och simulera det i CircuitVerse.
- Testa det genomsläppliga och det låsta tillståndet. Sätt `enable = 1`, ändra D , och observera vad som händer med Q och Q_n . Sätt sedan `enable = 0`, ändra D igen, och observera vad som händer. Förklara skillnaden mellan de två fallen.
- Kör D och `enable` genom kombinationerna 00, 01, 10, 11, två gånger. Bekräfta att du observerar båda fallen: låset låser efter att ha varit genomsläppligt med $D = 1$, och låset låser efter att ha varit genomsläppligt med $D = 0$. Förklara hur värdet på Q beror på värdet på D i det ögonblick `enable` ändras från 1 till 0.

ÖVNINGSGRUPP 2

D-vippan

Övning 3.2 Från lås till vippan

KRETS

Bygg ut D-låset från Övning 3.1 till en D-vippan, enligt avsnitt 3.3.

Vippan ska ha ingångarna `clock`, `reset_n`, D och `enable`, och utgångarna Q och Q_n .

Resetsignalen är **asynkron** (den får verkan utan att vänta på en klockflank) och **aktiv låg** (att sätta `reset_n = 0` ska tvinga Q till 0 omedelbart).

Tips

- Resetten måste göra två saker i vart och ett av de två interna låsen, inte en:
 - Mata in ett aktivt högt `reset` (`reset_n` genom en NOT-grind) i den NOR-grind som producerar Q . Det tvingar Q låg utan att vänta på en klockflank.
 - AND:a in `reset_n` i den AND-grind som producerar det låsets produkt av D och `enable`. Det frigör Q_n att gå hög.
 - Att grinda enbart återkopplingsvägen för Q räcker inte: med `enable = 1` och $D = 1$ lämnar det Q_n låg, och en låg Q_n är precis det som håller Q hög, så resetten skulle inte göra någonting.
- Driv de två interna låsens `enable`-ingångar med `clock` respektive inversen av `clock`.
- Följ avsnitt 3.3 för hur den yttre `enable`-ingången styr om vippan tar emot ett nytt värde.

- a) Realisera motsvarande grindnät: rita det för hand, återskapa och simulera det i Circuit-Verse, och sätt klockperioden till 1000 ms, långsamt nog för att observera beteendet med ögat.
- b) Testa det klockade beteendet: ändra D under klockans höga fas, och sedan under dess låga fas, och observera Q och Qn. Bekräfta att utgångarna ändras bara vid den aktiva klockflanken, aldrig mellan klockflankerna. Förklara hur de två interna låsen samverkar för att ge flanktrigget beteende.
- c) Testa den asynkrona resetten: sätt först Q = 1, aktivera sedan resetten genom att sätta reset_n = 0, och observera om Q nollställs omedelbart eller vid nästa klockflank. Förklara resultatet genom att hänvisa till skillnaden mellan en asynkron signal och en synkron signal.

ÖVNINGSGRUPP 3

Register

Övning 3.3 register4

VHDL

Skriv en fullständig VHDL-entitet med namnet `register4`: fyra vippor sida vid sida, som delar en klocka, en reset och en enable.

Det här är övningen där vippan slutar vara en kuriositet och blir en byggsten. En ensam D-vippa lagrar en bit, vilket inte är till mycket nytta i sig; vad en design egentligen vill ha är någonstans att förvara ett *tal*. Sätt fyra vippor på rad, koppla samma klocka till dem alla, och du har ett fyrabitars register. Ingenting hos den enskilda vippan ändras. Bara bredden gör det.

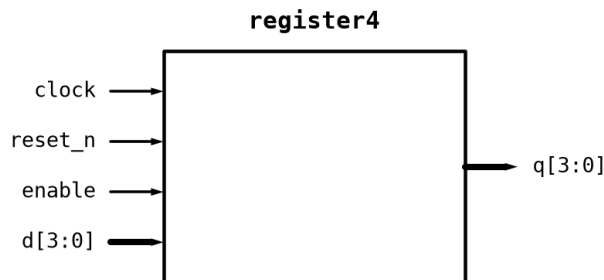
Det som är värt att lägga märke till är hur lite VHDL:en behöver ändras för att säga det. Du skriver samma synkrona processmall från avsnitt 3.6 som du redan använde för en bit, med `std_logic_vector(3 downto 0)` i stället för `std_logic`. Det finns ingen loop, ingen upprepning, inga fyra kopior av någonting. Det är ingen genväg som språket erbjuder dig; det är ett faktum om hårdvaran, och del c) nedan ber dig säga varför.

Entiteten ska ha:

Port	Riktn.	Typ	Beskrivning
clock	in	std_logic	Systemklocka.
reset_n	in	std_logic	Asynkron, aktiv låg. Nollställer registret till "0000".
enable	in	std_logic	'0' behåller nuvarande värde; '1' fångar d vid nästa stigande flank.
d	in	std_logic_vector(3 downto 0)	Värdet som ska fångas.
q	out	std_logic_vector(3 downto 0)	Det lagrade värdet.

Skriv ut processen själv i stället för att kopiera det genomarbetade vippexemplet i avsnitt 3.8. Att knappa in den är vad som får mallen att sluta vara något du känner igen och börja vara något du kan producera ur minnet, och du kommer att skriva den i varenda återstående kapitel i den här boken.

- Skriv entiteten och den synkrona processen, och anpassa mallen från en enda bit till en fyrabitars vektor.
- Kontrollera den med testbänken, så som beskrivs högst upp i det här avsnittet.
- Förklara varför samma mall skalar från en vipa till fyra utan någon strukturell ändring, medan en fyrabitars *räknare* inte helt enkelt kan vara fyra enbitsräknare sida vid sida. Vad skiljer de två fallen åt?



Självkontroll. Döp din entitet till `register4`, med portarna `clock`, `reset_n`, `enable`, `d` (`std_logic_vector(3 downto 0)`) och `q` (`std_logic_vector(3 downto 0)`), deklarerade i den ordningen; dess testbänk finns i `exercises/register4/`.

ÖVNINGSGRUPP 4

Flankdetektering

Övning 3.4 Växlingskretsen för hand

KRETS

Konstruera en krets som växlar en lysdiod en gång för varje samplad stigande flank hos en knappsignal. Följ avsnitt 3.7 och avsnitt 3.9.

Kretsen ska ha ingångarna `button`, `clock` och `reset`, och utgången `led`.

I den här övningen är `reset` aktiv hög, till skillnad från resetten i det genomarbetade exemplet. Den är fortfarande **asynkron**, som varje `reset` i den här boken: aktivera den så nollställs `led` omedelbart, utan att vänta på en klockflank. I mallen från avsnitt 3.6 betyder det att `resetgrenen` stannar utanför testet `rising_edge(clock)`, och att bara polariteten på `resettestet` ändras.

Tips

Använd två vipor. Den ena lagrar det föregående samplade värdet av `button`, och det värdet används för att detektera en stigande flank. Den andra lagrar lysdiodens nuvarande tillstånd, och växlas bara när en stigande flank detekteras.

a) Realisera motsvarande grindnät: rita det för hand, återskapa och simulera det i Circuit-Verse, och sätt klockperioden till 1000 ms.

b) Testa flankdetektorn: håll `button` hög under flera klockcykler i följd, bekräfta att `led` växlar exakt en gång, och att den inte växlar en gång per klockcykel. Förklara vilken lagrad signal som hindrar kretsen från att gå på gång på gång detektera samma höga nivå som en ny stigande flank.

Övning 3.5 led_toggle_single

VHDL

Implementera kretsen från Övning 3.4 i VHDL.

a) Skapa en entitet med namnet `led_toggle_single` med:

Port	Riktn.	Typ	Beskrivning
<code>clock</code>	in	<code>std_logic</code>	Systemklocka.
<code>reset</code>	in	<code>std_logic</code>	Asynkron, och aktiv hög , till skillnad från varje annan modul i den här boken. Aktivera den så nollställs <code>led</code> utan att vänta på en klockflank, så resetgrenen hör hemma utanför testet <code>rising_edge(clock)</code> .
<code>button</code>	in	<code>std_logic</code>	Växlar <code>led</code> en gång per tryck.
<code>led</code>	out	<code>std_logic</code>	Lysdiodens tillstånd.

Lägg märke till enbistyperna: det genomarbetade exemplet använder `std_logic_vector`-portar för två knappar och två lysdioder, medan den här driver ett enda par.

Kopiera inte `led_toggle.vhd` rakt av. Gör så här i stället:

- Härled de två klockade processerna själv: den ena lagrar det föregående värdet av `button`, den andra lagrar och växlar det nuvarande värdet av `led`.
- Härled flankdetekteringsekvationen för en stigande flank och en aktivt hög knapp, alltså avsnitt 3.7:s $edge = current \cdot previous'$.
- Använd inte ekvationen för fallande flank från det genomarbetade exemplet, som utgår från en aktivt låg knapp.

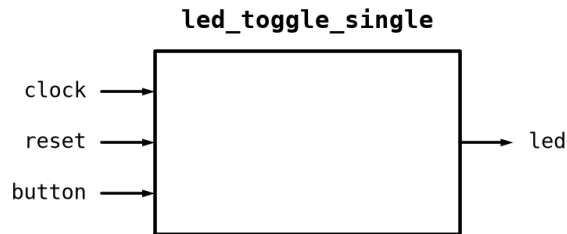
b) Verifiera designen med dess testbänk.

Testbänken driver samma sekvens som du skulle gå igenom för hand på ett kort:

- Den aktiverar `reset` och kontrollerar att `led` är nollställd.
- Den drar `button` hög och kontrollerar att `led` växlar exakt en gång.
- Den **håller `button` hög** under ytterligare några klockcykler och kontrollerar, efter var och en av dem, att `led` *inte* växlar igen.

- Den släpper och trycker igen, och kontrollerar att det andra trycket växlar led en gång till.
- Den aktiverar reset med led tänd och **utan någon klockflank efteråt**, och kontrollerar att led nollställs ändå. Det är den kontroll som faller om du lägger resetten inuti grenen `rising_edge(clock)`.

Den tredje kontrollen är den som faller om din flankdetektor är fel. En led som växlar varje klockcykel medan knappen hålls nedtryckt betyder att du drev växlingen från buttons **nivå** i stället för från dess flank. Läs om ekvationen ovan: `edge` är hög bara när `button` är hög *nu* och dess lagrade föregående värde är lågt.



Självkontroll. Döp din entitet till `led_toggle_single`, med portarna `clock`, `reset`, `button` (in) och `led` (out), deklarerade i den ordningen; dess testbänk finns i `exercises/led_toggle_single/`. Lägga märke till att `reset` är **aktiv hög** här, till skillnad från i det genomarbetade exemplet.

Metastabilitet och synkronisering

Att göra en asynkron ingång säker att använda i ett synkront system.

I DET HÄR KAPITLET

- Varför en asynkron ingång kan förstöra de rena flankdetektorer och register som byggdes i kapitel 3.
- Metastabilitet: vad det är, och varför det inte helt enkelt går att konstruera bort.
- Dubbelvippsynkroniseraren, och mönstret "aktivera asynkront, släpp synkront" för reset.
- Att återanvända samma kedja för flankdetektering, och vad den gör och inte gör åt studs.
- Generics: en modul som betjänar flera storlekar.
- `led_toggle_sync`, driven av en riktig, studsande tryckknapp.

Innan synkroniseraren sätts in: förutsäg vad en riktig, asynkron knapp gör med kapitel 3:s flankdetektor. Den förutsägelsen är skälet till hela kapitlet.

Argumentet som är värt att hålla fast vid är varför två vippor gör felet *osannolikt* snarare än omöjligt. Det är den delen som inte överlever att bli ihågkommen som en tumregel.

Repetition: Metastabilitet, dubbelvippsynkroniseraren och vad den inte löser står också i avsnitt 5.7–5.9 i *Digital Electronics*.

4.1 Problemet: en knapp är inte en testbänkssignal

I kapitel 3 byggde du flankdetektorer och resonerade om dem mot en ren klocka, där varje ingång ändrades prydligt mellan flankerna, precis så som en simulator ritat det. En tryckknapp, en extern resetledning, eller vilken signal som helst från en annan klockdomän ändras när omvärlden bestämmer sig för det, helt oberoende av din FPGA:s klocka.

Varje D-vippa har två tidskrav kring den aktiva klockflanken:

- en **setup-tid**: hur länge D måste vara stabil *före* flanken.
- en **hold-tid**: hur länge D måste förbli stabil *efter* den.

Om en ingång ändras inuti det fönstret är vippan inte garanterad att fånga en ren 0 eller 1, och du kan aldrig lova att den hamnar utanför, eftersom du inte styr när en extern signal

ändras. Resten av det här kapitlet löser det inte genom att undvika problemet utan genom att innesluta det.

4.2 Metastabilitet: vad som faktiskt händer inuti vippan

En D-vippa är byggd av korskopplade grindar som slår om till 0 eller 1 i samma ögonblick som klockan samplar ingången. Om D ändras för nära flanken kan den interna återkopplingslingan lämnas i balans mellan de två tillstånden, varken helt hög eller helt låg, längre än väntat. Det är **metastabilitet**.

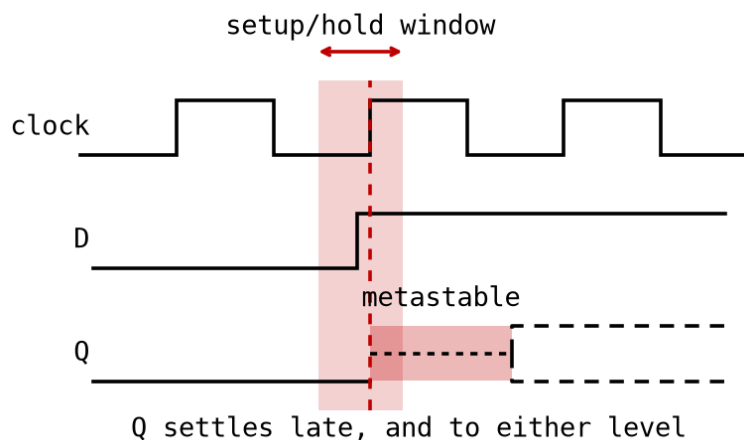
Tänk dig en kula som balanserar exakt på toppen av en kulle. Med tillräckligt med tid rullar den ner åt ena hållet, men exakt när den bestämmer sig, och åt vilket håll, går inte att förutsäga från det ögonblick den placerades där, och så länge den fortfarande balanserar är dess position inte något giltigt digitalt värde alls.

En metastabil utgång beter sig likadant:

- den stabiliserar sig nästan alltid till en giltig 0 eller 1 av sig själv, givet tillräckligt med tid.
- "tillräckligt med tid" är inte gratis, och är inte densamma varje klockcykel.
- om den ostabiliserade spänningen når logik nedströms innan den lagt sig kan olika grindar med all rätt läsa den olika: en ser 0, en annan 1, i samma ögonblick.

Den oenigheten är där riktiga system beter sig fel: en räknare som räknar upp i en del av kretsen men inte i en annan, eller en tillståndsmaskin i ett omöjligt tillstånd. Allt i det här kapitlet finns till för att se till att metastabiliteten stabiliserar sig *innan* en signal rör resten av din design.

I tid ser hela problemet ut så här:



Figur 4.1 D ändras inuti setup/hold-fönstret kring en stigande klockflank, vilket lämnar Q metastabil innan den sent lägger sig på endera nivån.

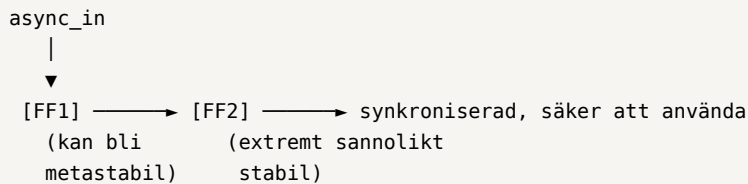
Läs det från vänster till höger. Det skuggade bandet är fönstret från avsnitt 4.1, t_{su} före flanken och t_h efter den. D ändras inuti det, vilket är precis vad en ingång du inte styr står fritt att göra. Q är sedan varken 0 eller 1 under en stund, och när den väl lägger sig är både *när* den lägger sig och *vilken nivå* den lägger sig på utanför din kontroll. Båda de streckade fortsättningarna är legitima utfall av samma ingång.

Lägg märke till vad diagrammet inte visar: ett felaktigt svar. Att sampla en signal mitt i en övergång får ge dig endera nivån, och det är i sig harmlöst, eftersom ingången faktiskt höll på att ändras i det ögonblicket. Problemet är fördröjningen innan du får ett svar över huvud taget, och det är den nästa avsnitt köper tid för.

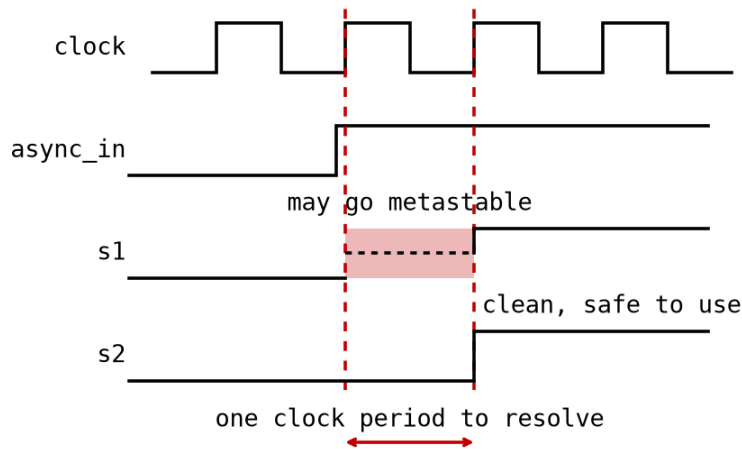
4.3 Dubbelvippsynkroniseraren

Standardlösningen är **dubbelvippsynkroniseraren**:

- Placera två D-vippor i serie på varje asynkron ingång: varje ingång som ändras utan någon fast relation till din klocka. En tryckknapp, en resetledning, en signal som anländer från en annan klockdomän.
- En ingång som redan klockas av *denna* klocka är inte asynkron, och får **inte** synkroniseras. Att göra det fördröjer den bara, och förskjuter dess tidsläge relativt allt runt omkring. `seq_detect_mealy` i kapitel 8 är precis det fallet: dess `din` är synkron seriell data, så den går rakt in i maskinen medan `reset_n` fortfarande får en synkroniserare.
- Använd aldrig något annat än den *andra* vippans utgång på andra ställen i designen.



Samma två vippor, i tid:



Figur 4.2 `async_in` ändras strax före en klockflank, `s1` blir metastabil under en klockperiod, och `s2` kommer ut ren en flank senare.

FF1 är den som är exponerad för den asynkrona ingången, så den är den som kan bli metastabil. Dess utgång behöver bara nå FF2, och den har en hel klockperiod på sig. Sannolikheten att en metastabil nod fortfarande är ostabiliserad efter en hel period är liten, och krymper ju längre tid den får, så när FF2 samplar på nästa flank har FF1 nästan alltid lagt sig. FF2:s utgång är det du bygger resten av din synkrona logik på.

Det här är sannolikhetsbaserat, inte absolut, och avsnitt 4.4 säger vad det innebär. Vid de hastigheter som används i den här boken, totalt MHz, är två steg det gängse, industriaccepterade svaret.

En regel knyter ihop det: **varje asynkron ingång får sin egen dubbelvippkedja innan den används någon annanstans.** Externa resetknappar, tryckknappar, och senare varje **enbitssignal** som korsar från en annan klockdomän.

Det sista ordet är bärande, och det är den enda begränsning som är värd att lära sig innan du överanvänder regeln. En dubbelvippkedja synkroniserar **en bit**. Kör en flerbittsbuss genom en kedja per bit och varje bit är säker för sig men de är inte säkra *tillsammans*: var och en stabiliserar sig oberoende till det gamla eller det nya värdet på samma flank, så mottagaren kan sampla en kombination som aldrig fanns på bussen. En räknare som stegar från 0111 till 1000 har alla bitar ändrade på en gång, och en synkroniserare per bit kan ge dig 1111 eller 0000, värden räknaren aldrig höll.

Bussar som korsar klockdomäner behöver en handskakning, en FIFO, eller Gray-kodning, som ordnar det så att bara en bit någonsin ändras åt gången. Ingenting i del I korsar klockdomäner med en buss, så du kommer inte att behöva dem där; gruppprojektet återkommer till begränsningen i kapitel 12, där samma synkroniserare skrivs en gång och återanvänds. Att veta att tvåvippsvaret stannar vid en bit räcker för nu.

4.4 Intuition för tidsförhållandena: varför "nästan säkert" duger

En synkroniserare köper inte "ingen metastabilitet". Den köper metastabilitet som får en hel klockperiod på sig att stabilisera sig innan den kan påverka något annat, och det är ett statistiskt argument snarare än ett bevis. Den korrekta hållningen är "rimligt säker, med stöd av en extremt låg, väl karakteriserad felfrekvens", inte "bevisbart säker under alla förhållanden", vilket är skälet till att extra steg finns för extremfall i stället för att två förklaras tillräckliga av en naturlag.

Sambandet har en standardform, värd att se en gång även om du inte kommer att räkna med den här:

$$MTBF = \frac{e^{t_r/\tau}}{T_w \cdot f_{\text{clock}} \cdot f_{\text{data}}}$$

Läs den för vad den säger snarare än för siffror. t_r är den stabiliseringstid ett steg får, ungefär en klockperiod; τ och T_w är båda egenskaper hos vippa, där τ är hur snabbt ett metastabilt värde avklingar och T_w bredden på det fönster kring klockflanken där ingången över huvud taget kan framkalla ett sådant. Att lägga till ett steg köper ytterligare ett t_r i exponenten, så **täljaren växer exponentiellt med varje steg du lägger till**, mot en klockcykels ökad latens. Nämnaren växer linjärt med klockfrekvensen och, precis lika viktigt, med f_{data} : hur ofta den asynkrona ingången faktiskt ändras. En knapp som trycks en gång i sekunden och en dataledning som växlar i 10 MHz skiljer sig sju tiopotenser åt enbart i den termen, vilket är skälet till att argumentet för ett tredje steg är en fråga om ingången, inte om klockan.

4.5 Att synkronisera en resetsignal: aktivera asynkront, släpp synkront

Reset förtjänar ett eget mönster, som `reset_sync` demonstrerar. Det finns ingen `reset_sync.vhd` att öppna i kursrepot: den är Övning 4.8, och du skriver den. Listningen nedan är hela modulens process.

```
process(clock, reset_n) is
begin
    if (reset_n = '0') then
        reset_s1_n <= '0';
        reset_s2_n <= '0';
    elsif (rising_edge(clock)) then
        reset_s1_n <= '1';
        reset_s2_n <= reset_s1_n;
    end if;
```

```
end process;
```

- **Aktivera asynkront.** När `reset_n` går låg tvingas båda stegen omedelbart till 0, utan något beroende av klockan. En reset som väntade på en klockflank skulle omintetgöra sitt eget syfte: den måste få verkan i samma ögonblick som den behövs, med eller utan klocka.
- **Släpp synkront.** När `reset_n` återgår till 1 försvinner inte resetvillkoret överallt på en gång. Det skiftas ut genom de två stegen som vilken annan asynkron ingång som helst, så `reset_s2_n`, den signal designen faktiskt använder, släpper rent på en klockflank tillsammans med allt annat. Utan detta skulle olika vippor kunna lämna reset på olika cykler och för ett ögonblick vara oense om systemets tillstånd, samma sorts fel som metastabilitet orsakar.

Det är därför `reset_n` synkroniseras av sin egen kedja innan något annat beror på den, inklusive knappsynkroniseraren, vars resetingång är `reset_s2_n` snarare än den råa `reset_n`.

4.6 Att återanvända kedjan för flankdetektering, och studsfiltrering i förbigående

Knappen behöver synkroniseras, och från kapitel 3 behöver du dessutom detektera *när* den trycks ned snarare än dess nivå. `button_sync`, som är Övning 4.9 och därmed också din att skriva snarare än en fil i kursrepot, får båda ur en enda kedja av **tre** vippor:



Vippa 1 och 2 är avsnitt 4.3:s synkroniserare, vilket gör `button_s2_n` till det stabila nuvarande värdet. Vippa 3 lagrar förra cykelns `button_s2_n` som `button_s3_n`, det föregående värdet. Vippa 1 och 2 är värda att lägga märke till för sig: Övning 5.4 bryter ut den dubbelvippkedjan till en återanvändbar sync-modul, och lämnar den tredje vippan här som flankdetektorns egen. Att jämföra steg 2 mot steg 3 ger flankdetektering gratis:

```
button_edge_s2 <= (not button_s2_n) and button_s3_n;
```

vilket är sant under exakt en klockcykel: den där knappen läses som nedtryckt nu och inte var det cykeln före.

Synkronisering löser i sig bara metastabilitet. Den säger ingenting om en mekanisk knappens benägenhet att studsas: att sluta och bryta kontakt flera gånger under den första millisekunden eller så, där varje studs ser ut som ett eget tryck. Det som håller den här bokens övningar från att dränkas i studs är *kombinationen* av kedjan och ett detekteringsfönster som är långsamt i förhållande till studsens:

- I det här kapitlets CircuitVerse-övningar är klockperioden en hel sekund. Studsen lägger sig på långt mindre än så, så vid nästa flank läser `button_s2_n` en enda ren nivå. Kedjan ser ut att studsfiltrera, men den samplar egentligen bara för sällan för att se studsens.
- På FPGA:n är klockan 50 MHz, ett sampel var 20 ns, medan studsen varar från under en millisekund till några tiotals millisekunder, många tusen cykler. Vid den hastigheten detekterar `button_sync` troget *varje* studsövergång som en egen flank, och lysdioden kan växla flera gånger för ett enda fysiskt tryck.

Så exakt: dubbelvippsynkroniseraren löser metastabilitet tillförlitligt vid de hastigheter som används här. Om den dessutom *ser ut* att studsfiltrera beror helt på att klockperioden är långsam i förhållande till knappens studstid, vilket stämmer i CircuitVerse och inte automatiskt stämmer på FPGA:n.

4.7 Vad en riktig studsfiltrering i hårdvara tillför

Den här boken har inget eget kapitel om studsfiltrering, och de genomarbetade exemplen fortsätter medvetet att använda avsnitt 4.6:s krets, studs och allt. Den duger för en demolysdiod. En produktionsdesign lägger till något av följande *ovanpå*:

- **Ett analogt RC-lågpasfilter**, eventuellt med en Schmitt-triggerbuffert, på knappingången innan den når en FPGA-pinne. RC-paret jämnar ut de snabba slut- och brytövergångarna till en enda gradvis flank, och Schmitt-triggern, med sina två omslagströsklar, gör tillbaka det till en enda ren digital övergång. Det löser studsens innan signalen ens är digital, men kostar komponenter på kortet och hjälper inte för ingångar som inte är mekaniska kontakter.
- **En digital studsfiltreringsräknare.** I stället för att agera på nästa detekterade flank, kräv att ingången håller en ny nivå under någon minsta tid, några millisekunders värde av klockcykler, innan den accepteras. Det bygger ditt eget konstgjort långsamma detekteringsfönster på chippet, samma effekt som CircuitVerses 1-sekundersklocka ger gratis, utan att sakta ner systemklockan. Du möter räknare i kapitel 6 och timers, som den här tekniken behöver, i kapitel 7.

Båda sitter ovanpå synkroniseraren, inte i stället för den: även en perfekt studsfiltrerad signal är fortfarande asynkron mot din klocka och behöver fortfarande en dubbelvippkedja.

4.8 Generics: en modul, flera storlekar

reset_sync och button_sync är byggda av samma idé, men bara den ena har en fråga att besvara. En reset är en bit och kommer alltid att vara det. Hur många *knappar* har en design? Det här kapitlets led_toggle_sync har en, kapitel 8:s fsm_led har två. Att skriva ut samma trevippskedja en gång per bredd är precis den dubbling som uppdelningen i moduler var till för att undvika.

En **generic** besvarar den: ett värde som en entitet deklarerar vid sidan av sina portar, fastlagt när designen byggs snarare än föränderligt medan den kör.

```
entity button_sync is
  generic(COUNT: natural range 1 to 3 := 1);
  port(clock, reset_s2_n: in std_logic;
        button_n      : in std_logic_vector(COUNT-1 downto 0);
        button_edge_s2 : out std_logic_vector(COUNT-1 downto 0));
end entity;
```

- COUNT betar sig som en konstant inuti arkitekturen. Den kan inte ändras medan kretsen kör; den avgörs innan en enda grind placerats.
- := 1 är dess standardvärde, som används av varje instansiering som inte säger något.
- Portbredderna är skrivna *i termer av den*, och så är varje intern signal som följer.

Att åsidosätta den

En instansiering skickar generics med en generic map, före sin port map:

```
button_sync1: entity work.button_sync
  generic map(2)
  port map(clock, reset_s2_n, button_n, button_edge_s2);
```

Generics matchas **positionellt**, i deklaraationsordning, precis som portar gör i avsnitt 2.6. Att utelämna generic map tillämpar standardvärdet; den här boken skriver ut den explicit även när den stämmer, eftersom en läsare inte ska behöva öppna modulen för att ta reda på hur bred en instans är.

När en generic gör rätt för sig

button_sync har en och reset_sync har ingen, och den skillnaden är hela regeln: **lägg till en generic när något genuint varierar mellan instanser, inte av princip.**

button_sync instansieras med COUNT = 1 av avsnitt 4.9:s led_toggle_sync och med COUNT = 2 av kapitel 8:s fsm_led: två bredder, en fil. reset_sync är en bit överallt, så en WIDTH-generic på den vore en parameter som ingen någonsin skickar något annat än 1 till, och en sak till för en läsare att kontrollera. Den andra instansieringen är det som gör en generic

värd att ha; en parameter med exakt ett värde i hela boken vore bara ett extra mellanled. `button_syncs` egen testbänk instansierar den med *båda* bredderna i samma simulering, just för att bevisa att en fil verkligen betjänar båda.

Du möter ytterligare en generic i kapitel 7: `timers TICK_COUNT`, som är ett antal snarare än en bredd, och som är det som låter samma fil mäta en femtiondels sekund i en design och en hel sekund i en annan.

4.9 Genomarbetat exempel: led_toggle_sync

Det här knyter ihop avsnitt 4.1–4.6 till kapitel 3:s växlingskrets, gjord säker för en riktig, asynkron knapp och reset.

Port	Riktn.	Beskrivning
clock	in	50 MHz systemklocka på DE0-CV-kortet.
reset_n	in	Aktiv låg asynkron reset, kopplad till en tryckknapp.
button_n	in	Aktiv låg tryckknapp; växlar lysdioden på sin fallande flank.
led	out	Lysdiod, växlas vid varje synkroniserat, flankdetekterat knapptryck.

Interna signaler: `led_s` håller lysdiodens tillstånd, `reset_s2_n` är reseten efter tvåvipp-synkroniseraren (suffixet `s2` är bokens konvention för "synkroniserad med två vippor"), och `button_edge_s2` är den encykelspuls som markerar en synkroniserad fallande flank på `button_n`.

Liksom kapitel 2:s tvåsiffriga display är den här designen byggd av mer än en entitet, med instansieringen från avsnitt 2.6 och genericen från avsnitt 4.8. Skillnaden är vad delarna gör: displayen instansierade en modul två gånger för att göra samma jobb på två halvor av sin ingång, medan den här instansierar två *olika* moduler med ett jobb var. Båda är samma mekanism. Att komponera en design av moduler är inte en särskild teknik för upprepad logik; det är så designer byggs när de vuxit ur en enda fil.

De två kedjorna bor i två subkomponenter:

- `reset_sync` är avsnitt 4.5:s tvåvippssynkroniserare. Den tar den råa `reset_n` och returnerar `reset_s2_n`.
- `button_sync` är avsnitt 4.6:s trevippssynkroniserare med flankdetektor. Den tar den redan synkroniserade `reset_s2_n` och pulsar `button_edge_s2` en gång per tryck.

Att hålla dem åtskilda betyder mer än det ser ut. En design utan egen knapp behöver ändå sin reset synkroniserad, och med två moduler instansierar den helt enkelt den den vill ha, som kapitel 8:s `seq_detect_mearly` gör. Hopsvetsade till en modul skulle den designen behöva instansiera alltihop och koppla bort den halva den inte använder.

`button_sync` tar en *vektor* av knappar så att en fil kan betjäna en design med två knappar. Den här designen har en, så toppnivån deklarerar ett par `enelementsvektorer` att koppla till den; det är rördragning, inte logik, två ledningar omdöpta. Fullständig källkod:

```

lectures/L04/led_toggle_sync/led_toggle_sync.vhd
-----
-- Toggle an LED on each synchronized, edge-detected press of button_n.
-- The LED is cleared on reset.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset.
--   - button_n: Active-low button.
-- Outputs:
--   - led: Toggled on the falling edge of button_n.
-----

library ieee;
use ieee.std_logic_1164.all;

entity led_toggle_sync is
    port(clock, reset_n, button_n: in std_logic;
         led : out std_logic);
end entity;

architecture behaviour of led_toggle_sync is
    -- Stable reset signal synchronized using a two-flip-flop synchronizer.
    signal reset_s2_n: std_logic;

    -- Pulse indicating a falling edge on button_n, synchronized using a two-flip-flop-
    --   synchronizer.
    signal button_edge_s2: std_logic;

    -- One-element vectors adapting the scalar button signals above to button_sync,
    -- whose button ports are COUNT-bit vectors.
    signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);

    -- Internal LED state.
    signal led_s: std_logic;

begin
    led <= led_s;

    -- Adapt the scalar button signals to button_sync's vector ports.
    button_n_v(0)    <= button_n;
    button_edge_s2 <= button_edge_s2_v(0);

    -- Synchronize the asynchronous reset.
    reset_sync1: entity work.reset_sync

```



```

    port map(clock, reset_n, reset_s2_n);

-- Synchronize button_n and pulse on each press (a single button, so COUNT = 1).
button_sync1: entity work.button_sync
    generic map(1)
    port map(clock, reset_s2_n, button_n_v, button_edge_s2_v);

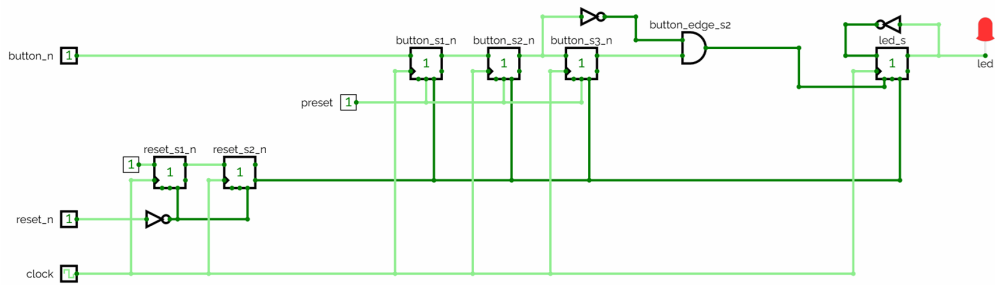
-- Toggle the LED on button press event.
LED_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        led_s <= '0';
    elsif (rising_edge(clock)) then
        if (button_edge_s2 = '1') then
            led_s <= not led_s;
        end if;
    end if;
end process;
end architecture;

```

Toppnivåarkitekturen innehåller ingenting annat än själva växlingsprocessen, och den rör aldrig annat än de redan synkroniserade `reset_s2_n` och `button_edge_s2`, aldrig de råa ingångarna.

De två subkomponenterna finns inte i kursrepot, eftersom att skriva dem är Övning 4.8 och 4.9: `reset_sync` är avsnitt 4.5 ovan, och `button_sync` är avsnitt 4.6 och avsnitt 4.8, båda utskrivna i sin helhet. Kopiera in dina egna i `led_toggle_sync/` för att bygga den.

För hand, i CircuitVerse först. Innan du rör VHDL, bygg den som ett grindnät: två D-vippor i serie för resetsynkroniseraren; två för `button_n` plus en tredje som lagrar det föregående synkroniserade värdet; en AND-grind som kombinerar det inverterade nuvarande värdet med det föregående för att producera `button_edge_s2`; och en växlingsvipa för `led_s`, med sin inverterade utgång återkopplad till sin ingång, aktiverad endast när `button_edge_s2 = 1`. Sätt klockperioden till 1000 ms så att du hinner följa varje steg med ögat.



Figur 4.3 Synkroniserad lysdiodsväxlingskrets: dubbelvippsynkroniserare för reset och knapp, flankdetektering, och en växlingsvippa för lysdioden.

En sak i ritningen behöver läsas noga: vipporna har en preset-ingång, och på knappsynkroniserarens tre steg är den bunden till 1. Det är ingen dekoration. I CircuitVerse är preset det värde en vippa laddar när dess reset aktiveras, så att binda den till 1 är det som gör att button_s1_n, button_s2_n och button_s3_n kommer ur reset med värdet 1 snarare än 0.

Det stämmer med button_sync, som resettar samma tre steg till (others => '1'), och det spelar roll av samma skäl: knapparna är aktivt låga, så 1 betyder "släppt". Resetta kedjan till 0 i stället och den startar med påståendet att knappen redan hålls nedtryckt. Resetvipporna lämnar preset i fred, eftersom en reset till 0 där verkligen är allt kretsen ber om.

Tryck på knappen i simuleringen och följ hur det fortplantar sig. Lysdioden växlar inte omedelbart: den väntar på att den synkroniserade, flankdetekterade pulsen ska nå växlingsvippan, vilket tar ett litet, fast antal klockpulser, precis den latens avsnitt 4.3 kallade preset för säkerheten.

På FPGA:n. Samma design demonstreras på Terasic DE0-CV (enheten 5CEBA4F23C7N) under passet, med clock på 50 MHz-oscillatorn, reset_n och button_n på två tryckknappar, och led på en lysdiod. Lysdioden växlar vid varje tryck, och enligt avsnitt 4.6 kan ett riktigt tryck ibland ge mer än en växling på grund av kontaktstuds.

4.10 Sammanfattning

Det här ska du kunna nu

- Förklara vad metastabilitet är, varför en asynkron ingång kan orsaka det, och varför två vippor gör det osannolikt snarare än omöjligt.
- Tillämpa dubbelvippsynkroniseraren på vilken asynkron ingång som helst, inklusive mönstret "aktivera asynkront, släpp synkront" som används för reset.
- Säg varför den kedjan dessutom fungerar som en praktisk men ofullständig studsfiltrering, och vad en riktig sådan skulle tillföra.
- Ge en modul en generic, åsidosätta den med en generic map, skriva en ports bredd i termer av den, och bedöma när en generic gör rätt för sig i stället för att läggas till av princip.

Frågor att testa dig själv med

1. Vad händer exakt när en asynkron ingång till en vippa ändras för nära en aktiv klockflank?
2. Hur hanterar dubbelvippsynkroniseraren det, och varför är det den *andra* vippans utgång som är säker att använda?
3. Varför beter sig kedjan som används här som en studsfiltrering i CircuitVerse men studsfiltrerar inte en riktig knapp fullt ut vid 50 MHz, och varför är det en egenskap hos klockperioden snarare än hos kretsen? Vad skulle du lägga till för att åtgärda det?
4. `button_sync` har en generic och `reset_sync` har ingen. Vad är regeln, och vad vore fel med att ge `reset_sync` en WIDTH-generic "för symmetrins skull"?

Vidare läsning

Erik Pihls videogenomgång av metastabilitet och dubbelvippsynkroniseraren finns på <https://www.youtube.com/watch?v=KrssJRgF13I> som ytterligare, frivillig visning. <https://nandland.com/lesson-13-metastability/> och <https://vhdlwhiz.com/terminology/metastability/> täcker samma begrepp från en något annan vinkel, användbart om det här kapitlet inte sitter vid första läsningen.

Blicken framåt

Kapitel 5 går vidare med:

- `variable`: den enda konstruktion du ännu inte mött, och den skarpa skillnaden mellan hur den uppdateras och hur en signal gör det.
- Varför samma ackumulering skriven med en signal och med en variable ger två olika svar, och vilket av de två som är felet.
- Vad verktygskedjan faktiskt bygger: LUT:ar och vippor, vad som begränsar en klockas hastighet, och låset du råkar beskriva av misstag.

4.11 Övningar

Så kontrollerar du ditt arbete. Varje övning nedan som ber dig skriva en VHDL-modul har en utdelad självkontrollerande testbänk under `lectures/L04/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7 för de tre kommandona.

ÖVNINGSGRUPP 1

Metastabilitetsbegrepp

Övning 4.1 Metastabilitet med egna ord

RESONEMANG

Förklara metastabilitet med egna ord. Beskriv i din förklaring:

- Vad som händer inuti en D-vippa när dess ingång ändras för nära den aktiva klockflanken.
- Varför vippans utgång tillfälligt kan bli kvar mellan en giltig logisk 0 och logisk 1.
- Varför utgången kan ta en oförutsägbar tid på sig att stabilisera sig.
- Varför två grindar nedströms kan tolka den ostabiliserade utgången olika: den ena som 0, den andra som 1.

Förklara varför den oenigheten kan få olika delar av en digital krets att hamna i inkonsistenta tillstånd.

Övning 4.2 Setup, hold, och varför en asynkron ingång inte kan undvikas

RESONEMANG

Setup-tid och hold-tid definierar ett fönster kring den aktiva klockflanken under vilket en vippas ingång måste förbli stabil.

Förklara:

- Varför en synkront genererad signal normalt kan konstrueras så att den uppfyller de här tidskraven.
- Varför en asynkron ingång, till exempel en mekanisk tryckknapp, inte har någon fast relation till systemklockan.
- Varför en asynkron ingång därför kan ändras under setup- och hold-fönstret oavsett hur den omgivande synkrona logiken är konstruerad.

Dra slutsatsen varför metastabilitetsrisken från en asynkron ingång måste hanteras snarare än helt enkelt undvikas.

ÖVNINGSGRUPP 2

Att konstruera en synkroniserare

Övning 4.3 Kedjan, ritad och förklarad

KRETS

Rita en dubbelvippsynkroniserare för en asynkron ingång. Kretsen ska innehålla en asynkron ingång, två D-vippor kopplade i serie, en gemensam systemklocka, och en synkroniserad utgång tagen från den andra vippan.

Förklara med egna ord:

- Varför den första vippan är direkt exponerad för den asynkrona ingången.
- Varför den första vippan kan bli metastabil.
- Hur intervallet mellan den första och den andra klockflanken ger det första steget tid att stabilisera sig.
- Varför den andra vippans utgång har en mycket lägre sannolikhet att vara metastabil.
- Varför det är det andra steget, och inte det första, som resten av designen ska använda.

En synkroniserare gör inte sannolikheten för metastabilitet exakt noll. Förklara varför den ändå sänker risken till en acceptabelt låg nivå i de flesta designen.

Tips: Fundera på hur mycket stabiliseringstid den första vippan får innan dess värde samplas av den andra vippan.

Övning 4.4 Ett tredje steg "för säkerhets skull"

RESONEMANG

En kollega föreslår att använda tre synkroniserarvippor på varje asynkron ingång i en långsam design som klockas i några megahertz, bara för säkerhets skull.

Ett tredje steg köper ytterligare en klockperiod för ett metastabilt värde att stabilisera sig på, och kostar ytterligare en klockcykels latens på varje synkroniserad ingång. Vid några megahertz är den extra klockperioden en enorm tid räknat i metastabilitetstermer, vilket är precis därför förslaget låter förnuftigt, och också därför det redan kan vara onödigt.

a) Resonera igenom avvägningen åt båda hållen. Under vilka omständigheter faller den ut till det tredje stegets fördel, och under vilka omständigheter räcker det andra steget redan? Ditt svar måste hänga på något annat än klockfrekvensen, eftersom frekvensen ensam inte avgör saken: säg vad det hänger på i stället.

b) Avsnitt 4.4 kom fram till MTBF utifrån den stabiliseringstid som finns per steg. Utan att göra om den räkningen, säg åt vilket håll var och en av de här faktorerna drar beslutet, och varför:

- Hur ofta den asynkrona ingången faktiskt ändras.
- Metastabilitetsegenskaperna hos den FPGA-familj du riktar in dig på.
- Vad ett synkroniseringsfel skulle kosta om det inträffade.

c) Ge din rekommendation för en vanlig långsam design av det slag som förekommer i den här boken. Beskriv sedan en design, verklig eller påhittad, där du i stället skulle argumentera för det tredje steget, och säg vilken av faktorerna i b) som gör jobbet i det fallet.

Kontrollera att ditt svar täcker de två saker ett tredje steg ger, ytterligare en klockperiods stabiliseringstid och en lägre sannolikhet att ett ostabiliserat värde når den funktionella logiken, och de två det kostar, ytterligare en cykels latens och ytterligare vippor. Om ditt svar på a) vilar på klockfrekvensen snarare än på den erforderliga medeltiden mellan fel, läs avsnitt 4.4 en gång till innan du går vidare.

Övning 4.5 Aktivera asynkront, släpp synkront

RESONEMANG

Förklara resetmönstret **aktivera asynkront, släpp synkront** från avsnitt 4.5.

Beskriv de två delarna var för sig:

- Asynkron aktivering: resetten får verkan omedelbart, och väntar inte på en klockflank.
- Synkront släpp: resetten tas bort bara vid en aktiv klockflank, och all berörd logik lämnar reset i en kontrollerad relation till klockan.

Förklara:

- Varför omedelbar aktivering av reset kan vara viktig när klockan är stoppad, när klockan är instabil, och när systemet måste gå in i sitt resettillstånd utan fördröjning.
- Varför ett släpp av en asynkron reset nära en klockflank kan bryta mot en vippas tidskrav för recovery eller removal.
- Varför synkronisering av släppet hindrar olika vippor från att lämna reset under olika klockcykler.

ÖVNINGSGRUPP 3

Att bygga kretsen

Övning 4.6 led_toggle_sync för hand

KRETS

Bygg kretsen `led_toggle_sync` från avsnitt 4.9 för hand i CircuitVerse.

a) Lägg till de yttre portarna: ingångarna `clock`, `reset_n` (asynkron, aktiv låg) och `button_n` (aktiv låg), samt utgången `led`.

b) Bygg tvåvippresetsynkroniseraren:

- Koppla de två vipporna i serie.
- Aktivera båda stegen asynkront med den råa `reset_n`-ingången.
- Låt resetten fortläpa sig ut ur kedjan synkront.

- Använd den synkroniserade resetutgången, `reset_s2_n`, i hela resten av kretsen, och använd inte den råa `reset_n`-ingången direkt utanför resetsynkroniseraren.
- c) Bygg trevippsknappsynkroniseraren och flankdetektorn:
- Använd de två första vipporna som dubbelvippsynkroniserare.
 - Använd den tredje vippa för att lagra det föregående synkroniserade knappvärdet.
 - Jämför det nuvarande och det föregående synkroniserade värdet, och detektera ett knapptryck när det nuvarande synkroniserade, aktivt låga knappvärdet är 0 och det föregående synkroniserade värdet är 1.
 - Använd en AND-grind med det nuvarande värdet inverterat för att generera `button_edge_s2`. Sambandet är $button_edge_s2 = previous \cdot current'$.
- d) Bygg växlingslogiken för lysdioden: lägg till en vippa som lagrar `led_s`, resetta den med `reset_s2_n`, växla den bara när `button_edge_s2 = 1`, och koppla utgången `led` till `led_s`.
- e) Simulera hela designen med klockperioden 1000 ms. Aktivera och släpp `reset_n`, tryck ned och släpp `button_n`, och observera de synkroniserade knappstegen, `button_edge_s2` och `led_s`. Avgör:
- Om lysdioden växlar vid den första klockflanken efter det fysiska knapptrycket.
 - Hur många klockcykler som krävs för att knappvärdet ska fortplanta sig genom synkroniseraren och flankdetektorn.
 - Om den observerade latensen stämmer med det beteende det här kapitlet förutsäger.

ÖVNINGSGRUPP 4

Studsfiltrering och tajming

Övning 4.7 Vad kedjan gör och inte gör

RESONEMANG

Du har nu byggt och betraktat kretsen, så det här är en fråga om vad du såg.

Trevippskedjan i Övning 4.6 gör tre jobb på en gång: den minskar risken för metastabilitet, den lagrar det föregående synkroniserade knappvärdet, och den detekterar ett knapptryck. Förklara varför den **inte** i sig garanterar att knappen är studsfiltrerad.

- a) Skilj på de tre saker som är lätta att blanda ihop:
- Synkronisering: gör om en asynkron ingång till en signal som går att använda i klockdomänen.
 - Flankdetektering: genererar en puls när den samplade signalen ändras.
 - Studsfiltrering: hindrar flera fysiska övergångar från att tolkas som flera separata tryck.

Vilka två av dessa gör din krets faktiskt?

b) I din simulering gav ett tryck en enda ren växling. Det är ett starkare påstående än det ser ut: det håller bara under ett särskilt villkor om var studsens hamnade i förhållande till klockflankerna. Formulera det villkoret exakt, i termer av vad din krets samplade och vad den därmed såg. Förklara sedan varför det villkoret är en tillfällighet som beror på din klockperiod snarare än en egenskap hos din krets. Var konkret med vilken av de två, studsens eller samplingen, din design faktiskt styr.

c) Din simulering kördes med klockperioden 1000 ms; FPGA:n går i 50 MHz. En tryckknapps kontakter studsar typiskt i storleksordningen 1 ms. Räkna ut konsekvenserna själv:

- Hur många gånger samplar en klocka med perioden 1000 ms knappen under den millisekunden av studs?
- Hur många gånger samplar en klocka på 50 MHz den?
- Varje samplad övergång ser ut som en ny flank för din flankdetektor. Förutsäg utifrån de två talen vad lysdioden gör på kortet vid ett enda fysiskt tryck, och säg varför exakt samma krets uppförde sig oklanderligt i CircuitVerse.

d) Beskriv en åtgärd av vardera slaget: en hårdvaruåtgärd (till exempel ett RC-lågpasfilter följt av en Schmitt-triggeringång) och en rent digital åtgärd (till exempel att kräva att den synkroniserade signalen håller en ny nivå under ett visst minsta antal klockcykler innan den accepteras, eller en räknare som ignorerar ytterligare övergångar under ett fast intervall efter ett tryck — vilket är vad kapitel 7:s timer skulle ge dig).

e) Förklara varför dubbelvippsynkroniseraren måste behållas även när en studsfiltreringsmetod lagts till. Vilket problem löser inte studsfiltreringen?

ÖVNINGSGRUPP 5

Synkroniserarna som egna moduler

Det genomarbetade exemplet lägger varje synkroniserare i en egen modul, så att varje senare design instansierar samma två filer i stället för att härleda dem på nytt. Det är därför både kapitel 7:s `walking_led` och kapitel 8:s `fsm_led` ber dig kopiera in de här två, och därför kapitel 8:s `seq_detect_mealy`, som inte har någon knapp, tar resetsynkroniseraren ensam och lämnar den andra därhän.

De två följande övningarna är du som skriver de modulerna. Avsnitt 4.5 och 4.6 ger dig logiken; vad de inte ger dig är entiteten runt den, och, för den andra, steget från en enda knapp till en vektor av dem. Skriv varje modul själv innan du öppnar det genomarbetade exemplets kopia, och jämför sedan. Övning 4.10 sätter därefter ihop båda till en fungerande design.

Övning 4.8 reset_sync

VHDL

Skriv resetsynkroniseraren som en modul med namnet reset_sync.

Entiteten har:

Port	Riktn.	Typ	Beskrivning
clock	in	std_logic	50 MHz systemklocka.
reset_n	in	std_logic	Aktiv låg asynkron reset, rakt från omvärlden.
reset_s2_n	out	std_logic	Aktiv låg synkroniserad reset, säker för resten av designen att använda. Går låg i samma ögonblick som reset_n gör det, utan att någon klockflank är inblandad, och återgår till hög två stigande flanker efter att reset_n gjort det.

a) Implementera **aktivera asynkront, släpp synkront** (avsnitt 4.5):

- En intern signal för det första steget, plus utgången för det andra.
- En process, känslig för både clock och reset_n.
- Vid reset_n = '0', driv båda stegen låga, utanför den klockade grenen.
- Vid en stigande klockflank, skifta in en konstant '1' genom de två stegen.

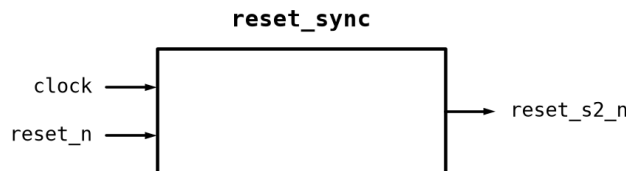
b) Värdet som tvingas fram vid reset och värdet som skiftas in på klockan är varandras motsatser, '0' och '1'. Förklara i en mening varför det inte är någon motsägelse, utifrån vad respektive gren är till för.

c) Verifiera designen med dess testbänk. Den kontrollerar mönstrets båda halvor var för sig: att en låg reset_n drar reset_s2_n låg utan någon klockflank däremellan, och att ett släpp av reset_n lämnar reset_s2_n låg tills två stigande flanker har passerat.

d) Svara i löpande text:

- Utgången heter reset_s2_n snarare än reset_n_sync. Vad säger s2, och vad skulle behöva ändras i den här modulen för att det namnet skulle bli fel?
- Varje annan modul i den här boken tar reset_s2_n som sin reset. Den här tar den råa reset_n. Varför måste exakt en modul i en design vara undantaget?

Tips: Släppet är den intressanta halvan. Vid reset bryr sig processen inte om klockan alls, så båda stegen går låga tillsammans; på klockan bryr den sig inte om reset_n, så '1':an behöver en flank per steg för att nå utgången. Två flankers latens vid släpp är priset för hela mönstret.



Självkontroll. Döp din entitet till `reset_sync`, med portarna `clock`, `reset_n` (in) och `reset_s2_n` (out), deklarerade i den ordningen; dess testbänk finns i `exercises/reset_sync/`.

Övning 4.9 `button_sync`

VHDL

Skriv knappsynkroniseraren och flankdetektorn som en modul med namnet `button_sync`.

Övning 4.10 hanterar två knappar genom att skriva en tvåbitars synkroniserare. Den här är samma krets med bredden lämnad öppen: en generic avgör hur många knappar den betjänar, så att `led_toggle_sync` kan använda den för en knapp och `fsm_led` för två utan att någondera filen behöver ändras. Se avsnitt 4.8 för syntaxen och för varför den här modulen har en generic där `reset_sync` inte har någon.

Entiteten har en generic:

Generic	Typ	Förval	Beskrivning
COUNT	natural range 1 to 3	1	Antalet knappar, och därmed bredden på båda vektorportarna.

och de här portarna:

Port	Riktn.	Typ	Beskrivning
clock	in	std_logic	50 MHz systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, redan synkroniserad reset. Den här modulen synkroniserar inte sin egen reset; det gjorde modulen i Övning 4.8.
button_n	in	std_logic_vector(COUNT - 1 downto 0)	Aktivt låga asynkrona tryckknappar.
button_edge_s2	out	std_logic_vector(COUNT - 1 downto 0)	En encykelspuls vid varje knapps fallande flank, två klockflanker efter trycket. Varje bit är oberoende av alla andra.

a) Bygg trestegskedjan från avsnitt 4.6, nu över vektorer i stället för enskilda bitar:

- Tre interna `std_logic_vector(COUNT - 1 downto 0)`-signaler, en per steg.
- Steg 1 och 2 är dubbelvippsynkroniseraren; steg 3 lagrar det föregående värdet av steg 2.
- Vid reset, sätt alla tre stegen till (`others => '1'`), inte (`others => '0'`).
- Härled utgången med en enda konkurrent tilldelning, och lägg märke till att samma uttryck fungerar oförändrat på vektorer som på bitar.

Håll den här i minnet. De två första vipporna är en synkroniserare i sin helhet och ingenting annat, och Övning 5.4 låter dig bryta ut dem igen som en egen modul, generisk både i bredd och i det värde de resettar till. Att skriva dem inline här först är vad som gör den övningen till en befästning i stället för en ny idé.

b) Förklara varför resetvärdet är enbart ettor. Enbart ettor betyder "släppt", eftersom knapparna är aktivt låga, så en reset till enbart nollor startar kedjan med påståendet att varje knapp redan hålls nedtryckt. Följ vad det kostar:

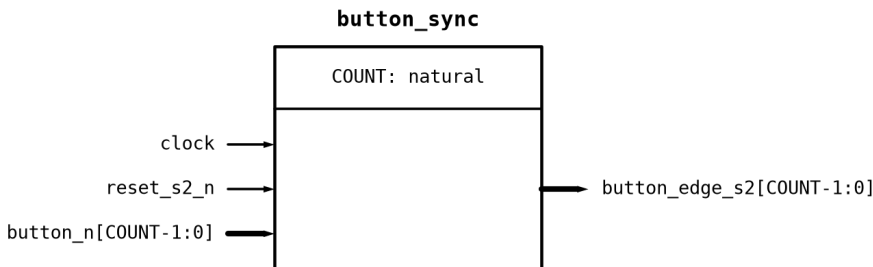
- Med knapparna släppta och en reset till enbart nollor, ger modulen ifrån sig en falsk puls när det släppta tillståndet fortplantar sig genom de tre stegen? Gå igenom utgångsuttrycket flank för flank innan du svarar, i stället för att gissa.
- Nu fallet som faktiskt går sönder. En användare håller en knapp nedtryckt i det ögonblick resetten släpps. Vad rapporterar modulen med en reset till enbart ettor, och vad rapporterar den med enbart nollor? Vilket av de två är det beteende du vill ha, och varför?

c) Verifiera designen med dess testbänk. Den instansierar din modul **två gånger**, en gång med `COUNT = 1` och en gång med `COUNT = 2`, och kontrollerar att ett tryck ger exakt en encykelspuls, att en nedhållen knapp inte ger några ytterligare pulser, att ett släpp inte ger någon alls, och att ett tryck på den ena knappen i ett par aldrig pulsar den andra.

d) Svara i löpande text:

- `COUNT` är begränsad till 1 to 3 snarare än lämnad som en obegränsad `natural`. Vad köper den begränsningen, givet att ingenting i arkitekturen beror på den övre gränsen?
- Den här modulen är medvetet inte hopslagen med Övning 4.8:s, trots att en design som har en knapp alltid behöver båda. Ange den design som motiverar uppdelningen, och säg vad den hade behövt göra i stället.

Tips: Skriv arkitekturen för `COUNT = 1` i huvudet först, och kontrollera sedan att varje rad du skrev redan är korrekt för en vektor. `and`, `not` och aggregatet (`others => '1'`) fungerar alla elementvis, så generaliseringen ska inte kosta dig någon extra kod alls. Gör den det är det värt en extra titt.



Självkontroll. Döp din entitet till `button_sync`, med genericen `COUNT` (`natural` range 1 to 3, standardvärde 1), ingångarna `clock`, `reset_s2_n`, `button_n` (`std_logic_vector(COUNT - 1 downto 0)`) och utgången `button_edge_s2` (`std_logic_vector(COUNT - 1 downto 0)`), deklarerade i den ordningen; dess testbänk finns i `exercises/button_sync/`.

ÖVNINGSGRUPP 6

Att sätta ihop dem till en design

Övning 4.10 led_toggle_sync2

VHDL

Gör nu kapitel 3:s led_toggle metastabilitetssäker, i VHDL, som en ny modul med namnet led_toggle_sync2.

kapitel 3:s led_toggle växlade två lysdioder från två tryckknappar, men matade in deras råa, asynkrona signaler rakt i flankdetektorn — logiskt korrekt, men osäkert på riktig hårdvara. Här bygger du om den med varje asynkron ingång skyddad av det här kapitlets dubbelvippsynkroniserare.

Entiteten har:

Port	Riktn.	Typ	Beskrivning
clock	in	std_logic	50 MHz systemklocka.
reset_n	in	std_logic	Aktiv låg asynkron reset.
button_n	in	std_logic_vector(1 downto 0)	Två aktivt låga asynkrona tryckknappar.
led	out	std_logic_vector(1 downto 0)	Två lysdioder; led(i) växlar en gång per tryck på button_n(i).

a) Synkronisera varje asynkron ingång:

- Tillämpa tvåvippsresetsynkroniseraren från avsnitt 4.5.
- Ge knapparna en dubbelvippsynkroniserare plus ett tredje steg för flankdetektering, som i avsnitt 4.6 — nu över en std_logic_vector(1 downto 0), så att båda knapparna synkroniseras parallellt.
- Låt aldrig den råa reset_n eller button_n nå växlingslogiken; använd bara den synkroniserade resetten och flankpulserna per knapp.

Tips: Du kan lägga synkroniseraren inline, eller bryta ut den till en subkomponent som tar en tvåbitars button_n, med instansieringssyntaxen från avsnitt 2.6. Att lägga den inline är den kortare vägen, och då är bygget en enda fil:

```
ghdl -a --std=93 led_toggle_sync2.vhd led_toggle_sync2_tb.vhd
```

Om du i stället bryter ut den, kopiera in de reset_sync.vhd och button_sync.vhd du skrev till Övning 4.8 och 4.9 (och instansiera button_sync med COUNT = 2), och namnge dem först på raden ghdl -a, före din egen modul, så att GHDL ser dem före den fil som instansierar dem:

```
ghdl -a --std=93 reset_sync.vhd button_sync.vhd \
    led_toggle_sync2.vhd led_toggle_sync2_tb.vhd
```

Se avsnitt 2.7.5. Ingen av modulerna delas ut någonstans i kursrepot, eftersom att skriva dem är Övning 4.8 och 4.9.

b) Bygg växlingslogiken:

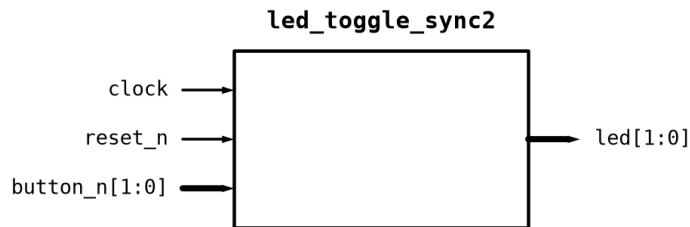
- En vippa för lysdiodstillstånd per lysdiod.
- Vid den synkroniserade resetten, släck båda lysdioderna.
- Växla `led(i)` bara när knapp `i`:s synkroniserade puls för fallande flank utlöses.
- Bekräfta att `led(i)` svarar på enbart `button_n(i)`, aldrig på den andra knappen.

c) Verifiera designen med dess testbänk. Testbänken trycker på varje knapp i tur och ordning och kontrollerar att reset släcker båda lysdioderna, att inget tryck betyder ingen växling, att ett tryck på knapp 0 växlar **bara** `led(0)` när synkroniserarlatsen har förflutit, att hålla den knappen nedtryckt inte växlar den igen, och att ett tryck på knapp 1 sedan växlar bara `led(1)` och lämnar `led(0)` som den var.

Var uppmärksam på **latensen**: testbänken kontrollerar den *exakta* flanken, inte bara att lysdioden till slut lägger sig rätt. Ett tryck måste nå växlingslogiken genom två synkroniserarvippor, så `led` är fortfarande oförändrad efter den första och den andra stigande flanken och växlar vid den tredje. Räkna vipporna ett tryck färdas genom och övertyga dig om att den tredje flanken är den rätta, och att ett defensivt extra pipelinesteg nu skulle fångas snarare än tolereras.

d) Svara i löpande text, genom att jämföra den här designen med din `led_toggle` från kapitel 3:

- Vad ändrar synkroniseraren när det gäller att trycka på en riktig, studsande knapp?
- Varför var den råa versionen osäker att koppla till en fysisk knapp vid 50 MHz?
- Din testbänk driver rena, perfekt tajmade övergångar. Vilket av det här kapitlets två problem, metastabilitet eller kontaktstuds, kan en testbänk därför aldrig demonstrera för dig?



Självkontroll. Döp din entitet till `led_toggle_sync2`, med ingångarna `clock`, `reset_n`, `button_n` (`std_logic_vector(1 downto 0)`) och utgången `led` (`std_logic_vector(1 downto 0)`), deklarerade i den ordningen; dess testbänk finns i `exercises/led_toggle_sync2/`. Den trycker på varje knapp i tur och ordning och kontrollerar att `led(i)` växlar på `button_n(i)`:s fallande flank,

att den gör det vid rätt klockflank snarare än bara lägger sig på rätt värde till slut, och att den aldrig utlöser fel lysdiod.

Variabler och hårdvaran under

Den sista konstruktionen som återstår, och vad verktygskedjan gör av alltihop.

I DET HÄR KAPITLET

- **variable:** processlokal lagring, som uppdateras direkt där en signal schemalägger.
- Varför samma ackumulering ger två olika svar beroende på vilken du griper efter.
- Vad en FPGA fysiskt består av: uppslagstabeller och vippor, och ingenting som är en grind.
- Kritiska vägen och Fmax, och vad som sätter en hastighetsgräns på en klocka.
- Låset du råkar beskriva av misstag, och varför det bara är en varning.

Det här kapitlets poäng ligger i dess första genomarbetade exempel: kod som ser korrekt ut, simulerar utan invändningar, och är fel. Enda sättet att se varför är att *spåra* den snarare än att läsa den, en loopiteration i taget, fram till fel svar. Gör det på papper innan du läser rättelsen.

Repetition: Fortplantningsfördröjningen bakom klockans hastighetsgräns förklaras på transistornivå i avsnitt 1.12 i *Digital Electronics*, och D-låset du kan få av misstag står i avsnitt 5.3 i samma bok.

5.1 Vad som återstår

Varje konstruktion du behöver för att beskriva hårdvara har du redan använt: `entity` och `architecture` i kapitel 1, `std_logic_vector`, `process` och `case` i kapitel 2, den klockade processen och schemaläggningsregeln i kapitel 3, subkomponenter och generics i kapitel 2 och 4.

Två saker återstår, och de är de två svåraste att lista ut på egen hand:

- Den första är **variable**, den enda konstruktion boken inte har behövt förrän nu. Den ser ut som en signal och beter sig inte alls som en, och skillnaden är precis den sort som ger en design som simulerar trovärdigt och är tyst felaktig. Avsnitt 5.2 går igenom fallet där det biter.
- Den andra är **vad** något av det här faktiskt blir. Sedan kapitel 1 har du litat på att syntes-verktyget bygger den krets du beskrev. Det gör det, men inte av något du skulle känna

igen. Avsnitt 5.3 öppnar lådan. Det visar sig att en FPGA inte innehåller några grindar alls, vilket är skälet till att kapitel 2:s Karnaughdiagram köpte dig mindre än de såg ut att göra. Din klocka har en hastighetsgräns (avsnitt 5.4), och något specifikt sätter den. Och ett case med en gren som saknas ger dig tyst ett minneselement som ingen bad om (avsnitt 5.5).

Inget av avsnitten svarar på ”är min logik rätt?” Den frågan har du besvarat sedan kapitel 1. De svarar på de två frågorna efter den: är det här vad jag menade, och kommer det att få plats och gå.

5.2 variable: processlokal lagring som uppdateras direkt

En *variable* deklareras inuti en process, mellan `process(...)` och `begin`. Den avgörande skillnaden mot en *signal*: variabeltilldelning använder `:=` och får verkan **omedelbart**, så redan nästa sats som läser den ser det uppdaterade värdet, precis som en tilldelning i C. Det finns ingen schemaläggning, ingenting av den vänta-tills-processen-suspenderar-fördröjning som avsnitt 3.6 stavade ut för signaler, och inget ”sista tilldelningen vinner”.

	signal (<=)	variable (:=)
Deklareras	I arkitekturens deklarativa del	I processens deklarativa del
Synlig från	Var som helst i arkitekturen	Endast inuti sin egen process
Uppdateringstillfälle	Schemalagd: tillämpas när processen nästa gång suspenderar	Omedelbar: tillämpas före nästa sats
Upprepad tilldelning i ett varv	Bara den sista överlever; tidigare kastas	Var och en får verkan, i tur och ordning

Varför det här faktiskt spelar roll: en XOR-paritetsgenerator

En entitet som beräknar XOR-pariteten hos en 8-bitars ingång, '1' om bits har ett udda antal ettställda bitar:

```
entity parity_gen is
  port(bits : in std_logic_vector(7 downto 0);
        parity: out std_logic);
end entity;
```

Med bakgrund i mjukvara är den första ingivelsen att ackumulera den löpande XOR-summan i en loop, med en signal som ackumulator:


```
-- Don't do this: shown to demonstrate why it doesn't work.
architecture broken of parity_gen is
  signal parity_s: std_logic;
begin
  parity <= parity_s;

  process(bits) is
  begin
    parity_s <= '0';
    for i in bits'range loop
      parity_s <= parity_s xor bits(i);
    end loop;
  end process;
end architecture;
```

Spåra ett varv, med schemalägningsregeln från avsnitt 3.6: varje läsning av `parity_s` returnerar det den höll *innan det här varvet började* (kalla det `old_parity`), och bara den sista tilldelningen överlever.

- `parity_s <= '0'`; schemalägger '0'. Ännu inte tillämpad.
- `i = 7` (bits'range på (7 downto 0) går från hög till låg): läser `parity_s`, får `old_parity`, eftersom '0':an inte har tillämpats. Schemalägger `old_parity xor bits(7)`, och kastar '0':an.
- `i = 6`: läser `parity_s`, återigen `old_parity`. Schemalägger `old_parity xor bits(6)`, och kastar den föregående.
- ...och så vidare för varje iteration ...
- `i = 0`, den sista satsen i varvet: läser fortfarande `old_parity`, schemalägger `old_parity xor bits(0)`, och kastar allt före den.
- Processen suspenderar och `parity_s` blir `old_parity xor bits(0)`, den enda av nio schemalagda tilldelningar som överlevde.

Resultatet beror bara på `bits(0)` och på vad `parity_s` råkade hålla sedan tidigare. Sju iterationer och den inledande '0':an beräknades och slängdes. Det här är inget randfall: det är vad en signal-som-ackumulator *alltid* gör, eftersom varje iteration läser samma frusna värde och bara den textuellt sista tilldelningen behålls.

Samma beräkning med en `variable`, och det här är det genomarbetade exemplet på disk:

```
lectures/L05/parity_gen/parity_gen.vhd

-----
-- 8-bit XOR-parity generator.
--
-- Inputs:
--   - bits: The eight bits to compute parity over.
```

```

-- Outputs:
--      - parity: '1' if 'bits' holds an odd number of set bits, '0' otherwise.
-----
library ieee;
use ieee.std_logic_1164.all;

entity parity_gen is
    port(bits : in std_logic_vector(7 downto 0);
          parity: out std_logic);
end entity;

architecture behaviour of parity_gen is
    -- Internal representation of 'parity'.
    signal parity_s: std_logic;
begin
    parity <= parity_s;

    PARITY_PROCESS: process(bits) is
        variable acc: std_logic;
    begin
        acc := '0';
        for i in bits'range loop
            acc := acc xor bits(i);
        end loop;
        parity_s <= acc;
    end process;
end architecture;

```

Nu tillämpas `acc := '0'` omedelbart; `i = 7` läser `'0'`, XOR:ar in `bits(7)` och uppdaterar `acc` på en gång; `i = 6` läser det just uppdaterade värdet, och så vidare genom varje iteration. Loopen slutar med att `acc` håller den sanna XOR:en av alla åtta bitarna, och `parity_s <= acc`; schemalägger det enda korrekta värdet, den enda tilldelningen till `parity_s` i varvet.

Vad den loopen inte är

Det är inte åtta steg som hårdvaran utför ett efter ett. Det är här en bakgrund i C vilseleder som mest, så det är värt att säga rakt ut: en `for`-loop vars gränser är fastlagda vid elaboreringen **rullas ut** av syntesverktyget. De åtta iterationerna blir åtta XOR-grindar i en kedja, som alla lägger sig inom en fortplantningsfördröjning, i samma klockcykel. Det finns ingen räknare för `i`, ingen förgrening, ingen iteration i tiden. Loopen skriver åtta grindar på tre rader; den gör inte åtta saker i följd.

Det är också därför `acc`, som läser sig som en löpande summa, inte kostar någon lagring alls. Den behöver aldrig överleva en klockflank: vart och ett av dess värden är bara ledningen

mellan två av de där XOR-grindarna. En *variable* är rätt verktyg just för att den aldrig överlever ett varv.

Detsamma gäller `bits'range`. **Attributet** `'range` ger indexintervallet för bits, här 7 downto 0, så att loopen täcker varje bit utan att gränserna skrivs två gånger. Det löses upp vid elaboreringen, som allt annat i loopens huvud, vilket är precis därför verktyget kan rulla ut den. `for i in 0 to 1 loop` i kapitel 3:s `led_toggle.vhd` betyder samma sak: båda bitarna får sina egna grindar och båda uppdateras i samma cykel.

Tumregeln

- Använd en *variable* för en tillfällig beräkning som konsumeras helt inom ett varv: loopackumulatorer, temporära byten, mellanvärden som lämnas över till en signal på slutet, precis som `acc`.
- Använd en *signal* för allt som en annan process eller omvärlden läser, och allt som måste hålla sitt värde över skilda varv (en räknare, eller ett skiftregisters lagrade bitar, båda i kapitel 6).

En variabel *kan* behålla sitt värde mellan varv genom samma process, och beter sig då som statisk lokal lagring snarare än en ny lokal variabel varje gång, så det går att bygga tillstånd med en. Men eftersom dess uppdateringar är omedelbara snarare än schemalagda beter den sig annorlunda än en signal som används på samma sätt, särskilt för allt som modellerar en kedja av register som stegar ett steg per klockflank. Om tillstånd genuint behöver bestå och läsas av annan logik, grip efter en *signal*: semantiken med schemalagd uppdatering är det som får kapitel 6:s räknare och skiftregister att fungera.

5.3 Vad FPGA:n faktiskt består av

En FPGA är inget oskrivet blad som grindar etsas in i. Den är ett fast rutnät av identiska små block, tillverkade innan någon skrev din VHDL, och "syntes" är att lista ut hur de ska konfigureras så att de beter sig som den krets du beskrev. Två av dem spelar roll här:

- En **uppslagstabell**, eller **LUT**: ett litet minne, typiskt fyra till sex ingångar och en utgång. Ladda den med sanningstabellen för vilken boolesk funktion som helst av de ingångarna, så beräknar den den funktionen. Det är det ärliga svaret på "vad blir en grind på en FPGA": ingenting blir en grind. En LUT med fyra ingångar beräknar $a \text{ and } b$, $a \text{ xor } b$ or $(c \text{ and not } d)$, eller vilken annan funktion som helst av upp till fyra ingångar till exakt samma kostnad, eftersom det i varje fall är samma block som håller en annan tabell.
- En **vippa**: en bit klockad lagring, som sitter bredvid varje LUT.

På DE0-CV:ns Cyclone V är de paketerade tillsammans som en **ALM**, som håller en delbar LUT med sex ingångar, användbar som en funktion av sex variabler eller två mindre, plus fyra vippor. Det är den enhet Quartus fitter-rapport räknar, vilket är skälet till att den talar om ALM:ar snarare än grindar.

Det är nästan hela historien. `register4` blir fyra vippor, `mux_8to1` en handfull LUT:ar, och kapitel 8:s tillståndsmaskin LUT:ar som matar de vippor som håller tillståndet.

Det förklarar också något från kapitel 2 som annars skulle se ut som ett brutet löfte: boken lät dig minimera uttryck med Karnaughdiagram för att spara grindar, och nämnde sedan aldrig besparingen igen. På en FPGA kostar en funktion av fyra variabler en LUT vare sig du minimerade den eller inte. Minimering spelar fortfarande roll: det är så du förstår en krets, den är precis rätt för de flöden med diskret logik och ASIC som Karnaughdiagram uppfanns för, och att reducera en funktion under LUT:ens ingångsbredd är det som hindrar den från att behöva en andra nivå av LUT:ar. Men på det här kortet köper den oftast ingenting, och det är bättre att veta det än att undra.

5.4 Varför en klocka har en hastighetsgräns

Signaler tar tid på sig genom en LUT och längs ledningarna mellan block. Den **fortplantningsfördröjningen** är skälet till att en klocka inte kan gå hur snabbt som helst.

Tänk dig den form varje design i den här boken har, en vippa, lite kombinatorik, ännu en vippa:

- På en stigande flank presenterar den första ett nytt värde, som måste färdas genom varje LUT och ledning mellan de två och *lägga sig* innan nästa flank anländer.
- Om det inte har gjort det fångar den andra vippan ett värde som fortfarande ändras, och designen gör fel sak tillförlitligt och gätfullt.

Så klockperioden måste täcka tre saker:

- Den första vippans **clock-to-Q-fördröjning**: tiden mellan flanken och att dess utgång faktiskt ändras.
- Den **kombinatoriska fördröjningen** genom LUT:arna och ledningarna mellan de två.
- Den andra vippans **setup-tid**.

Den långsammaste sådana vägen är **kritiska vägen**, och ett genom den summan är **Fmax**.

Quartus beräknar Fmax åt dig, i tidsanalyssteget (bilaga B), men bara om du talar om för den vad klockan är. Det innebär en `.sdc`-villkorsfil med ett `create_clock` på 50 MHz-ingången. Utan en sådan har analysatorn ingen period att jämföra något mot, så den rapporterar varje väg som obegränsad och ger dig ingen Fmax alls. Skilt från det är en design som "missar timingen" en där någon väg visade sig långsammare än den period du faktiskt deklarerade.

Inget av det ingår i den här boken. Du kommer inte att skriva någon `.sdc`, läsa någon timingrapport, eller möta en design som missar timingen, eftersom varje design här ryms inom 20 ns med marginal. Det är värt att veta att steget finns och vad det kräver, så att du känner igen det första gången en design av dig närmar sig gränsen.

Två följder värda att bära med sig:

- **Djup kombinatorik kostar dig hastighet, inte antalet vippor.** Att kedja ett långt uttryck mellan två register förlänger kritiska vägen; att dela upp det så att en del av arbetet sker

på en cykel och resten på nästa förkortar den. Den avvägningen, fler register mot en snabbare klocka, är det enskilt vanligaste greppet i snabb digital konstruktion.

- **50 MHz är en budget på 20 ns.** Det är den siffra varje väg i din design tävlar mot, och det är gott om tid vid de här hastigheterna.

5.5 Låset du får av misstag

Kapitel 3 byggde ett D-lås, visade att det var transparent medan det var aktiverat, och lade det sedan åt sidan till förmån för vippan. Här kommer det tillbaka objudet.

Avsnitt 2.5 krävde ett `when others` på varje `case`, och gav språkets skäl: ett `case` måste täcka varje värde av sin selektors typ. Den allmänna regeln bakom det är att en kombinatorisk process måste tilldela sin utgång på *varje* väg genom sig, och det skälet går nu att formulera:

- VHDL kräver att en signal behåller sitt gamla värde om ingenting tilldelar ett nytt.
- Om någon väg genom din process lämnar `x` otilldelad har du sagt att `x` måste *minnas* sitt föregående värde på den vägen, och att minnas är inte något ett kombinatoriskt nät kan göra.
- Så verktyget bygger det enda som kan: ett lås, precis det från avsnitt 3.2.

Det är ett **oavsiktligt lås**, och det är nästan aldrig vad någon menade. Det är en varning snarare än ett fel, designen kommer ofta att verka fungera, och meddelandet är värt att känna igen:

Warning: Inferred latch(es) for signal "x"

Åtgärden är aldrig att lägga till ett lås med flit. Den är att tilldela signalen på varje väg, vilket är regeln du redan följde.

5.6 Sammanfattning

Det här ska du kunna nu

- Säga vad som skiljer en signal från en variable, förutsäga det olika resultat var och en ger ur samma kod, och välja mellan dem medvetet.
- Förklara vad en FPGA fysiskt består av, varför två olika utseende beskrivningar av samma krets ger samma hårdvara, och varför ett Karnaughdiagram köper mindre här än på diskret logik.
- Förklara vad som begränsar en designs klockfrekvens, och varför fler register kan göra en design snabbare snarare än långsammare.
- Känna igen ett oavsiktligt lås på dess varning, säga vad som orsakade det och åtgärda det, och säga varför "det kompilerade utan fel" lovar mindre för en FPGA-design än för ett C-program.

Frågor att testa dig själv med

1. En signal tilldelas två gånger, med olika värden, i ett varv genom en process. Vad hamnar på den, och när sker den uppdateringen egentligen?
2. Samma fråga för en variable.
3. Varför ger ackumulering över en loop med en signal ett annat, oftast felaktigt, resultat?
4. Ingenting på en FPGA blir en grind. Vad blir en boolesk funktion i stället, och varför kostar en funktion av fyra variabler lika mycket oavsett om du minimerade den först?
5. Vad är kritiska vägen, och varför kan fler vippor få en design att gå snabbare?
6. Ett kombinatoriskt case tilldelar sin utgång i tre grenar av fyra. Vad bygger syntesverktyget för den fjärde, och varför är det det enda det *kan* bygga?
7. Varför är ett oavsiktligt lås en varning och inte ett fel, och varför gör det saken farligare snarare än mindre farlig?

Blicken framåt

Ingenting i kapitel 6 till 8 inför något nytt idiom. Räknare, skiftregister och timers håller alla tillstånd över klockflanker med hjälp av kapitel 3:s klockade process och schemalägningsregeln ovan, och kapitel 8:s tillståndsmaskiner lägger bara till en uppräknad typ och ett case på den. Det som ändras är vad tillståndet används till.

Två saker att hålla utkik efter. Kapitel 6:s `serial_rx8` är den första designen vars korrekthet du inte kan kontrollera med ögat på en lysdiod, vilket är där de utdelade testbänkarna slutar vara en formalitet. Och avsnitt 4.8:s generic sätts i arbete direkt: kapitel 7:s timer använder en för sitt tickantal, och `walking_led` instansierar den timern vid sidan av `reset_sync` och `button_sync`, och skriver nästan ingen egen logik.

5.7 Övningar

Så kontrollerar du ditt arbete. Varje övning nedan som ber dig skriva en VHDL-modul har en självkontrollerande testbänk under `lectures/L05/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7 för de tre kommandona.

ÖVNINGSGRUPP 1

variable kontra signal

Övning 5.1 Spåra ackumulatorbuggen

RESONEMANG

Den trasiga parity_gen-arkitekturen från avsnitt 5.2 återges nedan:

```
architecture broken of parity_gen is
  signal parity_s: std_logic;
begin
  parity <= parity_s;

  process(bits) is
  begin
    parity_s <= '0';
    for i in bits'range loop
      parity_s <= parity_s xor bits(i);
    end loop;
  end process;
end architecture;
```

Anta att bits har intervallet 7 downto 0, att bits = "10110110", och att parity_s håller '0' innan processen börjar.

a) Spåra processen för hand. Notera, för den inledande tilldelningen och för varje loopiteration: det aktuella indexet *i*, värdet på bits(*i*), värdet som läses från parity_s, värdet som schemaläggs för parity_s, och om den schemalagda tilldelningen överlever fram till att processen suspenderar.

Kom ihåg att en signaltilldelning inte uppdaterar signalen omedelbart, och att tilldelningar till samma signal under ett och samma varv ersätter tidigare schemalagda tilldelningar för samma simuleringstidpunkt.

b) Bestäm det slutliga värde som tilldelas parity_s efter att processen suspenderat. Beräkna sedan den korrekta XOR-pariteten hos "10110110" för hand, jämför den med resultatet från den trasiga arkitekturen, och förklara varför upprepade tilldelningar till signalen inte skapar en ackumulator.

c) Spåra den variabelbaserade arkitekturen behaviour från parity_gen.vhd med samma ingång. Notera, för varje loopiteration: värdet på bits(*i*), värdet på acc före XOR-operationen, och värdet på acc omedelbart efter den. Bekräfta att acc behåller varje iterations resultat så att nästa iteration kan använda det, och att det slutliga värde som tilldelas parity är den korrekta XOR-pariteten.

Övning 5.2 min_of_two

VHDL

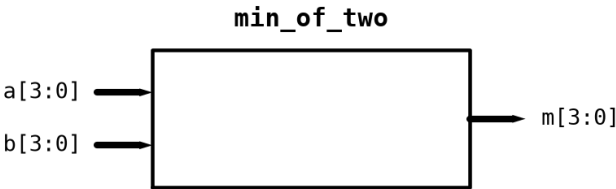
Skriv en entitet med namnet min_of_two.

Port	Riktn.	Typ	Betydelse
a	in	std_logic_vector(3 downto 0)	Första värdet.
b	in	std_logic_vector(3 downto 0)	Andra värdet.
m	out	std_logic_vector(3 downto 0)	Det minsta av de två.

Behandla a och b som teckenlösa fyrabitarsvärden. Implementera kretsen med en process:

- Ta med a och b i känslighetslistan.
- Deklarera en processlokal variabel som håller det minsta värdet.
- Jämför a och b som teckenlösa tal, med faciliteterna i `ieee.numeric_std` enligt avsnitt 2.3.
- Tilldela variabeln exakt en gång per varv, och tilldela den till m efter jämförelsen.

Den här övningen kräver ingen loop. Poängen med den är att använda en variabel som tillfällig processlokal lagring för något *annat* än en ackumulator: avsnitt 5.2:s genomarbetade exempel bygger upp ett värde över iterationerna i en loop, vilket är det fall en variabel oftast introduceras för, och det är lätt att gå därifrån och tro att det är det enda de duger till. Här håller variabeln helt enkelt ett mellanresultat under de få rader som skiljer beräkningen från användningen, vilket är den vanligare av de två användningarna i verklig kod.



Självkontroll. Döp din entitet till min_of_two, med portarna a, b (in) och m (out), alla `std_logic_vector(3 downto 0)`, deklarerade i den ordningen; dess testbänk finns i `exercises/min_of_two/`.

Övning 5.3 ones_count8

VHDL

Skriv en entitet med namnet `ones_count8` som räknar hur många bitar i en 8-bitars ingång som är ettställda.

Övning 5.1 lät dig spåra ackumulatorbuggen på papper, och Övning 5.2 använde en variabel som inte var någon ackumulator alls. Den här är det fall avsnitt 5.2 faktiskt handlar om: ett värde som byggs upp över iterationerna i en loop. Det är den enda övningen i boken där du skriver en sådan.

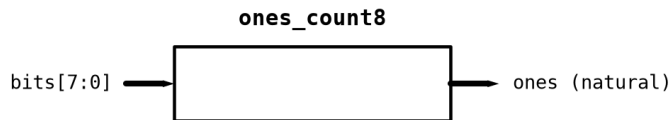
Port	Riktn.	Typ	Betydelse
bits	in	<code>std_logic_vector(7 downto 0)</code>	Värdet att räkna över.
ones	out	<code>natural range 0 to 8</code>	Antalet bitar i bits som är '1'.

Implementera den med en enda kombinatorisk process:

- Sätt `bits` i känslighetslistan.
- Deklarera en processlokal `variable` för den löpande räkningen, och sätt den till 0 överst i varje varv. En variabel behåller sitt värde mellan varv, så en räkning som aldrig nollställs fortsätter växa.
- Loopa över `bits`' range och lägg till 1 för varje ettställd bit.
- Tilldela variabeln till `ones` en gång, efter loopen.

Svara sedan, med en mening var:

- Skriv samma sak med en signal som ackumulator i stället, kör testbänken, och notera vad den rapporterar för ingången 00000010. Förklara talet du får med schemalägningsregeln, inte genom att gissa.
- Loopen körs åtta gånger. Hur många adderare lägger den här designen på FPGA:n, och hur många klockcykler tar det för den att producera ett svar? Om de två svaren överraskar dig, läs om avsnitt 5.2.
- `ones` är en `natural range 0 to 8` snarare än ett `std_logic_vector(3 downto 0)`. Båda rymmer svaret. Vad köper intervallet dig, och vad skulle GHDL göra om din räkning någonsin lämnade det?



Självkontroll. Döp din entitet till `ones_count8`, med ingången `bits` (`std_logic_vector(7 downto 0)`) och utgången `ones` (`natural range 0 to 8`), deklarerade i den ordningen; dess testbänk finns i `exercises/ones_count8/` och sveper igenom alla 256 ingångsvärden.

ÖVNINGSGRUPP 2

En modul värd att ha för sig själv

Övning 5.4 sync: synkroniseraren som egen modul

VHDL

Skriv dubbelvippssynkroniseraren som en egen modul, med namnet sync.

Du har redan byggt den här kretsen. Det är de två första vipporna i kapitel 4:s `button_sync`, skrivna inline där eftersom det vid den punkten var *idén* som spelade roll: en asynkron ingång behöver två vippor innan något annat får titta på den. Här bryter du ut de två till en modul, och övningen är inte logiken, som du redan kan, utan allt runt omkring.

Två saker gör det värt att ta en gång till. Den första är att en naken synkroniserare är vad de flesta asynkrona ingångar faktiskt behöver: `button_sync` lägger till en flankdetektor eftersom en knapptryckning är en händelse, men en seriell ledning, en statusflagga från en annan klockdomän, eller en strömbrytare vars nivå är allt du någonsin läser, behöver de två vipporna och inget mer. Kapitel 8:s seriemottagare instansierar precis det här på sin rx-pinne. Den andra är att den tar **två generics**, och en av dem är inte ett tal.

Generic	Typ	Förval	Betydelse
SIZE	natural range 1 to 15	1	Hur många oberoende signaler som ska synkroniseras, och därmed bredden på båda vektorportarna.
PRESET	std_logic	'0'	Värdet som båda stegen antar vid reset.

och de här portarna:

Port	Riktn.	Typ	Betydelse
clock	in	std_logic	50 MHz systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, redan synkroniserad reset. En synkroniserare synkroniserar inte sin egen reset; det gör kapitel 4:s <code>reset_sync</code> .
async_in	in	std_logic_vector(SIZE - 1 downto 0)	Den asynkrona ingången eller ingångarna, rakt utifrån klockdomänen.
sync_out	out	std_logic_vector(SIZE - 1 downto 0)	Samma signaler, säkra att använda, två stigande flanker senare. Varje bit synkroniseras oberoende.

a) Implementera den:

- Två interna signaler av typen `std_logic_vector(SIZE - 1 downto 0)`, en per steg.
- En process, känslig för `clock` och `reset_s2_n`.
- Vid reset, driv båda stegen till PRESET, utanför den klockade grenen.
- Vid en stigande klockflank, skifta `async_in` genom de två stegen.
- Driv `sync_out` från det andra steget med en konkurrent tilldelning.

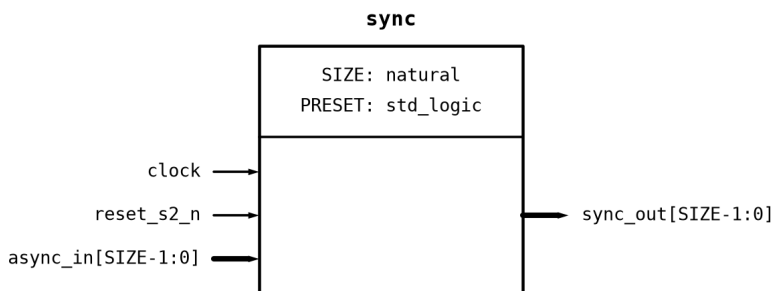
b) PRESET är en generic av typen `std_logic`, den första i den här boken som inte är ett tal, och den finns eftersom det säkra reset-värdet beror på vad signalen *betyder*. Bara den instansierande designen vet det. Svara:

- `button_sync` synkroniserar aktivt låga knappar, så den skulle skicka in '1'. Vad skulle '0' påstå om knapparna under de två cyklerna efter varje reset?
- En seriell ledning vilar hög, så kapitel 8:s mottagare skickar också in '1'. Hur skulle '0' se ut för en mottagare som håller utkik efter en startbit?
- Varför är *standardvärdet* ändå '0' snarare än '1', givet båda dessa, och vad säger det om vem som ansvarar för att få det rätt?

c) Det här är också en fråga om vad en generic kostar. `SIZE` är `natural` range 1 to 15 och `PRESET` är `std_logic`, och båda är fastlagda innan syntesen körs. Använd avsnitt 5.3:s bild av vad FPGA:n faktiskt består av och säg hur många vippor en instans med `SIZE = 4` använder, hur många en instans med `SIZE = 1` använder, och var i den byggda kretsen `PRESET` hamnar. Ingen av de två dyker upp som logik någonstans; säg vad som händer med dem i stället.

d) Verifiera den med dess testbänk. Den instansierar din modul två gånger, en gång en bit bred med `PRESET = '1'` och en gång tre bitar bred med `PRESET = '0'`, så en design som hårdkodar någon av de två generics fallerar. Den kontrollerar att `sync_out` följer `async_in` efter *exakt* två stigande flanker snarare än efter en eller så småningom, att varje bit i en vektor synkroniseras för sig, och att reset laddar `PRESET` utan att någon klockflank är inblandad.

Tips: Skriv arkitekturen för `SIZE = 1` i huvudet först, och kontrollera sedan att varje rad redan är korrekt för en vektor. Aggregatet (`others => PRESET`) och en vanlig vektortilldelning arbetar båda elementvis, så generaliseringen bör inte kosta någon extra kod.



Självkontroll. Döp din entitet till `sync`, med generics `SIZE` (`natural` range 1 to 15, standardvärde 1) och `PRESET` (`std_logic`, standardvärde '0'), ingångarna `clock`, `reset_s2_n`, `async_in` (`std_logic_vector(SIZE - 1 downto 0)`) och utgången `sync_out` (`std_logic_vector(SIZE - 1 downto 0)`), deklarerade i den ordningen; dess testbänk finns i `exercises/sync/`.

Spara den här filen. Kapitel 8:s capstone med seriemottagaren instansierar den på rx-pinnen, och ber dig kopiera in den.

ÖVNINGSGRUPP 3

Från VHDL till hårdvara

Quartus-flödet demonstreras på ett riktigt DE0-CV-kort under passen — först med bromsassistenten redan i kapitel 1 — och finns nedskrivet i bilaga B som referens. Du behöver varken Quartus eller ett kort själv; övningarna nedan handlar om att förstå vad det flödet gör med den VHDL du skriver.

Övning 5.5 Vad verktygskedjan gör

RESONEMANG

Besvara vart och ett av följande, med hänvisning till bilaga B och till det du såg demonstreras. En eller två meningar var.

- Ett Quartus-projekt behöver en vald **toppnivåentitet** innan det kompilerar. Vad gör verktyget annorlunda med toppnivåentiteten än med någon annan entitet i projektet? `min_of_two` från Övning 5.2 och dess testbänk är båda giltig VHDL. Varför kan bara den ena någonsin vara toppnivåentitet i ett FPGA-projekt?
- Pinntilldelning kopplar ett portnamn till en fysisk pinne. Varför måste pinnarna tilldelas *före* kompileringen snarare än efteråt? Vad skulle m fysiskt vara kopplad till om du kompilerade `min_of_two` utan att ha tilldelat några pinnar alls?
- Syntesen gör om din arkitektur till grindar på chippet. Kapitel 2:s övning `combo_logic` lät dig skriva $x = ab + c'd$ två gånger: en gång som ett enda uttryck, och en gång med $c'd$ namngivet av en intern signal. Vad bör syntesen producera för vardera, och varför är ditt svar detsamma för båda? Vilken av de två skulle du hellre läsa om sex månader?
- Kompileringen ger varningar såväl som fel. Varför är "det kompilerade utan fel" ett mycket svagare påstående om en FPGA-design än "det kompilerade utan fel" är om ett C-program? Nämn en sorts varning du aldrig skulle ignorera.

Övning 5.6 Fyra förutsägelser om pariteten

SIMULERING

`parity_gen` från avsnitt 5.2 är kapitlets genomarbetade exempel, och till det hör en referenstestbänk.

- Förutsäg, innan du kör något, `parity` för hand för var och en av de här ingångarna: "00000000", "10110110", "11111111" och "10000000".
- Kör referenstestbänken mot det genomarbetade exemplet och bekräfta dina fyra förutsägelser:

```
cd lectures/L05/parity_gen
ghdl -a --std=93 parity_gen.vhd parity_gen_tb.vhd
ghdl -e --std=93 parity_gen_tb
ghdl -r --std=93 parity_gen_tb --assert-level=error --stop-time=10ms
```

c) Ta vilken som helst av de ingångarna och vänd exakt en bit, och lämna de andra sju oförändrade. Vad händer med parity? Förklara varför en ändring av exakt en ingångsbit *alltid* måste invertera XOR-paritetsresultatet, vilken bit du än väljer och vilket startvärdet än är. Det är den egenskapen som gör en paritetsbit förmögen att upptäcka ett enbitsfel vid överföring. Vilken sorts fel missar den, och varför?

Övning 5.7 En avkodare som kompilerar rent och ändå är fel RESONEMANG

En kollega skickar dig den här kombinatoriska avkodaren för granskning. Den kompilerar, och Quartus rapporterar inga fel.

```
architecture behaviour of level_decode is
begin
  process(sel) is
  begin
    case sel is
      when "00" => leds <= "0001";
      when "01" => leds <= "0010";
      when "10" => leds <= "0100";
      when others => null;
    end case;
  end process;
end architecture;
```

a) Quartus ger Warning: Inferred latch(es) for signal "leds". Förklara med hjälp av avsnitt 5.5 vad verket byggt och varför. Vad bad VHDL-koden om som ett kombinatoriskt nät inte kan göra?

b) Författarens försvar är att sel är två bitar, så "11" är det enda värde when others kan matcha, och designen genererar det aldrig. Ge två skilda skäl till att det argumentet inte håller: ett om vad sel faktiskt kan hålla (avsnitt 1.4), och ett som skulle gälla även om sel verkligen bara kunde vara "00", "01" eller "10".

c) Åtgärda den, på två olika sätt: genom att ändra enbart grenen when others, och genom att lägga till en enda rad *före* case-satsen och lämna varje gren orörd. Vilken av de två skulle du hellre underhålla, och varför? Den andra skalar till ett case med tjugo grenar; den första gör det inte.

d) Det här är en varning, inte ett fel, och designen kommer ofta att verka fungera på kortet. Varför gör det saken farligare än ett fel snarare än mindre farlig?

Räknare och skiftregister

Byggblock för allt som behöver räkna eller flytta bitar.

I DET HÄR KAPITLET

- Räknare: ett register, en adderare och en konstant 1, och varför överslaget kommer gratis.
- Skiftregister: att flytta data en bit per klockflank, åt ett håll som används konsekvent.
- SIPO och PISO, och vilket som passar vilket jobb.
- En 8-bitars seriemottagare: ett skiftregister plus en biträknare, som vet när en byte anlänt.
- Två demonstrationer på kort: en rad lysdioder som räknar och slår över, och en byte som skiftas in en knapptryckning i taget.

Innan räknaren når 1111: förutsäg vad den kommer att visa härnäst, och leta sedan upp det element i ritningen som får det att hända. Det finns inget, vilket är poängen: ingenting i kretsen utför överslaget, och det är en konsekvens av registrets bredd snarare än logik någon lagt dit.

`serial_rx8` i avsnitt 6.5 är bokens första design vars korrekthet du inte kan kontrollera med ögat i full fart: kontinuerligt aktiverad sväljer den en byte på 160 ns. Det är därför den har en referenstestbänk, och därför kortdemonstrationen i avsnitt 6.7 driver `shift_enable` från en knapp i stället. Att sakta ned *klockan* vore mjukvaruinstinkten och fel drag; modulen har redan den styrning som låter dig ta en bit i taget.

Repetition: Räknare och skiftregister byggda av vippor och grindar står i avsnitt 6.1–6.2 i *Digital Electronics*.

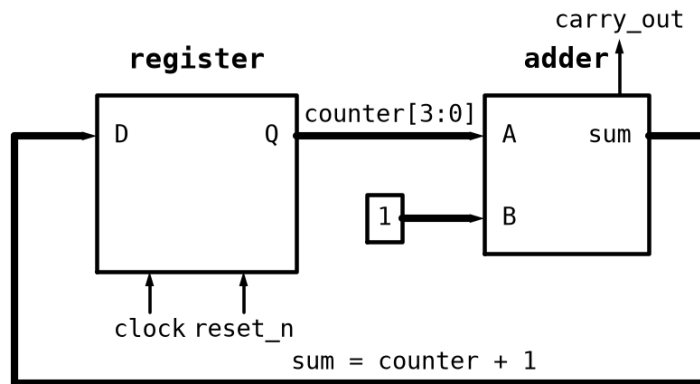
6.1 Från register till räknare

Ett register (kapitel 3) är en bank av D-vippor som lagrar vilket värde som än matas in vid nästa stigande flank. En **räknare** är samma idé med en vridning: i stället för ett nytt värde utifrån matar du registret med dess *eget nuvarande värde plus ett*. Vid varje flank fångar det `current_value + 1`, så det lagrade talet ökar en gång per klockcykel utan någon vidare inmatning.

Bygg den för hand först

Som med varje krets i den här boken: bygg den innan du skriver den. En 4-bitars räknare är tre element, och att se den köra är hela poängen:

- Ett **4-bitars register**: fyra D-vippor som delar en klocka, exakt registret från avsnitt 3.5.
- En **adderare** och en **konstant 1**: registrets fyra utgångar till adderarens ena ingång, konstanten till den andra.
- Adderarens summa kopplad tillbaka till registrets fyra D-ingångar. Den återkopplingen är hela kretsen: registret håller ett tal, adderaren beräknar det talet plus ett, nästa flank lagrar det. Det är avsnitt 3.1:s återkopplingsslinga igen, med en adderare i vägen i stället för en ren ledning.



Figur 6.1 En 4-bitars räknare: ett register vars egen utgång, plus ett, kopplas tillbaka till dess ingång.

Bekräfta sedan, en klockflank i taget, med en period på 1000 ms:

- Se utgångarna stega 0000, 0001, 0010, 0011, ..., en räkning per stigande flank.
- **Låt den köra förbi 1111.** Den återgår till 0000 av sig själv, och ingenting i din krets säger åt den att göra det: det finns ingen jämförelse, ingen nollställningslogik, inget "om räkningen står på maxvärdet" någonstans. Adderaren producerade 10000, den femte biten hade ingenstans att ta vägen, och de fyra bitar som överlevde är 0000. Det är **spill**, och i en räknare är det inte en bugg att arbeta runt utan den egenskap som resten av det här kapitlet och hela kapitel 7 vilar på.
- **Håll ögonen på carry_out på adderaren medan det sker.** Det är den femte biten, och ritningen visar exakt var den hamnar: hög under den enda flank där räkningen slår över, kopplad till ingenting, ut ur adderaren och stopp där. En bredare räknare skulle mata in den i ett femte steg; här kastas den bort, och att kasta bort den *är* överslaget. Så "räkningen slår över för att bitarna tar slut" är inte ett bildligt uttryck. Du kan peka på biten som tog slut.

Spara projektet. Kapitel 7 utgår från exakt den här kretsen och lägger till två saker till den, och att öppna den igen är bättre än att rita om den.

Samma räknare i VHDL

En signal och en process, precis som kapitel 3:s register, bortsett från att högerledet refererar till signalen själv:

```
signal counter: natural range 0 to 15;
...
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        counter <= 0;
    elsif (rising_edge(clock)) then
        counter <= counter + 1;
    end if;
end process;
```

`natural` är en fördefinierad heltalssubtyp i VHDL: ett icke-negativt heltal, till skillnad från `std_logic`-typerna som modellerar bitar direkt. VHDL:s aritmetiska operatörer fungerar på den så som vanlig heltalsmatematik gör, vilket är varför `counter + 1` inte behöver något extra maskineri. Syntesen gör ändå om den till ett register av vippor, precis som ett `std_logic_vector` skulle bli; skillnaden ligger enbart i hur du skriver och resonerar om värdet.

Intervallbegränsningen `0 to 15` är inte bara dokumentation. Den talar om för syntesverktyget att den här räknaren behöver fyra bitar, så att räkna förbi 15 faller tillbaka till 0 gratis, enbart som en konsekvens av att bitarna tar slut, utan någon uttrycklig "om counter = 15"-logik vare sig i ritningen eller i VHDL-koden.

Det gratis överslaget har ett villkor värt att säga ut: det inträffar bara när den övre gränsen är $2^N - 1$. `0 to 15` slår över gratis; `0 to 9` gör det inte, eftersom 10 inte är en tvåpotensgräns, och en modulo-10-räknare behöver en uttrycklig jämförelse. Det är precis vad Övning 6.4 ber dig skriva.

Ett ställe där simulatoren är oense med hårdvaran. Det gratis överslaget är en egenskap hos den *syntetiserade* kretsen, och ritningen visar att det är verkligt. En VHDL-simulator är strängare än en ledning:

- `natural range 0 to 15` intervallkontrolleras, så GHDL behandlar $15 + 1 = 16$ som ett fel och avbryter i stället för att slå över.
- Den syntetiserade hårdvaran har ingen sådan föreställning. Det finns inget sextonde värde för bitarna att anta, så de landar på 0000, precis som dina fyra lysdioder gjorde.

Grindnätet och chippet slår båda över; bara simulatören använder, eftersom ett intervallbegränsat heltal är ett löfte du gett och 16 bryter det. Regeln som följer är inte "lita på hårdvaran": den är att allt du tänker verifiera bör säga vad det menar. Skriv ut jämförelsen, så som `timer` och `serial_rx8` gör, så är ritningen, simulatören och chippet alla överens.

Det här överslaget som följer av bitbredden är räknarens enskilt viktigaste egenskap, och det är av det kapitel 7 bygger en timer: en räknare som håller utkik efter ett bestämt räknarvärde och höjer en flagga när det kommer.

6.2 Skiftregister

Ett **skiftregister** är, återigen, N D-vippor, kopplade så att varje vippas utgång matar *nästa* vippas ingång i stället för sin egen. Vid varje klockflank flyttar varje bit en position nedåt i kedjan, samtidigt:

```
serial_in -> [D Q]-->[D Q]-->[D Q]-->[D Q] -> serial_out
             FF0      FF1      FF2      FF3
             bit 0    bit 1    bit 2    bit 3
             ^clock  ^clock  ^clock  ^clock  (alla delar samma klocka)
```

Efter en flank ligger FF0:s innehåll i FF1, FF1:s i FF2, och så vidare; en ny bit kommer in i FF0 från `serial_in`, och det som låg i den sista vippan faller av änden som `serial_out`. Kör den i N flanker så har N seriella bitar vandrat hela vägen igenom.

Ett håll, som används genomgående i den här boken. Ett skiftregister kan flytta bitar åt båda hållen, och båda är lika giltiga; det är att blanda dem inom en och samma design som skapar förvirring. Så varje skiftregister här följer diagrammet ovan:

- kedjan avbildas på ett `std_logic_vector` med FF0 som **bit 0** och FF(N-1) som bit N-1.
- seriella data **kommer in vid bit 0** och **lämnar från bit N-1**.
- varje bit flyttar en position mot den **mest signifikanta** biten per flank.

I VHDL är det hållet ett enda uttryck, identiskt i varje exempel nedan:

```
shift_reg <= shift_reg(N-2 downto 0) & serial_in;
```

`shift_reg(N-2 downto 0)` är allt utom den översta biten, flyttat upp en position; den översta biten har ingenstans att ta vägen och faller av som `serial_out`; `serial_in` fyller den lediggjorda bit 0.

När du möter ett skiftregister utanför den här boken: kontrollera dess håll innan du antar något. Spegelbilden, `serial_in & shift_reg(N-1 downto 1)`, är precis lika vanlig och ser vid en snabb blick nästan likadan ut.

Skiftregister namnges efter hur data kommer in och lämnar:

Utförande	Data in	Data ut	Typisk användning
SISO	en bit per klocka	en bit per klocka	en ren fördröjningslinje
SIPO	en bit per klocka	alla N bitar på en gång	att ta emot en ström av seriella bitar, t.ex. en inkommande byte på en enda ledning
PISO	alla N bitar på en gång (laddade)	en bit per klocka	att sända ut N bitar över en enda ledning
PIPO	alla N bitar på en gång	alla N bitar på en gång	i praktiken ett vanligt register (avsnitt 3.5)

Det här kapitlet tar upp de två praktiskt användbara, SIPO och PISO.

6.3 Serie in, parallell ut (SIPO)

Ett SIPO-register är det du griper efter när bitar anländer en i taget på en enda ledning och du behöver det ackumulerade värdet på en gång. Klassikern är att deserialisera en inkommande byte, vilket är vad mottagarvägen i vilket seriellt protokoll som helst i grunden gör.

Vilken ände av byten som anländer först är varje protokolls eget beslut. SPI och CAN sänder som förval den mest signifikanta biten först, vilket är vad avsnitt 6.2:s håll förutsätter: den första biten som anländer hamnar i MSB när alla N har skiftats in. (Bitordningen i SPI går att ställa om på de flesta styrkretsar, så kontrollera komponenten.) Ett protokoll som sänder den minst signifikanta biten först, som UART, skiftar åt det spegelvända hållet, och dess idiom är också spegelbilden.

Idiomet är en rad inuti en klockad process, avsnitt 6.2:s skiftuttryck ordagrant:

```

signal shift_reg: std_logic_vector(7 downto 0);
...
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_reg <= (others => '0');
    elsif (rising_edge(clock)) then
        shift_reg <= shift_reg(6 downto 0) & serial_in;
    end if;
end process;
parallel_out <= shift_reg;

```

Efter 8 flanker har de mottagna bitarna helt ersatt registrets ursprungliga innehåll, och parallel_out exponerar alla 8 på en gång.

6.4 Parallell in, serie ut (PISO)

Ett PISO-register vänder på SIPO:s uppgift: ladda N bitar på en gång, sänd sedan ut dem en i taget. Det är det du griper efter när ett fast mönster ska ut över en enda ledning, till exempel till en kedja av lysdioder eller en display driven av ett externt skiftregisterchip.

Det behöver en styrsignal mer än SIPO: något som skiljer att *ladda* ett nytt parallellt värde från att *skifta* ut det befintliga.

```
process(clock, reset_s2_n) is
begin
  if (reset_s2_n = '0') then
    shift_reg <= (others => '0');
  elsif (rising_edge(clock)) then
    if (load = '1') then
      shift_reg <= parallel_in;           -- Parallel load.
    elsif (shift = '1') then
      shift_reg <= shift_reg(N-2 downto 0) & serial_in; -- Shift toward the-
-- MSB.
    end if;
  end if;
end process;
serial_out <= shift_reg(N-1);
```

Bit N-1 är alltid nästa bit att lämna, så ett laddat värde går ut med den mest signifikanta biten först: ladda "1010" i ett 4-bitars register så presenterar serial_out 1, 0, 1, 0 under de följande cyklerna.

Lägg märke till hur lite som skiljer det här från avsnitt 6.3:s SIPO. **Skiftuttrycket är identiskt**, samma håll, samma rad VHDL. Bara två saker skiljer: PISO lägger till en load-gren, och den tappar av shift_reg(N-1) som seriell utgång där SIPO exponerar hela vektorn. "SIPO" och "PISO" namnger hur du *kopplar* ett skiftregister, inte två sorters hårdvara, vilket är varför avsnitt 6.2 kunde beskriva skiftningen en gång innan något av namnen dök upp.

Om en design behöver ett verkligt nytt serial_in vid varje skiftning, eller nöjer sig med att återcirkulera det som faller av änden, avgörs av hur den är kopplad. Den återcirkulationen är precis det trick som exemplet med den vandrande lysdioden i kapitel 7 använder, när det väl har en timer som taktar skiftningen.

6.5 Genomarbetat exempel: en 8-bitars seriemottagare

Kapitlets två halvor möts i en modul. Ett skiftregister kan ta emot seriella bitar i all evighet men kan inte tala om *när* en komplett byte anlant, eftersom det inte har någon föreställning om "hur många bitar hittills". Det är ett räkneproblem, och avsnitt 6.1 löste det redan.

serial_rx8 kombinerar de två, plus en data_ready-puls på en cykel när den åttonde biten landar.

Det här är ingen leksak: det är främre halvan av varje seriemottagare som finns. Mottagarvägarna i SPI, CAN och UART börjar alla med ett skiftregister och en biträknare, och skiljer sig bara i vad som avgör när en bit är giltig.

Port	Riktn.	Typ	Betydelse
clock	in	std_logic	Systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, redan synkroniserad reset; nollställer registret och biträkningen. Asynkron, så den nollställer dem utan att någon klockflank är inblandad.
shift_enable	in	std_logic	Hög under exakt en klockcykel per inkommande bit.
serial_in	in	std_logic	Den inkommande biten, samplad vid varje aktiverad stigande flank.
data_out	out	std_logic_vector(7 downto 0)	Byten som mottagits hittills, mest signifikanta biten först.
data_ready	out	std_logic	Puls på en cykel när den åttonde biten landar.

Resetporten är den synkroniserade, och _s2 säger det. Den heter reset_s2_n snarare än reset_n eftersom det som hör hemma på den är utgången från en reset_sync (avsnitt 4.5), modulen du skrev i Övning 4.8: aktivera asynkront, släpp synkront. **Instansiera en i den design som omsluter den här mottagaren och koppla dess reset_s2_n hit**, vilket är precis vad avsnitt 6.7:s omslagsmodul på kortet gör. Koppla i stället en tryckknapp rakt in på den här porten så får du den kapplöpning kapitel 4 finns till för att förhindra: skiftregistret och biträknaren är ett dussin vippor, och en asynkront *släppt* reset låter dem lämna reset vid olika flanker, så mottagaren kan börja räkna en byte en cykel innan den börjar skifta in en.

counter, sipo8 och piso8 i övningarna tar reset_s2_n av samma skäl. Ingen av dem är en design du sätter på ett kort för sig själv: en räknare, ett skiftregister och en mottagare är delar, och en del har rätt att anta att designen omkring den har skött synkroniseringen. Så hela det här kapitlet tar den synkroniserade reset-signalen, och bara omslagsmodulerna i avsnitt 6.6 och 6.7, som äger pinnarna, får någonsin se en rå reset_n.

En **toppnivå** namnges tvärtom: den tar reset_n och instansierar reset_sync själv, eftersom det är den som håller pinnen. blinker och walking_led i kapitel 7, och båda capstones i kapitel 8, är skrivna så. Skaffa dig vanan att läsa _s2 som ett krav på anroparen, för härifrån och framåt är det mesta du skriver en subkomponent.

```
lectures/L06/serial_rx8/serial_rx8.vhd
```

```
-- 8-bit serial receiver: a SIPO shift register paired with a bit counter, so it
-- reports a complete byte rather than shifting forever. Bits arrive most
```

```

-- significant first, one per enabled clock edge. See Appendix A.5.
--
-- Inputs:
--   - clock      : 50 MHz system clock.
--   - reset_s2_n : Active-low, already-synchronized reset: drive it from a
--                   reset_sync (L04 A.5), never straight from a pin. ↪
--   ↪ Asynchronous
--                   here, so it clears the register and the bit count with no
--                   clock edge involved.
--   - shift_enable: High for one clock cycle per incoming bit; holds both the
--                   shift register and the bit count when low.
--   - serial_in   : Serial input bit, sampled on each enabled rising edge.
-- Outputs:
--   - data_out    : Received byte, valid while data_ready is high.
--   - data_ready  : One-cycle pulse as the eighth bit lands.
-----
library ieee;
use ieee.std_logic_1164.all;

entity serial_rx8 is
    port(clock, reset_s2_n, shift_enable, serial_in: in std_logic;
          data_out      : out std_logic_vector(7 downto 0);
          ↪ 0);
          data_ready    : out std_logic);
end entity;

architecture behaviour of serial_rx8 is

    -- Bit width.
    constant BIT_WIDTH: natural := 8;

    -- Most significant bit.
    constant MSB: natural := BIT_WIDTH - 1;

    -- Shift register.
    signal shift_reg: std_logic_vector(MSB downto 0);

    -- Bit counter.
    signal bit_count: natural range 0 to MSB;

begin
    data_out <= shift_reg;

    process(clock, reset_s2_n) is

```

```

begin
  if (reset_s2_n = '0') then
    shift_reg <= (others => '0');
    bit_count <= 0;
    data_ready <= '0';
  elsif (rising_edge(clock)) then
    data_ready <= '0';
    if (shift_enable = '1') then
      shift_reg <= shift_reg(MSB - 1 downto 0) & serial_in;
      if (bit_count >= BIT_WIDTH - 1) then
        data_ready <= '1';
        bit_count <= 0;
      else
        bit_count <= bit_count + 1;
      end if;
    end if;
  end if;
end process;
end architecture;

```

Vid varje stigande flank där `shift_enable = '1'` skiftas `serial_in` in i bit 0 med avsnitt 6.3:s idiom oförändrat, så den första biten som anländer har vandrat upp till bit 7 när den åttonde landar, och en intern biträknare av typen `natural range 0 to 7` räknar upp. Vid den flank där den räknaren redan står på 7 pulsar `data_ready` hög under en cykel och räknaren nollställs tillbaka till 0, redo för nästa byte. `data_out` drivs av en konkurrent tilldelning från skiftregistret, så den kompletta byten ligger på den under den cykel då `data_ready` är hög.

Lägg märke till att räknaren *nollställs* här snarare än lämnas att slå över: 0 to 7 är ett tvåpotensintervall så den skulle slå över gratis, men att skriva ut det håller simulatören överens med hårdvaran, enligt avsnitt 6.1.

Två detaljer är värda att dröja vid, eftersom båda är mönster du mött mer än en gång:

- **`data_ready` är en puls, inte en nivå.** Den drivs låg villkorslöst högst upp i den klockade grenen och höjs bara vid den flank som fullbordar en byte: samma encykelsform som flankdetektorn i avsnitt 3.7. En konsument som missar den har missat byten, vilket är precis därför den är parad med ett `data_out` som *håller* sitt värde efteråt. En riktig kringkrets skulle sätta ett hållregister eller en liten FIFO här så att en långsam konsument inte kan förlora data.
- **`shift_enable` grindar biträknaren, inte bara skiftningen.** Båda går framåt tillsammans eller ingendera, så ett avbrott i den inkommande strömmen pausar mottagaren mitt i en byte i stället för att korrumpera den, och den återupptar på exakt den bit den väntade på.

Att kontrollera den. Till exemplet hör en utdelad `serial_rx8_tb.vhd`, som skiftar in två byte med en paus emellan och kontrollerar att `data_ready` pulsar en gång per byte, aldrig för tidigt:

```
cd lectures/L06/serial_rx8
ghdl -a --std=93 serial_rx8.vhd serial_rx8_tb.vhd
ghdl -e --std=93 serial_rx8_tb
ghdl -r --std=93 serial_rx8_tb --assert-level=error --stop-time=10ms
```

Taktad på 50 MHz och kontinuerligt aktiverad sväljer den här mottagaren en byte på 160 ns, vilket ingen lysdiod kan visa dig. Det är därför testbänken spelar roll här på ett sätt den inte gjorde för en blinkande lysdiod, och så är det för det mesta av verklig digitalkonstruktion. Men det är inget skäl att hoppa över kortet, för `shift_enable` är just utvägen: se avsnitt 6.7.

6.6 På kortet: att se en räknare slå över

Räknaren demonstreras på DE0-CV under passet, byggd på samma sätt som varje design sedan kapitel 1: ett Quartus Prime Lite-projekt mot 5CEBA4F23C7N, .vhd-filen tillagd, kompilera, tilldela varje port en fysisk pinne i Pin Planner, kompilera sedan om och programmera kortet.

En sak måste ändras först, och det är värt att se varför. Taktad på 50 MHz går en 4-bitars räknare igenom alla sexton tillstånd och slår över var 320 ns, ungefär tre miljoner gånger i sekunden. En lysdiod driven direkt från den blinkar inte, den lyser helt enkelt med halv styrka. För att se ett överslag måste du titta på en mycket långsammare bit: räkna i ett bredare register, säg 28 bitar, och tilldela de **översta** fyra bitarna till lysdioderna.

Demonstrationen är därför en omslagsmodul kring två moduler snarare än övningens entitet för sig: en `reset_sync` som tar den råa `reset_n` från pinnen, och den breddade räknaren som tar sin `reset_s2_n` därifrån. Räknarens port heter `reset_s2_n` just för att den här kopplingen ska vara den enda som typkontrollerar, och avsnitt 6.7 upprepar mönstret med en tredje modul.

- clock till 50 MHz-oscillatorn, `reset_n` till en tryckknapp (in i `reset_sync`, aldrig rakt in i räknaren), `count(27 downto 24)` till fyra lysdioder.
- Bit 24 ändras en gång var 2^{24} klockcykler, ungefär en tredjedels sekund, så de fyra lysdioderna räknar synligt från 0000 upp till 1111 och sedan, utan att något i kretsen gör det, tillbaka till 0000. Ett helt varv tar ungefär fem sekunder.

Att dela en klocka genom att titta på en hög bit är den grövsta tänkbara timern, och dess begränsning är värd att namnge redan nu: perioden är låst till en tvåpotens, och det enda sättet att ändra den är att välja en annan bit. Kapitel 7 ersätter den med en räknare jämförd mot ett målvärde du väljer, vilket är allt en timer egentligen är.

6.7 På kortet: att skifta in en byte för hand

Avsnitt 6.5 sa att en byte anländer för fort för att kunna följas. Lösningen är inte att sakta ned klockan, vilket vore mjukvaruinstinkten och är fel drag i hårdvara: klockan är den enda sak i en synkron design som du inte pillar på. `serial_rx8` har redan rätt styrning, och det är `shift_enable`, som ”grindar biträknaren, inte bara skiftningen”. Håll den låg så stannar mottagaren tvärt, mitt i en byte, hur länge du vill. Höj den under exakt en cykel så går exakt en bit in.

Driv alltså `shift_enable` från en **knapp** i stället för att hålla den hög. Ett tryck, en bit.

Demonstrationen är en liten omslagsmodul kring tre moduler du redan har, och ingen ny logik utöver en enda vippa:

- `reset_sync` (avsnitt 4.5), som gör den råa `reset_n` till `reset_s2_n` för allt annat.
- `button_sync` (avsnitt 4.6) med `COUNT = 1`, som gör ett tryck till den encykelspuls som `shift_enable` vill ha. Det här är ingen valfri utsmyckning: en rå knapp kopplad till `shift_enable` skulle skifta in en slumpmässig handfull bitar per tryck, en per klockcykel så länge kontakterna studsar.
- `serial_rx8` själv, oförändrad, med sin `reset_s2_n` tagen från `reset_sync` ovan snarare än från knappen. Det är hela skälet till att porten heter `reset_s2_n`: omslagsmodulen äger den råa `reset_n`, och mottagaren får bara någonsin se den synkroniserade.
- En vippa som låser `data_ready`, satt när pulsen anländer och nollställd av reset. Utan den finns ingenting att se: `data_ready` är hög i 20 ns, och det kan ingen lysdiod visa dig.

På kortet: clock till 50 MHz-oscillatorn, `reset_n` till en tryckknapp, `serial_in` till en strömbrytare (det är den bit du är på väg att skicka), skiftknappen till en andra tryckknapp, `data_out` (7 downto 0) till åtta lysdioder, och den låsta `data_ready` till en nionde.

Ställ nu strömbrytaren, tryck på knappen, och se en bit komma in vid lysdioden längst till höger och hela mönstret stega åt vänster. Skicka 10110010 en bit i taget så byggs byten upp över lysdioderna framför dig, mest signifikanta biten först, precis som avsnitt 6.3:s idiom säger att den måste. Vid det åttonde trycket tänds den nionde lysdioden och biträknaren börjar om.

Tre saker som det här gör synliga och som testbänken bara kan hävda:

- **Bitordningen.** Den första biten du skickar hamnar i bit 7. Att läsa av det ur en vågform är ett tjugtåra; att se den vandra över en rad lysdioder är det inte.
- **Vad `shift_enable` är till för.** Släpp knappen en minut mitt i en byte så rör sig ingenting, ingenting går förlorat, och den åttonde biten fullbordar ändå byten. Det är egenskapen ”pausar mitt i en byte i stället för att korrumpdera den”, demonstrerad snarare än påstådd.
- **Puls mot nivå, en gång till.** Skälet till att en vippa måste läggas till för `data_ready` är hela den distinktionen, och det är samma skäl som flankdetektorn i avsnitt 3.7 fanns till för.

6.8 Sammanfattning

Det här ska du kunna nu

- Bygga en 4-bitars räknare som grindnät, och förklara, med fingret på din egen ritning, varför ingenting i den får räkningen att återgå till noll.
- Implementera samma räknare i VHDL, förklara överslaget som en konsekvens av registers bitbredd, och säga varför en simulator invänder mot ett överslag som hårdvaran utför utan att blinka.
- Implementera ett skiftregister i både SIPO- och PISO-utförande, veta vilket som passar vilket jobb, och veta vilket håll boken skiftar åt och varför det spelar roll att hålla sig till ett.
- Kombinera en räknare med ett skiftregister för att upptäcka att ett komplett ord anlärt, och verifiera en klockad design genom att köra dess testbänk.

Frågor att testa dig själv med

1. Varför återgår en räknare av typen `natural range 0 to 15` till 0 efter 15 utan någon uttrycklig "om maxvärdet, nollställ"-logik? Vad måste gälla för intervallet? Vilket element utför överslaget?
2. En VHDL-simulator avbryter på det överslag din CircuitVerse-ritning utför utan invändning. Vilken av de två modellerar hårdvaran, och vad säger det om att lita på någondera ensam?
3. Vad är den praktiska skillnaden mellan ett SIPO- och ett PISO-skiftregister? Vilket skulle du gripa efter för att ta emot en ström av seriella bitar, och vilket för att driva en kedja av lysdioder från ett fast mönster?
4. Ett skiftregister och en räknare är båda "N vippor som delar en klocka". Det som skiljer är vad som matar varje vippas ingång. Beskriv den skillnaden för respektive.
5. Varför är `data_ready` i `serial_rx8` en puls på en klockcykel snarare än en nivå som blir liggande hög när en byte anlärt?
6. Varför måste `shift_enable` grinda biträknaren och inte bara skiftningen?
7. Kortdemonstrationen driver `shift_enable` från en knapp i stället för att sakta ned klockan. Varför är nedsaktad klocka fel svar, och vad skulle gå sönder om knappen kopplades till `shift_enable` utan att först passera `button_sync`?
8. `data_ready` behöver en egen vippa innan en lysdiod kan visa den. Vilket kapitals problem är det, i annan skepnad?

Blicken framåt

Kapitel 7 går vidare med:

- Timers: en räknare med ett målvärde, vilket är det som gör klockcykler till riktig tid.
- Byggt genom att lägga exakt två saker till räknaren du ritade här, en jämförare och en nollställning, så spara det CircuitVerse-projektet.
- Den vandrande lysdioden: det här kapitlets skiftregister, taktat av den timern och startat av en synkroniserad knapp.

6.9 Övningar

Så kontrollerar du ditt arbete. Varje övning nedan som ber dig skriva en VHDL-modul har en självkontrollerande testbänk under `lectures/L06/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7 för de tre kommandona.

ÖVNINGSGRUPP 1

Räknare

Övning 6.1 Bredd och överslagspunkt

RESONEMANG

En signal deklareras som:

```
signal counter: natural range 0 to 1023;
```

- Hur många bitar bred blir det register som det här syntetiseras till?
- Vilket är det största värde counter kan hålla innan den slår över tillbaka till 0 vid nästa ökning?
- Skriv om deklarationen för en räknare som i stället behöver räkna från 0 upp till 63. Följ räknarmönstret i avsnitt 6.1.

Övning 6.2 Spåra ett överslag för hand

RESONEMANG

Spåra en räknare av typen `natural range 0 to 3` för hand (ingen VHDL behövs). Anta att räknaren startar på 2, att den ökas en gång per klockflank under fem flanker i följd, och att det inte finns någon reset däremellan.

Notera, klockflank för klockflank: värdet som hålls före flanken, och värdet som fångas på flanken. Förklara varför räknaren återgår till 0 utan någon uttrycklig "om räkningen når sitt maxvärde, nollställ den"-logik.

Observera: Det här är det *syntetiserade* beteendet, det som spelar roll för hårdvara. Avsnitt 6.1 flaggar för det simuleringsförbehåll som hör till: GHDL intervallkontrollerar en signal av typen `natural range 0 to 3`, så en simulering av den här räknaren avbryter vid överslaget i stället för att rulla över. Spåra den på papper som hårdvara, och grip efter en `unsigned` i full bredd om du någonsin behöver en räknare som slår över också i simulering.

Övning 6.3 Räknaren för hand, och biten som tar slut

KRETS

Bygg 4-bitarsräknaren från avsnitt 6.1 för hand i CircuitVerse, och använd den för att se spillet i stället för att ta det på förtroende.

a) Bygg den: ett 4-bitars register (fyra D-vippor som delar en klocka), en adderare med registrets utgångar på den ena ingången och en konstant 1 på den andra, adderarens summa kopplad tillbaka till registrets D-ingångar, och fyra utgångselement eller lysdioder, en per bit.

b) Kör den med en klockperiod på 1000 ms och notera vad du ser:

- Skriv ner sekvensen av räkningar över sexton klockflanker i följd, med start från 0000.
- Håll ögonen på `carry_out` på den översta adderaren medan räkningen rullar från 1111 till 0000: under hur många av de sexton flankerna är den hög, och vad är kopplat till den?
- Förklara, med en mening, vart den femte biten tar vägen, och varför registret lagrar 0000 snarare än 10000.

c) Resonera nu om det du *inte* byggde: peka på den del av din ritning som avgör att räknaren ska återgå till 0000. Det finns ingen. Säg vad som gör jobbet i stället. Vad skulle du behöva lägga till för att få den att räkna 0 till 9 och sedan slå över? (Övning 6.4 ber dig skriva precis det i VHDL, så spara ditt svar.)

d) Ändra bredden till tre bitar genom att ta bort den översta vippan och smalna av adderaren. Förutsäg den nya överslagspunkten innan du kör den, bekräfta den, och formulera regeln som kopplar en räknares bredd till det värde den slår över vid.

Spåra den här kretsen. Kapitel 7 bygger sin timer genom att lägga till två saker till den, en jämförare och en nollställning, och att utgå från en räknare du redan har sett fungera är hela poängen med övningen där.

Övning 6.4 counter: en räknare med godtycklig radix

VHDL

Skriv en entitet med namnet counter som räknar från 0 upp till ett steg under sin radix, och sedan slår över tillbaka till 0. Med standardvärdet på radix är det en decimalräknare, 0 till 9.

Entiteten har en generic:

Generic	Typ	Förval	Betydelse
RADIX	natural range 1 to 2**16	10	Hur många värden räknaren går igenom innan den upprepar sig. Den räknar 0 till RADIX-1 inklusive, så RADIX är cykelns längd, inte den största räkningen.

och de här portarna:

Port	Riktn.	Typ	Betydelse
clock	in	std_logic	Systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, redan synkroniserad : den kommer från din reset_sync, aldrig rakt från en pinne. Asynkron, så den nollställer utan att någon klockflank är inblandad.
count	out	natural range 0 to RADIX-1	Den löpande räkningen. 0 är ett värde den antar, så intervallet börjar där: reset driver den till 0, och den återvänder dit vid varje överslag.
tick	out	std_logic	En puls på en cykel vid den flank där count slår över från RADIX-1 tillbaka till 0.

Implementera den med en enda synkron process:

- Ta med clock och reset_s2_n i känslighetslistan.
- Vid reset, sätt count till 0 och tick till '0'.
- Vid varje stigande klockflank: driv tick låg som utgångsläge; när count har nått RADIX-1, slå över count till 0 och pulsa tick hög; öka annars count.

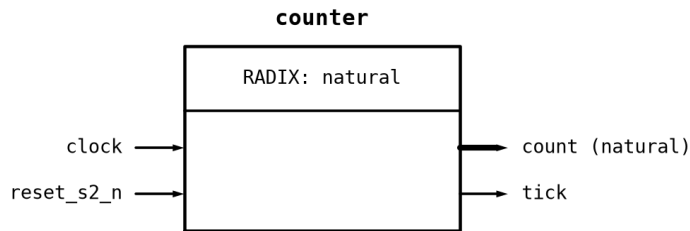
Skriv jämförelsen mot RADIX-1, aldrig mot 9. Genericen är hela skillnaden mellan en modul och en modul du kan använda två gånger: RADIX är fastlagd innan syntesen körs, så jämföraren den bygger blir inte större än en hårdkodad, och räknarens register dimensioneras efter den radix du ber om. Det är samma argument som avsnitt 4.8 för fram för button_sync:s COUNT, och skälet till att kapitel 7:s timer tar sitt målvärde som en generic snarare än som en konstant.

Till skillnad från det gratis överslaget hos en räknare av typen natural range 0 to 15 behöver en räknare med godtycklig radix en uttrycklig jämförelse, eftersom RADIX-1 i allmänhet inte är en tvåpotensgräns. Lägg märke till vad som händer när den är det: med RADIX = 16 skrivs jämförelsen fortfarande ut, men hårdvaran under behöver den inte längre.

Tips: Processen behöver läsa den löpande räkningen (för att jämföra den och för att öka den), så håll räkningen i en intern signal count_s: natural range 0 to RADIX-1; kör logiken på count_s, och driv utgången med count <= count_s; utanför processen, precis

som `d_flip_flop.vhd` använder `q_s` i kapitel 3. Det är inget stilval: varje ghdl-kommando i den här boken skickar `--std=93`, och under VHDL-93 kompilerar det inte alls att läsa en out-port. Senare standarder mjukar upp det, vilket är varför du kan ha sett det göras på annat håll.

Följ räknarmönstret i avsnitt 6.1, och den driv-låg-och-pulsa-sedan-form som `data_ready` har i avsnitt 6.5.



Självkontroll. Döp din entitet till `counter`, med genericen `RADIX` (`natural range 1 to 2**16`, standardvärde 10), ingångarna `clock`, `reset_s2_n` och utgångarna `count (natural range 0 to RADIX-1)` och `tick (std_logic)`, deklarerade i den ordningen; dess testbänk finns i `exercises/counter/`. Den elaborerar **två** instanser, en med `RADIX = 10` och en med `RADIX = 4`, och kör vardera genom två fulla cykler, och kontrollerar att `tick` är en puls på en cykel snarare än en nivå och att den aldrig utlöser halvvägs igenom. En arkitektur som tyst hårdkodar 10 klarar den första instansen och fallerar på den andra, vilket är poängen med att kontrollera två.

ÖVNINGSGRUPP 2

Skiftregister

Övning 6.5 Fyra bitar in i ett SIPO-register

RESONEMANG

Ett 8-bitars SIPO-skiftregister startar på 00000000 och använder idiomet från avsnitt 6.3:

```
shift_reg <= shift_reg(6 downto 0) & serial_in;
```

Det tar emot den seriella bitströmmen 1, 0, 1, 1 (en bit per klockflank, i den ordningen) över fyra stigande klockflanker i följd.

- Skriv ut hela det 8-bitars innehållet i `shift_reg` efter var och en av de fyra klockflankerna.
- Efter de fyra flankerna: vilka fyra bitar i `shift_reg` håller de mottagna data, och i vilken ordning ligger de, i förhållande till den ordning bitarna anlände i?

Övning 6.6 Fyra bitar ut ur ett PISO-register

RESONEMANG

Ett 4-bitars PISO-register, som följer idiomet från avsnitt 6.4, är parallellt laddat med `parallel_in = "1010"` (load = '1' under en klockflank), sedan skiftat med `shift = '1'` under de följande fyra klockflankerna, med `serial_in` hållen på '0' hela tiden.

Kom ihåg hållet från avsnitt 6.2: bitarna flyttar mot MSB, `serial_out` är bit 3 (värdet som är på väg att lämna), och `serial_in` kommer in vid bit 0.

- Skriv ut registrets innehåll och `serial_out` vid varje steg, en rad per klockflank: raden för laddningsflanken, före all skiftning, och sedan en rad för var och en av de fyra skiftflankerna.
- I vilken ordning dyker de fyra laddade bitarna upp på `serial_out`, i förhållande till sina positioner i `parallel_in`?
- Vad är registrets slutliga innehåll efter alla fyra skiftningarna, och varför?

Tips: Du behöver ingen riktig hårdvara för att kontrollera Övning 6.5 och 6.6 för hand. Gå igenom dem precis som du skulle spåra en sanningstabell: en klockflank, en rad, i taget.

Övning 6.7 sipo8 och piso8

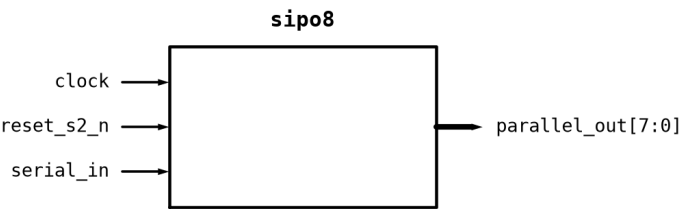
VHDL

Implementera kapitlets två användbara skiftregisterutföranden i VHDL.

- sipo8.** Skriv en entitet med namnet `sipo8`, ett register med serie in och parallell ut.

Port	Riktn.	Typ	Betydelse
<code>clock</code>	in	<code>std_logic</code>	Systemklocka.
<code>reset_s2_n</code>	in	<code>std_logic</code>	Aktiv låg, redan synkroniserad . Asynkron. Nollställer hela registret, så <code>parallel_out</code> läser "00000000" medan den hålls låg.
<code>serial_in</code>	in	<code>std_logic</code>	Den inkommande biten.
<code>parallel_out</code>	out	<code>std_logic_vector(7 downto 0)</code>	Registrets innehåll.

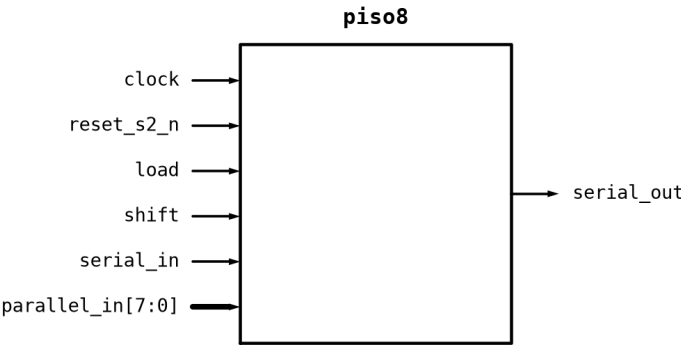
Följ SIPO-idiomet från avsnitt 6.3.



b) **piso8**. Skriv en entitet med namnet `piso8`, ett register med parallell in och serie ut.

Port	Riktn.	Typ	Betydelse
clock	in	std_logic	Systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, redan synkroniserad . Asynkron. Nollställer hela registret, så <code>serial_out</code> läser '0' medan den hålls låg.
load	in	std_logic	Fånga <code>parallel_in</code> vid nästa stigande flank.
shift	in	std_logic	Skifta en position vid nästa stigande flank.
serial_in	in	std_logic	Biten som matas in vid bit 0 vid en skiftning.
parallel_in	in	std_logic_vector(7 downto 0)	Värdet som fångas vid en laddning.
serial_out	out	std_logic	Registrets översta bit.

Följ PISO-idiotet från avsnitt 6.4: vid en laddningsflank (`load = '1'`), fånga `parallel_in`; vid en skiftflank (`shift = '1'`), flytta varje bit en position mot MSB och mata in `serial_in` vid bit 0; driv `serial_out` från bit 7, så att en laddad byte lämnar med den mest signifikanta biten först; och låt `load` ha företräde framför `shift` när båda är höga.



c) Namnge, för vart och ett av de fyra skiftregisterutförandena (SISO, SIPO, PISO, PIPO), en uppgift som det är det naturliga valet för. Följ sammanfattningstabellen i avsnitt 6.2.

Härled varje idiom på nytt ur kapitlet snarare än att kopiera `serial_rx8`:s skiftrad.

Självkontroll. sipo8: portarna clock, reset_s2_n, serial_in (in) och parallel_out (std_logic_vector(7 downto 0), out), deklarerade i den ordningen; testbänk i exercises/sipo8/. piso8: portarna clock, reset_s2_n, load, shift, serial_in, parallel_in (std_logic_vector(7 downto 0)) och serial_out (out), deklarerade i den ordningen; testbänk i exercises/piso8/.

Spara piso8. Kapitel 8:s capstone med sändaren instansierar den inte, men den är värd att läsa om där för idiomets skull, eftersom den maskinen skiftar ut byten inline.

ÖVNINGSGRUPP 3

Att läsa seriemottagaren

Till det genomarbetade exemplet hör en referenstestbänk. Kör den från exemplets egen katalog innan du börjar:

```
cd lectures/L06/serial_rx8
ghdl -a --std=93 serial_rx8.vhd serial_rx8_tb.vhd
ghdl -e --std=93 serial_rx8_tb
ghdl -r --std=93 serial_rx8_tb --assert-level=error --stop-time=10ms
```

Övning 6.8 Mottagarens beteende, och tre riktiga buggar

SIMULERING

Redogör för mottagarens beteende, med hänvisning till avsnitt 6.5.

a) Bitströmmen 1, 0, 1, 1, 0, 0, 1, 0 anländer, en bit per aktiverad klockflank. Vad ligger på data_out under den cykel där data_ready pulsar? Vilken anländande bit hamnar i data_out(7), och vilken i data_out(0)?

b) shift_enable går låg under tre klockcykler efter den fjärde biten, och sedan hög igen under de återstående fyra. Producerar mottagaren fortfarande samma byte? Förklara vad biträknaren gör under uppehållet. Vad skulle i stället gå fel om shift_enable grindade bara skiftregistret och inte räknaren?

c) Gör var och en av följande ändringar i din egen kopia av serial_rx8.vhd, en i taget, och **förutsäg** vad referenstestbänken kommer att rapportera innan du kör den. Kör den sedan och redogör för resultatet:

- Ändra skiftraden till det spegelvända hållet, shift_reg <= serial_in & shift_reg(7 downto 1);.
- Ta bort det villkorslösa data_ready <= '0'; högst upp i den klockade grenen.
- Ändra biträknarens jämförelse från 7 till 6.

Var och en av dem är en verklig bugg som någon har skickat ut skarpt. Säg för var och en vilken av testbänkens kontroller som fångar den, och hur felet hade sett ut på en verklig seriell länk om ingenting hade fångat det.

Timers

En räknare med ett målvärde, ritad för hand innan den skrivs.

I DET HÄR KAPITLET

- Timers: en räknare som höjer en flagga när den nått ett målvärde.
- Att göra klockcykler till riktig tid, och likhetsjämföraren som gör det.
- Samma timer i VHDL, återanvändbar tack vare en generic snarare än genom omkoppling.
- Att komponera en design av färdiga moduler i stället för att skriva om deras logik.
- walking_led: en enda tänd bit som vandrar längs en rad lysdioder.

Innan timern körs: förutsäg vid vilket räknarvärde timeout slår till, och varför det är var elfte flank snarare än var tionde. Det där ettfelet är hela skälet till att modulens generic heter TICK_COUNT och inte PERIOD.

walking_led i avsnitt 7.5 är utdelningen från de fyra senaste kapitlen: tre moduler, två av dem oförändrade från kapitel 4 och en skriven här, komponerade till något du kan se från bakre raden.

Repetition: Timern som en räknare med en jämförelse står i avsnitt 6.3 i *Digital Electronics*.

7.1 Timers: en räknare med ett målvärde

En **timer** svarar på en annan fråga än räknaren från avsnitt 6.1. En räknare svarar på "vad är räkningen just nu?"; en timer svarar på "har en viss tid gått?"

Den gör det med tre delar: en intern räknare som är osynlig utifrån, ett målvärde TICK_COUNT som är fast per instans, och en jämförelse som när den är sann nollställer räknaren och pulsar utgångsflaggan timeout under en klockcykel.

Eftersom systemklockan tickar med en känd, fast frekvens är att räkna tick samma sak som att mäta förfluten tid. På DE0-CV är det 50 000 000 tick per sekund:

$$TICK_COUNT = \text{sekunder} \times 50\,000\,000$$

- TICK_COUNT = 50_000_000: timeout en gång per sekund.
- TICK_COUNT = 25_000_000: två gånger per sekund.

- `TICK_COUNT = 5_000_000`: tio gånger per sekund.

Strikt taget löper räknaren 0 till och med `TICK_COUNT`, så en full period är `TICK_COUNT + 1` cykler. Det där extra ticket är mellan 0,02 och 0,2 ppm vid de här värdena, långt under kortoscillatorns egen tolerans, så de runda talen ovan används för tydlighetens skull; för ett exakt cykelantal, använd $TICK_COUNT = \text{sekunder} \times 50\,000\,000 - 1$.

Det här är motsatsen till en blockerande `_delay_ms()`. En blockerande fördröjning stoppar hela programmet för att vänta; en timer går permanent parallellt med allt annat och höjer helt enkelt en flagga, och ingenting i designen blockeras någonsin i väntan på den.

7.2 Att bygga en timer för hand i CircuitVerse

Innan någon VHDL skrivs: bygg en timer som ett grindnät, precis som du gjorde för kapitel 1 och 2:s kombinatoriska nät och kapitel 3:s vippor. En timer är liten nog att rita i sin helhet, och det är att rita den som gör att avsnitt 7.3:s VHDL läses som en beskrivning av något du redan förstår snarare än ett nytt idiom att lära utantill.

Bygg den i en storlek du kan följa: en **4-bitars räknare** med `TICK_COUNT = 10`, vilket är 1010. Den riktiga designen räknar till 50 000 000; ingenting i strukturen ändras, bara bredden.

Utgå från räknaren du redan byggt i avsnitt 6.1: ett 4-bitars register Q0 till Q3 som delar en klocka, en adderare och en konstant 1 med summan återkopplad till D-ingångarna, och ingenting mer. Om du inte sparade den är det tre element och ungefär fem minuter att rita om. Bekräfta att den fortfarande räknar och slår över innan du lägger till något, så att du vet att räknaren under är sund när timern senare krånglar.

Allt här lägger **två saker ovanpå det**: en jämförelse som säger "räkningen är framme", och en nollställning som får den att upprepa. Det är hela skillnaden mellan en räknare och en timer.

Lägg till jämförelsen. Du vill ha timeout hög exakt när de fyra bitarna stavar 1010, och det rena sättet att bygga "är de här två talen lika?" är en **XNOR-grind per bit**. En XNOR ger 1 när dess ingångar är *lika*, vilket är avsnitt 1.1:s XNOR-kolumn läst som ett likhetstest. Koppla in fyra, var och en som jämför en räknarutgång mot motsvarande målbit (Q3 mot 1, Q2 mot 0, Q1 mot 1, Q0 mot 0), och driv målsidan från en konstant i CircuitVerse snarare än från en ledning du senare kan råka ta för en signal. Kombinera sedan de fyra utgångarna med AND; den enda AND-grinden är timeout. Med $\odot$ för XNOR, den enda operator avsnitt 1.2 inte gav någon symbol:

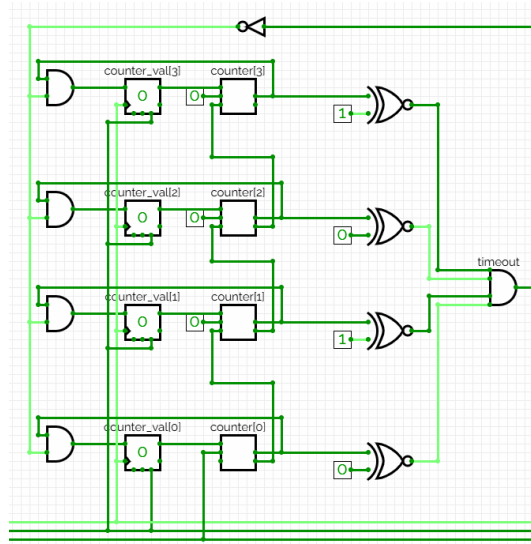
$$\text{timeout} = (Q_3 \odot 1) \cdot (Q_2 \odot 0) \cdot (Q_1 \odot 1) \cdot (Q_0 \odot 0)$$

Alla fyra bitar måste stämma samtidigt, vilket är vad "räknaren har nått 1010" betyder. Det här är en **allmän likhetsjämförare**, värd att känna igen som ett eget byggblock: samma fyra XNOR plus en AND jämför mot vilket 4-bitars värde som helst, och bara konstanterna ändras.

Lägg till nollställningen. Så att timern startar om i stället för att fortsätta: koppla tillbaka timeout för att tvinga räknaren till 0000 vid nästa flank, genom att grinda varje vippas D-ingång så att $\text{timeout} = 1$ går före adderarens utgång. Utan den fortsätter räknaren till 1111 och slår över av sig själv, så timeout slår till var sextonde flank i stället för var elfte. Båda är periodiska; bara den ena har den period du bad om.

Bekräfta sedan, en klockflank i taget.

- Räknaren går från 0000 till och med 1010, och timeout är låg för varje räknarvärde utom 1010. Håll ögonen på de fyra XNOR-utgångarna medan den räknar och lägg märke till hur ofta tre är höga samtidigt. Tre bitar som stämmer är ingen träff; det är AND-grinden som insisterar på alla fyra.
- Vid flanken efter att timeout slagit till är räknaren tillbaka på 0000 och timeout är låg, så den är hög under exakt en klockcykel, aldrig två.
- Hela cykeln är alltså elva flanker, räknarvärdena 0 till och med 10, upprepade i all oändlighet. Det är avsnitt 7.1:s ettfel $\text{TICK_COUNT} + 1$, och det här är bokens enklaste ställe att se det på, eftersom du kan räkna flankerna med ögat.



Figur 7.1 Jämförelselogiken: fyra XNOR mot 1010, ANDade till timeout.

Ritningen är beskuren till den del som det här avsnittet lägger till. Den märker räknarens bitar `counter[3]` ned till `counter[0]` där texten ovan kallar dem Q3 till Q0, och uppräknningen som matar dem är ritad som carry-logik på grindnivå snarare än det enda adderarblock avsnitt 6.1 använde, eftersom det är vad CircuitVerse ger dig när du bygger den en bit i taget.

De fyra XNOR-grindarna, deras konstanter 1, 0, 1, 0, och den AND som kombinerar dem är allt som är nytt här.

Två saker är värda att lägga märke till före VHDL:en, eftersom båda är vad språket köper dig:

- **Jämföraren är fast kopplad till ett målvärde.** Att ändra TICK_COUNT från 10 till 12 innebär att gå tillbaka in i ritningen och vända på två konstanter. Det är en liten ändring, men det är en ändring i *kretsen*, och varje instans behöver sin egen kopia. I avsnitt 7.3 blir målvärdet en generic.
- **Bredden är fast kopplad också.** Att räkna till 50 000 000 kräver 26 vippor och en 26-bitars jämförare, 26 XNOR över 52 ingångar: samma krets, en orimlig ritning. I avsnitt 7.3 är det samma sex rader VHDL hur som helst.

Det är avvägningen den här boken gör hela tiden: rita kretsen en gång, i en storlek du kan se, för att veta vad verktyget bygger åt dig, och låt sedan verktyget bygga den i en storlek du inte hade kunnat rita.

7.3 Modulen timer i VHDL

Samma krets som en återanvändbar modul. Den byggs live under passet, och Övning 7.4 ber dig skriva den själv efteråt, så den är tryckt här i sin helhet snarare än utdelad som en fil:

```
entity timer is
    generic(TICK_COUNT: natural := 50_000_000);
    port(clock, reset_s2_n, enable: in std_logic;
          timeout                : out std_logic);
end entity;

architecture behaviour of timer is
    -- Internal tick counter.
    signal counter: natural range 0 to TICK_COUNT;
begin
    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            timeout <= '0';
            counter <= 0;
        elsif (rising_edge(clock)) then
            timeout <= '0';
            if (enable = '1') then
                if (TICK_COUNT > counter) then
                    counter <= counter + 1;
                else
```

```

        timeout <= '1';
        counter <= 0;
    end if;
end if;
end if;
end process;
end architecture;

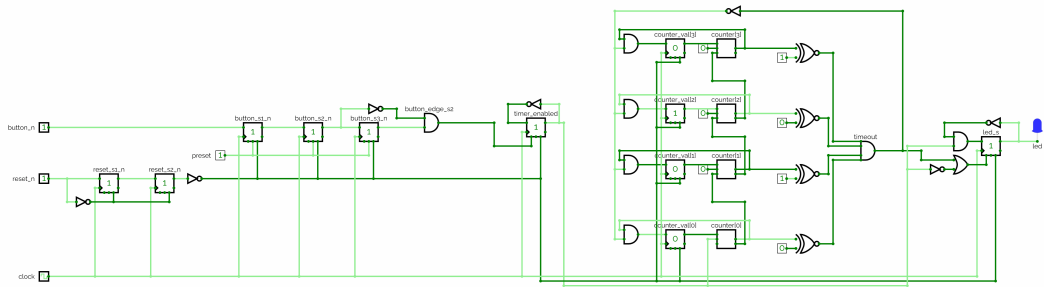
```

- `TICK_COUNT` är en **generic**, inte en konstant inbakad i arkitekturen, så samma entitet betjänar vilken frekvens som helst genom att instansieras med en annan `generic map`. Den interna räknarens intervall är skrivet i termer av den, så registret dimensioneras efter målvärdet snarare än efter vad ett obegränsat `natural` skulle ge, samma poäng som avsnitt 6.1 gör om `natural range 0 to 15`. En `generic` kan dimensionera en signal just för att den är fastlåst innan syntesen körs.
- `reset_s2_n` är den redan *synkroniserade* reset-signalen, aldrig den råa asynkrona `reset_n`. En timer rör aldrig en rå asynkron ingång direkt.
- `enable` **fryser** timern snarare än nollställer den. Medan den är '0' håller counter sitt värde och timeout förblir låg, och en avstängd timer fortsätter där den slutade snarare än startar om. Kapitel 8:s `fsm_led` hänger på det.
- `timeout` är en encykelspuls. Den `else`-gren som sätter den nås vid bara en flank: den flank där counter redan står på `TICK_COUNT` och nollställs tillbaka till 0. Vid den följande flanken körs den villkorlösa `timeout <= '0'`; utan något efter sig som kan skriva över, så pulsen är exakt en cykel bred.

Jämför det här med avsnitt 7.2:s grindnät. Logiken är densamma, och avsnitt 7.2 namngav redan vad genericen köper. En skillnad namngav den inte: avsnitt 7.2:s `timeout` är en kombinatorisk avkodning, hög under den cykel då räkningen visar 1010, medan den här är registrerad, hög en cykel senare, under den cykel då counter visar 0. Samma period, annan fas, och annat glitchbeteende, vilket är den distinktion som kapitel 8:s diskussion om Moore kontra Mealy vilar på.

7.4 Att komponera en komplett timerkrets

En timer ensam pulsar bara `timeout`; något måste reagera på den pulsen, och en knapp måste styra timern. Med kapitel 4:s dubbelvippsynkroniserare återanvänd ser en komplett knappstyrd, timerdriven lysdiodskrets ut så här:

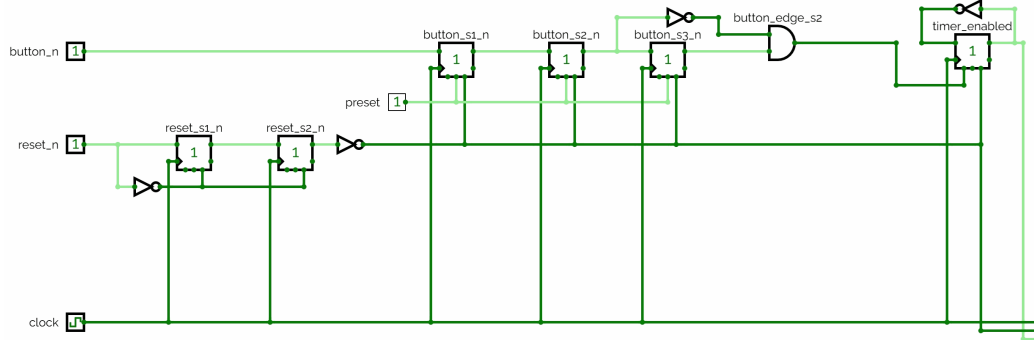


Figur 7.2 Hela kretsen på en gång: synkroniserad knapp, timer och växlad lysdiod. Detaljerna är läsbara i de två närbilderna nedan.

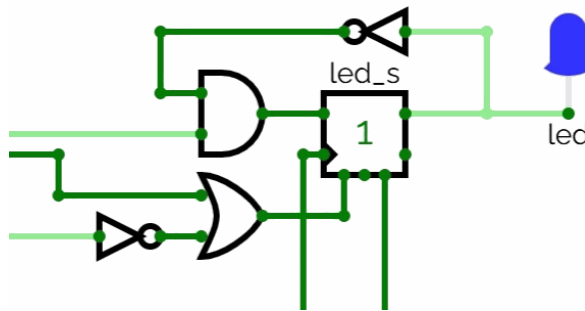
Läs den som en planritning snarare än som ett kopplingsschema. Det är en bred krets, så i sidstorlek blir dess etiketter små; vad den är till för är att visa hur få delar det finns och ungefär var var och en sitter. De två närbilderna nedan är där signalnamnen går att läsa, och tillsammans täcker de allt i den.

Från vänster till höger:

- **Synkronisering och flankdetektering.** `button_n` passerar två vippor för metastabilitets-skydd, och en tredje låter sedan kretsen jämföra "nu" mot "föregående" och detektera en fallande flank, vilket ger en `button_edge_s2`-puls på en cykel.
- **Timern** (avsnitt 7.3), aktiverad eller avstängd av `timer_enabled`, som i sin tur växlas av `button_edge_s2`.
- **Att växla lysdioden.** Timerns `timeout`-puls vänder lysdiodens läge varje gång den slår till, och lysdioden tvingas släckt så snart timern är avstängd eller kretsen resettas.



Figur 7.3 Synkroniserare och detektor för fallande flank.



Figur 7.4 Växlingslogiken för lysdioden, driven av timers timeout-puls.

Två mönster spelar roll här, oberoende av den här kretsen:

- **Varje asynkron ingång synkroniseras innan den rör någon annan logik**, och varje enable- eller växlingssignal som härleds ur en knapp är en synkron signal från den punkten och framåt. Du återanvänder det oförändrat i avsnitt 7.5 och igen för kapitel 8:s tillståndsmaskiner.
- **En klocka, och allt annat är enable.** timeout taktar lysdioden genom att vara en *enable* på en cykel för en vippa som går på systemklockan, inte genom att kopplas in i en klockingång. Det är frestande att dra en räknar- eller timerutgång in i en klockport för

att "sakta ned"; låt bli. En design med en klocka är en design vars tajming verktyget kan analysera, och det är det enskilt vanligaste misstaget en programmerare gör när en baudgenerator ska skrivas.

7.5 walking_led: ett skiftregister med vandrande lysdiod

walking_led är den första designen som komponerar tre kapitels byggblock till en krets som går att demonstrera i hårdvara: en enda tänd lysdiod som vandrar längs en rad, en position per timertick, startad och stoppad med en knapp. Den byggs live under passet, och Övning 7.6 ber dig skriva den själv efteråt, så den är tryckt här i sin helhet snarare än utdelad som en fil.

Den skriver nästan ingen ny logik, utan återanvänder reset_sync och button_sync oförändrade (avsnitt 4.5 och 4.6) för att synkronisera och flankdetektera knappen, timer (avsnitt 7.3) för att takta hastigheten, och skiftregistret från avsnitt 6.2–6.4 för själva vandringen. Det är det de fyra senaste kapitlen har samlat på sig: en liten uppsättning moduler som går att komponera.

Alla tre skriver du själv: reset_sync och button_sync i Övning 4.8 och 4.9, timer i Övning 7.4 i det här kapitlet. Kopiera in dina egna i övningskatalogen innan du bygger den här. Att återanvända en modul förutsätter att man har en, och det här är den första designen som ber dig om dem du redan byggt.

```
entity walking_led is
    generic(LED_COUNT : natural := 8;
           TICK_COUNT: natural := 25_000_000);
    port(clock, reset_n, button_n: in std_logic;
         led
           : out std_logic_vector(LED_COUNT-1 downto 0));
end entity;

architecture behaviour of walking_led is
    signal reset_s2_n, button_edge_s2 : std_logic;
    signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);
    signal shift_enable, shift_tick   : std_logic;
    signal shift_reg: std_logic_vector(LED_COUNT-1 downto 0);

begin
    led <= shift_reg;

    button_n_v(0) <= button_n;
    button_edge_s2 <= button_edge_s2_v(0);

    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);
```

```

button_sync1: entity work.button_sync
  generic map(1)
  port map(clock, reset_s2_n, button_n_v, button_edge_s2_v);

timer1: entity work.timer
  generic map(TICK_COUNT)
  port map(clock, reset_s2_n, shift_enable, shift_tick);

SHIFT_ENABLE_PROCESS: process(clock, reset_s2_n) is
begin
  if (reset_s2_n = '0') then
    shift_enable <= '0';
  elsif (rising_edge(clock)) then
    if (button_edge_s2 = '1') then
      shift_enable <= not shift_enable;
    end if;
  end if;
end process;

SHIFT_PROCESS: process(clock, reset_s2_n) is
begin
  if (reset_s2_n = '0') then
    shift_reg <= (0 => '1', others => '0');
  elsif (rising_edge(clock)) then
    if (shift_tick = '1') then
      shift_reg <= shift_reg(LED_COUNT-2 downto 0) & shift_reg(LED_COUNT-
-1);
    end if;
  end if;
end process;
end architecture;

```

Designval värda att notera:

- button_n_v och button_edge_s2_v överbryggar bara en breddskillnad: walking_led:s knappsignaler är enskilda bitar medan button_sync:s portar är vektorer med bredden COUNT. De två konkurrenta tilldelningarna kopplar skalären till element 0 och tillbaka. De kostar ingen hårdvara.
- Det här är ett PISO-register (avsnitt 6.4) i sin enklaste form: parallellt laddat exakt en gång, vid reset, med en enda tänd bit på position 0, och därefter bara skiftande. Det finns ingen load under körning, eftersom den här designen aldrig laddar ett nytt mönster.
- Det är ett **cirkulärt** skiftregister. Skiftningen är avsnitt 6.2:s uttryck oförändrat, så den tända biten vandrar mot MSB; den enda skillnaden mot ett vanligt PISO är vad som matar

bit 0, vilket är biten som faller av toppen snarare än ett `serial_in` någon annanstans ifrån. Ett engångsregister som "skiftar ut ett mönster" blir en oändligt upprepande mönstergenerator gratis, bara genom att återcirkulera sin egen utgång.

- `shift_enable` är avsnitt 7.4:s växlingsvippa återanvänd ordagrant: knappens synkroniserade flankpuls vänder en enda bit, och den biten blir timerns `enable`.
- `shift_tick` är timerns `timeout`-puls, återanvänd som skiftregistrets `klockenable` i stället för att driva en lysdiodsväxling. Samma byggblock, en annan konsument, och precis regeln "en klocka, allt annat är `enable`" från avsnitt 7.4.
- Både `LED_COUNT` och `TICK_COUNT` är generics, så `generic map(4, 25_000_000)` ger en rad med 4 lysdioder och `generic map(8, 5_000_000)` ett snabbare steg på 100 ms, utan att modulen rörs.

På kortet, demonstrerat under passet, tilldelas portarna via Pin Planner (bilaga B): `clock` till 50 MHz-oscillatorn, `reset_n` och `button_n` till två tryckknappar, och `led` till en rad lysdioder på kortet. Tryck på knappen en gång för att sätta biten i rörelse, en gång till för att frysa den.

Du kan se samma beteende utan kort genom att köra referenstestbänken, som skriver över båda generics så att ett helt varv tar en handfull klockcykler:

```
cd lectures/L07/exercises/walking_led
cp ../../../../L04/exercises/reset_sync/reset_sync.vhd . # the three you wrote ↵
  ↵ yourself
cp ../../../../L04/exercises/button_sync/button_sync.vhd .
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
      walking_led.vhd walking_led_tb.vhd
ghdl -e --std=93 walking_led_tb
ghdl -r --std=93 walking_led_tb --assert-level=error --stop-time=10ms
```

7.6 Sammanfattning

Det här ska du kunna nu

- Förklara vad som skiljer en timer från en vanlig räknare, och varför skillnaden är en jämförelse.
- Bygga en timer som grindnät: en likhetsjämförare av en XNOR per bit, och den nollställning som får den att upprepa i stället för att slå över.
- Implementera en timer i VHDL som en återanvändbar modul, omvandla en period i sekunder till ett `TICK_COUNT`, och säga varför `timeout` är en puls på en klockcykel snarare än en nivå.
- Komponera en design av färdiga moduler i stället för att skriva om dem, och säga varför en knapp som styr en timer ändå måste passera en synkroniserare först.

Frågor att testa dig själv med

1. I vilken mening är en timer "bara en räknare med en jämförelse"?
2. Varför måste den interna räknaren nollställas till 0 när den når sitt mål, i stället för att lämnas att räkna vidare?
3. Varför är timeout en puls på en enda cykel snarare än en nivå som ligger kvar hög?
4. I CircuitVerse-versionen innebär ett ändrat målvärde att jämföraren måste kopplas om. Vad ersätter den omkopplingen i VHDL-versionen, och när låses dess värde fast?
5. Varför är det fortfarande nödvändigt att köra en knapp genom en synkroniserare innan dess flank används för att aktivera en timer, trots att knappen inte har med räkningen att göra?
6. `walking_led` skriver nästan ingen egen logik. Nämn de tre moduler den komponerar och vilket kapitel var och en kom från.

Blicken framåt

Kapitel 8 går vidare med:

- Tillståndsmaskiner, bokens sista byggblock, först konstruerade för hand: tillståndsdia-gram, tillståndstabell, Karnaughhärledd logik och ett grindnät i CircuitVerse, precis som det här kapitlets timer ritades innan den skrevs.
- Därefter i VHDL, där en uppräkningsstyp och ett case ersätter tillståndstabellen och Karnaughdiagrammen helt.
- En timer är i sig en tillståndsmaskin med två tillstånd och ett enda syfte. Kapitel 8 generaliserar det till kretsar med många tillstånd och många övergångar mellan dem.

7.7 Övningar

Så kontrollerar du ditt arbete. Varje övning nedan som ber dig skriva en VHDL-modul kommer med en självkontrollerande testbänk under `lectures/L07/exercises/`. Skriv din modul i dess katalog `exercises/<module>/`, med det entitetsnamn och den **portordning** övningen anger, och kör den sedan med GHDL — se avsnitt 2.7.

Från och med det här kapitlet återanvänder designerna moduler du skrivit tidigare, och de delas inte ut en andra gång: varje övning säger vilka av dina egna filer som ska kopieras in, och skriver ut den `ghdl -a-rad` som behöver dem.

ÖVNINGSGRUPP 1

Timers för hand**Övning 7.1 En timer med ett annat målvärde**

KRETS

Bygg en andra timer för hand i CircuitVerse, enligt avsnitt 7.2, men med målvärdet `TICK_COUNT = 12` i stället för avsnittets 10.

a) Rikta om jämföraren:

- Skriv 12 binärt med 4 bitar.
- De fyra XNOR-grindarna sitter kvar precis där de sitter. Säg vilka av deras konstanta ingångar som ändras, och till vad.
- Säg, i en mening, varför en likhetsjämförare byggd av XNOR inte behöver någon grind tillagd, borttagen eller omkopplad för att jämföra mot ett annat värde.

b) Bygg den: återanvänd din 4-bitars räknare och jämförare från passet, och ändra bara konstanterna. Sätt klockperioden till 1000 ms och bekräfta att timeout slår till en gång var trettonde klockflank, under exakt en flank varje gång.

c) Ta nu bort nollställningen, så att timeout inte längre tvingar räknaren tillbaka till 0000. Förutsäg, innan du kör den, hur ofta timeout slår till nu. Kör den och bekräfta. Förklara resultatet utifrån räknarens eget överslag (avsnitt 6.1).

d) Gör sönder jämföraren med flit: koppla loss en XNOR:s utgång från AND-grinden och håll den AND-ingången hög i stället. Förutsäg vilka räknarvärden som nu höjer timeout, innan du simulerar, och bekräfta genom att stega räknaren genom en hel cykel. Det här är hur "tre av de fyra bitarna stämmer" ser ut, och det är det vanligaste sättet för en handritad jämförare att bli fel.

Tips: Att räkna 0 till och med 12 är tretton flanker, inte tolv. Avsnitt 7.1:s anmärkning om `TICK_COUNT + 1` är samma ettfel, och det är värt att reda ut här, där du kan räkna flankerna med ögat, snarare än senare vid 50 MHz.

Övning 7.2 Fyra frågor om modulen

RESONEMANG

Svara på var och en av frågorna nedan i en eller två meningar, med hänvisning till modulen timer i avsnitt 7.3:

- Varför är timeout en encykelspuls snarare än en signal som ligger kvar hög när målvärdet väl är nått?
- Medan en löpande timers enable-ingång hålls låg, vad händer med dess interna räknare från en klockflank till nästa: nollställs den, pausas den, eller räknar den vidare?
- Varför tar timer `reset_s2_n` (den synkroniserade reset-signalen) snarare än den råa `reset_n` som sin reset-ingång?

d) CircuitVerse-timern i Övning 7.1 behövde kopplas om för att målvärdet skulle ändras, medan VHDL-modulen inte behöver ändras alls. Vad ersätter omkopplingen, och när låses dess värde fast?

ÖVNINGSGRUPP 2

Timers i VHDL

Övning 7.3 Tre tickantal

RESONEMANG

DE0-CV-kortets systemklocka är 50 MHz. Räkna för varje mål nedan ut den `TICK_COUNT`-generic du skulle skicka in till modulen `timer`:

- a) En timeout-puls en gång per sekund.
- b) En timeout-puls var 250 ms.
- c) En timeout-puls tjugo gånger per sekund (20 Hz).

Tips: Sambandet mellan en period och dess tickantal är $TICK_COUNT = \text{sekunder} \times 50\,000\,000 - 1$. Termen -1 är samma ettfel som Övning 7.1 kretsade kring: räknaren löper 0 till och med `TICK_COUNT`, så en period är `TICK_COUNT + 1` klockcykler, inte `TICK_COUNT`.

Övning 7.4 timer

VHDL

Skriv själva modulen `timer`, som en entitet vid namn `timer`.

Den byggs live under passet och är tryckt i sin helhet i avsnitt 7.3, så det här är ingen gåta. Skriv den ändå, utan att kopiera: varje design härifrån till bokens slut instansierar den här modulen, båda capstones inräknade, och det är värt att ha byggt det du strax ska återanvända sex gånger.

Entiteten har en generic:

Generic	Typ	Förval	Betydelse
TICK_COUNT	natural	50_000_000	Antal tick att räkna innan en timeout. En sekund vid DE0-CV:ns 50 MHz-klocka, avrundat: enligt Övning 7.3:s regel är det exakta värdet 49_999_999, och 20 ns fel på en sekund är inte värt den mindre läsbara konstanten.

och dessa portar:

Port	Riktn.	Typ	Betydelse
clock	in	std_logic	Systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, redan synkroniserad reset. Asynkron: den nollställer räknaren och timeout utan att någon klockflank är inblandad.
enable	in	std_logic	Timerns enable. Medan den är låg håller räknaren sitt värde, och fortsätter därifrån i stället för att börja om.
timeout	out	std_logic	Encykelspuls, som slår till var TICK_COUNT + 1 cykler medan timern är aktiverad.

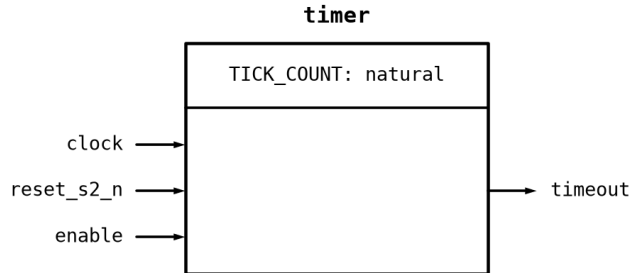
Arkitekturen ska:

- Deklarera den interna räknaren som `natural range 0 to TICK_COUNT`, så att registret dimensioneras av genericen snarare än lämnas obegränsat.
- Använda en enda process, känslig för `clock` och `reset_s2_n`, enligt mallen för klockad process från avsnitt 3.6.
- Nollställa både räknaren och timeout vid reset, utanför den klockade grenen.
- Driva timeout låg villkorslöst högst upp i den klockade grenen, och hög bara vid den flank som nollställer räknaren. Det mönstret, ett förval följt av en överskrivning, är vad som gör pulsen exakt en cykel bred snarare än två.

Svara sedan, i en mening var:

- `enable = '0'` måste **pausa** räknaren, inte nollställa den. Anta att du nollställde den i stället. Förutsäg vad referenstestbänken rapporterar, och vilken av de två egenskaperna ovan den mäter när den gör det. Kör den sedan och se. (Kapitel 8:s `fsm_led` hänger på pausen, vilket är varför testbänken bryr sig.)
- Räknaren räknar 0 till och med TICK_COUNT, så en period är TICK_COUNT + 1 cykler. Var i din kod är det där "+" faktiskt skrivet? Det står inte som en literal någonstans.

- Avsnitt 7.2:s handritade timeout är en kombinatorisk avkodning av räkningen; din är registrerad. Båda har samma period. Vilken cykel slår var och en till på, och varför spelar skillnaden roll för något nedströms som samplar timeout?



Självkontroll. Döp din entitet till `timer`, med genericen `TICK_COUNT (natural)`, ingångarna `clock`, `reset_s2_n`, `enable` och utgången `timeout`, deklarerade i den ordningen; dess testbänk ligger i `exercises/timer/`. Den kör tre hela perioder, så en timer som bara slår till en gång passerar inte, och den pausar mitt i räkningen för att bekräfta att räkningen fortsatte i stället för att börja om.

Spara den här filen. Övning 7.5 nedan återanvänder den, det gör `walking_led` i Övning 7.6 också, och likaså båda kapitel 8:s `capstones`.

Övning 7.5 blinker

VHDL

Skriv en entitet vid namn `blinker` som blinkar en lysdiod på och av en gång per sekund, genom att återanvända modulen `timer` du just skrev snarare än att räkna klockcykler själv.

Entiteten har en generic:

Generic	Typ	Förval	Betydelse
TICK_COUNT	natural	25_000_000	En halv sekund vid DE0-CV:ns 50 MHz-klocka. Exponera det som en generic i stället för att baka in antalet, så att en testbänk kan korta blinket; FPGA-bygget använder bara standardvärdet.

och dessa portar:

Port	Riktn.	Typ	Betydelse
clock	in	std_logic	Systemklocka.
reset_n	in	std_logic	Aktiv låg reset, rå från omvärlden. Asynkron: aktivera den och led slocknar omedelbart, utan att vänta på någon klockflank.
led	out	std_logic	Nollställd till '0' medan reset-signalen är aktiv, och lämnad där till den första timeouten efter att reset-signalen släppts.

Lägg märke till att den här entiteten tar den **råa** `reset_n` snarare än en redan synkroniserad `reset_s2_n`. Att synkronisera den är en del av övningen, och det är vad avsnitt 4.5 menar med "exakt en modul i en design är undantaget": här är den modulen blinker själv.

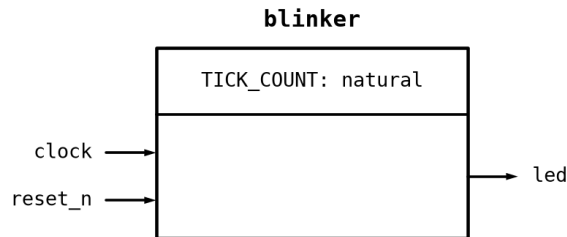
Arkitekturen ska:

- Instansiera din `reset_sync` från Övning 4.8, och använda dess utgång `reset_s2_n` som `reset` för allt inuti.
- Instansiera modulen `timer` från avsnitt 7.3, skicka din egen generic `TICK_COUNT` rakt igenom till timerens `TICK_COUNT`, och hålla dess `enable`-ingång hög.
- Växla led i en synkron process varje gång timerens `timeout`-puls slår till.

Implementera inte räknelogiken, eller synkroniseraren, på nytt själv. Återanvänd `timer` och `reset_sync` oförändrade, via en generic `map` och en port `map`, enligt kompositions-mönstret i avsnitt 7.4. Förklara varför en timerperiod på en **halv** sekund ger en blinkcykel på **en** sekund (en hel period av på och sedan av).

Svara sedan, i en mening var:

- Lysdioden måste vara släckt under reset, och testbänken kontrollerar att den slocknar utan någon klockflank däremellan. Vilken halva av mönstret aktivera-asynkront-släpp-synkront kontrollerar det, och vilken halva kontrollerar det inte?
- Din timer resettas av `reset_s2_n`, två flanker efter att `reset_n` släppts. Vad skulle gå fel om du matade den med den råa `reset_n` i stället? Namnge felet, inte bara regeln.



Självkontroll. Döp din entitet till `blinker`, med genericen `TICK_COUNT (natural)`, ingångarna `clock`, `reset_n` och utgången `led`, deklarerade i den ordningen; dess testbänk ligger i `exercises/blinker/`. **Kopiera in dina egna `reset_sync.vhd` och `timer.vhd` i den katalogen** innan du bygger; ingen av dem delas ut där, eftersom du skrivit båda:

```
cd lectures/L07/exercises/blinker
cp ../../L04/exercises/reset_sync/reset_sync.vhd .    # the two you wrote ↪
  ↳ yourself
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd timer.vhd blinker.vhd blinker_tb.vhd
```

Testbänken skriver över `TICK_COUNT` med ett litet värde och kontrollerar att lysdioden växlar om och om igen, ligger kvar exakt en timerperiod mellan växlingarna, och att den första växlingen kommer sent med din synkroniserades två flanker.

ÖVNINGSGRUPP 3

Den vandrande lysdioden

En enda tänd lysdiod som vandrar längs en rad, en position per timertick, startad och stoppad med en knapp. Den byggs live under passet och är beskriven i avsnitt 7.5.

Det här är utdelningen från de fyra senaste kapitlen, och den skriver nästan ingen egen logik: tre moduler, två av dem oförändrade från kapitel 4 och en skriven här, komponerade. Övning 7.6 bygger den, och Övning 7.7 och 7.8 ändrar det du byggt.

Övning 7.6 `walking_led`

VHDL

Skriv en entitet vid namn `walking_led`.

Entiteten har två generics:

Generic	Typ	Förval	Betydelse
<code>LED_COUNT</code>	<code>natural</code>	8	Hur många lysdioder biten vandrar längs. Raden är så här bred, och vandringen slår över efter så här många steg.
<code>TICK_COUNT</code>	<code>natural</code>	25_000_000	Skickas rakt igenom till din timer, så att biten flyttar en position per timerperiod: ett steg per halv sekund vid DE0-CV:ns 50 MHz-klocka.

och dessa portar:

Port	Riktn.	Typ	Betydelse
<code>clock</code>	in	<code>std_logic</code>	Systemklocka.
<code>reset_n</code>	in	<code>std_logic</code>	Aktiv låg reset, rå från omvärlden. Att synkronisera den är ditt jobb, precis som den var i blinker.
<code>button_n</code>	in	<code>std_logic</code>	Aktiv låg tryckknapp, rå. Varje tryck startar eller stoppar vandringen.
<code>led</code>	out	<code>std_logic_vector(LED_COUNT-1 downto 0)</code>	Lysdiodsraden. Exakt en bit är tänd åt gången; vid reset är det bit 0.

Arkitekturen komponerar tre moduler du redan skrivit, och lägger till två processer:

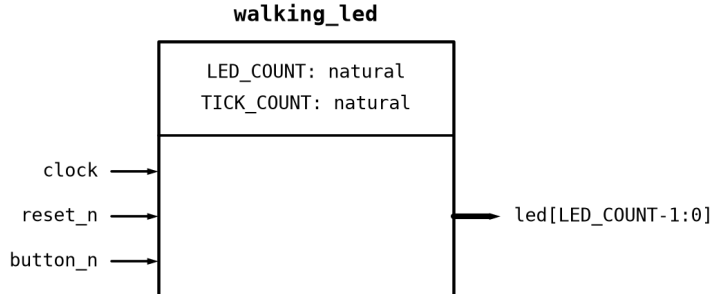
- Instansiera din **reset_sync**, och använd dess `reset_s2_n` som reset för allt annat i designen, de två instanserna nedan inräknade.
- Instansiera din **button_sync** med generic `map(1)`, för en encykelspuls per tryck. Dess knappportar är vektorer med `COUNT` bitar medan `button_n` här är en skalär, så överbrygga

bredden med en `std_logic_vector(0 downto 0)` med ett element i vardera riktningen. Avsnitt 7.5 visar det.

- Instansiera din **timer** med din egen `TICK_COUNT`, driv dess `enable` från skiftflaggan nedan och ta dess `timeout` som skiftticket.
- En process som växlar **shift_enable** vid varje puls från knappen, nollställd till '0' vid reset. Vandringen börjar därför stoppad.
- En process som håller det **skiftregister** som driver `led`: vid reset, ladda en enda tänd bit på position 0 och nollställ resten; vid varje timeout från timern, rotera ett steg mot MSB och låt den översta biten slå över tillbaka in i bit 0. Det är ett PISO-register (avsnitt 6.4) med sin egen seriella utgång kopplad tillbaka till sin seriella ingång, vilket är det som gör en engångsutskiftning till en oändlig vandring.

Svara sedan, i en mening var:

- `shift_enable` driver timerns `enable`, så att stoppa vandringen stoppar timern också. Övning 7.4 fick dig att visa att `enable` **pausar** räkningen snarare än nollställer den. Vad ser en betraktare på kortet, vid det tryck som startar om vandringen, om din timer nollställer i stället?
- `button_sync` instansieras med `COUNT = 1` här och med `COUNT = 2` i kapitel 8:s `fsm_led`, ur samma fil utan någon ändring. Vad hade du behövt skriva i stället om bredden var fastlagd i modulen snarare än inskickad som en generic?



Självkontroll. Döp din entitet till `walking_led`, med generics `LED_COUNT` och `TICK_COUNT` (båda `natural`), ingångarna `clock`, `reset_n`, `button_n` och utgången `led`, deklarerade i den ordningen; dess testbänk ligger i `exercises/walking_led/`. Den skriver över båda generics (`LED_COUNT = 4`, `TICK_COUNT = 3`) så att ett helt varv tar en handfull klockcykler snarare än två sekunder, och den kontrollerar resetmönstret, att ingenting rör sig före det första trycket, och att antalet skiftningar över ett fast fönster stämmer med den takt timern sätter. Kopiera först in de tre moduler den komponerar; kommandot står i avsnitt 7.5.

Övning 7.7 Att ändra takt och bredd

SIMULERING

Gör följande ändringar en i taget, och kör om testbänken efter var och en. Del **a)** och **b)** ändrar bara testbänkens konstanter — att de inte kräver någon ändring alls i din `walking_led.vhd` är hela poängen med dem. Del **c)** är den som rör modulen.

a) Ändra vandringshastigheten:

- Ändra konstanten `TICK_COUNT` i **testbänken** från 3 till 6.
- Den ska fortfarande passera, den här gången för att testbänken anpassar sig med dig: dess observationsfönster är fasta $\text{PULSES} * 6$ cykler, och den härleder det antal skiftningar den förväntar sig ur `TICK_COUNT`, en skiftning per $\text{TICK_COUNT} + 1$ cykler.
- Hitta nu var den anpassningen slutar betyda något. Fortsätt höja `TICK_COUNT` förbi fönstrets längd och räkna ut vad det förväntade antalet blir, och vad kontrollen då faktiskt bevisar. Ett test som passerar av fel skäl är värt att kunna känna igen.
- Förklara sedan varför ett ändrat tickantal inte kan göra sönder *designen*, bara sakta ner den.

b) Ändra bredden: ändra testbänkens konstant `LED_COUNT` från 4 till 8. Den ska fortfarande passera, utan någon ändring alls i din `walking_led.vhd`. Förklara hur modulen hade behövt se ut för att det här skulle ha krävt en ändring.

c) Ladda två tända bitar på var sin sida av registret vid reset (t.ex. "10001000" för `LED_COUNT = 8`), och låt dem vandra i takt:

- Peka ut den enda rad i din `walking_led.vhd` du behöver ändra.
- **Förutsäg** vad referenstestbänken kommer att rapportera, innan du kör någonting.
- Kör den sedan och se. Förklara resultatet: är *designen* fel, eller kontrollerar testbänken något som inte längre är specifikationen?

Tips: Både `LED_COUNT` och `TICK_COUNT` är generics, så del **a)** och **b)** kräver ingen ändring alls i modulens kropp. Det är hela poängen med en generic: en modul, många storlekar, inga ändringar.

Övning 7.8 En andra knapp som vänder riktningen

VHDL

Den nuvarande *designen* vandrar bara åt ett håll (mot MSB, där den översta biten slår över tillbaka till botten). Ändra din `walking_led.vhd` så att en **andra** knapp vänder vandringsriktningen.

a) Lägg till en andra, synkroniserad knappgång: dra den genom samma `button_sync`-instans, och skicka 2 till dess generic `COUNT` (`generic map(2)`, `positionsvis`, som överallt annars i den här boken).

b) Lägg till en signal `direction`, och växla den vid den nya knappens flank, precis så som `shift_enable` växlas av den första knappen.

c) Gör rotationen i `SHIFT_PROCESS` beroende av `direction`. När `direction = '0'`, rotera mot MSB (bokens konvention från avsnitt 6.2, och det du skrev i Övning 7.6):

```
shift_reg <= shift_reg(LED_COUNT-2 downto 0) & shift_reg(LED_COUNT-1);
```

När `direction = '1'`, rotera åt andra hållet, mot bit 0:

```
shift_reg <= shift_reg(0) & shift_reg(LED_COUNT-1 downto 1);
```

Det här är det enda stället i boken där ett skiftregister medvetet går emot konventionen: att vända riktningen är hela poängen med övningen. Lägg märke till att det andra uttrycket är precis den spegelbild avsnitt 6.2 varnar dig för att hålla utkik efter i andras kod.

d) Din entitets portar har nu ändrats: `button_n` är en tvåbitarsvektor snarare än en enda bit.

- Förklara varför `walking_led_tb.vhd` inte längre går att köra mot din version, och exakt vad GHDL kommer att klaga på när du försöker.
- Det här är den vanliga kostnaden för att ändra ett gränssnitt, och den är värd att känna en gång: en testbänk är skriven mot en bestämd entitet, och att bredda en port gör sönder varje instansiering av den, också de du inte skrivit själv.
- Beskriv, i löpande text, den följd av tryck du skulle använda för att övertyga dig själv om att designen fungerar, och vad du skulle förvänta dig att lysdioderna gör vid varje steg. Den följden är precis vad en testbänk för den nya entiteten skulle behöva driva.

Tillståndsmaskiner

Först konstruerade för hand, sedan uttryckta i språket.

I DET HÄR KAPITLET

- Ändliga tillståndsmaskiner: tillstånd, övergångar, ingångar och utgångar.
- Att konstruera en Mooremaskin för hand: tillståndsdigram, tillståndstabell, Karnaughhärledd logik.
- Tillståndskodning som ett designbeslut, och varför den genomarbetade maskinen använder Graykod.
- Samma maskin i VHDL: en uppräkningsstyp och case-mönstret för övergångar.
- Varför Moore är standardvalet här och i CAN-projektet.
- fsm_led: knappar som växlar lysdioden mellan släckt, blinkande och tänd.

Det här är det sista kapitlet i bokens första del, och dess byggblock är det sista som saknas. Utdelningen ligger i att se de två halvorna mot varandra: varje rad du härleder för hand i tillståndstabellen i avsnitt 8.2 blir en gren i ett case i avsnitt 8.4, och att se den överensstämmelsen är värt mer än något av stegen för sig.

Mealymaskiner (avsnitt 8.5) läses i stället för att presenteras, och det spelar större roll än det kan se ut. Allt som presenteras är Moore, och Moore är standardvalet både här och i CAN-projektet. Avsnitt 8.5 är där du får veta vad alternativet kostar och köper, vilket är enda sättet för det standardvalet att bli ett val i stället för en vana.

Repetition: Konstruktionsgången för hand, från tillståndsdigram via tillståndstabell och Karnaughdiagram till grindnät, och skillnaden mellan Moore och Mealy står i avsnitt 6.4–6.7 i *Digital Electronics*.

8.1 Vad en ändlig tillståndsmaskin är

En ändlig tillståndsmaskin (FSM) är en krets vars beteende inte bara beror på dess aktuella ingångar utan på en ändlig mängd *historik*, sammanfattad i ett enda värde som kallas dess **tillstånd**. I varje ögonblick befinner sig maskinen i exakt ett tillstånd; vid varje klockflank

kan den gå vidare till ett nytt, bestämt av dess aktuella tillstånd och ingångar; och dess utgång beräknas ur det tillståndet.

Inget av det här är ny hårdvara. En tillståndsmaskin kombinerar två saker du redan kan: ett **register** (kapitel 3) som håller det aktuella tillståndet, en vippa per tillståndsbit, och **kombinatorik** som beräknar nästa tillstånd och utgången ur det aktuella tillståndet och ingångarna.

Det nya är disciplinen. I stället för ad hoc-logik som matar en ad hoc-uppsättning vippor arbetar du i en fast ordning: rita först ett **tillståndsdiaqram**, några namngivna tillstånd med märkta övergångar mellan sig, och härled tillståndstabellen först när det är rätt, sedan logiken för nästa tillstånd, sedan grindnätet eller VHDL-koden.

Varje synkron krets hittills är tekniskt sett en trivial tillståndsmaskin: en räknares tillstånd är dess räkning, ett skiftregisters är dess bitmönster. Det som skiljer en tillståndsmaskin är att tillståndet varken är en räkning eller ett skiftat mönster utan en godtycklig etikett (STATE_OFF, STATE_BLINK, ...) vars innebörd du väljer och vars nästa värde får bero på ingången hur du vill.

Det finns två varianter, och skillnaden avgör vad en utgång får titta på:

- **Mooremaskiner:** utgången beror *bara* på det aktuella tillståndet. Det är den sorten som konstrueras för hand här.
- **Mealymaskiner:** utgången beror på det aktuella tillståndet *och* den aktuella ingången. Avsnitt 8.5 går kort igenom dem, bara i VHDL, så att du känner igen en när du möter den och kan skillnaden på en klockcykel.

8.2 Att konstruera en Mooremaskin för hand

Samma process som för varje grindnät i kapitel 1 och 2, tillämpad på en krets med minne. Det genomarbetade exemplet är en liten, sluten Mooremaskin med fyra tillstånd: en cirkulär räknare genom en Graykodssekvens, som stegas ett steg per knapptryck.

Specifikation

- Fyra tillstånd, vart och ett representerat av två vippor $\{Q_1, Q_2\}$:

Tillstånd	$\{Q_1, Q_2\}$
STATE_0	00
STATE_1	01
STATE_2	11
STATE_3	10

- En systemklocka clock, och en aktiv låg reset reset_n som återför maskinen till STATE_0.

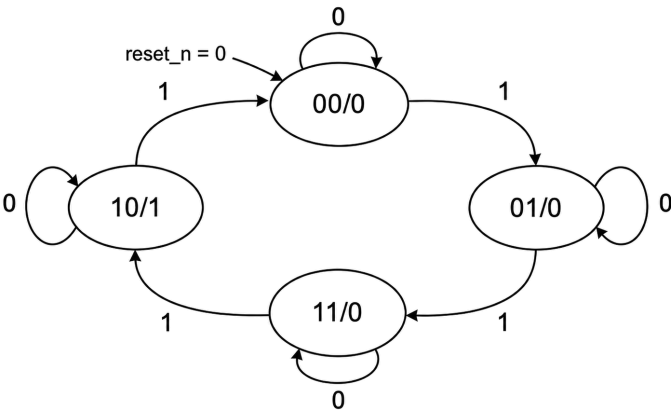
- En aktiv låg tryckknapp `button_n`, ur vilken en synkroniserad, flankdetekterad puls X härleds på exakt det sätt kapitel 4:s dubbelvippsynkroniserare gör. X är hög i exakt en klockcykel vid en fallande flank på `button_n`.
- En utgång Y , kopplad till en lysdiod, hög bara i `STATE_3`.
- Maskinen är **sluten**: tillståndet efter `STATE_3` är `STATE_0`.
- Det är en **Moore**-maskin: Y beror bara på $\{Q_1, Q_2\}$, aldrig på X .

Kodningen är en Graykod ($00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$), där tillstånd som följer på varandra skiljer sig i exakt en bit. Det är ett medvetet val, inte en slump:

- Med en vanlig binärräkning ($00 \rightarrow 01 \rightarrow 10 \rightarrow 11$) vänder en övergång som $01 \rightarrow 10$ två bitar på en gång, och de två vipporna kommer inte att slå om vid *exakt* samma ögonblick.
- Den skevheten är harmlös för maskinen själv. Båda vipporna uppdateras vid samma klockflank, och ingenting samplar tillståndsregistret däremellan, så maskinen agerar aldrig på mellanvärdet. Det är samma argument som avsnitt 5.4:s kritiska väg: så länge allt hinner stabilisera sig före nästa flank spelar det ingen roll vad som händer under tiden.
- Vad skevheten *kan* störa är allt som läser tillståndet kombinatoriskt. En utgång som avkodas direkt ur tillståndsbitarna, som Y nedan, kan glitcha kort medan de är skeva. Detsamma gäller ett värde som samplas av en *annan* klockdomän, vilket är fallet avsnitt 4.3 varnade för.
- En Graykod tar bort båda, eftersom bara en bit någonsin ändras per övergång. Därför spelar den störst roll för en räknare som korsar klockdomäner, och minst för en maskin vars utgång är registrerad.

Kodning är i allmänhet ett designbeslut, och det här är det enda stället i boken där du fattar det för hand. Binärt använder färrest vippor; Gray minimerar antalet bitar som ändras per övergång; **one-hot** lägger en vippa per tillstånd så att logiken för nästa tillstånd blir en enda bred OR-grind per tillstånd, vilket ofta är snabbast på en FPGA där vippor finns i överflöd. Så snart du skriver maskinen i VHDL som en uppräknad typ (avsnitt 8.4) lämnar du beslutet till syntesverktyget, som väljer en kodning åt dig och omkodar fritt om du inte säger något annat.

Tillståndsdigram



State label format: Q1Q2/Y
Example: 00/0 means Q1Q2 = 00 (current state in binary) and Y = 0 (output).
Arrow labels 0 and 1 denote X, the input that changes the state.

Figur 8.1 Tillståndsdigram för Mooremaskinen med fyra tillstånd och Graykod.

Varje tillståndsbubbla visar det aktuella tillståndet och utgången som Q1Q2/Y, och varje pil är märkt med den ingång X som utlöser den. Varje tillstånd går tillbaka till sig självt så länge X=0, och en synkroniserad puls från knappen (X=1) stegar vidare till nästa tillstånd och slår runt från STATE_3 tillbaka till STATE_0.

Tillståndstabell

Varje kombination av aktuellt tillstånd och ingång, med nästa tillstånd $\{Q_1^+, Q_2^+\}$ och utgången Y:

Q_1	Q_2	X	Q_1^+	Q_2^+	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	0
1	1	1	1	0	0

Karnaughhärledd logik

Om Q_1^+ och Q_2^+ var för sig behandlas som en boolesk funktion av $\{Q_1, Q_2, X\}$ och körs genom ett Karnaughdiagram, samma teknik som i kapitel 2:

$$Q_1^+ = Q_1X' + Q_2X \quad Q_2^+ = Q_2X' + Q_1'X$$

Lägg märke till formen på båda:

- var och en är "håll kvar när X är 0, ta något från den andra biten när X är 1".
- hållhalvorna, Q_1X' och Q_2X' , är verkligen spegelbilder av varandra.
- stegningshalvorna är Q_2X och $Q_1'X$, och komplementet på den andra är inte ett skrivfel: det är det som gör sekvensen till en Graykod i stället för en enkel rotation.

Utgången Y förenklas ytterligare, direkt ur tillståndstabellen, utan något beroende av X alls, vilket bekräftar att det här är en giltig Mooremaskin:

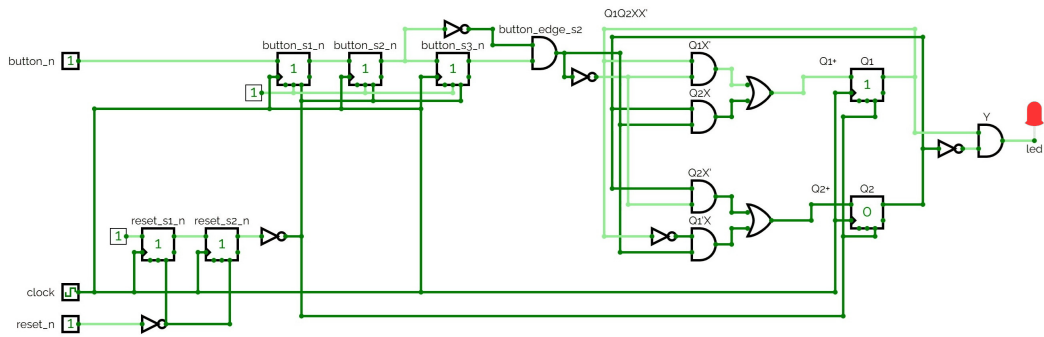
$$Y = Q_1Q_2'$$

Att realisera den som ett grindnät

$\{Q_1, Q_2\}$ är var sin D-vippa, precis som kapitel 3:s register: Q_1^+ och Q_2^+ matar D-ingångarna och Q_1, Q_2 läses tillbaka från Q-utgångarna. Y är en enda AND-grind med två ingångar där den ena är inverterad, och den läser det aktuella tillståndet. Det är hela maskinen: två vippor och en handfull grindar som beräknar Q_1^+, Q_2^+ och Y ur $\{Q_1, Q_2, X\}$.

8.3 Från grindnät till CircuitVerse

Innan något av det här skrivs i VHDL, realisera det för hand i CircuitVerse, precis som varje grindnät i kapitel 1: dubbelvippsynkronisera `button_n` och `reset_n`, flankdetektera den synkroniserade knappen för att producera X , koppla Q_1^+ och Q_2^+ till ett par D-vippor, och koppla Y rakt in i lysdioden.



Figur 8.2 Grindnät som realiserar Mooremaskinen med fyra tillstånd, inklusive dubbelvipsynkronisering och flankdetektering för button_n.

Läst från vänster till höger:

- button_n går genom tre synkroniseringsvippor (button_s1_n, button_s2_n, button_s3_n).
- En AND-grind härleder button_edge_s2 (den här kretsens X) ur andra och tredje steget.
- De fyra lodräta ledningarna märkta Q1Q2XX' bär tillståndsbitarna och flankpulsen vidare till grindarna till höger, som implementerar avsnitt 8.2:s ekvationer för Q_1^+ och Q_2^+ .
- De två märkta vipporna lagrar Q_1 och Q_2 och matar de avslutande AND- och NOT-grindar som producerar Y, kopplad till led.
- reset_n synkroniseras av sitt eget vippapar nere till vänster, precis som varje annan synkroniserad reset i den här boken.

Öppna den körbara kretsen själv: [lectures/L08/appendix/images/fsm_gate_network.cv](#). Stega den för hand, ett knapptryck i taget, och bekräfta att den besöker STATE_0 → STATE_1 → STATE_2 → STATE_3 → STATE_0 i ordning, med lysdioden tänd bara i STATE_3.

8.4 Samma maskin i VHDL

Det var sista gången i den här boken du konstruerar en maskin för hand. Allt härifrån skriver den direkt i VHDL, vilket är värt att ha förtjänat snarare än att ha börjat med: syntesverktyget gör exakt det du just gjorde på papper, och när en tillståndsmaskin betar sig fel är det diagrammet du felsöker, inte VHDL-koden.

Översättningen ersätter den explicita tillståndstabellen och Karnaughekvationerna med en **uppräknad typ** som namnger varje tillstånd, en **case-sats** som beskriver varje tillstånds övergångar, och ett syntesverktyg som lämnas att lista ut vippkodningen och logiken på grindnivå.

Den uppräknade tillståndstypen

Anta en maskin med tre tillstånd som styr en lysdiod: STATE_OFF, STATE_BLINK (blinkar i en fast takt) och STATE_ON. VHDL deklarerar exakt de tre värdena som en ny typ:

```
type state_t is (STATE_OFF, STATE_BLINK, STATE_ON);
signal state: state_t;
...
state <= STATE_ON;
```

Mönstret med case-satsen

Maskinen är nästan alltid en synkron process som uppdaterar state vid stigande flank via ett case, en gren per tillstånd, med en asynkron reset som återför den till ett känt starttillstånd. Antag att to_next_state och to_prev_state är synkroniserade, flankdetekterade pulser producerade på exakt samma sätt som X i avsnitt 8.2 och 8.3:

```
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        -- On reset, go to STATE_OFF.
        state <= STATE_OFF;
    elsif (rising_edge(clock)) then
        case (state) is
            when STATE_OFF =>
                if (to_next_state = '1') then
                    state <= STATE_BLINK;
                elsif (to_prev_state = '1') then
                    state <= STATE_ON;
                end if;
            -- STATE_BLINK and STATE_ON follow the same shape.
            when others =>
                state <= STATE_OFF;
        end case;
    end if;
end process;
```

case (state) is ... end case; måste täcka varje värde state_t kan anta, och when others håller det uttömmande om du lägger till ett tillstånd senare och glömmer en gren.

Vad when others *inte* är, är ett skyddsnät mot ett korrumperat tillståndsregister. Varje värde state_t kan anta är redan namngivet ovanför den, så grenen är onåbar i källkoden och syntesen optimerar normalt bort den i stället för att bygga återhämtningslogik. När Quartus väl har omkodat maskinen är de flesta bitmönster otillåtna, och att ta sig ur ett av dem kräver Quartus inställning Safe State Machine snarare än något du kan skriva i VHDL.

Genomarbetat exempel: fsm_led

fsm_led.vhd implementerar exakt den maskinen med tre tillstånd, utökad till två knappar: button_n(0) stegar framåt, button_n(1) går bakåt, tillstånden är slutna i en loop, och STATE_BLINK växlar lysdioden var 100:e ms.

Tre delblock återanvänds, alla moduler du skrivit själv:

- reset_sync: kapitel 4:s resetsynkroniserare, oförändrad (Övning 4.8).
- button_sync: kapitel 4:s knappsynkroniserare, också oförändrad, men instansierad med generic map(2) så att den hanterar båda knapparna på en gång och producerar en pulsvektor på två bitar. Det är utdelningen från den generic:en: filen är identisk med den kapitel 4 och 7 instansierar med 1 (Övning 4.9).
- timer: kapitel 7:s timer, oförändrad (Övning 7.4). fsm_led deklarerar en egen generic TIMER_TICK_COUNT och skickar den rakt vidare, med förvalet 5_000_000 för 100 ms vid 50 MHz, och aktiverar timern bara i STATE_BLINK.

Ingen av de tre är utdelad i fsm_led/: kopiera in dina egna innan du bygger exemplet. Vid det här laget i boken är det fyra kapitels egna moduler komponerade till en enda design, vilket är det som är värt att lägga märke till.

```
lectures/L08/fsm_led/fsm_led.vhd
-----
-- Moore FSM cycling an LED through OFF / BLINK / ON via two push buttons.
-- (Moore: the LED depends only on the state, never directly on the buttons;
-- contrast seq_detect_mealy. See Appendix A.5.)
--
-- Generics:
--   - TIMER_TICK_COUNT: The timer's tick count, so the interval between LED
--     toggles in STATE_BLINK (default = 5 000 000, i.e. about 100 ms at
--     50 MHz; override to shorten it in a testbench).
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset.
--   - button_n: Active-low buttons; (0) = next state, (1) = previous.
--
-- Outputs:
--   - led: The controlled LED.
-----
library ieee;
use ieee.std_logic_1164.all;

entity fsm_led is
    generic(TIMER_TICK_COUNT: natural := 5_000_000);
```

```

    port(clock, reset_n: in std_logic;
          button_n      : in std_logic_vector(1 downto 0);
          led            : out std_logic);
end entity;

architecture behaviour of fsm_led is
-- Enumeration of states.
type state_t is (STATE_OFF, STATE_BLINK, STATE_ON);

-- Synchronized input signals after two-flip-flop metastability protection.
signal reset_s2_n      : std_logic;
signal button_edge_s2: std_logic_vector(1 downto 0);

-- Timer signals.
signal timer_enable, timer_timeout: std_logic;

-- Internal LED state.
signal led_s: std_logic;

-- Current state.
signal state: state_t;

-- Signals indicating whether to change state.
signal to_prev_state, to_next_state: std_logic;
begin
-- Assign the internal LED state to the output port.
led <= led_s;

-- Enable the timer in STATE_BLINK only.
timer_enable <= '1' when STATE_BLINK = state else '0';

-- Indicate a change to the previous state on a falling edge of button_n(1).
-- Each direction also requires the other button's edge to be '0', so two
-- presses landing on the same clock cycle move the machine nowhere.
to_prev_state <= '1' when button_edge_s2(1) = '1' and button_edge_s2(0) = '0'
↳ else '0';

-- Indicate a change to the next state on a falling edge of button_n(0).
to_next_state <= '1' when button_edge_s2(0) = '1' and button_edge_s2(1) = '0'
↳ else '0';

-- Synchronize the asynchronous reset (Appendix A.4, reusing L04's reset_sync)↳
↳
reset_sync1: entity work.reset_sync

```

```

    port map(clock, reset_n, reset_s2_n);

-- Synchronize both buttons and pulse on each press (Appendix A.4, reusing
-- L04's button_sync at COUNT = 2: the same file, a wider vector).
button_sync1: entity work.button_sync
    generic map(2)
    port map(clock, reset_s2_n, button_n, button_edge_s2);

-- Instantiate the 100 ms timer (Appendix A.4, reusing L07's timer unchanged).
timer1: entity work.timer
    generic map(TIMER_TICK_COUNT)
    port map(clock, reset_s2_n, timer_enable, timer_timeout);

-- State-transition process:
--   - On reset: reset the current state to STATE_OFF.
--   - On rising clock edge: update the current state when a synchronized,
--     edge-detected button press is seen.
STATE_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        state <= STATE_OFF;
    elsif (rising_edge(clock)) then
        case (state) is
            when STATE_OFF =>
                if (to_next_state = '1') then
                    state <= STATE_BLINK;
                elsif (to_prev_state = '1') then
                    state <= STATE_ON;
                end if;
            when STATE_BLINK =>
                if (to_next_state = '1') then
                    state <= STATE_ON;
                elsif (to_prev_state = '1') then
                    state <= STATE_OFF;
                end if;
            when STATE_ON =>
                if (to_next_state = '1') then
                    state <= STATE_OFF;
                elsif (to_prev_state = '1') then
                    state <= STATE_BLINK;
                end if;
            when others =>
                state <= STATE_OFF;
        end case;
    end if;
end process;

```



```

        end if;
    end process;

    -- LED (Moore output) process, depending only on the current state:
    --   - On reset: disable the LED.
    --   - On rising clock edge, based on the current state:
    --       - STATE_OFF : disable the LED.
    --       - STATE_BLINK: toggle the LED on every timer timeout.
    --       - STATE_ON  : enable the LED.
    LED_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            led_s <= '0';
        elsif (rising_edge(clock)) then
            case (state) is
                when STATE_OFF =>
                    led_s <= '0';
                when STATE_BLINK =>
                    if (timer_timeout = '1') then
                        led_s <= not led_s;
                    end if;
                when STATE_ON =>
                    led_s <= '1';
                when others =>
                    led_s <= '0';
            end case;
        end if;
    end process;
end architecture;

```

Två synkrona processer implementerar maskinen:

- STATE_PROCESS är övergångslogiken med case ovan, driven av to_next_state och to_prev_state, som härleds kombinatoriskt ur button_edge_s2. Var och en kräver att den *andra* knappens puls är '0' i samma cykel, så att trycka ned båda så nära i tid att deras pulser hamnar på samma klockcykel flyttar maskinen ingenstans i stället för att godtyckligt utse en vinnare. En tvetydig ingång förtjänar ett definierat svar, och "gör ingenting" är det som inte kan överraska dig.
- LED_PROCESS driver led_s enbart ur det aktuella tillståndet: släckt i STATE_OFF, tänd i STATE_ON, växlande vid varje timeout från timern i STATE_BLINK. Eftersom den aldrig läser button_n eller button_edge_s2 direkt, bara state, är det här en äkta Mooremaskin: lysdioden reagerar på ett tryck bara indirekt, genom det tillstånd trycket orsakade.

8.5 Mealymaskiner: skillnaden på en klockcykel

Allt hittills har varit en **Moore**-maskin, med flit: Moore är det säkrare standardvalet och den form du konstruerar för hand. En **Mealy**-maskin luckrar upp regeln som definierar en Moore-maskin: utgången får bero på det aktuella tillståndet *och* den aktuella ingången, beräknad kombinatoriskt. Den lagras inte i ett register, och den ändras inom samma klockcykel som ingången gör.

Beteendet: detektera två 1-bitar i följd

En seriell ingång *din* bär en bit per klockcykel, och *y* ska indikera att de två senaste bitarna båda var 1, där överlappande träffar räknas, så 111 rapporterar en träff två gånger.

Som Mealymaskin behöver det här bara två tillstånd, eftersom utgången kan reagera på ingången *innan* den syns i nästa tillstånd: STATE_IDLE (föregående bit var 0, eller det här är den första) och STATE_ONE (föregående bit var 1).

Tillstånd	din	Nästa tillstånd	y
STATE_IDLE	0	STATE_IDLE	0
STATE_IDLE	1	STATE_ONE	0
STATE_ONE	0	STATE_IDLE	0
STATE_ONE	1	STATE_ONE	1

Tillståndsprocessen är avsnitt 8.4:s case-mönster; bara utgången skiljer sig:

```
lectures/L08/seq_detect_mealy/seq_detect_mealy.vhd

-----
-- Mealy FSM detecting two consecutive '1's on serial input din (overlapping
-- matches allowed, e.g. "111" reports two). y is combinational in (state, din),
-- so it asserts the same cycle the second '1' arrives - one cycle earlier than
-- the Moore equivalent. See Appendix A.5.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset, from a push button.
--   - din: Synchronous serial data (already clocked by clock, so it is NOT run
--         through a synchronizer - doing so would shift the sequence being detected)
--
--
-- Outputs:
--   - y: Asserted during the cycle the second '1' of a "11" is seen on din.
-----

library ieee;
use ieee.std_logic_1164.all;
```

```

entity seq_detect_mealy is
    port(clock, reset_n: in std_logic;
          din          : in std_logic;
          y            : out std_logic);
end entity;

architecture behaviour of seq_detect_mealy is
    -- Enumeration of states. Only two are needed: this is the state count the
    -- Mealy design saves relative to the three-state Moore equivalent.
    type state_t is (STATE_IDLE, STATE_ONE);

    -- Synchronized reset signal after two-flip-flop metastability protection.
    signal reset_s2_n: std_logic;

    -- Current state.
    signal state: state_t;
begin
    -- Instantiate reset_sync (Appendix A.5 / L04) to synchronize reset_n. This
    -- example has no push button of its own, so the button synchronizer is
    -- simply not instantiated.
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    -- State-transition process:
    --   - On reset: reset the current state to STATE_IDLE.
    --   - On rising clock edge: move to STATE_ONE whenever din = '1', back to
    --     STATE_IDLE whenever din = '0'. Note this process only ever reads din
    --     to decide the *next* state; it never assigns y.
    STATE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            state <= STATE_IDLE;
        elsif (rising_edge(clock)) then
            case (state) is
                when STATE_IDLE =>
                    if (din = '1') then
                        state <= STATE_ONE;
                    else
                        state <= STATE_IDLE;
                    end if;
                when STATE_ONE =>
                    if (din = '1') then
                        state <= STATE_ONE;
            end case;
        end if;
    end process;

```

```

        else
            state <= STATE_IDLE;
        end if;
        when others =>
            state <= STATE_IDLE;
        end case;
    end if;
end process;

-- Mealy output: a combinational (unregistered) function of the current
-- state AND the current input, assigned outside any clocked process.
-- Asserted the same cycle the second '1' arrives, one cycle earlier than
-- a Moore machine could report the same event.
y <= '1' when (state = STATE_ONE and din = '1') else '0';
end architecture;
```

Tilldelningen av utgången är en enda konkurrent sats, helt utanför varje klockad process:

```
y <= '1' when (state = STATE_ONE and din = '1') else '0';
```

Det är en **villkorlig signaltilldelning**, och det är den sista biten VHDL-syntax den här boken introducerar. Den är en konkurrent sats, så den hör hemma i arkitekturröparen snarare än inuti en process, och den beskriver en multiplexer: välj det första värde vars villkor gäller, annars värdet efter else. Det är samma hårdvara du skulle få ur ett if/else inuti en kombinatorisk process, skriven på en rad i stället för fem, och den är värd att ha eftersom en tilldelning av utgången på en rad gör det uppenbart vid en blick vad utgången beror på.

```

-- These two describe exactly the same multiplexer.
x <= a when sel = '1' else b;

process(sel, a, b) is
begin
    if (sel = '1') then
        x <= a;
    else
        x <= b;
    end if;
end process;
```

Ge den alltid ett else. Utelämnar du ett har du beskrivit en signal som behåller sitt gamla värde när inget villkor gäller, vilket är ett lås, precis som i avsnitt 5.5. fsm_led använder den här formen tre gånger, och Övning 8.4 ber dig använda den.

Den enda raden är hela skillnaden. Den läser din, alltså är den Mealy. Flytta samma villkor in i den klockade processen, eller ta bort din ur det, så har du en Mooremaskin i stället.

Vad den kostar och vad den köper

Mooreversionen av den här detektorn behöver ett **tredje** tillstånd, `M_TWO`, som finns enbart för att i en cykel minnas att träffen inträffade, så att utgången har något att läsa som inte beror på ingången. Mata båda maskinerna med 1, 1, 0, 1, 1, 1:

Cykel	din	Mealytillstånd (före)	Mealy y	Mooretillstånd (före)	Moore y
1	1	STATE_IDLE	0	M_IDLE	0
2	1	STATE_ONE	1	M_ONE	0
3	0	STATE_ONE	0	M_TWO	1
4	1	STATE_IDLE	0	M_IDLE	0
5	1	STATE_ONE	1	M_ONE	0
6	1	STATE_ONE	1	M_TWO	1

Moorens y-kolumn är exakt Mealys y-kolumn förskjuten en rad nedåt. Det är hela avvägningen: Mealy rapporterar träffen *samma* cykel som den andra 1:an kommer där Moore tar en till, och Mealy behöver 2 tillstånd där Moore behöver 3.

Vilken du ska välja

Moore, nästan alltid, och det är den resten av boken använder, grupprojektet inräknat:

- dess utgång beror inte på någon primär ingång, så den kan inte följa en brusig sådan.
- en Mealyutgång kan i princip glitcha kombinatoriskt om dess ingångar inte redan är rena.
- observera att "Moore" i sig inte betyder glitchfri. En utgång som avkodas kombinatoriskt ur flera tillståndsbitar, som avsnitt 8.2:s $Y = Q_1 Q'_2$, kan fortfarande glitcha medan de bitarna är skeva vid en övergång. Det som gör en utgång glitchfri är att *registrera* den, så som fsm_led:s LED_PROCESS gör, och det är vad du vill ha på allt som driver en ledning ut från kretsen. En seriell tx-ledning som avkodas direkt ur tillståndsbitarna kommer att sända ut falska flanker.
- en maskin vars utgång beror på färre saker är en maskin med färre sätt att bli fel på.

Ta till Mealy bara när du specifikt behöver den lägre latensen eller det mindre antalet tillstånd, och var medveten om att du då byter en synkron utgång mot en kombinatorisk.

8.6 På kortet

fsm_led syntetiseras och körs precis som varje VHDL-design gjort sedan kapitel 1: skapa ett Quartus Prime Lite-projekt med DE0-CV:s 5CEBA4F23C7N som målkrets, lägg till .vhd-filerna

för exemplet och dess delblock, kompilera, tilldela varje port en fysisk pinne i Pin Planner, och kompilera sedan om och programmera kortet.

För fsm_led, tilldela clock pinnen för 50 MHz-oscillatorn, reset_n och button_n(1 downto 0) tre tryckknappar, och led en lysdiod på kortet. Bekräfta sedan att upprepade tryck på "nästa" får lysdioden att cykla släckt → blinkande → tänd → släckt, att "föregående" går åt andra hållet, och att reset återför den till släckt.

seq_detect_mealy demonstreras med flit **inte** på kortet. Den konsumerar en bit per klockcykel, och en omkopplare som slås om för hand håller sitt värde i miljontals cykler, så det enda en lysdiod kan visa är det degenererade fallet: din hållen hög, y hög från andra cykeln och framåt. Skillnaden på en klockcykel som hela avsnitt 8.5 handlar om sker alldeles för fort för att se. Kör dess testbänk i stället, vilket är den allmänna lärdomen snarare än ett undantag:

```
cd lectures/L08/seq_detect_mealy
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the one you wrote in L04
ghdl -a --std=93 reset_sync.vhd seq_detect_mealy.vhd seq_detect_mealy_tb.vhd
ghdl -e --std=93 seq_detect_mealy_tb
ghdl -r --std=93 seq_detect_mealy_tb --assert-level=error --stop-time=10ms
```

Exakta pinnummer beror på vilka knappar och lysdioder på DE0-CV du väljer, tilldelade på samma sätt som för varje design sedan kapitel 1. Slå upp DE0-CV:s egen dokumentation för dess fysiska pinnkonfiguration om du behöver bekräfta vilken headerpinne som motsvarar vilken märkt knapp, omkopplare eller lysdiod.

8.7 Sammanfattning

Det här ska du kunna nu

- Förklara vad en ändlig tillståndsmaskin är, och peka ut dess tillstånd, övergångar, ingångar och utgångar i en given krets.
- Konstruera en Mooremaskin för hand, från specifikation via tillståndsdigram och tillståndstabell till Karnaughhärledda ekvationer, och bygga resultatet i CircuitVerse.
- Modellera samma maskin i VHDL med en uppräknad typ state_t och en case-process.
- Känna igen en Mealymaskin på sekunden och säga vad dess utgång kostar och köper, en klockcykels latens mot ett extra tillstånd, och varför Moore är standardvalet här och i CAN-projektet.
- Komponera en tillståndsmaskin med tidigare byggda subkomponenter i stället för att skriva om dem. I capstonen betyder det tre tidigare kapitels moduler runt en maskin som är den enda nya logiken, med skift- och räknaridiom från ytterligare ett.

Frågor att testa dig själv med

1. Vad gör en krets till en tillståndsmaskin, snarare än bara ett register med lite logik omkring?
2. Vad är X exakt i den handkonstruerade maskinen, och varför måste det vara en puls på en klockcykel snarare än knappens nivå?
3. Varför är den maskinens tillståndskodning en Graykod? Vad kan gå fel med en vanlig binärräkning?
4. En räknare och ett skiftregister är tekniskt sett också tillståndsmaskiner. Vad skiljer dem från maskinen som konstruerades här?
5. Vilka signaler får en Mooreutgång bero på, och vilka får en Mealyutgång bero på? Hur avgör du, givet en enda rad VHDL som driver en utgång, vilken av dem du tittar på?
6. Varför behöver Mooreversionen i avsnitt 8.5:s jämförelse ett extra tillstånd, och varför ligger dess utgång exakt en klockcykel efter Mealyversionens?
7. Varför läser LED_PROCESS i fsm_led.vhd bara state, aldrig button_edge_s2 eller button_n direkt? Vad skulle ändras om den gjorde det?
8. Varför går inte din i seq_detect_mealy genom en synkroniserare, till skillnad från reset_n?
9. Vad köper when others dig i ett case för tillståndsövergångar, när state_t bara har de värden du deklarerat?
10. Varför går timern i capstone-mottagaren sexton gånger så fort som baudhastigheten snarare än en gång per bit, och varför gör det att designen tål en sändare vars klocka skiljer sig något från din, att sampla mitt i en bit?
11. Capstone-sändaren använder en helbitstimer och behöver ingenting mitt i biten. Vad har den som mottagaren inte har, som gör skillnaden?

Blicken framåt

Det här är sista kapitlet i bokens första del. Över åtta kapitel har vi gått från grindar och Karnaughdiagram, via register, metastabilitet och själva VHDL-språket, till räknare, skiftregister och timers, och nu ändliga tillståndsmaskiner, i varje steg med resonemang om kretsen för hand innan den skrevs i syntetiserbar VHDL och kördes på en riktig FPGA. Varje mönster som byggts längs vägen, dubbelvippsynkroniseraren, kodstilen med process och case, och nu Mooremaskinen, har återanvänts i det här kapitlets egna exempel, och följer med oförändrat in i det som kommer härnäst.

Härnäst är kapitel 9, den första praktiska tentamen: tre timmar, individuell, vid datorn med GHDL, över allt de åtta första kapitlen har byggt. Efter den börjar grupprojektet, där en CAN-kontroller konstrueras i VHDL över tio kapitel och avslutas mot en riktig mikrokontroller över SPI.

8.8 Övningar

Så kontrollerar du ditt arbete. Övning 8.1 och 8.2 är penna och papper: de handlar om att få en tillståndsmaskin rätt *innan* den blir kod, och inget i dem kräver VHDL, GHDL eller kort.

Från Övning 8.4 och framåt har varje övning som ber dig skriva en VHDL-modul en utdelad självkontrollerande testbänk under `lectures/L08/exercises/`, och ingenting annat; Övning 8.3 ändrar en modul du redan har och kontrolleras för hand. Att sätta ihop resten av katalogen är en del av arbetet: modulerna du skrev i tidigare kapitel (`reset_sync`, `button_sync`, `sync`, `timer`) kopieras in från där du än skrev dem. Varje övning skriver ut de exakta kommandon den behöver.

Övning 8.6 och 8.7 är bokens capstones: de två ändarna av en och samma seriella länk, en mottagare och en sändare. Var och en komponerar moduler från fyra olika kapitel. Avsätt riktig tid för dem, och konstruera varje tillståndsdiagram på papper innan du skriver någon VHDL. Gör mottagaren först: den är den svårare av de två, eftersom den måste återskapa den bittajming den blir tilldelad snarare än att definiera den själv.

ÖVNINGSGRUPP 1

Tillståndsdiagram och tillståndstabeller

Övning 8.1 En maskin med binär tillståndsordning

KRETS

Konstruera en ny Mooremaskin med fyra tillstånd, $\{Q_1, Q_2\}$, som styr en lysdiod via en enda synkroniserad, flankdetekterad puls X från en knapp (producerad precis som X i det genomarbetade exemplet i avsnitt 8.2).

Den skiljer sig från det genomarbetade exemplet på två sätt:

- Tillståndsordningen är den vanliga binärräkningen $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$, **inte** den Graykod som används i avsnitt 8.2.
- Lysdioden (Y) lyser i **två** tillstånd i stället för ett: närhelst $Q_1 = 1$ (alltså i både 10 och 11).

a) Rita tillståndsdiagrammet: fyra tillstånd, en övergångspil per tillstånd för $X = 1$, och en självloop per tillstånd för $X = 0$.

b) Fyll i hela tillståndstabellen (alla åtta rader av $\{Q_1, Q_2, X\}$).

c) Härled Q_1^+ , Q_2^+ och Y med Karnaughdiagram, på samma sätt som avsnitt 8.2 härleder dem för Graykodsversionen.

d) Realisera grindnätet för hand: två D-vippor för $\{Q_1, Q_2\}$, grindar som beräknar Q_1^+ , Q_2^+ och Y , och bygg och simulera det i CircuitVerse, med dubbelvippsynkronisering och flankdetektering av knappgången precis som i avsnitt 8.3.

e) Verifiera den i CircuitVerse, mot din egen tillståndstabell. Det finns ingen testbänk för den här maskinen, och det är inte meningen att det ska finnas: tillståndstabellen du härledde

i b) är specifikationen, och att kontrollera en krets mot en specifikation du själv arbetat fram är hela poängen med övningen.

Stega klockan för hand och kontrollera, i den här ordningen:

- **Stegningsvägen.** Pulsa X en gång per steg och gå ett helt varv, $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$. Läs av det förväntade nästa tillståndet ur din tabell före varje flank. Läs av $\{Q_1, Q_2\}$ på vippornas utgångar efter flanken. Alla fyra övergångarna måste stämma.
- **Hållvägen.** Stega klockan med $X = 0$ i vart och ett av de fyra tillstånden och bekräfta att maskinen står kvar. Det är fyra kontroller till, och det är den halva folk hoppar över; en maskin som stegar rätt men inte håller är en maskin vars X' -termer är fel.
- **Utgången.** Kontrollera Y mot din tabell i varje tillstånd: hög i 10 och 11 , låg i 00 och 01 . Gör det som en egen genomgång, inte medan du tittar på tillståndsbitarna.

Var de skiljer sig åt säger dig vilken grind du ska titta på. Ett felaktigt Q_1^+ syns bara på de övergångar där Q_1 skulle ha ändrats. Ett felaktigt Y syns med tillståndsföljden fortfarande helt korrekt, vilket är varför det är värt en egen genomgång.

Tips: Eftersom tillstandsordningen ändrades från Graykod till binär, räkna med en övergång ($01 \rightarrow 10$) där *två* tillståndsbitar vänder på en gång. Enligt avsnitt 8.2 är det harmlöst för maskinen själv, eftersom ingenting samplar tillstandsregistret mellan flankerna, och CircuitVerses ideala simulering visar dig ingenting åt något håll. Vad det däremot påverkar är Y , som här avkodas kombinatoriskt ur tillståndsbitarna: på riktig hårdvara kan Y glitcha kort medan de två bitarna är skeva. Lägg märke till vilken övergång det är, och vad du skulle göra åt det om Y drev något som brydde sig.

ÖVNINGSGRUPP 2

Sekvensdetektering

Övning 8.2 En Mooredetektor för 101

RESONEMANG

Konstruera en **Mooremaskin** som detekterar sekvensen $1, 0, 1$ när den kommer en bit per klockcykel på en ingång d_{in} , och som höjer sin utgång y så snart träffens tredje bit setts. Icke-överlappande duger här: efter en fullständig träff, börja leta efter ett nytt $1, 0, 1$ från början i stället för att återanvända de avslutande bitarna.

- Rita tillståndsdigrammet: ett tillstånd per hur stor del av mönstret som matchats hittills, en pil ut ur varje tillstånd för $d_{in} = 0$ och för $d_{in} = 1$ (en sekvensdetektor är aldrig sysslolös, den antingen stegar framåt eller faller tillbaka), och markera vilket tillstånd som driver $y = 1$.
- Fyll i tillståndstabellen, en rad per kombination av (tillstånd, d_{in}).
- Följ den. Välj en sexbitarssekvens för d_{in} som innehåller minst en träff, och anteckna, cykel för cykel: tillståndet före flanken, värdet på d_{in} , och utgången y .

d) Fällan i den här maskinen är pilarna tillbaka, inte de framåt. Säg för varje tillstånd vad som händer på den ingång som *inte* för träffen vidare: vilka av dem går tillbaka till starten, och finns det någon ingång som borde skicka dig tillbaka till ett *delvis* matchat tillstånd i stället för hela vägen till början? Motivera ditt svar för det här mönstret.

Tips: Räkna dina tillstånd innan du ritar dem. "Inget matchat än", "sett 1", "sett 10", "sett 101" är den naturliga uppdelningen, och det sista av dem är det som driver y.

Spara ditt svar. Övning 8.4 implementerar samma detektor i VHDL, som en Mealymaskin — som, enligt avsnitt 8.5, behöver ett tillstånd färre än det du just ritade. Att lista ut vilket av dina tillstånd som försvinner, och varför, är en del av den övningen.

ÖVNINGSGRUPP 3

Ändliga tillståndsmaskiner i VHDL

Övning 8.3 Ett fjärde tillstånd i fsm_led

VHDL

Utöka fsm_led med ett fjärde tillstånd, STATE_FAST_BLINK, som blinkar med 20 ms i stället för STATE_BLINK:s 100 ms. Följ det genomarbetade exemplet i avsnitt 8.4.

Arbeta på en **kopia** av modulen, inte på lectures/L08/fsm_led/fsm_led.vhd självt. Kopiera först hela katalogen till ett eget ställe, tillsammans med de tre egna moduler den instansierar:

```
cp -r lectures/L08/fsm_led /tmp/fsm_led_fast
cp lectures/L04/exercises/reset_sync/reset_sync.vhd /tmp/fsm_led_fast # the ↵
↵ three you wrote
cp lectures/L04/exercises/button_sync/button_sync.vhd /tmp/fsm_led_fast # ↵
↵ yourself
cp lectures/L07/exercises/timer/timer.vhd /tmp/fsm_led_fast
cd /tmp/fsm_led_fast
```

Det finns ingen testbänk att köra här, men maskinen med fyra tillstånd ska ändå gå att analysera, och det är värt att göra efter varje ändring: det fångar en saknad case-gren, eller ett state_t-värde du lagt till i den ena processen men inte den andra.

```
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd fsm_led.vhd
```

Skälet att arbeta på en kopia är värt att känna till, eftersom det är en verklig fallgrop och inte sysselsättning för dess egen skull. fsm_led har en utdelad referenstestbänk, fsm_led_tb.vhd, och Övning 8.5 ber dig köra den. Den testbänken kontrollerar cykeln med **tre** tillstånd: den trycker på knappen en gång från STATE_BLINK och kontrollerar att lysdioden lyser fast, eftersom nästa tillstånd i maskinen som den är skriven är STATE_ON. I din version med fyra tillstånd

landar det trycket i STATE_FAST_BLINK, lysdioden blinkar fortfarande, och assertionen fallerar. Ingenting i felet talar om vilken av de två övningarna det hör till. Håll referensmaskinen intakt så uppstår aldrig den frågan.

Det finns ingen testbänk för maskinen med fyra tillstånd. Kontrollera den så som avsnitt 8.2 lät dig kontrollera ett tillståndsdigram: förutsäg tillståndsföljden för en serie knapptryck, läs sedan tillbaka dina case-grenar och bekräfta att de stämmer.

Du behöver:

- Lägga till det nya värdet i state_t.
- Lägga till en gren i varje case-sats, i både STATE_PROCESS och LED_PROCESS.
- Hantera de två blinktakterna, antingen genom att lägga till en andra timerinstans, eller genom att låta den befintliga timerns tickantal växla utifrån det aktuella tillståndet.

Övning 8.4 seq_detect_101_mealy

VHDL

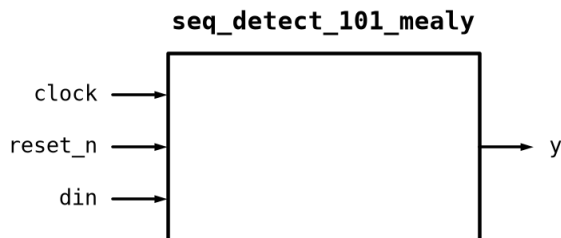
Implementera detektorn för 1, 0, 1 från Övning 8.2 i VHDL, i en ny katalog seq_detect_101_mealy — den här gången som en **Mealy**maskin.

Utgå från ditt tillståndsdigram för Mooremaskinen och konvertera det:

- Ditt sista tillstånd, "sett 101", finns bara för att hålla utgången hög i en cykel.
- En Mealymaskin behöver det inte: utgångsvillkoret kan läsa din direkt i det tillstånd som redan matchat 1, 0.
- Ta bort det tillståndet, och flytta in dess utgång i villkoret för y.

Följ sedan mönstret type state_t is (...) och case från avsnitt 8.4 och 8.5:

- Återanvänd reset_sync för reset_n, precis som seq_detect_mealy.vhd gör. Det finns ingen knapp i den här designen, så button_sync instansieras helt enkelt inte.
- Se till att tilldelningen av utgången y är en enda konkurrent sats, utanför varje klockad process, som beror på både state och din, inte bara state. Annars har du i tysthet byggt en Mooremaskin i stället, och dess utgång kommer en cykel för sent — vilket är precis vad testbänken kontrollerar.



Självkontroll. Döp din entitet till `seq_detect_101_mealy`, med portarna `clock`, `reset_n`, `din` (in) och `y` (out), deklarerade i den ordningen för att matcha `seq_detect_mealy`. En testbänk finns i `exercises/seq_detect_101_mealy/`; den kontrollerar en icke-överlappande 101-detektor. Din modul instansierar `reset_sync`, som inte är utdelad där eftersom du skrev den i Övning 4.8: kopiera in din egen.

```
cd lectures/L08/exercises/seq_detect_101_mealy
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the one you wrote in L04
ghdl -a --std=93 reset_sync.vhd seq_detect_101_mealy.vhd \
    seq_detect_101_mealy_tb.vhd
ghdl -e --std=93 seq_detect_101_mealy_tb
ghdl -r --std=93 seq_detect_101_mealy_tb --assert-level=error --stop-time=10ms
```

ÖVNINGSGRUPP 4

Att läsa den genomarbetade maskinen

Övning 8.5 Blinktakt, testbänk och tillståndsföljd

SIMULERING

`fsm_led` demonstreras på DE0-CV (avsnitt 8.6). Den här övningen kontrollerar att du kan redogöra för det du såg, och den har en utdelad referenstestbänk du kan köra själv. Den återanvänder dina två synkroniserare från kapitel 4 och din timer från kapitel 7, och ingen av dem är utdelad, så kopiera in dem först:

```
cd lectures/L08/fsm_led
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the three you wrote yourself
cp ../../L04/exercises/button_sync/button_sync.vhd .
cp ../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
    fsm_led.vhd fsm_led_tb.vhd
ghdl -e --std=93 fsm_led_tb
ghdl -r --std=93 fsm_led_tb --assert-level=error --stop-time=10ms
```

a) Räkna ut blinktakten ur källkoden, inte ur demonstrationen. `fsm_led` instansierar timer med `TIMER_TICK_COUNT` som har förvalet 5_000_000, på en klocka på 50 MHz. Hur lång är en timeoutperiod? `STATE_BLINK` växlar lysdioden vid varje timeout. Vilken blinkfrekvens ger det, och varför är den hälften av vad du kanske först skriver ned?

b) Testbänken skriver över `TIMER_TICK_COUNT` med ett mycket mindre värde än förvalet. Hitta den överskrivningen i `fsm_led_tb.vhd`. Förklara varför en testbänk inte rimligen kan

använda det riktiga värdet, och ungefär hur länge en simulering av en verklig blinkperiod skulle behöva köra.

c) Redogör för tillståndsmaskinens beteende:

- Alla tre tillstånden går att nå åt båda hållen. Följ de tryck på `button_n(0)` och `button_n(1)` som besöker varje tillstånd och återvänder till `STATE_OFF`.
- Tryck på båda knapparna under samma klockcykel. Vad gör `fsm_led`, och vilka två rader i arkitekturen avgör det?
- `timer_enable` drivs ur `state`, så timern går bara i `STATE_BLINK`. Lägg märke till att `timer` fryses snarare än nollställs när den är avstängd (avsnitt 7.3). Vad betyder det för ögonblicket då du återvänder till `STATE_BLINK` efter att ha lämnat det, och skulle det vara bättre eller sämre att hålla `timer_enable` hög permanent?

ÖVNINGSGRUPP 5

Capstone: en seriell länk

Övning 8.6 uart_rx8: mottagaren

VHDL

Det här är första halvan av bokens capstone, och den komponerar fyra kapitel till en enda design. Övning 8.7 bygger den andra änden av samma seriella länk, så stanna inte här. Ingenting i den är en ny idé; allt handlar om att koppla ihop moduler du redan har, runt en tillståndsmaskin du konstruerar själv:

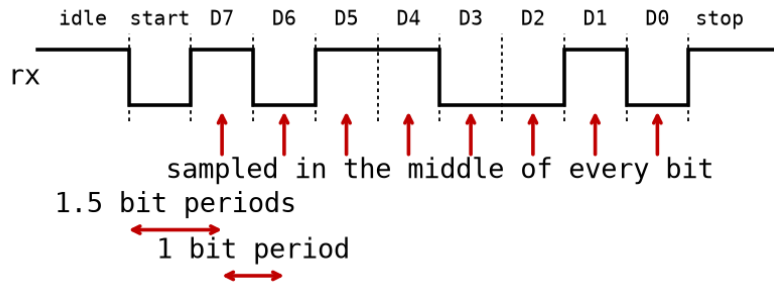
- kapitel 4:s `reset_sync`, som synkroniserar reset.
- kapitel 5:s `sync`, som synkroniserar ledningen `rx`, vilken kommer från någon annans kristall och är asynkron i precis den mening kapitel 4 avser.
- kapitel 7:s `timer`, som genererar översamplingsticket.
- Det här kapitlets tillståndsmaskin, som ramar in alltihop och sköter sin egen skiftning och biträkning. Det byggde du som en modul i kapitel 6; här är det fyra rader inuti maskinen som redan har de räknare den behöver, vilket är det vanligare sättet en mottagare skrivs på.

Skriv en entitet med namnet `uart_rx8`: en mottagare för en byte asynkron seriell data, i det format en UART sänder.

Ledningens format

Ledningen `rx` bär ramar, och mellan ramarna ligger den i viloläge:

- **Vila:** ledningen hålls hög.
- **Startbit:** ledningen går låg i exakt en bitperiod. Det är det som markerar början på en ram; det finns ingen separat klockledning, vilket är vad "asynkron seriell" betyder.
- **Åtta databitar**, mest signifikant först, en bitperiod var.
- **Stoppbit:** ledningen går tillbaka hög i en bitperiod.



Figur 8.3 En UART-ram: en ledning som vilar hög, en startbit, åtta databitar med den mest signifikanta först, och en stoppbit, med mottagaren som samplar mitt i varje bit.

Ramen ovan bär 10110010, som är den första byten testbänken skickar. Pilarna är de åtta ögonblick då mottagaren tar upp en **databit**: en och en halv bitperiod efter den fallande flanken, och sedan en gång per bitperiod därefter. Den tittar på ledningen även däremellan — efter den fallande flanken, mitt i startbiten för att kontrollera att den fortfarande är låg, och mitt i stoppbiten — men bara de åtta pilarna bidrar med bitar till den mottagna byten.

Mottagaren måste återskapa bittajmingen enbart ur startbitens fallande flank. Det är hela problemet den här övningen löser, och det är därför timern finns här.

En medveten avvikelse från en riktig UART. En riktig UART sänder sina databitar med den **minst** signifikanta först, som avsnitt 6.3 påpekar. Den här övningen sänder dem med den mest signifikanta först:

- ramningen och tajmingen, som är vad de här två övningarna faktiskt handlar om, är identiska åt båda hållen.
- det är den riktning avsnitt 6.3:s SIPO-idiom skiftar åt, så mottagarens skift blir en rad, och sändaren i Övning 8.7 blir dess spegelbild.
- de utdelade testbänkarna sänder och förväntar sig MSB först för att matcha, så paret är konsekvent med sig självt.
- för att prata med en riktig UART skulle du vända skiftriktningen till avsnitt 6.2:s spegelvända form och inte ändra något annat. Värt att veta innan du kopplar det här till en dators serieport och undrar varför varje byte kommer bitvänd.

Översampling: sexton tick per bit

Mottagaren måste hitta mitten av en bit vars början den bara kan sluta sig till. Tekniken heter **översampling**: kör timer med **sexton gånger** baudhastigheten, så att det finns sexton tick per bitperiod, och räkna dem.

- Startbitens fallande flank nollställer tickräkningen till 0. Timern går kontinuerligt, före, under och efter en ram; det är *räkningen* som nollställs, aldrig timern.
- **Tick 8** ligger en halv bit senare, mitt i startbiten. Kontrollera att ledningen *fortfarande är låg* där. Har den gått hög var det en glitch och ingen startbit, och mottagaren återgår till viloläge. Den omkontrollen är det som hindrar brus på en vilande ledning från att läsas som data.
- Därifrån är vart **sextonde** tick mitten på nästa bit: sampla ledningen och skifta in den. Åtta av dem är databitarna, och det nionde är stoppbiten.

Att sampla mitt i biten i stället för vid flankerna är det som ger mottagaren dess tajming-marginal. Sändarens klocka är en annan fysisk kristall än din, så de två driver isär under en ram; en mottagare som samplade nära gränserna skulle läsa fel bit så snart de gjorde det. Sexton tick är det konventionella valet, och sändaren i Övning 8.7 går på samma tick.

Entiteten

Generic	Typ	Förval	Beskrivning
OVERSAMPLE_TICK_COUNT	natural	325	Tickantalet för timer för en sextondel av en bitperiod. 9600 baud vid 50 MHz; del b) ber dig härleda det.

Port	Riktn.	Typ	Beskrivning
clock	in	std_logic	Systemklocka.
reset_n	in	std_logic	Asynkron, aktiv låg; går genom reset_sync, som alltid.
rx	in	std_logic	Den seriella ledningen, direkt från en pinne och därför asynkron . Synkronisera den till rx_s2 innan något annat läser den.
data_out	out	std_logic_vector(7 downto 0)	Den mottagna byten, giltig under den cykel då byte_valid är hög.
byte_valid	out	std_logic	En puls på en cykel: en komplett, korrekt ramad byte ligger på data_out.
frame_err	out	std_logic	En puls på en cykel: stoppbiten var inte hög, så ramen är trasig och ingen byte lämnas över.

a) Konstruera tillståndsmaskinen **på papper först**, precis som avsnitt 8.2 lät dig göra. Fyra tillstånd:

- STATE_IDLE, som väntar på att ledningen ska gå låg.
- STATE_START, som vid tick 8 kontrollerar att ledningen fortfarande är låg.

- STATE_DATA, som samplar en bit vart sextonde tick, åtta gånger.
- STATE_STOP, som samplar stoppbiten sexton tick efter den sista databiten och väljer mellan byte_valid och frame_err.

Rita den innan du skriver någon VHDL, och på diagrammet:

- Ge varje tillstånd sin **självloop**: pilen för "ticket jag väntar på har inte kommit än, stå kvar". Det är den pil som brukar saknas, och ett tillstånd utan den faller igenom dit else råkar peka.
- Skriv bredvid varje tillstånd **hur många tick det väntar på**, så att tickbokföringen ligger på papperet i stället för i huvudet.
- Markera var ticks **nollställs**. Timern själv stannar aldrig, så de nollställningarna är de enda ställen där mottagaren avgör vad "början på en bit" betyder.
- Märk pilen ut ur STATE_IDLE med en **flank**, inte en nivå: rx_s2 låg och hög vid föregående tick. Det kräver en bit historik, så anteckna var du tänker hålla den.

b) Räkna ut tajmingen innan du implementerar den:

- DE0-CV:s klocka är 50 MHz och 9600 baud betyder 9600 bitar per sekund. Hur många klockcykler är en bitperiod? Hur många är en sextondel av en?
- timer pulsar var TICK_COUNT + 1 klockcykel (avsnitt 7.1). Vilket OVERSAMPLE_TICK_COUNT ger det dig? Det exakta svaret är inget heltal, så det finns två kandidater på var sin sida om det: räkna ut felet var och en av dem lämnar, och säg vilken av dem förvalet ovan är och varför.
- Det felet är en bråkdel av en procent, och det ackumuleras över en ram. Hur långt har din samplingspunkt tio bitperioder efter startflanken drivit från mitten av stoppbiten, mätt i översamplingstick? Hur många tick marginal hade den från början?
- Och nu skälet till att svaret är bekvämt: du omsynkroniserar vid varje startbit. Vad skulle behöva gälla för ramlängden för att det skulle sluta räcka?

c) Implementera uart_rx8. Den instansierar tre moduler och gör resten själv:

- reset_sync för reset. Allt annat i designen tar dess reset_s2_n.
- **sync**, modulen du skrev i Övning 5.4, på pinnen rx, som ger det rx_s2 tillståndsmaskinen läser. Instansiera den med SIZE = 1 och PRESET = '1': en vilande ledning är hög, och en synkroniserare som kommer ur reset och läser '0' skulle för den här maskinen se ut som en startbit. rx kommer från en pinne som drivs av en sändare med sin egen kristall, så den är asynkron i precis den mening kapitel 4 avser, och varje regel därifrån gäller oförändrad. Lägg märke till att den tar en std_logic_vector med ett element på var sida, så du deklarerar ett par sådana och plockar ut rx_s2 ur utgången.
- timer, med OVERSAMPLE_TICK_COUNT skickat rakt igenom, enable bundet till '1', och dess timeout som din enda uppfattning om förfluten tid. Kalla den baud_tick. Timern frilöper: den stoppas aldrig, nollställs aldrig, och inget tillstånd rör den någonsin. Det maskinen fasar om i stället är sin egen räknare ticks.

- Ingen skiftregistermodul. Maskinen behöver ändå en biträknare, så den skiftar in ledningen direkt i en signal frame själv, en rad VHDL per samplad bit.

Tillståndsmaskinen, tillstånd för tillstånd

Två räknare: ticks som räknar 0 till 15 inom en bit, och bit_count som räknar de åtta databitarna. Allt nedan händer **bara när baud_tick = '1'**; vid varje annan klockflank står maskinen still. Eftersom timern frilöper är ett tick alltid på väg, i varje tillstånd inklusive vila.

Tillstånd	Väntar på	Sedan	Till
STATE_IDLE	fallande flank på rx_s2	nollställ ticks	STATE_START
STATE_START	ticks = 8	rx_s2 fortfarande låg? nollställ ticks rx_s2 hög? det var en glitch	STATE_DATA STATE_IDLE
STATE_DATA	ticks = 15	skifta in rx_s2 i frame, nollställ ticks	STATE_DATA tills 8 bitar, sedan STATE_STOP
STATE_STOP	ticks = 15	rx_s2 hög? data_out ≤ frame, pulsa byte_valid rx_s2 låg? pulsa frame_err, kasta byten	STATE_IDLE STATE_IDLE

Det finns inget "klart"-tillstånd. Samplingen av stoppbiten är där ramen bedöms, så det är där byten lämnas över, och maskinen går rakt tillbaka till viloläge.

Varför vilotillståndet väntar på en flank i stället för en nivå

Håll ett tick historik för rx_s2, och lämna vilotillståndet bara vid övergången från hög till låg. Att i stället testa nivån, "om rx_s2 är låg, starta en ram", ser likvärdigt ut och klarar allt utom det enda fall som spelar roll.

Betrakta ett **break**: ledningen hålls låg långt längre än en ram, vilket är hur en sändare som tappar strömmen ser ut. Ett nivåtest återaktiveras i samma ögonblick som den föregående ramens stoppbit bedöms, så mottagaren tillbringar hela breaket med att marschera genom ram efter ram av idel nollor. Var och en slutar på en låg stoppbit och förkastas korrekt, och den delen är i sin ordning. Problemet är den ram som är i luften när breaket *slutar*: dess stoppbit samplas efter att ledningen gått tillbaka hög, så det är en fullkomligt välformad ram så vitt mottagaren kan avgöra, och en byte gjord av ingenting lämnas över som riktiga data. Flanktestet återaktiveras inte alls under breaket, eftersom det efter den första fallande flanken inte kommer någon till förrän ledningen har hämtat sig.

Det är hela skillnaden, den kostar en vippa, och det är därför en mottagare som "fungerar" på rena data ändå kan lämna över skräp första gången en kabel dras ur.

Skiftet

Åtta databitar kommer med den mest signifikanta först, så SIPO-idioten från avsnitt 6.3 klarar det helt utan indexaritmetik:

```
frame <= frame(6 downto 0) & rx_s2;
```

bit_count räknar då bara *hur många* bitar som kommit, aldrig *var* de hamnar. Att i stället lagra med `frame(bit_count) <= rx_s2` lägger den först ankomna biten i bit 0, vilket vänder byten: det är den ordning med minst signifikant först som en riktig UART använder, och det skulle vara rätt val om den här mottagaren måste prata med en sådan. Här ligger MSB först på ledningen, så skiftet är det som matchar.

Fyra saker avgör om det här fungerar:

- **Allt är villkorat på baud_tick.** Ett tillstånd som agerar på en vanlig klockflank körs sexton gånger per tick och ungefär femtusent gånger per bit.
- **Varje tillstånd behöver sin "inte än"-väg.** Om ticket du väntar på inte har kommit, eller räkningen inte har nått sitt mål, måste maskinen **stå kvar där den är**. Att skicka den någon annanstans i `else` är det överlägset vanligaste sättet den här designen fallerar på, och den fallerar fullständigt snarare än subtilt: maskinen hamnar i att studsa fram och tillbaka mellan två tillstånd och når aldrig det tredje.
- **Se upp med ettfelet i räknaren.** Med ticks som börjar på 0 och ökar vid varje tick är det n :te ticket `ticks = n - 1` i det ögonblick du testat det. Om du jämför före eller efter ökningen avgör vilket tick du faktiskt samplar på, och att vara ett eller två tick sen går att överleva medan åtta inte gör det.
- **Timern frilöper, så din fas kommer ur ticks, aldrig ur timern.** Startflanken kommer när sändaren skickar den, vilket är någonstans inuti en tickperiod mottagaren inte valt, så det första ticket efter den flanken kan komma allt från ett helt tick till nästan ingen tid senare. Att nollställa ticks på vägen ut ur vilotillståndet är det som fasar om samplingen mot startbiten. Den överblivna bråkdelen av ett tick är priset för att aldrig stoppa timern, och det är billigt: en sextondel av en bit, mot den halva bit marginal del b) ber dig räkna ut.

d) Kör testbänken. Den behöver fyra designfiler, delblocken först, och inget av de tre delblocken är utdelat i övningskatalogen: alla tre är moduler du skrivit.

```
cd lectures/L08/exercises/uart_rx8
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the three you wrote ~
  ~ yourself
cp ../../L05/exercises/sync/sync.vhd .
cp ../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 sync.vhd reset_sync.vhd timer.vhd uart_rx8.vhd uart_rx8_tb.vhd
ghdl -e --std=93 uart_rx8_tb
```

```
ghdl -r --std=93 uart_rx8_tb --assert-level=error --stop-time=10ms
```

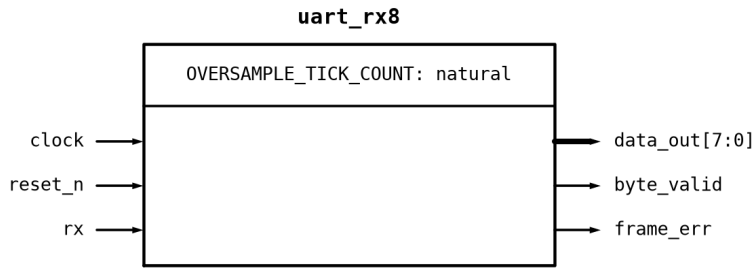
Den agerar sändare, och kontrollerar:

- två rena ramar, som kommer fram som rätt byte, utan något `frame_err`.
- en vilande ledning, före, mellan och efter dem, som inte producerar något.
- en ram vars stoppbit är **låg**: `frame_err` måste pulsa och ingen byte får lämnas över.
- ledningen hållen låg långt längre än en ram, vilket är ett **break**: varje ram mottagaren tror sig se där slutar med en låg stoppbit, så ingen av dem är en byte.
- en ram där varje databit bär sitt värde bara i **mitten** av bitperioden och motsatt nivå på var sida. En mottagare som samplar nära tick 8 läser den perfekt; en som samplar nära en gräns läser grannbiten.
- att `byte_valid` och `frame_err` var för sig aldrig är höga två cykler i rad.

Tips: Bygg den i etapper och kör testbänken efter varje, i stället för att skriva alla fyra tillstånden och felsöka dem på en gång. Få först maskinen att lämna vilotillståndet på en startbit och nå `STATE_DATA`; kommer den aldrig dit är det "inte än"-vägen du ska titta på. Sampla sedan en databit och kontrollera att det är rätt bit innan du bryr dig om de andra sju. Sedan stoppbiten och de två pulserna.

e) Ha nu sönder den med flit, och fundera på vad resultatet bevisar och inte bevisar:

- Ta bort stoppbitstillståndet, och lämna över byten så snart den åttonde databiten är inne. Förutsäg vad som borde gå fel, och kör den sedan. Två kontroller fångar det här, och det är värt att veta vilken som utlöser först: ramen med låg stoppbit höjer inte längre `frame_err`, och den lämnar över en byte som skulle ha kastats.
- Nu andra halvan av lärdomen. Att passera är den svagare signalen, så hitta en sabotage-ring testbänken **inte** fångar. Här är en: **ta bort omkontrollen mitt i startbiten**, så att `STATE_START` går vidare till `STATE_DATA` vid tick 8 utan att bekräfta att ledningen fortfarande är låg. Testbänken lägger aldrig någon glitch på en vilande ledning, så ingenting märker det, och på riktig hårdvara startar en enda brusspik en spökram. Läs `uart_rx8_tb.vhd` och säg vad du skulle lägga till för att fånga det.
- Här är en andra: **sampla vid tick 3 i stället för tick 8**. Räkna ut varför testbänken fortfarande passerar, och vad du har gett bort. Svaret ligger i marginalberäkningen i del b).
- Och en tredje, som är den viktigaste av de tre: **ta bort synkroniseraren för rx** och läs pinnen direkt. Varje kontroll passerar fortfarande, och det kommer den alltid att göra, oavsett vad som läggs till i testbänken. Säg varför, i termer av vad en simulator modellerar och vad den inte modellerar, och läs sedan om avsnitt 4.4. Det här är den enda sortens bugg i boken som testning inte kan nå, vilket är precis varför kapitel 4 ger dig en regel att följa snarare än ett symptom att leta efter.
- Med några meningar: vad lovar egentligen "passerar sin testbänk", och vad lovar det inte?



Självkontroll. Döp din entitet till `uart_rx8`, med generic `OVERSAMPLE_TICK_COUNT (natural)`, ingångarna `clock`, `reset_n`, `rx`, och utgångarna `data_out (std_logic_vector(7 downto 0))`, `byte_valid` och `frame_err`, deklarerade i den ordningen. Dess testbänk finns i `exercises/uart_rx8/`, och inget annat: **kopiera in dina egna `reset_sync`, `sync` och `timer`**, som del **d**) visar. En projektkatalog innehåller allt den byggs av, och att sätta ihop den är en del av övningen.

Övning 8.7 `uart_tx8`: sändaren

VHDL

Den andra halvan av länken: skriv `uart_tx8`, en sändare som skickar en byte i samma format som Övning 8.6 tar emot.

En sändare är den enklare av de två, och skälet är värt att förstå innan du börjar:

- Mottagaren måste *återskapa* bittajmingen ur en startflank den inte styrde över, vilket är varför den samplade mitt i biten och varför dess timer gick på en sextondels bitperiod.
- Sändaren **definierar** tajmingen. Ingenting behöver återskapas, så timern går på en hel bitperiod och maskinen håller helt enkelt varje bit på ledningen under en timeout.

Den sänder den mest signifikanta biten först, vilket matchar mottagaren i Övning 8.6 snarare än en riktig UART, av skälet som anges där.

Entiteten

Generic	Typ	Förval	Beskrivning
BIT_TICK_COUNT	natural	5207	Tickantalet för timer för en hel bitperiod, 9600 baud vid 50 MHz. Sexton av Övning 8.6:s översamplingstick, så när som på den avrundning dess del b) ber dig räkna ut, så att en uart_tx8 och en uart_rx8 byggda med de matchande förvalen pratar i samma takt.

Port	Riktn.	Typ	Beskrivning
clock	in	std_logic	Systemklocka.
reset_n	in	std_logic	Asynkron, aktiv låg; genom reset_sync, som alltid.
data_in	in	std_logic_vector(7 downto 0)	Byten som ska skickas.
send	in	std_logic	En begäran om att sända data_in. Ignoreras medan sändaren är upptagen.
tx	out	std_logic	Den seriella ledningen. Vilar hög , även under reset.
busy	out	std_logic	Hög från ögonblicket en ram startar tills stoppbiten är över.

a) Rita tillståndsmaskinen först. Fyra tillstånd är den naturliga uppdelningen: vila, startbit, databitar, stoppbit, och vart och ett håller ledningen på en definierad nivå under en bitperiod.

- Markera vad som driver tx i varje tillstånd. Lägg märke till att tx aldrig beror på send eller någon annan primär ingång: det är tillståndet, plus skiftregistrets aktuella utgång. Det gör det här till en Mooremaskin, som allt annat i den här boken.
- Markera var busy kommer ifrån. Det är en enda jämförelse mot tillståndet.
- Bestäm var byten fångas. Den måste läsas när ramen startar, inte läsas kontinuerligt ur data_in: producenten står fritt att ändra data_in i samma ögonblick som busy går hög, och en verklig sådan kommer att göra det.

b) Skifta ut byten med den mest signifikanta biten först, i maskinen själv, spegelbilden av vad mottagaren gör: läs in data_in i en signal frame när ramen startar, driv tx från den översta biten i frame medan du är i datatillståndet, och skifta frame ett steg uppåt per bitperiod, med räkning på de åtta bitar som går ut.

Det här skrev du som en modul i Övning 6.7, och piso8 är värd att läsa om för idiomets skull. Att instansiera den här skulle fungera, men maskinen äger redan den biträknare ett PISO-register behöver, så skiftet blir en enda rad inuti det tillstånd där det hör hemma. Så är båda halvorna av den här länken byggda, och samma avvägning återkommer i kapitel 14, där tx_shift_reg är en egen modul just för att can_controller *inte* redan äger den räknare den behöver.

c) Implementera den, med `reset_sync` instansierad för reset och timer med `BIT_TICK_COUNT` skickat rakt igenom.

Här är timerens `enable` driven av tillståndet, hög bara medan en ram är i luften, och det är det enda stället där den här designen medvetet skiljer sig från Övning 8.6. Mottagaren måste låta sin timer frilöpa eftersom den inte styr när en ram börjar: den kan bara fasa om sin egen tickräkning när startflanken redan har kommit. Sändaren har det motsatta problemet. Den *bestämmer* när ramen börjar, så den kan starta timern i det ögonblicket och få en första bitperiod som är exakt lika lång som alla andra. Låt den frilöpa här i stället, så blir startbiten kort med precis så mycket av en tickperiod som råkade återstå när `send` kom, vilket är en defekt i det enda en sändare är ansvarig för.

Två krav testbänken är sträng med:

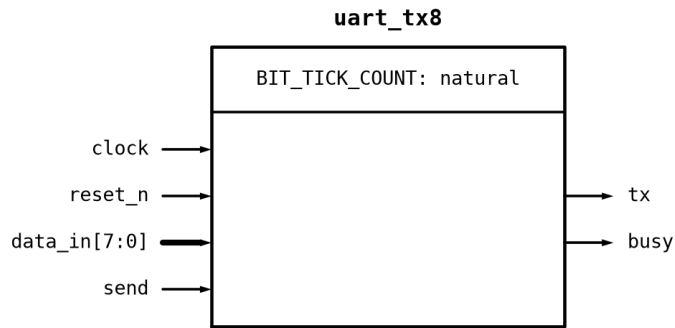
- **send ignoreras medan sändaren är upptagen.** Att ladda om mitt i en ram skulle starta om byten och korrumpiera det som redan ligger på ledningen. En begäran som kommer under en ram kastas, den köas inte.
- **busy ligger kvar hög genom hela stoppbiten**, inte bara databitarna. Ramen är inte över förrän ledningen hållits hög under den sista bitperioden.

d) Kör testbänken. Den agerar mottagare: den väntar på din startbit, samplar mitten av varje bitperiod, och kontrollerar ramningen, byten, handskakningen med `busy`, och att ett `send` som matas in två bitperioder in i en ram ignoreras.

```
cd lectures/L08/exercises/uart_tx8
cp ../../L04/exercises/reset_sync/reset_sync.vhd .      # the two you wrote ↵
  ↵ yourself
cp ../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd timer.vhd uart_tx8.vhd uart_tx8_tb.vhd
ghdl -e --std=93 uart_tx8_tb
ghdl -r --std=93 uart_tx8_tb --assert-level=error --stop-time=10ms
```

e) Svara, i löpande text:

- En sändare som hoppar över stoppbiten helt lämnar ändå ledningen hög efteråt, eftersom vila och stopp är samma nivå. Vad går egentligen fel, och när visar det sig först? Testbänken fångar den här: vilken av dess kontroller gör det, och varför hade det aldrig gått att fånga genom att bara titta på `tx`?
- Din mottagare i Övning 8.6 samplar mitt i varje bit; din sändare ändrar ledningen vid flankerna. Om båda gick på kristaller som skilde sig med 1 %, ungefär hur många bitar in i en ram skulle mottagarens samplingspunkt driva in i fel bit? Vad säger det dig om varför seriella format har en startbit per byte, snarare än en enda i början av ett långt meddelande?



Självkontroll. Döp din entitet till `uart_tx8`, med generic `BIT_TICK_COUNT (natural)`, ingångarna `clock`, `reset_n`, `data_in (std_logic_vector(7 downto 0))`, `send`, och utgångarna `tx`, `busy`, deklarerade i den ordningen. Dess testbänk finns i `exercises/uart_tx8/`, och inget annat: kopiera in dina egna `reset_sync` och `timer`, som del **d**) visar.

Praktisk tentamen 1: sekvensnät

Tre timmar, individuellt, vid datorn. Omfattar del I i sin helhet, med tyngdpunkt på sekvensnät.

I DET HÄR KAPITLET

- Tentamens form: fristående uppgifter, var och en med en utdelad självkontrollerande testbänk.
- Vad som är tillåtet att använda, och vad som inte är det.
- Hur man förbereder sig, och vad som är den bästa förberedelsen som finns.
- Vad tentamen faktiskt mäter.

Det här är kursens första examinationstillfälle. Det innehåller ingen ny teori; kapitlet säger vad tentamen mäter, vilken form uppgifterna har och hur man förbereder sig, och det är värt att läsa i förväg snarare än dagen innan. Poäng och betygsgränser står i bilaga C.

Tentamen omfattar kapitel 1 till 8 i sin helhet, med tyngdpunkt på sekvensnät: register, räknare, skiftregister, timers, synkroniserare och enkla Mooremaskiner.

9.1 Upplägg

Tentamen består av ett antal fristående uppgifter. Varje uppgift ger dig en **utdelad, självkontrollerande testbänk** och en beskrivning av den modul den driver: portordning, typer och beteende. Din uppgift är att skriva modulen så att testbänken passerar.

- Uppgifterna är oberoende av varandra. En modul du inte får att fungera stoppar ingen annan.
- Uppgifterna är ordnade ungefär efter stigande svårighet, men läs igenom alla innan du börjar.
- En modul som inte är helt klar rättas på det som finns. Delvis korrekt logik ger delpoäng, så lämna aldrig in en tom fil för att den inte blev färdig.
- Testbänken är facit under skrivningen, inte efter den. `all checks passed!` betyder att den uppgiften är klar; rättningen läser dessutom koden.

Ett övningsprov med uppgifter av samma slag och samma omfattning delas ut i förväg.

9.2 Hjälpmedel

Tillåtet:

- Allt kursmaterial: den här boken, föreläsningarnas README, samtliga appendix, och all VHDL du själv skrivit under del I, inklusive dina lösningar på övningarna.
- Din egen dator, med GHDL. Se avsnitt 2.7 om du behöver påminna dig om hur en testbänk körs.
- CircuitVerse, om du vill rita en krets innan du skriver den. Det är ofta snabbare, inte långsammare.

Inte tillåtet: kommunikation med andra i någon form; språkmodeller och andra verktyg som genererar eller förklarar VHDL åt dig; och sökning på nätet efter färdiga lösningar. Gränsen går inte vid vem som trycker på tangenterna utan vid vems förståelse som prövas. Att slå upp GHDL:s eller VHDL:s dokumentation är tillåtet; att söka efter en färdig modul är det inte.

Den fullständiga och auktoritativa hjälpmedelslistan står i bilaga C; står något annat här gäller den.

9.3 Före tentamen

- Gå igenom övningsprovet, under tidspress och utan att titta i lösningarna först.
- Övning 8.6 och 8.7, bokens två capstones, är den bästa förberedelsen som finns: de tvingar dig att konstruera ett tillståndsdigram själv och komponera tidigare moduler runt det, vilket är precis vad tentamen mäter.
- Se till att GHDL fungerar på din maskin *innan* du kommer. En trasig verktygskedja är inget skäl till förlängd skrivtid.

9.4 Det här mäter tentamen

- Att du kan skriva den synkrona processmallen korrekt, utan att slå upp den.
- Att du kan bygga en räknare, en timer och ett skiftregister och veta vad som skiljer dem.
- Att du synkroniserar varje asynkron ingång, utan att bli påmind.
- Att du kan konstruera en liten Mooremaskin ur en specifikation i ord, och skriva den som en uppräknad typ och ett case.
- Att du kan läsa en testbänks felutskrift och hitta tillbaka till raden som orsakade den.

9.5 Efter tentamen

Nästa kapitel startar **grupprojektet**, som löper över kapitel 10 till 19 och står för halva kursens poäng. Läs projektspecifikationen i bilaga C före kapitel 10; grupperna sätts samman där.

Kapitel 10 tar upp projektstarten: CAN-bussen, ramen och arbitreringen, och projektets första VHDL, det delade paketet `can_def`.

Del II

Grupprojektet: en CAN-nod

Projektstart: CAN-bussen, ramen och can_def

Varför protokoll behöver ramar, vad CAN faktiskt är, och projektets första VHDL.

I DET HÄR KAPITLET

- Varför protokoll behöver ramar alls: nyttolast, längdfält, checksumma, och varför avsändare, mottagare och sekvensnummer uppstår så snart en länk delas av fler än två noder.
- CAN:s differentiella buss och multimasterdesign.
- Dominanta och recessiva bitar, och varför arbitrerings ersätter en bussmästare.
- Broadcast plus ID-filtrering i stället för adresser.
- CAN-ramen fält för fält: SOF, identifierare, RTR, kontrollfält, DLC, data, CRC-15, ACK, EOF.
- Bitstopning: femregeln, och varför den finns.
- Wired-AND-arbitrerings, genomräknad för hand.
- Bittajming och sampelpunkten, på begrepps nivå.
- can_def: det delade paketet som håller varje konstant kapitlet definierar.

Det här är första kapitlet i grupprojektet. Läs projektspecifikationen i bilaga C innan du börjar: den beskriver arbetsformen, modulerna och vad som bedöms. Grupperna sätts samman i det här passet.

Avsnitt 10.1 bygger upp ett eget litet ramformat från ingenting och motiverar varje fält CAN sedan har. Det är bokens enda C++-material, och det finns för att göra resten av kapitlet till en jämförelse snarare än en uppräknings: CAN besvarar samma fyra frågor, men besvarar en av dem helt annorlunda.

Protokollreferensen. Den här boken handlar om att bygga en CAN-nod, inte om CAN som standard. Protokollet självt beskrivs i *CAN: bussen, ramen och kontrollern*, en kort bok skriven för båda halvorna av ett CAN-system — den som bygger kontrollern i hårdvara och den som skriver drivrutinen i mjukvara — och den är protokollreferensen genom hela del II.

Det här kapitlet ger den arbetsförståelse som behövs för att skriva `can_def` och komma igång; går du vilse i ett fält, en regel eller ett tidsförhållande är det där du slår upp det. För den som bygger kontrollern är dess **kapitel 1–6 kravspecifikationen, bit för bit**, och dess kapitel 7 beskriver precis den förenklade kontrollen del II bygger: vad den implementerar av standarden och vad den medvetet låter bli. Bilaga E har den fullständiga kapitelkartan mellan de två böckerna.

10.1 Ramar och serialisering

Kapitel 1 till 8 slutade med tillståndsmaskiner som styrde ett enda kort: en blinkande lysdiod, en UART på en FPGA. Projektet utgår från ett annat problem: att få oberoende hårdvarudelar att utbyta information tillförlitligt över en delad ledning, där ingen av sidorna kan se in i den andra.

Det här är centralt i inbyggda system eftersom kommunikationen sker som råa byteströmmar, RAM och flash är begränsade, och robusthet är kritisk. Noder kommunicerar oftast över seriella gränssnitt som UART, SPI, I2C eller RS-485. Data skickas som en följd av bytes, och att tolka den byteströmmen kräver en struktur. Den strukturen kallas en **ram**.

En ram är en strukturerad representation av data som:

- Innehåller tydligt definierade fält.
- Har en definierad längd.
- Kan valideras (till exempel via en checksumma).
- Kan serialiseras till en bytearray (`std::uint8_t[]`).
- Kan deserialiseras från en bytearray.

Betrakta nod A som skickar värdet `0x2A` till nod B över en delad ledning, utan att något annat är överenskommet i förväg:

- Hur vet B när ett nytt värde börjar i en ström av bitar som ser likadan ut mitt i ett värde som mellan två värden?
- Hur vet B hur långt (hur många bytes) värdet är?
- Hur vet B att bitarna den samplade inte förvanskades av elektriskt brus?
- Vad händer om två noder sänder samtidigt?

Varje riktigt protokoll besvarar de här med någon kombination av följande, var och en namngiven här efter det fält den här bokens ram bär den i:

- **Ramning (SOF)**: en igenkännbar startmarkör (och ofta slutmarkör), så att en mottagare kan avgöra var ett meddelande börjar och var det slutar.
- **Ett längdfält (LEN)**: så att mottagaren vet hur mycket nyttolast den ska vänta sig.
- **En checksumma (CHK)**: extra bitar, beräknade ur nyttolasten, som låter mottagaren upptäcka (inte nödvändigtvis rätta) många sorters överföringsfel.
- **En regel för mediaåtkomst**: vad som händer när fler än en nod vill sända samtidigt.

Ett TYPE-fält sällar sig till dem för att säga vad meddelandet betyder, så att mottagaren kan agera utifrån det.

En liten men realistisk ram

För att göra det konkret: konstruera en ram för att skicka korta meddelanden mellan noder på en delad, byteorienterad länk (UART, RS-485, ...):

Example Frame Format

SOF (2 bytes)	LEN (1 byte)	TYPE (1 byte)	DST (1 byte)	SRC (1 byte)	SEQ (2 bytes)	DATA (N bytes)	CHK (2 bytes)
------------------	-----------------	------------------	-----------------	-----------------	------------------	-------------------	------------------

Field	Size	Description
SOF	2 bytes	Start-of-frame sync word, 0xA5F7
LEN	1 byte	Payload length, in bytes
TYPE	1 byte	Message type
DST	1 byte	Destination address
SRC	1 byte	Source address
SEQ	2 bytes	Sequence number (correlates a response with its request)
DATA	N bytes	Payload
CHK	2 bytes	Checksum

Figur 10.1 Ramformatet som resten av det här avsnittet bygger, fält för fält.

Tre regler spelar roll så snart man börjar räkna ut de här fälten för hand:

- **Byteordning:** varje fält bredare än en byte (SOF, SEQ, CHK) är big-endian, mest signifikanta byte först.
- **LEN** räknar bara DATA-fältet, aldrig huvudet och aldrig CHK.
- **CHK** är en 16-bitars summa av varje byte från SOF till och med den sista nyttolastbyten, som slår runt vid spill. Den beräknas över den *serialiserade* byteströmmen snarare än över fältvärdena som heltal, så SOF bidrar med 0xA5 + 0xF7, inte 0xA5F7 behandlat som ett enda 16-bitarstal.

Varför just DST, SRC och SEQ. SOF, LEN, TYPE och CHK besvarar redan de fyra frågorna ovan. De här tre finns eftersom en delad länk med fler än två deltagare väcker nya:

- **DST och SRC:** vem ramen är till och vem den är från, så att en nod kan ignorera trafik adresserad någon annanstans och ett svar kan hitta tillbaka. CAN besvarar det här helt annorlunda, som avsnitt 10.3 visar.
- **SEQ:** matchar ett svar tillbaka mot *just den* förfrågan, inte någon annan som är i luften.

Att välja SOF

- En enda synkbyte är lätt att träffa av misstag: vilken nyttolastbyte som helst som råkar vara lika med den ser ut som början på en ram.
- Två bytes är därför vanligt, och det här protokollet använder 0xA5F7.
- SOF är det *första* en parser kontrollerar när den läser en ram. Stämmer den inte kastas datan, och parsern fortsätter söka efter nästa SOF.
- Lägg märke till vad det ger och inte ger: det gör en falsk synkronisering mycket mindre sannolik, men inte omöjlig, eftersom nyttolastbytes fortfarande kan råka stava A5 F7. Den kvarvarande risken är varför LEN och CHK ändå måste kontrolleras efteråt.

Genomräknat exempel: PING och PONG

Anta att nod 0x01 pingar nod 0x02, med sekvensnummer 0x0020 och utan nyttolast:

- SOF = 0xA5F7, LEN = 0x00, TYPE = 0x00 (Ping), DST = 0x02, SRC = 0x01, SEQ = 0x0020.
- CHK = 0xA5 + 0xF7 + 0x00 + 0x00 + 0x02 + 0x01 + 0x00 + 0x20 = 0x01BF.

SOF	LEN	TYPE	DST	SRC	SEQ	DATA	CHK
A5 F7	00	00	02	01	00 20	--	01 BF

Nod 0x02 svarar med en PONG:

- SOF = 0xA5F7, LEN = 0x00, TYPE = 0x01 (Pong), DST = 0x01, SRC = 0x02, SEQ = 0x0020.
- CHK = 0xA5 + 0xF7 + 0x00 + 0x01 + 0x01 + 0x02 + 0x00 + 0x20 = 0x01C0.
- Lägg märke till att DST och SRC är omkastade, SEQ är oförändrad, och bara TYPE och de två adresserna skiljer sig från PING:en: identisk aritmetik med andra indata.

Det täcker tre av de fyra inledande frågorna, plus de extra som en delad länk med flera noder väcker. Den fjärde står fortfarande öppen: vad händer om två noder sänder samtidigt? CAN besvarar den annorlunda, och det är ämnet för avsnitt [10.2](#).

Ramtyper

TYPE är en byte på ledningen, så en ramtyp färdas som ett teckenlöst heltal. Varje typ tilldelas sitt eget id enligt konvention: Ping = 0, Pong = 1, StatusReq = 2, StatusResp = 3. Ytterligare typer som protokollet får fortsätter helt enkelt sekvensen.

Två följer värda att notera innan någon kod skrivs:

- Den mottagna byten är obetrodd indata. Varje värde mottagaren inte känner igen måste förkastas i stället för att agera på, precis som en felaktig SOF eller en felaktig checksumma.
- Att reservera ett ID direkt ovanför den sista riktiga typen, ett "okänt"-värde, gör den kontrollen till en enda jämförelse. Utan det behöver valideringen en lista över giltiga ID:n som måste hållas i takt varje gång en typ läggs till.

En enkel 16-bitars checksumma

För den här bokens syfte behöver en checksumma bara vara enkel att implementera, enkel att testa, och tillräckligt bra för att fånga många slumpmässiga bitfel. En 16-bitars summa uppfyller alla tre: börja på 0, och för varje byte i intervallet, $\text{sum} += \text{byte}$. Resultatet är checksumman, modulo 2^{16} .

```
/**
 * @brief Compute a simple 16-bit checksum (sum of bytes modulo 2^16).
 *
 * @param[in] data Pointer to the bytes to checksum.
 * @param[in] dataLen Number of bytes to checksum.
 *
 * @return The 16-bit checksum.
 */
std::uint16_t checksum16(const std::uint8_t* data, const std::size_t dataLen) ↵
↵ noexcept
{
    if ((nullptr == data) || (0U == dataLen)) { return 0U; }
    std::uint16_t checksum{};
    for (std::size_t i{}; i < dataLen; ++i) { checksum += data[i]; }
    return checksum;
}
```

En checksummas begränsningar, värda att vara tydlig med. En checksumma upptäcker bara fel som ändrar den aritmetiska summan. En enda vänd bit ändrar alltid summan, så enbitsfel fångas alltid. De riktiga svagheterna ligger någon annanstans: två hela bytes som byter plats lämnar summan oförändrad, och *vissa* kombinationer av flera vända bitar tar ut varandra och slinker igenom oupptäckta. CRC:er fångar en mycket bredare, matematiskt välförstådd klass av fel, och används därför av CAN i stället för en enkel summa. De introduceras i avsnitt 10.4 och implementeras i kapitel 13.

Det här är inte lika starkt som en CRC, men det räcker för att öva ramning, parsning och robust felhantering, vilket är vad den här introduktionen är till för.

Serialisering och deserialisering

Serialisering omvandlar strukturerad data till en följd av bytes (`std::uint8_t[]`) så att den kan skickas över ett gränssnitt. **Deserialisering** tolkar en mottagen bytearray och plockar ut den strukturerade datan igen: fälten valideras först (SOF, längd, typ, checksumma), och först när varje kontroll passerat plockas datan ut.

Att validera före utplockning spelar roll eftersom UART och RS-485 levererar bytes som en ström, så en mottagare rutinmässigt ser skräp mellan ramar, tappade bytes och förvanskade ramar.

I inbyggda system görs det här nästan alltid med C-arrayer (`std::uint8_t buffer[Size]`), pekare till de arrayerna vid anropsgränser (`std::uint8_t*`), och en explicit längd (`std::size_t`).

Båda riktningarna behöver samma hantering av byteordning på varje flerbytesfält, vilket är repetitivt och lätt att få subtilt fel. Två funktionsmallar tar bort upprepningen. Båda begränsar T med `std::is_unsigned`, så var de än hamnar behöver de `<type_traits>`:

```
#include <type_traits>

/**
 * @brief Write a value to a buffer, most significant byte first.
 *
 * The caller is responsible for the buffer being large enough.
 *
 * @tparam T The value type. Must be unsigned.
 *
 * @param[out] buf The buffer to write to.
 * @param[in] offset Value offset, i.e. the index at which to write the value.
 * @param[in] value The value to write.
 */
template <typename T>
void writeToBuf(std::uint8_t* buf, const std::size_t offset, const T value) ↪
    ↪ noexcept
{
    static_assert(std::is_unsigned<T>::value, "Failed to write to buffer: T must ↪
        ↪ be unsigned!");
    constexpr std::size_t len{sizeof(T)};
    constexpr std::size_t msb{len - 1U};
    constexpr std::size_t bitsPerByte{8U};

    for (std::size_t i{}; i < len; ++i)
    {
        const std::size_t shift{bitsPerByte * (msb - i)};
        buf[offset + i] = static_cast<std::uint8_t>((value >> shift));
    }
}

/**
 * @brief Read a value from a buffer, most significant byte first.
 *
```

```

* @tparam T The value type. Must be unsigned.
*
* @param[in] buf Buffer to read from.
* @param[in] offset Value offset in the buffer.
*
* @return The retrieved value.
*/
template <typename T = std::uint8_t>
[[nodiscard]] T readFromBuf(const std::uint8_t* buf, const std::size_t offset) noexcept
{
    static_assert(std::is_unsigned<T>::value, "Failed to read from buffer: T must be unsigned!");
    constexpr std::size_t len{sizeof(T)};
    constexpr std::size_t msb{len - 1U};
    constexpr std::size_t bitsPerByte{8U};
    T value{};

    for (std::size_t i{}; i < len; ++i)
    {
        const std::size_t shift{bitsPerByte * (msb - i)};
        value |= static_cast<T>(buf[offset + i]) << shift;
    }
    return value;
}

```

En fälla värd att namnge, eftersom kompilatorn inte fångar den. Antalet bytes som skrivs är `sizeof(T)`, härlett ur *argumentet*, inte ur fältet. Att skicka in en `std::size_t` där fältet är en byte skriver åtta bytes och skriver i tysthet över fälten som följer:

```

constexpr std::size_t payloadLen{0U};
writeToBuf(buf, FrameOffset::Len, payloadLen); //↪
    ↳ Writes 8 bytes (error).
writeToBuf(buf, FrameOffset::Len, static_cast<std::uint8_t>(payloadLen)); //↪
    ↳ Writes 1 byte.

```

Casta alltid till fältets egen bredd på anropsstället.

Från papper till kod

Precis den här ramen görs körbar i C++ under `lectures/L10/appendix/code/`: en `Frame`-struct som serialiserar sig själv till en bytebuffert, deserialiserar sig själv tillbaka, och förkastar allt med felaktig SOF, längd, typ eller checksumma. Att serialisera PING:en ovan producerar

exakt de bytes som räknades ut ovan, `A5 F7 00 00 02 01 00 20 01 BF`. `comm/def.hpp` tillhandahåller redan fältoffseten och fältlängderna, så du behöver aldrig räkna bytepositioner för hand.

Att göra typ-ID:na ovan till en C++-uppräkning, deklarerera structen och implementera båda metoderna själv är Övning 10.2. En uppsättning enhetstester följer med koden, i `test/frame_test.cpp`: de kontrollerar båda metoderna mot ramarna som räknades ut på papper, byte för byte. Du skriver dem inte; make kör dem, och din uppgift är att få dem att passera.

Vad som kommer härnäst

En fråga från början av det här avsnittet står fortfarande öppen: **vad händer om två noder sänder samtidigt?**

Ingenting i det här ramformatet besvarar den. Ett synkord kan inte hjälpa, eftersom båda sändarna skickar ett; en checksumma kan inte hjälpa, eftersom den bara rapporterar skadan i efterhand. Att besvara den kräver ett annat slags protokoll, ett där bussens *elektriska* beteende avgör vem som fortsätter och vem som faller ifrån, innan någon data går förlorad. Det protokollet är CAN.

10.2 Introduktion till CAN

CAN konstruerades för fordons- och industrisystem: många oberoende styrenheter (motor, bromsar, instrumentpanel, sensorer) som utbyter korta meddelanden tillförlitligt, utan någon enskild felpunkt.

Punkt-till-punkt-kablage slutar skala nästan omedelbart. Det ramade protokollet i avsnitt 10.1 förutsatte en enda delad länk; en styrenhet som pratar med en instrumentpanel, en ABS-enhet och tre sensorer skulle behöva fem separata länkar, var och en med eget kablage, egna kontakter och egen transceiver. En enda delad buss ersätter allihop: varje nod kopplas in på samma ledningspar, och varje nod kan både lyssna och sända på den.

Den fysiska bussen

En riktig CAN-buss är ett tvinnat differentialpar, `CAN_H` och `CAN_L`. En nod signalerar en bit genom att driva isär de två ledningarna (**dominant**) eller låta dem flyta ihop genom ett motstånd (**recessiv**). Ett externt transceiverchip sköter det analoga lagret.

Den här boken arbetar helt ovanför det: ett logiskt värde per bit, 0 för dominant och 1 för recessiv. Det är exakt vad en transceiver presenterar för, och tar emot från, den digitala logiken på vardera sidan.

Dominant och recessiv

Hela arbitreringsschemat vilar på en enda egenskap:

- **Recessiv (1)** är bussens viloläge. Varje nod släpper bussen och transceivrarernas förspänningsnät drar båda ledningarna till samma nivå.
- **Dominant (0)** hävdas genom att aktivt driva isär ledningarna. Om *någon* nod driver dominant medan resten är recessiva läser bussen dominant, oavsett hur många noder som är inblandade eller vilken som hävdade det först.

Det är precis en **wired-AND**: betrakta varje nuds utgång som en ingång till en busstäckande AND-grind, och 0 dominerar. Riktiga transceivrar implementerar det i hårdvara med öppen dränering; systemtestbänken i kapitel 17 modellerar identiskt beteende digitalt genom att kombinera varje nuds par (`tx_bus`, `bus_en`) på samma sätt.

Multimaster, och varför det kräver arbitrerings

De flesta inbyggda bussar besvarar frågan "vem får sända?" med dekret: SPI har en enda master, och I2C, som visserligen definierar en wired-AND-arbitrerings ungefär som den nedan, körs nästan alltid med en enda master den också. CAN gör multimaster till normalfallet: **vilken nod som helst får sända så snart bussen är ledig**, och om två startar i samma ögonblick avgör bussen själv vem som fortsätter. Det finns ingen omsändningsfördröjning och en kollision är inget fel alls. Det här är CAN:s signaturidé, **arbitrerings**.

Mekanismen i korthet; det genomräknade exemplet kommer i avsnitt 10.6:

1. Varje meddelande börjar med sin identifierare, sänd bit för bit, MSB först.
2. Varje sändande nod övervakar också bussen medan den sänder varje identifierbit.
3. Så länge biten den sände stämmer med det bussen läser fortsätter den sända.
4. I det ögonblick en nod sänder recessivt (1) men observerar dominant (0) sänder någon annan nod en 0 där den själv sände en 1. Den har förlorat arbitrerings: den slutar sända omedelbart och blir mottagare. Noden som fortsatte driva dominant vinner, utan att ha märkt någonting.

Två följer av det:

- **Lägre identifierare vinner alltid**, eftersom de bär fler inledande dominanta bitar. Därav "prioriteter" i CAN-folkloren.
- **Varje identifierare måste vara unik**. CAN lägger det kravet på nätverkskonstruktören; kontrollern upprätthåller det inte i hårdvara.

En förenkling att flagga tidigt. Steg 4 beskriver en *produktionsfärdig* kontrollern, och att sömlöst bli mottagare mitt i en ram är en del av varför riktig CAN behandlar en kollision som en icke-händelse. Den här bokens `can_controller` (kapitel 16–17) slutar driva i exakt samma ögonblick, vilket är den del som spelar roll för bussen, men överger sedan ramen och sätter `error` i stället för att avkoda vinnarens meddelande. Avsnitt 19.7 återkommer till det i sin lista över skillnader mot en produktionsfärdig kontrollern.

Kapitel 1 i *CAN: bussen, ramen och kontrollern* tar bussen på egen hand: topologin, den dominanta och recessiva asymmetrin, wired-AND:et, de två termineringsmotstånden, och sambandet mellan bithastighet och busslängd — 1 Mbit/s räcker ungefär 40 m, och skälet är arbitreringen, som kräver att en bit hinner ut till bussens bortersta nod och tillbaka innan biten är slut.

10.3 Broadcast, inte adressering

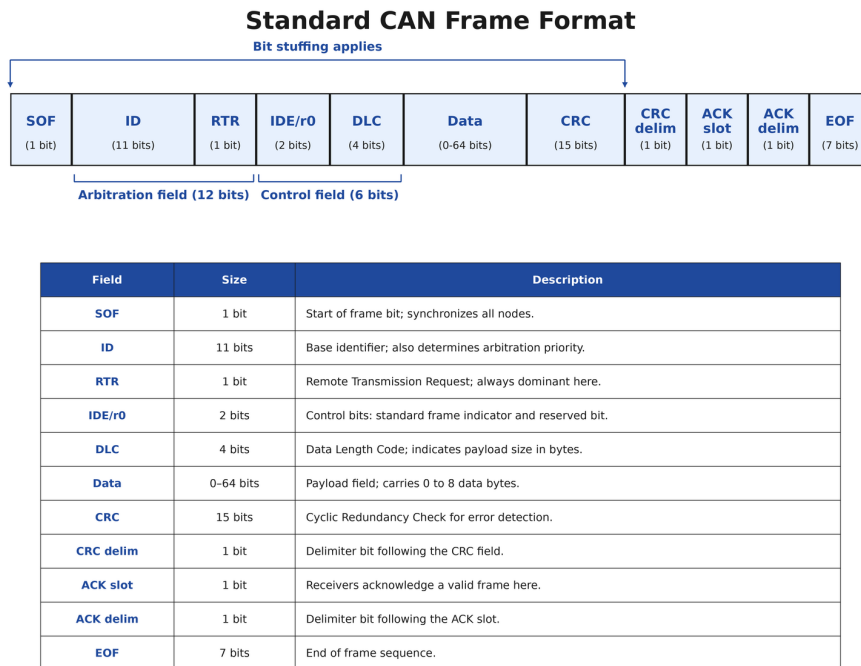
Det ramade protokollet i avsnitt 10.1 behövde DST och SRC eftersom fler än en nod kunde lyssna. CAN står inför samma problem på samma sorts buss och besvarar det helt annorlunda: **en CAN-ram bär ingen destinationsadress alls.**

Varje nod ser varje ram. Identifieraren som vinner arbitreringen namnger ingen mottagare; den namnger *sortens meddelande*, utsänt till alla och relevant för den som bryr sig. Varje nod avgör själv om en identifierare betyder något, typiskt genom att jämföra den mot en liten uppsättning den är intresserad av, i mjukvara eller i kontrollerns filtreringshårdvara. En nod som inte är intresserad kastar helt enkelt ramen; ingenting på protokollnivå styrde undan den.

Exempel. Anta att en konstruktion reserverar identifierare enligt konvention, $StatusReq = 0x300 + nodeId$ och $StatusResp = 0x400 + nodeId$. Sensornod 3 tillfrågas om status på $0x303$ och svarar på $0x403$. Varje nod tar emot ramen $0x303$; bara den som känner igen den som sin egen förfrågningsidentifierare svarar. Inget fält sa "det här är till nod 3".

Det är ett genuint annorlunda designval. Protokollet i avsnitt 10.1 skalar adressering genom att bredda DST och SRC när antalet noder växer; CAN skalar genom konvention för hur identifierare tilldelas och filtreras, och det finns inget adressfält att bredda.

10.4 CAN-ramformatet



Figur 10.2 CAN:s standarddataram, fält för fält, i sändningsordning.

Den här boken implementerar CAN:s **standarddataram (basformatet)**, bit för bit som standarden definierar den: en 11-bitars identifierare och upp till 8 databytes. Fjärramar, utökade 29-bitars identifierare och allt standarden definierar *runt* dataramar (fel- och överlastramar, och den 3-bitars intermission som skiljer en ram från nästa) ligger utanför bokens ram, men varje ram den här kontrollern lägger ut på ledningen är en fullt standardenlig dataram. Det är samma förenkling som riktiga konstruktioner ofta utgår från, och den räcker gott och väl för att bygga en genuin, fungerande kontrollern.

Diagrammet ger sändningsordningen och varje fälts bredd. Bitstoppning gäller varje fält från **SOF till och med CRC-fältet**, och inte CRC-avgränsaren, ACK-fältet eller EOF.

Fält för fält

SOF, 1 bit, alltid dominant. Sänds ut på en i övrigt ledig, recessiv buss, så den producerar den enda recessiv-till-dominant-flank varje nod väntar på. Den flanken markerar både att en

ram börjar och resynkroniserar varje nods bittajming mot den; `bit_timer.resync` i kapitel 12 är precis det här.

Identifierare, 11 bitar, MSB först. Fungerar samtidigt som meddelandets arbitreringsprioritet. Varje sändande nod jämför varje bit den sänder mot vad bussen faktiskt läser, och faller ifrån i det ögonblick den sänder recessivt men observerar dominant.

RTR, 1 bit, alltid dominant här. Remote Transmission Request-biten avslutar det 12 bitar långa arbitreringsfältet: dominant markerar en dataram, recessiv en *fjärram*, en begäran om att den som äger identifieraren sänder sin data. Den här boken stöder bara dataramar, så dess kontroller sänder alltid RTR dominant, och en mottagare som samplar den recessiv överger ramen och sätter error (kapitel 17 implementerar den kontrollen). Formellt spänner arbitreringen över identifieraren och RTR; i riktig CAN är det det som låter en dataram slå en fjärram med samma identifierare. När varje nod här sänder RTR dominant avgör identifieraren alltid ensam.

Kontrollfält, 6 bitar: IDE, r0, DLC. IDE är alltid 0 här; 1 skulle markera en ram med utökad identifierare, utanför bokens ram. r0 är reserverad, alltid 0. DLC är det 4 bitar breda längdfältet, 0–8 bytes kodade som 0000–1000. Det är längdfältet från avsnitt 10.1 i ny skepnad: utan det skulle en mottagare inte kunna avgöra om datafältet slutar efter den här byten eller fortsätter.

Datafält, 0 till 8 bytes. Sänds MSB först, byte 0 först, med den längd DLC annonserade. När DLC är 0 saknas fältet helt och CRC:n följer direkt på kontrollfältet.

CRC-fält, 15 bitar plus en avgränsare. CRC:n täcker varje bit från SOF till och med datafältets slut, och är långt mer robust än den summerade checksumman i avsnitt 10.1. En detalj spelar roll längre fram: den täcker bara de *riktiga* bitarna. Stoppbitar sätts in efter att CRC:n beräknats på vägen ut och kastas innan den kontrolleras på vägen in, så de kommer aldrig in i beräkningen (kapitel 14 och 15 är där den grindningen byggs). Kapitel 13:s motor beräknar den bitseriellt, en bit i taget medan ramen sänds eller tas emot, utan något separat "beräkna checksumman"-steg på slutet.

CRC-avgränsaren är den 16:e biten, alltid recessiv och aldrig stoppad. Den finns för att en bit som är oberoende av stoppbitsregeln alltid ska följa på CRC-värdet, vilket är det som låter en mottagare hitta gränsen mellan CRC- och ACK-fälten.

ACK-fält, 2 bitar. Sändaren sänder båda recessiva. Varje *annan* nod som tog emot ramen korrekt (ramningen intakt, stoppningen respekterad, CRC:n giltig) drar ACK-luckan dominant. Sändaren kontrollerar aldrig sin egen CRC mot sig själv, så den dominanta biten från någon annan är dess enda bekräftelse på att ramen kom fram intakt någonstans. **ACK-avgränsaren** är recessiv och ostoppad, och markerar gränsen av samma skäl som CRC-avgränsaren.

EOF, 7 recessiva bitar, ostoppade. Sju identiska bitar i rad är långt fler än stoppbitsregeln någonsin skulle tillåta inuti ett stoppat fält, vilket är precis det som gör EOF omisskännligt. Det håller bara om varje nod är överens om att inte stoppa det, vilket är varför EOF och de två avgränsarna definieras som undantagna från stoppning.

Kapitel 2 i *CAN: bussen, framen och kontrollern* går igenom samma ram fält för fält, och tar med det som ligger utanför den här bokens kontroller: fjärramarnas RTR-semantik, vad ACK-luckan betyder för en sändare som läser tillbaka den, och de tre bitars intermission som skiljer en ram från nästa.

10.5 Bitstoppnings

Regeln. Genom varje stoppat fält, SOF till och med CRC-fältet: efter **fem bitar i följd med samma värde** sätter sändaren in en extra bit med *motsatt* värde. Den biten bär ingen data, och varje mottagare kastar den genom att räkna på samma sätt.

Varför den finns, av två besläktade skäl:

1. **Klockåtervinning.** Varje nods bittimer går på sin egen oscillator. En lång följd av identiska bitar innehåller inga övergångar, så en mottagare har ingenting som bekräftar att den fortfarande ligger i linje med sändarens bitgränser. Att garantera en övergång minst var femte riktig bit begränsar hur långt dess tajming kan driva. Riktig CAN resynkroniserar på de flanker bitstoppningsen garanterar; den här bokens kontroller är enklare och resynkroniserar bara vid SOF (`bit_timer.resync`, kapitel 12), vilket gör den begränsade driften ännu viktigare.
2. **Att skilja riktig data från EOF.** EOF är sju identiska recessiva bitar, ostopgade. Ett stoppat fält kan aldrig av misstag producera den följd, vilket är det som gör sju ostopgade recessiva bitar till ett otvetydigt ramslut snarare än något identifieraren eller datafältet kunde ha producerat av en slump.

Genomräknat exempel. För att sända den 8 bitar långa sekvensen 11111000 slår regeln till efter den femte 1:an i följd: sätt in en 0, fortsatt sedan med den riktiga datan. Varje kolumn nedan är en bit som faktiskt ligger på ledningen, så raden med riktiga bitar lämnar en lucka där stoppbiten sitter:

Riktiga bitar:	1	1	1	1	1	-	0	0	0
Sänd bitström:	1	1	1	1	1	0*	0	0	0
Antal i följd:	1	2	3	4	5	1	2	3	4

^ stoppbit - inte riktig data

Läs antalsraden mot den sända raden, inte mot raden med riktiga bitar: regeln räknar bitar på ledningen. Stoppbitten är själv den första biten i den nya följd, vilket är varför antalet står som 1 under den och 2 under den första riktiga 0 som följer. Kapitel 14 bygger precis den här räknaren i hårdvara.

En mottagare räknar samma fem 1:or, förväntar sig att nästa bit är en stoppbit, och kastar den om den inverterar följd. Dyker det i stället upp en *sjätte* 1 i följd där stoppbiten skulle ha suttit är det en stoppningsöverträdelse: ett detekterat ramfel, behandlat som fatalt, rapporterat av `rx_shift_reg.stuff_error` i kapitel 15.

En finess värd att nämna redan nu, eftersom den överraskar folk senare. Räkningen av bitar i följd nollställs *inte* vid fältgränser. En följd som börjar i ett fält och fortsätter in i nästa räknas rakt igenom, och kan utlösa en stoppbit precis vid gränsen. Stoppningen bryr sig bara om bitarna som faktiskt ligger på ledningen, aldrig om vilket namngivet fält de tillhör.

Kapitel 3 i *CAN: bussen, framen och kontrollern* härleder femregeln ur klockåtervinningsproblemet och tar CRC-15:an i samma kapitel, av det skäl kapitel 13 och 14 sedan lever med: stoppningen och checksumman delar bitström men inte bitar, och den som förstår den ena utan den andra får fel CRC.

10.6 Arbitrering, genomräknad i sin helhet

Noderna P (identifierare 0x0F0) och Q (identifierare 0x100) börjar sända vid samma SOF. Båda sänder SOF som 0, så ingen strid uppstår än. Sedan identifierarna, MSB först:

bit:	1	2	3	4	5	6	7	8	9	10	11
P (0x0F0) sänder:	0	0	0	1	1	1	1	0	0	0	0
Q (0x100) sänder:	0	0	1	-	-	-	-	-	-	-	-
bussen läser:	0	0	0	1	1	1	1	0	0	0	0

(förlorar vid bit 3, slutar driva)

Bitarna 1–2 stämmer för båda noderna, så ingenting händer. Vid bit 3 sänder P dominant och Q recessivt, så bussen läser dominant. Q, som övervakar medan den sänder, ser sin egen 1 mot en buss som läser 0: **Q har förlorat arbitreringen** och slutar driva omedelbart. P ser 0 på en buss som läser 0, exakt vad den förväntade sig, och fortsätter utan att någonsin få veta att en strid ägde rum.

I en produktionsfärdig kontroller skulle Q nu ta emot resten av P:s ram. Den här bokens can_controller slutar driva i samma ögonblick men överger ramen, som flaggades ovan och som avsnitt 19.7 återkommer till.

Mekanismen generaliserar till godtyckligt antal samtidiga sändare: den numeriskt lägsta identifieraren vinner (likvärdigt: den längsta följd av inledande dominanta bitar), och varje förlorare faller ifrån i det ögonblick dess recessiva bit övertrumfas, aldrig senare än exakt den biten.

Kapitel 4 i *CAN: bussen, framen och kontrollern* räknar igenom både tvånods- och trenodsfallet, och tar sedan de två frågor det här kapitlet inte ställer: vad svält och prioritetsinversion betyder på en buss där låga identifierare alltid vinner, och vad en sändare aldrig får veta om varför den förlorade.

10.7 Bittajming och sampelpunkten

Arbitrering fungerar bara om varje nod är överens om *när* en bit ligger på bussen, vilket väcker en fråga: vid vilken punkt inom en bitperiod läser en nod den?

Varje nod delar upp sin egen klocka i bitperioder med fast längd med hjälp av sin egen lokala bittimer. Eftersom de oscillatorerna är oberoende, och eftersom signaler tar tid på sig att utbreda sig längs bussen, samplar en nod **inte** vid periodens början. Den väntar till en **sampelpunkt** längre in i perioden, då den sända nivån hunnit stabilisera sig.

Den här boken samplar 70 % in i varje bitperiod, ett rimligt och enkelt val för en kort buss på ett enda kort vid 1 Mbit/s.

Riktiga CAN-kontrollrar gör sampelpunkten konfigurerbar och delar upp bitperioden i namngivna segment (utbredningssegment, fasssegment) avstämda mot busslängd, utbredningsfördröjning och oscillatortolerans. Det ligger utanför den här bokens fasta sampelpunkt, men inte utanför kapitel 5 i *CAN: bussen, ramen och kontrollern*, som delar upp bitperioden i sina segment, räknar ut tajmingen ur en klockfrekvens och en bithastighet, och beskriver den resynkronisering vid varje flank som den här kontrollern ersätter med en enda puls vid SOF.

Avsnitt 10.8 gör om de 70 % till den konkreta VHDL-konstanten `SAMPLE_TICK`, härledd ur klockfrekvensen och bithastigheten i stället för hårdkodad.

10.8 Paketet `can_def`

Varför använda ett VHDL-paket?

- Del I behövde aldrig något: varje exempel där var självständigt, och där en övning återanvände en modul du redan skrivit — `reset_sync`, `button_sync`, `timer` — kopierade du filen till den katalog som behövde den.
- Det här projektet är annorlunda. `TICKS_PER_BIT`, `SAMPLE_TICK`, `ID_WIDTH`, `CRC_WIDTH` och fler behövs var och en av *flera* av blocken som byggs från kapitel 12 och framåt, och de måste stämma exakt överens, överallt, annars går designen sönder i tysthet. Så mycket delad information är värd att definiera en gång.
- Varje konstant i det kommer från något det här kapitlet just lärt ut: fältbredderna är de du lägger ut för hand i övningarna, och `SAMPLE_TICK` är den 70 %-punkt avsnitt 10.7 resonerade om. Det här paketet är där de siffrorna slutar vara prosa och blir den enda källan till de definitioner varje modul använder.
- Det ändrar också upplägget av *filerna*. Varje modul här är del av **en enda** design, så de bor allihop i `controller/` och läser den enda `can_def.vhd` bredvid sig. Ingenting dupliceras, och det finns exakt en fil att ändra när en konstant ändras.

Att läsa `can_def.vhd`

Live-kodad tillsammans under passet, och skriven **en gång**, till `controller/can_def.vhd`. Det här är den första VHDL boken skriver. Varje modul du skriver från kapitel 11 och framåt hamnar i samma katalog, och alla utom `meta_prev` läser den här enda filen.

Bittajmingskonstanterna, härledda snarare än handplockade, så att de förblir konsekventa med varandra:

```

constant CLOCK_FREQ_HZ: natural := 50_000_000;           -- System clock ↪
    ↪ frequency: 50 MHz.
constant BIT_RATE_HZ   : natural := 1_000_000;           -- CAN bit rate: 1↪
    ↪ Mbit/s.
constant TICKS_PER_BIT : natural := CLOCK_FREQ_HZ / BIT_RATE_HZ; -- Clock ticks per↪
    ↪ CAN bit: 50.
constant SAMPLE_TICK   : natural := (TICKS_PER_BIT * 7) / 10; -- Sample point: ↪
    ↪ 35 (70%).

```

TICKS_PER_BIT måste gå jämnt ut för att den här enkla designen ska träffa en exakt bithastighet: 50 000 000/1 000 000 är exakt 50. SAMPLE_TICK beräknas medvetet som en *procentandel* av TICKS_PER_BIT, inte som ett hårdkodat ticknummer, så att en senare ändring av bithastigheten inte i tysthet lämnar sampelpunkten fel. Det här är avsnitt 10.7:s 70 %-sampelpunkt, förvandlad till ett tal som `bit_timer` kan räkna till i kapitel 12.

Ramfältens bredder, direkt från ramdiagrammet i avsnitt 10.4:

```

constant ID_WIDTH   : natural := 11;           -- Standard CAN identifier width in ↪
    ↪ bits.
constant DLC_WIDTH  : natural := 4;           -- Data Length Code field width in ↪
    ↪ bits.
constant CRC_WIDTH  : natural := 15;           -- CAN CRC-15 sequence width in bits.
constant DLC_MAX    : natural := 8;           -- Maximum payload length in bytes.
constant DATA_WIDTH: natural := DLC_MAX * 8; -- Maximum data-field width in bits.

```

Stoppbitsregelns följdlängd, från avsnitt 10.5:s regel om fem identiska bitar:

```

constant MAX_RUN: natural := 5; -- Maximum run of identical bits before a stuff ↪
    ↪ bit is inserted.

```

Båda skiftregistren räknar följdär mot det här enda talet (kapitel 14, 15): sändaren för att avgöra när en stoppbit måste *sättas in*, mottagaren för att förutsäga var en måste *dyka upp*. Att länkens båda ändar är överens om det exakt är hela det här paketets argument i miniatyr — en definition, flera moduler, ingen möjlighet att glida isär.

Gemensamma subtyper, så att varje modul namnger likadant formade signaler på samma sätt:

```

subtype byte_t is std_logic_vector(7 downto 0);           -- 8-bit byte.
subtype id_t   is std_logic_vector(ID_WIDTH-1 downto 0); -- Standard CAN ↪
    ↪ identifier.
subtype dlc_t  is std_logic_vector(DLC_WIDTH-1 downto 0); -- Data Length Code ↪
    ↪ field.
subtype crc_t  is std_logic_vector(CRC_WIDTH-1 downto 0); -- CAN CRC-15 sequence.
subtype data_t is std_logic_vector(DATA_WIDTH-1 downto 0); -- CAN data field.

```

Varje modul utom `meta_prev` börjar med `use work.can_def.all;` och använder de här namnen direkt: `bit_timers` räknarintervall, `crc15s` registerbredd, och `tx_shift_reg/rx_shift_regs` byteportar. Allt går tillbaka till den här enda filen.

CRC-15-polynomet, som en 15 bitar bred konstant. Kapitel 13:s `crc15` använder det direkt; matematiken tas upp där, inte här. Behandla det tills vidare som en fast standardkonstant som varje CAN-kontroller använder:

```
constant CRC_POLY: crc_t := "100010110011001";
```

Den motsvarar $x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$.

Vad som kommer härnäst

Kapitel 11 avbildar det här kapitlets protokoll på en blockarkitektur och skriver kontrollernas toppnivå, `can_controller.vhd`, vars portar typas ur det här paketet (`id_t`, `dlc_t`, `data_t`). Den introducerar också simuleringsflödet och lämnar den toppnivåns arkitektur tom. Kapitel 12 bygger de två första blocken som ska in i den, `meta_prev.vhd` och `bit_timer.vhd`.

10.9 Sammanfattning

Det här ska du kunna nu

- Säga varför kommunikationsprotokoll använder ramar, checksummor och arbitrering.
- Beskriva CAN-ramens format fält för fält, och konstruera en komplett ram för hand.
- Förklara hur wired-AND-arbitrering löser upp konkurrens utan bussmästare, och avgöra vilken av två noder som vinner.
- Tillämpa stoppbitsregeln på en bitföljd, för hand.
- Skriva `can_def.vhd`: konstanterna, subtyperna och CRC-polynomet som resten av projektet läser.
- Redogöra för gruppens arbetssätt: branchning, PR:er och granskning.

Frågor att testa dig själv med

1. Varför behöver ett protokoll ett längdfält när ramen ändå har en tydlig början?
2. Varför har CAN ingen destinationsadress, och vad använder den i stället?
3. Två noder börjar sända samtidigt. Vilken vinner, och vad gör förloraren?
4. Varför måste en sändare stoppa in en bit efter fem lika i rad, och varför gäller regeln inte hela ramen?
5. Vad händer med CRC-beräkningen för en instoppad bit?
6. Varför ligger sampelpunkten runt 70 % in i bitperioden i stället för i mitten?

Riktvärde

`can_def.vhd` bör vara skriven och mergad när passet är slut. Det är en kort fil och resten av projektet läser den, så den är värd att göra klar direkt.

Riktvärdena i de här kapitlen är just riktvärden. De är inte milstolpar, de betygsätts inte, och gruppen bestämmer själv i vilken ordning modulerna byggs; se bilaga C.

Blicken framåt

Kapitel 11 går från protokoll till blockschema: kontrollerns arkitektur, den toppnivå varje senare modul instansieras i, registerkartan som drivrutinsklassen ska konsumera, och projektets simuleringsflöde i GHDL.

10.10 Övningar

Övningarna är upplagda så här. Övning 10.1 och 10.2 hör till avsnitt 10.1: den första bygger ramar för hand, på papper, och den andra gör samma ram till en fungerande C++-implementation. Övning 10.3 slår bron mellan det ramformatet och CAN. Övning 10.4 till 10.8 befäster avsnitt 10.2 till 10.7; de görs på papper och bygger upp till att sätta ihop en hel ram för hand. Övning 10.9 skriver paketet `can_def` och arbetar sedan mot det vid tangentbordet.

ÖVNINGSGRUPP 1

Ramar för hand och i C++

Övning 10.1 Att konstruera två ramar för hand

RESONEMANG

Utgå från ramstrukturen ur avsnitt 10.1:

Fält	Storlek	Beskrivning
-----	-----	-----
SOF	2 byte	0xA5F7
LEN	1 byte	Nyttolastens längd
TYPE	1 byte	Meddelandetyp
DST	1 byte	Destinationsadress
SRC	1 byte	Källadress
SEQ	2 byte	Sekvensnummer
DATA	N byte	Nyttolast
CHK	2 byte	Checksumma (summan av föregående byte)

Checksumman är summan av varje byte från SOF till och med den sista nyttolastbyten, som en `std::uint16_t` (så den slår runt vid overflow). Fält bredare än en byte (SOF, SEQ, CHK) är

big-endian. Ramtyperna är Ping = 0x00, Pong = 0x01, StatusReq = 0x02 och StatusResp = 0x03.

En enhet på adress 0x17 skickar en statusförfrågan till en sensor på adress 0x25, med sekvensnummer 0x7F05. Det ger TYPE = 0x02, DST = 0x25, SRC = 0x17, SEQ = 0x7F05, och en tom DATA.

a) Rita motsvarande StatusReq-ram på papper, och fyll i varje fält: SOF, LEN, TYPE, DST, SRC, SEQ och CHK. Räkna ut checksumman för hand.

b) Sensorn svarar med ett 16-bitars sensorvärde, 0x3201, buret i nyttolasten. Rita motsvarande StatusResp-ram på papper. Reglerna: TYPE ändras till StatusResp, DST och SRC byter plats, SEQ är oförändrat, och DATA är 0x3201 (2 byte). Räkna ut även den här ramens checksumma för hand.

Övning 10.2 Ramen i C++

C++

Implementera ramen i C++ som en struct `comm::Frame`. Din implementation valideras av en befintlig uppsättning enhetstester under `lectures/L10/appendix/code/`.

1. Undersök filstrukturen

- `test/frame_test.cpp` innehåller enhetstesterna som validerar din ramimplementation. Dem skriver du inte; du får dem att passera.
- `include/comm/def.hpp` innehåller ramfältens offset (indexpositioner), fältstorlekarna med mera. Uppräkningsklassen `comm::FrameType` ska implementeras här.
- `include/comm/frame.hpp` innehåller deklarationen av structen `comm::Frame`.
- `source/comm/frame.cpp` ska innehålla implementationsdetaljerna för structen `comm::Frame`, alltså dess metoddefinitioner.

2. Skapa ramtyperna

Implementera uppräkningsklassen `comm::FrameType` i `comm/def.hpp`, och ge var och en av de fyra typerna det id som listas i Övning 10.1. Lägg till en avslutande uppräknare `Unknown`, ett steg bortom den sista riktiga typen, så att en typ utanför intervallet kan avvisas vid deserialisering med en enda jämförelse.

3. Skapa ramstructen

Lägg till medlemsvariablerna för structen `comm::Frame` i `comm/frame.hpp`, där `@todo` markerar stället: `std::uint8_t data[MaxDataLen]`, `std::uint16_t seq`, `std::uint8_t dst`, `std::uint8_t src`, `std::uint8_t len` och `FrameType type`.

Värdeinitiera var och en med klammerparenteser: de numeriska medlemmarna med `{}`, och type med `{FrameType::Unknown}`, så att en default-konstruerad ram aldrig håller en typ som råkar vara giltig.

Bredderna är ingen smaksak. Enhetstesterna jämför en medlem mot en literal av fältets egen bredd genom en mall som härleder `en` typ ur båda argumenten, så att `len` deklarerat

som `std::size_t` eller ett `seq` deklarerat som `std::uint32_t` slösar inte bara utrymme; testerna slutar kompilera.

`serialize()` och `deserialize()` är **redan deklarerade** i den filen, dokumenterade och korrekt kvalificerade (`const noexcept` respektive `noexcept`). Låt de deklarationerna vara; du skriver deras definitioner i steg 4 och 5.

4. Implementera serialiseringsmetoden

Avsnitt 10.1:s `checksum16()`, `writeToBuf()` och `readFromBuf()` ingår **inte** i den utdelade koden, och båda metoderna nedan behöver alla tre. Kopiera in dem i `comm/frame.cpp`, i en anonym namnrymd ovanför metoddefinitionerna, så att de förblir privata för den filen:

```
namespace
{
    // checksum16(), writeToBuf() and readFromBuf() from the framing introduction go ↵
    ↵ here.
} // namespace
```

Två saker om den kopieringen, som kompilatorn båda berättar för dig först i efterhand:

- `writeToBuf()` och `readFromBuf()` är begränsade med `std::is_unsigned`, och ingenting i inkluderingsskedjan för `frame.cpp` drar in `<type_traits>`. Lägg till den inkluderingen högst upp i filen.
- Ha avsnitt 10.1:s varning i åtanke när du anropar dem: `writeToBuf()` skriver `sizeof(T)` byte härledda ur *argumentet*, inte ur fältet, så casta till fältets egen bredd vid varje anropsställe.

Implementera sedan metoden `comm::Frame::serialize()` så att den skriver följande till den givna bufferten: SOF = 0xA5F7 (big-endian), LEN, TYPE, DST, SRC, SEQ (big-endian), DATA i samma ordning som den ligger i ramens nyttolastbuffert, och CHK, beräknad över alla föregående fält, byte för byte.

Returnera ramens totala längd om serialiseringen lyckades, och 0U vid fel, vilket betyder en buf som är null eller en `bufLen` som är mindre än ramen som ska skrivas. Båda har ett eget test; ingen av dem är valfri.

5. Implementera deserialiseringsmetoden

Implementera i samma fil metoden `comm::Frame::deserialize()` så att den validerar den givna datan, **i den här ordningen**, eftersom varje kontroll gör nästa säker att utföra:

- buf är inte null, och `bufLen` är minst `MinFrameLen`. Ingenting har lästs ännu vid den här punkten, så det är det här som över huvud taget gör det tillåtet att läsa huvudet. Hoppa över det och läsningarna av SOF och LEN nedan springer utanför en för kort buffert innan någon annan kontroll får en chans.
- SOF är korrekt.
- LEN ligger inom gränserna, alltså högst `MaxDataLen`.

- Den givna bufferten rymmer **hela** ramen som huvudet utger sig för. Det här är en andra, striktare längdkontroll, och den första ersätter den inte: en buffert som kapats efter nyttolasten lämnar ändå *något* där CHK borde ligga, och att läsa det som checksumman får en trunkerad ram att se hel ut.
- Ramtypen är giltig, alltså att den mottagna byten ligger under Unknown.
- Checksumman stämmer.

Lagra den givna datan i ramen, men först när allt av den är giltigt.

Två av de punkterna är kontroller som testerna straffar hårdast. En saknad null-kontroll får inget test att fallera, den **kraschar hela körningen**, så du ser en segfault och inget testnamn. En buffertlängdskontroll som stannar vid huvudet faller `DeserializeRejectsShortBuffer` och `DeserializeRejectsTruncatedPayload` och ingenting annat, vilket är ett betydligt vänligare sätt att få det sagt.

6. Validera din implementation

Enhetstesterna ligger bakom ett makro, så ingenting körs förrän du slår på det. Lägg till `-DL01` i `CXX_FLAGS` i makefilen, där en `Todo`-kommentar markerar stället:

```
CXX_FLAGS := -Wall -Werror -std=c++17 -Iinclude -DL01
```

Bygg och kör dem sedan från en Linuxterminal:

```
cd lectures/L10/appendix/code
make
```

Tills du gör det rapporterar programmet att testerna är avstängda och avslutas med lyckat resultat, oavsett om din implementation fungerar eller inte.

Om bygget avbryts direkt med det här, är det inte din kod:

```
make[1]: *** No rule to make target 'lib'. Stop.
make: *** [Makefile:26: ../../../../libs/test/libqcademy_test.a] Error 2
```

Testramverket ligger som en submodul i `libs/test`, och en klon utan `--recurse-submodules` lämnar katalogen tom. Att den ändå *finns* är varför felet handlar om ett saknat make-mål och inte om en saknad katalog: det är ramverkets makefil som inte är hämtad än. Hämta den en gång, från repots rot, med `git submodule update --init`. Övningen behöver också en C++-kompilator.

Testerna kontrollerar båda metoderna mot ramarna du byggde för hand i Övning 10.1: PING-bytesen ur avsnitt 10.1 och båda ramarna ur den övningen, byte för byte; en rundtur genom serialisering och deserialisering som måste bevara varje fält; och avvisningsfallen (felaktig SOF, felaktig checksumma, ogiltig typ, trunkerad buffert, nullpekare).

Varje test som passerar skrivs ut i grönt; ett som fallerar skrivs ut i rött och namnger uttrycket, båda värdena och raden, så att du kan jämföra direkt mot din egen pappersuträkning. Räkna med att se alla fallera först, och gå gröna en grupp i taget medan du arbetar dig igenom steg 2–5.

Två saker som är värda att veta i förväg:

- Testerna läser `frame.type`, `frame.dst`, `frame.src`, `frame.seq`, `frame.len` och `frame.data` direkt, så de medlemsnamnen måste stämma. Har du döpt dina till något annat står det dig fritt att ändra testerna så att de matchar.
- `DeserializeLeavesFrameUntouchedOnFailure` ger `deserialize()` en korrupt buffert och kontrollerar sedan att ramens fält är *oförändrade*. En `deserialize()` som kopierar in fält i ramen medan den parsar dem passerar alla andra tester och faller det här.

ÖVNINGSGRUPP 2

Från ramformatet till CAN

Övning 10.3 Samma ram, fast som CAN

RESONEMANG

Ta den `StatusResp`-ram du byggde för hand i Övning 10.1 b), och skicka den som en CAN-ram i stället. Dess fält, som påminnelse: `TYPE = StatusResp`, `DST = 0x17`, `SRC = 0x25`, `SEQ = 0x7F05`, nyttolast 2 byte med värdet `0x3201`.

Utgå från konventionen i avsnitt 10.3: identifieraren kombinerar en bas per typ med id:t för den sensornod meddelandet gäller, alltså den nod som frågas i en förfrågan och den nod som svarar i ett svar (nod 3 frågas på `0x303` och svarar på `0x403`):

$$ID = TYPE_BASE + nodeId$$

Typbaserna är `StatusReq = 0x300`, `StatusResp = 0x400`, `Ping = 0x500` och `Pong = 0x600`.

- Skriv motsvarande CAN-ram på formen `ID | DLC | DATA`.
- Ange vilka fält i avsnitt 10.1:s ram som saknar motsvarighet i CAN-ramen. Säg för varje saknat fält vad CAN gör i stället, eller förklara varför CAN inte behöver något motsvarande fält.
- Anta att två noder börjar sända på CAN-bussen i exakt samma ögonblick. Förklara kort hur bussen avgör vilken nod som fortsätter sända, och hur den förlorande noden upptäcker att den har förlorat. Det här är arbitreringsmekanismen från avsnitt 10.6; återge den med egna ord.
- Förklara varför det inte finns något separat checksummefält att fylla i när man konstruerar en CAN-ram, till skillnad från avsnitt 10.1:s ramformat.

ÖVNINGSGRUPP 3

Bitstoppning, arbitrering och tajming**Övning 10.4 Bitstoppning för hand**

RESONEMANG

- a) Tillämpa stoppbitsregeln på följande 10-bitarssekvens:

```
0011111111
```

Skriv den kompletta sända bitströmmen, med alla ursprungliga bitar och varje instoppad stoppbit tydligt markerad.

- b) En mottagare observerar följande sända ström på 9 bitar:

```
101111101
```

Mottagaren vet att bitstoppning är i bruk. Avstoppa strömmen: identifiera eventuella stoppbitar, ta bort dem, och återskapa den ursprungliga följd av riktiga databitar.

- c) En mottagare observerar sex dominanta (0) bitar i rad på ett ställe där en stoppbit skulle ha brutit en följd av fem. Förklara vad mottagaren drar för slutsats, och vad den bör göra härnäst.

- d) Avsnitt 10.5 påpekar att räknaren för lika bitar i rad inte nollställs vid fältgränser. Anta att ett fält slutar med tre lika bitar och att nästa börjar med två till av samma värde. Använd regeln och säg var stoppbiten hamnar, räknat från fältgränsen, och förklara varför inget av fälten på egen hand skulle ha orsakat den.

Övning 10.5 Att avkoda en riktig bitström

RESONEMANG

En mottagare observerar följande ström på 21 bitar:

```
0 00000010101 0 00 0010 11
```

Strömmen är redan avstoppad, så att den här övningen håller fokus på fältens uppdelning i stället för på stoppningen. Mellanrummen finns bara med för läsbarhetens skull; den riktiga CAN-bussen har inga glapp mellan fälten.

- a) Dela upp strömmen i dess namngivna fält, med bredderna från avsnitt 10.4: SOF, ID, RTR, IDE, r0, DLC. Bestäm sedan identifieraren i hexadecimal form, DLC-värdet, och vilket fält som har börjat i de två bitar som blir över när alla namngivna fält är avräknade.
- b) Bestäm utifrån DLC:n från a) hur många databitar mottagaren förväntar sig totalt, och hur många ytterligare databitar som återstår innan CRC-fältet börjar.

c) Anta att DLC i stället hade varit 0000. Vilket fält skulle då följa omedelbart efter kontrollfältet?

Övning 10.6 Arbitrering mellan tre noder

RESONEMANG

Tre noder försöker sända vid samma SOF: nod R med ID 0x155, nod S med ID 0x154, och nod T med ID 0x1D5.

a) Arbeta igenom arbitreringen bit för bit, i samma stil som avsnitt 10.6:s genomräknade exempel. Bestäm vilken nod som vinner, och vid vilken identifierarbit varje förlorande nod faller ur.

b) De två förlorande noderna faller ur vid olika bitpositioner. Förklara varför genom att ange var varje förlorande nods identifierare först skiljer sig från vinnarens identifierare, och varför just den första avvikande biten avgör när noden förlorar arbitreringen.

c) En förlorande nod måste sluta driva bussen, och den intressanta frågan är *när*. (Kapitel 11 introducerar portparet `tx_bus/bus_en` som bokens kontroller slutar driva med, och kapitel 16–17 bygger logiken; här resonerar du på protokollnivå.) Förklara, med en eller två meningar, vad varje förlorande nods `can_controller` måste göra på alla första klockcykeln efter att den upptäckt avvikelsen, och varför den tajmingen spelar roll.

Ledtråd: Tänk igenom vad "ögonblicket" betyder i avsnitt 10.6:s genomräknade arbitringsexempel.

Övning 10.7 Resonemang om sampelpunkten

RESONEMANG

Den här boken samplar varje bit 70 % in i bitperioden, inte vid 0 % (allra först i perioden) eller 100 % (allra sist).

a) Förklara vad som skulle kunna gå fel om en mottagare samplade vid 0 %. Utgå från signalutbredning, varför en bitperiod behöver en minsta varaktighet, och om det sända värdet har hunnit stabilisera sig.

b) Förklara vad som skulle kunna gå fel om en mottagare samplade vid 100 %, på den sista möjliga klockpulsen. Gör förklaringen specifik för att det i bokens design gäller att varje nod har sin egen lokala bittimer, att timern omsynkroniseras bara vid SOF, och att den aldrig omsynkroniseras mitt i ramen (riktig CAN omsynkroniserar på flanker även mitt i ramen; bokens kontroller gör inte det).

c) Arbitrering kräver att varje konkurrerande nod läser bussen under *samma* bitperiod som den driver den. Förklara varför en sampelpunkt för nära periodens början är riskabel, varför en sampelpunkt för nära periodens slut är riskabel, och varför faran växer ju längre in i ramen man kommer.

Övning 10.8 Att konstruera en CAN-ram för hand

RESONEMANG

Övning 10.4 till 10.7 arbetade med enskilda regler. Den här sätter ihop en komplett ram utifrån ramdiagrammet i avsnitt 10.4.

En anmärkning om CRC:n. Du får CRC-15-värdet för varje ram nedan. Att räkna ut en 15-bitars CRC över drygt 30 bitar för hand är mekaniskt snarare än lärorikt; i kapitel 13 arbetar du igenom rekursionen ordentligt, över en 4-bitarssekvens, när motorn i sig är ämnet.

- a) Ta CAN-ramen du tog fram i Övning 10.3: ID = 0x425, DLC = 2, data 32 01. Skriv ut varje fält som bitar, i sändningsordning, med bredderna från avsnitt 10.4: SOF; ID (11 bitar, MSB först); RTR; IDE och r0; DLC; data (MSB först, byte 0 först); CRC (given: 010101100100011); CRC-avgränsare, ACK-plats, ACK-avgränsare och EOF.
- b) Hur många bitar blir det totalt, före all stoppning?
- c) Tillämpa nu stoppbitsregeln från Övning 10.4 över SOF till och med CRC-fältet, och kom ihåg att följdräknaren inte nollställs vid fältgränser. Hur många stoppbitar sätts in, och hur många bitar upptar ramen på ledningen?
- d) Vid bokens bithastighet på 1 Mbit/s, hur lång tid tar det att sända den ramen?
- e) Upprepa a) till d) för en ram **utan nyttolast**: ID = 0x100, DLC = 0, CRC 011100000001010. Notera vilka delar av ramen som är strukturellt oförändrade trots det saknade datafältet, och var CRC-fältet nu börjar.
- f) Jämför dina två svar på c). Den andra ramen bär ingen data alls, och ändå behöver dess stoppade område *fler* stoppbitar än den första. Förklara varför, utifrån vad stoppbitsregeln faktiskt reagerar på.

ÖVNINGSGRUPP 4

Att arbeta med paketet

Övning 10.9 can_def och dess härledningar

VHDL

Varje konstant i can_def går att spåra tillbaka till något det här kapitlet definierat. De här frågorna skriver paketet, kontrollerar att härledningarna håller när indata ändras, och utökar det med värden som avsnitt 10.4 namngav men som paketet ännu inte håller.

- a) Skriv controller/can_def.vhd, med konstanterna och subtyperna som avsnitt 10.8 ställer upp. Var du med på passet har du den redan; var du inte det är det här den kommer ifrån, och varje kapitel från kapitel 11 och framåt förutsätter att den finns.

Det här är den enda modulen i boken som delas ut i sin helhet i stället för som en specifikation, och det av ett skäl värt att nämna: ett paket innehåller inget beteende att

designa, så det finns ingenting mellan värdena och koden för dig att lista ut. Det nya här är själva VHDL-koden, eftersom en paketdeklaration är en konstruktion boken inte använt tidigare. Skriv av den för hand i stället för att klistra in den. Namnen är de kommande åtta kapitlens vokabulär, och att skriva dem en gång är billigare än att slå upp dem åtta gånger.

b) Anta att DE0-CV:ns klocka vore 40 MHz i stället för 50 MHz, med bithastigheten fortfarande 1 Mbit/s. Räkna om `TICKS_PER_BIT` och `SAMPLE_TICK` (fortfarande 70 %).

c) Anta i stället att klockan ligger kvar på 50 MHz men att bussen körs på 125 kbit/s, en verklig CAN-hastighet för "low speed". Räkna om båda konstanterna.

d) `TICKS_PER_BIT` är `CLOCK_FREQ_HZ / BIT_RATE_HZ`, med VHDL:s heltalsdivision för natural, som trunkerar. Ge ett par av klocka och bithastighet där divisionen *inte* går jämnt ut, och förklara med en mening varför det vore ett problem för den här designen, trots att VHDL-koden fortfarande skulle kompilera och elaboreras utan invändningar.

e) Lägg till en konstant `BIT_PERIOD_NS`, längden på en CAN-bitperiod i nanosekunder, härledd ur `BIT_RATE_HZ` i stället för hårdkodad. Ange sedan sambandet mellan den, `TICKS_PER_BIT` och 50 MHz-systemklockans periodtid på 20 ns, och bekräfta att de tre stämmer överens.

f) Avsnitt 10.4 anger CRC-avgränsaren, ACK-fältet och EOF som 1, 2 respektive 7 bitar, men paketet håller ingen av dem: `can_controller` kommer att använda de talen som nakna literaler. Lägg till `CRC_DELIM_BITS`, `ACK_BITS` och `EOF_BITS`, och säg vilken av de två formerna en senare läsare är bäst betjänt av.

g) Kontrollera att paketet faktiskt kompilarar. Installera GHDL om du inte redan gjort det (bilaga A), och kör sedan `make build-project` från roten av gruppens repo. Du bör se:

```
--> controller/can_def.vhd analyzes cleanly
```

följt av de två utdelade `bridge/-`-filerna, och därefter varje testbänk rapporterad som överhoppad, eftersom modulerna de kontrollerar inte finns ännu:

```
0 testbench(es) run, 8 skipped.
```

Gör nu sönder något med flit, ett saknat semikolon eller en felstavad typ, kör om, och läs vad GHDL säger om det. Ställ tillbaka det.

Det här är hela verifieringsberättelsen för ett paket: ett paket har inget beteende att simulera, så "den analyseras" är allt som finns att kontrollera. Kapitel 11 introducerar verktyget ordentligt, och därifrån får varje modul en testbänk också.

Arkitektur, toppnivå och registerkartan

Från protokoll till blockschema, och den toppnivå varje senare modul instansieras i.

I DET HÄR KAPITLET

- Att avbilda CAN-begrepp på hårdvarublock: bittimer, CRC-motor, skiftregister, och den ramsekvenserande tillståndsmaskin som styr dem.
- Vem som äger vad: `bit_timer` äger *när*, skiftregistren äger bitstoppningen, `crc15` vet ingenting om ramar, och `can_controller` är det enda block som känner till ramformatet.
- En första titt på registerkartan som drivrutinsklassen ska konsumera.
- Att skriva `can_controller.vhd` från grunden: dess femton portar, och en arkitektur som börjar tom och växer kapitel för kapitel.
- Vilka av portarna som är asynkrona, och varför kontrollern synkroniserar dem själv.

Kapitel 10 gav protokollet. Det här kapitlet gör det till en *arkitektur*: fem block med ett ansvar var, en entitet som håller ihop dem, och ett gränssnitt som ligger fast resten av boken. Ingen av delarna byggs här. Det som skrivs är entiteten de alla ska bo i, och dess arkitektur lämnas medvetet tom.

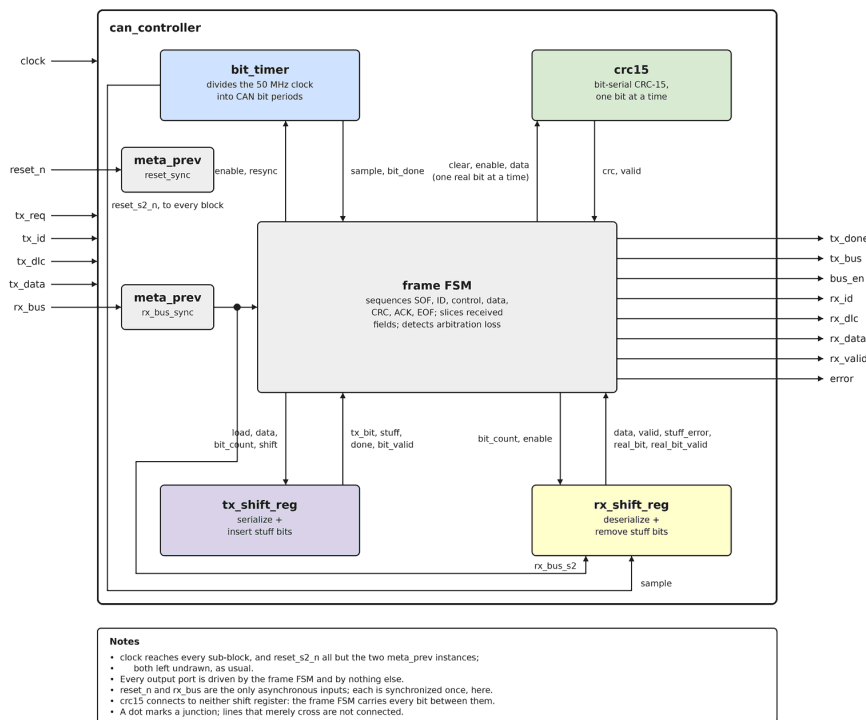
11.1 Kontrollerns arkitektur

Varje regel från kapitel 10 avbildas på ett av fem block:

- **bit_timer** äger ingenting om *vad* som sänds, bara *när*. Den är det enda i den här konstruktionen som ens gränsar till en klockdomänövergång: den frilöper kontinuerligt så snart den är aktiverad, och resynkroniserar exakt en gång per ram, vid SOF (avsnitt 10.5:s resonemang om klockåtervinning, tillämpat).
- **tx_shift_reg** och **rx_shift_reg** äger bitstoppningen och ingenting annat: de serialiserar eller deserialiserar upp till 8 riktiga bitar i taget och sätter transparent in eller tar bort

stoppbiter. De har ingen aning om huruvida bitarna de skiftar är ett ID, en databyte eller ett CRC-värde; det är `can_controllers` jobb.

- **crc15** äger CRC-15-beräkningen, en bit i taget, utan någon föreställning om "ramar" eller "fält" den heller; den ackumulerar bara den bit den får, när den blir tillsagd.
- **can_controller** är det enda block som känner till det faktiska CAN-ramformatet: det sekvenserar genom SOF, arbitrering, kontrollfältet, data, CRC, ACK och EOF, i ordning, driver de andra fyra blocken och läser deras utgångar. Detektering av förlorad arbitrering bor också här, eftersom den specifikt handlar om att jämföra vad den här kontrollern bestämde sig för att sända mot vad bussen faktiskt visar.



Figur 11.1 `can_controllers` blockarkitektur.

Två konventioner håller bilden läsbar. `clock` når varje delblock, och den synkroniserade `reset_s2_n` når alla utom de två `meta_prev`-instanser som producerar den och dess bussmot-svarighet; båda lämnas oritade, som brukligt i ett blockschema. Varje utgångs-port (`tx_done`, `tx_bus`, `bus_en`, `rx_id`, `rx_dlc`, `rx_data`, `rx_valid`, `error`) drivs av ramtillståndsmaskinen och av ingenting annat.

Tre av de elva interna kopplingarna är värda att läsa två gånger, eftersom det är de som folk ritar om fel ur minnet:

- De två linjerna vid `crc15` går i **motsatta** riktningar. `clear`, `enable` och `data` går upp in i den, en riktig bit i taget; `crc` och `valid` kommer tillbaka ner ur den. (`clear` är kapitel 13:s reset per ram — motorn måste börja varje ram på noll, och en avbruten ram skulle annars lämna rester efter sig; kapitel 16 driver den från `STATE_START`.)
- `rx_shift_reg` har **tre** linjer som anländer: `sample` från `bit_timer`, `rx_bus_s2` från synkroniseraren som kapitel 12 lägger till, och `bit_count` plus `enable` från ramtillståndsmaskinen. Utan de två första har den ingen datakälla alls.
- `tx_shift_reg` och `rx_shift_reg` kopplas aldrig till varandra, och ingen av dem kopplas till `crc15`. Ramtillståndsmaskinen bär varje bit mellan dem. `rx_shift_reg` är på samma sätt det enda delblock som över huvud taget läser bussen, och till och med det läser den synkroniserade kopian snarare än porten; se avsnitt 11.5.

Hur de nästlas. Diagrammet läses som en lagerbild, med det billigaste bekymret överst, men den faktiska VHDL-hierarkin är inneslutning, inte en pipeline: `can_controller` instansierar alla fyra av `bit_timer`, `crc15`, `tx_shift_reg` och `rx_shift_reg` inuti sig själv, vid sidan av den `meta_prev`-synkroniserare som kapitel 12 lägger till. `can_controller` är därför den enda entitet du någonsin instansierar för hand, och hela kontrollern är en komponent utifrån sett, exakt som registerbanken i kapitel 18 kommer att se den.

Två lager till sitter sedan ovanför den: `register_bank` (kapitel 18), som gör om de portarna på registernivå till den registerkarta en drivrutin pollar, och `can_spi_node` (kapitel 19), toppnivån som lägger till SPI-transporten och kortets pinnar. Båda hör till att nå kontrollern, inte till att tala CAN, vilket är varför de dyker upp i de två sista kapitlen snarare än här.

Den här uppdelningen — ett block per *ansvar*, inte ett block per *ramfält* — är medveten: `tx_shift_reg` återanvänds för ID:t, kontrollfältet, varje databyte och CRC-värdet, helt enkelt omladdat med olika data och olika bitantal varje gång, i stället för att skriva nästan identisk skiftlogik fem separata gånger.

11.2 Att följa en ram genom blocken

Att namnge de fem blocken är inte samma sak som att veta hur de samarbetar. Det här avsnittet tar en konkret ram, ID 0x123, DLC 2, databytesen A1 B2, och följer den genom arkitekturen ovan. Det finns ingen VHDL här; frågan är bara vilket block som gör vad, och vad som passerar mellan dem.

Först, det enda block som räknar tid. `bit_timer` delar upp 50 MHz-klockan i bitperioder och avger två pulser per period: `sample`, 70 % in i perioden, när nivån på bussen hunnit stabilisera sig, och `bit_done`, allra sist, när den aktuella biten är klar. Varje annat block agerar på en av de två pulserna, och inget av dem räknar klockcykler mot bussen själv. Sändning rör sig på `bit_done`, vid gränsen där en ny bit börjar. Mottagning rör sig på `sample`, mitt i

en bit som redan ligger där. Just den skillnaden är varför en mottagare läser det värde en sändare lagt ut, i stället för att fånga det mitt i en förändring.

Sändning: 60 bitar, 50 av dem genom ett register. Med fältbredderna från kapitel 10 är den här ramen 1 (SOF) + 11 (ID) + 1 (RTR) + 2 (IDE, r0) + 4 (DLC) + 16 (data) + 15 (CRC) = **50 bitar i det stoppade området**, följt av en ostoppad svans på $1 + 1 + 1 + 7 = 10$ bitar. Sextio bitar innan någon stoppbit satts in.

De 50 bitarna når bussen som **åtta laddningar av ett enda tx_shift_reg**:

#	Grupp	Bitar
1	SOF	1
2	Identifieraren, hög del	8
3	Identifieraren, låg del + RTR	4
4	IDE, r0, DLC	6
5	Databyte 0 (A1)	8
6	Databyte 1 (B2)	8
7	CRC, hög del	8
8	CRC, låg del	7

can_controller laddar om samma 8-bitarsregister åtta gånger, varje gång med ett annat värde och ett annat bitantal. Det är påståendet "ett block per ansvar" ovan, gjort konkret: identifieraren är inget fält för skiftregistret, den är grupp 2. De återstående 10 svansbitarna når aldrig något skiftregister alls; de är fasta recessiva nivåer, och can_controller driver dem direkt medan den räknar bit_done-pulser.

Inom en grupp händer fyra saker per bit:

- tx_shift_reg producerar nästa bit. Laddningen presenterar den första omedelbart; varje bit_done därefter skiftar ut nästa. Registret avgör också, på egen hand, om en stoppbit ska sättas in, och annonserar på stufv vilken sorts bit det här var.
- can_controller gör om biten till bussnivåer. Dominant betyder bus_en = '1' med tx_bus = '0'; recessiv betyder bus_en = '0', vilket släpper bussen. En nod driver aldrig recessivt aktivt (avsnitt 10.2:s wired-AND).
- crc15 matas med samma bit, men bara när den var en riktig. Stoppbitar hoppas över. crc15 har ingen aning om vad en stoppbit är; att bli tillsagd när den *inte* ska ackumulera är hela dess gränssnitt på den sidan.
- can_controller jämför rx_bus mot biten den just drev. Den jämförelsen är arbitreringen, och den stannar här snarare än i ett delblock eftersom den behöver både vad den här noden avsåg och vad bussen faktiskt visar, och can_controller är det enda block som håller båda.

Efter grupp 6 håller crc15 CRC:n över allt som sänts hittills. can_controller läser den och matar tillbaka den rakt in i tx_shift_reg som grupp 7 och 8. Lagg märke till vad det betyder: motorn som beräknar CRC:n och registret som sänder den kopplas aldrig till

varandra. `can_controller` bär värdet över, vilket är varför inget av blocken behöver veta att ett CRC-fält finns.

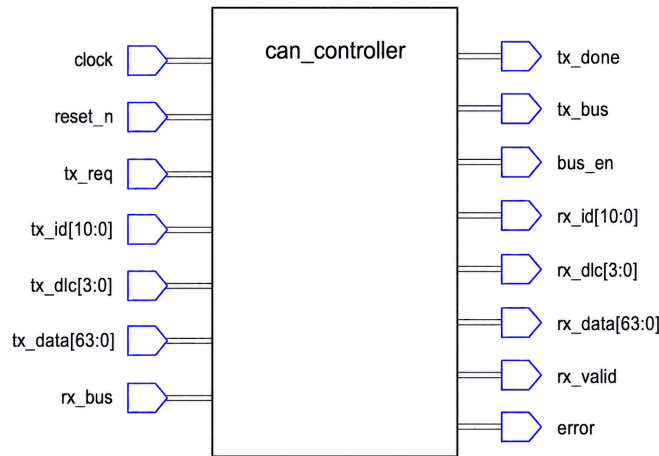
Mottagning: samma väg, speglad. `rx_shift_reg` samplar `rx_bus` på `bit_timer.sample`, tar bort stoppbitar och rapporterar varje riktig bit på `real_bit` och `real_bit_valid`. De två signalerna matar `crc15` precis som `tx_shift_regs` gör vid sändning: samma instans, samma regel om att bara ackumulera riktiga bitar, ingen lägesomkoppling någonstans (vilket är kapitel 13:s poäng om att en motor gör båda jobben). Fullbordade grupper anländer på data, och `can_controller` skivar tillbaka dem till `rx_id`, `rx_dlc` och `rx_data`. En stoppbit som uteblir höjer `stuff_error`, och `rx_shift_reg` stannar där: den rapporterar brottet utan att avgöra vad det betyder. Att avgöra är `can_controllers` jobb, och den bestämmer sig för att avbryta ramen och höja error.

Vad genomgången visar. Inget delblock vet vilket fält det hanterar. Vart och ett rapporterar fakta om *bitar*: en är klar, den där sattes in, en stoppbit saknades, den här gruppen är fullbordad. `can_controller` levererar varenda del av meningen. Tre saker följer, och alla tre är värda att ha:

- De fyra delblocken går att testa utan CAN. Deras testbänkar i kapitel 12 till 15 driver bitmönster och kontrollerar bitmönster; inte en enda av dem behöver en hel ram för att vara meningsfull.
- `can_controller` är det enda block som skulle ändras om ramformatet gjorde det. Utökade 29-bitars identifierare skulle till exempel betyda andra grupper, inte andra block.
- Varje koppling mellan block är en signal på en bit eller en byte med exakt en skrivare. Det finns ingen delad buss eller gemensam struktur som två block båda griper in i, vilket är det som håller kopplingsarbetet i kapitel 12 till 15 vid en port map i taget snarare än en omkonstruktion varje kapitel.

11.3 Kontrollernas gränssnitt

`can_controllers` portar är vad omvärlden ser, och de ligger fast resten av boken: kapitel 12 till 17 lägger till delblock *inuti* den här entiteten utan att ändra en enda port. Två saker kopplas till den, och varje port hör till en av dem.



Figur 11.2 can_controllers femton portar.

Bussidan, fem portar. Det är de portar som når omvärlden som fysiska pinnar; de tre första vetter mot CAN-transceivern:

- **rx_bus** är den aktuella bussnivån, en bit, direkt från transceivern och asynkron mot clock. Kontrollern synkroniserar den själv, med en `meta_prev`-instans som kapitel 12 bygger, så att kortets toppnivå kan koppla den här porten till en pinne och inte tänka mer på det. Kontrollern läser den för att ta emot, och också för att övervaka sina egna sändningar under arbitreringen.
- **tx_bus** är biten den här noden vill driva, och **bus_en** säger om den driver över huvud taget. Paret modellerar beteendet hos öppen dränering: `bus_en = '1'` med `tx_bus = '0'` drar bussen dominant, och `bus_en = '0'` släpper den så att en annan nod kan göra det. En nod driver aldrig recessivt aktivt; den släpper helt enkelt taget. Det är det som får avsnitt 10.2:s wired-AND att fungera.
- **clock** är 50 MHz-systemklockan, och **reset_n** en rå, aktivt låg reset, direkt från en pinne. Precis som **rx_bus** är den asynkron, och precis som **rx_bus** synkroniserar kontrollern den själv. Den synkroniserade kopian är en intern signal som heter `reset_s2_n`, namnet övningen `reset_sync` gav den i kapitel 4 och namnet varje delblock i projektet fortfarande tar.

Sändningssidan, fem portar. Det är dem en anropare fyller i för att sända en ram:

- **tx_id**, **tx_dlc** och **tx_data** bär själva ramen, typade som `id_t`, `dlc_t` och `data_t` från `can_def`. **tx_data** håller byte 0 i sina mest signifikanta bitar, i samma ordning som bussen sänder dem.

- `tx_req` begär sändning. Kontrollern agerar på den bara när den själv är i vila **och bussen också har varit ledig**, eftersom en nod inte får börja driva ovanpå en ram som redan pågår (kapitel 16). En begäran som höjs medan bussen är upptagen kastas i stället för att köas, så anroparen får begära igen.
- `tx_done` pulsar högt under en cykel så snart den här nodens egen ram har sänts färdigt.

Mottagningssidan, fem portar. De speglar sändningssidan exakt:

- `rx_id`, `rx_dlc` och `rx_data` håller den senast mottagna ramen, samma typer och samma byteupplägg som sina `tx_-motsvarigheter`.
- `rx_valid` pulsar högt under en cykel när en fullständig, giltig ram har anlänt.
- `error` är den enda port som *inte* är en puls: den går hög vid förlorad arbitrering, en stoppningsöverträdelse, en misslyckad CRC-kontroll eller en mottagen fjärram (en recessiv RTR-bit), och stannar hög tills nästa ram börjar. En anropare som pollar den då och då ser ändå felet.

Fyra saker med det här gränssnittet är värda att lägga märke till nu, eftersom de formar allt som följer:

- **Det finns ingen serialisering i det.** Anroparen lämnar över en identifierare, en längd och bytes. Varenda del av ramningen, stoppningen och CRC:n som kapitel 10 beskrev sker inuti, på vägen till `tx_bus`. Det är hela poängen med att bygga en kontroll.
- **Sändning och mottagning är symmetriska men inte samtidiga.** En nod sänder antingen sin egen ram eller tar emot någon annans, aldrig båda, så gränssnittets två halvor är aldrig aktiva samtidigt.
- **`tx_done`, `rx_valid` och `error` är hela statusgränssnittet.** Varje statusbit registerkartan erbjuder en drivrutin (kapitel 18) härleds ur någon av de tre.
- **Varje port på registersidan antas synkron mot clock.** `rx_bus` är undantaget, och det hanterar kontrollern själv med `meta_prev`. Resten, sändningsportarna en anropare driver och status- och mottagningsportarna den läser, förutsätter en anropare i samma klockdomän, vilket är exakt vad `register_bank` (kapitel 18) är. En anropare i en *annan* domän behöver en handskakning framför dem, inte en bredare `meta_prev`: att köra 64 bitar `tx_data` genom en bit-för-bit-synkroniserare tar bort metastabiliteten men låter fortfarande kontrollern låsa ett värde som aldrig skrevs, samma skevhetsproblem som kapitel 19 lägger ut för omkopplarbanken. En synkroniseringskedja är rätt verktyg för en bit vars grannar är oberoende, och fel verktyg för ett värde vars bitar måste stämma överens.

11.4 Att skriva `can_controller.vhd`

Det här kapitlet skriver filen från grunden, till `controller/can_controller.vhd`, bredvid den `can_def.vhd` som skrevs i kapitel 10.

Entiteten deklarerar de femton portarna som beskrevs ovan, i exakt den här ordningen, vilket är det `can_controller_tb` binder till:

1 clock	4 tx_id	7 rx_bus	10 bus_en	13 rx_data
2 reset_n	5 tx_dlc	8 tx_done	11 rx_id	14 rx_valid
3 tx_req	6 tx_data	9 tx_bus	12 rx_dlc	15 error

Varje ingång först, sedan varje utgång, vilket är den här bokens konvention överallt och är varför `rx_bus` sitter sju snarare än bredvid de bussutgångar den begreppsligt hör ihop med. Läs blockschemat ovan rakt ned för dess vänstra kolumn och sedan dess högra, så får du 1 till 15 i ordning. Typerna kommer från `can_def`, så filen öppnar med `use work.can_def.all`; vid sidan av det vanliga `ieee.std_logic_1164`, och portlistan blandar rent `std_logic` med paketets subtyper. De första raderna sätter mönstret för alla femton:

```
library ieee;
use ieee.std_logic_1164.all;
use work.can_def.all;

entity can_controller is
  port(clock    : in std_logic;
        reset_n : in std_logic;
        tx_req   : in std_logic;
        tx_id    : in id_t;
        ...
```

Få **ordningen** exakt rätt, eftersom det är den allt binder till: `can_controller_tb` associerar sina portar positionellt, och det gör varje instansiering i den här boken. Namnen är dina. Ett omkastat par analyserar alldeles utmärkt i dag, och till skillnad från ett felstavat namn kommer det inte att misslyckas med att analysera i kapitel 17 heller. Det kommer i tysthet att koppla ihop fel signaler, och testbänken kommer att rapportera beteende som inte går ihop, sex kapitel senare.

Arkitekturen börjar tom. Inte en platshållare som ska bytas ut senare, utan genuint tom:

```
architecture behaviour of can_controller is
begin
end architecture;
```

Det analyserar och elaborerar rent, och `make build` bekräftar att det gör det. Skriv det, kontrollera det, och titta på det ett ögonblick: en entitet utan något beteende alls är fortfarande en fullständig designenhet. Sedan lägger resten av projektet in det första inuti den.

Varför skriva den nu, tom? Därför att gränssnittet är ett beslut, och det är ett det här kapitlet är i läge att fatta. Du vet vad en CAN-ram innehåller (kapitel 10), du vet vilka block

som ska göra jobbet (avsnitt 11.1), och du vet vad en drivrutin så småningom behöver läsa och skriva (avsnitt 11.6). Allt kapitel 12 och framåt lägger till är *beteende bakom portar som redan finns*.

Alternativet, att skriva delblocken först och upptäcka toppnivåns gränssnitt på slutet, är så konstruktionen skulle växa fram om ingen hade tänkt på den i förväg. Det betyder också att formen på det du bygger förblir ett mysterium ända till det allra sista kapitlet.

Vad varje kapitel lägger till. Härifrån bygger varje kapitel ett block och instansierar det här, så att arkitekturen fylls i inifrån. Kapitel 12 gör två, eftersom båda är små:

Kapitel	Lägger till i <code>can_controller</code>
12	<code>meta_prev</code> två gånger, som synkroniserar <code>reset_n</code> och <code>rx_bus</code> , sedan <code>bit_timer</code>
13	<code>crc15</code> , och signalerna som matar den en bit i taget
14	<code>tx_shift_reg</code>
15	<code>rx_shift_reg</code>
16	Den ramsekvenserande tillståndsmaskinen, sändningsvägen
17	Mottagningsvägen och arbitreringen, som fullbordar konstruktionen

Filen måste analysera rent i vart och ett av de stegen, inte bara på slutet. `make build` kontrollerar varje modul för sig så snart den finns, så en entitet som inte kompilerar fångas i det kapitel som skrev den.

11.5 Vad kontrollern synkroniserar, och varför det är kontrollerns jobb

Ett beslut följer direkt ur den sista punkten i avsnitt 11.3, och det är värt att göra uttryckligt här eftersom varje senare kapitel hänger på det. `rx_bus` anländer från en transceiver på ett annat chip, och `reset_n` från en knapp eller en övervakare, så ingenting i den här FPGA:n har någonsin styrt någon av dem. Att sampla den ena direkt in i `clocks` domän kan lämna en vippa metastabil, problemet kapitel 4 löste med två vippor i serie.

De två är kontrollerns *enda* asynkrona ingångar, och de är asynkrona vem som än instansierar den, så **k kontrollern synkroniserar dem själv** i stället för att lämna det till kortets toppnivå. Den som integrerar kopplar båda rakt till pinnar och tänker inte mer på det. Allt annat på entiteten är registergränssnittet, som är ett annat kontrakt: ett som en anropare i den här klockdomänen uppfyller gratis, och som en anropare i en annan domän måste uppfylla med en handskakning snarare än en bredare synkroniserare.

Internt heter de synkroniserade kopiorna `reset_s2_n` och `rx_bus_s2`, enligt kapitel 4:s konvention från `reset_sync`, och det är de namn varje delblock från kapitel 12 och framåt tar. **Inget block läser någonsin `reset_n` eller `rx_bus` direkt**, arbitreringsjämförelsen i kapitel 17 inräknad. Modulen som producerar dem, `meta_prev`, är kapitel 12:s, vid sidan av bittimern; det här kapitlet slår fast att kontrollern äger problemet, och nästa bygger det som löser det.

11.6 En första titt på registerkartan

En parallellklass skriver en C++-drivrutin mot den här kontrollern, och talar med den genom **register som nås över SPI** snarare än via VHDL-portar direkt. Du bygger lagren som presenterar de registren: `register_bank` i kapitel 18 och `spi_reg_bridge` i kapitel 19. Hela kartan och transaktionsformatet finns i bilaga D, och tillsammans är de hela kontraktet mellan de två klasserna.

Den är värd en förhandstitt här, sju kapitel i förväg, eftersom det är exakt samma information som `can_controllers` portar redan bär — de senare lagren ändrar informationens *form*, aldrig dess innehåll:

Begrepp (<code>can_controller</code>)	Register i kartan
<code>tx_id</code> , <code>tx_dlc</code> , <code>tx_data</code>	<code>TX_ID</code> , <code>TX_DLC</code> , <code>TX_DATA_LO/HI</code>
<code>tx_req</code> , <code>tx_done</code>	<code>TX_SEND</code> , en bit i <code>STATUS</code>
<code>rx_id</code> , <code>rx_dlc</code> , <code>rx_data</code>	<code>RX_ID</code> , <code>RX_DLC</code> , <code>RX_DATA_LO/HI</code>
<code>rx_valid</code>	en bit i <code>STATUS</code> , nollställd via <code>RX_ACK</code>
<code>error</code>	<code>ERROR_FLAGS</code>

Ha den här motsvarigheten i minnet genom kapitel 19: allt parallellklassens drivrutin läser eller skriver är bara den här kontrollerns eget gränssnitt på portnivå, inslaget i register.

11.7 Sammanfattning

Det här ska du kunna nu

- Rita kontrollerns blockschema och säga vad varje block ansvarar för, och lika viktigt, vad det inte ansvarar för.
- Skriva toppnivåns entitet med rätt portordning och rätt typer.
- Förklara varför arkitekturen är tom nu och vilket kapitel som fyller vilken del av den.
- Köra GHDL:s två första steg på den tomma toppnivån, läsa vad vart och ett rapporterar, och säga vad det tredje steget kräver som ännu inte finns.
- Säga vad positionell portbindning köper och vad den ger upp.

Frågor att testa dig själv med

1. Varför äger `bit_timer` bara *när*, och inte *vad*?
2. Vilket block känner till CAN-ramens format, och varför bara ett?
3. Varför synkroniserar `can_controller` sina egna asynkrona ingångar i stället för att kräva att den som instansierar den gör det?
4. Vad går sönder om du byter plats på två portar av samma typ, och när skulle du märka det?
5. Vad gör `ghdl -a`, `ghdl -e` och `ghdl -r`, var för sig?

6. Vilken registerkarterad operation svarar mot `tx_req`, och vilken mot `tx_done`?

Riktvärde

Toppnivåns entitet bör finnas och analysera rent. Därefter är det upp till gruppen: någon kan mycket väl börja på `bit_timer` eller `crc15` redan nu. De fyra lövmodulerna, `bit_timer`, `crc15`, `tx_shift_reg` och `rx_shift_reg`, beror inte på varandra och har var sin egen testbänk, så en grupp om 4–5 kan bygga dem parallellt. Det är den enda punkt i projektet där ordningen på riktigt spelar roll för hur snabbt ni går framåt; se bilaga C.

Blicken framåt

Kapitel 12 bygger de två första byggblocken som går in i den tomma arkitekturen, båda små: synkroniseraren `meta_prev`, som inte vet någonting alls, och `bit_timer`, det första blocket som vet något om CAN.

11.8 Övningar

De här övningarna befäster avsnitt 11.1 till 11.6: blockarkitekturen, gränssnittet den landar i, och registerkartan den avbildas på. Paketet `can_def` som de alla vilar på skrevs i kapitel 10. Övning 11.1 och 11.2 görs på papper. Övning 11.3 och 11.4 görs vid tangentbordet, mot den controller/`can_controller.vhd` som skrevs under det här passet. `meta_prev` och dess övningar hör till kapitel 12, tillsammans med `bit_timer`.

ÖVNINGSGRUPP 1

Resonemang om arkitekturen

Övning 11.1 Vem äger vad

RESONEMANG

a) `tx_shift_reg` och `rx_shift_reg` vet inte om bitarna de skiftar hör till ID:t, kontrollfältet, en databyte eller CRC-värdet. Förklara, med hänvisning till blockschemat, *vilket* block som vet det, och hur det kan återanvända samma skiftregister till alltihop i stället för att behöva fem olika.

b) `crc15` matas en bit i taget och får veta när den ska sluta av den av `tx_shift_reg` och `rx_shift_reg` som är aktiv för tillfället (via respektive utgångar `stuff` och `real_bit_valid`, som grindar `crc15.enable`). Varför måste `crc15` få veta att den ska *hoppa över* just stoppbitar, i stället för att bara matas med varje bit som dyker upp på ledningen?

c) `can_controller` instansierar de fyra andra blocken inuti sig, så utifrån sett är hela kontrollern en enda komponent. Anta att någon föreslår att kontrollen av arbitreringsförlust

bryts ut till ett eget block *bredvid* `can_controller`. Förklara vad det blocket skulle behöva tillgång till, och varför det gör lösningen sämre än att låta kontrollen ligga kvar där den är.

Övning 11.2 Att dela upp en annan ram i grupper

RESONEMANG

Avsnitt 11.2 följer en ram med DLC 2 genom blocken och kommer fram till att den når bussen som åtta laddningar av `tx_shift_reg`, tillsammans 50 bitar, plus en 10-bitars svans som drivs direkt.

- Gör om den uppdelningen för en ram med ID `0x7A0` och DLC 5. Ange listan av grupper (varje grupps innehåll och antal bitar, i sändningsordning), det totala antalet bitar i det stoppade området, och det totala antalet bitar i ramen innan någon stoppbit satts in.
- Hur många grupper skulle en ram med DLC 0 behöva, och vilka poster ur din lista i a) försvinner? Vad säger det dig om hur `can_controller` måste vara strukturerad kring datafältet, jämfört med identifieraren eller CRC:n?
- Fyra av grupperna i din lista är smalare än 8 bitar, och `tx_shift_reg`:s dataingång är alltid en hel byte. Namnge dem, och säg vad `can_controller` måste skicka med tillsammans med datan för att registret ska sända rätt antal bitar. (Du behöver inte kapitel 14:s svar på *hur*; frågan är vilken information registret inte kan räkna ut på egen hand.)
- Inget av dina gruppantal är antalet bitar som faktiskt drivs ut på ledningen. Förklara vad som saknas, varför det inte går att veta enbart utifrån DLC:n, och vilket block som ansvarar för det.

ÖVNINGSGRUPP 2

Entiteten och gränssnittet

Övning 11.3 Entiteten, kontrollerad

SIMULERING

- Kör `make build-project` från roten av gruppens repo, precis som i kapitel 10. Två egna filer bör nu rapporteras, vid sidan av de två utdelade `bridge/-`filerna, och varje testbänk bör fortfarande hoppas över:

```
--> controller/can_def.vhd analyzes cleanly
--> controller/can_controller.vhd analyzes cleanly
--> bridge/spi_def.vhd analyzes cleanly
--> bridge/spi_slave.vhd analyzes cleanly

==> meta_prev_tb (skipped: no meta_prev.vhd yet)
==> bit_timer_tb (skipped: no bit_timer.vhd yet)
```

```

==> crc15_tb (skipped: no crc15.vhd yet)
==> tx_shift_reg_tb (skipped: no tx_shift_reg.vhd yet)
==> rx_shift_reg_tb (skipped: no rx_shift_reg.vhd yet)
==> can_controller_tb (skipped: no meta_prev.vhd bit_timer.vhd crc15.vhd
    tx_shift_reg.vhd rx_shift_reg.vhd yet)
==> register_bank_tb (skipped: no register_bank.vhd yet)
==> spi_reg_bridge_tb (skipped: no spi_reg_bridge.vhd yet)

0 testbench(es) run, 8 skipped.

```

Den andra raden är hela poängen med den här övningen: en entitet med en tom arkitektur är ändå en komplett, giltig designenhet. Gå nu längre än vad `make build-project` gör och elaborera den för hand, vilket är det andra av GHDL:s tre steg:

```

cd controller
ghdl -a --std=93 can_def.vhd can_controller.vhd
ghdl -e --std=93 can_controller

```

Den elaboreras rent, utan några underblock inuti sig och utan något att simulera. Det finns inget tredje steg att köra här, eftersom en design utan beteende inte har något att kontrollera och ingen egen testbänk; `can_controller_tb` driver den *färdiga* kontrollern och väntar på kapitel 17. Det är i kapitel 12 det tredje steget dyker upp, med den första modulen som har en egen testbänk.

b) `can_controller_tb` ligger redan i `controller/`, och den behöver sju moduler, varav du har skrivit två. Säg vilka fem som saknas och i vilket kapitel var och en kommer. Varför är det bättre att bygget hoppar över den testbänken än att det försöker och misslyckas?

c) *Valfritt, och bara om du har Quartus Prime Lite installerat.* Ta samma fil genom Quartus, enligt bilaga B: New Project Wizard, enhet 5CEBA4F23C7N, lägg till `can_def.vhd` och `can_controller.vhd`, sätt `can_controller` som toppnivåentitet, och kompilera.

Det går igenom. Läs varningarna i stället för att hoppa över dem, och säg vilka av dem som är oundvikliga för en arkitektur som inte driver någonting alls. Leta sedan upp antalet pinnar i fitterns rapport och jämför det med vad ett DE0-CV faktiskt erbjuder en människa: tio omkopplare, fyra knappar, tio lysdioder. Skriv ner vad du tror måste sitta mellan den här entiteten och kortet, och spara anteckningen; kapitel 18 och 19 bygger precis det, och kallar det `register_bank` och `can_spi_node`.

Ingenting senare i boken beror på det här, och kapitel 12 till 19 behöver bara GHDL. Det står här för att just det antalet pinnar är hela argumentet för de två lagren ovanför kontrollern, sju kapitel i förväg, och att läsa av det ur en riktig fitterrapport gör det till ditt eget i stället för en siffra som boken påstår.

d) Byt tillfälligt plats på `tx_bus` och `bus_en` i entitetens portlista, så att de står i omvänd ordning, och kör om `make build-project`. Båda är utgångar av typen `std_logic`, så filen analyseras fortfarande rent. Förklara varför ingenting fångar det nu, i vilket kapitel det först skulle ställa till skada, och hur den skadan skulle se ut på bussen (ledtråden är avsnitt 11.3:s avsnitt om bussidans fem portar). Säg sedan varför en förväxling mellan två portar av samma typ är svårare att hitta än ett felstavat namn skulle ha varit, givet att allt i den här boken binds positionsvis. Byt tillbaka dem.

Övning 11.4 Att läsa gränssnittet

RESONEMANG

a) En anropare vill sända en ram med ID 0x123, DLC 3 och databytesen AA BB CC. Lista varje port den måste driva, och värdet på var och en. `tx_data` är `data_t`, 64 bitar bred, och byte 0 ligger i de mest signifikanta bitarna: ange den i sin helhet.

b) Samma anropare vill nu veta om ramen blev sänd. Den har tre statusportar till sitt förfogande (`tx_done`, `rx_valid`, `error`). Vilken pollar den, och vad är den enda sak den *aldrig* kan få veta av dem, hur ofta den än pollar? (Ledtråden är avsnitt 11.3:s anmärkning om vad `error` betyder; skälet är avsnitt 10.4:s ACK-fält.)

c) `error` ligger hög tills nästa ram börjar, medan `tx_done` och `rx_valid` är pulser på en cykel. Förklara vad som skulle gå fel för parallellklassens C++-drivrutin om `error` vore en puls som de andra två.

Synkronisering och bittimern

De två första byggblocken som går in i den tomma arkitekturen: det som inte vet någonting alls, och det första som vet något om CAN.

I DET HÄR KAPITLET

- `meta_prev`: dubbelvippsynkroniseraren, skriven en gång med en breddgeneric, och instansierad två gånger inuti `can_controller`, för `reset_n` och `rx_bus`.
- `bit_timer`: en fritt löpande räknare som producerar pulserna `sample` och `bit_done`.
- Att köra en utdelad testbänk mot vardera modulen och läsa dess rapport. Det här är kapitlet där GHDL-flödet får sitt tredje steg.
- Att instansiera båda inuti `can_controller`.

Två moduler i stället för en, eftersom båda är små och de är designens enda block som inte håller något protokolltillstånd: `meta_prev` vet ingenting alls, och `bit_timer` vet bara *när*.

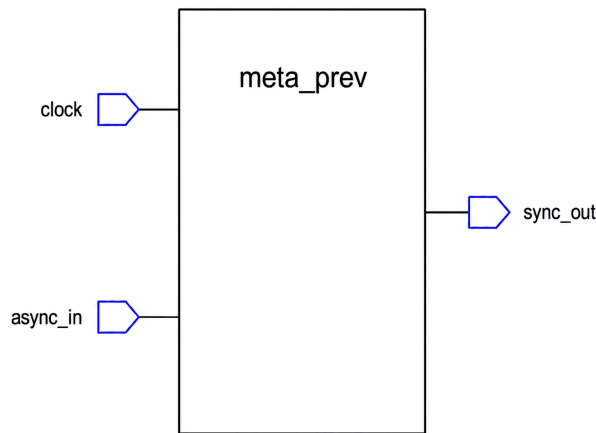
Konstruera varje modul utifrån kontraktet och beteendet nedan, och verifiera sedan mot den utdelade testbänken, utan att gripa efter någon referens. Bilaga A är GHDL-referensen för projektet.

12.1 Att konstruera `meta_prev`

`rx_bus` anländer från en transceiver på ett annat chip, och `reset_n` från en knapp eller en övervakare, så ingenting i den här FPGA:n har någonsin styrt någondera. Att sampla en av dem direkt in i `clocks` domän kan lämna en vippa metastabil, vilket är problemet kapitel 4 löste med två vippor i serie. Du har byggt den här modulen förut, som `sync` i Övning 5.4; den här är samma idé, skriven en gång, ordentligt, för att behållas i den här designen.

Avsnitt 11.5 avgjorde redan att kontrollern synkroniserar sina egna två asynkrona ingångar, och varför: de är asynkrona vem som än instansierar `can_controller`, så kontrollern äger dem och en integratör kopplar båda rakt till pinnar utan att tänka mer på det. Det här avsnittet bygger modulen som gör det. Den är den enda modulen i hela boken som inte innehåller någonting alls om CAN, vilket är precis därför den är återanvändbar nog att sluta med tre instanser vid kapitel 19.

12.1.1 Gränssnitt



Port / generic	Riktn.	Typ	Betydelse
WIDTH	generic	natural := 1	Antal oberoende bitar att synkronisera.
clock	in	std_logic	50 MHz systemklocka, domänen som korsas <i>in i</i> .
async_in	in	std_logic_vector(WIDTH-1 downto 0)	Den eller de asynkrona ingångarna.
sync_out	out	std_logic_vector(WIDTH-1 downto 0)	Samma bitar, säkra att använda i den här domänen.

`meta_prev_tb` binder sina portar positionellt, som allt i den här boken gör, så ordningen och typerna ovan måste stämma exakt. **Generic-mappningen är också positionell**, så `WIDTH` måste vara modulens enda generic (eller åtminstone dess första). Testbänken instansierar modulen två gånger, en gång med standardbredden och en gång med bredd 4, så att både standardvärdet och genericen övas. Den är också den enda modulen här som inte läser något paket alls, så den behöver varken `can_def` eller något annat.

12.1.2 Beteende

Två vippor per bit, i serie, på `clock`s stigande flank: den första kan bli metastabil, och den andra ger den en hel klockperiod att stabilisera sig innan något nedströms ser den. `sync_out` följer därför `async_in` efter exakt två stigande flanker.

Tre saker skiljer sig från modulen `sync` du skrev i Övning 5.4, och alla tre är avsiktliga:

- **Ingen resetport.** `sync` tog en `reset_s2_n`. Den här tar ingen: efter två klockflanker håller en synkroniserare riktig data oavsett vad den startade från. Att stryka porten är också det som

låter *samma* modul synkronisera en reset, utan cirkulariteten i att en resetsynkroniserare behöver en synkroniserad reset. Testbänken klockar igenom den ursprungliga nivån innan den kontrollerar någonting, av precis det skälet.

- **Ingen generic PRESET.** Kapitel 5 argumenterade utförligt för att det säkra resetvärdet beror på vad signalen *betyder*, och att bara den instansierande designen vet det. Det argumentet handlade om de två cyklerna efter reset, och det överlever inte att reseten stryks: utan resetgren finns inget värde att förinställa. Det som ersätter det är anroparens skyldighet att ignorera `sync_out` under de två första flankerna, vilket är samma skyldighet uttryckt annorlunda.
- **SIZE heter WIDTH.** Samma generic, samma uppgift, omdöpt till namnet varje delblock i det här projektet använder.

Den här designen instansierar den tre gånger, alla med `WIDTH = 1`: två gånger inuti `can_controller`, för `reset_n` och för `rx_bus`, och en gång på `can_spi_nodes` toppnivå för dess egen reset (kapitel 19). En modul, tre instanser, inga kopior. Genericen gör ändå skäl för sin plats — testbänken övar `WIDTH = 4`, och en synkroniserare skriven för en bit är en synkroniserare du kopierar nästa gång du behöver fyra.

En reservation att nämna redan nu, eftersom kapitel 19 lutar sig mot den: en synkroniserare hanterar metastabilitet, inte *skevh*et. Den andra vippa gör det överväldigande osannolikt att ett metastabilt värde fortplantar sig (aldrig omöjligt; sannolikheten faller exponentiellt med stabiliseringstiden, vilket är därför två vippor är den accepterade avvägningen), men varje bit korsar fortfarande oberoende, så två bitar som ändras tillsammans kan landa en klocka isär. Det är skälet till att en synkroniserare inte är ett generellt svar för en buss vars bitar måste hålla ihop — och det är därför `can_spi_node` synkroniserar sin reset med den här modulen men lämnar SPI-ledningarna till `spi_slave`, som ramar in dem, snarare än att bredda en `meta_prev` över dem.

12.1.3 Vad testbänken låser fast

- `sync_out` följer `async_in` efter exakt två stigande flanker: inte en (en enda vippa är ingen synkroniserare) och inte tre.
- Detsamma gäller från hög till låg som från låg till hög.
- Med `WIDTH = 4` anländer alla fyra bitarna tillsammans, två flanker efter att ingången ändrats.
- En stabil ingång lämnar `sync_out` stabil.

Det här är den första testbänken i projektet som faktiskt *körs* snarare än bara analyseras, så det är också här det tredje GHDL-steget anländer. Kapitel 11 analyserade och elaborerade; se bilaga A för vad `ghdl -r` tillför och varför `--assert-level=error` inte är valfritt.

12.1.4 Att lägga in den i can_controller

meta_prevs portar är vektorer även vid bredd 1, så en skalär port behöver en elementssignal på vardera sidan. I arkitekturs deklarativa del (mellan is och begin):

```
-- meta_prev, synchronizing the two inputs that arrive from off-chip.
signal reset_n_v, reset_s2_n_v : std_logic_vector(0 downto 0);
signal rx_bus_v, rx_bus_s2_v : std_logic_vector(0 downto 0);
signal reset_s2_n, rx_bus_s2 : std_logic;
```

och efter begin:

```
reset_n_v(0) <= reset_n;
rx_bus_v(0) <= rx_bus;

reset_s2_n <= reset_s2_n_v(0);
rx_bus_s2 <= rx_bus_s2_v(0);

reset_sync: entity work.meta_prev
  port map(clock, reset_n_v, reset_s2_n_v);

rx_bus_sync: entity work.meta_prev
  port map(clock, rx_bus_v, rx_bus_s2_v);
```

Två instanser i stället för en med WIDTH => 2. Båda bitarna skulle åka med tillsammans alldeles utmärkt, eftersom de är oberoende och skevhet mellan dem inte betyder något, men en namngiven instans per angelägenhet läser sig bättre på diagrammet, och att senare låta reseten hävdas asynkront skulle då inte riva upp någonting.

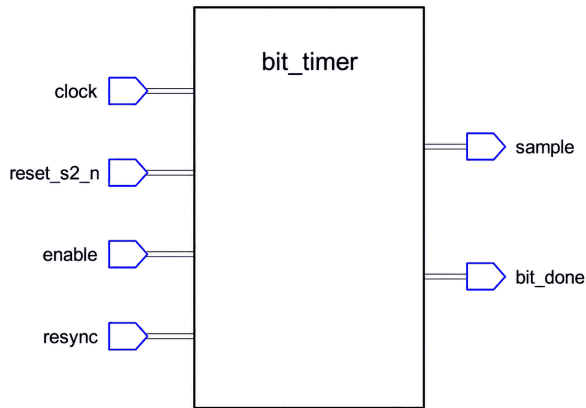
Från och med nu tar varje block reset_s2_n, aldrig reset_n, och varje block som läser bussen läser rx_bus_s2, aldrig rx_bus. Det är hela skälet till att de här två instanserna finns, det gäller bit_timer redan i nästa avsnitt, och det är lätt att glömma igen när rx_shift_reg kopplas in i kapitel 15.

12.2 Att konstruera bit_timer

bit_timer delar 50 MHz-systemklockan i CAN-bitperioder och producerar två encykelspulser som resten av kontrollern går på: en som markerar var i biten bussen ska samplas, en som markerar bitens slut.

Varför en bitperiod alls delas upp, och varför sampelpunkten ligger sent i den snarare än mitt i, står i kapitel 5 i *CAN: bussen, framen och kontrollern*; det här avsnittet bygger den modul som gör det den kapitlet beskriver, med en fast sampelpunkt i stället för konfigurerbara segment.

12.2.1 Gränssnitt



Port	Riktn.	Typ	Betydelse
clock	in	std_logic	50 MHz systemklocka.
reset_s2_n	in	std_logic	Aktiv låg reset, synkroniserad med två vippor.
enable	in	std_logic	Kör räknaren medan '1'; håller den medan '0'.
resync	in	std_logic	Tvingar räknaren tillbaka till tick 0 på nästa klockflank.
sample	out	std_logic	Flagga som anger att bussen ska samplas.
bit_done	out	std_logic	Flagga som anger att den här biten är klar och att nästa börjar.

Inga generics. `bit_timer_tb` binder sina portar **positionellt**, så ordningen och typerna ovan måste stämma exakt. Namnen är dina; att behålla de här får boktexten och din kod att tala om samma signaler.

Ur `can_def` (kapitel 10) behövs `TICKS_PER_BIT` (= 50, ticken i en bitperiod) och `SAMPLE_TICK` (= 35, alltså 70 % in, sent nog för att bussen ska ha hunnit stabilisera sig).

12.2.2 Beteende

En enda fritt löpande räknare över $0 \dots \text{TICKS_PER_BIT} - 1$ räcker. Vid varje klockflank: ge båda pulserna förvalet '0' och sedan, i prioritetsordning:

1. **reset_s2_n = '0'**: nollställ räknaren och båda pulserna.
2. **resync = '1'**: starta om bitperioden; sätt räknaren tillbaka till 0. Det här vinner över `enable`, och används en gång per ram, vid SOF, så att den här nodens bittajming börjar om från början, i linje med den flank som inleder ramen i stället för var en tidigare räkning råkade stå.

3. **enable = '1'**: pulsa sample under det enda tick där counter = SAMPLE_TICK. Vid counter = TICKS_PER_BIT - 1, sätt bit_done och låt räknaren slå runt till 0; annars öka den.
4. **Annars (avstängd)**: håll räknaren, båda pulserna låga.

Det här är samma idiom med räknare och prioriterade grenar som kapitel 7:s timer, bara med två pulsutgångar i stället för en.

En notering om enable, eftersom det är den enda porten vars slutliga användning inte är uppenbar. Den naturliga gissningen är att can_controller håller timern medan bussen är ledig, och det visar sig att den inte gör det: kapitel 16:s regel för ledig buss räknar recessiva **bitperioder**, så timern måste fortsätta gå även i STATE_IDLE, och den färdiga designen binder enable högt i varje tillstånd. Bygg porten ändå; testbänken kontrollerar den, och en timer som inte går att stoppa är ett sämre byggblock än en som går.

12.2.3 En bitperiod, tick för tick

Reglerna ovan är lättare att lita på när de setts på en tidsaxel. Med TICKS_PER_BIT = 50 och SAMPLE_TICK = 35 ser en aktiverad bitperiod ut så här. Varje kolumn är en stigande klockflank; counter är värdet processen ser vid den flanken, och de två raderna under den är pulserna som den flanken producerar.

counter	0	1	2	..	34	35	36	..	48	49		0	1	..
sample	0	0	0	..	0	1	0	..	0	0		0	0	..
bit_done	0	0	0	..	0	0	0	..	0	1		0	0	..

Tre saker att läsa ur det. Båda utgångarna är **ett tick breda**, inte nivåer. sample kommer vid tick 35 och bit_done vid tick 49, så **sample slår alltid till först, 14 tick tidigare i samma period**; en mottagare läser bussen 70 % in och först därefter tar perioden slut. Och periodgränsen är omslaget från 49 tillbaka till 0, vilket är samma flank som bit_done markerar.

Nu samma bild med en resync. Anta att timern är 20 tick in i en period när SOF anländer och can_controller pulsar resync:

tick	19	20	21	22	..	55	56	..	69	70	71
resync	0	1	0	0	..	0	0	..	0	0	0
counter	19	20	0	1	..	34	35	..	48	49	0
sample	0	0	0	0	..	0	1	..	0	0	0
bit_done	0	0	0	0	..	0	0	..	0	1	0

Räknaren kastas mitt i räkningen och startar om, så counter och tick stämmer inte längre överens. Det som spelar roll är följden: bit_done anländer vid tick 70, en hel TICKS_PER_BIT efter omsynkroniseringen, snarare än vid tick 49 där den avbrutna perioden skulle ha slutat. Noden har antagit sändarens bitgräns i stället för sin egen, vilket är hela poängen med resync och exakt vad testbänkens fjärde fall mäter.

Lägg också märke till att resync startar om perioden trots att enable är hög hela tiden; det är därför den står över enable i prioritetsordningen.

12.2.4 Vad testbänken låser fast

- Hållen i reset stannar sample och bit_done på '0'.
- Medan enable = '0' går räknaren aldrig framåt; båda pulserna stannar på '0'.
- Medan den är aktiverad pulsar sample exakt vid SAMPLE_TICK och bit_done exakt vid det sista ticket, en gång per period.
- Efter en resync mitt i en period kommer nästa bit_done exakt TICKS_PER_BIT tick senare, inte där den avbrutna perioden skulle ha tagit slut.

12.2.5 Att lägga in den i can_controller

bit_timer sällar sig till de två meta_prev-instanserna från avsnitt 12.1 inuti toppnivån som skrevs i kapitel 11, och är det första blocket där inne som vet något om CAN. Att instansiera den är samma tvådelade ändring i controller/can_controller.vhd som meta_prev var, och den tar reset_s2_n från synkroniseraren du kopplade in nyss snarare än porten reset_n.

Deklarera först de fyra signaler som instansen kopplas till, i arkitekturens deklarativa del:

```
-- bit_timer.  
signal bt_enable, bt_resync, bt_sample, bt_bit_done : std_logic;
```

Instansiera sedan själva entiteten, efter begin:

```
bit_timer1: entity work.bit_timer  
    port map(clock, reset_s2_n, bt_enable, bt_resync, bt_sample, bt_bit_done);
```

Ingenting driver bt_enable eller bt_resync än, och ingenting läser bt_sample eller bt_bit_done; tillståndsmaskinen som gör bådadera är kapitel 16:s. Det är väntat i det här läget. Det som spelar roll nu är att can_controller.vhd fortfarande analyserar rent, vilket make_build kontrollerar så snart filen finns, och att bit_timer själv passerar bit_timer_tb.

12.3 Sammanfattning

Det här ska du kunna nu

- Implementera en breddgenerisk dubbelvippsynkroniserare, och förklara varför den inte behöver någon reset och vad en breddning *inte* löser.
- Implementera en räknarbaserad bittimer som producerar rätt tajmade sample- och bit_done-pulser, med en resync-ingång.
- Förklara vad resync gör och varför sample kommer före bit_done.

- Läs ett spår på tickvis nivå över en bitperiod och för varje tick säga vad varje utgång bör vara.
- Köra en utdelad testbänk hela vägen och spåra ett fel tillbaka till den tick i designen som orsakade det.

Frågor att testa dig själv med

1. Varför behöver en synkroniserare två vippor snarare än en, och varför behöver den ingen reset?
2. En breddad `meta_prev` tar bort metastabilitet men inte skevhet mellan bitarna. Vad är skillnaden, och när slutar en synkroniserare vara rätt verktyg?
3. Vad gör `bit_timer.resync`, och när används den?
4. Varför kommer `sample` före `bit_done` snarare än tvärtom?
5. `bit_timer` håller sin räknare medan `enable = '0'`. Vilket tillstånd i `can_controller` är det enda som någonsin skulle vilja ha det, och varför visar det sig inte vilja det heller?
6. När en utdelad testbänk misslyckas, vad skriver GHDL ut, och hur hittar du vilken kontroll som fallerade?

Riktvärde

`meta_prev` och `bit_timer` bör vara klara och mergade inom ungefär en vecka. Båda är korta, och de är de enda modulerna som ingen annan modul kan ersätta. Det här är också de första riktiga Pull Requesterna: använd granskningen, och kontrollera portordning, att testbänken passerar, och att inga konstanter är hårdkodade förbi `can_def`.

Blicken framåt

Kapitel 13 bygger `crc15`, en bitseriell checksummemotor som både genererar och kontrollerar en CRC med samma logik och utan lägesomkoppling.

12.4 Övningar

Övning 12.1 befäster avsnitt 12.1; Övning 12.2 till 12.4 befäster avsnitt 12.2. Konstruera varje modul utifrån specifikationen i sitt eget avsnitt; de utdelade testbänkarna, körda enligt bilaga A, är kontrollen av att de beter sig korrekt.

ÖVNINGSGRUPP 1

Synkroniseraren

Övning 12.1 meta_prev

VHDL

Du har byggt den här modulen förut, så det mesta här handlar om de två sakerna som är nya.

a) Skriv din egen `meta_prev.vhd` utifrån avsnitt 12.1 och kör dess testbänk:

```
cd controller
ghdl -a --std=93 meta_prev.vhd meta_prev_tb.vhd
ghdl -e --std=93 meta_prev_tb
ghdl -r --std=93 meta_prev_tb --assert-level=error
```

Lägg märke till att ingen `can_def.vhd` står med på den första raden. `meta_prev` är den enda modulen i boken som inte läser något paket alls, vilket också är skälet till att det är den som återanvänds tre gånger.

b) Ta bort den ena av de två vipporna, så att ett enda register blir kvar, och kör om. Vilken kontroll fallerar, och vad säger dess meddelande dig? Sätt tillbaka den. (En version med en vippa är inte bara svagare; den är precis den bugg testbänken finns till för att fånga, och den skulle annars passera varje funktionell simulering i boken, eftersom ingenting i en simulator någonsin är metastabilt.)

c) `meta_prev` har ingen reset. Säg vad `sync_out` läser på den första klockflanken efter uppstart, varför det är ofarligt här, och vad som skulle gå fel i `can_controller` självt om modulen hade haft en ingång `reset_s2_n`: instansen `reset_sync` är den som producerar `reset_s2_n`, så vad skulle den ta för sin egen reset?

d) `can_controller` instansierar den två gånger, båda med standardbredden. Säg vilka två av dess femton portar det gäller, och varför just de två och inga andra. Säg sedan vad den tredje instansen i designen synkroniserar — den som `can_spi_node` äger på sin toppnivå i kapitel 19 — och varför just den inte kan ligga inuti `can_controller` i stället.

e) Två omkopplare slås om i samma ögonblick in i en instans med `WIDTH = 10`. Förklara varför `sync_out` under ett kort ögonblick kan visa den ena omkopplarens nya värde och den andras gamla, varför det inte är en bugg i `meta_prev`, och varför det skulle spela betydligt större roll om de tio bitarna vore en räknare som lästes över en klockdomängrens i stället för omkopplare som ställs för hand.

ÖVNINGSGRUPP 2

Bittimern**Övning 12.2 bit_timer**

VHDL

Skriv din egen `bit_timer.vhd` utifrån avsnitt 12.2. Verifiera den sedan:

```
cd controller
ghdl -a --std=93 can_def.vhd bit_timer.vhd bit_timer_tb.vhd
ghdl -e --std=93 bit_timer_tb
ghdl -r --std=93 bit_timer_tb --assert-level=error
```

Om en kontroll fallerar, läs assertion-meddelandet (bilaga A); det namnger exakt vilket tick och vilket förväntat värde det gäller.

Övning 12.3 En konfigurerbar omsynkroniseringsoffset

SIMULERING

Riktig CAN-hårdvara behöver ibland en bittimer som kan omsynkronisera *och* omedelbart hoppa fram ett fast antal tick (en förenklad ersättare för de justeringar av fassegment som riktig CAN-bittajmingslogik utför, nämnda i avsnitt 10.7).

a) Lägg till en ny generic, `RESYNC_OFFSET`: `natural := 0`, i `bit_timer`. När `resync = '1'` ska räknaren sättas till `RESYNC_OFFSET` (i stället för alltid 0) på nästa klockflank.

b) Med `RESYNC_OFFSET = 0` måste din ändrade entitet bete sig exakt som originalet; bekräfta det genom att köra den *oförändrade* `bit_timer_tb.vhd` mot den; den ska fortfarande passera utan några ändringar i testbänken.

c) Räkna ut på papper var **båda** utgångarna hamnar när `RESYNC_OFFSET` är 10: hur många tick efter en `resync`-puls som `sample` slår till, och hur många tick efter den som `bit_done` gör det. Båda flyttar sig, och av samma skäl — räknaren startar om på 10 i stället för 0, så hela perioden är tio tick kortare.

Låt sedan den utdelade testbänken mäta upp det åt dig. Ändra tillfälligt din generics standardvärde till 10 och kör om `bit_timer_tb`. Den binder ingen generic map, så den plockar upp vilket standardvärde du än sätter, och fall 4 är det som märker det:

```
bit_timer_tb.vhd:132:17:@6832ns:(assertion error): bit_timer: unexpected sample at-
↳ tick 25 after resync!
```

Lägg märke till **vilken** av de två kontrollerna som fallerar först. Fall 4 kontrollerar inte bara att `bit_done` kommer på rätt tick utan också att sampelpunkten flyttar med om, och sampelpunkten ligger först i perioden — så det är den som slår till, fjorton tick innan `bit_done`-kontrollen skulle ha gjort det. Under `--assert-level=error` stannar simuleringen där, så meddelandet om `bit_done` får du aldrig se.

Meddelandet namnger själv ticket: `sample` kom på tick 25, alltså `SAMPLE_TICK - RESYNC_OFFSET`. Vill du se `bit_done` också, sänk sampelkontrollens `severity` error till `severity note` i din egen kopia av testbänken och kör om; den rapporterar då tick 39, alltså `TICKS_PER_BIT - 1 - RESYNC_OFFSET`. Bekräfta att båda stämmer med ditt pappersvar, sätt sedan tillbaka standardvärdet till 0 och kontrollera att den *oförändrade* testbänken passerar igen.

Det är värt att lägga märke till i sig: en testbänk som fallerar *användbart* säger inte bara att en design är fel utan också hur mycket, vilket är skälet till att assertion-meddelandena i det här projektet alla namnger det tick eller det värde de förväntade sig.

Övning 12.4 Hur länge kan en bittimer frilöpa?

RESONEMANG

`bit_timer` omsynkroniserar **en gång per ram**, vid SOF, och frilöper därifrån (avsnitt 10.5:s resonemang om klockåtervinning). Det fungerar bara om nodens egen oscillator håller sig tillräckligt nära varenda annan nods under hela ramen. Den här övningen sätter en siffra på "tillräckligt nära".

Anta att en nods 50 MHz-oscillator i själva verket går 1 % för fort, så att dess tick är 1 % kortare än sändarens, och att allt annat är som specificerat: `TICKS_PER_BIT = 50`, `SAMPLE_TICK = 35`.

- a) Hur många tick har den här nodens räknare vunnit på sändarens efter en bitperiod? (Uttryck det som en andel av ett tick.)
- b) Noden samplar vid tick 35 av 50, så i förhållande till den sanna biten kan den driva 15 tick *senare* innan den samplar bortom bitens slut, eller 35 tick *tidigare* innan den samplar innan biten börjar. En snabb klocka driver sampelpunkten tidigare. Efter hur många bitperioder har den drivit hela 35 tick?
- c) En standardram med 8 databytes går på ungefär 110–130 bitar på ledningen när stoppbittarna är inräknade. Jämför det med ditt svar på **b**): samplar den här noden fortfarande rätt vid EOF? Vid ungefär vilket fält börjar den sampla fel bit?
- d) Riktiga CAN-kontrollrar omsynkroniserar *under* en ram, inte bara vid SOF, med hjälp av de fassegment och den synchronization jump width som avsnitt 10.7 nämner. Förklara med en eller två meningar vad det ger dem, uttryckt i ditt svar på **b**).

CRC-15-motorn

En motor, två jobb.

I DET HÄR KAPITLET

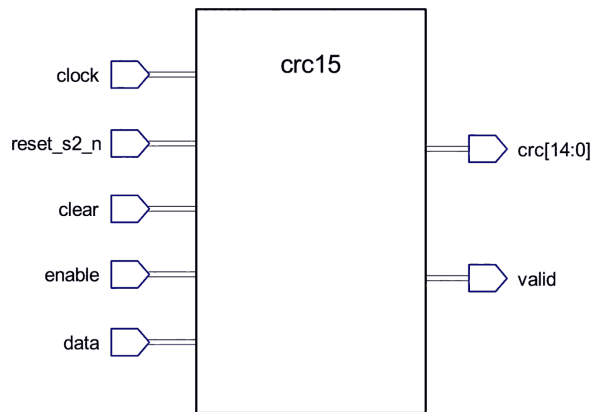
- `crc15`: en bitseriell CRC-15-motor.
- Varför samma motor både genererar och kontrollerar en CRC, utan lägesomkoppling.
- Varför den ändå behöver ett `clear`, och vad en avbruten ram lämnar efter sig utan det.
- En kort CRC arbetad genom rekursionen för hand.
- Att instansiera `crc15` inuti `can_controller`.

CAN:s CRC-15 beräknas **bitseriellt**: en bit inmatad per aktiverad klockflank, utan något separat "beräkna checksumman nu"-steg. Egenskapen som gör den här bokens konstruktion enkel: **exakt samma motor, körd på exakt samma sätt, både genererar en CRC och kontrollerar en.**

Kapitel 3 i *CAN: bussen, ramen och kontrollern* är protokollsidan av det här kapitlet: vad polynomet är, vilka fält som räknas in, och varför stoppbitar hålls utanför. Det här kapitlet bygger motorn; den boken säger vad den ska räkna på.

Konstruera den utifrån kontraktet och rekursionen nedan, och verifiera sedan mot den utdelade testbänken. `crc15` beror inte på `bit_timer` och kan mycket väl ha byggts redan.

13.1 Gränssnitt



Port	Riktn.	Typ	Betydelse
clock	in	std_logic	50 MHz systemklocka.
reset_s2_n	in	std_logic	Aktiv låg, synkroniserad reset.
clear	in	std_logic	Återför registret till idel nollor vid den här flanken, utan en reset.
enable	in	std_logic	Integrera data i registret vid den här flanken när '1'.
data	in	std_logic	Den aktuella biten, biten som sänds (TX) eller nyss samplats av bussen (RX).
crc	out	crc_t	Registrets aktuella 15-bitars CRC-värde.
valid	out	std_logic	'1' närhelst registret för närvarande läser idel nollor.

Inga generics. `crc15_tb` binder sina portar **positionellt**, så ordningen och typerna ovan är det som måste stämma; namnen är dina. `crc_t` (en 15-bitars vektor), dess bredd `CRC_WIDTH` och polynomet `CRC_POLY` kommer från `can_def` (kapitel 10).

13.2 Rekursionen

Håll ett 15-bitars register (`crc_t`), nollställt till idel nollor av `reset_s2_n`. Vid varje klockflank där `enable = '1'`, integrera en bit:

```
feedback = crc(CRC_WIDTH-1) xor data      -- Inkommande bit XOR registrets översta bit.
shifted  = crc skiftat ett steg vänster, ny lägsta bit '0'
crc      = shifted xor CRC_POLY            -- om feedback = '1'
          = shifted                        -- annars
```

Det är hela motorn, femton vippor och en skiftning XOR-grindad av polynomets avgreningar. När `enable` är '0', håll registret oförändrat. Driv `crc` kontinuerligt från registret, och `valid` som "registret är idel nollor".

`clear` gör med registret vad `reset_s2_n` gör, men den är ett **synkront kommando, inte en reset**: den hör hemma inuti den klockade grenen, prövad före `enable` så att den vinner när båda är höga vid samma flank. Vik inte in den i det asynkrona resetvillkoret; det skulle göra `clear` asynkron och nivåkänslig, en annan konstruktion som råkar passera testbänken (som alltid håller `clear` över en hel klockcykel). Gränssnittstabellen säger "vid den här flanken" och menar det.

`clear` finns eftersom motorn återanvänds för varje ram och håller inget eget tillstånd per ram: `can_controller` pulsar den en gång i början av varje ram (kapitel 16). En ram som får löpa till slut återför registret till noll av sig själv, genom generera-sedan-kontrollera-egenskapen nedan, så på den lyckliga vägen ändrar `clear` ingenting. En ram som *avbryts* halvvägs gör inte det, och det är det fallet den finns till för.

13.3 En motor, två jobb

- **Generera:** aktivera motorn över SOF till och med datafältets slut. Läs sedan `crc`; det är värdet som ska sändas som ramens CRC-fält.
- **Kontrollera:** aktivera den över SOF till och med datafältet *och fortsatt sedan*, och mata in det *mottagna* CRC-fältets egna bitar också. Om ramen anlände intakt driver det registret tillbaka till exakt noll, en egenskap hos aritmetiken, inte något motorn blivit tillsagd att förvänta sig. `valid` rapporterar precis det. `can_controller` (kapitel 17) kontrollerar `valid` bara vid just den punkten; den behöver ingen andra, separat kontrollkrets.

Båda jobben förbrukar **bara riktiga bitar**. Stoppbitar (kapitel 14) kommer aldrig in i CRC:n på någondera sidan av länken; anroparens grindning av `enable` är det som håller dem ute (kapitel 16 och 17 bygger precis det), och motorn själv vet aldrig att de finns. Det är därför `crc15` inte behöver någon kunskap om ramfält eller stoppning: den som driver `enable` har redan tagit de besluten.

Båda jobben förutsätter att registret börjar en ram på noll, vilket är exakt vad `clear` garanterar. Lägg märke till hur mycket som vilar på det: en nod som överger en ram halvvägs (den förlorar arbitringen, eller ser en stoppningsöverträdelse, eller underkänns i CRC-kontrollen, allt sådant som kapitel 17 gör) slutar mata motorn med ett *partiellt* värde i den. Utan en nollställning per ram får nästa ram den resten som utgångsvärde, så noden beräknar en CRC ingen annan håller med om och beräknar en felaktig kontroll på allt den tar emot, för varje ram från då till reset. En enda avbruten ram skulle ta noden ur bussen permanent.

`valid` läser '1' vid reset också, före någon data alls; meningslöst där, meningsfullt bara vid kontrollpunkten ovan.

13.4 Ett genomräknat exempel, för hand

Börja med det enklaste fallet. Mata in en enda 1 i en nyss resetad motor: feedback är 0 xor 1 = 1, shifted är fortfarande idel nollor, så crc blir 0 xor CRC_POLY, polynomet självt. Det är det första testbänken kontrollerar, och en bra rimlighetskontroll på din egen kod.

Nu fyra bitar, 1001, MSB först. top är registrets bit 14 *före* flanken, och CRC_POLY är 100010110011001:

	top	data	fb	shifted	xor poly	crc
reset	-	-	-	-	-	000000000000000
bit 1	0	1	1	000000000000000	ja	100010110011001
bit 2	1	0	1	000101100110010	ja	100111010101011
bit 3	1	0	1	001110101010110	ja	101100011001111
bit 4	1	1	0	011000110011110	nej	011000110011110

Läs en rad i taget så slutar motorn se ut som aritmetik. Varje flank gör samma tre saker: XOR:a den inkommande biten med biten som faller av toppen, skifta vänster, och XOR:a tillbaka polynomet bara om det resultatet var 1.

Två rader är värda att stanna vid. **Bit 2** matar in en 0 och utlöser ändå XOR:en, eftersom 1:an som lämnar toppen är det som sätter feedback; den inkommande biten är bara halva beslutet. **Bit 4** matar in en 1 och utlöser den *inte*, eftersom den översta biten också är 1 och de två tar ut varandra. Det är därför en CRC svarar på *var* en bit sitter och inte bara på hur många ettor det finns, och det är den egenskap Övning 13.3 använder för att fånga en förvanskning som en enkel summa inte kan se.

Lägg också märke till att varje rads shifted-kolumn är föregående rads crc skiftad ett steg vänster, med en 0 som kommer in nedifrån. Ingenting adderas någonsin i den låga änden; registrets innehåll kommer helt och hållet från polynomet som XOR:as in om och om igen vid olika förskjutningar.

13.5 Vad testbänken låser fast

- Strax efter reset är crc idel nollor och valid = '1'.
- En enda '1'-bit ger crc = CRC_POLY.
- clear återför ett laddat register till idel nollor utan att reset_s2_n rörs.
- En 19-bitars sekvens av SOF, ID, RTR och kontrollfält ger en CRC som stämmer med ett värde beräknat oberoende i Python (inte härlett ur den här VHDL-koden).
- Att mata den nyss beräknade CRC:ns egna bitar rakt in igen återför registret till idel nollor (valid = '1'), generera-sedan-kontrollera-egenskapen.

13.6 Att lägga in den i can_controller

Samma tvådelade ändring som kapitel 12:s, ett block längre in. I controller / can_controller.vhd, lägg till signalerna under bit_timer-gruppen:

```
-- crc15.
signal crc_clear, crc_enable, crc_data_in, crc_valid : std_logic;
signal crc_value                                     : crc_t;
```

och instansen under bit_timer1:

```
crc1: entity work.crc15
    port map(clock, reset_s2_n, crc_clear, crc_enable, crc_data_in, crc_value,
    ↪   crc_valid);
```

Lägg märke till att crc15 kopplas till ingenting annat än klockan, resetten och sina egna signaler: den tar sina bitar från vilket skiftregister som än är aktivt, och båda de registren är ännu oskrivna. Kopplingen som förenar dem är kapitel 16:s, och den är ett enda uttryck snarare än en port map.

13.7 Sammanfattning

Det här ska du kunna nu

- Implementera en bitseriell CRC-15-motor som matchar CAN:s polynom.
- Förklara varför generering och kontroll är samma operation, och vad ett nollställt CRC-register vid slutet av en mottagen ram bevisar.
- Räkna några bitar genom rekursionen för hand och jämföra med simuleringen.
- Säga varför motorn behöver nollställas mellan ramar, och vilket fel som uppstår om den inte gör det.

Frågor att testa dig själv med

1. Varför behöver motorn inget läge för "generera" respektive "kontrollera"?
2. Vad betyder det att CRC-registret är noll efter att en mottagen ram matats igenom?
3. Vad händer om en ram avbryts och nästa ram börjar matas in utan clear?
4. Varför matas en instoppad bit inte in i CRC-motorn?
5. Vilken signal i can_controller avgör när crc15 får uppdatera sig, och varför är det inte bit_done?

Riktvärde

Vid det här laget bör två av de fyra lövmodulerna vara klara. Vilka två spelar ingen roll.

Blicken framåt

Kapitel 14 bygger sändningsskiftregistret: att serialisera ut på bussen, med samma stoppbits-regel som i kapitel 10, nu i VHDL. Kapitel 14 och 15 bygger tillsammans de moduler som avgör vilka bitar som når `crc15` och när.

13.8 Övningar

De här övningarna befäster kapitlet. Konstruera din egen `crc15.vhd` utifrån avsnitt 13.1 till 13.4; den utdelade testbänken, körd enligt bilaga A, är kontrollen av att den beter sig korrekt.

Övning 13.1 `crc15`

VHDL

Skriv din egen `crc15.vhd` utifrån specifikationen i det här kapitlet. Verifiera den sedan:

```
cd controller
ghdl -a --std=93 can_def.vhd crc15.vhd crc15_tb.vhd
ghdl -e --std=93 crc15_tb
ghdl -r --std=93 crc15_tb --assert-level=error
```

Om en kontroll fallerar, läs assertion-meddelandet (bilaga A).

Övning 13.2 CRC för hand, sedan i simulering

SIMULERING

- a) Räkna för hand ut, med avsnitt 13.2:s rekursion, vilket CRC-15-resultat som blir följden av att mata in 4-bitarssekvensen 1010 (MSB först) i en nyss nollställd motor. Visa uträkningen en bit i taget (återkoppling, skift, villkorlig XOR).
- b) Bekräfta ditt handräknade svar med GHDL: skriv en kort testbänk (eller anpassa strukturen i `crc15_tb.vhd`) som nollställer `crc15`, matar in 1010 och rapporterar det resulterande `crc`-värdet, antingen via en `assert` eller via en `report ... severity note`; som du läser direkt.
- c) Utgången valid hos `crc15` läser '1' direkt efter reset, innan någon riktig data alls har matats in. Förklara varför det inte är en bugg, med hänvisning till vad `can_controller` (kapitel 17) faktiskt gör med valid, och när.

Övning 13.3 Felet som en summerad checksumma missar

SIMULERING

Avsnitt 10.1 påstod att en CRC "fångar en betydligt bredare, matematiskt välförstådd klass av fel" än den enkla bytevisa summa som dess eget ramformat använder. Den här övningen gör det konkret genom att köra en korruption som summan bevisligen inte kan se genom motorn du just byggt.

Ta nyttolasten 0x3C, 0x91 och korruptionen 0x34, 0x99: bit 3 nollställd i den första byten, bit 3 satt i den andra, så att den ena byten förlorar exakt det den andra vinner.

- a) Räkna ut den bytevisa checksumman för båda nyttolasterna och bekräfta att de är **identiska**: båda blir 0xCD. En enkel summa registrerar bara totalen, så en mottagare som kontrollerar den skulle godta den korrupta ramen som hel.
- b) Mata nu in alla 16 bitarna av den *ursprungliga* nyttolasten i en nyss nollställd crc15 (MSB först, första byten först) och notera det resulterande crc-värdet. Nollställ sedan och gör samma sak med den *korrupta* nyttolasten. Stämmer de två CRC-värdena överens? Använd simulering; en report av båda värdena räcker.
- c) Egenskapen att generering och kontroll är samma operation (avsnitt 13.3) säger att en ram vars egna CRC-bitar matas tillbaka in återför registret till noll. Mata in den **korrupta** nyttolasten följd av den **ursprungliga** nyttolastens CRC-bitar, alltså precis det en mottagare skulle se om den här korruptionen inträffade under transporten. När registret noll? Går valid hög? Förklara vad mottagaren drar för slutsats.
- d) Säg med en mening vilken egenskap hos CRC-rekursionen som gör att den märker en förändring som en summa inte kan, med hänvisning till vad varje bit gör med registret i avsnitt 13.2.

Sändningsskiftregister och bitstoppning

Att serialisera ut på bussen, med samma stoppbitsregel som i kapitel 10, nu i VHDL.

I DET HÄR KAPITLET

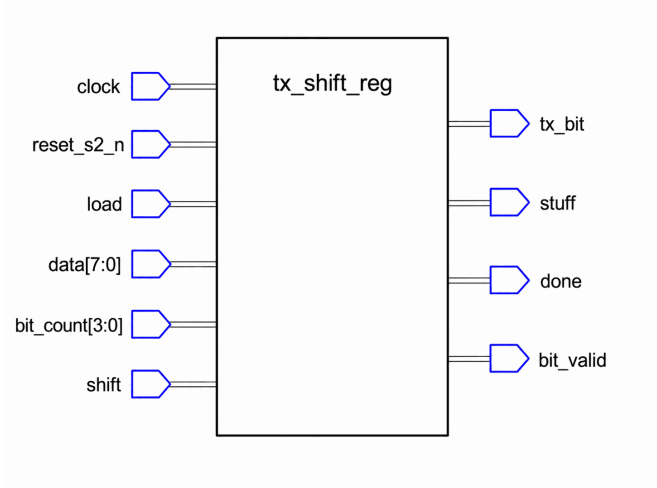
- tx_shift_reg: serialisering med mest signifikanta biten först, och insättning av stoppbitar.
- Varför en laddad grups första bit måste presenteras omedelbart vid omladdning, och varför done väntar en extra skiftning.
- Varför stoppningstillståndet lever kvar över omladdningar.
- stuff och bit_valid, och varför CRC-motorn grindas på dem.
- Att instansiera tx_shift_reg inuti can_controller.

tx_shift_reg serialiserar ut en grupp på upp till 8 bitar MSB först på bussen och **sätter in** en stoppbit efter fem lika utsända bitar i rad. Dess spegelbild, rx_shift_reg (som *kastar* de stoppbitarna), är kapitel 15.

Själva stoppbitsregeln är oförändrad från avsnitt 10.5 och kapitel 3 i *CAN: bussen, framen och kontrollern*, där du tillämpade den på papper på en hel bitföljd i taget, med möjlighet att se tillbaka över de bitar du redan hade skrivit ned. Här körs den kontinuerligt, en bit per klocka, utan någon följd att se tillbaka över och utan något "buffertslut" att stanna vid: vad regeln än behöver minnas måste bo i ett register. can_controller (kapitel 16–17) återanvänder **ett enda** tx_shift_reg för varje stoppat fält, och laddar om det och ändrar dess bitantal grupp för grupp. Den ostopgade svansen (CRC-avgränsare, ACK-plats och ACK-avgränsare, EOF) är fasta bitar som can_controller driver direkt, förbi det här registret.

Det här är den första modulen där det lönar sig att rita tidsdiagrammet innan du skriver koden. Ta tid till det; de flesta fel här är tajmingfel med en cykel, inte logikfel.

14.1 Gränssnitt



Port	Riktn.	Typ	Betydelse
clock, reset_s2_n	in	std_logic	50 MHz klocka, synkroniserad aktiv låg reset.
load	in	std_logic	Läser data/bit_count som nästa grupp och presenterar dess första bit den här cykeln.
data	in	byte_t	Gruppen, med sina bit_count giltiga bitar packade i de mest signifikanta bitarna (första biten som sänds är bit 7).
bit_count	in	std_logic_vector(3 downto 0)	Antal giltiga bitar i data, 1–8.
shift	in	std_logic	Pulsar en gång per CAN-bit (från bit_timer.bit_done) för att driva strömmen framåt.
tx_bit	out	std_logic	Registrerad bit att driva under den kommande bitperioden.
stuff	out	std_logic	'1' på just den skiftning som lägger ut en stoppbit; maskar bort den biten ur CRC-matningen.
done	out	std_logic	Pulsar när gruppen är helt utsänd (se tajmingen).
bit_valid	out	std_logic	'1' den enda cykel då en ny bit lades ut på tx_bit.

tx_bit är den enda utgången som är en *nivå*, hållen tills nästa bit presenteras. stuff, done och bit_valid är alla **pulsar på en cykel**: ge dem förvalet '0' högst upp i den klockade processen och höj dem bara i den gren som förtjänar dem, samma idiom som bit_timer använder för sample och bit_done.

Det spelar störst roll för `bit_valid` och `stuff`, eftersom `can_controller` bygger `crc15s` enable av paret, som `bit_valid` and not `stuff` (kapitel 16): `bit_valid` säger att en bit lades ut, och `stuff` maskar bort de utlagda bitar som CRC:n aldrig får se, eftersom CRC:n bara omfattar riktiga bitar. Håll `bit_valid` som en nivå i stället, så integrerar motorn samma bit vid varje klockflank under bitperioden, femtio gånger om, och förstör CRC:n i tysthet.

`load` och `shift` utesluter varandra inom en cykel; `can_controller` aktiverar aldrig båda. `tx_shift_reg_tb` binder **positionellt**, så ordningen och typerna ovan är det som måste stämma; namnen är dina.

14.2 Vad modulen måste minnas

Börja där inledningen pekade: pappersversionen av stoppbitsregeln såg *tillbaka över följden*, och hårdvaran har ingen följd att se tillbaka över. Så innan någon process skrivs, lista vad som måste bo i ett register. Allt nedan följer av modulens två uppgifter — att hålla en grupp i rörelse, och att hålla stoppbitsregeln ärlig — och varje regel i resten av det här kapitlet talar i termer av exakt de här fem signalerna:

```
signal shift_reg    : byte_t;                -- Remaining chunk bits, next ↵
    ↳ one in bit 7.
signal bits_left    : natural range 0 to 8;   -- Real bits of the chunk still ↵
    ↳ to present.
signal last_bit     : std_logic;              -- The bit most recently placed ↵
    ↳ on the wire.
signal consecutive  : natural range 0 to MAX_RUN; -- Current run length; 0 = none.
signal stuff_pending : std_logic;             -- A stuff bit is owed on the ↵
    ↳ next shift.
```

- **shift_reg** och **bits_left** är gruppen i flykt. Registret håller de osända bitarna vänsterjusterade så att "nästa bit" alltid är `shift_reg(7)`; `bits_left` är hur många riktiga bitar som återstår. Mer grupphållning än så finns inte.
- **last_bit** och **consecutive** är stoppbitsregelns minne, och tillsammans kallas de nedan för **följdspåraren**: biten som senast sändes, och hur många lika bitar i rad den avslutade. `MAX_RUN` lika bitar — fem, en konstant som `can_def` tillhandahåller, eftersom länkens båda ändar måste vara exakt överens om den — är en full följd; paret är allt regeln någonsin behövde av att "se tillbaka". Båda skrivs på exakt ett ställe, en procedur som processen deklarerar för ändamålet (nedan).
- **stuff_pending** är regelns enbitsskuld. När en följd når fem sänds stoppbiten *inte* omedelbart — den femte biten behöver fortfarande sin egen fulla period på bussen. Att nå fem *armerar* därför bara en insättning, och den här flaggan bär skulden vidare till nästa shift.

Utgångarna registreras i samma process: `tx_bit` är en nivå, hållen tills nästa bit ersätter den; `stuff`, `done` och `bit_valid` är encykelspulsarna som avsnitt 14.1 beskrev, som förval låga högst upp i processen och höjda bara i den gren som förtjänar dem.

14.3 Processen, uppifrån och ned

En klockad process håller allt ovanstående. Genomgången nedan följer koden i den ordning den skrivs, så varje rad har ett hem innan nästa dyker upp; kodfragmenten är processens deklarativa del följd av på varandra följande skivor av en enda `if/elsif`-kedja, och lagda i följd inuti skelettet nedan, vars resetgren du fyller i utifrån dess enradiga beskrivning, är de hela processen. En rad använder `to_integer(unsigned(...))`, så filen behöver use `ieee.numeric_std.all`; vid sidan av `ieee.std_logic_1164` och `work.can_def`.

Skelettet. Processen deklarerar sin egen procedur före `begin`; sedan asynkron reset först, allt annat på den stigande flanken:

```
process(clock, reset_s2_n) is
    -- One procedure, the run tracker (below).
begin
    if (reset_s2_n = '0') then
        -- Clear everything.
    elsif (rising_edge(clock)) then
        -- Defaults, then the load branch or the shift branch.
    end if;
end process;
```

14.3.1 Följdspåraren, skriven en gång

Varje gren som lägger ut en bit behöver samma få rader, så de bor i en procedur som deklaras i processens egen deklarativa del, mellan `process(...)` `is` och `begin`:

```
-- The run tracker: runs on the bit just placed on the wire.
procedure track_run(new_bit: in std_logic) is
begin
    if ((new_bit = last_bit) and (consecutive /= 0)) then
        consecutive <= consecutive + 1;
        if (consecutive = MAX_RUN - 1) then -- Run reaches MAX_RUN.
            stuff_pending <= '1';
        end if;
    else
        consecutive <= 1; -- Start a new run.
    end if;
    last_bit <= new_bit;
```

```
end procedure;
```

Det här är bokens första underprogram, så fyra saker om det innan något anropar det:

- **Den når signalerna direkt.** `last_bit`, `consecutive` och `stuff_pending` är arkitektursignaler, synliga inuti proceduren eftersom proceduren är deklarerad inuti processen, som ligger inuti arkitekturen. De är inte parametrar och ska inte vara det; bara den *utlagda biten* skiljer sig mellan anroparna, och det är den enda parameter som finns.
- **Att deklarerar den inuti processen är det som gör signaltilldelningarna lagliga alls.** Ett underprogram får tilldela en signal bara i två situationer: signalen är en formell parameter av klassen `signal`, eller underprogrammet är deklarerat inuti en process, och då använder dess tilldelningar den processens drivare, precis som om raderna hade skrivits på varje anropsställe. Den här proceduren vilar på det andra fallet. Flytta upp den oförändrad till arkitektursens deklarativa del så slutar den kompilera, ett fel per tilldelning: `signal "consecutive" is not a formal parameter`. Att träda `consecutive`, `last_bit` och `stuff_pending` genom som formella parametrar av klassen `signal` skulle uppfylla regeln, till priset av tre extra argument vid varje anrop och en verklig fråga om vilken process som äger drivarna. Deklarerad här kan bara den här processen anropa den, så bara den här processen driver dem.
- **Tajmingen är oförändrad.** En in-parameter evalueras vid anropet, så `new_bit` bär det värde före flanken som anroparen skickade in, och signalläsningarna inuti kroppen är också från före flanken (regeln om variabler kontra signaler från kapitel 5). Proceduren betar sig precis som samma rader skrivna inline.
- **Syntesen inlinar den.** Resultatet är samma logik replikerad under varje anropares grenvillkor; ingenting delas, multiplexas eller anropas vid körning. Det här är faktorisering på källkodsnivå, och nätlistan blir identisk hur som helst.

Två detaljer inuti kroppen, och en om var den anropas:

- **`consecutive = MAX_RUN - 1` betyder "följden är nu full".** Läsningen ger värdet före flanken, så jämförelsen sker mot följdlängden *innan* den här biten förlängde den. Den semantiken är precis rätt här, och det är därför koden jämför mot `MAX_RUN - 1` och inte `MAX_RUN`.
- **Håll reda på vilket `if` som äger vilken gren.** `consecutive <= 1` är det *yttre* `if`-satsens `else` (varje utlagd bit som bryter följden startar en ny), och `last_bit <= new_bit` står efter hela `if`-satsen och uppdateras vid *varje* utläggning, lika eller inte. Häng någon av dem på den inre `MAX_RUN - 1`-kontrollen, så startar en bruten följd aldrig om, eller så fryser `last_bit`.
- **Anropa den från en gren som lägger ut en bit, aldrig från processens toppnivå.** Ingenting på ledningen ändras under de dryga fyrtionio klockflanker som ligger mellan presentationerna, så ett anrop som körs vid varje flank räknar spökbiter. Grupper vet den ingenting om åt något håll: ledningen har ingen aning om var en grupp slutar och nästa börjar, så det har inte spåraren heller.

14.3.2 Reset och förval

Reset nollställer allt: utgångarna, gruppen (`shift_reg`, `bits_left`) och följdspåraren (`last_bit`, `consecutive`, `stuff_pending`). Det här är det **enda** stället spåraren någonsin nollställs. `load` får inte röra den: stoppningen ser den utsända strömmen som sammanhängande över hela ramen, så en följd som avslutar en grupp och fortsätter in i nästa tvingar fortfarande fram en stoppbit precis vid gränsen. (Det är scenariot med kedjad omladdning, test 3 i testbänken: en grupp slutar med fyra '1':or, nästa öppnar med en '1', och stoppbiten landar före den gruppens andra riktiga bit.)

Förvalen härnäst, och bara de här tre. Pulsutgångarna faller vid varje klockflanks början, `bit_timer`-idiomet, så grenarna nedan bara *höjer* dem:

```
stuff      <= '0';
done       <= '0';
bit_valid  <= '0';
```

`stuff_pending` hör **inte** hemma här, hur mycket den än liknar dem. Den är tillstånd, inte en puls: armerad vid den skiftning som fullbordar en följd, betald vid *nästa* skiftning, en hel bitperiod (femtio klockflanker) senare. Ge den förvalet lågt här, så avdunstar skulden vid den första lediga flanken däremellan, och då stoppar modulen helt enkelt aldrig in något. Den skrivs på exakt tre ställen: nollställd vid reset, armerad inuti `track_run`, och nollställd av den gren som betalar den.

14.3.3 Laddgrenen

load = '1' presenterar den första biten omedelbart. En bitperiod börjar i samma ögonblick som den föregående gruppens sista `bit_done` går, vilket också är när nästa grupps `load` går. Om `load` bara låste data och väntade på den första `shift`-pulsen skulle den första perioden fortfarande visa den föregående gruppens sista bit, en hel bit försent. Så grenen lägger ut `data(7)` nu, för registret i förväg förbi den, sätter `bits_left` till `bit_count - 1`, och kör sedan den utlagda biten genom följdspåraren:

```
if (load = '1') then
  tx_bit      <= data(7);                -- Output the first bit immediately.
  shift_reg   <= data(6 downto 0) & '0'; -- Remove the output bit and prepare the-
  ↳ next one.
  bits_left   <= to_integer(unsigned(bit_count)) - 1; -- bit_count is 1-8 by ↵
  ↳ contract:
  ↳
  ↳                               -- 0 would underflow ↵
  ↳ bits_left here.
  bit_valid   <= '1';                    -- Indicate that a valid bit has been ↵
  ↳ output.
```



```
track_run(data(7)); -- data(7) was just placed on the wire.
```

Anropet är varken valfritt eller en detalj: load lägger ut en bit på ledningen, så spåraren måste se den som vilken annan som helst. Utelämna det, så ligger följdträskningen en bit efter under resten av ramen; test 1 i testbänken fångar det vid skiftning 5, där stoppbiten som de fem '1':orna förtjänat uteblir.

14.3.4 Skiftgrenen

shift = '1' gör exakt en av tre saker (aldrig i samma cykel som load; det är därför grenen är ett elsif), provade i den här prioritetsordningen. Först den stoppbitt som är skyldig, om det finns någon:

```
elsif (shift = '1') then
  if (stuff_pending = '1') then
    tx_bit      <= not last_bit; -- The stuff bit inverts the run.
    stuff       <= '1';
    bit_valid   <= '1';
    stuff_pending <= '0';      -- The debt is paid.

    track_run(not last_bit); -- A stuffed bit is a placed bit like any ↪
    -- other.
```

En stoppbitt går genom spåraren precis som en riktig, och att skicka in not last_bit är det som gör att det blir rätt utan något specialfall: inuti kroppen är jämförelsen new_bit = last_bit då falsk per konstruktion, så anropet tar else-vägen och gör precis det regeln vill, startar om följden på 1 med stoppbittens eget värde i last_bit. Ingenting armerar någon ny skuld här heller, eftersom armeringen sitter i den gren som inte kan tas. Gruppen själv gör inga framsteg: shift_reg och bits_left lämnas orörda.

Härnäst den *avslutande* skiftningen:

```
elsif (bits_left = 0) then
  done <= '1'; -- The closing shift changes nothing else.
```

Varje bit har lagts ut och den sista har nu fått sin fulla period på bussen; tx_bit i synnerhet behåller sitt värde. I annat fall: sänd nästa riktiga bit, samma form som load-grenen med shift_reg(7) i data(7):s ställe:

```
else
  tx_bit  <= shift_reg(7); -- Output the next bit.
  shift_reg <= shift_reg(6 downto 0) & '0';
```

```

        bits_left <= bits_left - 1;
        bit_valid <= '1';

        track_run(shift_reg(7));           -- May arm a stuff bit for the next --
    -- shift.
    end if;
end if;

```

Prioritetsordningen är bärande: en väntande stoppbit går före den avslutande skiftningen, och det är det som får specialfallet nedan att fungera utan någon extra kod; en grupp vars *sista* riktiga bit fullbordar en följd får ändå sin stoppbit utsänd innan *done* går. (load har aldrig någon utestående stoppbit att bekymra sig om: *can_controller* laddar bara efter *done*, och *done* kommer efter att varje väntande stoppbit har sänts.)

14.4 Utgångarnas tajming, den del testbänken låser exakt

- *tx_bit* sätts av **både** load och shift; den avslutande skiftningen lämnar den oförändrad.
- stuff pulsar bara vid den skiftning som faktiskt lägger ut en stoppbit, aldrig vid load (en grupp:s första bit kan *utlösa* en stoppbit, men den stoppbiten sänts vid en senare skiftning).
- *done* pulsar **inte** vid den skiftning som presenterar gruppens sista riktiga bit eller stoppbit; den biten behöver först sin egen fulla period på bussen. Den pulsar en skiftning senare, vid den avslutande skiftningen, som inte presenterar något nytt. Att låta *done* gå tidigt skulle låta *can_controller* ladda nästa grupp en hel bitperiod för tidigt.
- *bit_valid* är '1' vid load och vid varje skiftning som lägger ut en ny bit (riktig eller stoppbit), och '0' vid den avslutande skiftningen. Konsumenter som för *tx_bit* vidare bit för bit grindar på *bit_valid*, inte på "load eller shift", annars skulle de sampla den avslutande skiftningens oförändrade *tx_bit* som en dubblett. *crc15* är den konsument som spelar roll, och den behöver stuff också: kapitel 16 grindar dess enable på *bit_valid* and not stuff, eftersom en stoppbit läggs ut på ledningen men aldrig får gå in i CRC:n.
- Specialfall: om *bit_count* = 1 och den enda biten utlöser en stoppbit, väntar *done* ändå på att stoppbiten sänts först.

14.5 En grupp, presentation för presentation

Fyra tajmingregler samspelar här, och prosa är ett dåligt sätt att se dem. Nedan följer en komplett grupp: *data* = "11111000", *bit_count* = 8, laddad in i ett nyss resettat register. Varje rad är en *presentation*, antingen load eller en shift-puls, och visar utgångarna efter den. Det här är samma sekvens som *tx_shift_reg_tb* kontrollerar i sitt första test, så din egen modul måste återge den rad för rad.

	databit	tx_bit	stuff	bit_valid	done	consecutive
ladda	1	1	0	1	0	1
skifta 1	2	1	0	1	0	2
skifta 2	3	1	0	1	0	3
skifta 3	4	1	0	1	0	4
skifta 4	5	1	0	1	0	5
skifta 5	-	0	1	1	0	1
skifta 6	6	0	0	1	0	2
skifta 7	7	0	0	1	0	3
skifta 8	8	0	0	1	0	4
skifta 9	-	0	0	0	1	4

Tio presentationer för att sända åtta bitar. Var och en av de fyra reglerna syns i den:

- **load presenterar omedelbart.** Första raden har redan databit 1 på tx_bit, med bit_valid hög. Ingen shift har hänt ännu. Hade load bara låst gruppen skulle den här raden fortfarande visa den *föregående* gruppens sista bit, och hela ramen skulle gå en bitperiod försent.
- **Följdräkningen driver allt.** Den klättrar från 1 till 5 över de fem '1':orna, och vid skiftning 4 når den fem. Det lägger inte ut någon stoppbit; det *armerar* en till nästa presentation.
- **stuff är en presentation bred och bär inga data.** Skiftning 5 driver not last_bit, markerar stuff = '1' och nollställer följdén till 1. Databitkolumnen visar -: gruppen har inte gjort några framsteg, vilket är varför åtta databitar behöver nio bitläggande presentationer här. bit_valid är fortfarande '1', eftersom en bit verkligen lades ut på ledningen; den var bara inte en av dina.
- **done släpar med en, och bit_valid markerar skillnaden.** Skiftning 8 lägger ut den sista riktiga biten, och done förblir låg, eftersom den biten ännu inte fått sin bitperiod på bussen. Skiftning 9 är den avslutande skiftningen: tx_bit håller sitt tidigare värde, ingenting nytt läggs ut, bit_valid faller till '0', och först nu pulsar done.

Den sista raden är den som är värd att stirra på. tx_bit läser fortfarande '0', precis som vid skiftning 8, så vad som helst som grindar enbart på shift skulle mata crc15 med samma bit en andra gång och tyst korrumpiera checksumman. bit_valid är den enda utgången som skiljer "en bit lades ut" från "en period förflöt", vilket är varför den finns och varför kapitel 16 grindar CRC:n på den.

Lägg slutligen märke till att consecutive slutar på 4 på en följd av '0':or, och inte nollställs. Nästa load ärver både räkningen och last_bit, så om den gruppen öppnar med en '0' når följdén fem redan vid sin allra första bit och en stoppbit tvingas fram före dess andra. Öppnar den i stället med en '1' bryts följdén och räkningen börjar om på 1. Det är samma mekanism som den kedjade omladdningen i testbänkens test 3, där en grupp i stället ärver en följd av fyra '1':or och öppnar med en '1'.

14.6 Vad testbänken låser fast, och att lägga in modulen

- "11111000" ger en stoppbit ('0') direkt efter den femte '1':an; en följd på exakt fyra stoppas *inte*.
- Att ladda "10101010" direkt efter en grupp som slutade med fyra '1':or (utan reset emellan) tvingar fram en stoppbit före den här gruppens andra riktiga bit, vilket bevisar att följdtilståndet lever kvar över load.
- bit_valid är '1' vid load och vid varje skiftning med riktig bit eller stoppbit, '0' vid den avslutande skiftningen; done pulsar bara vid just den avslutande skiftningen.
- bit_valid är tillbaka på '0' cykeln *efter* varje presentation, så en version som håller nivån underkänns även om den lägger ut varje bit korrekt.

14.6.1 Att lägga in den i can_controller

Tredje blocket, samma redigering. Signalgruppen i controller/can_controller.vhd är större här, eftersom tx_shift_reg har mer att säga:

```
-- tx_shift_reg.
signal txsr_load                : std_logic;
signal txsr_data                : byte_t;
signal txsr_bit_count          : std_logic_vector(3--
-- downto 0);
signal txsr_shift               : std_logic;
signal txsr_tx_bit, txsr_stuff, txsr_done, txsr_bit_valid : std_logic;
```

```
tx_shift_reg1: entity work.tx_shift_reg
  port map(clock, reset_s2_n, txsr_load, txsr_data, txsr_bit_count, --
-- txsr_shift,
          txsr_tx_bit, txsr_stuff, txsr_done, txsr_bit_valid);
```

En enda instans, återanvänd för varje stoppat fält i varje ram. Om det fortfarande ser förvånande ut, läs om kapitlets inledning: registret laddas om med en ny byte och ett nytt bit_count per fält, vilket är precis det som gör att en kopia räcker. (Ta reset_s2_n-kopplingen som den är tills vidare; kapitel 17 byter den mot en reset per ram, av ett skäl bara arbitreringsen kan visa.)

14.7 Sammanfattning

Det här ska du kunna nu

- Implementera ett skiftregister som serialiserar MSB först och sätter in stoppbitar enligt femregeln.
- Förklara varför stoppningsräknaren inte får nollställas när en ny grupp laddas.
- Skilja på `bit_valid` och `bit_done`, och säga vilken av dem CRC-motorn ska grindas på.
- Säg vad stuff är till för, och varför den behövs utanför modulen.

Frågor att testa dig själv med

1. Varför måste den första biten i en nyladdad grupp ligga ute direkt, i stället för vid nästa shift?
2. Varför nollställs inte stoppningsräknaren vid omladdning?
3. Vad skulle gå fel om `can_controller` matade CRC-motorn på `bit_done` i stället för på `bit_valid` and not stuff?
4. En stoppbit har alltid motsatt värde mot de fem föregående. Varför räcker inte "sätt in en nolla"?
5. Varför väntar `done` en extra skiftning i stället för att gå hög direkt när sista databiten skiftats ut?

Riktvärde

Ungefär halvvägs genom projektet. Tre av fyra lövmoduler brukar vara klara här, och det är också ungefär nu det blir tydligt vilken grupp som faktiskt granskar varandras kod och vilken som bara klickar "Approve".

Blicken framåt

Kapitel 15 bygger `rx_shift_reg`, mottagarsidans spegelbild: deserialisering, avstoppning och flaggning av brott mot stoppbitsregeln. Kapitel 16 sätter sedan båda registren i arbete inuti `can_controller`.

14.8 Övningar

De här övningarna befäster kapitlet. Konstruera din egen `tx_shift_reg.vhd` utifrån specifikationen i avsnitt 14.1 till 14.5; den utdelade testbänken, körd enligt bilaga A, är kontrollen av att den beter sig korrekt.

Övning 14.1 tx_shift_reg

VHDL

Skriv din egen tx_shift_reg.vhd utifrån kapitlet. Verifiera den:

```
cd controller
ghdl -a --std=93 can_def.vhd tx_shift_reg.vhd tx_shift_reg_tb.vhd
ghdl -e --std=93 tx_shift_reg_tb
ghdl -r --std=93 tx_shift_reg_tb --assert-level=error
```

Om en kontroll fallerar, läs assertion-meddelandet; det namnger exakt vilket skift det gäller och vad som förväntades.

Övning 14.2 Följ en stoppad sekvens för hand, sedan i simulering

SIMULERING

a) Följ för hand tx_shift_reg laddad med data = "11100000", bit_count = 8, med start vid en gruppgräns där den föregående gruppens sista bit var 0 med en följdlength på 3 (alltså consecutive = 3 och last_bit = '0' in i gruppen). Skriv ut, bit för bit: vad som presenteras vid load, och sedan vid varje efterföljande shift, med varje instoppad stoppbit markerad och med angivande av vilket skift som höjer done.

b) Bekräfta din handföljning i simulering: skriv en kort testbänk (eller återanvänd strukturen i tx_shift_reg_tb.vhd) som förladdar samma startvillkor med följdlength 3, sedan laddar "11100000" och vid varje skift assert:ar tx_bit, stuff och done mot värdena du förutsade (eller report:ar dem så att du kan läsa dem). Stämmer dess utfall av passera eller falla med din handföljning?

Övning 14.3 Enbitsgruppen som ändå är skyldig en stoppbit

SIMULERING

Avsnitt 14.4:s sista punkt namnger ett gränsfall utan att arbeta igenom det: om bit_count = 1 och den enda biten utlöser en stoppbit, väntar done ändå på att stoppbiten skickas först. Det är inte hypotetiskt; can_controller laddar enbitsgrupper (SOF) och fyrabitsgrupper (ID:ts undre halva plus RTR), så en grupp kan verkligen sluta mitt i en följd.

Sätt upp det precis som test 2 och 3 i tx_shift_reg_tb gör, utan reset emellan: ladda "00001111" med bit_count = 8 och skifta ut den helt, så att följdräknaren står på fyra '1':or i rad. Ladda sedan en **enbitsgrupp** vars enda bit också är '1' (data = "1-----", bit_count = 1).

a) Följ vad som händer, presentation för presentation: vad load lägger på tx_bit, vad följdräknaren når, och sedan vad varje följande shift gör. Markera vilken presentation som bär stoppbiten och vilken som höjer done.

b) Hur många shift-pulser förbrukar den här enbitsgruppen totalt, och hur många skulle det ha förbrukat om följdräknaren i stället stått på noll? Säg var skillnaden kommer ifrån.

- c) Anta att done hade fått pulsa så snart gruppens enda riktiga bit presenterats, utan hänsyn till den väntande stoppbiten. `can_controller` skulle då ladda nästa grupp under nästa bitperiod. Beskriv exakt vad den mottagande noden i så fall skulle se på ledningen, och vilken av `rx_shift_reg`:s utgångar som till slut skulle rapportera det (kapitel 15).
- d) Bekräfta din genomgång i simulering, genom att utöka mönstret från test 2 och test 3 i `tx_shift_reg_tb.vhd` med den här enbitsgruppen som ett fjärde fall.

Övning 14.4 Varifrån `track_run` anropas

SIMULERING

Avsnitt 14.3.1 lägger följdräknaren i en procedur och är tydlig med var den proceduren får anropas: från en gren som placerar en bit, aldrig från processens toppnivå. Båda halvorna av den regeln är värda att bryta med flit, i en kladdkopia av din modul.

- a) Flytta ut anropet `track_run(data(7))`; ur grenen `load`, så att `load` låser gruppen och presenterar dess första bit men räknaren aldrig ser den biten. Förutsäg vad följdräknaren då visar under resten av ramen, och vilket av testbänkens tre tester som fallerar först. Kör det.
- b) Lägg nu ett enda `track_run(tx_bit)`; högst upp i grenen `elsif (rising_edge(clock))`, ovanför standardvärdena, och ta bort alla tre anropen inuti grenarna. `tx_bit` är en out-port, så börja med att säga varför det inte ens kompilerar, och upprepa sedan experimentet med en intern kopia av den sända biten i dess ställe. Med `TICKS_PER_BIT` satt till 50, hur många gånger körs räknaren nu per CAN-bit, och vad gör consecutive på den andra klockflanken i den allra första bitperioden?
- c) Avsnitt 14.3.1 säger också att proceduren måste deklareras *inuti* processen i stället för i arkitekturen. Flytta ut den till arkitektursens deklarativa del, oförändrad, och analysera filen. Den kompilerar inte; läs de fyra felen innan du går vidare. Gör sedan en version på arkitektursnivå som *faktiskt* kompilerar, med identiskt beteende och en testbänk som passerar. Jämför de två anropsställena och säg vad den andra versionen kostar, och vilken ny fråga den väcker som versionen inuti processen inte kan väcka.

Mottagningskiftregister och avstoppning

Mottagarsidans spegelbild: deserialisering, borttagning av stoppbitar, och flaggning av stoppningsfel.

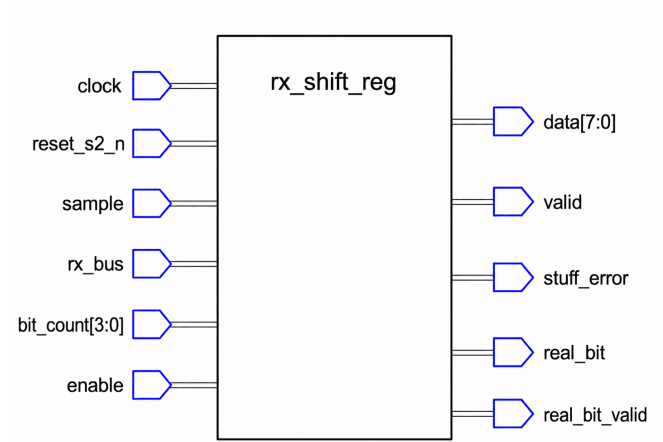
I DET HÄR KAPITLET

- `rx_shift_reg`: deserialisering, avstoppning och detektering av stoppningsfel.
- Att sampla mitt i bitperioden (`bit_timer.sample`) snarare än vid `bit_done`.
- `real_bit` och `real_bit_valid`: att mata `crc15` en avstoppad bit i taget.
- Att instansiera `rx_shift_reg` inuti `can_controller`.

`rx_shift_reg` är mottagarsidans spegelbild av kapitel 14:s `tx_shift_reg`: den deserialiserar en grupp MSB först från bussen och **kastar** de stoppbitar som `tx_shift_reg` satte in, och flaggar för ett brott om en väntad stoppbit uteblir. Följdräkningen och regeln om fem lika i rad är identiska, bara omvända — kapitel 3 i *CAN: bussen, ramen och kontrollern* är samma regel sedd från protokollet, och det är värt att läsa den en gång till härifrån, eftersom det är den enda regel i boken som två moduler måste vara oberoende överens om. `can_controller` (kapitel 17) återanvänder **ett enda** `rx_shift_reg` för varje stoppat fält i en inkommande ram, och ändrar dess `bit_count` mellan grupperna; den ostopgade svansen (CRC-avgränsare, ACK, ACK-avgränsare, EOF) läser den direkt, förbi det här registret.

Det här kapitlet innehåller också projektets **obligatoriska kodgranskningsseminarium**; se bilaga C.

15.1 Gränssnitt



Port	Riktn.	Typ	Betydelse
clock, reset_s2_n	in	std_logic	50 MHz klocka, synkroniserad aktiv låg reset.
sample	in	std_logic	En puls per CAN-bit (<code>bit_timer.sample</code>) som markerar när <code>rx_bus</code> ska läsas.
rx_bus	in	std_logic	Bussnivån, giltig vid sampelpunkten.
bit_count	in	std_logic_vector(3 downto 0)	Antal riktiga bitar att samla in, 1–8; läses live, får ändras mellan grupper.
enable	in	std_logic	Avgör om sample-pulser behandlas.
data	out	byte_t	Den insamlade gruppen, högerjusterad i sina lägsta <code>bit_count</code> bitar.
valid	out	std_logic	Pulsar när <code>bit_count</code> riktiga bitar samlats in.
stuff_error	out	std_logic	Pulsar om en väntad stoppbit inte inverterar.
real_bit	out	std_logic	Den enda avstoppade bit som accepterades vid det här samplet.
real_bit_valid	out	std_logic	'1' vid ett sampel som accepterade en riktig bit; '0' vid en kastad stoppbit.

`data` och `real_bit` är nivåer; `valid`, `stuff_error` och `real_bit_valid` är **pulser på en cykel**, sätta till '0' som förval högst upp i den klockade processen, precis som `tx_shift_regs`. `real_bit_valid` är den som biter: `can_controller` grindar `crc15s` `enable` direkt på den (kapitel 17), så en nivå som ligger kvar till nästa sampel matar in samma bit under en hel bitperiod och förstör CRC:n i tysthet. Testbänken kontrollerar att den fallit igen cykeln efter varje sampel, av precis det skälet.

Det finns ingen `load`: samplingen sker mitt i perioden (`sample`, inte `bit_done`), när alla noders signaler har hunnit stabilisera sig. En sändare ändrar bussen vid en periodgräns, så en mottagare som också agerade på `bit_done` skulle läsa i samma ögonblick som nivån ändras;

avsnitt 12.2.3:s tickspår visar de två pulserna och de 14 ticken mellan dem. `can_controller` ändrar bara `bit_count` mellan grupperna; `valid` rapporterar själv varje grupps fullbordan. `rx_shift_reg_tb` binder **positionellt**, så ordningen och typerna ovan är det som måste stämma; namnen är dina.

15.2 Vad modulen måste minnas

Som i kapitel 14: lista tillståndet innan någon process skrivs. Mottagarsidan har samma två uppgifter, speglade — hålla en grupp *under påfyllning*, och hålla stoppbitsregeln ärlig — plus en som den inte hade: att *förutsäga* var sändaren måste ha satt in en stoppbit. Varje regel nedan talar i termer av exakt de här fem signalerna:

```

signal data_reg      : byte_t;                -- Collected real bits, filling-
    <- from the right.
signal collected     : natural range 0 to 8;  -- Real bits accepted toward -
    <- this chunk so far.
signal last_bit      : std_logic;             -- The bit most recently -
    <- sampled off the bus.
signal consecutive   : natural range 0 to MAX_RUN; -- Current run length; 0 = none-
    <- .
signal stuff_expected: std_logic;            -- The next sampled bit must be-
    <- a stuff bit.

```

- **data_reg** och **collected** är gruppen som fylls på. Varje accepterad bit skiftas in från höger (`data_reg(6 downto 0) & rx_bus`), vilket är där högerjusteringen kommer ifrån, och `collected` räknar accepterade bitar mot `bit_count`. Registret bakom `data` måste vara den här interna signalen, med `data <= data_reg`; som en konkurrent tilldelning utanför processen: en out-port kan inte *läsas*, och en skiftning är en läs-modifiera-skriv.
- **last_bit** och **consecutive** är samma följdspårare som kapitel 14:s, tecken för tecken: biten som senast låg på ledningen, och hur lång den identiska följd den avslutade är. Länkens båda sidor kör samma spårare över samma bitar, mot samma `MAX_RUN` från `can_def`; det är det som håller dem överens om var stoppbitarna hör hemma. Som i kapitel 14 skrivs paret från en enda procedur som processen deklarerar.
- **stuff_expected** är spegelbilden av kapitel 14:s **stuff_pending** — samma flagga med motsatt uppgift. Sändarens flagga säger "jag är *skyldig* en stoppbit härnäst"; mottagarens säger "nästa bit gör klokt i att *vara* en". Samma armeringsvillkor (en följd som når fem), samma tajming (kontrolleras vid nästa händelse), motsatt riktning på skyldigheten — och mottagarens version kan bli *besviken*, och det är där `stuff_error` kommer ifrån.

Utgångarna registreras i samma process: `data` och `real_bit` är nivåer; `valid`, `stuff_error` och `real_bit_valid` är encykelpulserna som avsnitt 15.1 beskrev.

15.3 Processen, uppifrån och ned

En klockad process håller allt ovanstående, plus en konkurrent tilldelning utanför den. Som i kapitel 14 följer genomgången koden i den ordning den skrivs. **Följdspåraren** är rad för rad kapitel 14:s, med bara flaggan omdöpt efter vad den nu betyder: samma procedur, deklarerad inuti den här modulens process och anropad från varje gren som tar en bit av ledningen utom en, och undantaget noteras där det uppstår. Som i kapitel 14 jämförs `collected` mot `bit_count` via `to_integer(unsigned(...))`, så filen behöver use `ieee.numeric_std.all`;

Skelettet. Den konkurrenta raden först: `data` är en out-port och kan inte *läsas*, medan en skiftning är en läs-modifiera-skriv, så registret är den interna `data_reg` och porten speglar bara den. Sedan processen, med sin procedur deklarerad före `begin`, asynkron reset först och allt annat på den stigande flanken:

```
data <= data_reg; -- Concurrent, outside the process.

process(clock, reset_s2_n) is
    -- One procedure, the run tracker (below).
begin
    if (reset_s2_n = '0') then
        -- Clear everything.
    elsif (rising_edge(clock)) then
        -- Defaults, then the sample branch.
    end if;
end process;
```

15.3.1 Följdspåraren, skriven en gång

Kapitel 14:s procedur med `stuff_expected` i stället för `stuff_pending`; den enda omdöpningen är hela skillnaden på mottagarsidan. Allt som avsnitt 14.3.1 säger om idiomet (varför den läser signalerna direkt, varför den deklarerar inuti processen i stället för i arkitekturen, varför en in-parameter bevarar tajmingen före flanken, och varför syntesen inlinar den) gäller här oförändrat:

```
-- The run tracker: runs on the bit just taken off the wire.
procedure track_run(new_bit: in std_logic) is
begin
    if ((new_bit = last_bit) and (consecutive /= 0)) then
        consecutive <= consecutive + 1;
        if (consecutive = MAX_RUN - 1) then -- Run reaches MAX_RUN.
            stuff_expected <= '1';
        end if;
    else
        consecutive <= 1; -- Start a new run.
```

```

end if;
last_bit <= new_bit;
end procedure;

```

15.3.2 Reset och förval

Reset nollställer allt: gruppen (data_reg, collected), följdspåraren (last_bit, consecutive, stuff_expected) och utgångarna. Det här är det **enda** stället spåraren någonsin nollställs. Grupperna nollställer den inte: sändarens ström är sammanhängande över hela ramen, så mottagarens bild av den måste vara det också.

Förvalen härnäst, och bara de här tre, så att grenarna nedan bara *höjer* dem:

```

valid          <= '0';
stuff_error    <= '0';
real_bit_valid <= '0';

```

stuff_expected hör **inte** hemma här, av exakt samma skäl som stuff_pending inte hörde hemma i kapitel 14:s förval: den är tillstånd, armerad vid det sampel som fullbordar en följd och avgjord vid *nästa* sampel, en hel bitperiod (femtio klockflanker) senare. Ge den förvalet lågt så avdunstar förutsägelsen innan den kan förfalla, och då blir stoppbitar varken kastade eller kontrollerade. Den skrivs på exakt tre ställen: nollställd vid reset, armerad inuti track_run, och nollställd när förutsägelsen förfaller (båda utfallen nedan).

15.3.3 Samplingsgrenen

(sample = '1') and (enable = '1') är den **enda arbetande grenen**; det finns ingen load på mottagarsidan. Den gör exakt en av tre saker, prövade i den här prioritetsordningen. Först: en utestående förutsägelse förfaller, och den samplade biten avgör den åt det ena eller andra hållet:

```

if ((sample = '1') and (enable = '1')) then
  if (stuff_expected = '1') then
    if (rx_bus /= last_bit) then
      stuff_expected <= '0';      -- The stuff bit arrived; prediction settled.
      .

      track_run(rx_bus);          -- A stuff bit is a bit off the wire like ↵
      . any other.
    else
      stuff_error    <= '1';      -- Six identical bits: a stuffing violation.
      stuff_expected <= '0';
      consecutive    <= 1;        -- Restart the run at the violating bit; the

```

```

-- tracker must never count past MAX_RUN.

end if;

```

Den anlända stoppbiten *kastas*: ingen skiftning, ingen ändring av `collected`, och `real_bit_valid` förblir '0', eftersom en stoppbitt aldrig är en riktig bit (det här är biten som `crc15` inte får se). Anropet till spåraren är spegelbilden av kapitel 14:s i dess stoppgren: en kastad stoppbitt resynkar spåraren precis som en utsänd gör, eftersom ledningen inte skiljer riktiga bitar från stoppbitar och spåraren inte heller får göra det. Här behöver den inget särskilt argument, eftersom `rx_bus /= last_bit` är den här grenens eget villkor, så `track_run(rx_bus)` tar vägen för en ny följd per konstruktion och startar om räkningen på 1.

I `else`-grenen är sex identiska bitar i rad omöjliga på en korrekt stoppad buss, så `stuff_error` pulsar. Det snyggaste för resten är att nollställa flaggan, inte acceptera något, och starta om följdräkningen vid den brytande biten (`last_bit` håller redan dess värde), eftersom det inte är den här modulens uppgift att avgöra vad ett brott *betyder*. Omstarten är inte valfri prydlighet: utan den ligger consecutive kvar på `MAX_RUN`, nästa identiska bit kör spårarens + 1, och signalens deklarerade intervall överskrids; testbänken matar in en bit förbi brottet för att låsa fast precis det. Det här är också den enda placering `track_run` inte kan tjäna, och av samma skäl: `rx_bus = last_bit` här, så anropet skulle ta + 1-vägen med consecutive redan på `MAX_RUN`. Den brytande biten är undantaget från regeln proceduren kodar, så dess två rader står kvar utskrivna. `can_controller` behandlar den som fatal och avbryter mottagningen (kapitel 17), precis som en riktig controller skulle göra.

I annat fall: acceptera en riktig bit, skifta in den från höger, exponera den bitvis för `crc15`, avsluta gruppen om den här biten fullbordar den, och kör spåraren på biten som just accepterats:

```

else
    data_reg      <= data_reg(6 downto 0) & rx_bus; -- Shift in from the
-- right.
    real_bit      <= rx_bus;
    real_bit_valid <= '1';

    if (collected = to_integer(unsigned(bit_count)) - 1) then
        valid      <= '1';                -- This bit completes the chunk.
        collected <= 0;
    else
        collected <= collected + 1;
    end if;

    track_run(rx_bus); -- rx_bus was just accepted off the wire.
end if;
end if;

```

Jämförelsen här är läsningen av signalen före flanken från kapitel 14: `collected` mot `bit_count - 1` är idiomat "följd-är-nu-full" tillämpat på gruppens fullbordan, samma form som `consecutive = MAX_RUN - 1` inuti `track_run`. Ingenting annat händer vid en gruppgräns: `bit_count` läses live, så nästa grupp börjar helt enkelt med nästa accepterade bit i den bredd `can_controller` satt vid det laget, och `data_reg` nollställs *inte*; den nya gruppens bitar skiftas in under vad den gamla gruppen än lämnade ovanför dem (Övning 15.3 handlar om precis det).

Två spegelnoteringar värda att stanna vid:

- **Det finns inget avslutande sampel.** Kapitel 14:s `done` var tvungen att släpa en skiftning, eftersom en sänd bit ändå behövde sin fulla period på bussen. En mottagare samplar mitt i perioden, när biten redan fått sin stabiliseringstid, så i samma ögonblick som den sista riktiga biten accepteras är gruppen helt enkelt *där*: `valid` går vid just det samplet, utan något att vänta på.
- **`data/valid` är gruppvisa, `real_bit/real_bit_valid` bitvisa.** En hel grupp är för grov för `crc15`, som behöver varje riktig bit när den anländer; `real_bit` och `real_bit_valid` exponerar exakt den enda accepterade biten, mottagarsidans motsvarighet till `tx_shift_regs` par `bit_valid/stuff`. Motsvarighet, inte spegelbild: på sändarsidan bär stoppbitar fortfarande `bit_valid = '1'` och anroparen maskar bort dem med `stuff`, medan den kastade stoppbiten här aldrig höjer `real_bit_valid` alls, så `real_bit_valid` ensam är redan CRC-grindningen (kapitel 17).

15.4 En grupp, sampel för sampel

Att ta emot är inte helt enkelt kapitel 14 baklänges. En sändare *bestämmer sig* för att sätta in en stoppbitt; en mottagare *förutsäger* en och måste sedan kontrollera om den faktiskt kom. Den skillnaden är där båda den här modulens intressanta utgångar kommer ifrån.

Här är ledningssekvensen som `tx_shift_reg` producerade för "11111000" i avsnitt 14.5, anländande till ett nyss resettat `rx_shift_reg` med `bit_count = 8`. Varje rad är en samplepuls. Kolumnen följd är antalet lika bitar i rad efter samplet, och `data` visar registret fyllas nedifrån.

	<code>rx_bus</code>	åtgärd	<code>real_bit</code>	<code>real_bit_valid</code>	följd	<code>data</code>	<code>valid</code>
sampel 1	1	acceptera	1	1	1	00000001	0
sampel 2	1	acceptera	1	1	2	00000011	0
sampel 3	1	acceptera	1	1	3	00000111	0
sampel 4	1	acceptera	1	1	4	00001111	0
sampel 5	1	acceptera	1	1	5	00011111	0
sampel 6	0	kasta	-	0	1	00011111	0
sampel 7	0	acceptera	0	1	2	00111110	0
sampel 8	0	acceptera	0	1	3	01111100	0
sampel 9	0	acceptera	0	1	4	11111000	1

Nio sampel för att återvinna åtta bitar, exakt de nio presentationer kapitel 14:s sändare behövde för att skicka dem. Tre rader bär lärdomen:

- **Sampel 5** tar den femte '1':an i rad. Ingenting kastas här. Att nå fem betyder bara att *nästa* bit förväntas vara en stoppbit; den här biten är riktig och samlas in som vanligt.
- **Sampel 6** är den kastade stoppbiten. `real_bit_valid` faller till '0' och data rör sig inte: gruppen gör inga framsteg. Det här är raden som spelar roll för kapitel 13:s motor, eftersom `crc15` inte får se den här biten. Följden resynkar till 1 och tar stoppbittens eget värde som början på den nya följden.
- **Sampel 9** fullbordar den åttonde *riktiga* biten, så `valid` pulsar och data läser 11111000, den ursprungliga gruppen. Stoppbiten har försvunnit utan att `can_controller` någonsin fått veta att den fanns.

Lägg märke till att data fylls från höger, en position per accepterad bit. Efter nio sampel råkar gruppen uppta alla åtta bitarna, men en 6-bitarsgrupp skulle sluta med sina bitar i `data(5 downto 0)` och lämna det som ligger ovanför i fred. Det är högerjusteringen, och Övning 15.3 handlar om vad som sitter ovanför.

När stoppbiten inte kommer. Samma mekanism, en bit annorlunda. Sex '0':or i rad där den sjätte skulle ha inverterat:

	<code>rx_bus</code>	väntat	åtgärd	<code>real_bit_valid</code>	följd	<code>stuff_error</code>
sampel 1	0	riktig bit	acceptera	1	1	0
sampel 2	0	riktig bit	acceptera	1	2	0
sampel 3	0	riktig bit	acceptera	1	3	0
sampel 4	0	riktig bit	acceptera	1	4	0
sampel 5	0	riktig bit	acceptera	1	5	0
sampel 6	0	stopp '1'	brott	0	1	1

Brottet upptäcks vid sampel 6, inte sampel 5. Fem identiska bitar är fullt lagliga och krävs till och med innan någon stoppbit alls existerar; det är den sjätte, som anländer där en inverterad bit var skyldig, som är omöjlig på en korrekt stoppad buss. `stuff_error` pulsar under en cykel och modulen fortsätter samla in; att avgöra vad ett brott *betyder* hör till `can_controller`, som avbryter ramen (kapitel 17).

15.5 Vad testbänken låser fast

- Ledningssekvensen för "11111000" (stoppbit efter den femte '1':an) avstoppas tillbaka till "11111000", `valid` pulsar vid det sista samplet, och `real_bit_valid` är '0' vid den kastade stoppbiten och '1' (med `real_bit = rx_bus`) vid varje riktig bit.
- `real_bit_valid` är tillbaka på '0' cyklern *efter* varje sampel, så en version som håller nivån underkänns även om den avstoppas perfekt.

- Sex '1':or i rad höjer stuff_error vid det sjätte samplet, och en sjunde identisk bit accepteras som vanligt efteråt: följdrukningen startade om vid den brytande biten i stället för att räkna förbi MAX_RUN.
- En 6-bitarsgrupp pulsar valid vid sitt sjätte sampel, inte sitt åttonde, och landar högerjusterad i data(5 downto 0).
- En 3-bitarsgrupp som matas in omedelbart därefter, **utan någon reset emellan**, plockar upp den nya bit_count och landar i data(2 downto 0). Det här är den omdimensionering från grupp till grupp som can_controller utför mellan varje fält.

15.6 Att lägga in den i can_controller

Det sista av de fyra CAN-blocken, och det första vars port map kopplas till något annat än klockan och sina egna signaler:

```
-- rx_shift_reg.
signal rxsr_bit_count          : std_logic_vector(3 downto 0);
signal rxsr_enable             : std_logic;
signal rxsr_data               : byte_t;
signal rxsr_valid, rxsr_stuff_error : std_logic;
signal rxsr_real_bit, rxsr_real_bit_valid : std_logic;
```

```
rx_shift_reg1: entity work.rx_shift_reg
  port map(clock, reset_s2_n, bt_sample, rx_bus_s2, rxsr_bit_count,
    rxsr_enable,
      rxsr_data, rxsr_valid, rxsr_stuff_error, rxsr_real_bit,
    rxsr_real_bit_valid);
```

Två av kopplingarna där är värda att stanna vid. sample kommer direkt från bit_timer1:s utsignal, så mottagarvägen samplar på samma 70-procentstick som resten av designen tajmar mot; ingenting däremellan bestämmer när bussen ska läsas. Och bussnivån är rx_bus_s2, utgången från kapitel 12:s instans rx_bus_sync, **inte** porten rx_bus själv. rx_shift_reg är det enda block som läser bussen, och det läser den synkroniserade kopian; ingenting i designen läser den råa pinnen. (Som med tx_shift_reg: ta reset_s2_n-kopplingen som den är tills vidare; kapitel 17 flyttar båda skiftregistren till en reset per ram.)

Med det här håller can_controllers arkitektur alla fyra CAN-delblocken och deras signaler, vid sidan av kapitel 12:s meta_prev, och inget rambeteende alls. Varje återstående koppling mellan dem är kapitel 16:s tillståndsmaskin.

15.7 Sammanfattning

Det här ska du kunna nu

- Implementera ett skiftregister som deserialiserar från bussen och kastar stoppbitar.
- Upptäcka ett stoppningsfel och säga exakt vilket villkor som utgör det.
- Förklara varför mottagaren samplar vid `sample` och sändaren skiftar vid `bit_done`.
- Mata CRC-motorn från mottagarsidan utan att räkna stoppbitar.
- Läså någon annans VHDL och skriva en granskning som är användbar snarare än artig.

Frågor att testa dig själv med

1. Varför samplar mottagaren vid `sample` snarare än vid `bit_done`?
2. Exakt vilket villkor utgör ett stoppningsfel, och varför är det sex lika bitar som är gränsen på mottagarsidan när sändaren räknar fem?
3. Vad händer med stoppbiten när den är korrekt?
4. Varför får `crc15` bara se `real_bit`, och vad skulle en felaktigt matad stoppbit ställa till?
5. Varför räcker det inte att räkna mottagna bitar för att veta när en byte är klar?

Kodgranskningsseminariet

Ungefär en timme, i helgrupp. Varje grupp får tio minuter och visar en mergad Pull Request (vad den ändrade, hur den var uppdelad, och vad commit-meddelandena säger), en granskning gruppen skrivit (vad granskaren tittade efter, och vad som faktiskt ändrades av kommentarerna), och en sak som inte fungerat i arbetssättet, och vad ni gjort åt det.

Välj ut i gruppen en mergad Pull Request och en granskning någon i gruppen skrivit att visa. Det behöver inte vara den finaste; en PR som fick befogad kritik är mer värd att visa än en som ingen läste.

Seminariet är obligatoriskt och är den ena av projektets två fasta hållpunkter. Det betygsätts inte för sig, men det ingår i den helhetsbedömning av arbetssättet som beskrivs i bilaga C.

Riktvärde

Med `rx_shift_reg` klar finns alla fyra lövmoduler, och `can_controller` är den enda som återstår på CAN-sidan. Grupper som ligger efter bör lägga tiden här snarare än att börja på tillståndsmaskinen: den behöver alla fyra.

Blicken framåt

Kapitel 16 tar ramen som tillståndsmaskin, först förklarad och sedan byggd på sändarsidan.

15.8 Övningar

De här övningarna befäster kapitlet. Konstruera din egen `rx_shift_reg.vhd` utifrån specifikationen i avsnitt 15.1 till 15.4; den utdelade testbänken, körd enligt bilaga A, är kontrollen av att den betar sig korrekt.

Övning 15.1 `rx_shift_reg`

VHDL

Skriv din egen `rx_shift_reg.vhd` utifrån kapitlet. Verifiera den:

```
cd controller
ghdl -a --std=93 can_def.vhd rx_shift_reg.vhd rx_shift_reg_tb.vhd
ghdl -e --std=93 rx_shift_reg_tb
ghdl -r --std=93 rx_shift_reg_tb --assert-level=error
```

Om en kontroll fallerar, läs assertion-meddelandet; det namnger exakt vilket sampel det gäller och vad som förväntades.

Övning 15.2 Ett medvetet stoppbitsbrott

SIMULERING

- a) Konstruera (på papper) en 6-bitars sekvens på ledningen som utgör ett äkta brott mot stoppbitsregeln: fem lika riktiga bitar i rad, följda av en sjätte lika bit där en stoppbit skulle ha brutit följden.
- b) Mata in din sekvens från a) i `rx_shift_reg` (antingen genom att utöka `rx_shift_reg_tb.vhd` eller genom att skriva en kort ny testbänk) och bekräfta att `stuff_error` pulsar på exakt den sjätte biten, inte tidigare.
- c) `rx_shift_reg` försöker inte återhämta sig eller omsynkronisera efter ett `stuff_error`; den rapporterar det bara under en cykel och fortsätter att ackumulera som om ingenting hänt. Förklara, med en eller två meningar, varför det är en godtagbar design här, givet hur `can_controller` (kapitel 17) kommer att reagera på det.

Övning 15.3 Högerjusteringen, och det som ligger ovanför den

SIMULERING

Avsnitt 15.1 säger att en färdig grupp hamnar **högerjusterad i sina lägsta bit_count bitar**, och att can_controller förlitar sig på det när det skivar upp rekonstruerade fält. Test 3 och 4 i den utdelade testbänken prövar det: en 6-bitars grupp, sedan en 3-bitars grupp, utan reset emellan, med bit_count ändrat mellan grupperna på precis det sätt can_controller gör mellan ett fält och nästa.

Anta en nyss nollställd rx_shift_reg, enable = '1' och inga stoppbitar någonstans (ingen följd när någonsin fem). Processen i avsnitt 15.3 skiftar in varje accepterad bit i registrets minst signifikanta ände och nollställer aldrig registret mellan grupper; anta att din gör detsamma. Den utdelade testbänken kräver egentligen inte det, och del d) återkommer till varför.

a) Mata med bit_count = 6 in de sex bitarna 0 1 0 1 0 1 (i den ordningen, en per sample). Skriv ut alla åtta bitarna i data när valid pulsar. Vad står i data(7 downto 6), och varför?

b) Sätt bit_count = 3 utan att nollställa, och mata in 0 1 1. Skriv ut alla åtta bitarna i data när valid pulsar igen. Vad står nu i data(7 downto 3)?

c) Ditt svar på b) visar att bitarna ovanför bit_count håller rester från den *föregående* gruppen. Förklara varför det är ofarligt här, och vad can_controller måste göra (och aldrig får göra) när det läser rxsr_data för ett fält som är smalare än 8 bitar.

d) rx_shift_reg håller två ganska olika sorters tillstånd: skiftregistret som ackumulerar den aktuella gruppen, och följdräknaren (consecutive, last_bit) som avgör var stoppbitar hör hemma. Att nollställa dem är inte lika säkert i båda fallen.

- Anta att modulen nollställde sitt **skiftregister** vid varje valid. Vilket av dina svar ovan skulle ändras, och skulle något fält någonsin bli *felläst* som en följd? Notera att den utdelade testbänken bara kontrollerar de lägsta bit_count bitarna i data, så båda designerna passerar den.
- Anta att den nollställde **följdräknaren** i samma ögonblick. Namnge vad som går sönder, och säg vilken regel från kapitel 14 det speglar.

Skillnaden mellan de två svaren är skälet till att avsnitt 15.3.2 säger att följdräknaren nollställs bara av reset och inte säger någonting alls om skiftregistret.

CAN-kontrollern I: tillståndsmaskinen och sändvägen

Ramen som tillståndsmaskin, först förklarad och sedan byggd på sändarsidan.

I DET HÄR KAPITLET

- Den ramsekvenserande tillståndsmaskinen som begrepp: tillståndsdigrammet, fälttabellen och mönstret `STATE_LOAD_*/skifta`, innan någon VHDL skrivs.
- Att lägga till tillståndstypen och maskinens signaler i `can_controller`, och sedan skriva dess sändväg.
- Att driva den öppna dräneringsbussen från skiftregistrets aktuella bit.
- Att mata `crc15` korrekt: grinda på `bit_valid`, inte på `bit_done`.

`can_controller` är det enda block som känner till det faktiska CAN-ramformatet (kapitel 10). Den instansierar `bit_timer`, `crc15`, `tx_shift_reg` och `rx_shift_reg` internt och sekvenserar en hel ram fält för fält. Det här kapitlet bygger **sändvägen** (en nod som skickar en komplett ram); kapitel 17 lägger till mottagning och arbitrering.

Entiteten och dess femton portar skrevs i kapitel 11 och är oförändrade; se avsnitt 11.3 för vad var och en är till för. Ingenting i det här kapitlet eller i kapitel 17 ändrar dem: hela `can_controllers` sänd- och mottagarbeteende byggs *innanför* ett gränssnitt som legat fast sedan kapitel 11.

Två påminnelser som spelar roll för tillståndsmaskinen nedan:

- `can_controller_tb` binder **positionellt**, så portordningen som fastställdes i kapitel 11 är det som måste stämma; namnen är dina.
- `tx_bus` och `bus_en` hör ihop. Att driva bussen dominant betyder `bus_en = '1'` med `tx_bus = '0'`; att släppa den betyder `bus_en = '0'`. En nod driver aldrig recessivt aktivt.

Det här är projektets största enskilda kodinsats och den enda del som är svår att dela upp på flera personer. Ett rimligt upplägg är att två skriver och resten granskar tätt, snarare än att fyra skriver var sin del av samma process.

16.1 En nod, två roller

En nod antingen **sänder** sin egen begärda ram eller **tar emot** någon annans, aldrig båda för samma ram. En registrerad role ('1' = sänder, '0' = tar emot), låst en gång per ram på den flank som lämnar STATE_IDLE för STATE_START och hållen tills ramen är slut, avgör vilka av delblockens portar som spelar roll. Båda vägarna ut ur STATE_IDLE kräver **en buss som har legat ledig**: med det etablerat startar tx_req = '1' en sändning (role = '1'), och i annat fall startar ett dominant rx_bus_s2 (någon annans SOF) en mottagning (role = '0'). Följ role = '1'-vägen i det här kapitlet.

Varför "en buss som har legat ledig" gör verkligt arbete

Villkoret förtjänar sin plats två gånger om, av olika skäl vid vardera ingången.

Vid mottagaringången. En dominant bit i sig betyder inte att en ram börjar; de flesta bitarna i de flesta ramar är dominanta. STATE_IDLE måste skilja den dominanta bit som *inleder* en ram från de många dominanta bitarna inuti en, och en nod i STATE_IDLE har ingen ramkontext att skilja dem åt med.

Två frestande regler misslyckas båda:

- **En dominant nivå.** Slår till på vilken dominant bit som helst, så en nod som sitter i STATE_IDLE medan en annan nod sänder försöker omedelbart "ta emot" en ram som redan är halvvägs klar.
- **En flank från recessiv till dominant.** Låter som avsnitt 10.4:s beskrivning av SOF, men bitstoppning tvingar fram en övergång efter var femte lika bit, så flanker i båda riktningarna, den från recessiv till dominant inräknad, förekommer ständigt mitt inne i en ram. Flanktestet slår till inom några få bitar från nivåtestet.

Regeln som faktiskt fungerar är en **räkning av recessiva bitperioder**, och dess tröskel kräver ett steg mer omsorg än stoppbitsregeln ensam. Inuti det stoppade området garanterar avsnitt 10.5:s regel ingen följd längre än fem lika bitar. Men svansen är ostoppad och nästan helt recessiv: en kvitterad ram slutar med ACK-avgränsaren plus EOF, åtta recessiva bitperioder i rad, och en ram som ingen kvitterar visar tio (CRC-avgränsare, recessiv ACK-lucka, ACK-avgränsare, EOF). En tröskel på sex till tio skulle förklara bussen ledig medan en rams egen svans fortfarande passerade över den, och ett väntande tx_req skulle då driva ut SOF över någons sista EOF-bitar.

Räkna recessiva bitperioder i STATE_IDLE och kräv **elva**, samma antal som riktig CAN ger en nod som måste bedöma om bussen är ledig utan någon ramkontext: den kvitterade svansen på åtta plus standardens intermission på tre bitar, och förbi även den tio bitar långa

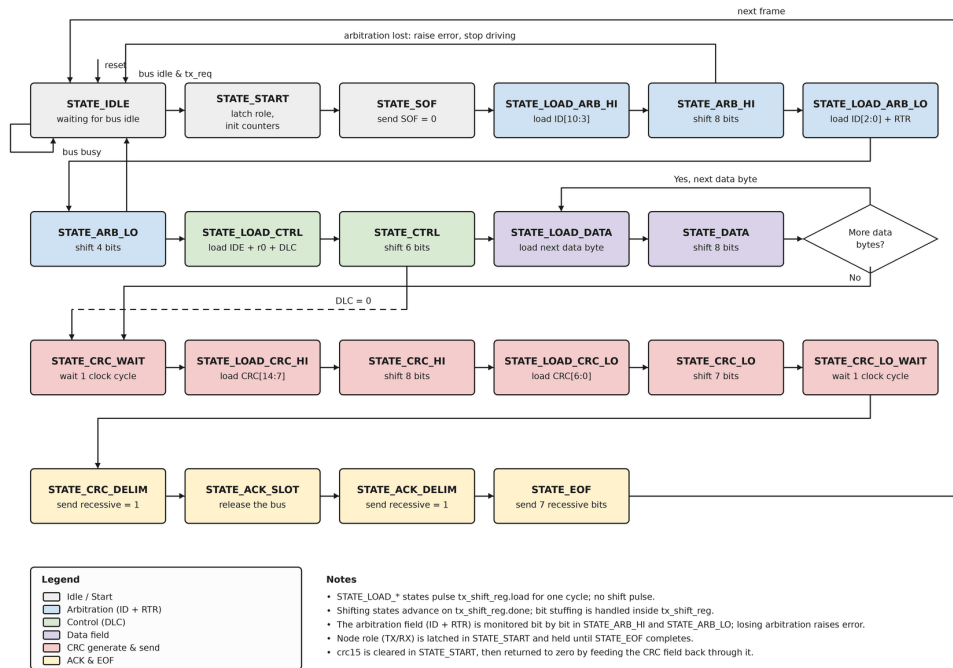
svansen hos en okvitterad ram. Först när räkningen är full betyder ett dominant `rx_bus_s2` SOF. Två följder för resten av designen:

- `bit_timer` måste fortsätta gå i `STATE_IDLE`, eftersom räkningen sker i bitperioder.
- Reset måste starta räkningen **full**. En nod som kommer ur reset har ingen anledning att tro att en ram pågår, och startade den tom skulle den strunta i den första ramen på bussen.

Vid sändaringången. Samma räkning svarar på en annan fråga: får den här noden börja driva bussen över huvud taget? Riktig CAN säger att en nod bara får börja sända på en ledig buss, och skälet är inte artighet. Arbitrering avgör en tvist enbart mellan noder som startade vid *samma* SOF, genom att jämföra vad var och en sände mot vad bussen läser, bit för bit, ned genom identifieraren. En nod som startar mitt inne i en ram deltar aldrig i den tävlingen; den driver bara dominanta bitar över någon annans fält. Ramen som pågår korrumpas, och skadan dyker upp i andra änden som ett stoppbitsbrott eller ett CRC-fel, utan att något pekar tillbaka på noden som orsakade den. Så `tx_req` väntar också på räkningen.

Tillsammans håller de här villkoren error observerbar. En nod som förlorar arbitreringen (kapitel 17) faller tillbaka till `STATE_IDLE` medan vinnaren fortfarande driver bussen, och `STATE_START` nollställer error. Under någon av de två misslyckade mottagarreglerna ovan skulle den gå in i `STATE_START` igen inom några få bitperioder och sudda den error den just höjt; med sändaringången ogrundad skulle en anropare som fortfarande håller `tx_req` hög sudda den redan nästa cykel. Att grinda båda betyder att förloraren väntar, korrekt, på nästa ram, och att en anropare som pollar error då och då fortfarande ser felet.

16.2 Ramen, fält för fält



Figur 16.1 can_controllers sändningstillståndsmaskin.

Varje **stoppat** fält går genom tx_shift_reg som en grupp, sänd MSB först. Varje fält får **två** tillstånd: ett STATE_LOAD_* på en cykel som pulsar tx_shift_reg.load, och sedan skifttillståndet som väntar på tx_shift_reg.done.

Tillstånd	Fält	Bitar	Sänt värde	Stoppat
STATE_START / STATE_SOF	SOF	1	dominant '0'	ja
STATE_LOAD_ARB_HI STATE_ARB_HI	/ ID hög	8	tx_id(10 downto 3)	ja
STATE_LOAD_ARB_LO STATE_ARB_LO	/ ID låg + RTR	4	tx_id(2 downto 0), sedan RTR='0'	ja
STATE_LOAD_CTRL STATE_CTRL	/ Kontroll	6	IDE='0', r0='0', sedan 4 bitars tx_dlc	ja
STATE_LOAD_DATA STATE_DATA	/ Databyte	8	aktuell byte, upprepad DLC gånger	ja
STATE_CRC_WAIT	(stabilisering)	0	låt crc15 stabilisera sig, läs sedan in den i crc_tx	nej
STATE_LOAD_CRC_HI STATE_CRC_HI	/ CRC hög	8	crc_tx(14 downto 7)	ja
STATE_LOAD_CRC_LO STATE_CRC_LO	/ CRC låg	7	crc_tx(6 downto 0)	ja
STATE_CRC_LO_WAIT	(stabilisering)	0	låt crc_valid stabilisera sig	nej
STATE_CRC_DELIM	CRC-avgränsare	1	recessiv	nej
STATE_ACK_SLOT	ACK-lucka	1	recessiv (mottagaren driver dominant)	nej
STATE_ACK_DELIM	ACK-avgränsare	1	recessiv	nej
STATE_EOF	EOF	7	recessiv	nej

Noteringar om skiftregistrets laddningar:

- tx_shift_reg.data är en byte_t, alltid 8 bitar, och sänder de översta bit_count av dem (kapitel 14). En grupp som är smalare än en byte är alltså **vänsterjusterad och nollutfylld** vid laddning, och utfyllnaden sänds aldrig. Tabellen ovan namnger varje grupp efter dess riktiga bitar; utfyllnaden är mekanisk.
- Kontrollbyten laddas som "00" & tx_dlc & "00" med bit_count = 6, så de sex sända bitarna är de sex översta: 0, 0, DLC[3:0].
- Den låga CRC-gruppen laddas som crc_tx(6 downto 0) & '0' med bit_count = 7.
- Den låga ID-gruppen laddas som tx_id(2 downto 0) & '0' & "0000" med bit_count = 4: de tre återstående bitarna av identifieraren, och sedan den alltid dominant RTR som avslutar arbitreringsfältet.
- Data väljer en byte ur tx_data per data_byte_idx; loopa STATE_LOAD_DATA/STATE_DATA en gång per byte, DLC gånger (hoppa direkt till STATE_CRC_WAIT när DLC = 0).

- `tx_dlc` över 8 ligger utanför kontraktet: avsnitt 10.4:s DLC kodar 0 till 8, och bytelooopen går utanför `data_byte_idx`s intervall för allt större. Riktiga kontrollrar begränsar kodningarna 9 till 15 till åtta byte; den här designen kräver helt enkelt att anroparen inte skickar en sådan.

De **ostoppade** avslutande fälten (CRC-avgränsare, ACK-lucka, ACK-avgränsare, EOF) går inte genom `tx_shift_reg`; de är fasta bitantal som drivs direkt, en bit per `bit_timer.bit_done`, identiskt för båda rollerna.

16.3 Vad du lägger till i arkitekturen

`can_controller.vhd` håller redan entiteten (kapitel 11), synkroniseraren `meta_prev` (kapitel 12) och alla fyra CAN-delblocksinstanserna med de signaler de kopplas till (kapitel 12 till 15). Den håller inget rambeteende alls. Allt nedan är nytt i det här kapitlet.

Tillståndstypen och tillståndsmaskinens egna signaler hör hemma i arkitekturens deklarativa del, ovanför de fyra delblockens signalgrupper:

```
-- One state per frame field. Every stuffed field is a LOAD/shift pair; the
-- trailer fields (CRC delimiter, ACK slot/delim, EOF) are driven directly.
type state_t is (
    STATE_IDLE,
    STATE_START,
    STATE_SOF,
    STATE_LOAD_ARB_HI, STATE_ARB_HI,
    STATE_LOAD_ARB_LO, STATE_ARB_LO,
    STATE_LOAD_CTRL,   STATE_CTRL,
    STATE_LOAD_DATA,   STATE_DATA,
    STATE_CRC_WAIT,
    STATE_LOAD_CRC_HI, STATE_CRC_HI,
    STATE_LOAD_CRC_LO, STATE_CRC_LO,
    STATE_CRC_LO_WAIT,
    STATE_CRC_DELIM,
    STATE_ACK_SLOT,
    STATE_ACK_DELIM,
    STATE_EOF);

signal state : state_t;

-- '1' while transmitting this frame, '0' while only receiving. Latched per
-- frame.
signal role : std_logic;
```

```

-- Consecutive recessive bit periods seen in STATE_IDLE, saturating at 11. ↪
↪ Only once this is
-- full may the node leave STATE_IDLE at all. Resets to 11, not 0.
signal idle_bits : natural range 0 to 11;

-- Data-byte index, and a countdown for the fixed-width trailer fields.
signal data_byte_idx  : natural range 0 to 8;
signal field_bit_count : natural range 0 to 7;

-- Accumulators for the frame being received.
signal rx_id_acc  : id_t;
signal rx_dlc_acc : dlc_t;
signal rx_data_acc : data_t;

-- This frame's own CRC, captured at STATE_CRC_WAIT before the engine is fed
-- the CRC field's own bits and moves on.
signal crc_tx : crc_t;

-- The data byte selected by data_byte_idx, MSB-first (byte 0 first).
signal tx_data_shifted : data_t;
signal tx_current_byte : byte_t;

```

De tre ackumulatorerna rx*_acc används inte förrän i kapitel 17; att deklarerat dem med resten håller ihop gruppen i stället för att dela den över två kapitel.

Två konkurrenta tilldelningar plockar ut den databyte som just nu sänds. De hör hemma efter begin, vid sidan av delblocksinstanserna:

```

tx_data_shifted <= std_logic_vector(shift_left(unsigned(tx_data), 8 * ↪
↪ data_byte_idx));
tx_current_byte <= tx_data_shifted(63 downto 56);

```

shift_left och unsigned kommer från numeric_std, så lägg till use ieee.numeric_std.all; i kontextklausulen om den inte redan finns där.

Allt annat i det här kapitlet är de två processer du skriver: en **kombinatorisk** som driver delblocken per tillstånd, och en **sekventiell** som för tillståndsmaskinen framåt och uppdaterar bokföringen.

Vilken process driver vad. Uppdelningen är värd att fastställa före genomgången, eftersom varje rad i den landar i en av de två processerna, och vilken det är ska inte vara någon gåta:

- Den **kombinatoriska** processen driver varje ingång på delblocken (bt_enable, bt_resync, de fyra txsr*-ingångarna, de två rxsr*-ingångarna i kapitel 17) och bussparet tx_bus/bus_en. Ge dem alla ett förval högst upp och skriv över per tillstånd, i samma stil som bit_timers egna grenar.

- `bt_enable` är '1' i **varje** tillstånd, `STATE_IDLE` inräknat, eftersom räkningen i `STATE_IDLE` mäts i bitperioder. Porten finns för att `bit_timer` ska kunna hållas, och den här designen vill aldrig det.
- `txsr_shift` är `bt_bit_done`, men bara under sändning och bara i **skifftillstånden**, aldrig i ett `STATE_LOAD_*`-tillstånd. Det är det som håller kapitel 14:s löfte om att "load och shift utesluter varandra": ett laddtillstånd utfärdar en laddning och inget annat.
- Den **sekventiella** processen äger state och bokföringen (`role`, `idle_bits`, `data_byte_idx`, `field_bit_count`, `crc_tx`, ackumulatorerna `rx_*_acc`) och varje utgångsport på registersidan: `tx_done`, `rx_valid`, `error` och `rx_*`-registren. Bussparet förblir kombinatoriskt, drivet per tillstånd enligt ovan. `tx_done` och `rx_valid` är pulser på en cykel, så ge dem förvalet '0' vid varje flank och höj dem där de hör hemma.
- `error` nollställs av `reset_s2_n` såväl som av `STATE_START`. Den är en nivå som en anropare pollar, så den måste läsa '0' ur `reset` i stället för vad nu en vipa råkade starta upp som.
- `tx_id`, `tx_dlc` och `tx_data` låses **inte** vid `STATE_START`; laddtillstånden läser dem live, vart och ett i det ögonblick dess grupp laddas. Det är en medveten förenkling och den lägger skyldigheten på anroparen: håll alla tre stabila från `tx_req` till `tx_done`. Ett registerblock i samma klockdomän gör det gratis, och det är precis den anropare avsnitt 11.3 förutsätter.

16.4 Den kombinatoriska processen

Den driver delblockens ingångar och bussparet utifrån state, med förvalen först och överskrivningarna per tillstånd efter, och den är kompakt nog att visas hel. Tre saker att se i den innan koden. Varje `STATE_LOAD_*`-arm laddar sin grupp och delar arm med det parade skifftillståndet, eftersom paret också delar ett `bit_count`. `txsr_shift` är `bt_bit_done` bara i skifftillstånden, aldrig i ett `STATE_LOAD_*`-tillstånd: load presenterar redan den nya gruppens första bit under laddcykeln (kapitel 14), så ett dedikerat laddtillstånd på en cykel är det som håller det löftet; att få det här om bakfoten är ramnivåns version av det enbitsfel i tajmingen som kapitel 14 varnade för. Och bussen har öppen dränering: en nod driver bara *aktivt* när biten är dominant, i annat fall släpper den, så att en annan nod kan dra samma ledning dominant under samma period.

```
COMBINATIONAL_PROCESS: process(state, role, bt_bit_done, tx_id, tx_dlc,
                                tx_current_byte, crc_tx, txsr_tx_bit) is
begin
    -- Defaults: every sub-block input, every cycle, then per-state overrides.
    bt_enable      <= '1';                -- Never held; the idle count needs it.
    bt_resync      <= '0';
    txsr_load      <= '0';
    txsr_data      <= (others => '0');
    txsr_bit_count <= "0000";
    txsr_shift     <= '0';
```

```

rxsr_enable    <= '0';           -- The receive side arrives in L17.
rxsr_bit_count <= "0000";
tx_bus         <= '1';           -- Release the bus unless driving ↵
↳ dominant.
bus_en         <= '0';

-- Each STATE_LOAD_* arm loads its chunk. The paired shifting state keeps
-- the same bit_count applied, so grouping the pair in one arm is natural.
case state is
  when STATE_START =>
    bt_resync <= '1';           -- Adopt SOF as this node's bit boundary.
    ↵ ,

    if (role = '1') then
      txsr_load    <= '1';
      txsr_data    <= "00000000";           -- SOF: one ↵
↳ dominant bit.
      txsr_bit_count <= "0001";
    end if;
  when STATE_LOAD_ARB_HI | STATE_ARB_HI =>
    if ((role = '1') and (state = STATE_LOAD_ARB_HI)) then
      txsr_load <= '1';
      txsr_data <= tx_id(10 downto 3);
    end if;
    txsr_bit_count <= "1000";
  when STATE_LOAD_ARB_LO | STATE_ARB_LO =>
    if ((role = '1') and (state = STATE_LOAD_ARB_LO)) then
      txsr_load <= '1';
      txsr_data <= tx_id(2 downto 0) & '0' & "0000"; -- ID low, ↵
↳ then RTR.
    end if;
    txsr_bit_count <= "0100";
  when STATE_LOAD_CTRL | STATE_CTRL =>
    if ((role = '1') and (state = STATE_LOAD_CTRL)) then
      txsr_load <= '1';
      txsr_data <= "00" & tx_dlc & "00";           -- IDE, r0, ↵
↳ DLC.
    end if;
    txsr_bit_count <= "0110";
  when STATE_LOAD_DATA | STATE_DATA =>
    if ((role = '1') and (state = STATE_LOAD_DATA)) then
      txsr_load <= '1';
      txsr_data <= tx_current_byte;
    end if;
    txsr_bit_count <= "1000";

```

```

when STATE_LOAD_CRC_HI | STATE_CRC_HI =>
    if ((role = '1') and (state = STATE_LOAD_CRC_HI)) then
        txsr_load <= '1';
        txsr_data <= crc_tx(14 downto 7);
    end if;
    txsr_bit_count <= "1000";
when STATE_LOAD_CRC_LO | STATE_CRC_LO =>
    if ((role = '1') and (state = STATE_LOAD_CRC_LO)) then
        txsr_load <= '1';
        txsr_data <= crc_tx(6 downto 0) & '0';
    end if;
    txsr_bit_count <= "0111";
when others =>
    null;
end case;

-- Shifting states advance the register once per bit period; a LOAD
-- state never does, which keeps load and shift mutually exclusive.
if (role = '1') then
    case state is
        when STATE_SOF | STATE_ARB_HI | STATE_ARB_LO | STATE_CTRL
             | STATE_DATA | STATE_CRC_HI | STATE_CRC_LO =>
            txsr_shift <= bt_bit_done;
        when others =>
            null;
    end case;
end if;

-- Driving the bus, open-drain: only the owning states, only dominant
-- bits. A receiver's one dominant bit is the ACK slot (L17).
case state is
    when STATE_SOF
        | STATE_LOAD_ARB_HI | STATE_ARB_HI
        | STATE_LOAD_ARB_LO | STATE_ARB_LO
        | STATE_LOAD_CTRL   | STATE_CTRL
        | STATE_LOAD_DATA   | STATE_DATA
        | STATE_LOAD_CRC_HI | STATE_CRC_HI
        | STATE_LOAD_CRC_LO | STATE_CRC_LO =>
        if ((role = '1') and (txsr_tx_bit = '0')) then
            bus_en <= '1';
            tx_bus <= '0';
        end if;
    when others =>
        null;
end case;

```

```

    end case;
end process;

```

Avgränsningen av bussdrivningen i den sista case-satsen förtjänar sin egen mening: txsr_tx_bit är en nivå och håller sitt senaste värde i all evighet, så en oavgränsad version fortsätter driva efter den grupp som producerade den. En ram vars CRC slutar på en dominant bit skulle fortfarande dra ned bussen genom hela svansen, och en nod som sitter i STATE_IDLE med role fortfarande '1' från sin förra ram skulle hålla hela bussen dominant för alltid. Svansens fält är recessiva för båda rollerna och driver ingenting, med det enda undantaget en mottagares ACK-lucka (kapitel 17).

16.5 Den sekventiella processen

Den äger state, bokföringen och utgångsportarna. Skelett, reset och förval först:

```

SEQUENTIAL_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        state          <= STATE_IDLE;
        role           <= '0';
        idle_bits      <= 11;          -- Full: an idle bus is assumed at power-
        -- up.
        data_byte_idx  <= 0;
        field_bit_count <= 0;
        rx_id_acc       <= (others => '0');
        rx_dlc_acc      <= (others => '0');
        rx_data_acc     <= (others => '0');
        crc_tx          <= (others => '0');
        tx_done         <= '0';
        rx_valid        <= '0';
        error           <= '0';
        rx_id           <= (others => '0');
        rx_dlc          <= (others => '0');
        rx_data         <= (others => '0');
    elsif (rising_edge(clock)) then
        tx_done <= '0';          -- The two pulse outputs default low;
        rx_valid <= '0';         -- error is a level and holds.

        case state is

```

Allt nedan är armar i den här enda case-satsen. Först de två tillstånd varje ram passerar en gång:

```

when STATE_IDLE =>
  if (bt_sample = '1') then
    if (rx_bus_s2 = '1') then
      if (idle_bits /= 11) then
        idle_bits <= idle_bits + 1;
      end if;
    else
      idle_bits <= 0;
    end if;
  end if;
  if (idle_bits = 11) then
    if (tx_req = '1') then
      role      <= '1';
      idle_bits <= 0;
      state     <= STATE_START;
    elsif (rx_bus_s2 = '0') then
      role      <= '0';    -- Someone else's SOF: receive (→
-- L17).

      idle_bits <= 0;
      state     <= STATE_START;
    end if;
  end if;
when STATE_START =>
  error      <= '0';
  data_byte_idx <= 0;
  rx_data_acc <= (others => '0');
  state      <= STATE_SOF;

```

STATE_IDLE räknar recessiva bitperioder vid sampelpunkten och mättar vid 11; ett dominant sampel nollställer räkningen. Ingenting lämnar tillståndet förrän räkningen är **full**, vad tx_req och rx_bus_s2 än gör, så en tx_req-puls som anländer medan bussen är upptagen **förkastas**, den köas inte; anroparen begär om. STATE_START behöver ingen kod för SOF-laddningen eller resynken, eftersom den kombinatoriska processen utfärdar båda utifrån tillståndet självt; uppsättning som måste *minnas*, att nollställa error, byteindexet och ackumulatort för mottagen data, är det som landar här. rx_data_acc behöver nollställningen eftersom en mottagen ram bara skriver de byte dess DLC täcker (kapitel 17): en fembytesram som anländer efter en åttabytesram måste rapportera nollor i byte 5 till 7, inte den förra ramens rester. rx_id_acc och rx_dlc_acc skrivs alltid i sin helhet, så de behöver ingen. (crc15 nollställs också av det här tillståndet; se avsnitt 16.8.)

Sedan LOAD/skift-paren. Det här är formen varje stoppat fält följer, visad en gång för identifierarens höga grupp; sändarhalvan av ett LOAD-tillstånd går vidare efter sin enda cykel, och skifttillståndet väntar på tx_shift_reg.done.

En tajmingnotering innan vågformerna överraskar dig: kapitel 14 beskrev load som att den utfärdas i samma ögonblick som den förra gruppens sista bit_done, och här utfärdas den två klockflanker senare, en för att tillståndsmaskinen ska hinna se txsr_done och en för själva LOAD-tillståndet. Varje grupps första bit når därför ledningen omkring 40 ns in i sin bitperiod på 1000 ns; sampelpunkten vid 70 % absorberar det med marginal, och förskjutningen ackumuleras inte, eftersom varje grupp startar om från samma bit_done.

```

when STATE_SOF =>
  if (role = '1') then
    if (txsr_done = '1') then
      state <= STATE_LOAD_ARB_HI;
    end if;
  end if;                                     -- role = '0': L17.
when STATE_LOAD_ARB_HI =>
  if (role = '1') then
    state <= STATE_ARB_HI;                   -- One cycle; the load is issued.
  end if;                                     -- role = '0': L17.
when STATE_ARB_HI =>
  if (role = '1') then
    if (txsr_done = '1') then -- Arbitration watching joins
      state <= STATE_LOAD_ARB_LO;
    end if;
  end if;                                     -- role = '0': L17.
when STATE_LOAD_ARB_LO =>
  if (role = '1') then
    state <= STATE_ARB_LO;
  end if;                                     -- role = '0': L17.
when STATE_ARB_LO =>
  if (role = '1') then
    if (txsr_done = '1') then -- Also watched in L17; RTR is
      state <= STATE_LOAD_CTRL; -- so it can never be the
    end if;
  end if;                                     -- role = '0': L17.

```

Kontroll- och datatillstånden har samma form med två beslut påhängda: kontrollfältets done konsulterar DLC:n, och dataparet loopar en gång per byte:

```

when STATE_LOAD_CTRL =>
  if (role = '1') then
    state <= STATE_CTRL;
  end if;
  -- role = '0': L17.
when STATE_CTRL =>
  if (role = '1') then
    if (txsr_done = '1') then
      if (unsigned(tx_dlc) = 0) then
        state <= STATE_CRC_WAIT;    -- No data field at ↵
      all.

      else
        state <= STATE_LOAD_DATA;
      end if;
    end if;
  end if;
  -- role = '0': L17.
when STATE_LOAD_DATA =>
  if (role = '1') then
    state <= STATE_DATA;
  end if;
  -- role = '0': L17.
when STATE_DATA =>
  if (role = '1') then
    if (txsr_done = '1') then
      if (data_byte_idx + 1 = to_integer(unsigned(tx_dlc))) ↵
    then

      state <= STATE_CRC_WAIT;
    else
      data_byte_idx <= data_byte_idx + 1;
      state <= STATE_LOAD_DATA;
    end if;
  end if;
end if;
-- role = '0': L17.

```

Sedan CRC:n. STATE_CRC_WAIT är en tom cykel så att den sista databiten hinner integreras klart innan `crc_value` läses, och dess verkliga uppgift är låsningen: båda CRC-grupperna laddas från `crc_tx`, aldrig från den levande motorn (avsnitt 16.6 förklarar vad som går fel annars). De två CRC-paren har arbitreringsparens form igen:

```

when STATE_CRC_WAIT =>
  crc_tx <= crc_value;    -- Capture before the engine ↵
  moves on.

  state <= STATE_LOAD_CRC_HI;
when STATE_LOAD_CRC_HI =>
  if (role = '1') then

```

```

        state <= STATE_CRC_HI;
    end if; -- role = '0': L17.
when STATE_CRC_HI =>
    if (role = '1') then
        if (txsr_done = '1') then
            state <= STATE_LOAD_CRC_LO;
        end if;
    end if; -- role = '0': L17.
when STATE_LOAD_CRC_LO =>
    if (role = '1') then
        state <= STATE_CRC_LO;
    end if; -- role = '0': L17.
when STATE_CRC_LO =>
    if (role = '1') then
        if (txsr_done = '1') then
            state <= STATE_CRC_LO_WAIT;
        end if;
    end if; -- role = '0': L17.

```

Till sist svansen, där skiftregistret bugar och går och tillståndsmaskinen själv räknar `bt_bit_done`-pulser. `STATE_CRC_LO_WAIT` är en klockcykel för en sändare, **inte** en bitperiod: `tx_shift_reg.done` pulserar vid den avslutande skiftningen, som själv är ett `bit_done`, så sändaren står redan på en bitgräns och svansen kan börja omedelbart; att vänta en hel period skulle trycka ut en extra recessiv bit på ledningen. (En mottagare väntar *däremot* ett helt `bit_done` här, och kontrollerar `crc_valid` medan den väntar; kapitel 17.)

```

    when STATE_CRC_LO_WAIT =>
        if (role = '1') then
            state <= STATE_CRC_DELIM; -- One clock cycle, not a bit ↵
        end if; -- role = '0': L17 checks ↵
    ↵ period.
    ↵ crc_valid here.
    when STATE_CRC_DELIM =>
        if (bt_bit_done = '1') then
            state <= STATE_ACK_SLOT;
        end if;
    when STATE_ACK_SLOT =>
        if (bt_bit_done = '1') then -- Transmitter releases; a ↵
    ↵ receiver
            state <= STATE_ACK_DELIM; -- acknowledges here (L17).
        end if;
    when STATE_ACK_DELIM =>
        if (bt_bit_done = '1') then

```

```

        field_bit_count <= 6;      -- Arm the 7-bit EOF countdown.
        state <= STATE_EOF;
    end if;
    when STATE_EOF =>
        if (bt_bit_done = '1') then
            if (field_bit_count = 0) then
                if (role = '1') then
                    tx_done <= '1';
                end if;
                -- role = '0': L17 latches rx_*
            here.
            state <= STATE_IDLE;
        else
            field_bit_count <= field_bit_count - 1;
        end if;
    end if;
end case;
end if;
end process;

```

Svansens tillstånd behöver inget *role*-test för att gå vidare, eftersom båda rollerna räknar samma hela bitperioder genom samma fält; bara det som händer *inuti* dem skiljer sig, och de skillnaderna är alla kapitel 17:s.

16.6 Att fånga CRC:n

STATE_CRC_WAIT läser *crc_value* i *crc_tx*, och båda CRC-grupperna laddas från *crc_tx* snarare än från motorn. Den omvägen ser överflödigt ut och är det inte, av ett skäl värt att följa noga, eftersom det är det enda stället i den här designen där två regler som var för sig är korrekta tillsammans bildar en bugg.

Regel ett: motorn fortsätter gå genom CRC-fältet, eftersom det är det som återför den till noll (avsnitt 16.7). Regel två: varje grupp laddas när dess STATE_LOAD_*-tillstånd körs.

Sätt nu ihop dem. STATE_LOAD_CRC_HI körs först och läser rätt värde. Sedan skiftar STATE_CRC_HI ut åtta bitar, och var och en av dem går tillbaka in i motorn, som pliktskyldigt för registret vidare. När STATE_LOAD_CRC_LO väl körs är *crc_value* **inte längre ramens CRC**: det är registret åtta bitar längre fram, vilket för en korrekt ram betyder de återstående sju CRC-bitarna uppskiftade med nollor inskiftade under dem. Ladda *crc_value*(6 downto 0) där, och ramens sista sju CRC-bitar går ut som sju nollor, i varje ram, alltid.

Felet är tyst på värsta tänkbara sätt. Sändaren är fullkomligt nöjd; ingenting den kan se är fel. Mottagarens motor når inte noll, så den rapporterar ett CRC-fel, och det uppenbara stället att leta på är mottagarvägen, vilket är helt i sin ordning. Att låsa värdet en gång, i det ögonblick det är färdigt och innan något annat rör motorn, tar bort samspelet helt.

16.7 Att mata crc15

crc15 måste se varje riktig bit från SOF till och med slutet av **CRC-fältet**, och inga andra. Grinda dess enable på tx_shift_reg.bit_valid ('0' vid varje grupps avslutande skiftning, så att den oförändrade biten inte matas in två gånger), och stäng ute stoppbitarna och mottagarvägen:

```

crc_enable <= txsr_bit_valid and not txsr_stuff and role;
crc_data_in <= txsr_tx_bit;

```

Tre saker följer av att det här är en enkel konkurrent tilldelning utan någon tillståndsterm i sig.

CRC-fältet matas tillbaka in i motorn, och det är avsiktligt. Värdet som sänds ligger tryggt i crc_tx, så ingenting behöver läsa motorn igen när STATE_CRC_WAIT väl fångat det, och uttrycket har ingen tillståndsterm som skulle stoppa matningen vid datafältet. På sändarsidan är återmatningen bara harmlös: kapitel 13:s egenskap generera-och-sedan-kontrollera betyder att ett meddelande följt av sin egen CRC återför registret till noll, och STATE_STARTs nollställning (avsnitt 16.8) lämnar hur som helst över en fräsch motor till nästa ram. Där återmatningen förtjänar sin plats är på **mottagarsidan**: kapitel 17:s CRC-kontroll är observationen att registret återvänt till noll när det mottagna CRC-fältet gått in, och den observationen finns bara därför att CRC-fältet matas tillbaka. En regel för båda rollerna, varje riktig bit från SOF till och med slutet av CRC-fältet, är det som låter en enda motor generera på en nod medan den kontrollerar på en annan.

Svensen utesluter sig själv. CRC-avgränsare, ACK-lucka, ACK-avgränsare och EOF går aldrig genom tx_shift_reg alls, så txsr_bit_valid är redan låg över allihop. Ingen tillståndsterm behövs för att hålla dem utanför checksumman.

role är plathållaren för mottagarsidan. Med role = '0' håller det här uttrycket crc15 avstängd, vilket är korrekt för det här kapitlet (bara sändning) men inte för den färdiga kontrollern. Kapitel 17 ersätter hela tilldelningen med en som matar rx_shift_reg.real_bit på rx_shift_reg.real_bit_valid under mottagning; sändarhalvan ovan är oförändrad av det.

16.8 Att nollställa crc15

Ännu en konkurrent tilldelning, och det är den som gör motorn säker att återanvända ram efter ram:

```

crc_clear <= '1' when state = STATE_START else '0';

```

På den lyckliga vägen gör den ingenting alls, eftersom en ram som körs hela vägen redan har återfört registret till noll. Den finns där för de ramar som *inte* blir klara. Kapitel 17 ger den här noden tre sätt att överge en ram halvvägs (den förlorar arbitringen, den ser

ett stoppbitsbrott, dess CRC-kontroll misslyckas), och vart och ett lämnar motorn med ett halvfärdigt värde och ingen utsikt att någonsin nå noll av sig själv. Varje ram noden sänder därefter skulle bära en CRC beräknad ur den resten, och varje ram den tog emot skulle underkännas i kontrollen, tills någon tryckte på reset.

Två saker är värda att lägga märke till. `STATE_START` är rätt hem för den eftersom det redan är tillståndet "sätt upp den här ramen", vid sidan av att nollställa error och byteindexet; nollställningen hör hemma där snarare än bland CRC-tillstånden. Och `can_controller_tb` kan inte fånga en utebliven nollställning, eftersom ingen av dess två scenarier låter en nod sända igen efter ett avbrott. Det gör det här till en regel att få rätt genom resonemang snarare än genom att vänta på en röd testbänk, samma lärdom som `crc_tx` gav i avsnitt 16.6: de tysta felen är de man ska resonera sig fram till i förväg.

16.9 Sammanfattning

Det här ska du kunna nu

- Rita CAN-ramens tillståndsdigram och säga vilket fält varje tillstånd svarar mot.
- Förklara varför varje fält behöver ett `LOAD`-tillstånd före sitt skifttillstånd.
- Implementera sändvägen: ladda skiftregistret, skifta ut, och gå vidare vid rätt puls.
- Driva bussen med öppen dränering korrekt: `bus_en` styr, `tx_bus` bestämmer vad.
- Grinda CRC-motorn rätt, och säga varför fel grindning ger en CRC som ser rimlig ut och är fel.

Frågor att testa dig själv med

1. Varför behöver varje fält ett eget `LOAD`-tillstånd?
2. En nod som sänder släpper bussen under `ACK`-luckan. Varför, och vad förväntar den sig se där?
3. `bus_en` och `tx_bus` är två separata signaler. Vad hade gått förlorat med bara en?
4. Varför grindas `crc15` på `bit_valid` and `not stuff` och inte på `bit_done`?
5. Vilka fält täcks av `CRC:n`, och var slutar täckningen exakt?
6. Varför är tillståndsmaskinen en Mooremaskin här?

Riktvärde

Sändvägen bör vara på plats när kapitel 17 börjar, eftersom mottagarvägen byggs in i samma maskin. Kommer du inte hela vägen: gör klart sändningen fram till och med CRC-fältet, och lämna `ACK` och `EOF` till nästa pass.

Blicken framåt

Kapitel 17 färdigställer `can_controller` med mottagning och arbitrering, och kör systemtestbänken hela vägen.

16.10 Övningar

Du bygger sändningsvägen, och `can_controller_tb` driver två noder genom ett komplett utbyte, så den kan inte passera förrän kapitel 17 lägger till mottagningshalvan. `make build-project` vet det och rapporterar den som *överhoppad* i stället för att köra den och misslyckas:

```
==> can_controller_tb (skipped: can_controller.vhd has no receive path yet; ↵
    ↵ ...)
```

Så bygget förblir grönt det här kapitlet, och de här övningarna resonerar sig igenom sändningsvägen och analyserar den i GHDL i stället. Testbänken blir domen om ett kapitel.

Övning 16.1 Följ en kort ram för hand

RESONEMANG

Skriv, utifrån ramformatet i avsnitt 10.4 och det här kapitlets fälttabell, ut hela följden av tillstånd som `can_controller` passerar (i TX-rollen) för att sända en ram med ID `0x001`, DLC 0 (inget datafält), från `STATE_IDLE` tillbaka till `STATE_IDLE`. Notera för varje tillstånd ungefär hur många bitperioder det upptar.

Övning 16.2 Varför tillståndet `STATE_LOAD_*` finns

RESONEMANG

- Förklara med egna ord vad som skulle gå fel om ett stoppat fält sekvenserades med ett enda tillstånd som gav `tx_shift_reg:s load` och väntade på `done`, i stället för ett separat `STATE_LOAD_*` på en cykel följt av skifttillståndet.
- Peka ut den specifika tajmingregeln från kapitel 14 (att `tx_shift_reg:s load` presenterar direkt, och dess `done` som kommer ett skift extra) som gör uppdelningen i två tillstånd nödvändig.

Övning 16.3 Vad som når `crc15`, och vad som inte gör det

RESONEMANG

- a) Sortera för en sänd ram de här kategorierna av bitar i "matas in i `crc15`" och "utesluts": SOF-biten, riktiga bitar ur ID, kontrollfält och data, instoppade stoppbitar, CRC-fältets egna bitar, och den ostoptrade svansen (CRC-avgränsare, ACK-plats, ACK-avgränsare, EOF). Säg för varje utesluten kategori vad som utesluter den: antingen en term i `crc_enable <= txsr_bit_valid and not txsr_stuff and role;`, eller det faktum att fältet aldrig passerar genom `tx_shift_reg` över huvud taget.
- b) En av de kategorierna kan överraska dig: CRC-fältets egna bitar går tillbaka in i motorn. Säg, med kapitel 13:s egenskap att generering och kontroll är samma operation, vilket värde registret håller efter att den sista CRC-biten matats in, och namnge den kontroll i kapitel 17 som beror på precis det värdet. Förklara sedan varför sändningsvägen, tagen för sig, inte beror på det: vad garanterar `STATE_START:s clear` redan om den andra ram en nod sänder?
- c) Anta att du i stället hade grindat bort `crc_enable` under `STATE_CRC_HI` och `STATE_CRC_LO`. Skulle den första ram en nod sänder bära en korrekt CRC? Skulle den andra? Var noga: att `crc15` nollställs är en del av svaret. Säg sedan om någon simulering i den här boken skulle fånga ändringen, och vad det säger dig om var det återmatade CRC-fältet faktiskt gör nytta.

Övning 16.4 Hur lång är en ram, exakt?

RESONEMANG

Att svara på det kräver att hela fälttabellen och stoppbitsregeln hålls i huvudet samtidigt, vilket är själva poängen. Ta ramen `can_controller` sänder för ID `0x123`, DLC 3, databytesen `FF 00 00`.

- a) Skriv ut det stoppade området bit för bit, SOF till och med datafältets slut, med det här kapitlets fälttabell. (Det ska bli 43 bitar: $1 + 11 + 1 + 2 + 4 + 24$.) Identifieraren är `0x123`; kom ihåg att RTR, IDE och r0 alla är 0 och att DLC är `0011`.
- b) CRC-15 över de 43 bitarna är `0x0FEE`, alltså `000111111101110` (du kan bekräfta det mot din `crc15` från kapitel 13 om du vill). Lägg till den, vilket ger hela det stoppade området på 58 bitar, SOF till och med CRC.
- c) Tillämpa stoppbitsregeln över alla 58 bitarna och räkna de instoppade stoppbitarna. Håll koll på gränsen mellan datafältets avslutande nollor och CRC:ns inledande nollor: följdräknaren nollställs inte vid fältgränser.
- d) Lägg till den ostoptrade svansen (CRC-avgränsare, ACK-plats, ACK-avgränsare, EOF) och ange det totala antalet bitar som faktiskt drivs ut på ledningen. Hur många mikrosekunder är det vid 1 Mbit/s?
- e) Samma ram med datan `11 22 33` behöver bara en stoppbit i stället för antalet du fick i c). Förklara med en mening varför en rams varaktighet på en CAN-buss beror på *nyttolastens*

värde och inte bara på dess längd, och vad det innebär för den som försöker räkna ut värsta fallets bustajning.

CAN-kontrollern II: mottagning och arbitrering

Att färdigställa can_controller. Efter det här kapitlet är CAN-halvan av projektet klar.

I DET HÄR KAPITLET

- Mottagarrollen: tajmingen för rx_shift_regs enable, och den enda cykelns föregripande.
- Att sätta ihop en mottagen ram igen, och att mata crc15 på mottagarsidan.
- Ett rent bord mellan ramar, och varför båda skiftregistren hålls i reset medan bussen är ledig.
- Detektering av förlorad arbitrering, kontrollerad bit för bit över arbitreringsfältet.
- Att köra can_controller_tb hela vägen, sändning och mottagning tillsammans.

Kapitel 16 byggde sändvägen. Det här kapitlet färdigställer can_controller med role = '0'-vägen (mottagning) och den arbitreringslogik som avgör vilken av två tävlande sändare som får fortsätta. Ramuppställningen, tillståndssekvensen och gruppstorlekarna per fält är desamma som i kapitel 16; mottagarsidan läser bara tillbaka varje fält i stället för att driva det.

17.1 Att läsa tillbaka bussen

En mottagande nod samlar in riktiga (avstoppade) bitar genom rx_shift_reg, grindade vid bit_timer.sample-punkten (mitt i perioden, när bussen hunnit stabilisera sig), inte vid bit_done. Den sätter rx_shift_reg.bit_count till samma gruppstorlekar som sändarsidan använder, och läser varje färdig grupp ur rx_shift_reg.data, som är **högerjusterad** i sina lägsta bit_count bitar (kapitel 15). Den sätter ihop ramen igen i ackumulatorer:

- **ID**: de höga 8 bitarna från ARB_HI (rxsr_data(7 downto 0)), de låga 3 från ARB_L0 (rxsr_data(3 downto 1)). Gruppens sista bit, rxsr_data(0), är RTR: dominant markerar de dataramar den här boken stöder, så en recessiv RTR höjer error och avbryter ramen (en fjärram, som inte stöds).

- **DLC:** från CTRL (`rxsr_data(3 downto 0)`). Gruppens två översta bitar, IDE och r0, kontrolleras **inte**: båda är alltid dominanta i den här bokens ramar. En mottagare som mötte den recessiva IDE:n i en riktig ram med utökad identifierare skulle feltolka den som en standardram ända tills CRC-kontrollen misslyckades; att kontrollera IDE och hantera det den annonserar hör hemma i en produktionskontroller (avsnitt 19.7 listar det bland luckorna).
- **Data:** varje mottagen byte placeras på sin **fasta** position efter byteindex, byte 0 i bitarna 63 downto 56, byte 1 i 55 downto 48, och så vidare. Det här är en positionsstyrd case, **inte** en löpande vänsterskiftning (som bara skulle landa byte 0 högst upp för en full 8-bytesram).

I slutet av STATE_EOF, läs `rx_id`, `rx_dlc` och `rx_data` från ackumulatorerna och pulsa `rx_valid`.

17.2 Mottagarvägen, tillstånd för tillstånd

Samma tillståndssekvens som kapitel 16:s sändväg, gången som mottagare. `role = '0'` hela vägen. Läs det här vid sidan av avsnitt 16.2:s fälttabell: gruppstorlekarna är identiska, bara det som händer med varje grupp skiljer sig.

”Föregriper” är det `rxsr_bit_count/rxsr_enable` som det **kombinatoriska** blocket driver medan state fortfarande läser STATE_LOAD_*-värdet, en cykel innan skifttillståndet blir synligt; avsnitt 17.5 förklarar varför det är nödvändigt.

Tillstånd	Föregriper	Går vidare på	Akkumulator / åtgärd
STATE_IDLE	inget	dominant rx_bus_s2, idle_bits full	lås role = '0', nollställ idle_bits
STATE_START	inget	omedelbart	nollställ error, återställ byteindexet, bt_resync
STATE_SOF	1	rxsr_valid	samla in SOF som en enbitsgrupp; själva biten kastas
STATE_LOAD_ARB_HI	8	bt_sample	(väntan: förra gruppens sista bit ligger fortfarande på ledningen)
STATE_ARB_HI	8	rxsr_valid	rx_id_acc(10 downto 3) <= rxsr_data(7 downto 0)
STATE_LOAD_ARB_LO	4	bt_sample	
STATE_ARB_LO	4	rxsr_valid	rx_id_acc(2 downto 0) <= rxsr_data(3 downto 1); rxsr_data(0) är RTR: om recessiv, error <= '1' och avbryt
STATE_LOAD_CTRL	6	bt_sample	
STATE_CTRL	6	rxsr_valid	rx_dlc_acc <= rxsr_data(3 downto 0); DLC = 0 hoppar till STATE_CRC_WAIT
STATE_LOAD_DATA	8	bt_sample	
STATE_DATA	8	rxsr_valid	placera rxsr_data på den fasta positionen för byte data_byte_idx
STATE_CRC_WAIT	inget	omedelbart	en stabiliseringscykel; ingen väntan på bt_sample
STATE_LOAD_CRC_HI	8	bt_sample	
STATE_CRC_HI	8	rxsr_valid	fortsätt mata crc15; själva värdet behålls inte
STATE_LOAD_CRC_LO	7	bt_sample	
STATE_CRC_LO	7	rxsr_valid	sista bitarna in i crc15
STATE_CRC_LO_WAIT	inget	bt_bit_done	kontrollera crc_valid: om låg, error <= '1' och avbryt
STATE_CRC_DELIM	inget	bt_bit_done	
STATE_ACK_SLOT	inget	bt_bit_done	driv bussen dominant för att kvittera
STATE_ACK_DELIM	inget	bt_bit_done	armera 7-bitars EOF-nedräkningen
STATE_EOF	inget	bt_bit_done x7	vid den sista biten, läs rx_id, rx_dlc, rx_data, pulsa rx_valid

Två mönster löper genom hela tabellen. Varje STATE_LOAD_*-rad finns enbart för att förbruka den förra gruppens avslutande bitperiod: tillståndsmaskinen skördar ingenting där, även om det bt_sample som avslutar tillståndet är precis det som rx_shift_reg samlar in nästa grupps första bit på. Varje avslutande (ostoppad) rad går vidare på bt_bit_done i stället för bt_sample, eftersom de fälten går förbi rx_shift_reg helt och räknas som hela bitperioder.

Varför STATE_SOF aktiverar rx_shift_reg för en bit som ingen behåller

Det insamlade värdet kastas; SOF är alltid dominant och STATE_IDLE visste redan det när den startade den här ramen. Det som spelar roll är att biten passerar genom avstoppningens följdspårare. Den spåraren (consecutive, last_bit) lever kvar över gruppgränser och nollställs bara av reset (kapitel 15), och sändarens kopia inuti tx_shift_reg räknade SOF, eftersom kapitel 16 skickar den genom tx_shift_reg som vilket annat stoppat fält som helst. Hoppa över den här, och mottagaren kör hela ramen en dominant bit efter sändaren.

Det är inte något litet fönster. Varje identifierare vars fyra översta bitar är noll, alltså **varje ID under 0x080**, ger SOF plus fyra dominanta identifierarbitar: sändarens följd når fem och den sätter in en stoppbit, medan en mottagare som missade SOF bara står på fyra, inte väntar sig någon, och tar den för en riktig bit. Allt efter den är förskjutet. Felet dyker upp långt senare som ett CRC-fel eller ett stoppbitsbrott, och det slår mot precis de högprioriterade identifierare som en riktig design reserverar för sina viktigaste meddelanden.

Regeln under det är värd att formulera för sig: **båda följdspårarna måste se exakt samma bitar, SOF till och med slutet av CRC-fältet, och inget annat**. Svansen utesluts på båda sidor eftersom inget av skiftregistren är aktiverat för den. Över *ramgränser* är däremot "samma bitar" ingen garanti som designen kan luta sig mot alls, vilket är avsnitt 17.3.

STATE_SOF behöver inget föregripande för det här. STATE_START pulsar bt_resync, så SOF-bitens sampelpunkt ligger ett helt SAMPLE_TICK framåt (kapitel 12); tillståndsmaskinen är med god marginal inne i STATE_SOF vid det laget, och en vanlig kombinatorisk enable räcker.

17.3 Ett rent bord mellan ramar

Följdspårarna nollställs bara av sin reset (kapitel 14, 15), och inom en och samma ram är den beständigheten precis rätt. Låt den sträcka sig *över* ramgränser, och en subtil asymmetri dyker upp: en spårarens tillstånd vid SOF beror nu på vilka bitar just den noden råkade skicka genom den tidigare, och olika noder har olika historia. En nod som sände den förra ramen avslutar den med sin tx_shift_reg-spårare hållande CRC-fältets svans; en nod som bara tog emot håller den historien i sin rx_shift_reg-spårare i stället, och dess sändarspårare visar fortfarande vad nu dess *egen* senaste sändning lämnade, eller ingenting alls efter reset.

Låt nu två sådana noder börja sända vid samma SOF, vilket är precis vad arbitrering finns till för. Identiska identifierarbitar kommer ut ur båda, men noden som bär på en kvarlämnad följd når fem lika bitar *tidigare* och sätter in sin stoppbit en position före den andra noden. På den positionen sänder den en recessiv stoppbit medan den andra noden fortfarande sänder en riktig dominant bit; wired-AND:en läser dominant, den stoppande noden sänder recessivt inne i arbitreringsfältet, och den drar slutsatsen att den förlorat arbitreringen. Ingen tvist ägde rum: den *blivande vinnaren* går ifrån sin egen ram med error höjd. Det här är inte hypotetiskt; det är precis så can_controller_tbs fall 2 misslyckas om spårarna lever kvar, med nod A som går in i det fortfarande hållande svansen av fall 1:s CRC.

Riktig CAN har inget sådant fel, eftersom stoppbitsräkningen börjar om från noll vid varje SOF, och den här designen får samma garanti med en enda signal: båda skiftregistren

hålls i reset medan bussen är ledig, så varje ram börjar med rena spårare på varje nod, vad var och en än gjorde innan. I den deklarativa delen:

```
-- Shift-register reset: asserted while the bus is idle, so every frame
-- starts with clean run trackers on every node.
signal srs_reset_n : std_logic;
```

efter begin:

```
srs_reset_n <= '0' when ((reset_s2_n = '0') or (state = STATE_IDLE)) else '1';
```

och båda skiftregisterinstanserna byter sin andra association från `reset_s2_n` till `srs_reset_n`, så att port map-anropen från kapitel 14 och 15 blir:

```
tx_shift_reg1: entity work.tx_shift_reg
  port map(clock, srs_reset_n, txsr_load, txsr_data, txsr_bit_count, ↵
↵ txsr_shift,
           txsr_tx_bit, txsr_stuff, txsr_done, txsr_bit_valid);

rx_shift_reg1: entity work.rx_shift_reg
  port map(clock, srs_reset_n, bt_sample, rx_bus_s2, rxsr_bit_count, ↵
↵ rxsr_enable,
           rxsr_data, rxsr_valid, rxsr_stuff_error, rxsr_real_bit, ↵
↵ rxsr_real_bit_valid);
```

Reseten släpper på den flank som lämnar `STATE_IDLE`, en hel cykel innan SOF-laddningen klockas in i `tx_shift_reg`, så registren är vakna i tid inför varje ram. Lägg märke till vad det här *inte* ändrar: inom en ram nollställs aldrig något, så det kedjade omladdningsbeteende som kapitel 14:s test 3 låser fast (en följd som fortsätter över en gruppgräns) är orört, och argumentet om SOF-hoppet ovan står sig bit för bit. Det rena bordet börjar vid `STATE_IDLE`, och `STATE_IDLE` går bara att nå mellan ramar.

17.4 Att mata crc15 på mottagarsidan

Spegelbilden av kapitel 16:s grindning på sändarsidan: aktivera `crc15` på `rx_shift_reg.real_bit_valid` (som redan utesluter både "ingen ny bit än" och varje kastad stoppbit), matad med `rx_shift_reg.real_bit`, över SOF till och med CRC-fältet. Att mata tillbaka det mottagna CRC-fältets egna bitar återför registret till exakt noll om ramen kom fram intakt (kapitel 13:s generera-och-sedan-kontrollera), vilket rapporteras av `crc_valid`.

Det finns en `crc15` och en `enable`, så det här ersätter kapitel 16:s tilldelning i stället för att stå vid sidan av den. `role` väljer nu källa i stället för att stänga av motorn:

```

crc_enable <= (txsr_bit_valid and not txsr_stuff and role) or
               (rxsr_real_bit_valid and not role);
crc_data_in <= txsr_tx_bit when role = '1' else rxsr_real_bit;

```

Låt kapitel 16:s version stå kvar, och en mottagande nod ackumulerar aldrig något, så `crc_valid` läser '1' (resetvärdet) hela vägen och kontrollen i `STATE_CRC_L0_WAIT` passerar för varje ram, korrumperad eller inte.

`crc_clear` berörs inte av något av det här och står kvar exakt som kapitel 16 skrev den, en puls i `STATE_START` oavsett roll. Den spelar större roll nu än den gjorde då: det är i det här kapitlet noden får tre sätt att gå ur en ram i förtid, och vart och ett lämnar motorn mitt i ett meddelande den aldrig kommer att avsluta. Ingen av avbrottsvägarna nedan behöver nollställa den på vägen ut, eftersom nästa ram nollställer den på vägen in.

17.5 Väntan på ett extra sampel: den avgörande tajminglärdomen

På sändarsidan lägger `tx_shift_reg.done` redan en extra bitperiod på att låta en grupp sista bit löpa ut innan nästa grupp laddas. En mottagare reagerar på `rx_shift_reg.valid` utan någon sådan inbyggd väntan, så `can_controller` måste lägga till den uttryckligen vid vart och ett av RX-sidans `STATE_LOAD_*`-tillstånd: **vänta på ytterligare ett `bt_sample`** innan den går vidare, eftersom ledningen fortfarande visar förra gruppens sista bit under hela dess avslutande period.

Det finns en andra hake: `state` blir skifttillståndet först **en cykel efter** det `bt_sample` som utlöste övergången, en cykel för sent för att aktivera `rx_shift_reg` för just det samplet. Därför **föregriper** det kombinatoriska styrblocket `rxsr_enable` (och sätter den kommande gruppens `bit_count`) för exakt det samplet medan det fortfarande står i `STATE_LOAD_*`-tillståndet.

Hur det ser ut cykel för cykel. Här är en gruppgräns på mottagarsidan, där den 8 bitar breda höga ID-gruppen lämnar över till den 4 bitar breda gruppen med låga ID plus RTR. Cyklerna är 50 MHz-klockans, så de två `bt_sample`-pulserna ligger 50 cykler isär; följderna av identiska cykler mellan dem är utelämnad.

klockcykel	bt_sample	state	rxsr_bit_count	rxsr_enable	rxsr_valid	
n	0	STATE_ARB_HI		8	1	0
n + 1	1	STATE_ARB_HI		8	1	0
n + 2	0	STATE_ARB_HI		8	1	1
n + 3	0	STATE_LOAD_ARB_LO		4	1	0
...	0	STATE_LOAD_ARB_LO		4	1	0
n + 51	1	STATE_LOAD_ARB_LO		4	1	0
n + 52	0	STATE_ARB_LO		4	1	0

Läs de två sampelcyklerna mot varandra. Vid $n + 1$ tas den åttonde biten av den höga ID-gruppen, och `rxsr_valid` dyker upp en cykel senare vid $n + 2$, vilket är när tillståndsmaskinen bestämmer sig för att gå vidare. `state` läser inte `STATE_LOAD_ARB_L0` förrän vid $n + 3$.

Titta nu på $n + 51$, samplet som måste fånga den *första* biten av den låga ID-gruppen. `state` läser fortfarande `STATE_LOAD_ARB_L0` i det ögonblicket. Den blir inte `STATE_ARB_L0` förrän vid $n + 52$, en cykel efter att biten redan kommit och gått. Det är hela problemet på en rad: tillståndet som namnger gruppen anländer alltid en cykel senare än samplet som startar den.

Föregripandet är det som gör `rxsr_bit_count = 4` och `rxsr_enable = '1'` sanna vid $n + 51$ ändå, genom att driva dem från `STATE_LOAD_ARB_L0`-armen i stället för `STATE_ARB_L0`-armen. Driv dem från `STATE_ARB_L0` i stället, och samma gräns ser ut så här:

klockcykel	bt_sample	state	rxsr_bit_count	rxsr_enable	rxsr_valid
$n + 3$	0	STATE_LOAD_ARB_L0		0	0
...	0	STATE_LOAD_ARB_L0		0	0
$n + 51$	1	STATE_LOAD_ARB_L0		0	0
$n + 52$	0	STATE_ARB_L0	4	1	0

`rx_shift_reg` är avstängt över det enda sampel som spelade roll. Den första biten av identifierarens låga grupp samlas aldrig in, så gruppen blir färdig en bit kort mot ledningen, varje senare grupp är förskjuten med en, och `crc15` matas med en ström som aldrig stämmer. Ingenting rapporterar något fel vid $n + 51$; felet dyker upp vid `STATE_CRC_L0_WAIT`, ungefär fyrtio bitperioder senare, som en CRC som inte går till noll.

Lägg märke till att det här inte kostar något extra på ledningen. `STATE_LOAD_ARB_L0` förbrukar redan förra gruppens avslutande bitperiod, vilket är varför den går vidare på `bt_sample` och samlar inget eget; föregripandet ställer bara registret i rätt konfiguration innan den perioden är slut.

Det här mönstret upprepas vid varje gruppgräns och är designens allra viktigaste lärdom: **en signal som ser "klar" ut en cykel för tidigt eller för sent tar inte ut sig själv över gruppgränserna; den ackumuleras till en tyst felinriktning** mellan sändare och mottagare som först långt senare dyker upp som ett CRC-fel eller ett stoppbitsbrott, långt ifrån sin orsak.

17.6 Detektering av förlorad arbitring

Exakt avsnitt 10.6:s regel — kapitel 4 i *CAN: bussen, framen och kontrollern* är den utförliga versionen — kontrollerar en gång per bit vid sampelpunkten enbart under `STATE_ARB_HI` och `STATE_ARB_L0`, vilket är precis arbitreringsfältet, ID plus RTR. Unika ID:n garanterar att arbitringen avgörs inom den 11 bitar långa identifieraren; kontrollen täcker harmlöst även RTR, där varje nod här sänder dominant, en bit som aldrig kan förlora:

```

if ((bt_sample = '1') and (txsr_tx_bit = '1') and (rx_bus_s2 = '0')) then
    error    <= '1';    -- Sent recessive, read dominant: lost.
    tx_done  <= '1';    -- The attempt is over; say so, win or lose.
    role     <= '0';
    idle_bits <= 0;
    state    <= STATE_IDLE;
end if;

```

Kontrollen bor inuti STATE_ARB_HIs och STATE_ARB_LOs if (role = '1')-gren, och det är det som avgränsar den till en sändande nod; de tre termerna ovan är själva avkänningsvillkoret.

Varför tx_done pulsar även här, på en ram som aldrig sändes. Det läser fel först: tx_done betyder "ramen blev klar", och det blev den här inte. Läs den i stället som *"sändningsförsöket är över, och porten är din igen"* — vilket är den enda fråga en anropare kan agera på. error är det som säger om försöket lyckades, och tillsammans bär de två hela utfallet: tx_done ensam betyder sänd, tx_done med error betyder förlorad.

Utelämna den, och anroparen har ingen flank alls att vänta på. En nod som förlorar arbitreringen återvänder till STATE_IDLE och är fullt kapabel att sända igen, men ingenting annonserar det någonsin, så varje anropare som väntar på tx_done innan den begär om väntar för evigt. Det är inte hypotetiskt: det är precis så register_bank (kapitel 18) sätter sin TX-redo-bit, och utan den här pulsen skulle en enda förlorad arbitrering — normalfallet på en buss med två noder — lämna hela noden oförmögen att sända tills den resettades.

Lägg märke till att det är rx_bus_s2, den synkroniserade kopian, och inte porten rx_bus. Avsnitt 11.5:s regel gäller överallt, den här jämförelsen inräknad: ingenting i designen läser den råa pinnen. Det vore lätt att prata sig ur den här, eftersom sampelpunkten ligger 35 tick in i en bit som stabiliserade sig nära tick 0, så nivån är med säkerhet stabil vid det laget. Det argumentet handlar om *när* värdet är stabilt på ledningen, och metastabilitet handlar om vad en vippa gör när den samplar en signal som ingen klocka i det här chippet styr. De två extra vipporna kostar två klocktick av femtio och köper den enda jämförelse hela arbitreringsupplägget vilar på.

Att nollställa idle_bits är inte städning i bokföringen. Utan den landar noden tillbaka i STATE_IDLE med idle_bits fullt medan vinnaren är mitt i sin ram, så vinnarens nästa dominanta bit läses som ett nytt SOF, noden går in i STATE_START, och STATE_START suddar den error den just höjde. Samma nollställning krävs vid varje annat avbrott nedan, av samma skäl.

17.7 Att färdigställa processerna

Samma regler som i kapitel 16:s genomgång: kodfragmenten är på varandra följande skivor i den ordning koden läses, och ingenting från kapitel 16 skrivs om utöver dess markerade platshållare; varje skiva nedan fyller i en av dem.

Den kombinatoriska processen

Den får två nya block, och `crc_valid` ansluter sig till dess känslighetslista (ACK-drivningen nedan läser den). Först, mellan sändarsidans case för skiftgrindningen och case-satsen för bussdrivningen, mottagarsidans enable och gruppbredder, vilket är föregripandet gjort konkret:

```
-- The receive-side enable and chunk widths: the pre-empt. Active
-- through every stuffed field's LOAD and shifting state, so the sample
-- that falls while state still reads STATE_LOAD_* is collected at the
-- new width.
if (role = '0') then
  case state is
    when STATE_SOF =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0001";
    when STATE_LOAD_ARB_HI | STATE_ARB_HI
      | STATE_LOAD_DATA   | STATE_DATA
      | STATE_LOAD_CRC_HI | STATE_CRC_HI =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "1000";
    when STATE_LOAD_ARB_LO | STATE_ARB_LO =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0100";
    when STATE_LOAD_CTRL | STATE_CTRL =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0110";
    when STATE_LOAD_CRC_LO | STATE_CRC_LO =>
      rxsr_enable    <= '1';
      rxsr_bit_count <= "0111";
    when others =>
      null;
  end case;
end if;
```

Varje `STATE_LOAD_*`-tillstånd föregriper den *kommande* gruppens bredd, ett `bt_sample` innan state synligt ändras, precis som avsnitt 17.5 härledde; att gruppera det med sitt skifttillstånd i en och samma arm är det som gör det automatiskt. För det andra, mottagarens enda dominanta bit: en ny arm i case-satsen för bussdrivningen, ovanför dess `when others`:

```
when STATE_ACK_SLOT =>
  if ((role = '0') and (crc_valid = '1')) then
    bus_en <= '1';
    tx_bus <= '0';
```

```
end if;
```

Den sekventiella processen

Varje markering -- role = '0': L17. blir armens andra halva. Det finns bara två mottagarformer. Ett mottagande STATE_LOAD_*-tillstånd väntar på ett extra bt_sample (samma elsif i alla sex), och ett mottagande skifttillstånd går vidare på rxsr_valid och skördar sin grupp om ramen behåller något av den. Här är båda formerna ifyllda, på STATE_SOF och identifierarens höga par; lägg märke till att STATE_ARB_HIs sändarhalva också växte, och tog upp avsnitt 17.6:s arbitreringskontroll före done-testet. (De två kan aldrig slå till samma cykel, eftersom bt_sample ligger mitt i perioden och done följer ett bit_done, så ordningen handlar om läsbarhet snarare än prioritet; en förlust på en grupp:s sista bit vinner ändå helt enkelt genom att komma först, vid den bitens sampelpunkt.)

```
when STATE_SOF =>
  if (role = '1') then
    if (txsr_done = '1') then
      state <= STATE_LOAD_ARB_HI;
    end if;
  else
    if (rxsr_valid = '1') then
      state <= STATE_LOAD_ARB_HI;
    end if;
  end if;
when STATE_LOAD_ARB_HI =>
  if (role = '1') then
    state <= STATE_ARB_HI;
  elsif (bt_sample = '1') then
    state <= STATE_ARB_HI;
  end if;
when STATE_ARB_HI =>
  if (role = '1') then
    if ((bt_sample = '1') and (txsr_tx_bit = '1') and (↵
↵ rx_bus_s2 = '0')) then
      error      <= '1';      -- Sent recessive, read dominant:↵
↵ lost.

      tx_done    <= '1';
      role       <= '0';
      idle_bits  <= 0;
      state      <= STATE_IDLE;
    elsif (txsr_done = '1') then
      state <= STATE_LOAD_ARB_LO;
    end if;
```

```

else
  if (rxsr_valid = '1') then
    rx_id_acc(10 downto 3) <= rxsr_data(7 downto 0);
    state <= STATE_LOAD_ARB_L0;
  end if;
end if;

```

STATE_LOAD_ARB_L0, STATE_LOAD_CTRL, STATE_LOAD_DATA, STATE_LOAD_CRC_HI och STATE_LOAD_CRC_L0 tar alla den identiska väntan elsif (bt_sample = '1'), och STATE_CRC_HI/STATE_CRC_L0 tar det identiska rxsr_valid-framsteget utan att skörda något: deras bitar når crc15 bit för bit genom real_bit, och de hopsatta grupperna betyder ingenting. Tillstånden med verkligt mottagararbete i sig:

```

when STATE_ARB_L0 =>
  if (role = '1') then
    if ((bt_sample = '1') and (txsr_tx_bit = '1') and (↵
↵ rx_bus_s2 = '0')) then
      error      <= '1';
      tx_done    <= '1';
      role       <= '0';
      idle_bits  <= 0;
      state      <= STATE_IDLE;
    elsif (txsr_done = '1') then
      state <= STATE_LOAD_CTRL;
    end if;
  else
    if (rxsr_valid = '1') then
      rx_id_acc(2 downto 0) <= rxsr_data(3 downto 1);
      if (rxsr_data(0) = '1') then
        error      <= '1';    -- Recessive RTR: a remote ↵
↵ frame.

        idle_bits <= 0;
        state      <= STATE_IDLE;
      else
        state <= STATE_LOAD_CTRL;
      end if;
    end if;
  end if;
when STATE_CTRL =>
  if (role = '1') then
    if (txsr_done = '1') then
      if (unsigned(tx_dlc) = 0) then
        state <= STATE_CRC_WAIT;

```

```

        else
            state <= STATE_LOAD_DATA;
        end if;
    end if;
else
    if (rxsr_valid = '1') then
        rx_dlc_acc <= rxsr_data(3 downto 0);
        if (unsigned(rxsr_data(3 downto 0)) = 0) then
            state <= STATE_CRC_WAIT;
        else
            state <= STATE_LOAD_DATA;
        end if;
    end if;
end if;
when STATE_DATA =>
    if (role = '1') then
        if (txsr_done = '1') then
            if (data_byte_idx + 1 = to_integer(unsigned(tx_dlc))) --
-- then
                state <= STATE_CRC_WAIT;
            else
                data_byte_idx <= data_byte_idx + 1;
                state <= STATE_LOAD_DATA;
            end if;
        end if;
    else
        if (rxsr_valid = '1') then
            case data_byte_idx is -- Fixed position per byte --
-- index.
                when 0 => rx_data_acc(63 downto 56) <= rxsr_data;
                when 1 => rx_data_acc(55 downto 48) <= rxsr_data;
                when 2 => rx_data_acc(47 downto 40) <= rxsr_data;
                when 3 => rx_data_acc(39 downto 32) <= rxsr_data;
                when 4 => rx_data_acc(31 downto 24) <= rxsr_data;
                when 5 => rx_data_acc(23 downto 16) <= rxsr_data;
                when 6 => rx_data_acc(15 downto 8) <= rxsr_data;
                when others => rx_data_acc(7 downto 0) <= --
-- rxsr_data;

            end case;
            if (data_byte_idx + 1 = to_integer(unsigned(rx_dlc_acc--
-- ))) then
                state <= STATE_CRC_WAIT;
            else
                data_byte_idx <= data_byte_idx + 1;

```

```

        state      <= STATE_LOAD_DATA;
    end if;
end if;
end if;

```

Två läsningar före flanken att lägga märke till, båda samma regel som kapitel 14:s MAX_RUN - 1. STATE_CTRL avgör DLC-grenen direkt utifrån rxsr_data(3 downto 0), eftersom rx_dlc_acc tilldelas på just den här flanken och fortfarande läser den förra ramens värde. Och STATE_DATA jämför mot rx_dlc_acc, som vid det laget *är* stabil, låst ett helt fält tidigare.

Svensen skiljer sig från kapitel 16:s i exakt två armar. STATE_CRC_LO_WAIT är där mottagaren förbrukar den sista CRC-bitens avslutande period och fäller sin dom, och STATE_EOFs sista bit är där en mottagare publicerar ramen:

```

when STATE_CRC_LO_WAIT =>
    if (role = '1') then
        state <= STATE_CRC_DELIM;    -- One clock cycle, not a bit-
        period.
    elsif (bt_bit_done = '1') then
        if (crc_valid = '1') then
            state <= STATE_CRC_DELIM;
        else
            error      <= '1';        -- The engine did not return a
            to zero.
            idle_bits <= 0;
            state      <= STATE_IDLE;
        end if;
    end if;
end if;

```

```

when STATE_EOF =>
    if (bt_bit_done = '1') then
        if (field_bit_count = 0) then
            if (role = '1') then
                tx_done <= '1';
            else
                rx_id      <= rx_id_acc;
                rx_dlc     <= rx_dlc_acc;
                rx_data    <= rx_data_acc;
                rx_valid   <= '1';
            end if;
            state <= STATE_IDLE;
        else

```

```

        field_bit_count <= field_bit_count - 1;
    end if;
end if;

```

Till sist en pålagring som inte hör hemma i något enskilt tillstånd. Efter hela case-satsen, fortfarande inuti den klockade grenen:

```

-- A stuffing violation aborts a reception from any stuffed state.
if ((role = '0') and (rxsr_stuff_error = '1')) then
    error      <= '1';
    idle_bits <= 0;
    state      <= STATE_IDLE;
end if;

```

Placerad efter case-satsen vinner den över vad än tillståndsarman bestämde på samma flank, vilket är hela poängen: ett brott är fatalt var det än slår till, och ett sampel kan slå till i vilken som helst av armarna för stoppade fält, LOAD-tillstånden inräknade (deras föregripna sampel behandlas som vilket annat som helst). Att skriva den en gång är bättre än att upprepa tre rader i ett dussin armar.

Att kvittera, kontrollera och avbryta

- **ACK-luckan:** en mottagare vars `crc_valid` är satt driver bussen dominant under den enda biten för att kvittera ramen.
- **CRC-kontrollen:** i `STATE_CRC_LO_WAIT` läser en mottagare `crc_valid`. Är den satt väntar den på `bt_bit_done` och går sedan vidare till `STATE_CRC_DELIM`; är den det inte avbryter den.
- **Stoppbitsbrott:** `rx_shift_reg.stuff_error` avbryter mottagningen från vilket tillstånd som helst. En sändare kan inte bryta mot stoppbitsregeln, så det här gäller bara RX.
- **Arbitreringsförlust** är det enda avbrott som sker under *sändning*, och därmed det enda som också pulsar `tx_done`. Avbrotten på mottagarsidan avslutar en mottagning, och en mottagning har ingen anropare som väntar på en färdigpuls: `rx_valid` uteblir helt enkelt, vilket är precis rätt, eftersom ingen ram anlände.
- **error ligger kvar:** nollställd i `STATE_START`, hållen i övrigt, så att en anropare ser den ända tills nästa ram börjar. Det är precis därför `STATE_IDLE` kräver en **ledig buss** innan noden lämnar det tillståndet över huvud taget, och varför varje avbrott här måste nollställa `idle_bits` på vägen ut. Varje avbrott landar tillbaka i `STATE_IDLE` mitt i en ram, medan bussen fortfarande är upptagen. Utan kravet på ledig buss skulle noden gå in i `STATE_START` igen inom några få bitperioder och sudda den error den höjde, så att en anropare som pollar `error` aldrig skulle se den. Att först kräva elva recessiva bitperioder är exakt den garanti som "ända tills nästa ram börjar" utfärdar.

17.8 Att verifiera den: can_controller_tb

Nu när mottagarvägen är komplett körs controller/can_controller_tb.vhd äntligen hela vägen. Den kopplar **två** can_controller-noder till en enda buss med öppen dränering, kombinerar varje nods par (tx_bus, bus_en) på det sätt avsnitt 10.2 beskrev en wired-AND, och kör två fall.

Fall 1, mottagning. Nod A sänder ID 0x123, DLC 5, data 11 22 F8 44 55 medan nod B tar emot. Den kontrollerar tx_done och rx_valid, och att B återskapar ID:t, DLC:n och **varje** databyte på sin fasta position. Byten 0xF8 är vald för att tvinga fram en stoppbit inne i datafältet, så det här övar sändvägen, mottagarvägen, avstopningen och flerbytesplaceringen i en och samma ram. En ensam nod kan inte testa något av det: rx_shift_reg är aktivt bara medan role = '0', så en sändare tar aldrig emot sin egen ram.

Fall 2, arbitrerings. Båda noderna begär under samma cykel, A med ID 0x000 och B med 0x001, åtskilda bara i den sista identifierarbiten. B måste upptäcka förlusten på exakt den biten och sluta driva bussen; A måste bli klar som vanligt, ovetande om att en tvist ägt rum. Den sista assertionen är den att lägga märke till: den läser B:s error *efter* att A:s ram blivit klar, så den faller om inte både den kvarliggande error och disciplinen kring idle_bits ovan håller.

17.9 Sammanfattning

Det här ska du kunna nu

- Implementera mottagarvägen i samma tillståndsmaskin som sänder.
- Förklara hur en nod avgör att den förlorat arbitrerings, och vad den gör då.
- Sätta ihop rx_id, rx_dlc och rx_data ur den mottagna bitströmmen och höja rx_valid vid rätt tillfälle.
- Köra systemtestbänken med två noder på en buss och läsa vad den rapporterar.
- Säga vad error betyder, vilka fyra orsaker som sätter den, och när den nollställs.

Frågor att testa dig själv med

1. Hur upptäcker en nod att den förlorat arbitrerings, och exakt vilka bitar jämför den?
2. Vad gör en nod som förlorat arbitrerings, och varför är det inte "försök igen omedelbart"?
3. Varför måste rx_shift_regs enable föregripas en cykel?
4. En nod kan inte sända och ta emot samtidigt. Var i tillståndsmaskinen syns det?
5. Vad sätter error, och vad nollställer den?
6. Vad bevisar can_controller_tb som de fyra modultestbänkarna tillsammans inte gör?

Riktvärde

Här går projektet över från CAN till SPI. En grupp som inte fått `can_controller_tb` att passera bör prioritera det framför registerbanken: SPI-halvan går att bygga och testa mot en kontroller som ännu inte fungerar, men bring-upen i kapitel 19 gör det inte.

Systemtestbänken hoppas över av bygget tills kontrollern har en mottagarväg. Ser du fortfarande "skipped" när du tycker att den borde köra, kör med `CI_BUILD_ALL=1`; bilaga A förklarar varför den kontrollen fungerar så.

Blicken framåt

Kapitel 18 tar registerbanken, och SPI från vågformen och uppåt. Här börjar den halva som gör kontrollern nåbar från en mikrokontroller.

17.10 Övningar

När din mottagningslogik är klar verifierar `controller/can_controller_tb.vhd` sändning och mottagning tillsammans, och `make build-project` slutar hoppa över den: alla sex testbänkar i `controller/` körs härifrån och framåt. De två i `bridge/` fortsätter hoppas över tills kapitel 18 och 19 skriver modulerna de prövar.

Om den fortfarande rapporteras som överhoppad efter att du tror att mottagningsvägen är klar, läs meddelandet före din design. Bygget avgör saken genom att leta efter kapitel 17:s CRC-grindning på mottagningssidan inuti `can_controller.vhd`, så en korrekt kontroller som stavar de två uttrycken annorlunda hoppas över i stället för att köras. `CI_BUILD_ALL=1 make build-project` tvingar fram den, och bilaga A förklarar varför kontrollen fungerar så.

Övning 17.1 Resonemang om tajmingfixen på RX-sidan

RESONEMANG

- Avsnitt 17.5 visar att `rx_shift_reg` måste aktiveras *kombinatoriskt*, under själva `STATE_LOAD_*`-tillståndet, i stället för att vänta tills `state` synligt blir skifttillståndet. Förklara med egna ord varför det att först vänta in att tillståndsovergången blir klar skulle få `rx_shift_reg` att missa just det sampel som utlöste väntan från början.
- `STATE_CRC_LO_WAIT` väntar på `bt_bit_done`, inte på `bt_sample`, till skillnad från varje väntan vid en gruppgräns i designen (tillstånden `STATE_LOAD_*`). Förklara med egna ord varför just den övergången måste ligga i linje med en *bitperiodsgräns* och inte med en sampel-punkt. Vilka andra tillstånd delar utlösaren `bt_bit_done`, och vad har de gemensamt med den?
- Anta (hypotetiskt) att `STATE_CRC_WAIT` också väntade på `bt_sample` för RX-rollen, utöver att `STATE_LOAD_CRC_HI` gör detsamma omedelbart efteråt. Förklara vad som skulle gå fel,

och varför det inte så mycket skulle visa sig som ett rent haveri som att en mottagande nod hamnar en bitperiod längre efter en sändande nod än den borde.

Övning 17.2 Arbitrering för hand

RESONEMANG

Två noder begär sändning på samma cykel: nod A med ID 0x2A0, nod B med ID 0x2C0.

- Skriv båda ID:na binärt (11 bitar, MSB först) och markera den första bitposition där de skiljer sig.
- Säg med avsnitt 10.6:s arbitreringsregel (dominant 0 vinner) vilken nod som vinner och vid vilken bit förloraren faller ur.
- Den förlorande noden upptäcker sin förlust med `bt_sample = '1'` and `txsr_tx_bit = '1'` and `rx_bus_s2 = '0'`. Säg, för förloraren vid den biten, vad var och en av de tre termerna evalueras till och varför villkoret slår till.
- Förloraren sätter `error <= '1'` och faller ner till `STATE_IDLE` medan vinnaren fortfarande driver bussen. `error` nollställs i `STATE_START`. Förklara varför förlorarens `error` skulle bli oobserverbart för en anropare om `STATE_IDLE` gick in i en mottagning på en dominant *nivå*.
- En lockande lagning är att i stället leta efter *flanken* från recessiv till dominant, i linje med hur avsnitt 10.4 beskriver SOF. Visa att det egentligen inte fungerar, med hjälp av stoppbitsregeln: vad sätter sändaren in efter fem dominanta bitar i rad, och vad gör bussen på biten efter det? Ungefär hur långt in i vinnarens ram hinner förloraren innan flanktestet slår till ändå?
- Ange regeln designen faktiskt använder (avsnitt 16.1). Två fakta gör den hållbar; ge båda: vad stoppbitsregeln begränsar inne i det stoppade området, och vad den ostoppade svansen lägger till ovanpå det som skjuter tröskeln till elva. Säg sedan varför tomgångsräknaren måste nollställas till **fullt** och inte till tomt när noden kommer ur reset.

Övning 17.3 SOF-biten som ingen behåller

RESONEMANG

`STATE_SOF` aktiverar `rx_shift_reg` under en bit och kastar sedan värdet. Avsnitt 17.2 förklarar varför; den här övningen handlar om att se felet med egna ögon.

- Ta ID 0x000, DLC 0. Skriv ut bitarna en sändare lägger på ledningen från SOF till och med kontrollfältets slut, med avsnitt 16.2:s fälttabell, och markera var `tx_shift_reg` sätter in varje stoppbit. Kom ihåg att följdräknaren räknar med SOF.
- Gör nu samma sak sett från en mottagare som hoppade över SOF, så att dess följdräknare ligger en dominant bit efter. Markera var *den* förväntar sig stoppbitar. Ställ upp de två mot varandra och ange den första bitposition där de är oense.
- Vid den positionen skickar sändaren en stoppbit och mottagaren väntar sig ingen. Vad gör mottagaren med den, och vad gör det av varje bit efter den?

d) Upprepa a) och b) för ID 0x123, det ID som `can_controller_tb` använder i fall 1, och bekräfta att de två räknarna är överens hela vägen. Ange den allmänna regeln för vilka identifierare som går sönder: vad måste gälla för de första identifierarbitarna, och hur många av de 2048 standard-ID:na uppfyller det?

e) Fall 2 i `can_controller_tb` skickar *faktiskt* ID 0x000. Förklara varför buggen ändå inte skulle visa sig där, och säg vad du skulle lägga till i testbänken för att fånga den.

Övning 17.4 Att ta emot en ram utan data

RESONEMANG

Gå igenom **mottagningsrollen** genom en komplett ram med `DLC = 0`, med hjälp av tabellen per tillstånd i avsnitt 17.2. En mottagare vet inte `DLC:n` förrän den har avkodat kontrollfältet, så det här är också den väg som prövar om din övergång i `STATE_CTRL` är rätt.

a) Lista varje tillstånd noden passerar, från `STATE_IDLE` tillbaka till `STATE_IDLE`. Markera den enda övergång som skiljer sig från en ram som bär data, och säg vad som får den att slå till.

b) Ange för varje `STATE_LOAD_*`-tillstånd i din lista det `rxsr_bit_count` som det kombinatoriska nätet föregriper, och säg vilket `bt_sample` det föregripandet måste vara på plats för: det som utlöser övergången ut ur `STATE_LOAD_*`, eller nästa.

c) Anta att föregripandet vore en cykel för sent överallt, alltså att `rxsr_enable` gick hög först när `state` synligt läste skifttillståndet. Hur många riktiga bitar skulle `rx_shift_reg` ha missat för en ram med `DLC = 0` när noden når `STATE_CRC_LO_WAIT`, och vad skulle `crc_valid` läsa där?

d) Ditt svar på c) är ett CRC-fel som rapporteras många bitperioder efter det faktiska misstaget. Relatera det till avsnitt 17.5:s påstående att en felställning på en cykel ackumuleras till en tyst felställning som visar sig först senare, långt från sin orsak, och säg varför det gör den här sortens bugg dyr att felsöka enbart utifrån testbänkens utskrifter.

Registerbanken och SPI från vågformen

Här börjar den halva som gör kontrollern nåbar från en mikrokontroller.

I DET HÄR KAPITLET

- Varför `tx_done` och `rx_valid` som encykelspulser är oanvändbara för en pollande drivrutin, och vad registerkartan lovar i stället: klibbiga, pollbara nivåer med uttryckliga nollställningar.
- `register_bank`: STATUS-låsningen, `TX_SEND` som en skrivutlöst händelse, infångning av mottagen ram, och maskning vid skrivning.
- SPI från vågformen och uppåt: lägen, MSB först, SS som ramning.
- Varför slaven **inte** klockar på SCK utan översamlar den i 50 MHz-domänen.
- Den utdelade `spi_slave`: byte-motorn som får SCK, MOSI och SS säkert in i systemklockdomänen och lämnar över en ren byte i taget.

`can_controller` är färdig från och med kapitel 17. Det här kapitlet bygger det första av de två lager som gör den nåbar utifrån. Modulen kan skrivas långt innan kontrollern är färdig — den behöver bara `can_def` — så den är ett fullvärdigt arbetsspår som kan starta parallellt.

Den auktoritativa källan för varje registernummer i det här kapitlet är bilaga D. Där kapitlet och specifikationen är oense vinner specifikationen.

18.1 Varför det här lagret finns

`can_controllers` statusutgångar är gjorda för en anropare i samma klockdomän som ser varje cykel: `tx_done` och `rx_valid` är **encykelspulser**, 20 ns vid 50 MHz (kapitel 11). En drivrutin som pollar över SPI samplar systemet några *mikrosekunder* i taget i bästa fall — en puls är borta tusentals cykler innan nästa pollning kommer. Registerkartan (bilaga D) lovar därför något annat: **klibbiga, pollbara nivåer** med uttryckliga nollställningar (`RX_ACK`, `ERROR_FLAGS`), och skrivutlösta händelser (`TX_SEND`).

Att konvertera mellan de två världarna är hela den här modulens uppgift. Den innehåller ingen protokollkunskap i någon riktning: ingenting om CAN-ramar (det stannar i

can_controller) och ingenting om SPI (det kommer senare i det här kapitlet, och i kapitel 19). Det är register, läsningar och en pulsgenerator — vilket är precis därför den kan verifieras fullständigt under ett enda pass, innan det finns någon SPI alls.

Du har sett båda dess kärntrick förut, i kapitel 3 och 4. En klibbig statusbit är kapitel 3:s vippa med de två frågorna uttryckligen besvarade: vad som sätter den, och vad som nollställer den. Och att göra om en hållen nivå till en encykelspuls är precis vad button_syncs flankdetektering gjorde i kapitel 4 — här går den åt samma håll för TX_SEND (en skrivning blir en puls) och åt motsatt håll för tx_done (en puls blir en nivå som ligger kvar). Det här kapitlet befordrar båda från en övning till ett riktigt arkitekturlager med ett dokumenterat kontrakt.

18.2 Gränssnittet

Sexton portar, **alla ingångar först, sedan alla utgångar**, samma konvention som varje modul i projektet — och, som överallt annars i det, är bindningen *positionell*, så ordningen nedan är det kontrakt register_bank_tb håller dig till:

1 clock	5 reg_wdata	9 rx_data	13 tx_id
2 reset_s2_n	6 tx_done	10 rx_valid	14 tx_dlc
3 reg_index	7 rx_id	11 error	15 tx_data
4 reg_write	8 rx_dlc	12 reg_rdata	16 tx_req

Typerna kommer från can_def (use work.can_def.all;), så portarna mot kontrollern har exakt samma typer som can_controllers egna. Vid sidan av clock och reset_s2_n (portarna 1 och 2) faller de fjorton övriga i två grupper.

Sidan mot bryggan (portarna 3–5 och 12). En medvetet minimal registeråtkomstport — det som kapitel 19:s brygga kommer att driva, och det som testbänken driver direkt i dag:

- reg_index : std_logic_vector(3 downto 0) — vilket register, protokollets index (offset delat med 4).
- reg_write : std_logic — en **encykelsstrob**; på dess stigande klockflank verkställs reg_wdata i det adresserade registret.
- reg_wdata : std_logic_vector(31 downto 0) — värdet som ska skrivas.
- reg_rdata : std_logic_vector(31 downto 0) — det adresserade registrets aktuella värde, **kombinatoriskt**: det följer reg_index utan att någon klockflank är inblandad. Regeln om att låsa en gång vid läsning ur protokollspecifikationen är *bryggans* uppgift (kapitel 19), inte den här modulens — banken berättar alltid den nuvarande sanningen, och bryggan avgör när den tar sitt enda atomära sampel.

Sidan mot kontrollern (portarna 6–11 och 13–16). Speglar can_controllers portar på registersidan en mot en: tx_id, tx_dlc, tx_data och tx_req ut till kontrollern, och tx_done,

rx_id, rx_dlc, rx_data, rx_valid och error tillbaka från den. I kapitel 19:s toppnivå kopplas de rakt över, port till port, ingenting emellan.

reset_s2_n är den *synkroniserade* reseten, som för varje delblock i projektet. Kapitel 19:s toppnivå äger en meta_prev för den, precis som can_controller äger sin egen internt.

18.3 De tre STATUS-bitarna: en låsning var

Varje STATUS-bit är en vippa med ett namngivet sättvillkor och ett namngivet nollställningsvillkor, och ingenting mer:

Bit	Betydelse	Sätts av	Nollställs av
0	TX klar	reset; kontrollerns puls tx_done; en stigande flank på error medan den här biten är låg; en accepterad skrivning till TX_ABORT	en accepterad skrivning till TX_SEND
1	RX giltig	kontrollerns puls rx_valid	en skrivning av 0x1 till RX_ACK
2	Fel	en stigande flank på kontrollerns nivå error	en skrivning av 0x0 till ERROR_FLAGS

Tre av dem förtjänar en andra blick:

- **Bit 0 börjar på '1'.** Ur reset är kontrollern i vila, så banken är redo att ta emot en ram — en drivrutin som pollar innan den någonsin sänt måste se "klar", annars skulle send() misslyckas för alltid på ett nyss spänningssatt system.
- **Bit 2 låser en flank, inte en nivå.** Kontrollern håller error hög tills nästa ram börjar (kapitel 11), på sitt eget schema. Att låsa den stigande flanken ger *drivrutinen* en egen livscykel för flaggan: en nollställning tar bort den även medan kontrollerns nivå fortfarande är hög, och låsningen sätts inte igen förrän ett genuint *nytt* fel producerar en ny stigande flank. Att i stället låsa på nivån skulle få nollställningen att verka trasig — nollställd och omedelbart satt igen — så länge kontrollern håller ledningen.
- **Bit 0 har tre vägar tillbaka upp, och det är avsiktligt.** Läs de två extra som livrem och hängslen kring det enda fel som kan slå ut hela noden.

En sändning som förlorar arbitringen slutar utan att någonsin nå EOF. Kapitel 17 låter kontrollern pulsa tx_done också på den vägen, just för att den här biten ska komma tillbaka — men kontrollern är *din*, och en version som glömmet det lämnar biten låg för alltid, så varje senare TX_SEND kastas och noden kan inte sända igen förrän den resettas. Banken förlitar sig därför inte på att kontrollern gör rätt. **En stigande flank på error medan bit 0 redan är låg kan bara betyda att en sändning slutade illa**, eftersom en nod antingen sänder eller tar emot, och ett mottagningsfel inte kan inträffa medan en sändning pågår. Så den flanken sätter biten också, och det behövs ingen role-port för att veta det.

TX_ABORT är den tredje vägen: en uttrycklig nödutgång som drivrutinen kan dra i om den bedömer att den väntat länge nog. Ingenting i konstruktionen ska någonsin behöva den. Bygg

in den ändå — kostnaden är ett enda register som bara går att skriva till, och alternativet är en nod som en drivrutin inte har någon möjlighet att få loss.

En enda ordningsregel binder ihop läsningarna: i den klockade processen **tillämpas kontrollernas händelser efter bryggans skrivningar**, så om en TX_SEND-skrivning och en tx_done-puls någonsin landade på samma flank vinner hårdvaran och banken förblir klar. En så sällsynt konflikt bör lösas åt det håll som inte kan köra fast systemet, och det är samma princip som stycket ovan tillämpar tre gånger om.

18.4 TX_SEND: en skrivning blir en puls

En TX_SEND-skrivning med `reg_wdata(0) = '1'`, som anländer medan TX klar är satt, gör exakt två saker på den klockflanken: höjer `tx_req` **under en cykel**, och nollställer TX klar. Mer precist: `tx_req` är hög under den enda cykeln *efter* skrivningens klockflank, och låg igen på nästa — testbänken kontrollerar båda ändarna av det påståendet.

Allt annat med det här registret är vad det *ignorerar*:

- En skrivning medan TX klar är låg (en sändning pågår) kastas helt — ingen puls, ingen tillståndändring. Att skriva över `tx_id`, `tx_dlc` eller `tx_data` mitt i en sändning är den korruption som drivrutinens egen klar-kontroll skyddar mot från mjukvarusidan; det här är samma skydd på hårdvarusidan, och ingendera sidan får anta att den andra finns där.
- En skrivning med `reg_wdata(0) = '0'` gör ingenting: registerkartan säger "skriv 0x1 för att utlösa", och banken håller den till det.
- Läsningar ger noll — TX_SEND lagrar ingenting, det *gör* något.

Varför pulsdisciplinen spelar roll syns tydligast i felet den förhindrar. `can_controller` agerar på `tx_req` så fort den är hög och bussen är ledig (kapitel 16), så en `tx_req` som lämnas *hållen* begär på nytt i samma ögonblick som en ram tar slut, för alltid, och suddar error vid varje passage tillbaka genom starttillståndet. Begäran måste vara en händelse, och något måste göra den till en. På det här lagret är det enklare — en skrivstrob är redan en händelse, inte en nivå — men det måste ändå *upprätthållas*: en skrivning, en puls, oavsett vad mastern gör härnäst.

18.5 TX_ABORT: nödutgången

Index 12, bara skrivbart, och spegelbilden av TX_SEND: en skrivning av 0x1 sätter TX klar tillbaka till '1'. En skrivning av något annat gör ingenting, och en läsning ger 0x00000000.

Det rör ingenting annat. Det når inte `can_controller` — kontrollern har ingen ingång för att avbryta, dess femton portar spikades i kapitel 11 och `can_controller_tb` binder dem positionellt — och den behöver ingen. En controller som avbrutit en sändning är redan

tillbaka i STATE_IDLE och fullt kapabel att sända igen; det enda som sitter fast är den här bankens klabbiga bit. TX_ABORT lossar den, och det är hela modulen.

Det är värt att stanna vid ett ögonblick, eftersom det är buggens allmänna form. Ingenting i *hårdvaran* hade kört fast. Det som kört fast var en **statusbit vars enda sättvillkor hängde på en händelse som inte längre inträffade**. Varje klabbig flagga med en väg upp och en väg ner är en missad händelse från samma fel, och lösningen är alltid densamma: se till att varje väg ut ur upptagetillståndet också höjer flaggan, och ge anroparen ett sätt att tvinga fram den ifall du har fel om ”varje”.

18.6 RX-sidan: infångning vid pulsen

När rx_valid pulsar gör banken två saker på samma flank: sätter STATUS bit 1, och **fångar in** rx_id, rx_dlc och rx_data i de fyra RX_*-registren — rx_datas övre ord (bitarna 63–32, databyte 0–3, byte 0 i den mest signifikanta positionen) i RX_DATA_LO, dess undre ord i RX_DATA_HI.

Att fånga in, i stället för att koppla kontrollerns portar rakt till läsmuxen, är det som ger drivrutinen en stabil ram att läsa: RX_*-registren håller sina värden medan drivrutinen arbetar sig igenom fyra separata SPI-lästransaktioner, oavsett vad kontrollerns portar gör under tiden. En skrivning till RX_ACK nollställer *bara flaggan* — dataregistren behåller sitt innehåll till nästa infångning, vilket är ofarligt: registerkartans kontrakt är att RX_* är meningsfullt *medan RX giltig är satt*.

Vad infångningen **inte** ger drivrutinen är buffring. En ny ram som fullbordas innan den förra kvitterats fångas helt enkelt in ovanpå den — en ram hålls, den nyaste vinner. Det är inte den här modulen som är lat; det är en dokumenterad lucka som ärvts från kontrollern själv, som håller exakt en mottagen ram och skriver över den vid nästa (kapitel 11). Banken gör den luckan synlig på registernivå i stället för att dölja den, och kapitel 19 återkommer till vad den kostar en drivrutin.

18.7 Maskning vid skrivning

Skrivbara register lagrar bara de bitar som finns i *hårdvaran*, och läses tillbaka som nollor ovanför dem:

Register	Lagrade bitar	En skrivning av 0xFFFFFFFF läses tillbaka som
TX_ID	10–0 (id_t)	0x000007FF
TX_DLC	3–0 (dlc_t)	0x0000000F
TX_DATA_LO/HI	alla 32	0xFFFFFFFF

Lägg märke till vad banken *inte* gör: validerar. En TX_DLC på 0xF lagras och lämnas vidare till kontrollern som den är — att avvisa dlc > 8 är drivrutinens ansvar, i den parallella mjukvaruklassen, och att göra det två gånger på två språk är precis så de två kopiorna glider isär. Banken maskar till de bitar som fysiskt finns; drivrutinen upprätthåller betydelsen.

STATUS och RX_*-registren ignorerar skrivningar helt, och de reserverade indexen 13–15 läses som noll och ignorerar skrivningar, precis som protokollspecifikationen kräver.

Utgångarna mot kontrollern kommer direkt från TX-registren: `tx_data <= TX_DATA_LO & TX_DATA_HI` — 64 bitar med databyte 0 i den mest signifikanta positionen, den layout `can_controller` väntar sig (kapitel 16) och samma packning som drivrutinen utför på andra sidan kartan (se diagrammet över databyteordningen i bilaga D).

18.8 Att skriva `register_bank.vhd`

Filen hamnar i `bridge/`, bredvid dina kopierade kontrollermoduler. Formen är två processer och en handfull konkurrenta tilldelningar:

- **En klockad process** som äger varje register och läsning: resetvärdena (TX klar '1', allt annat noll), skriv-case:et över `reg_index`, och sedan de tre uppdateringarna från kontrollerns händelser, i den ordningen.
- **En kombinatorisk process** för läsmuxen: ge `reg_rdata` förvalet idel nollor, och skriv sedan över de bitar varje register faktiskt har — samma stil med "tilldela ett förval, skriv över i specifika grenar" som varje räknare och tillståndsmaskin från kapitel 6 och framåt.
- Konkurrenta tilldelningar som kopplar ut `tx_id`, `tx_dlc`, `tx_data` och `tx_req` från TX-registren och pulsvippan.

Analysera den och kör sedan den utdelade testbänken. `can_def.vhd` ligger en katalog bort, i `controller/`: banken läser `can_def` för sina typer mot kontrollern men hör hemma på SPI-sidan av protokollgränsen.

```
cd bridge
ghdl -a --std=93 ../controller/can_def.vhd register_bank.vhd register_bank_tb.vhd
ghdl -e --std=93 register_bank_tb
ghdl -r --std=93 register_bank_tb --assert-level=error
```

Vad testbänken kontrollerar. `register_bank_tb.vhd` går igenom kontraktet i elva fall, i tur och ordning: reset-tillståndet (bit 0 satt, allt annat noll); skrivning och återläsning med maskning på varje skrivbart register, och utgångarna mot kontrollern som följer dem; TX_SENDS encykelspuls och nollställningen av klar; skrivningen som kastas medan sändning pågår; `tx_done` som återställer klar; en TX_SEND-skrivning av 0x0 som inte gör något; infångning av mottagen ram, att den håller trots att ingångarna ändras, att den är skrivskyddad, och RX_ACK; fellåsningens flanksemantik, inklusive regeln om skrivning av noll och en nollställning medan nivån fortfarande är hög; de reserverade indexen; och sist de två återhämtningsvägar som ger tillbaka TX klar efter en förlorad arbitrerings, en automatisk och en driven av TX_ABORT.

Varje fallkommentar i filen säger vad ett underkänt fall betyder, så läs testbänken innan du skriver modulen — den är den körbara formen av det här kapitlet. De två sista fallen är särskilt värda att läsa i förväg: utan dem fastnar noden efter sin första förlorade arbitrerings, och eftersom en tvånodsbuss provocerar det i normal drift är det inte ett hörnfall.

18.9 SPI från vågformen och uppåt

Registerbanken presenterar register. Något måste nå dem utifrån, och det är SPI. Det här avsnittet tar transporten från vågformen och uppåt, och slutar i den utdelade `spi_slave`, som du läser snarare än skriver.

Läge 0, MSB först, SS som ramning

Projektet kör **SPI läge 0** (CPOL = 0, CPHA = 0) rakt igenom:

- SCK vilar lågt. Mastern genererar den; slaven producerar aldrig en klocka.
- Båda sidor **samplear på den stigande flanken**. MISO ändras därför på den **fallande** flanken, så att värdet hunnit stabilisera sig till nästa stigande.
- **MSB först**, i varje byte.
- SS är aktiv låg och är **ramningssignalen**: en transaktion per låg period. Mastern måste höja SS mellan transaktioner.
- SCK går i högst 1 MHz. Skälet står i avsnitt 18.9.1.

Rita vågformen för hand innan du läser vidare: SS som faller, åtta stigande SCK-flanker med byten 0xA5 på MOSI, mest signifikant först, och SS som stiger. Det är hela bilden, och Övning 18.4 bygger en femtransaktionsbyte av den.

18.9.1 Varför slaven inte klockar på SCK

Den frestande läsningen är att SCK är en klocka, så gör den till *din* klocka: klocka skiftregistret på den och var klar. Låt bli, av två skilda skäl.

Det första är metastabilitet, och det är kapitel 4:s regel oförändrad. SCK, MOSI och SS kommer alla in på pinnar som drivs av en mikrokontroller med sin egen kristall. De är asynkrona mot `clock` i precis den mening kapitel 4 avser, och varje en av dem måste genom två vippor innan någon logik får titta på den.

Det andra är vad en design med två orelaterade klockor kostar i en FPGA. En andra klockdomän betyder en andra uppsättning tidsvillkor, ett verkligt klockdomänkors på varje signal som passerar mellan dem, och ett syntesverktyg som inte längre kan analysera designen som en helhet. För ett gränssnitt som går i 1 MHz mot en väv som går i 50 är det priset onödigt.

Svaret är **översampling**: synkronisera in SCK i 50 MHz-domänen och *detektera dess flanker* i stället för att klocka på den. Vid 1 MHz är det minst 50 systemcykler per SPI-bit, alltså en storleksordnings marginal. Protokollet garanterar därför korrekt funktion upp till 1 MHz; snabbare kan fungera, och lovas medvetet inte.

Det kostar några 50 MHz-cykler per flank, och det är därifrån protokollets tidskrav kring SS kommer: minst 60 ns (tre cykler) före den första SCK-flanken, mellan transaktioner, och efter den sista flanken. Bilaga D har hela tabellen.

18.9.2 Att läsa spi_slave.vhd

spi_slave delas ut i stället för att skrivas: den är transport, inte bokens ämne. Men du måste kunna läsa den, och kapitel 20 förväntar sig att du kan det.

```
bridge/spi_slave.vhd
-----
-- Byte-level SPI slave.
--
-- Inputs:
--   - clock      : 50 MHz system clock.
--   - reset_s2_n: Asynchronous active-low reset.
--   - sclk       : SPI clock.
--   - mosi       : Master Out Slave In.
--   - ss         : SPI Chip Select.
--   - tx_data    : Next byte to shift output.
-- Outputs:
--   - miso       : Master In Slave Out.
--   - rx_data    : Incoming byte.
--   - rx_valid   : '1' if incoming byte is fresh.
--   - ss_active  : '1' while selected (ss low).
-----

library ieee;
use ieee.std_logic_1164.all;

entity spi_slave is
    port(clock, reset_s2_n : in std_logic;
         sclk, mosi, ss    : in std_logic;
         tx_data          : in std_logic_vector(7 downto 0);
         miso             : out std_logic;
         rx_data          : out std_logic_vector(7 downto 0);
         rx_valid, ss_active: out std_logic);
end entity;

architecture behaviour of spi_slave is

    -- Two synchronizer stages, plus a third flop holding last cycle's s2 so an edge ↵
    -- ↵ can be detected
    -- by comparing the two. sclk and ss are used for their edges and need all three; ↵
    -- ↵ mosi is only
    -- ever read at an edge detected on sclk, so it needs the two synchronizer stages ↵
    -- ↵ and no history.
    type sync_t is record
        s1: std_logic; -- First synchronizer stage; may go metastable.
```

```

    s2: std_logic; -- Second synchronizer stage: the current, safe-to-use value.
    s3: std_logic; -- Previous cycle's s2, for edge detection.
end record;

-- Synchronized SPI signals.
signal sclk_s, ss_s      : sync_t;
signal mosi_s1, mosi_s2 : std_logic;

-- Bit counter.
signal bit_counter: natural range 0 to 7;

-- Shift registers.
signal rx_sh, tx_sh: std_logic_vector(7 downto 0);

begin
    ss_active <= not ss_s.s2;
    miso      <= tx_sh(7) when ss_s.s2 = '0' else '0';

    -- Synchronize SPI inputs.
    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            -- Each chain resets to the level its line idles at, so the module ↵
            ↵ comes out of reset
            -- believing what is actually true of an idle bus. sclk idles low in ↵
            ↵ SPI mode 0; ss
            -- is active low and idles HIGH, so resetting its chain to '0' would ↵
            ↵ make ss_active
            -- read '1' out of reset and the bridge see a frame nobody started. ↵
            ↵ This is the same
            -- rule L04's button_sync exercise exists to enforce.
            sclk_s <= ('0', '0', '0');
            ss_s   <= ('1', '1', '1');
            mosi_s1 <= '0';
            mosi_s2 <= '0';
        elsif (rising_edge(clock)) then
            sclk_s.s1 <= sclk;
            sclk_s.s2 <= sclk_s.s1;
            sclk_s.s3 <= sclk_s.s2;
            ss_s.s1   <= ss;
            ss_s.s2   <= ss_s.s1;
            ss_s.s3   <= ss_s.s2;
            mosi_s1   <= mosi;
            mosi_s2   <= mosi_s1;
        end if;
    end process;
end

```

```

    end if;
end process;

-- Run SPI slave logic.
process(clock, reset_s2_n) is
variable sclk_rising, sclk_falling, ss_falling: std_logic := '0';
begin
    if (reset_s2_n = '0') then
        bit_counter <= 0;
        rx_sh        <= (others => '0');
        tx_sh        <= (others => '0');
        rx_data      <= (others => '0');
        rx_valid     <= '0';
    elsif (rising_edge(clock)) then
        rx_valid <= '0';

        -- Detect edges on sclk and ss.
        sclk_rising := sclk_s.s2 and (not sclk_s.s3);
        sclk_falling := (not sclk_s.s2) and (sclk_s.s3);
        ss_falling  := (not ss_s.s2) and ss_s.s3;

        -- Deselected: hold the bit counter at zero and ignore the SPI lines ↵
        ↵ entirely. SCK
        -- belongs to whichever slave the master is addressing, and a line ↵
        ↵ that is merely
        -- ringing is not data. Without this gate the branches below would ↵
        ↵ shift MOSI and
        -- pulse rx_valid on edges arriving while ss is high, and ↵
        ↵ spi_reg_bridge would lose
        -- byte alignment for every transaction afterwards.
        if (ss_s.s2 = '1') then
            bit_counter <= 0;

            -- ss just fell: a transaction is starting. Load the first byte to ↵
            ↵ send and start the
            -- frame at bit 0. This arm comes before the SCK arms rather than ↵
            ↵ beside them, so a
            -- rising SCK edge landing on this same system-clock cycle cannot ↵
            ↵ advance the counter
            -- past the start of the frame and misalign every byte in it.
            elsif (ss_falling = '1') then
                bit_counter <= 0;
                tx_sh        <= tx_data;

```

```

else
    -- Sample MOSI, MSB first on rising SCLK edge.
    if (sclk_rising = '1') then
        if (bit_counter = 7) then
            rx_data    <= rx_sh(6 downto 0) & mosi_s2;
            rx_valid   <= '1';
            bit_counter <= 0;
        else
            rx_sh      <= rx_sh(6 downto 0) & mosi_s2;
            bit_counter <= bit_counter + 1;
        end if;
    end if;

    -- Present the next MISO bit on falling edge.
    if (sclk_falling = '1') then
        if (bit_counter = 0) then
            tx_sh <= tx_data;
        else
            tx_sh <= tx_sh(6 downto 0) & '0';
        end if;
    end if;
end if;
end process;
end architecture;

```

Fyra saker är värda att läsa noga:

Posten `sync_t` har tre fält, men bara två är synkroniseringssteg. `s1` och `s2` är dubbelvipp-synkroniseraren; `s3` är förra cykelns `s2`, och finns bara för att en flank ska gå att detektera genom att jämföra de två. `sclk` och `ss` används för sina *flanker* och behöver alla tre; `mosi` läses bara vid en flank som redan detekterats på `sclk`, så den behöver de två synkroniseringsstegen och ingen historik.

De två kedjorna resettar till olika värden. `sclk_s` nollställs till idel nollor och `ss_s` till idel ettor. Varje kedja resettar till den nivå dess ledning vilar på, så att modulen kommer ur reset och tror på det som faktiskt är sant om en ledig buss: `sclk` vilar lågt i läge 0, och `ss` är aktiv låg och vilar högt. Att nollställa `ss`-kedjan till '0' skulle få `ss_active` att läsa '1' ur reset och bryggan att se en ram som ingen startat. Det är exakt samma regel som `button_sync` i Övning 4.9 finns till för att upprätthålla.

En avvald slav ignorerar SPI-ledningarna helt. Hanteringen av SCK-flanker ligger inuti `else`-armen i ett test på `ss_s.s2`. SCK tillhör den slav mastern adresserar; för den här slaven är den en asynkron ingång som kommer in över en ledning på ett kopplingsdäck, och en störning på den ledningen ser likadan ut som en flank. Utan den grindningen skulle grenarna

nedanför skifta in MOSI och pulsa `rx_valid` på flanker som anländer medan `ss` är hög, och `spi_reg_bridge` skulle tappa bytejusteringen för varje transaktion därefter.

`ss_falling` är ett `elsif` före SCK-grenarna. Ordningen spelar roll vid exakt en klockflank: den där `SS` nyss föll *och* en stigande SCK-flank detekteras samtidigt. Genom att `ss_falling`-armen kommer först kan den SCK-flanken inte stega biträknaren förbi ramens början och kasta om varje byte i den.

`spi_slave` lämnar över en färdig byte i taget på `rx_data`, med `rx_valid` som en puls, och tar nästa byte att sända på `tx_data`. Den vet ingenting om register. Kapitel 19 bygger `spi_reg_bridge`, lagret som gör de byteströmmarna till femtransaktionsbytes och till registeråtkomster mot banken du just skrivit.

18.10 Sammanfattning

Det här ska du kunna nu

- Förklara skillnaden mellan en puls och en klibbig nivå, och varför registerlagret måste konvertera mellan dem.
- Implementera `register_bank.vhd` enligt registerkartan: STATUS-bitarnas sättning och nollställning, `TX_SEND` som encykelspuls, och maskningen vid skrivning.
- Läs ett SPI-tidsdiagram i läge 0 och säg vilken bit som ligger på MOSI vid varje flank.
- Förklara varför en SPI-slav i en FPGA översamplar SCK i stället för att använda den som klocka.
- Läs och använd `spi_slave.vhd` utan att skriva den.

Frågor att testa dig själv med

1. Varför kan en drivrutin som pollar var femte mikrosekund inte se en puls på 20 ns?
2. Vad sätter STATUS bit 0, och vad nollställer den?
3. Varför låser felbiten på en *stigande flank* av `error` och inte på nivån?
4. En andra ram tas emot innan den första kvitterats. Vad händer, och varför är det ett medvetet val snarare än en bugg?
5. STATUS bit 0 sätts av tre saker under drift, utöver reset. Vilka, och varför räcker det inte med `tx_done` ensamt?
6. Vad gör `TX_ABORT`, och vad når det *inte*?
7. En skrivning av `0xFFFFFFFF` till `TX_DLC` läses tillbaka som `0x0000000F`. Var sker maskningen, och varför validerar banken inte att värdet är högst 8?
8. Varför använder `spi_slave` inte SCK som klocka? Vad skulle gå fel om den gjorde det?
9. Varför byter MISO värde på SCK:s fallande flank?

Riktvärde

`register_bank` är en avgränsad modul med en fullständig testbänk och bör gå att bli klar med under passet och veckan efter. Den beror inte på SPI alls, så den kan skrivas parallellt med att någon annan i gruppen läser in sig på transaktionsformatet. Lägg in den utdelade `spi_slave.vhd` i repot oförändrad, och läs igenom `spi_reg_bridge_tb.vhd` inför nästa kapitel: den är kontraktet för modulen du skriver då.

Blicken framåt

Kapitel 19 bygger SPI-bryggan, toppnivån `can_spi_node`, och tar hela kedjan ut på riktig hårdvara. Det är projektets sista pass.

18.11 Övningar

Skriv din egen `register_bank.vhd` utifrån avsnitt 18.2 till 18.8; den utdelade testbänken, körd enligt bilaga A, är kontrollen av att den beter sig korrekt. Övning 18.4 och 18.5 handlar om `spi_slave.vhd`, som delas ut i stället för att skrivas — den läser du, och kapitel 20 förväntar sig att du kan den.

Den auktoritativa källan för varje registernummer nedan är bilaga D. Där en övning och en specifikation är oense vinner specifikationen, och det är övningen som är buggen.

Övning 18.1 `register_bank`

VHDL

Skriv din egen `register_bank.vhd` utifrån kapitlet. Verifiera den:

```
cd bridge
ghdl -a --std=93 ../controller/can_def.vhd register_bank.vhd register_bank_tb.vhd
ghdl -e --std=93 register_bank_tb
ghdl -r --std=93 register_bank_tb --assert-level=error
```

Om en kontroll fallerar, läs assertion-meddelandet; vart och ett av de elva fallen säger vad ett haveri i just det fallet betyder.

Övning 18.2 De tre STATUS-bitarna

RESONEMANG

- a) Ur reset läser STATUS `0x00000001`. Säg vilken bit det är och varför resetvärdet är '1' och inte '0'. Beskriv sedan vad en drivrutins `send()` skulle göra, i all evighet, om det vore '0'.
- b) Bit 2 läser en **stigande flank** hos kontrollerns error, inte dess nivå. Anta att den i stället läste nivån. Skriv ner den följd av drivrutinsoperationer — `polla`, `nollställ`, `polla` — som då skulle bete sig annorlunda, och säg vad drivrutinens författare skulle dra för slutsats om vad som var trasigt.
- c) Bit 1 sätts av `rx_valid` och nollställs av en skrivning till `RX_ACK`. Bit 0:s huvudsakliga sättvillkor är `tx_done`, och den nollställs av en skrivning till `TX_SEND`. Det ena av de paren har sina villkor för att sätta och nollställa i den "naturliga" ordningen, och det andra är omvänt. Säg vilket som är vilket, och varför det omvända ändå är rätt.
- d) Avsnitt 18.3 ger en ordningsregel: kontrollerhändelser tillämpas **efter** bryggans skrivningar i den klockade processen. Konstruera den enda klockflank där den regeln ändrar utfallet, namnge båda händelserna, och säg vilken av de två möjliga lösningarna som kan låsa systemet och vilken som inte kan det.
- e) Bit 0 har *tre* sättvillkor, och ett av dem är en stigande flank hos `error` medan bit 0 redan är låg. Förklara varför det villkoret inte behöver veta något om huruvida kontrollern sände eller tog emot — vad är det i en nods beteende som gör att "error steg medan vi var upptagna" betyder "en sändning tog just slut"?
- f) Ta bort det villkoret ur din bank och kör om testbänken. Vilket fall fallerar, och vilken konsekvens säger dess meddelande att det skulle ha fått? Förklara sedan varför det här är skydd i flera lager snarare än dubbelarbete, givet att kapitel 17 redan låter kontrollern pulsa `tx_done` vid en avbruten sändning. (Vem skriver `can_controller`? Vem skrev testbänken som skulle fånga det?)
- g) `TX_ABORT` är den tredje vägen, och till skillnad från de andra två motsvarar den inte något hårdvaran gjorde — det är drivrutinen som går förbi banken. Säg vad den *inte* gör: namnge minst två saker du kanske skulle vänta dig att ett register som heter `TX_ABORT` når fram till, och förklara varför den inte når någondera. (Ledtråden är `can_controllers` portlista, som fastställdes i kapitel 11.)

Övning 18.3 TX_SEND och pulsdisciplinen

SIMULERING

- a) En skrivning av 0x1 till TX_SEND höjer tx_req under exakt en cykel. Säg exakt vilken cykel, i förhållande till den klockflank som bar reg_write, och vilka två saker register_bank_tb kontrollerar om den.
- b) Gör nu sönder det med flit. Ändra din TX_SEND-gren så att tx_req blir en **nivå**: sätt den vid en accepterad skrivning och nollställ den först när tx_done kommer. Kör om register_bank_tb. Vilken kontroll fallerar först, och vad säger dess meddelande?
- c) Din trasiga version från b) skulle fortfarande *se* korrekt ut på ett oscilloskop som tittar på en enda ram. Beskriv, med avsnitt 16.1:s regel för när can_controller agerar på tx_req, vad den gör vid den andra ramen och varje ram därefter. Säg sedan varför det här är en bugg som simulering fångar lätt och ett test på bänken inte gör.
- d) En skrivning till TX_SEND medan STATUS bit 0 är låg kastas helt — ingen puls, ingen tillståndsändring. En annan design skulle i stället kunna köa den och avfyra när kontrollern rapporterar redo. Ge ett konkret argument för regeln att kasta den, givet att drivrutinen kan polla STATUS. Ställ tillbaka modulen som den var.

Övning 18.4 Att läsa av en SPI-transaktion ur vågformen

RESONEMANG

Det här kapitlet heter "från vågformen" av ett skäl: kapitel 20 kan mycket väl räcka dig en och fråga vad den betyder. Läge 0 rakt igenom — SCK vilar lågt, MOSI samplas på **stigande** flank, MISO ändras på **fallande** flank, MSB först.

- a) Rita för hand hela fembytestransaktionen som skriver 0x00000001 till TX_SEND. Räkna ut kommandobyten ur formatet i bilaga D i stället för att kopiera den: bit 7 är skrivbiten, bit 6–4 är reserverade, bit 3–0 är registerindexet, och TX_SEND är index 5. Märk upp SS, SCK och MOSI, och markera de 40 stigande flankerna i grupper om åtta.
- b) Markera på samma ritning var mastern inte får släppa SS, och vad slaven måste göra om den ändå gör det. Återge regeln ur protokollspecifikationens avsnitt om avbrott med egna ord.
- c) En läsning av STATUS är spegelbilden: kommandobyte 0x00, fyra dummybytes ut, fyra bytes registervärde tillbaka på MISO. Protokollspecifikationen säger att slaven läser registervärdet **en gång**, vid slutet av kommandobyten. Förklara vad en drivrutin skulle kunna observera om slaven i stället läste registret på nytt för var och en av de fyra bytesen, och varför just STATUS är det register där det skulle göra ont.
- d) Räkna SCK-flankerna i en hel transaktion och räkna, vid det maximum på 1 MHz som protokollspecifikationen tillåter, ut hur lång tid en transaktion tar. Jämför det med den 20 ns långa puls som registerbanken finns till för att översätta. Det förhållandet är hela argumentet för det här lagret, i ett enda tal.

Övning 18.5 Att läsa `spi_slave.vhd`

RESONEMANG

Du skriver inte den här modulen, men du måste kunna läsa den.

- a) `spi_slave` använder inte SCK som klocka. Den synkroniserar in den i 50 MHz-domänen och detekterar dess flanker i stället. Ge två skilda skäl, ett om metastabilitet (kapitel 4) och ett om vad en design med två orelaterade klockor kostar i en FPGA.
- b) Posten `sync_t` har tre fält, men bara två av dem är synkroniseringssteg. Säg vad det tredje är till för, och varför `mosi` behöver två fält där `sclk` och `ss` behöver tre.
- c) I resetgrenen nollställs `sclk_s` till idel nollor och `ss_s` till idel **ettor**. Förklara varför de skiljer sig, och följ sedan vad `ss_active` skulle läsa under de två första klockflankerna efter reset om `ss_s` nollställdes till nollor utan någon master inkopplad. Det här är samma regel som Övning 4.9 byggdes kring; säg vilken kontroll i den övningens testbänk som motsvarar den.
- d) Hanteringen av SCK-flanker ligger inuti `else`-armen i ett test på `ss_s.s2`, så en avvald slav ignorerar SPI-ledningarna helt. Konstruera felet som skulle följa om den inte gjorde det: utgå från en vilken SCK-flank medan `SS` är hög, och följ den fram till vad *nästa* registerskrivning hamnar på.
- e) Grenen `ss_falling` är ett `elsif` före SCK-grenarna i stället för ett separat `if` bredvid dem. Säg vid vilken enda klockflank den ordningen spelar roll, och vad som skulle vara fel med de mottagna bytesen om de två grenarna kördes i omvänd ordning.

Övning 18.6 Infångningen, och gapet den inte täpper till

RESONEMANG

- a) När `rx_valid` pulsar fångar banken in `rx_id`, `rx_dlc` och `rx_data` i `RX_*`-registren i stället för att koppla kontrollernas portar rakt in i läsmultiplexern. Förklara vad en drivrutin som läser fyra separata SPI-transaktioner skulle se utan den infångningen.
- b) En skrivning till `RX_ACK` nollställer `STATUS` bit 1 men lämnar `RX_*`-registren med sitt innehåll kvar. Säg varför det är ofarligt, och citera meningen i registerkartan som gör det så.
- c) En andra ram anländer innan den första har kvitterats. Säg exakt vad drivrutinen observerar, och varför det här är ett dokumenterat gap som ärvts från `can_controller` (kapitel 11) snarare än en brist hos registerbanken.
- d) Konstruera, enbart på papper, den minsta ändring i *registerkartan* som skulle låta en drivrutin upptäcka att den missat en ram. Du lägger inte till någon mottagningskö; du lägger till möjligheten att veta att en behövdes. Säg vad det kostar i hårdvara, och i vilket befintligt register det skulle kunna bo utan att bryta det kontrakt parallellklassen redan skriver mot.

SPI-bryggan, `can_spi_node` och bring-up

Projektets sista pass, och dess capstone: transaktionslagret, toppnivån som binder ihop allt, och hela kedjan verifierad först i simulering och sedan på riktig hårdvara.

I DET HÄR KAPITLET

- Systemverifiering: två noder på en wired-AND-buss, och vad den testbänken inte täcker.
- Transaktions-FSM:en: kommandobyte, fyra databytes, läsning en gång vid läsning, verkställe på femte byten, och avbrott när SS går hög för tidigt.
- `can_spi_node`: att komponera `spi_slave`, `spi_reg_bridge`, `register_bank` och `can_controller` till en nod.
- Bring-up i två halvor: två kort på en delad busslinje, sedan en mikrokontroller som SPI-master mot dem, i åtta steg.
- Vad riktiga CAN-kontrollrar gör som den här designen inte gör.

`can_controller` är färdig från och med kapitel 17: sändning, mottagning och arbitrering. Den instansierar `bit_timer`, `crc15` och båda skiftregistren inuti sig själv, så utifrån sett är hela kontrollern en enda komponent med det 15-portars gränssnitt på registernivå som kapitel 11 förhandsvisade.

Sedan går två lager framför den: `register_bank` (kapitel 18) gör om det gränssnittet till registerkartan, och `spi_reg_bridge` plus den utdelade `spi_slave` gör om registerkartan till en fembytes SPI-transaktion. Det parallellklassen skriver sin drivrutin mot är den bortre änden av den kedjan — registerkartan och protokollspecifikationen i bilaga D, aldrig `can_controllers` portar.

Det här är bokens tightaste kapitel, och det är värt att säga rakt ut: två moduler skrivs, hela kedjan ska simuleras, hårdvaran ska upp, och varje grupp redovisar.

Ligg därför före. `register_bank` och `spi_reg_bridge` beror inte på `can_controller` alls — den förra behöver bara `can_def`, den senare bara `spi_def` — så båda går att skriva parallellt med

kontrollern, från kapitel 10 och framåt. En grupp som kommer hit med registerbanken klar och transaktionsformatet läst har passat till att koppla ihop och felsöka. En grupp som börjar här hinner inte allt.

19.1 Att verifiera hela systemet

Innan någon del av transporten går att lita på måste kontrollern visas fungera som ett *system*. Det betyder inte en modul mot en attrapp, utan två oberoende noder på en ledning som bråkar om den så som riktiga noder gör.

Att modellera bussen

Ingenting i konstruktionen modellerar den delade bussen, och det är medvetet. `can_controller` driver ett par (`tx_bus`, `bus_en`) och läser `rx_bus`, vilket är precis vad en riktig CAN-transceiver presenterar på sin digitala sida (kapitel 10). Wired-AND:et bor i testbänken, och på hårdvara bor det i transceivern.

`can_controller_tb.vhd` instansierar två noder på en ledning och kombinerar dem direkt. Varje nods effektiva utgång är dess drivna värde när `bus_en = '1'`, annars släppt ('1'), och bussen är deras AND, så den läser dominant i samma ögonblick som *någon* av noderna drar låg:

```
bus_line <= (a_tx_bus or not a_bus_en) and (b_tx_bus or not b_bus_en);
```

Den enda raden är hela avsnitt 10.2:s ”vilken dominant som helst slår alla recessiva”, och det är den som gör arbitreringen testbar i simulering.

Värd att räkna igenom snarare än att läsa. Varje nod bidrar med `tx_bus or not bus_en`, som är 1 så fort den släppt bussen, och ledningen är de två bidragen AND:ade:

Nod A	Nod B	a_tx_bus	a_bus_en	b_tx_bus	b_bus_en	bus_line
släppt	släppt	-	0	-	0	1 recessiv
driver dominant	släppt	0	1	-	0	0 dominant
släppt	driver dominant	-	0	0	1	0 dominant
driver dominant	driver dominant	0	1	0	1	0 dominant

En släppt nods `tx_bus` är - därför att den faktiskt inte spelar någon roll: `not bus_en` ensamt gör dess bidrag till 1, så en nod som släppt taget kan inte påverka ledningen oavsett vad dess utgång råkar ligga på. Över alla sexton ingångskombinationer läser ledningen dominant exakt när minst en nod har `bus_en = '1'` med `tx_bus = '0'`, vilket är regeln uttryckt som hårdvara.

Två rader bär arbitreringsargumentet. **Rad två är själva tvekampen:** A driver dominant, B sänder recessivt och har därför släppt, och ledningen läser dominant. B, som övervakar

medan den sänder, ser att den sänt recessivt och att bussen läser dominant, och det är precis det förlustvillkor kapitel 17 kontrollerar. Samtidigt läser A tillbaka exakt den bit den sände och märker ingenting.

Rad ett är varför förloraren kan gå. När B slutat driva bidrar den med 1 för alltid, så ledningen är A:s ensam. Förlorarens reträtt är osynlig för vinnaren, och det är det som gör en kollision till en icke-händelse i stället för något båda noderna måste återhämta sig från.

Rad fyra är värd en blick också: när båda noderna driver dominant är ledningen dominant och var och en läser tillbaka det den sände, så ingen av dem kan märka att den andra finns. Det är det normala tillståndet för varje dominant bit i identifieraren innan de två ID:na skiljer sig åt.

De två scenarierna

1. **Mottagning.** Nod A sänder en ram på 5 bytes (ID 0x123, data 11 22 F8 44 55) medan nod B tar emot den. 0xF8 framtvingar en stoppbit inne i datafältet, så båda sidors CRC-motorer måste hoppa över den. Testbänken kontrollerar A:s tx_done och B:s rx_valid, att B rekonstruerade ID:t, DLC:n och **varje** databyte på sin fasta position, och att ingen av noderna höjde error.
2. **Arbitrering.** Noderna A (ID 0x000) och B (ID 0x001) begär båda under samma cykel, och skiljer sig bara i identifierarens sista bit. A måste vinna och fullborda normalt, ovetande om att någon tvekamp ägt rum; B måste upptäcka förlusten vid den biten, höja error och sluta driva. Kontrollen körs efter att A:s ram är klar, så den bekräftar också att B:s error fortfarande går att läsa då, i stället för att ha suddats av ett återinträde i STATE_START.

En sändande nod tar aldrig emot sin egen ram (rx_shift_reg går bara medan role = '0'), så båda scenarierna behöver verkligen två noder; inget av dem skulle gå att testa med en ensam nod.

De två encykelpulser det första scenariot väntar på, A:s tx_done och B:s rx_valid, **läses** i testbänken, eftersom två oberoende tillståndsmaskiner inte behöver pulsa på samma flank, och ett rakt wait until (a = '1' or b = '1') skulle återuppta på den första och missa den andra.

Vad testbänken inte täcker

Två godkända scenarier är inte samma sak som en bevisad konstruktion, och att veta var en testbänk slutar hör till att läsa den. Fem luckor är värda att namnge, eftersom var och en är ett ställe där den här konstruktionen antingen är förenklad med flit eller helt enkelt otestad:

- **Båda noderna delar en clock och en reset_n.** Riktiga noder går på oberoende oscillatorer och driver isär. Var precis med vad det lämnar otestat, för det är inte resync självt: båda noderna pulsar resync vid varje rams början (kapitel 16), och den pulsen är det som fasrikter B:s sampelpunkter mot det godtyckliga ögonblick då A började sända, så fall 1 misslyckas utan den. Vad en delad klocka aldrig kan pröva är **drift**, två timrar som

startar i fas och vandrar isär mitt i ramen, fallet som Övning 12.4 kvantifierar och skälet till att riktig CAN synkroniserar om vid varje flank från recessiv till dominant i stället för en gång per ram. Det här är den största luckan, och att sluta den i simulering kräver en andra klockgenerator med en avsiktlig frekvensavvikelse. Avsnitt 19.6 får en ram tvärs över **två kort** med två separata kristaller, äntligen genuint oberoende klockor, med ett ärligt förbehåll noterat där.

- **Ingen ram förvanskas någonsin.** Ingenting matar in ett stoppfel, vänder en bit på ledningen eller skadar en CRC, så mottagarsidans error-vägar provas bara på modulnivå av `rx_shift_reg_tb` och `crc15_tb`, aldrig hela vägen igenom.
- **ACK-luckan drivs men läses aldrig tillbaka.** En mottagare med giltig CRC drar den dominant (kapitel 17), men sändaren kontrollerar den aldrig, så en ram som ingen kvitterat fullbordas precis som en som blev kvitterad. Det är en medveten förenkling (avsnitt 19.7), och testbänken ärver den i stället för att fånga den.
- **Arbitreringen är alltid tvåvägs, och förloras alltid vid samma bit.** `0x000` mot `0x001` skiljer sig i identifierarens sista bit, så förlusten upptäcks i samma tillstånd varje körning. Övning 19.1 lägger till en tredje nod och flyttar den förlorande biten tidigare.
- **Ingen nod sänder någonsin igen efter ett avbrott.** Nod B förlorar arbitreringen i fall 2 och blir aldrig ombedd att skicka en ram till, så ingenting här kontrollerar att en nod över huvud taget återhämtar sig från ett avbrott. Det spelar större roll än det låter: `can_controller` nollställer `crc15` i `STATE_START` (kapitel 16) just för att en övergiven ram inte ska kunna lämna sina rester efter sig, och den här testbänken skulle passera precis lika glatt med den nollställningen borttagen. Övning 19.7 är där du får reda på vad den gjorde.

19.2 Transaktionsbryggan `spi_reg_bridge`

`spi_slave` (avsnitt 18.9.2) talar bara *bytes*. `register_bank` (kapitel 18) talar bara *registersemantik*. Mellan dem sitter ett lager som talar *transaktioner*, och det är `spi_reg_bridge`. Den vet ingenting om vilka register som finns, och ingenting om vad någon av dem betyder.

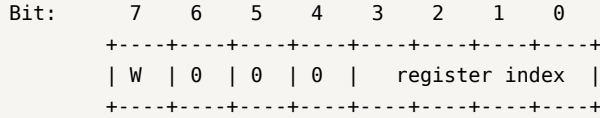
Gränssnitt

Tio portar, i den här ordningen: `clock`, `reset_s2_n`, `ss_active`, `rx_data`, `rx_valid`, `reg_rdata`, `tx_data`, `reg_addr`, `reg_wdata`, `reg_write`. De fem första plus `reg_rdata` är ingångar; de fyra sista är utgångar. Bindningen är positionell, som överallt annars.

De tre första kommer från `spi_slave`: `ss_active` är hög medan slaven är vald, `rx_data` bär den senast mottagna byten, och `rx_valid` pulsar en gång per byte. `tx_data` är nästa byte slaven ska skicka ut. De fyra registersignalerna kopplas rakt till `register_banks` sida mot bryggan.

Transaktionen

Varje transaktion är exakt **5 bytes** lång: en kommandobyte, sedan fyra databytes. SS faller före kommandobyten och stiger efter den femte byten; den måste ligga låg hela transaktionen.



- **Bit 7 (W):** 1 = skrivning, 0 = läsning.
- **Bit 6–4:** reserverade, måste vara 0.
- **Bit 3–0:** registerindexet, offset / 4 ur registerkartan (0–12).

Skrivning (W = 1). Mastern skickar registervärdet i de fyra databytesen, **MSB först** (bitarna 31–24 i den första databyten). Slaven verkställer det ihopsatta värdet i det adresserade registret **först när den femte byten är hel**. Vad slaven än driver på MIS0 under en skrivning är meningslöst; mastern kastar det.

Läsning (W = 0). Vid slutet av kommandobyten **läser** slaven det adresserade registrets aktuella värde **en gång**, in i sitt svarsskiftregister. De fyra databytesen klockar sedan ut det värdet på MIS0, MSB först, medan mastern skickar dummybytes 0x00. Regeln om att låsa en gång är inte en implementationsdetalj utan en del av kontraktet: de fyra bytesen tillhör alltid ett sammanhängande sampel av registret, taget i ett enda ögonblick — STATUS i synnerhet får aldrig vara ihopsytt av två skilda ögonblick.

Avbrott. Går ss_active låg innan den femte byten är hel överges transaktionen **utan sido-effekter**: ingenting verkställs, ingen utlösare avfyras, och slaven återgår till vila, redo för nästa fallande flank på SS. Det är det som gör slaven självläkande efter en glitch eller en master som resettas.

Ogiltiga kommandon. En läsning av ett reserverat index (13–15) ger 0x00000000; en skrivning till ett ignoreras. En kommandobyte med någon reserverad bit (6–4) satt är ogiltig på samma sätt: ingen läsning läses och ingen skrivning verkställs, och slaven förbrukar ändå hela fembytesramen innan den återgår till vila. Inget av fallen är ett fel slaven rapporterar — det finns ingen felkanal på det här lagret. Att upprätthålla regeln om reserverade *kommando*-bitar är spi_reg_bridges jobb; register_bank ser aldrig mer än ett 4-bitars index, aldrig ett reserverat kommando.

reg_addr **håller** det avkodade indexet tills nästa kommandobyte avkodar ett nytt: det är ett låst värde, inte en puls som följer den pågående byten. Testbänken kontrollerar indexet när alla fem bytes är förbrukade, så en brygga som nollställer reg_addr när den återgår till vila passerar de första fallen och faller på det.

Att verifiera den

Bryggan läser inget register alls och behöver därför bara spi_def:

```
cd bridge
ghdl -a --std=93 spi_def.vhd spi_reg_bridge.vhd spi_reg_bridge_tb.vhd
ghdl -e --std=93 spi_reg_bridge_tb
ghdl -r --std=93 spi_reg_bridge_tb --assert-level=error
```

Testbänken modellerar spi_slave i stället för att instansiera den: den pulsar rx_valid en gång per byte, inramat av ss_active, så att du kan skriva och kontrollera bryggan innan någon SPI-vågform är inblandad. Fyra fall: en skrivning, en läsning byte för byte, ett avbrott på ss_active, och en kommandobyte med reserverad bit satt. Fall 3 och 4 är de två som gör den här modulen till mer än en byteräknare — läs deras kommentarer innan du börjar, inte efteråt.

19.3 Att konstruera can_spi_node

can_controller är **inte** den entitet som ska upp på kortet, och det är värt att vara precis med. Dess gränssnitt ligger på registernivå: tx_id är 11 bitar, tx_dlc 4, tx_data 64, plus tx_req och fem statusutgångar. Ingenting på ett DE0-CV erbjuder ett så brett gränssnitt, och i det färdiga systemet kommer de ingångarna från drivrutinens register snarare än från en mänsklig hand.

Kapitel 18 byggde lagret som gör dem nåbara, och avsnitt 19.2 transporten framför det. Den sista modul boken skriver är alltså can_spi_node.vhd: toppnivån som binder ihop spi_slave, spi_reg_bridge, register_bank och can_controller till en nod, och exponerar exakt två saker mot omvärlden, SPI och CAN-bussen.

Till skillnad från varje annan modul i projektet kommer den utan egen testbänk på modulnivå, och det är avsiktligt: varje block inuti den har redan en, och det som återstår — pinnar, kopplingar och spänningsnivåer — är precis det en simulering inte kan modellera. Kortet är dess testbänk, och bring-up-stegen i avsnitt 19.6 är hur man läser den. (Det finns en utdelad systemtestbänk, can_spi_node_tb, som driver två kompletta noder från modellerade SPI-mastrar; den är den enda i boken som driver sclk, mosi och ss som riktiga vågformer.)

Gränssnitt

Port	Riktn.	Typ	Betydelse
<code>clock</code>	in	<code>std_logic</code>	50 MHz-oscillatorn på kortet. Konstruktionens enda klocka.
<code>reset_n</code>	in	<code>std_logic</code>	Rå, aktiv låg reset, direkt från en pinne. Asynkron; den här nivån synkroniserar den.
<code>sclk</code>	in	<code>std_logic</code>	SPI-klockan från mastern. Används aldrig som klocka.
<code>mosi</code>	in	<code>std_logic</code>	Ut från mastern, in i slaven.
<code>ss</code>	in	<code>std_logic</code>	Chip select, aktiv låg. Ramar in transaktionen.
<code>miso</code>	out	<code>std_logic</code>	Ut från slaven, in i mastern.
<code>rx_bus</code>	in	<code>std_logic</code>	CAN-bussens nivå, direkt från pinnen och asynkron. <code>can_controller</code> synkroniserar den själv.
<code>tx_bus</code>	out	<code>std_logic</code>	Biten den här noden vill driva.
<code>bus_en</code>	out	<code>std_logic</code>	Om den driver över huvud taget.

De tre bussportarna är det gränssnitt på digitalsidan som en riktig CAN-transceiver presenterar (kapitel 10), och det är därför de lämnar konstruktionen som tre separata signaler i stället för en ledning. Att göra en enda fysisk ledning av dem är ett jobb på kortnivå, och avsnitt 19.4 handlar om det.

Ordningen är SPI:s fyra ledningar som en grupp, `sclk/mosi/ss/miso`, och därefter CAN-bussens tre. `miso` ligger alltså bland SPI-portarna och inte hos de andra utgångarna, så att de fyra pinnarna står i samma ordning som i kopplingstabellen i avsnitt 19.6. Det spelar roll: `can_spi_node_tb` binder positionellt som allt annat, och `miso` och `rx_bus` har samma typ, så en förväxling av dem passerar analysen utan ett ord och ger en nod som aldrig svarar.

Det finns inga generics. Allt konstruktionen behöver som en pinne inte kan leverera är antingen en konstant i `can_def` eller ett register som drivrutinen skriver.

Beteende: strukturellt, och det är poängen

`can_spi_node` har ingen tillståndsmaskin, ingen protokollkunskap och nästan ingen logik alls. Den har exakt tre uppgifter:

- **Instansiera de fyra blocken och koppla dem på rad.** `spi_slave` gör om SCK/MOSI/SS till bytes; `spi_reg_bridge` gör om bytes till registeråtkomster; `register_bank` gör om registeråtkomster till signaler på kontrollernivå; `can_controller` gör om dem till en ram. Varje gräns är ett av de kontrakt som redan verifierats av en utdelad testbänk, så om kedjan betar sig illa sitter felet i en koppling här, inte i ett block.
- **Synkronisera reseten.** En enda `meta_prev`-instans tar den råa `reset_n` och producerar den `reset_s2_n` som varje delblock tar. `can_controller` är undantaget som bekräftar regeln: den äger sin resetsynkroniserare internt, vid sidan av den för `rx_bus`, så den här nivån matar den med den råa `reset_n` och synkroniserar bara åt blocken på SPI-sidan. Det ger tre `meta_prev`-instanser i hela konstruktionen — två inuti `can_controller`, en här.

- **Låt portarna mot kontrollern gå rakt över.** register_banks tx_id, tx_dlc, tx_data och tx_req till can_controller, och dess tx_done, rx_id, rx_dlc, rx_data, rx_valid och error tillbaka. Port till port, ingenting emellan. Kapitel 18 konstruerade de två gränssnitten så att de speglar varandra just för att det ska stämma.

Om en regel i den här filen någonsin behöver nämna ett CAN-fält eller en SPI-byte hör den hemma i en annan fil. Det är testet att tillämpa medan du skriver den.

19.4 Att få ut bussen på en ledning

tx_bus och bus_en är ett *par*, inte en signal: konstruktionen säger "driv dominant", "släpp", aldrig "driv recessivt". Två noders push-pull-utgångar knutna till en ledning skulle slåss med varandra i samma ögonblick som den ena drev låg och den andra hög. Två sätt att få den delade ledningen att uppföra sig:

Med en transceiver. Vilket är vad en riktig nod använder: tx_bus går till dess TXD, dess RXD kommer tillbaka som rx_bus, och transceivern sköter arbetet med öppen dränering analogt.

Utan en. Med två DE0-CV kopplade direkt ihop för tvånodsdemot gör FPGA:ns egen I/O jobbet. Välj en GPIO-pinne och driv den med öppen dränering:

```
GPIO_0(2) <= '0' when ((bus_en = '1') and (tx_bus = '0')) else 'Z';
rx_bus    <= GPIO_0(2);
```

med en pull-up som återställer '1' medan varje nod släppt ledningen — FPGA:ns svaga pull-up aktiverad på den pinnen i Pin Planner, eller ett externt motstånd. Det är avsnitt 10.2:s wired-AND-regel gjord fysisk, i en enda konkurrent tilldelning. GPIO_0(0) och GPIO_0(1) är värda att driva med tx_bus och bus_en oregistrerade också, enbart för att en logikanalysator ska kunna se vad noden *avsåg* vid sidan av vad ledningen faktiskt gjorde; varje oanvänt index drivs 'Z'.

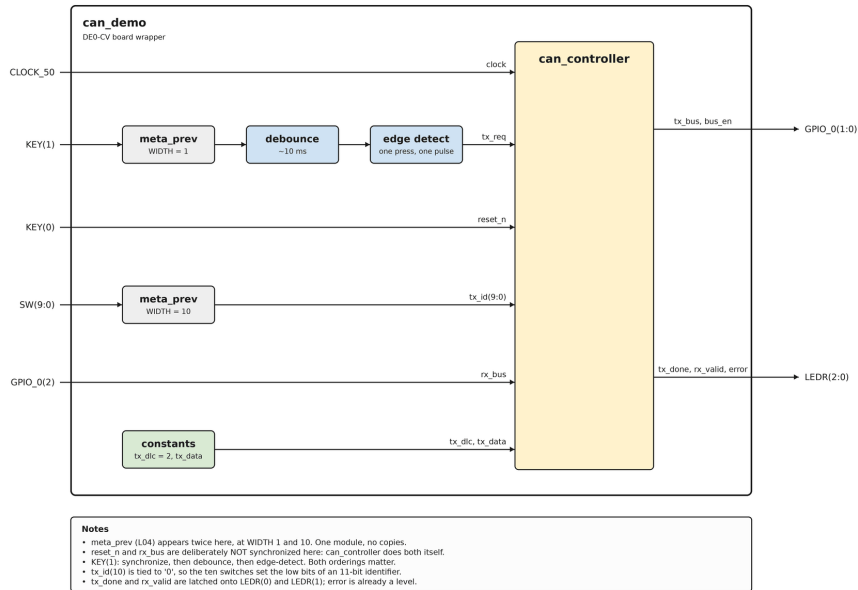
Ingenting här synkroniseras, av samma skäl som reset_n inte synkroniseras innan den når can_controller: kontrollern äger sina egna asynkrona ingångar, så pinnen dras rakt till porten rx_bus.

Pinnindexen är en bekvämlighet i Pin Planner, inte ett krav. Vilka tre pinnar som helst på listen fungerar, så länge två ihopkopplade kort är överens; se bilaga B.

19.5 Valfritt: en wrapper på kortnivå för felsökning

Allt ovanstående går bara att nå över SPI, vilket betyder att när länken inte fungerar har du ingen möjlighet att se in i den. En liten wrapper med knappar och lysdioder, som står bredvid can_spi_node i stället för att ersätta den, är värd en timme om bring-upen kör fast — och de tre beslut den tvingar fram är värda att förstå oavsett om du bygger den eller inte.

Wrappern driver register_banks portar direkt från knappar och omkopplare och låser dess statusutgångar på lysdioder:



Figur 19.1 En wrapper på kortnivå: knappar och omkopplare in i registerbanken, statusbitarna ut på lysdioder.

- **En hållen knapp måste bli en encykelspuls**, och inte bara för snygghetens skull. En knapp som hålls i några millisekunder är hundratusentals klockcykler vid 50 MHz. Mata in det rakt i tx_req så startar kontrollern en ny ram i samma ögonblick som den förra tar slut, om och om igen — och varje passage genom starttillståndet suddar error innan något hunnit läsa det. Alltså: **synkronisera** den råa pinnen med en meta_prev, **studsfiltrera** den synkroniserade nivån med en räknare som accepterar ett nytt värde först när det legat kvar i ungefär 10 ms, och **därefter** flankdetektera den. Alla tre, i den ordningen: att flankdetektera före studsfiltreringen ger en puls per studs, och att studsfiltrera före synkroniseringen matar räknaren med en signal som aldrig samplats säkert. Det här är samma konvertering som register_bank utför för TX_SEND (kapitel 18), och det är därför banken gör wrappern valfri snarare än nödvändig.
- **En omkopplarbank är bedömningsfrågan**. Tio omkopplare som korsar in i klockdomänen synkroniseras bit för bit, vilket tar bort metastabiliteten men inte *skewheten*: två omkopplare som slås om samtidigt kan landa en klocka isär, så tx_id skulle kortvarigt kunna läsa ett värde som aldrig stod på omkopplarna. Det är acceptabelt bara därför att en människa ställer in dem och sedan låter dem vara. Koppla samma tio bitar till något som ändras varje cykel och en bit-för-bit-synkroniserare är helt fel verktyg — då krävs

en handskakning, eller en kodning där bara en bit ändras i taget. Värt att säga rakt ut, eftersom "bredda bara synkroniseraren" är en vana som förr eller senare biter ifrån.

- **Statuslysdioder behöver låsas**, eftersom `tx_done` och `rx_valid` är pulser på 20 ns. Det är samma problem som `register_bank` löser åt drivrutinen, och att lösa det två gånger på två ställen är precis så de två kopiorna glider isär — så driv lysdioderna från bankens STATUS-bitar i stället för från kontrollerns pulser, så förblir wrappern en wrapper.

19.6 Bring-up

Bring-upen har två halvor: CAN-sidan mellan två kort, och SPI-sidan mellan en mikrokontrollerhatt och kortet. Ta dem i den ordningen. CAN-halvan behöver ingen mikrokontroller alls, så den går att köra medan hatten fortfarande ligger på lödbordet, och den utesluter en hel felklass innan någon SPI-ledning finns att misstänka.

När konstruktionen passerar simuleringen fullt ut, syntetisera och programmera `can_spi_node`: New Project Wizard, enheten 5CEBA4F23C7N, Pin Planner, Compile, Programmer. Stegen står i bilaga B, som är den auktoritativa referensen för det flödet, ända ner till tillägget för Cyclone V-enheter och till att få USB-Blastern synlig för Programmer. Notera att `CLOCK_50` är den enda pinne du inte får välja fritt.

CAN-halvan: två kort på en buss

Två kort, deras busslinjer på `GPIO_0(2)` knutna ihop till en ledning med gemensam jord, och drivningen med öppen dränering ur avsnitt 19.4 gör den delade ledningen till ett wired-AND.

1. **Ett ensamt kort.** Busslinjen bär ingenting annat än pull-upen, så noden läser tillbaka exakt den nivå den driver: en loopback. `tx_bus` och `bus_en` ska växla fält för fält på en logikanalysator via `GPIO_0(0)/GPIO_0(1)`, `tx_done` pulsa vid slutet av EOF, och error aldrig gå hög.
2. **Två kort, en ledning.** Tryck på en knapp på det ena kortet och en lysdiod ska tändas på det andra. Det är första gången konstruktionen möter två *oberoende* oscillatorer; i `can_controller_tb` delade noderna en clock.
3. **Arbitrering på riktigt.** Låt båda korten sända samtidigt och se att den med lägst identifierare vinner, och att förloraren backar.

En loopback är också där avsnitt 10.7:s sampelpunkt på 70 % i tysthet gör rätt för sig. Nodens egen signal tar nu en riktig rundtur, ut genom en pad, längs ledningen, in igen, och genom `rx_bus`-synkroniserarens två vippor, till en kostnad av en handfull klockcykler. Med `TICKS_PER_BIT = 50` och `SAMPLE_TICK = 35` läser noden bussen 35 tick in i en bit den började driva vid tick 0, så dess eget eko har sedan länge stabiliserat sig och arbitreringsövervakningen läser aldrig fel på det. Att sampla vid bitperiodens *början* skulle i stället läsa ekot av biten före, och arbitreringsövervakningen (sänd recessivt, läs dominant, kapitel 17) skulle rapportera en falsk förlust vid den första recessiva bit som följer på en dominant.

En loopback kan medvetet inte visa noden *ta emot* den ramen. `can_controller` läser `role = '1'` för hela ramen och `rx_shift_reg` går bara medan `role = '0'` (kapitel 17), så en sändare avkodar aldrig sin egen trafik; det är precis därför systemtestbänken behöver två noder.

Det andra kortet sluter den lucka avsnitt 19.1 kallar den största. Testbänkens två noder delar en clock: `resync` fasriktar dem fortfarande vid varje SOF, men deras timrar kan aldrig *driva isär* mitt i ramen. Två kort har två kristaller, som går på 50 MHz bara inom den tolerans komponenterna är specificerade till, så deras bitperioder skiljer sig verkligen åt, och mottagaren frikör på sin egen timer resten av ramen efter sin enda `resync` vid SOF. Ärlig aritmetik håller förväntningarna i schack: vid typiska kristalltoleranser på några tiotals ppm är driften över en ram en liten bråkdel av en tick, så det demot synligt bevisar är att hela kedjan fungerar över genuint oberoende klockor från en godtycklig startfas; hur mycket drift konstruktionen skulle tåla innan sampelpunkten vandrar av biten är beräkningen i Övning 12.4.

SPI-halvan: en mikrokontrollerhatt mot kortet

Kopplingen, per nod. Den är inte din att välja: den står i bilaga D, eftersom parallellklassen skriver sin drivrutin mot samma tabell.

Signal	AVR32DB28	DE0-CV
SCK	PC2	sclk
MOSI	PC0	mosi
MISO	PC1	miso
SS	PC3	ss
I/O-matning	VDDI02	3,3 V
Kärnmatning	VDD	(hattens egen 5 V)
GND	GND	GND

Ingen nivåomvandlare behövs, och det är ett val och inte en förenkling: AVR32DB28:ans `PORTC` är en egen spänningsdomän, matad från `VDDI02` i stället för `VDD`, så den läggs på DE0-CV:ns 3,3 V medan kärnan går kvar på 5 V. Varje nivå FPGA:n ser är då en nivå den är specificerad för, och de 3,3 V FPGA:n driver tillbaka läses mot en 3,3 V-tröskel i stället för en 5 V-tröskel. `SPI0` flyttas dit med `PORTMUX`. `SPIROUTEA = ALT1`.

Tre krav på hatten, och de måste vara på plats innan den etsas.

- **PC0–PC3 är reserverade.** De är `SPI0`:s `ALT1`-mux och får inte bära någon annan kringutrustning. Ingen lysdiod, ingen knapp, och framför allt ingen avkopplingskondensator på `SS` — en knapp med 100 nF på pinnen gör `SS`-flanken obrukbar vid 1 MHz.
- **Ingenting annat på `PORTC`.** Hela porten ligger i `VDDI02`-domänen på 3,3 V, så en 5 V-kringutrustning där skulle både sluta fungera och mata 5 V in i en 3,3 V-domän.
- **VDDI02 är en egen bana.** Den får inte knytas till `VDD` på kortet, utan ska ut till en pinne där DE0-CV:ns 3,3 V kopplas in, med 100 nF nära chippet.

Att domänen över huvud taget är separat styrs dessutom av `MVSYSCFG` i `FUSE.SYSCFG1`: i enkelmatningsläge knyts `VDDI02` internt till `VDD`, `PORTC` blir en 5 V-port, och kopplingen ovan lägger då 5 V rakt in i ingångar som inte är 5 V-toleranta — på en hatt som ser precis likadan ut som en rätt byggd. Kontrollera fusen före första inkopplingen. Det är det enda felet i det här kapitlet som kan kosta ett kort.

Stegen tas i den här ordningen, och inte i någon annan. Varje steg utesluter en felkälla, så att det som fortfarande inte fungerar efter steg n med säkerhet ligger i steg $n + 1$:

4. **SPI-loopback på enbart hatten**, med `PC1` kopplad till `PC0` och `FPGA:n` urkopplad. Bevisar att mastern fungerar, och på köpet att `PORTMUX` och `VDDI02` är rätt satta.
5. **Registereko**: skriv `TX_ID`, läs tillbaka det. Bevisar att hela transaktionskedjan bär.
6. **STATUS-polling**: läs `STATUS` och se att bit 0 är satt efter reset.
7. **En nod sänder på kommando**: skriv en ram och utlös `TX_SEND`, och se `STATUS` bit 0 gå låg och tillbaka — nu utlöst över SPI i stället för av en knapp.
8. **Två noder, hela kedjan**: den ena sänder på SPI-kommando, den andra tar emot och kvitterar med `RX_ACK`. Det är systemet.

Vad som återstår. När passet är slut har båda halvorna gått på riktig hårdvara. Det som återstår är inte kopplingen utan **motparten**: din nod har svarat på ett testprogram som är skrivet för att lyckas, inte på en drivrutin skriven av någon annan mot enbart specifikationen. Det är därför protokollspecifikationens tidskrav — de 60 ns kring `SS`, regeln om vad `MISO` bär under kommandobyten, och avbrottsregeln — är skrivna som krav och inte som råd: ett testprogram som aldrig gör något oväntat bekräftar dem aldrig, och en drivrutin som gör det fäller dig på dem.

19.7 Hur produktionsklara CAN-kontrollrar går längre

Den här bokens kontroller är medvetet förenklad: en undervisningskonstruktion snarare än en certifierbar. Listan nedan är den här bokens version av den; kapitel 7 i *CAN: bussen, ramen och kontrollern* ställer samma lista från protokollhållet, tillsammans med blocköversikten och de tre begränsningar som når ända upp i drivrutinens kod, och kapitel 6 i *CAN: bussen, ramen och kontrollern* beskriver den felhantering — felramar, felräknare, error-passive och bus-off — som den här konstruktionen helt saknar.

Anmärkningsvärda luckor, värda att namnge uttryckligen:

- **Bara standardramar med data**: ledningsformatet är en bit-exakt standard-CAN-dataram, RTR inräknad, men det finns inga 29-bitars utökade identifierare (mottagaren kontrollerar inte ens IDE, så en utökad ram skulle tolkas fel tills dess CRC-kontroll misslyckas), och en fjärram (en recessiv RTR) avvisas med error i stället för att besvaras med de begärda data.

- **Inga felramar och ingen återhämtning från bus-off:** en riktig CAN-nod räknar sändnings- och mottagningsfel och går in i alltmer begränsade tillstånd (error-passive, bus-off) för att skydda en frisk buss från en trasig nod; den här bokens `can_controller` avbryter bara den pågående ramen lokalt.
- **Ingen detektering av bit- eller formfel:** den här sändaren jämför vad den sände mot vad bussen läste bara under arbitreringen (kapitel 17). En riktig kontroller övervakar varje bit den sänder, flaggar för ett bitfel i samma ögonblick som bussen säger emot utanför arbitreringsfältet och ACK-luckan, och kontrollerar vid mottagning att svansens fasta fält har sina föreskrivna värden.
- **Ingen omsändning vid uteblivet kvitto:** en riktig kontroller försöker om automatiskt. Den här konstruktionen märker inte ens saken, eftersom sändaren aldrig läser tillbaka ACK-luckan.
- **Ingen acceptansfiltrering och ingen mottagningsbuffring:** varje ram på bussen landar i samma enda uppsättning `rx_*`-portar, och en andra ram som anländer innan drivrutinen läst den första skriver helt enkelt över den. Produktionsklara kontrollrar erbjuder identifierarfilter i hårdvara och en mottagnings-FIFO eller en uppsättning brevlådor. Det här är den lucka parallellklassens drivrutin känner mest direkt, och kapitel 18 visar exakt var den blir synlig på registernivå.
- **En fast modell för bittajmingen:** den här boken samplar vid en fast punkt på 70 %; produktionsklara kontrollrar exponerar konfigurerbara propagerings- och fasssegment och en synchronization jump width, avstämda efter busslängd och antal noder.
- **En avbruten mottagning (efter förlorad arbitrering) kastas helt enkelt,** i stället för att sömlöst fortsätta ta emot den ram som vann; en riktig nod fortsätter ta emot oavsett vem som vann bussen. Den här konstruktionens nod väntar ut resten av ramen i `STATE_IDLE` (avsnitt 16.1) och tar vid från nästa.
- **En förenklad regel om ledig buss:** den här konstruktionen kräver elva recessiva bitperioder, samma antal som riktig CAN begär av en nod som ansluter sig till busstrafiken, innan någon nod får lämna `STATE_IDLE`. Riktig CAN är mer finkornig: en nod som just deltagit i en ram får sända igen redan efter den 3-bitars intermission som följer på EOF, och en nod som varit bus-off väntar in 128 förekomster av elva recessiva bitar innan den får ansluta sig igen.

19.8 Konstruktionen, sedd i backspegeln

Varje kapitel från kapitel 10 och framåt lade till exakt ett ansvar, aldrig duplicerat någon annanstans:

Block	Äger	Byggt i
can_def	Delade konstanter och subtyper som varje annat block är överens om	kapitel 10
meta_prev	Att få en asynkron signal säkert in i den här klockdomänen, utan någon aning om vad signalen betyder	kapitel 12
bit_timer	När det ska samplas eller skiftas, utan någon aning om vad som sänds	kapitel 12
crc15	Beräkningen av CRC-15, en bit i taget, utan någon aning om vad en "ram" är	kapitel 13
tx_shift_reg rx_shift_reg	/ Bitstopppning och avstopppning, utan någon aning om vad ett ID eller en databyte är	kapi- tel 14–15
can_controller	Det enda block som känner till det faktiska CAN-ramformatet, arbitreringen inräknad	kapitel 11 (portar), ka- pitel 16–17 (beteende)
register_bank	Pulser till pollbara nivåer, och registerkartans semantik; ingen aning om vad SPI eller CAN är	kapitel 18
spi_slave	Bytes av en seriell ledning, säkert in i 50 MHz-domänen; ingen aning om vad ett register är	utdelad, läses i kapitel 18
spi_reg_bridge	Fembytestransaktionen; ingen aning om vad något enskilt register betyder	kapitel 19
can_spi_node	Kopplingarna, och pinnarna bussen bor på. Ingen egen logik	kapitel 19

Det är hela noden: fyra små block med ett ansvar var och ett delat paket komponerade inuti `can_controller`, och sedan tre lager till framför den, där vart och ett bara känner till det under sig. `can_spi_node` står utanför dem alla, eftersom att anpassa sig till ett visst kort är en annan sak än att tala CAN, eller SPI, eller register.

Läs tabellen uppfifrån och ned så är den också en lista över vad som skulle behöva ändras för att nå ett annat system. Byt ut `spi_slave` och `spi_reg_bridge` mot ett I2C-par så är allt under dem orört. Byt ut `can_controller` mot en annan protokollmotor så flyttar registerkartan men inte transporten. Det är vad gränserna köpte.

Registerkartan i sammanfattning. Varje port på registersidan av `can_controller` motsvarar ett register som parallellklassens C++-drivrutin läser eller skriver; de återstående fem (`clock`, `reset_n` och de tre bussignalerna) är fysiska och ingen drivrutin ser dem någonsin. Motsvarigheten är inte längre en plan, som den var när avsnitt 11.6 först ritade den här tabellen — `register_bank` är modulen som förverkligar den, så läs högerkolumnen som "registret banken presenterar vid det indexet":

can_controller-port	Register i kartan
tx_id, tx_dlc, tx_data	TX_ID, TX_DLC, TX_DATA_LO/HI
tx_req, tx_done	TX_SEND, en bit i STATUS
rx_id, rx_dlc, rx_data	RX_ID, RX_DLC, RX_DATA_LO/HI
rx_valid	en bit i STATUS, nollställd via RX_ACK
error	ERROR_FLAGS

Den motsvarigheten är där drivrutinsarbetet börjar: allt parallellklassen skriver mot står i bilaga D, och ingenting i den här konstruktionens VHDL.

19.9 Sammanfattning

Det här ska du kunna nu

- Implementera transaktions-FSM:en enligt protokollspecifikationen, inklusive avbrottsregeln.
- Förklara varför en läsning låser registervärdet en gång, och vad som går fel utan den regeln.
- Komponera hela noden till en toppnivå och verifiera den i simulering.
- Koppla ihop två DE0-CV till en delad buss, och en mikrokontroller som master över dess MVIO-domän, och felsöka kedjan systematiskt nerifrån och upp.
- Säga vilka krav SPI-kopplingen ställer på hattens konstruktion, och varför de inte går att rätta i efterhand.
- Säga vad simulering har bevisat och vad bara hårdvara kan bevisa.

Frågor att testa dig själv med

1. Varför läses registervärdet en gång vid slutet av kommandobyten i stället för att läsas medan databytesen skiftas ut? Vilket register lider mest av att den regeln bryts?
2. SS går hög efter tre bytes. Vad har hänt med transaktionen, och vad har ändrats i noden?
3. En skrivning till ett reserverat index. Vad gör bryggan, och varför konsumerar den ändå alla fem bytesen?
4. Varför finns ingen felkanal i transaktionslagret?
5. Vilket bring-up-steg skulle avslöja en förväxlad MISO och MOSI, och vilket skulle avslöja ett VDDI02 som aldrig kopplades in?
6. Hatten lägger en lysdiod på PC2. Vilket steg fallerar först, och varför är det ett fel som inte går att programmera sig ur?
7. Vad bevisade stegen 1 till 3 som stegen 4 till 8 inte behövde bevisa om igen?
8. Vad har simuleringen bevisat om designen, och vad har den inte kunnat bevisa?

Slutredovisning

Varje grupp visar, på ungefär femton minuter: en körning av `make build-project` med alla testbänkar gröna; en kort genomgång av `can_spi_node`, vad varje block gör och var gränserna går; bring-upen, så långt ni kommit, på riktig hårdvara; och `CONTRIBUTORS.md` plus en snabb genomgång av repots historik, branchar, PR:er och granskningar.

Slutredovisningen är projektets andra och sista fasta hållpunkt. Projektet lämnas in samma dag; se checklisten i bilaga C.

Bring-upen bedöms inte. Kommer du till steg 3 av åtta är det ett bra resultat; det som bedöms är simuleringen bakom den.

Blicken framåt

Kapitel 20 är bokens andra praktiska tentamen, med uppgifter på projektets moduler. Efter den är kursen slut.

Kontrollern du byggt lämnas över till parallellklassen, som skriver C++-drivrutinen mot samma register över samma SPI-transport. Det gemensamma kontraktet är bilaga D, och ingenting annat.

19.10 Övningar

Övning 19.1, 19.2 och 19.7 befäster avsnitt 19.1. Övning 19.3 till 19.6 befäster avsnitt 19.2 till 19.7 och bilaga D, som är auktoriteten överallt där den och en övning är oense.

Övning 19.1 En tredje nod, och ett nytt arbitreringsutfall

SIMULERING

a) Utöka `can_controller_tb.vhd` med en tredje nod, C, som delar samma `bus_line`. Ge C ID:t `0x002` och låt den begära sändning på samma cykel som nod A (`0x000`) och B (`0x001`). Bekräfta (genom simulering, inte bara förutsägelse) att A fortfarande vinner, och att *både* B och C upptäcker arbitreringsförlust.

b) Nod A:s `0x000` är den lägsta identifierare en standard dram kan bära, så inget val av ID låter C slå den. Höj A:s ID till `0x010`, och välj sedan ett ID till C som gör C till vinnare medan B (`0x001`) fortfarande förlorar. Motivera ditt val med avsnitt 10.6:s arbitreringsregel, och bekräfta sedan den nya vinnaren i simulering. En följd att lägga märke till innan dina assertions överraskar dig: med A på `0x010` slår B:s `0x001` även A, så i den här omgången **förlorar A också** och höjer error; bara C fullföljer.

Övning 19.2 En funktion ur en riktig kontroller, som designskiss

RESONEMANG

Välj **en** post ur avsnitt 19.7:s lista över vad riktiga CAN-kontrollrar gör som den här designen inte gör. Skissa, utan att nödvändigtvis implementera den i VHDL (i prosa, plus ett tillägg i tillståndsdigrammet eller en ny portlista om det hjälper), vad som skulle behöva ändras i `can_controller` för att stödja den. Ange vilket eller vilka befintliga block (`bit_timer`, `crc15`, `tx_shift_reg/rx_shift_reg`, eller `can_controller` självt) som skulle behöva ändras, och vilka som kunde stå exakt som de är.

Övning 19.3 `spi_reg_bridge`

VHDL

Skriv din egen `spi_reg_bridge.vhd` utifrån avsnitt 19.2 och bilaga D. Verifiera den:

```
cd bridge
ghdl -a --std=93 spi_def.vhd spi_reg_bridge.vhd spi_reg_bridge_tb.vhd
ghdl -e --std=93 spi_reg_bridge_tb
ghdl -r --std=93 spi_reg_bridge_tb --assert-level=error
```

- Få alla fyra fallen att passera. Fall 3 och 4 är de två som gör den här modulen till mer än en byteräknare — läs deras kommentarer innan du börjar, inte efteråt.
- Gör nu sönder den med flit. Ta bort varje användning av `ss_active` och räkna bytes modulo fem i stället. Fall 1 och 2 passerar fortfarande. Säg vilken kontroll i fall 3 som fallerar, citera värdet som felmeddelandet rapporterar, och förklara var de bytesen kom ifrån.
- En brygga som ignorerar de reserverade kommandobitarna passerar också fall 1 och 2. Förklara varför protokollspecifikationen lägger regeln om reserverade bitar i *det här* lagret i stället för i `register_bank`, givet att banken aldrig ser mer än ett 4-bitars index.
- Läsvägen läser `reg_rdata` **en gång**, vid slutet av kommandobyten. Säg vad `register_bank` skulle behöva göra i stället om bryggan läste det på nytt för varje databyte, och varför det vore fel plats att skjuta ner regeln i banken. Ställ tillbaka modulen.

Övning 19.4 Komponera `can_spi_node`

VHDL

Skriv toppnivån utifrån avsnitt 19.3: `spi_slave`, `spi_reg_bridge`, `register_bank` och `can_controller`, plus en `meta_prev` för reseten.

a) Skriv den, och kör `can_spi_node_tb` mot den. Fyra utdelade testbänkar täcker redan allt *inuti* filen; säg vilka fyra, och säg sedan den enda sortens fel som `can_spi_node_tb` kan fälla och som ingen av de fyra kan. (Ledtråden är den positionella portbindningen.)

b) `can_spi_node_tb` håller 200 ns tyst kring varje SS-flank, där bilaga D kräver minst 60 ns. Den är alltså snäll mot tidskraven med flit, och ingen utdelad testbänk pressar dem. Kopiera den, krymp den tysta perioden stegvis ner mot 60 ns, och ta reda på vid vilket värde din nod slutar svara.

Säg sedan **varför** just där, i klockcykler räknat, och var i `spi_slave` gränsen sitter. Jämför det uppmätta värdet med de 60 ns specifikationen lovar: har du marginal, eller ligger din nod precis på gränsen? Det här är enda stället i boken där du *mäter* en marginal i stället för att läsa den, och det är värt att göra noga.

c) Räkna `meta_prev`-instanserna i den färdiga designen och säg vad var och en synkroniserar. Säg sedan varför `can_spi_node` räcker `can_controller` den *råa* `reset_n` i stället för den `reset_s2_n` den ger varje annat block, och vad som skulle gå fel om den synkroniserade den två gånger.

d) `tx_bus` och `bus_en` lämnar designen som två skilda portar i stället för en ledning. Förklara vad som måste hända mellan dem och en fysisk ledning som delas med ett andra kort, och vad som skulle fallera om du tilldelade `tx_bus` rakt till en pinne och band ihop två kort.

Övning 19.5 Vad kortet visade

RESONEMANG

Bring-upen kördes i åtta steg, i två halvor: tre på två DE0-CV och fem med hatten som master. Den här övningen handlar om vad de fastställde och vad de inte gjorde. Den kräver ingen egen hårdvara.

a) Stegen är ordnade så att varje steg utesluter en felklass innan nästa beror på den. Säg för vart och ett av steg 1, 4 och 5 (ensamt kort, loopback på hatten, registereko) vilken enda sak det bevisar som steget före inte gjorde. Säg sedan vilka av de åtta stegen som `can_spi_node_tb` **redan** hade fastställt i simulering, och vilka som bara hårdvara kan fastställa.

b) Två kopplingsfel att skilja åt. Säg vilket steg som fallerar först om MISO och MOSI är förväxlade, och vilket som fallerar först om VDDI02 aldrig kopplades in, så att PORTC står utan matning. Förklara varför de fallerar vid olika steg, och vad den skillnaden låter dig sluta dig till utan att röra en mätare. Säg också vad MVI0.STATUS hade svarat i det andra fallet, och varför det svaret är snabbare än mätaren.

- c) Testprogrammet som delades ut är skrivet för att lyckas: det håller SS låg i god tid, skickar aldrig en sjätte byte och avbryter aldrig mitt i en transaktion. Säg vilken av protokollspecifikationens regler — de 60 ns kring SS, MIS0 under kommandobyten, eller avbrottsregeln — som **inget** av de åtta stegen skulle ha avslöjat som trasig, och varför just den därför är den dyraste att ha fel i när en drivrutin du inte skrivit möter noden.
- d) En ensam nod kan aldrig få `rx_valid` att pulsa, hur dess pinnar än är kopplade, vilket är skälet till att passet lade en ram över **två** kort. Säg varför en nod aldrig kan ta emot sin egen ram (skälet är kapitel 17:s `role`), och vad det andra kortet därmed bevisar som varken en `loopback` eller `can_controller_tb` kan.
- e) De två korten går på två oberoende 50 MHz-oscillatorer. Avsnitt 19.1 kallar den delade klockan i `can_controller_tb` för testbänkens **största** gap av precis det skälet. Var noga med vad som ändras: `bit_timer.resync` gör redan riktigt arbete i simulering, så säg vad de två oberoende klockorna lägger till som ingen simulering med delad klocka kan visa, och ungefär hur långt isär de två oscillatorerna skulle kunna driva innan en ram slutade komma fram hel (Övning 12.4 gjorde den uträkningen; återanvänd den). Jämför den gränsen med en typisk kristalls tolerans och säg om demonstrationen egentligen sätter den på prov.
- f) `rx_bus` kommer utifrån FPGA:n, och `can_spi_node` skickar den osynkroniserad vidare till `can_controller` med flit, eftersom `can_controller` innehåller sin egen `meta_prev` (kapitel 12). Förklara vad som skulle kunna gå fel om den instansen togs bort, varför felet skulle vara betydligt svårare att diagnostisera än en felkopplad ledning, och varför ingen mängd simulering i den här boken någonsin skulle återskapa det.

Övning 19.6 Att följa ett `send()` från ände till ände

RESONEMANG

- a) Följ en enskild ramförfrågan hela vägen ner. Utgå från de fem bytesen på MOSI som skriver `0x1` till `TX_SEND`, och följ dem genom `spi_slave` (bytes), `spi_reg_bridge` (en registerskrivning) och `register_bank` (ett `tx_req` på en cykel) fram till `can_controllers` portar. Namnge vad varje lager räcker vidare till nästa. Gör sedan detsamma uppåt, för hur drivrutinen får veta att ramen är klar.
- b) Fyra lager är tre fler än `can_controller_tb` behöver. Namnge för varje gräns den utdelade testbänk som spikar fast den, och säg vad som skulle vara oprövat om den testbänken inte fanns.
- c) Pollning är det enda den här designen erbjuder: det finns ingen avbrottsutgång någons. Säg vad som skulle behöva läggas till för att ge drivrutinen en sådan, ur vilken befintlig signal den skulle härledas, och i vilket lager den skulle behöva ligga. Förklara sedan varför det hade varit hopplöst för en drivrutin att polla ett `tx_done` som varar en cykel, och vilken modul som gjorde att det inte är det.
- d) En drivrutin skriver `TX_ID`, sedan `TX_DLC`, sedan `TX_SEND`, som tre separata transaktioner. Mellan den andra och den tredje går bussen till tomgång och en annan nod sänder. Säg om något av det drivrutinen skrivit är i fara, och vilken regel i registerkartan det är som gör ditt svar sant.

Övning 19.7 Livet efter ett avbrott

SIMULERING

Avsnitt 19.1 listar "ingen nod sänder någonsin igen efter ett avbrott" som något systemtestbänken inte täcker. Den här övningen täcker det, och visar vad `clear`-ingången hos `crc15` var till för.

- a) Utöka `can_controller_tb.vhd` med ett tredje fall. Ge båda noderna tid att återgå till tomgång efter att fall 2 körts, alltså efter att nod B förlorat arbitreringen och övergett sin ram mitt i identifieraren, och låt sedan B sända en egen ram (ID `0x123`, DLC 2, valfri nyttolast) med A som mottagare. Bekräfta i simulering att A tar emot den: A:s `rx_valid` pulsar, A:s error förblir låg, och ID:t kommer tillbaka korrekt.
- b) Ta nu bort nollställningen: ändra `crc_clear` till en konstant '0' och kör om. Fall 1 och 2 passerar fortfarande. Fall 3 gör det inte. Säg vilken nod som höjer error, vid vilket tillstånd, och förklara varför det *inte* är den noden som har buggen.
- c) Räkna ut vad B:s `crc15`-register håller i det ögonblick den avbryter i fall 2, och varför ingen mängd korrekt beteende därefter någonsin återför det till noll. (Ledtråden är kapitel 13:s egenskap att generering och kontroll är samma operation: vad förutsätter den om registret vid en rams *början*?)

- d) Hur många ramar måste nod B skicka innan den fungerar igen, med nollställningen fortfarande borttagen? Förklara ditt svar, och säg vad en drivrutin som pollar STATUS och ERROR_FLAGS över SPI skulle observera varje gång, med kapitel 18:s regel för hur felbiten läses.
- e) Kapitel 16 lägger nollställningen i STATE_START i stället för i STATE_CRC_WAIT eller på vägen ut ur ett avbrott. Ge ett skäl vardera till varför de andra två placeringarna är sämre.

Praktisk tentamen 2: CAN-moduler

Tre timmar, individuellt, vid datorn. Omfattar projektets moduler: bittajming, CRC, bitstoppning, ramsekvensering, registersemantik och SPI-transaktionen.

I DET HÄR KAPITLET

- Varför en individuell tentamen följer på ett grupparbete.
- Tentamens form: skrivuppgifter med utdelad testbänk, och läsuppgifter på papper.
- De fyra slagen av uppgifter: varianter, reparationer, ett enskilt register, läsning.
- Vad som är tillåtet att använda — inklusive gruppens egen projektkod.
- Hur man förbereder sig, vad tentamen mäter, och vart konstruktionen tar vägen efteråt.

Det här är kursens andra och sista examinationstillfälle. Som kapitel 9 innehåller det ingen ny teori; kapitlet säger vad tentamen mäter, vilken form uppgifterna har och hur man förbereder sig. Poäng och betygsgränser står i bilaga C.

Tentamen omfattar **projektets moduler**, alltså kapitel 10 till 19: bittajmingen, CRC:n, bitstoppningen åt båda hållen, ramsekvenseringen, registersemantiken och SPI-transaktionen. Del I:s material är inte borta — varje modul i projektet är byggd av den synkrona processmallen, en räknare, ett skiftregister och en Mooremaskin — men tyngdpunkten ligger på del II.

20.1 Varför en individuell tentamen på ett grupparbete

Projektet bedöms i grupp och står för halva kursen. Den här tentamen mäter något annat: att var och en förstår det gruppen byggde, inte bara den del man själv skrev.

Det är inte en misstro mot arbetsformen utan en följd av den. En grupp om fyra eller fem delar upp elva moduler mellan sig, och den uppdelningen är precis vad som gör projektet genomförbart på tio pass. Baksidan är lika förutsägbar: den som skrev `crc15` kan komma undan utan att någonsin ha behövt förstå varför `bit_timer` har två pulser, och den som skrev

registerbanken utan att ha tänkt på vad stoppningen gör åt CRC:n. Uppgifterna hämtas därför ur *alla* delar av projektet, oavsett vem som skrev vilken modul.

Det finns en praktisk konsekvens av det här som är värd att ta på allvar långt före tentamen: **kodgranskningen under projektet är förberedelsen**. En grupp som granskar varandras PR:er på riktigt, i stället för att godkänna dem osedda, har redan läst allt som tentamen frågar om. Kapitel 15:s kodgranskningsseminarium är tillfället att öva det; det är också det tillfälle som betalar sig här.

20.2 Upplägg

Samma form som kapitel 9: fristående uppgifter, oberoende av varandra och ordnade ungefär efter stigande svårighet.

De flesta är **skrivuppgifter** — en utdelad, självkontrollerande testbänk och en beskrivning av modulen den driver, med portordning, typer och beteende. Uppgiften är att skriva modulen så att testbänken passerar.

Därutöver finns **läsuppgifter**: ett tidsdiagram eller en SPI-transaktion att tolka på papper. De har ingen testbänk, av precis det skäl som gör dem värda att ställa — att läsa en vågform är en färdighet i sig, och den examineras inte av att skriva VHDL. De rättas mot ett facit, och delvis rätt svar ger delpoäng på samma sätt som delvis korrekt logik gör.

De fyra slagen av uppgifter

Varianter. En `bit_timer` med en annan sampelpunkt eller en annan bithastighet. En stoppningsdetektor som räknar ett annat antal. En CRC med ett annat polynom. Modulen är bekant; den enda punkt där den skiljer sig från projektets är den punkt som kräver att du förstått den.

Reparationer. En given tillståndsmaskin eller ett givet skiftregister som är *nästan* rätt. Hitta felet med hjälp av testbänkens utskrift och åtgärda det. Det är kapitel 14:s och 15:s arbetssätt satt på prov: läs vad testbänken rapporterar, och gå bakåt därifrån till raden som orsakade det.

Ett enskilt register. Ett register ur registerbanken, med sin sättnings- och nollställningsregel, isolerat från resten. Kapitel 18:s skillnad mellan en puls och en klibbig nivå är hela uppgiften.

Läsning. Ett tidsdiagram eller en SPI-transaktion att tolka: vad ligger på bussen, och vad borde ha legat där?

Delvis korrekt logik ger delpoäng, så lämna aldrig in en tom fil för att modulen inte blev klar; detsamma gäller ett halvt uttolkat tidsdiagram. Rättningen läser dessutom koden, inte bara testbänkens utfall: en modul som passerar genom att göra något annat än det som efterfrågades ger inte full poäng.

20.3 Hjälpmedel

Tillåtet:

- Allt kursmaterial: den här boken, föreläsningarnas README och samtliga appendix. Registerkartan och protokollspecifikationen (bilaga D) hör förstås dit; flera uppgifter är skrivna mot dem.
- **Gruppens egen projektkod.** Ni byggde den; det vore konstigt att låtsas annat. Uppgifterna är skrivna så att avskrift inte hjälper — varianterna skiljer sig på just den punkt som kräver att man förstått modulen, och reparationerna har ingen motsvarighet i repot alls.
- Din egen dator med GHDL, och CircuitVerse om du vill rita en krets eller en vågform innan du skriver den. Det är ofta snabbare, inte långsammare.

Inte tillåtet: kommunikation med andra i någon form; språkmodeller och andra verktyg som genererar eller förklarar VHDL åt dig; och sökning på nätet efter färdiga lösningar. Gränsen går inte vid vem som trycker på tangenterna utan vid vems förståelse som provas.

Den fullständiga och auktoritativa hjälpmedelslistan står i bilaga C; står något annat här gäller den. Är du osäker på om något är tillåtet: fråga före tentan, inte under.

20.4 Före tentamen

- Gå igenom de moduler i projektet som *någon annan* i gruppen skrev. Det är där luckorna finns, och det är det enda rådet i listan som är svårt att följa dagen innan.
- Kontrollfrågorna sist i kapitel 12 till 19 är avsiktligt skrivna som repetitionsmaterial. Kan du svara på alla utan att slå upp något är du klar.
- Läs igenom de utdelade testbänkarna en gång till. De är det tydligaste som finns om exakt vad varje modul lovar, och en skrivuppgift är alltid lättare för den som vet hur en testbänk formulerar sina krav.
- Rita om `bit_timers` två pulser och `can_controllers` tillståndsdigram på ett papper, ur minnet. De två bilderna bär mer av projektet än någon enskild fil gör.
- Se till att GHDL fungerar på din maskin *innan* du kommer. En trasig verktygskedja är inget skäl till förlängd skrivtid.

20.5 Det här mäter tentamen

- Att du kan omsätta en specifikation i ord till en korrekt modul, med rätt portordning.
- Att du förstår bittajmingen: varför sampelpunkten ligger där den ligger, och vilken puls som gör vad.
- Att du kan resonera om bitstoppning åt båda hållen — att sätta in och att ta bort är samma regel sedd från två håll, och båda sidorna måste hålla CRC:n utanför.
- Att du kan skilja en puls från en klibbig nivå, och veta vilket lager som ansvarar för vilket.

- Att du kan läsa en testbänks utskrift och hitta tillbaka till orsaken.

20.6 Efter kursen

Kontrollern går vidare: parallellklassen skriver C++-drivrutinen mot samma register över samma SPI-transport, och noden följer med till CAN-labben i nästa kurs i inbyggda system. Det som håller ihop de två halvorna är registerkartan och protokollspecifikationen i bilaga D — inte koden på någondera sidan. Det är själva poängen med ett gränssnitt: två grupper som aldrig läst varandras källkod kan ändå mötas på en buss, därför att specifikationen mellan dem är skriven innan någon av dem började.

Över tjugo kapitel har boken gått från en grind på ett papper till en CAN-nod som en mikrokontroller kan prata med. Varje mönster på vägen är allmängods i digital konstruktion och följer med oförändrat in i vad du än bygger härnäst:

- dubbelvippsynkroniseraren (kapitel 4), som varje asynkron ingång passerar,
- den klockade processmallen (kapitel 3), som varje sekventiell modul i boken är skriven med,
- Mooremaskinen (kapitel 8), som en uppräknad typ och ett case,
- skiftregistret och räknaren (kapitel 6) som byggstenar snarare än som specialfall, och
- till sist uppdelningen i block med ett tydligt ansvar var (kapitel 11), som är det enda skälet till att elva moduler går att skriva av fem personer parallellt och ändå passa ihop.

Vill du gå vidare från konstruktionen i den här boken — till en kontroller med felräknare och ACK-kontroll, till verifiering bortom en handskriven testbänk, eller till nästa protokoll byggt med samma metod — pekar bilaga E ut vad som finns att läsa härnäst.

Simulering med GHDL

Det här är bokens permanenta referens för simulering. Varje kapitel som verifierar en modul mot en testbänk pekar hit i stället för att upprepa stegen. Avsnitt 2.7 är den mjukare introduktionen, skriven för första gången du kör en testbänk över huvud taget; den här bilagan är den du kommer tillbaka till, och den täcker gruppprojektets bygge i synnerhet.

A.1 Varför simulera

Kapitel 1 till 8 verifierar varje konstruktion för hand, i CircuitVerse eller direkt på DE0-CV. Det fungerar för små kretsar med en handfull ingångar som en människa kan växla.

Det slutar fungera så snart en konstruktions korrekthet beror på exakta tidsförhållanden över flera cykler mellan flera moduler, vilket är gruppprojektets situation rakt igenom.

- En **testbänk** är ett litet, utdelat VHDL-program som driver en konstruktions ingångar och kontrollerar dess utgångar automatiskt.
- **GHDL** är verktyget som kompilerar och kör den, helt i programvara, utan att någon FPGA är inblandad.
- Du ombeds aldrig skriva den testbänk som *verifierar* en modul du bygger. Varje sådan är utdelad; du kör den och läser vad den rapporterar.
- Några övningar bjuder in dig att skriva en kort *provisorisk* rigg, tjugo rader som matar in din egen bitsekvens i en modul och report:ar vad som kommer ut. Det är utforskande, och alltid frivilligt till formen: ingenting du bedöms på beror på en testbänk du själv skrivit.

A.2 Att installera GHDL

På Ubuntu eller WSL:

```
sudo apt-get update && sudo apt-get install -y ghdl
```

Bekräfta att det gick vägen:

```
ghdl --version
```

GHDL 3.x eller senare. Kursens CI kör samma GHDL-kommandon som nedan, så det som passerar lokalt passerar också i CI.

A.3 Var filerna bor

Grupprojektets VHDL ligger i **två platta kataloger i repots rot**: modulerna du skriver, vid sidan av de utdelade testbänkarna (*_tb.vhd) som kontrollerar dem. Det finns inga kataloger per pass och ingenting att kopiera mellan dem; can_def.vhd i synnerhet skrivs *en* gång, i kapitel 10, och varje senare kontrollermodul läser den enda filen.

Uppdelningen mellan de två katalogerna är protokollgränsen: allt i controller/ vet något om CAN och ingenting om SPI, och allt i bridge/ tvärtom.

controller/	can_def.vhd	kap 10, din
	can_controller.vhd	kap 11, din (först bara portar, växer genom kap 17)
	meta_prev.vhd	kap 12, din
	bit_timer.vhd	kap 12, din
	crc15.vhd	kap 13, din
	tx_shift_reg.vhd	kap 14, din
	rx_shift_reg.vhd	kap 15, din
	+ de sex utdelade *_tb.vhd	
bridge/	register_bank.vhd	kap 18, din
	spi_reg_bridge.vhd	kap 19, din
	can_spi_node.vhd	kap 19, din
	+ utdelade spi_slave.vhd, spi_def.vhd,	
	register_bank_tb.vhd, spi_reg_bridge_tb.vhd	

De två katalogerna innehåller från början bara de utdelade filerna: åtta testbänkar, plus spi_slave.vhd och spi_def.vhd. Varje kapitel från kapitel 10 lägger till en modul, med undantaget att can_controller.vhd skrivs tidigt (kapitel 11) som en entitet med en tom arkitektur och sedan *växer*: kapitel 16 fyller i dess sändväg och kapitel 17 dess mottagväg, utan att någon ny fil dyker upp för någondera. Det är därför can_controller_tb, den enda testbänk som prövar hela kontrollern, är grindad på mer än att filerna finns; avsnitt A.5 återkommer till det.

A.4 De tre GHDL-stegen

Varje VHDL-konstruktion (och testbänk) går igenom samma tre steg, i den här ordningen:

1. **Analysera** (ghdl -a): tolkar en .vhd-fil, kontrollerar den mot syntax- och typfel, och registrerar dess designenheter (entiteter, arkitekturer, paket) i ett arbetsbibliotek. Be-

roenden måste analyseras före de enheter som använder dem: `can_def.vhd` före varje modul som säger `use work.can_def.all;`, och en modul före sin egen testbänk.

2. **Elaborera** (`ghdl -e`): löser upp en viss toppnivåentitets fullständiga instansieringshierarki till något körbart.
3. **Köra** (`ghdl -r`): simulerar den på riktigt.

För en modul med utdelad testbänk, till exempel kapitel 12:s `meta_prev`, som inte läser något paket alls och därför bara behöver sina egna två filer:

```
cd controller
ghdl -a --std=93 meta_prev.vhd meta_prev_tb.vhd
ghdl -e --std=93 meta_prev_tb
ghdl -r --std=93 meta_prev_tb --assert-level=error
```

Varje annan kontrollermodul läser `can_def`, så paketet kommer först:

```
cd controller
ghdl -a --std=93 can_def.vhd bit_timer.vhd bit_timer_tb.vhd
ghdl -e --std=93 bit_timer_tb
ghdl -r --std=93 bit_timer_tb --assert-level=error
```

`bridge/` fungerar likadant, med en krumelur: `register_bank` läser `can_def`, som ligger en katalog bort.

```
cd bridge
ghdl -a --std=93 ../controller/can_def.vhd register_bank.vhd register_bank_tb.vhd
ghdl -e --std=93 register_bank_tb
ghdl -r --std=93 register_bank_tb --assert-level=error
```

- `--std=93` väljer VHDL-93 genom hela kursen. Varje utdelad testbänk avslutar sig själv snyggt, genom att sätta en `done`-signal som klockgenereringsprocessen kontrollerar och sedan anropa `wait;`, i stället för att förlita sig på någon "stoppa simulatorn"-procedur ur en senare standard.
- `--assert-level=error` säger åt GHDL att stanna simuleringen, med en utgångskod skild från noll, vid varje misslyckad kontroll som rapporterats på nivån `error` eller högre. **Skicka alltid med den när du kör en projekttestbänk.** Nästa avsnitt säger varför den inte är valfri här.

A.4.1 Varför `--assert-level=error` spelar roll, och var den inte gör det

GHDL stannar på en misslyckad assertion bara när assertionens allvarlighetsgrad når assert-nivån, och GHDL:s förvalda nivå är `failure`. Bokens två halvor ligger på var sin sida om den gränsen, vilket är värt att veta eftersom det förklarar varför flaggan ser valfri ut i del I och inte är det i del II:

	Grad	Utan flaggan	Med flaggan
Kapitlens testbänkar	failure	stannar, avslutar 1	stannar, avslutar 1
Projektets testbänkar	error	skriver ut, kör vidare, avslutar 0	stannar, avslutar 1

För en projekttestbänk gör alltså ett utelämnande av flaggan om ett verkligt fel till ett meddelande som rullar förbi ovanför en nollställd utgångskod, vilket är precis vad ett "tyst godkänt" ser ut som. Skicka med den varje gång så slutar skillnaden spela roll.

--stop-time=10ms är värd att lägga till av samma slags skäl. En testbänk som väntar på en händelse din modul aldrig producerar *fallerar* inte; den hänger. Stopptiden gör om det till en körning som tar slut. Den står inte i kommandona ovan därför att projektets testbänkar alla begränsar sina egna väntor, men den kostar ingenting och avsnitt 2.7 gör den till en vana.

Varje testbänk rapporterar bara godkänt eller underkänt (och, vid underkänt, orsaken), ingenting annat. En godkänd körning slutar med en rad som denna:

```
bit_timer_tb.vhd:148:9:@7312ns:(report note): bit_timer: all checks passed!
```

En underkänd körning stannar tidigt med ett (assertion error)-meddelande som namnger exakt vilken kontroll som fallerade och varför. Läs det meddelandet; det är skrivet för att tala om specifikt vad som gick fel.

A.5 Att bygga allt på en gång

make build-project, från repots rot, gör allt det här automatiskt för varje testbänk vars moduler du skrivit. Det är användbart för att bekräfta att hela ditt träd fortfarande passerar, men det är förståelsen av de tre enskilda stegen ovan som låter dig felsöka en enskild modul snabbt under ett pass.

Eftersom projektet byggs upp en modul i taget **hoppa** make build-project över varje testbänk vars moduler ännu inte finns, i stället för att falla på dem, och säger det:

```
==> meta_prev_tb
controller/meta_prev_tb.vhd:114:9:@311ns:(report note): meta_prev: all checks
↳ passed!
==> bit_timer_tb (skipped: no bit_timer.vhd yet)
```

Så ser kapitel 12 ut halvvägs: meta_prev.vhd skriven och godkänd, bit_timer.vhd kvar att göra. can_def.vhd namnges inte som saknad, eftersom kapitel 10 redan skrev den.

Överhoppad är inte underkänd, med flit. Projektet löper över tio pass och det mesta av trädets saknas under de flesta av dem; ett bygge som lyste rött tills den sista modulen landade skulle inte säga något användbart under någon av de nio föregående. Antalet testbänkar som faktiskt kör växer alltså allteftersom: inga efter kapitel 10 eller 11, två efter kapitel 12,

sedan en till i vart och ett av kapitel 13, 14 och 15, en till i kapitel 17, en i kapitel 18, och den sista i kapitel 19.

`can_controller_tb` är **grindad på mer än att filen finns**, och det är värt att veta innan det förbryllar dig. `can_controller.vhd` skrivs i kapitel 11 som en entitet med tom arkitektur och växer genom kapitel 16 och 17, så från kapitel 15 och framåt finns varje fil testbänken behöver medan kontrollern fortfarande inte kan ta emot en ram. Att köra den då skulle fallera, två gånger om: en gång i kapitel 15, och igen i kapitel 16, som kan sända men ännu inte ta emot. Eftersom testbänken fungerar genom att låta en nod ta emot en annans ram är det den faktiskt kräver en **mottagväg**, och den kommer i kapitel 17. Byggskriptet tittar därför inuti din `can_controller.vhd` efter kapitel 17:s CRC-grindning på mottagarsidan och rapporterar:

```
==> can_controller_tb (skipped: can_controller.vhd has no receive path yet;
      set CI_BUILD_ALL=1 to run it anyway)
```

Den kontrollen är en heuristik snarare än ett verkligt beroende, och den har ett felläge värt att namnge: om din kapitel 17 är korrekt men stavar de uttrycken annorlunda än kapitlet får du det här hoppmeddelandet när testbänken borde ha körts. Ingenting är trasigt; kör `CI_BUILD_ALL=1 make build-project` för att tvinga fram varje testbänk oavsett. Kontrollen kan inte fallera åt andra hållet, eftersom markören inte kan dyka upp innan en mottagväg gör det.

A.6 Att städa upp

GHDL:s analyssteg skapar `work-obj93.cf` och liknande filer i den katalog du körde dem från. Byggskriptet håller sig i stället med en egen `work/-`katalog, så de filerna skräpar inte ner källträdet. Kör du de tre stegen för hand i `controller/` eller `bridge/`, städa med:

```
make clean
```

som tar bort de genererade `work/-`katalogerna, eventuella kvarglömnda `work-obj*.cf`, och de vågforms- och exekverbara filer en handkörning lämnar efter sig.

Quartus Prime Lite och DE0-CV

Det här är bokens verktygsreferens för syntes och hårdvara. Genom kapitel 1 till 18 är den **lärarvärd**: Quartus- och DE0-CV-arbetet demonstreras från katedern, med början i bromsassistenten i kapitel 1, och ingen deltagare behöver Quartus eller ett eget kort.

Kapitel 19 är undantaget, och det är det som räknas. Bring-upen demonstreras först och lämnas sedan över: varje grupp syntetiserar och programmerar sin egen `can_spi_node` och demonstrerar sin egen tvånodslänk. Läs därför avsnitt B.3 till B.7 som deltagarvärda — du behöver avsnitt B.3 och B.4 för att skapa projektet och lägga till källfilerna, och avsnitt B.6 är kompileringen som producerar den .sof avsnitt B.7 programmerar — och läs avsnitt B.9 *innan* du kopplar in något, eftersom det är avsnittet som avgör om Programmer över huvud taget kan se kortet.

Ingenting som **bedöms** kräver hårdvaran (bilaga C). Bring-upen bedöms på simuleringen bakom den, inte på om ledningen fungerade just den dagen. Allt som beskrivs här slutar på riktig FPGA-hårdvara, verifierad för hand. Det är ett annat lager än övningarna, som du verifierar på din egen laptop med en självkontrollerande testbänk under GHDL (bilaga A).

B.1 Vad som behövs

- **Quartus Prime Lite** (gratisutgåvan): tillverkarens FPGA-verktyg, som täcker syntes, placering och routning ("fitting"), och generering av programmeringsfil.
- **Enhetsstöd för Cyclone V**: ett separat tilläggspaket. DE0-CV:s FPGA, 5CEBA4F23C7N, är en Cyclone V-krets, och Quartus installerar inte varje familjs stöd som förval.
- **USB-Blaster-drivrutin**: DE0-CV programmeras över USB med USB-Blaster-protokollet. Drivrutinen installeras oftast tillsammans med Quartus, men behöver ibland en manuell knuff (avsnitt B.8).
- Ett **Terasic DE0-CV-kort** och en USB-kabel.

Quartus Prime Lite finns för både Windows och Linux, och konstruktionsstegen nedan är desamma oavsett värddoperativsystem. **Att få kortet synligt för Programmer är det inte**, och det är där varje värdspecifikt problem i den här kursen bor: avsnitt B.9 täcker Linux och WSL, avsnitt B.8 täcker Windows.

En rekommendation direkt, eftersom den sparar en kväll: kursens GHDL-verktygskedja kör på WSL eller Ubuntu, men **WSL2 kan inte se en USB-enhet alls utan extra arbete** (avsnitt B.9). Är du på Windows, installera Quartus nativt på Windows och behåll WSL för GHDL. De två behöver aldrig se varandra; de delar bara .vhd-filerna, som ligger i Windows-filsystemet hur som helst.

B.2 Att installera Quartus Prime Lite

1. Hämta **Quartus Prime Lite Edition** från tillverkarens nedladdningsportal. Sök efter "Quartus Prime Lite download": Intels FPGA-verksamhet blev **Altera** igen 2024, så sidorna har flyttat och äldre djuplänkar ruttar. Det som är stabilt att söka på är produktnamnet, inte URL:en.
 - Välj **Lite**-utgåvan, som är den fria och inte behöver någon licensfil. Standard och Pro täcker inte Cyclone V gratis.
 - Vilken någorlunda färsk Lite-utgåva som helst fungerar; kursen beror inte på en viss delversion.
 - Välj Windows- eller Linux-installeraren efter var du tänker köra den, och läs varningen ovan om du är på Windows med WSL.
2. Se under installationen till att **Cyclone V** är valt bland enhetsfamiljerna. Är Quartus redan installerat utan det, lägg till Cyclone V-stödet efteråt, antingen genom att köra installeraren igen eller via Quartus egen "Additional Software"-installerare.
3. När DE0-CV-kortet ansluts över USB första gången ska Windows upptäcka USB-Blastern och antingen installera drivrutinen automatiskt eller be dig peka ut Quartus egna drivrutinsfiler (vanligen under Quartus installationskatalog). Hittar Windows ingen drivrutin automatiskt, se avsnitt B.8.

B.2.1 Två fallgropar i Windows 11

Ett par av Windows 11:s säkerhetsförval kan störa installationen, eller programmeringen av kortet senare:

- **Om installeraren inte ens öppnar sig:** leta upp Smart App Control i Windows Säkerhet (sök efter det i startmenyn), stäng av det och starta om.
- **Om allt installeras snyggt men kortet ändå inte går att programmera** (avsnitt B.7): gå till Windows Security → Device security → Core isolation och slå av **Memory integrity**.

Core isolation

Security features available on your device that use virtualisation-based security.

Memory integrity

Prevents attacks from inserting malicious code into high-security processes.

 Memory integrity is off. Your device may be vulnerable. [Dismiss](#)

 Off

[Learn more](#)

Figur B.1 Memory integrity under Windows 11:s inställningar för kärnisolering.

Ingendera inställningen rör Quartus i sig; båda är allmänna hårdningsförval i Windows 11 som råkar vara strikta nog att störa USB-åtkomst på drivrutinsnivå.

B.3 Att skapa ett nytt projekt

1. Öppna Quartus och starta **New Project Wizard** (File → New Project Wizard).
2. Välj en arbetskatalog och ett projektnamn. Quartus använder projektnamnet som förvalt namn på toppnivåentiteten också, så låt det matcha din VHDL-entitets namn för att slippa skriva över det senare (för `or_gate.vhd`: döp projektet till `or_gate`).
3. Välj **Empty Project** som projekttyp. Kursen använder varken Quartus IP-katalog eller någon projektmall.
4. Hoppa över att lägga till filer i det här steget (avsnitt B.4 täcker det), om du inte redan vet exakt vilken fil det gäller; endera går bra.
5. På sidan för enhetsval, välj exakt den krets som sitter på DE0-CV: filtrera på familjen **Cyclone V**, och leta sedan upp och välj enheten 5CEBA4F23C7N.
6. Hoppa över sidan för EDA-verktyg (lämna den på "none"); kursen kör sina simuleringar direkt i GHDL i stället för genom Quartus.
7. Avsluta guiden.

B.4 Att lägga till sin VHDL

- Lägg till källfilen: Project → Add/Remove Files in Project..., bläddra fram din .vhd-fil och lägg till den.

- Har konstruktionen mer än en fil (en toppnivåentitet plus en eller flera delkomponenter), lägg till dem allihop. Quartus löser upp instansieringshierarkin automatiskt så länge varje fil finns i projektet.
- Bekräfta att toppnivåentiteten är rätt satt: Project → Set as Top-Level Entity, med din huvudfil markerad. För en enfilskonstruktion sker det automatiskt.

B.5 Att tilldela pinnar i Pin Planner

Varje port i din entitet behöver en fysisk pinne på FPGA:n innan Quartus kan producera en fungerande programmeringsfil; annars finns det ingen väg från konstruktionens portar till en verklig strömbrytare eller lysdiod på kortet.

1. Kör Processing → Start → Start Analysis \& Elaboration först, och öppna sedan Pin Planner via Assignments → Pin Planner. I ett alldeles nytt projekt är portlistan tom tills Quartus har elaborerat konstruktionen och känner till dess portar.
2. Var och en av entitetens portar dyker upp som en rad. Fyll i kolumnen **Location** med pinnbeteckningen för den strömbrytare eller lysdiod du vill använda.
3. **En strömbrytare eller en lysdiod är fritt val. En klocka är det inte.** För SW, KEY och LEDR går det bra att ta vilka som helst som är bekväma: konstruktionen bryr sig inte om vilken strömbrytare som driver vilken ingång, bara att varje port når en. CLOCK_50 är en bestämd pad på kretsen och går inte att flytta. På DE0-CV är den **PIN_M9**, oscillatoren på 50 MHz. Varje konstruktion från kapitel 3 och framåt är klockad, så från det kapitlet är det här det enda pinnnummer som är värt att kunna utan att slå upp. Bekräfta det mot ditt eget korts filer (steg 4) innan du förlitar dig på det.
4. **Skriv inte av pinntabellen för hand.** Tio LEDR-pinnar avskrivna ur en PDF är tio chanser att kasta om två siffror, och felet det ger är det otäckaste i avsnitt B.8: en ren kompilering som betar sig obegripligt.
 - DE0-CV:s system-CD innehåller en DE0_CV.qsf där varje kortpinne redan är namngiven och tilldelad.
 - Assignments → Import Assignments..., peka ut den filen, och varje CLOCK_50-, SW[n]-, KEY[n]-, LEDR[n]- och GPIO_0[n]-tilldelning landar på en gång.
 - Döp din entitets portar så att de matchar de signalnamnen så återstår ingenting att tilldela. Det är därför kapitel 19:s bring-up döper sina portar till CLOCK_50, KEY, SW, LEDR och GPIO_0 i stället för till något snyggare.
 - Pin Planner (och kortets pinntabeller i användarmanualen) är fortfarande vägen att kontrollera en enskild pinne, eller tilldela en som .qsf-filen inte täcker.
5. Sätt kolumnen **I/O Standard** om Quartus inte härleder ett vettigt förval för pinnen du valde. DE0-CV:s allmänna I/O är 3,3 V LVTTTL/LVCMOS. Att importera kortets .qsf (steg 4) sätter det här också, vilket är ännu ett skäl att föredra den vägen.

6. Spara tilldelningarna (de skrivs in i projektets .qsf-fil). Varje ändring här kräver omkompilering (avsnitt B.6) innan den får effekt; Quartus lappar inte i efterhand en .sof som kompilerades före ändringen.

B.5.1 En klocka behöver ett tidskrav också

Att tilldela CLOCK_50 en pinne talar om för Quartus *var* klockan kommer in. Det säger inte hur fort den går, och tills du säger det har tidsanalysen ingenting att kontrollera mot och rapporterar en obegränsad konstruktion. Lägg till en .sdc-fil i projektet med en rad:

```
create_clock -name CLOCK_50 -period 20.000 [get_ports CLOCK_50]
```

20 ns är 50 MHz. Med den på plats blir tidsanalysen i Processing → Start Compilation meningsfull och rapporten ger dig ett verkligt Fmax, vilket är siffran avsnitt 5.4 skickar dig hit för att läsa. Utan den saknas Fmax, eller är inte att lita på.

B.6 Att kompilera konstruktionen

Processing → Start Compilation (eller ► i verktygsraden) kör hela flödet: analys och syntes, placering och routning, tidsanalys, och hopsättning till en programmeringsfil.

Läs kompileringsrapporten när den är klar, i synnerhet **Warnings**: en oansluten port, en ofullständig känslighetslista, eller en latch som härletts där du menade kombinatorik dyker alla upp där som varningar snarare än fel. De är lätta att missa om man bara kontrollerar att boken blev grön.

En lyckad kompilering producerar en .sof-fil (SRAM Object File) i projektets output_files-katalog. Det är den avsnitt B.7 laddar ner till kortet. Den är flyktig: den konfigurerar FPGA:ns SRAM direkt och försvinner när strömmen bryts. Kursen har inget steg som bränner configurationen permanent.

B.7 Att programmera DE0-CV

1. Anslut DE0-CV till datorn över USB och slå på det.
2. Öppna Programmer: Tools → Programmer.
3. Välj USB-Blastern under **Hardware Setup**. Den ska hittas automatiskt om drivrutinen installerades rätt (annars avsnitt B.8).
4. Se till att .sof-filen från avsnitt B.6 står i listan och att dess kryssruta **Program/Configure** är ibockad. Står den inte där, använd Auto Detect eller lägg till filen för hand.
5. Klicka **Start**. Förloppsindikatorn ska nå 100 % och rapportera att det gick bra.
6. Testa konstruktionen direkt på kortet: växla de strömbrytare du tilldelade i avsnitt B.5 och bekräfta att lysdioden följer entitetens sanningstabell.

B.8 Felsökning

USB-Blastern syns inte i Programmer. Kontrollera Enhetshanteraren i Windows efter en okänd enhet eller en med drivrutinsfel, och peka manuellt ut USB-Blaster-drivrutinen under Quartus installationskatalog.

Kortet går inte att programmera, utan något tydligt fel. Slå av Memory integrity under Core isolation på Windows 11 (avsnitt B.2.1). Det är den enskilt vanligaste orsaken när drivrutinen väl är installerad.

Installeraren startar inte. Kontrollera Smart App Control under Windows Säkerhet (avsnitt B.2.1).

Kompileringen går igenom men konstruktionen betar sig fel på kortet. Kontrollera tilldelningarna i Pin Planner (avsnitt B.5) *innan* du misstänker din VHDL. En omkastad eller saknad pinntilldelning ger ett kort som kompilerar rent och betar sig obegripligt, eftersom det inte är kopplat som du tror.

Fitter rapporterar otilldelade pinnar som fel. Vissa projektinställningar behandlar det som fatalt snarare än som en varning. Tilldela antingen varje port i Pin Planner, eller, för en kvarglömd oanvänd port tidigt i utvecklingen, säg åt Quartus hur resten ska behandlas: Assignments → Device → Device and Pin Options... → Unused Pins. För DE0-CV är **"As input tri-stated (with weak pull-up)"** det säkra värdet. Förvalet i vissa flöden driver oanvända pinnar låga, vilket på ett kort där pinnar når kontaktlistor och kringkretsar är ett sätt att slåss med en annan drivare. Lämna inte portar tyst otilldelade och förvänta dig att Quartus gissar.

B.9 Att programmera från Linux och från WSL

Avsnitt B.8 är Windows. På Linux är drivrutinen ingen drivrutin alls: Programmer talar med USB-Blastern genom libusb, och behöver rättighet att göra det.

Linux, nativt

Ur lådan visar Programmer ingen hårdvara, eftersom enhetsnoden ägs av root. Ge den en udev-regel med raderna nedan i /etc/udev/rules.d/51-usbblaster.rules:

```
# Altera USB-Blaster
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6001", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6002", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6003", MODE="0666"
# USB-Blaster II
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6010", MODE="0666"
SUBSYSTEM=="usb", ATTR{idVendor}=="09fb", ATTR{idProduct}=="6810", MODE="0666"
```

Ladda om reglerna och dra sedan ur och i kortet igen:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Quartus cachar sin hårdvarulista i en bakgrundsdaemon, så om Programmer fortfarande inte visar något, starta om den:

```
killall jtagd 2> /dev/null
jtagconfig # ska nu lista USB-Blastern och kedjan av enheter
```

jtagconfig är den snabbaste vägen att skilja ett rättighetsproblem från ett kabelproblem: listar den kortet är hårdvaran i sin ordning och problemet sitter inuti Quartus; gör den det inte, stanna och lös det innan du rör Programmer.

WSL2

Det här är fallet kursens egen verktygskedja går rakt in i, så det är värt att säga rent ut: **WSL2 har ingen USB-genomkoppling**. Kortet är osynligt för en WSL2-installation hur många udev-regler du än skriver, eftersom kärnan aldrig ser enheten. Det finns ingen inställning att ändra. Två vägar ut:

- **Kör Quartus nativt på Windows** (rekommenderas). Behåll WSL för GHDL, som inte behöver någon hårdvara alls. Dina .vhd-filer ligger i Windows-filsystemet och båda verktygen läser dem. Det är vägen kursen förutsätter, och den med minst rörliga delar.
- **Koppla vidare enheten med usbipd-win**, om du specifikt vill ha Quartus inuti WSL. Från en PowerShell som administratör på Windows-sidan:

```
winget install usbipd
usbipd list # hitta USB-Blasterns BUSID
usbipd bind --busid <BUSID> # en gång per enhet, består
usbipd attach --wsl --busid <BUSID>
```

Enheten dyker då upp inuti WSL och udev-regeln ovan gäller. Den måste kopplas om efter varje urdragnings och varje omstart av WSL, och Quartus grafiska gränssnitt behöver dessutom WSLg eller en X-server. Görbart, men tre saker till som kan gå fel på en redovisningsdag.

Gruppprojektet

Det här är projektets specifikation, samlad på ett ställe: arbetsformen, strukturen, modulernas gränssnitt, testplanen och bedömningen. Kapitel 10 till 19 är genomgången; det här är kontraktet, och där de två skiljer sig gäller det som står här.

Uppgiften är att konstruera, simulera och köra en förenklad CAN-kontroller i VHDL, och göra den nåbar från en mikrokontroller över SPI. Parallellklassen skriver C++-drivrutinen mot samma register, så det som byggs är ena halvan av ett riktigt hårdvaru- och mjukvarusystem, inte en övning som slutar i simulatoren.

När projektet är klart har gruppen implementerat en **bittimer** som producerar sampel- och bitslutspulser, en **CRC-15-motor** som både beräknar och verifierar checksumman, ett **sändningsskiftregister** som serialiserar och sätter in stoppbitar, ett **mottagningsskiftregister** som deserialiserar och tar bort dem, en **CAN-kontroller** vars tillståndsmaskin sekvenserar ramens fält och hanterar arbitrering, en **registerbank** som översätter kontrollerns pulser till pollbara register, en **SPI-brygga** som gör registren nåbara utifrån, och en **toppnivå** som binder ihop alltihop till en nod en mikrokontroller kan prata med.

C.1 Arbetsform

Projektet genomförs i grupper om **4–5 studenter**. Gruppen ansvarar gemensamt för hela projektet, och arbetssättet följer industriell praxis.

C.1.1 Git och GitHub

- Skapa ett privat repository och bjud in alla gruppmedlemmar samt handledaren som collaborator.
- Arbeta i feature-branchar. En branch per modul är en rimlig utgångspunkt.
- Committa ofta, med meddelanden som säger vad ändringen gör och inte bara att den finns.
- **Pusha aldrig direkt till main.** All kod når main genom en Pull Request. Skydda branchen i GitHubs inställningar så att regeln inte beror på att alla kommer ihåg den.

C.1.2 Kodgranskning

- Varje PR granskas av **minst en annan gruppmedlem** innan den mergas.
- Granskaren kontrollerar: att portordningen stämmer mot gränssnittstabellen, att modulens testbänk passerar, att inga värden är hårdkodade förbi `can_def`, och att varje entitets gränssnitt och varje viktig intern signal är kommenterad.
- Lägg granskningskommentarer i GitHub, inte i chatt eller muntligt. En granskning som inte finns skriftligt har inte hänt.
- En granskning som bara säger "ser bra ut" är ingen granskning. Ställ minst en riktig fråga, även när koden är bra: *varför* den gör som den gör är oftare intressant än *att* den gör det.

C.1.3 Arbetsfördelning

Alla ska skriva VHDL och alla ska granska VHDL. Fördelningen får vara ojämn i mängd, inte i art. Dokumentera fördelningen löpande i `CONTRIBUTORS.md` i repos rot.

C.2 Så här drivs projektet

Projektet har **inga milstolpar**. Varje kapitel från kapitel 10 och framåt anger ett *riktvärde*, formulerat som "ni bör vara ungefär här". Riktvärdena är inte krav och betygsätts inte.

Det är ett medvetet val och inte generositet. Efter `can_def.vhd` har de fyra lövmodulerna `bit_timer`, `crc15`, `tx_shift_reg` och `rx_shift_reg` **inga beroenden till varandra**, och var och en har sin egen utdelade testbänk. En grupp om 4–5 kan därför bygga dem parallellt, i vilken ordning som helst. Att låsa fast en ordning skulle bara hindra det.

Men beroendegrafen är inte platt, och det är värt att planera efter:

```

        meta_prev                                (läser inget paket alls)
can_def  → bit_timer, crc15, tx_shift_reg, rx_shift_reg (de fyra löven, parallella)
                                     ↳ can_controller (kap 16 sändning, kap 17 mottagning)
can_def  → register_bank                        (oberoende, byggbar från kap 10)
spi_def  → spi_reg_bridge                       (oberoende, byggbar från kap 10)
spi_slave                                (utdelad, skrivs inte av gruppen)
    can_controller + register_bank + spi_reg_bridge + spi_slave → can_spi_node (kap 19)
```

Två saker följer av den:

- **can_controller är den kritiska vägen.** Den har ingen intern parallellitet, den sträcker sig över två pass, och dess testbänk är en systemtestbänk som antingen går igenom eller inte — ingen delvis grön. Den som ska äga den bör börja läsa kapitel 16 tidigt, och gruppen bör räkna med att den tar mest tid av allt.
- **register_bank och spi_reg_bridge är två fullvärdiga arbetsspår som kan starta direkt.** Den ena behöver bara `can_def`, den andra bara `spi_def`, och båda har utdelade testbänkar.

Ingen behöver vänta på kontrollern för att ha något att göra, och en grupp som skjuter dem till kapitel 18 och 19 har lagt sitt tigtaste arbete sist.

Två hållpunkter är obligatoriska:

När	Vad
Kapitel 15	Kodgranskningsseminarium. Varje grupp visar en mergad PR och en granskning gruppen skrivit, och berättar om en sak i arbetssättet som inte fungerat.
Kapitel 19	Slutredovisning och inlämning. Gröna testbänkar, genomgång av <code>can_spi_node</code> , bring-up på hårdvara så långt gruppen kommit, och repots historik.

Det som bedöms är slutresultatet och arbetssättet: designen, Git-historiken, PR:erna och granskningarna. **Inte takten.** En grupp som ligger efter halvvägs igenom projektet och lämnar in en komplett, välgranskad nod har gjort projektet rätt.

C.3 Projektstruktur

Två platta kataloger, med modulerna bredvid de testbänkar som kontrollerar dem:

```
controller/  can_def.vhd, meta_prev.vhd, bit_timer.vhd, crc15.vhd,
              tx_shift_reg.vhd, rx_shift_reg.vhd, can_controller.vhd
              + de sex utdelade *_tb.vhd
bridge/      register_bank.vhd, spi_reg_bridge.vhd, can_spi_node.vhd
              + utdelade spi_slave.vhd, spi_def.vhd,
              register_bank_tb.vhd, spi_reg_bridge_tb.vhd
ci/          build_project.sh, kopierad från kursrepot
Makefile     med målet build-project
CONTRIBUTORS.md
.gitignore
```

Gränsen mellan de två katalogerna går vid protokollet: allt i `controller/` känner till CAN och ingenting om SPI, och **ingen modul i någondera katalogen känner till båda**. `register_bank` är mitten: den känner varken CAN-ramformatet eller SPI utan bara register-semantik, och läser `can_def` enbart för porttyperna.

De utdelade filerna hämtas från kursrepot och läggs in oförändrade. `spi_slave.vhd` skriver gruppen inte: den är transport, inte kursens ämne, och den läses i kapitel 18 i stället.

C.4 Namnkonventioner och kodstil

All kod och alla kommentarer i koden skrivs på **engelska**. Instruktionerna är på svenska.

- Modul- och signalnamn i `snake_case`, till exempel `timer_enable`.
- Konstanter i versaler, till exempel `TICKS_PER_BIT`.
- `std_logic` och `std_logic_vector` för allt som är ledningar; `natural` för räknare.

- Inga hårdkodade värden. Allt som är protokoll ligger i `can_def`.
- Kommentera varje entitets gränssnitt och varje viktig intern signal.
- Håll gränssnitten stabila. Testbänkarna är utdelade och binder positionellt.

Positionell portbindning. Varje utdelad testbänk binder till sin modul **positionellt**, och det gör varje instansiering inuti `can_controller` också. Kontraktet är alltså **ordningen och typerna** på portarna, exakt som tabellerna i avsnitt C.6 och C.7 listar dem. Portarnas *namn* är gruppens egna.

Byt plats på två portar av samma typ och ingenting klagar vid analys; konstruktionen betar sig bara fel i simuleringen. Det är priset för att gränssnittstabellen får vara enda sanningskälla.

C.5 Det delade paketet `can_def`

Skrivs i kapitel 10, i `controller/can_def.vhd`, och läses av allt annat.

Konstanter: `CLOCK_FREQ_HZ`, `BIT_RATE_HZ`, `TICKS_PER_BIT`, `SAMPLE_TICK`, `ID_WIDTH`, `DLC_WIDTH`, `DLC_MAX`, `CRC_WIDTH`, `DATA_WIDTH`, `MAX_RUN`, `CRC_POLY`.

Subtyper: `byte_t`, `id_t`, `crc_t`, `dlc_t`, `data_t`.

Riktvärden för ett DE0-CV med 50 MHz systemklocka och CAN på 1 Mbit/s:

Konstant	Värde	Kommentar
<code>CLOCK_FREQ_HZ</code>	<code>50_000_000</code>	Systemklockan.
<code>BIT_RATE_HZ</code>	<code>1_000_000</code>	1 Mbit/s.
<code>TICKS_PER_BIT</code>	<code>CLOCK_FREQ_HZ / BIT_RATE_HZ</code>	50 klockcykler per CAN-bit. Räkna ut den, skriv den inte.
<code>SAMPLE_TICK</code>	<code>TICKS_PER_BIT * 7 / 10</code>	Sampelpunkten, ungefär 70 % in i bitperioden.
<code>ID_WIDTH</code>	11	Standardidentifierare.
<code>DLC_WIDTH</code>	4	DLC är fyra bitar.
<code>DLC_MAX</code>	8	Högst åtta databytes.
<code>DATA_WIDTH</code>	<code>DLC_MAX * 8</code>	64 bitar nyttolast.
<code>CRC_WIDTH</code>	15	CAN:s CRC-15.
<code>MAX_RUN</code>	5	Femregeln för bitstoppning.
<code>CRC_POLY</code>	<code>"100010110011001"</code>	Motsvarar $x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$.

Att `TICKS_PER_BIT` och `SAMPLE_TICK` är *uträknade* och inte inskrivna spelar roll: byter någon bithastighet ska allt följa med.

C.6 Modulerna i `controller/`

Kapitlen beskriver modulerna i detalj; det här är portordningen samlad, eftersom det är den som är kontraktet.

meta_prev (kapitel 12)

En generic, WIDTH (natural := 1), och tre portar: clock (in), async_in (in, std_logic_vector(WIDTH-1 downto 0)), sync_out (out, samma typ). Dubbelvippsynkroniseraren från kapitel 4, skriven en gång med en breddgeneric. Den behöver ingen reset: efter två klockcykler är innehållet ändå ersatt. Instansieras två gånger inuti can_controller, för reset_n och för rx_bus.

bit_timer (kapitel 12)

#	Port	Riktning	Typ	Betydelse
1	clock	in	std_logic	Systemklocka.
2	reset_s2_n	in	std_logic	Synkroniserad, aktiv låg reset.
3	enable	in	std_logic	Räknar medan '1', håller medan '0'.
4	resync	in	std_logic	Tvingar tillbaka räknaren till tick 0.
5	sample	out	std_logic	Puls vid sampelpunkten.
6	bit_done	out	std_logic	Puls vid bitperiodens slut.

En räknare från 0 till TICKS_PER_BIT-1 som pulsar sample vid SAMPLE_TICK och bit_done vid periodens slut. resync används en gång per ram, vid SOF; däremellan löper timern fritt.

crc15 (kapitel 13)

#	Port	Riktning	Typ	Betydelse
1	clock	in	std_logic	Systemklocka.
2	reset_s2_n	in	std_logic	Synkroniserad, aktiv låg reset.
3	clear	in	std_logic	Nollställer registret utan reset.
4	enable	in	std_logic	Integrerar data denna flank.
5	data	in	std_logic	Aktuell bit.
6	crc	out	crc_t	Registrets nuvarande värde.
7	valid	out	std_logic	'1' när registret är idel nollor.

Ett 15-bitars register med återkoppling. Samma motor både genererar och kontrollerar: sändaren matar in sina bitar och läser ut crc; mottagaren matar in samma bitar *plus* den mottagna CRC:n, och valid går hög om allt stämmer. Ingen lägesomkoppling behövs. clear behövs ändå: en avbruten ram lämnar skräp i registret.

tx_shift_reg (kapitel 14)

#	Port	Riktning	Typ	Betydelse
1	clock	in	std_logic	Systemklocka.
2	reset_s2_n	in	std_logic	Synkroniserad, aktiv låg reset.
3	load	in	std_logic	Laddar data/bit_count och lägger ut första biten samma cykel.
4	data	in	byte_t	Gruppen, med sina giltiga bitar högst.
5	bit_count	in	std_logic_vector(3 downto 0)	Antal giltiga bitar, 1–8.
6	shift	in	std_logic	En puls per CAN-bit, från bit_timer.bit_done.
7	tx_bit	out	std_logic	Biten som ska drivas nästa bitperiod.
8	stuff	out	std_logic	'1' på just den skiftning som lägger ut en stoppbit.
9	done	out	std_logic	Pulsar när gruppen är helt utsänd.
10	bit_valid	out	std_logic	'1' den cykel en ny bit lades ut.

Skiftar ut MSB först, räknar lika bitar i rad, och sätter in en stoppbit med motsatt värde efter MAX_RUN lika. **Stoppningstillståndet lever kvar över omladdningar:** regeln gäller bitströmmen, inte den enskilda gruppen. stuff och bit_valid finns för can_controllers skull: CRC:n matas på bit_valid and not stuff, eftersom en stoppbit inte ingår i checksumman.

rx_shift_reg (kapitel 15)

#	Port	Riktning	Typ	Betydelse
1	clock	in	std_logic	Systemklocka.
2	reset_s2_n	in	std_logic	Synkroniserad, aktiv låg reset.
3	sample	in	std_logic	En puls per CAN-bit, från bit_timer.sample.
4	rx_bus	in	std_logic	Bussnivån, giltig vid sampelpunkten.
5	bit_count	in	std_logic_vector(3 downto 0)	Antal <i>riktiga</i> bitar att samla in, 1–8.

#	Port	Riktning	Typ	Betydelse
6	enable	in	std_logic	Avgör om sample-pulser behandlas.
7	data	out	byte_t	Den insamlade gruppen, högerjusterad.
8	valid	out	std_logic	Pulsar när bit_count riktiga bitar samlats in.
9	stuff_error	out	std_logic	Pulsar om en väntad stoppbit inte inverterar.
10	real_bit	out	std_logic	Den avstoppade bit som accepterades.
11	real_bit_valid	out	std_logic	'1' när en riktig bit accepterades, '0' för en kastad stoppbit.

Sändarens spegelbild. Efter MAX_RUN lika bitar i rad *måste* nästa bit vara den motsatta; är den det kastas den och räkningen börjar om, är den det inte är det ett stuff_error.

can_controller (kapitel 11, 16, 17)

Toppnivån för CAN-halvan, och det enda block som känner till ramformatet. Femton portar, i exakt den här ordningen, alla ingångar först:

1 clock	4 tx_id	7 rx_bus	10 bus_en	13 rx_data
2 reset_n	5 tx_dlc	8 tx_done	11 rx_id	14 rx_valid
3 tx_req	6 tx_data	9 tx_bus	12 rx_dlc	15 error

Ramsekvensen en sändning går igenom: SOF (1 bit), arbitreringsfältet (11 bitars identifierare + RTR), kontrollfältet (IDE, r0, DLC = 6 bitar), datafältet (0–64 bitar enligt DLC), CRC (15 bitar + avgränsare), ACK (lucka + avgränsare) och EOF (7 bitar). Varje fält får ett eget laddningstillstånd före sitt skifttillstånd.

Bussen modelleras som öppen dränering: bus_en avgör om noden driver alls, tx_bus vad den driver. Bussvärdet är '1' bara om ingen nod drar ned det. Under ACK-luckan släpper den sändande noden bussen, och en mottagare med godkänd CRC drar ned den. **Sändaren läser aldrig tillbaka ACK-luckan**; en ram som ingen kvitterar fullbordas precis som en kvitterad, och tx_done pulsar ändå.

Arbitrering: under arbitreringsfältet jämförs varje sänd bit med det som faktiskt ligger på bussen. Skickar noden en recessiv bit och läser tillbaka en dominant har den förlorat: den sätter error, pulsar tx_done, lämnar sändarrollen och går till tomgång. Den **tar inte** emot ramen som vann.

Att tx_done pulsar även här är avsiktligt: pulsen betyder "sändningsförsöket är över och porten är din igen", inte "ramen kom fram". error säger vilketdera. Utan den pulsen har en anropare ingen flank att vänta på, och registerbanken får aldrig tillbaka sin TX-klar-bit.

error går hög av fyra orsaker: förlorad arbitrerings, `stuff_error`, misslyckad CRC-kontroll, och en mottagen fjärrram. Den ligger kvar tills nästa sändning eller mottagning startar. `tx_done` och `rx_valid` är pulser på exakt en klockcykel, vilket är precis det som gör registerbanken nödvändig.

C.6.1 Kända begränsningar som drivrutinen märker av

Förenklingarna mot riktig CAN listas i sin helhet i avsnitt 19.7, och från protokollhålllet i kapitel 7 i *CAN: bussen, ramen och kontrollern*. Tre av dem syns ändå ut i registerkartan och måste därför stå här också, eftersom parallellklassen skriver kod mot dem — det är samma tre som *CAN: bussen, ramen och kontrollern* lyfter fram, och den boken läser båda klasserna:

- **En förlorad arbitrerings går inte att skilja från en lyckad sändning på STATUS bit 0.** Båda ger tillbaka biten, eftersom kontrollern pulsar `tx_done` också när den avbryter. Det är avsevärt: biten betyder ”porten är fri”, inte ”ramen kom fram”. En drivrutin som vill veta utfallet läser `ERROR_FLAGS` efter att bit 0 kommit tillbaka.
Det här var en gång en riktig bugg, och det är värt att veta varför den inte är det längre. Sattes bit 0 enbart av `tx_done`, och pulshade `tx_done` bara vid EOF, blev biten liggande låg efter varje förlorad arbitrerings och noden kunde inte sända igen förrän den nollställdes på resetknappen — på en tvånodsbuss alltså i normal drift. Tre saker gör att den inte kan uppstå igen, och de är medvetet överlappande: kontrollern pulsar `tx_done` även vid avbrott (kapitel 17), banken sätter bit 0 också på en stigande flank av `error` medan biten är låg (kapitel 18), och `TX_ABORT` finns som nödutgång. `can_controller_tb` fall 2 och `register_bank_tb` fall 10–11 kontrollerar de tre.
- **error är en bit för fyra orsaker.** Drivrutinen kan se att något gick fel, aldrig vad. Om-sändningslogik som skiljer på förlorad arbitrerings och trasig CRC går inte att skriva mot det här gränssnittet.
- **Ingen buffring på mottagarsidan.** Kontrollern håller en ram. Kommer nästa innan den förra lästs ut skrivs den över. Registerbanken vidarebefordrar begränsningen i stället för att dölja den; se bilaga D.

C.7 Modulerna i bridge/

register_bank (kapitel 18)

Sexton portar, i den här ordningen:

1 clock	5 reg_wdata	9 rx_data	13 tx_id
2 reset_s2_n	6 tx_done	10 rx_valid	14 tx_dlc
3 reg_index	7 rx_id	11 error	15 tx_data
4 reg_write	8 rx_dlc	12 reg_rdata	16 tx_req

Översätter kontrollernas encykelspulser till klibbiga, pollbara nivåer, och registerskrivningar till händelser. Semantiken i sin helhet står i bilaga D; den är kontraktet mot parallellklassen och får inte ändras ensidigt. `reg_rdata` är **kombinatorisk**: att låsa ett läst värde är bryggans ansvar, inte bankens.

spi_reg_bridge (kapitel 19)

Tio portar, i den här ordningen: `clock`, `reset_s2_n`, `ss_active`, `rx_data`, `rx_valid`, `reg_rdata`, `tx_data`, `reg_addr`, `reg_wdata`, `reg_write`.

Implementerar transaktionen ur bilaga D: en kommandobyte plus fyra databytes, alltid exakt fem. Vid läsning **läses registervärdet en gång** i slutet av kommandobyten; vid skrivning verkställs ingenting förrän femte byten är hel. Går `ss_active` låg dessförinnan avbryts transaktionen helt, utan sidoeffekter. `reg_addr` **håller** det avkodade indexet tills nästa kommandobyte avkodar ett nytt.

can_spi_node (kapitel 19)

Nio portar, i den här ordningen:

1 <code>clock</code>	4 <code>mosi</code>	7 <code>rx_bus</code>
2 <code>reset_n</code>	5 <code>ss</code>	8 <code>tx_bus</code>
3 <code>sclk</code>	6 <code>miso</code>	9 <code>bus_en</code>

Rent strukturell toppnivå. Instansierar `spi_slave`, `spi_reg_bridge`, `register_bank` och `can_controller`, plus en `meta_prev` för reset. Ingen funktionell logik här.

Ordningen är SPI:s fyra ledningar som en grupp, `sclk/mosi/ss/miso`, och därefter CAN-bussens tre. `miso` ligger alltså bland SPI-portarna och inte hos de andra utgångarna, så att de fyra pinnarna står i samma ordning som i kopplingstabellen. Det spelar roll: `can_spi_node_tb` binder positionellt som allt annat, och `miso` och `rx_bus` har samma typ, så en förväxling av dem passerar analysen utan ett ord och ger en nod som aldrig svarar.

C.8 Testplan

Gruppen skriver inga testbänkar i det här projektet. Nio är utdelade, och de är kontraktet: sju modultestbänkar och två systemtestbänkar.

Testbänk	Kontrollerar
<code>meta_prev_tb</code>	Två vippors fördröjning, för både en bit och en vektor.
<code>bit_timer_tb</code>	<code>sample</code> och <code>bit_done</code> vid rätt tick; <code>resync</code> ; <code>enable</code> .
<code>crc15_tb</code>	CRC-värden för kända sekvenser; <code>valid</code> ; <code>clear</code> .
<code>tx_shift_reg_tb</code>	MSB-först skiftning, stoppning efter fem lika, <code>done</code> -tajmingen.
<code>rx_shift_reg_tb</code>	Deserialisering, avstoppning, <code>stuff_error</code> , <code>real_bit</code> .
<code>register_bank_tb</code>	Elva fall, från resettillståndet till <code>TX_ABORT</code> och de två vägarna tillbaka till TX klar.
<code>spi_reg_bridge_tb</code>	Fyra fall: en skrivning, en läsning byte för byte, ett avbrott på <code>ss_active</code> , och en kommandobyte med reserverad bit satt.

`can_controller_tb` sätter två noder på en gemensam wired-AND-buss och kontrollerar att den ena tar emot den andras ram, och att arbitreringslösa lösningar uppträder rätt när båda sänder samtidigt. Den hoppas över av bygget tills kontrollern har en mottagarväg (bilaga A).

`can_spi_node_tb` gör samma sak ett lager ut: två kompletta noder på samma buss, men varje nod driven av en modellerad SPI-master på sina fyra pinnar. Den spelar upp protokollspecifikationens genomarbetade exempel genom hela kedjan och följer en ram från en SPI-skrivning på den ena noden till en SPI-läsning på den andra. Sju fall: resettillståndet, det genomarbetade exemplet, ogiltiga kommandobytes och reserverade index, `TX_ABORT`, avbrottsregeln, extra bytes under samma SS, och till sist sändningen och mottagningen.

Det är den **enda** testbänk i kursen som driver `sclk`, `mosi` och `ss` som riktiga vågformer. `spi_reg_bridge_tb` matar bryggan färdiga bytes och rör aldrig en pinne, så allt mellan ledningen och byten — synkroniserarna, flankdetekteringen på SCK, `ss_active`-ramningen — provas här eller ingenstans.

C.8.1 Egen verifiering

`can_spi_node_tb` är snäll mot tidskraven med flit: den håller 200 ns tyst kring varje SS-flank, där protokollet kräver minst 60 ns. Det är rätt för en testbänk som ska skilja logikfel från tidsfel, men det betyder också att **ingen** utdelad testbänk pressar specifikationens tidsminima.

Det är den verifiering projektet ber gruppen göra själv: kopiera `can_spi_node_tb`, krymp den tysta perioden ner mot 60 ns, och ta reda på vid vilket värde noden slutar svara och varför. Svaret ligger i `spi_slaves` synkroniserare, och det är det enda stället där kursen låter dig *mäta* en marginal i stället för att läsa den. Övning 19.4(b) ställer frågan i sin helhet.

C.8.2 Checklista före inlämning

- Alla nio utdelade testbänkar passerar, `can_controller_tb` och `can_spi_node_tb` inräknade. **Kontrollera att de verkligen kördes och inte hoppades över**, se rutan nedan.
- `make build-project` går grönt från ett rent utcheckat repo.
- Portordningen stämmer mot tabellerna i avsnitt C.6 och C.7.
- Inga värden hårdkodade förbi `can_def`.

- Varje entitets gränssnitt är kommenterat.
- CONTRIBUTORS.md är aktuell.
- main innehåller inga commits som inte gått via en granskad PR.

Överhoppad är inte godkänd. can_controller_tb grindas på om can_controller.vhd innehåller kapitel 17:s CRC-grindning, letad upp med en textsökning på de interna signalnamn kapitlet använder. Skriver ni en korrekt mottagarväg med andra namn rapporteras testbänken som *skipped*, och eftersom överhoppad aldrig är underkänd ser checklisten uppfyllt ut medan kursens pass/fail-testbänk aldrig kördes. Kör därför

```
CI_BUILD_ALL=1 make build-project
```

minst en gång före inlämning, och lämna in det utfallet.

C.9 Examination och betyg

Kursen examineras genom **ett gruppprojekt** och **två praktiska tentamina**. Totalt 100 poäng.

Moment	Var	Form	Poäng
Praktisk tentamen 1, sekvensnät	kapitel 9	Individuell, 3 h	25
Gruppprojekt, en CAN-nod i VHDL	kapitel 10–19	Grupp om 4–5	50
Praktisk tentamen 2, CAN-moduler	kapitel 20	Individuell, 3 h	25

Betyg	Krav
IG	Under 50 poäng, eller ett underkänt projekt.
G	Minst 50 poäng totalt och godkänt projekt.
VG	Minst 80 poäng totalt och väl godkänt projekt.

Projektet är alltså en spärr i båda riktningarna. Det står för halva kursen, det är där det mesta av arbetet ligger, och ett underkänt projekt går inte att kompensera med två starka tentor.

C.9.1 Projektets 50 poäng

Del	Poäng	Vad som bedöms
Designen	30	Modulerna fungerar enligt specifikation och testbänkarna passerar.
Arbetssättet	12	Git-historik, PR:er, granskningarnas kvalitet, seminariet, CONTRIBUTORS.md.
Redovisningen	8	Slutredovisningen: att gruppen kan förklara vad den byggt och varför.

Betyget sätts på gruppen. Är insatsen påtagligt ojämn kan enskilda betyg justeras; det är den enda anledningen till att `CONTRIBUTORS.md` ska hållas aktuell löpande och inte skrivas kvällen före.

Godkänt projekt kräver en **komplett nod**: alla nio utdelade testbänkar ska passera, alltså `CI_BUILD_ALL=1 make build-project` grönt rakt igenom. Kontrollern ensam räcker inte, och skälet ligger efter kursen: noden följer med till CAN-labben i nästa kurs i inbyggda system, där en drivrutin ska nå den över SPI. En kontroller utan registerbank och brygga går inte att nå från någon drivrutin alls, och en nod vars delar fungerar var för sig men inte ihop går inte heller att nå.

Väl godkänt kräver dessutom den egna verifieringen i avsnitt C.8.1, att koden är genomgående dokumenterad och stilistiskt konsekvent, och att granskningskulturen syns i repots historik.

Bring-up på riktig hårdvara bedöms **inte**. Ett trasigt kit eller en trasig koppling får inte kosta poäng. Allt som bedöms går att köra i GHDL på en vanlig laptop.

C.9.2 De praktiska tentorna

Båda är individuella, tre timmar, vid datorn med GHDL. Formen är densamma i båda: fristående uppgifter, oberoende av varandra och ordnade ungefär efter stigande svårighet.

De flesta uppgifterna är **skrivuppgifter**: en utdelad, självkontrollerande testbänk och en beskrivning av modulen den driver, med portordning och beteende. Kapitel 20 har därutöver **läsuppgifter**: ett tidsdiagram eller en SPI-transaktion att tolka på papper. De har ingen testbänk, av precis det skäl som gör dem värda att ställa — att läsa en vågform är en färdighet i sig, och den examineras inte av att skriva VHDL. De rättas mot ett facit, och delvis rätt svar ger delpoäng på samma sätt som delvis korrekt logik gör.

Delvis korrekt logik ger delpoäng: lämna aldrig in en tom fil för att modulen inte blev klar, och samma sak gäller ett halvt uttolkat tidsdiagram. Rättningen läser koden, inte bara testbänkens utfall; en modul som passerar genom att göra något annat än det som efterfrågades ger inte full poäng.

Kapitel 9 omfattar kapitel 1 till 8, med tyngdpunkt på sekvensnät: register, räknare, skiftregister, timers, synkroniserare och enkla Mooremaskiner. Ett övningsprov delas ut i förväg. Kapitel 20 omfattar projektets moduler: bittajming, CRC, bitstoppling åt båda hållen, ramsekvensering, registersemantik och SPI-transaktionen.

C.9.3 Hjälpmedel

Det här avsnittet är den **auktoritativa** listan. Står något annat någon annanstans gäller det som står här.

Tillåtet vid båda tentorna:

- Allt kursmaterial: den här boken, föreläsningarnas README och samtliga appendix. Vid kapitel 20 hör registerkartan och protokollspecifikationen (bilaga D) förstås dit; vid kapitel 9 finns de också tillgängliga, men examineras inte.

- All VHDL du själv skrivit under kursen, och vid kapitel 20 även gruppens projektkod.
- Egen dator med GHDL, och CircuitVerse. Båda tentorna: att rita en krets innan man skriver den är ofta snabbare.

Inte tillåtet:

- **Kommunikation med andra**, i någon form.
- **Verktyg som genererar eller förklarar VHDL åt dig**: språkmodeller, kodassistenter och liknande, oavsett om de skriver koden eller bara resonerar om den. Gränsen går inte vid vem som trycker på tangenterna utan vid vems förståelse som prövas.
- **Sökning på nätet efter lösningar**. CircuitVerse körs i en webbläsare, så datorn är uppkopplad, och det förutsätter att du håller dig till kursmaterialet av egen kraft. Att slå upp GHDL:s eller VHDL:s dokumentation är tillåtet; att söka efter en färdig modul är det inte.

Är du osäker på om något är tillåtet: fråga före tentan, inte under.

Att gruppens projektkod är tillåten vid kapitel 20 är ett medvetet val. Ni byggde den. Uppgifterna är i stället skrivna som varianter och reparationer, så att de skiljer sig från projektets moduler på just den punkt som kräver att man förstått dem.

C.9.4 Omexamination och frånvaro

Båda tentorna går att skriva om. Projektet lämnas in en gång, med möjlighet till komplettering efter besked från handledaren: en komplettering rättas mot godkänt, inte mot väl godkänt.

Kodgranskningsseminariet i kapitel 15 och slutredovisningen i kapitel 19 är projektets två obligatoriska hållpunkter. Missad närvaro tas igen efter överenskommelse med handledaren. Övriga pass har ingen närvaroplikt; gruppprojektet har däremot en gruppdynamik som inte tål att någon uteblir, vilket är ett annat och starkare skäl att komma.

C.10 Praktiska tips

- Rita tidsdiagrammet innan ni skriver skiftregistren. De flesta fel där är tajmingfel på en cykel, inte logikfel, och de syns på papper innan de syns i simulatorn.
- Börja med `bit_timer` och `crc15` om ni är osäkra på var ni ska börja. De är de enklaste, de beror inte på något, och de går att bli helt klar med.
- Läs de utdelade testbänkarna. De säger mer exakt vad varje modul lovar än någon prosa gör, och de är skrivna för att läsas.
- Testa stoppningen noggrant, åt båda hållen. Det är projektets vanligaste felkälla.
- Håll `can_spi_node` strukturell. Så fort det smyger sig in logik i toppnivån blir den omöjlig att testa isolerat.
- Prata med parallellklassen tidigt, och gör det om registerkartan snarare än om er kod.

Gemensam dokumentation. Två dokument delas med mjukvarugruppen och är kontraktet mellan klasserna: registerkartan (vad registren betyder) och protokollspecifikationen (hur de nås), båda återgivna i bilaga D. Ändras något i dem måste båda klasserna veta om det innan ändringen mergas. Dokumenten uppdateras *före* koden, aldrig efter.

Registerkartan och SPI-protokollet

Det här är kontraktet i projektets mitt, och det är skrivet för **två klasser samtidigt**: den här kursen bygger FPGA-sidan som en SPI-slav (kapitel 18 och 19), och parallellklassen bygger mikrokontrollersidan som master. Båda halvorna skrivs mot det här dokumentet, inte mot varandras kod. **Är en implementation och den här bilagan oense är implementationen fel.**

Kontraktet säger ingenting om CAN som protokoll; det förutsätter det. *CAN: bussen, framen och kontrollern* är beskrivningen, och dess kapitel 7 är den som förklarar varför det här lagret behövs alls: en kontrollern som signalerar med encykelspulser går inte att polla över en seriebuss.

Två saker hålls isär, och det är värt att veta vilken som är vilken. *Registerkartan* (avsnitt D.1 till D.4) säger **vad** registren betyder. *Transaktionsformatet* (avsnitt D.6 till D.8) säger **hur** de nås över ledningarna.

En äldre version av det här projektet lät drivrutinen nå registren via minnesmappad I/O på basadressen 0xFF200000. Det förutsätter en processor med en adressbuss ut mot FPGA-väven, vilket mikrokontrollern i det här systemet inte har. Registren nås över SPI, och kolumnen **Index** nedan, inte offseten, är det som står i kommandobyten.

D.1 Registren

Index	Register	Offset	Åtkomst	Beskrivning
0	STATUS	0x00	R	Bit 0: TX klar. Bit 1: RX giltig. Bit 2: Fel.
1	TX_ID	0x04	R/W	11-bitars sändar-ID (bitarna 10–0).
2	TX_DLC	0x08	R/W	Data Length Code (bitarna 3–0, värde 0–8).
3	TX_DATA_LO	0x0C	R/W	Sändardata byte 0–3 (byte 0 = MSB).
4	TX_DATA_HI	0x10	R/W	Sändardata byte 4–7 (byte 4 = MSB).
5	TX_SEND	0x14	W	Skriv 0x1 för att utlösa sändning.
6	RX_ID	0x18	R	Mottagen identifierare (bitarna 10–0).
7	RX_DLC	0x1C	R	Mottagen DLC (bitarna 3–0).
8	RX_DATA_LO	0x20	R	Mottagen data byte 0–3.
9	RX_DATA_HI	0x24	R	Mottagen data byte 4–7.
10	RX_ACK	0x28	W	Skriv 0x1 för att kvittera och tömma RX-bufferten.
11	ERROR_FLAGS	0x2C	R/W	Felregistret; skriv 0x0 för att nollställa.
12	TX_ABORT	0x30	W	Skriv 0x1 för att tvinga tillbaka TX klar efter en avbruten sändning.

Index 13–15 är reserverade: en läsning ger 0x00000000, en skrivning ignoreras.

Index är Offset / 4. Offseten finns kvar i tabellen därför att drivrutinens konstanter är skrivna i den formen, men det är indexet som går ut på ledningen.

D.2 STATUS

Bit	Betydelse	Sätts av	Nollställs av
0	TX klar	reset; kontrollerns puls tx_done; en stigande flank på error medan biten är låg; en accepterad skrivning till TX_ABORT	en accepterad skrivning till TX_SEND
1	RX giltig	kontrollerns puls rx_valid, som också låser RX_*	en skrivning av 0x1 till RX_ACK
2	Fel	en stigande flank på kontrollerns nivå error	en skrivning av 0x0 till ERROR_FLAGS

Poängen med hela lagret ligger i den tabellen. Kontrollern signalerar med encykelspulser, 20 ns vid 50 MHz. En drivrutin som pollar över SPI ser systemet några mikrosekunder i taget i bästa fall, så en puls är borta tusentals cykler innan nästa pollning kommer. Registerbanken gör om pulserna till **klibbiga, pollbara nivåer med uttrycklig nollställning**.

D.3 Semantik, register för register

- **TX_SEND** utlöser en enda tx_req-puls när ett värde med **bit 0 satt** skrivs och STATUS bit 0 är satt. Övriga bitar i det skrivna ordet ignoreras. Är STATUS bit 0 låg ignoreras skrivningen: en sändning pågår. En skrivning med bit 0 låg gör ingenting. Läsning ger 0x00000000, eftersom registret inte lagrar något utan *gör* något. Skrivningar är flankhändelser på det här lagret, så en upprepad skrivning är en upprepad begäran, aldrig en hållen nivå.
- **RX_ACK** nollställer STATUS bit 1. Kräver bit 0 satt; övriga bitar ignoreras. Läsning ger 0x00000000.
- **TX_ABORT** sätter STATUS bit 0 tillbaka till 1, och gör ingenting annat. Bara en skrivning med **bit 0 satt** gör något; läsning ger 0x00000000. Registret når inte kontrollern — en kontroller som avbrutit en sändning är redan tillbaka i vila och kan sända igen; det är bara bankens klubbiga bit som fastnat. Det här är nödutgången: i en riktig konstruktion ska den aldrig behövas, eftersom bit 0 redan har två automatiska vägar tillbaka. En drivrutin som märker att den behöver den har hittat en bugg värd att rapportera.
- **ERROR_FLAGS** är en lås-och-nollställ-på-noll-konstruktion, inte ett allmänt register: bara en skrivning där **hela ordet är noll** nollställer. 0x000000FF nollställer alltså inte, trots att bit 0 är satt. Bit 0 är låsningen och speglar STATUS bit 2; bitarna 31–1 läses som 0. Namnet är i plural av historiska skäl.
- **Maskning vid skrivning.** TX_ID lagrar bitarna 10–0 och TX_DLC bitarna 3–0; överflödiga bitar läses tillbaka som noll. En skrivning av 0xFFFFFFFF läses alltså tillbaka som 0x000007FF respektive 0x0000000F. TX_DATA_LO/HI lagrar alla 32 bitarna.
- **Ingen validering.** En TX_DLC på 0xF lagras och lämnas vidare till kontrollern som den är. Att avvisa dlc > 8 är drivrutinens ansvar. Att göra det på båda sidor är precis så de två kopiorna av regeln glider isär.
- **Ingen buffring på mottagarsidan.** Kommer en ny ram innan den förra kvitterats skrivs den över, och den nyaste vinner. Det är ett medvetet, dokumenterat val och inte en bugg; en riktig kontroller har en mottagningskö.

D.3.1 Detaljer en drivrutin behöver

Punkterna nedan hör till kontraktet lika mycket som tabellerna ovan.

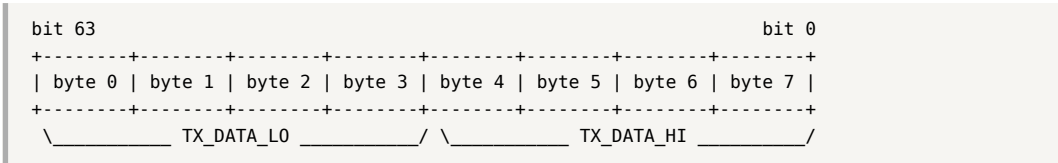
- **Varje register är 32 bitar brett.** Alla fem bytes i en transaktion överförs alltid, även för ett register som bara har fyra meningsfulla bitar.
- **STATUS bitarna 31–3 läses som 0.** Bankens läsmux nollar allt ovanför bit 2, så en jämförelse mot 0x1 är säker.
- **ERROR_FLAGS skiljer inte på felorsaker.** De fyra orsakerna — förlorad arbitrerings, stoppfel, misslyckad CRC och mottagen fjärrram — ger alla samma enda bit.
- **RX_DATA_LO/HI bortom RX_DLC läses som 0x00.** Kontrollern nollställer sin dataackumulator vid varje ramstart, så bytes ovanför den mottagna längden bär aldrig rester från en tidigare ram: en ram med DLC = 2 som följer på en med DLC = 8 läses som två bytes

- data och sex nollor. **Maskera ändå mot RX_DLC** — nollorna är kontrollernas garanti, inte ramens innehåll, och en drivrutin som kopierar åtta bytes rapporterar sex bytes som aldrig sändes.
- **TX_DATA_LO/HI bortom TX_DLC skickas aldrig**, så de behöver inte nollställas.
 - **Skrivningar till STATUS, RX_ID, RX_DLC och RX_DATA_LO/HI ignoreras tyst**. Transaktionen fullbordas normalt; inget fel rapporteras.
 - **Det finns ingen tidsgräns att härleda**. Kartan säger inte hur lång en sändning är, så en drivrutin som vill ge upp får välja sin timeout själv. Ett riktvärde: en maximal ram är omkring 130 bitar, och vid 1 Mbit/s alltså omkring 130 µs, plus väntan på en ledig buss.
 - **En förlorad arbitrering återställer STATUS bit 0**, precis som en lyckad sändning gör. En drivrutin behöver alltså inte skilja på utfallen för att kunna sända igen: polla bit 0, och läs ERROR_FLAGS för att veta hur det gick.

Vad "sändningen är slut" betyder här. Att STATUS bit 0 kommer tillbaka säger att sändarporten är fri igen, inte att ramen levererades. En förlorad arbitrering ger tillbaka samma bit som en ren sändning. ERROR_FLAGS är det som skiljer dem åt, och den vanliga pollningsslingan är därför: vänta på bit 0, läs sedan ERROR_FLAGS för att avgöra om ramen ska skickas om. Avsnitt C.6.1 listar de övriga förenklingarna mot riktig CAN.

D.4 Databyteordning

tx_data och rx_data är 64 bitar breda, med databyte 0 i de mest signifikanta bitarna:



Namnen L0 och HI syftar på registerparets ordning, inte på bitsignifikans. TX_DATA_L0 bär de *första* fyra databyten.

D.5 Motsvarigheter i hårdvaran

Register	Port på can_controller
TX_ID, TX_DLC, TX_DATA_LO/HI	tx_id, tx_dlc, tx_data
TX_SEND	tx_req (en puls)
STATUS bit 0	tx_done (en puls), låst till en nivå
RX_ID, RX_DLC, RX_DATA_LO/HI	rx_id, rx_dlc, rx_data
STATUS bit 1	rx_valid (en puls), låst till en nivå
STATUS bit 2, ERROR_FLAGS	error (en nivå), låst på stigande flank

Översättningen mellan de två kolumnerna är exakt vad `register_bank` gör, och ingenting mer.

D.6 Elektriska parametrar och ramning

Parameter	Värde
Roller	Mikrokontrollern är SPI-master, FPGA:n slav.
Läge	SPI-läge 0 (CPOL = 0, CPHA = 0): SCK vilar lågt, båda sidor samplar på stigande flank.
Bitordning	MSB först, i varje byte.
SCK-frekvens	≤ 1 MHz (AVR32DB28 går på 4 MHz och delar med 4). Se avsnitt D.8 för varför.
SS	Aktiv låg. Ramsignal: en transaktion per låg period.
Spänning	AVR32DB28:s kärna går på 5 V, DE0-CV på 3,3 V, och ingen ledning passerar en nivåomvandlare. De fyra SPI-ledningarna sitter på PORTC, AVR DB:s andra matningsdomän, med VDDI02 matad från DE0-CV:s 3,3 V-skena.

D.6.1 Pinnindelning

Båda sidor spikas fast här. Mikrokontrollersidan använder SPI0:s **ALT1**-pinnmux på AVR32DB28, som lägger periferienheten på PC0–PC3; FPGA-sidan är fri, och det här avsnittet lägger den friheten på att göra korten utbytbara.

ALT1 är inget godtyckligt val. PORTC är AVR DB:s **MVIO**-domän: den matas från VDDI02 i stället för från VDD, så att knyta VDDI02 till DE0-CV:s 3,3 V-skena lägger alla fyra SPI-ledningar i en 3,3 V-domän medan kärnan fortsätter gå på 5 V. Varje nivå FPGA:n ser är då en nivå den är specificerad för, och de 3,3 V FPGA:n driver tillbaka läses mot ett 3,3 V-tröskelvärde i stället för ett på 5 V. På SPI0:s förvalda mux skulle samma fyra signaler ligga på PA4–PA7 i VDD-domänen, och bänken skulle behöva en nivåomvandlare för att vara säker. Det är därför muxen är en del av kontraktet i stället för varje grupps eget val.

Mikrokontrollersidan är därmed skyldig två saker utöver pinnumren, och båda hör till drivrutinsklassen: PORTMUX. SPIROUTEA måste välja ALT1, och VDDI02 måste ha spänning innan PORTC gör någonting alls. MVIO.STATUS rapporterar om den har det.

Det här begränsar kortet, inte bara firmware. Hatten som bär AVR32DB28 är byggd för det här systemet, och pinntabellen nedan ställer tre krav på dess layout. PC0–PC3 bär SPI0 och ingenting annat — ingen lysdiod, ingen knapp, och framför allt ingen avstudsningsskondensator på SS, som vid 1 MHz skulle göra ramflanken oanvändbar. Ingenting annat sitter någonstans på PORTC, eftersom hela porten är en 3,3 V-domän så snart VDDI02 är det. Och VDDI02 dras som sitt eget nät ut till en pinne, avkopplad med 100 nF nära kapseln, i stället för att knytas till VDD på kortet. Ingen av de tre går att rätta i firmware när kortet väl är etsat.

En enda fuse avgör om något av ovanstående är sant. MVIO:s tvåmatningsläge väljs av MVSYS_CFG i FUSE.SYS_CFG1. I enmatningsläge knyts VDDI02 internt till VDD, PORTC blir en 5 V-port, och

kopplingen i det här avsnittet driver då 5 V in i FPGA-ingångar som inte tål 5 V — medan bänken ser exakt likadan ut som en riktig. Bekräfta fusen mot databladet innan den första noden kopplas, inte efter att det första kortet betett sig konstigt.

Signal	AVR32DB28	DE0-CV-port	DE0-CV-pinne
SCK	PC2	sclk	GPIO_0(4)
MOSI	PC0	mosi	GPIO_0(5)
MISO	PC1	miso	GPIO_0(6)
SS	PC3	ss	GPIO_0(7)
Matning I/O	VDDI02	—	3,3 V
Jord	GND	—	GND

GPIO_0(0) till GPIO_0(2) är reserverade för CAN-sidan — tx_bus, bus_en och den wired-AND:ade busslinjen — och GPIO_0(3) lämnas medvetet oanvänd som ett glapp mellan de två grupperna, så att ett kabelknippe som sitter en pinne fel inte kortsluter något mellan domänerna.

Varför alls spika fast dem, när Quartus inte bryr sig? Ingenting i konstruktionen går sönder av att en grupp väljer andra pinnar; poängen är vad som händer vid bänken. Med en tilldelning delad av båda klasserna passar vilken mikrokontrollernod som helst i vilket FPGA-kort som helst: ett kabelknippe kan byggas en gång i stället för per par, och en mjukvarugrupp vars partnerkort inte är klart kan låna ett fungerande i stället för att falla tillbaka på simulering. Det sista fallet är inte hypotetiskt — de två kurserna håller sin bring-up och sina slutredovisningar samma vecka.

En grupp *får* använda andra pinnar, men bara om kortet och den nod det paras med är överens, och bara om det är nedskrivet där den andra klassen kan se det. Ett odokumenterat pinnbyte visar sig som en nod som svarar på registerläsningar och aldrig sänder en ram, vilket är ett dyrt sätt att upptäcka en kopplingskonvention.

D.7 Transaktioner

Varje transaktion är exakt **5 bytes** lång: en kommandobyte, sedan fyra databytes. SS faller före kommandobyten och stiger efter den femte byten; den måste ligga låg hela transaktionen.

Bit:	7	6	5	4	3	2	1	0
	+	+	+	+	+	+	+	+
		w		0		0		0
								register index
	+	+	+	+	+	+	+	+

- **Bit 7 (w):** 1 = skrivning, 0 = läsning.
- **Bit 6–4:** reserverade, måste vara 0.
- **Bit 3–0:** registerindexet, offset / 4 ur avsnitt [D.1](#) (0–12).

Skrivning ($W = 1$).. Mastern skickar registervärdet i de fyra databyten, **MSB först** (bitarna 31–24 i den första databyten). Slaven verkställer det ihopsatta värdet i det adresserade registret först när den femte byten är hel. Vad slaven än driver på MISO under en skrivning är meningslöst; mastern kastar det.

Läsning ($W = 0$).. Vid slutet av kommandobyten **läser** slaven det adresserade registrets aktuella värde **en gång**, in i sitt svarsskiftregister. De fyra databyten klockar sedan ut det värdet på MISO, MSB först, medan mastern skickar dummybytes $0x00$. Regeln om att låsa en gång är ingen implementationsdetalj utan en del av kontraktet: de fyra bytena tillhör alltid ett sammanhängande sampel av registret, taget i ett enda ögonblick — STATUS i synnerhet får aldrig vara ihopsytt av två skilda ögonblick.

Avbrott.. Stiger SS innan den femte byten är hel överges transaktionen **utan sidoeffekter**: ingenting verkställs, ingen utlösare avfyras, och slaven återgår till vila, redo för nästa fallande flank på SS. Det är det som gör slaven självläkande efter en glitch eller en master som resettas.

Ogiltiga kommandon.. En läsning av ett reserverat index (13–15) ger $0x00000000$; en skrivning till ett ignoreras. En kommandobyte med någon reserverad bit (6–4) satt är ogiltig på samma sätt: ingen läsning läses och ingen skrivning verkställs, och slaven förbrukar ändå hela fembytesramen innan den återgår till vila. Inget av fallen är ett fel slaven rapporterar — det finns ingen felkanal på det här lagret. Att upprätthålla regeln om reserverade kommando-bitar är `spi_reg_bridges` jobb; `register_bank` ser aldrig mer än ett 4-bitars index.

D.8 Tidskrav

- Slaven använder **aldrig** SCK som klocka. Alla tre ingångarna (SCK, MOSI, SS) synkroniseras in i 50 MHz-domänen med två vippor var, och SCK-flanker *detekteras*, de klockas inte på (kapitel 4 för varför, kapitel 18 för modulen som gör det). `spi_slave` bygger sin synkroniserare på plats i stället för att instansiera `meta_prev`, så att `spi_slave` inte beror på något i `controller/`.
- Tvåvippssynkronisering plus flankdetektering kostar några 50 MHz-cykler per flank; vid $SCK \leq 1$ MHz finns det ≥ 50 systemcykler per SPI-bit, alltså en tiopotens marginal. Protokollet garanterar därför korrekt drift bara upp till 1 MHz; snabbare kan fungera, och det är medvetet inte utlovat.
- MISO ändras på SCK:s fallande flank och samplas av mastern på den stigande, som läge 0 föreskriver.
- Mastern måste höja SS mellan transaktioner. Transaktioner rygg mot rygg under en enda låg SS stöds inte, och att försöka är ofarligt: **en transaktion börjar bara på första byten efter att SS fallit**, så en sjätte byte som anländer medan SS fortfarande är låg kastas i stället för att tas för en ny kommandobyte.

D.8.1 Sättnings- och hålltid kring SS

Eftersom SS synkroniseras och sedan flankdetekteras i stället för att användas direkt behöver den några systemklockcykler för att få effekt. Vid 50 MHz är en cykel 20 ns, och talen är:

Parameter	Minimum	Varför
SS låg före första stigande SCK-flanken	60 ns (3 cykler)	ramstartsflanken detekteras på tredje systemklockflanken efter att SS fallit
SS hög mellan transaktioner	60 ns (3 cykler)	samma detektor måste se linjen återvända innan den kan armas igen
SS låg efter sista SCK-flanken	60 ns (3 cykler)	så att sista byten registreras innan ramen rivs

Räkna med 100 ns för var och en i en drivrutin så är ingenting av det här nära. En AVR som växlar SS genom `PORTC.OUTCLR` och `PORTC.OUTSET` kring en byte skriven till `SPI0.DATA` är redan långt långsammare än så, så gränserna spelar roll bara om mastern driver SS från hårdvara, eller går mycket fortare än AVR32DB28 gör.

Det finns **inget** minsta glapp mellan bytes: bytes får komma rygg mot rygg inom en transaktion, och slaven bryr sig inte om hur länge mastern pausar mellan dem, så länge SS ligger kvar låg.

D.8.2 MISO utanför en läsning

- MISO drivs **lågt** medan slaven är avvald, den släpps inte till högimpedans. Konstruktionen förutsätter en enda slav på bussen. Att lägga till en andra skulle kräva att den här utgången gjordes trestatad.
- Byten mastern klockar ut **under kommandobyten** är meningslös: slaven laddar sitt svarsskiftregister vid kommandobyten *slut*, så det som lämnar under den är vad som råkade ligga kvar. Kasta den.
- Under en skrivning är alla fyra MISO-databytesen meningslösa också.

D.9 Genomarbetat exempel

Att skicka en ram (id 0x123, dlc 2, data AA BB) är fem transaktioner, och sedan en pollning:

```
Write TX_ID      : 81 00 00 01 23
Write TX_DLC     : 82 00 00 00 02
Write TX_DATA_LO : 83 AA BB 00 00
Write TX_DATA_HI : 84 00 00 00 00
Write TX_SEND    : 85 00 00 00 01
Read  STATUS     : 00 xx xx xx xx  -> MISO ger 00 00 00 00 medan sändningen pågår,
                                   sedan 00 00 00 01 när tx_done satt TX klar igen.
```

Lägg märke till 83 AA BB 00 00: byte 0 i den mest signifikanta positionen. Det är packningen på båda sidor om ledningen — drivrutinen sätter ihop den, `register_bank` presenterar

TX_DATA_L0 som de *första* fyra bytesen, och can_controller läser tx_data med byte 0 i bitarna 63–56. Diagrammet i avsnitt [D.4](#) är bilden av det, och register_bank_tb kontrollerar det.

Värt att läsa av samma exempel: de fyra första MIS0-bytesen av en skrivning är meningslösa, och det är MIS0-byten som returneras *under* vilken kommandobyte som helst också. Kasta den.

Att ändra i det här kontraktet. Registerkartan och protokollet är ett kontrakt mellan två klasser som inte kan kompilera mot varandra. Ändras något måste båda sidor veta om det innan ändringen mergas. I praktiken: ändringar diskuteras med båda handledarna först, och dokumenten uppdateras *före* koden, aldrig efter.

Vidare läsning

Den här boken slutar med en CAN-nod som en mikrokontroller kan prata med, och den slutar där med flit: konstruktionen är förenklad på flera medvetna punkter (avsnitt 19.7), och drivrutinen på andra sidan SPI-länken skrivs av någon annan. Den här bilagan säger var grunderna boken förutsätter kan repeteras, var protokollet självt står beskrivet, vilka standarder konstruktionen förhåller sig till, och vart man går härnäst.

E.1 Grunderna

Boken förutsätter digitalteknikens grunder och repeterar dem bara där språket kräver det. *Digital Electronics* (<https://github.com/qrtech-academy/digital-electronics>) är boken att gå till när de behöver fräschas upp. Den är på engelska, fritt tillgänglig, och slutar ungefär där den här boken börjar: med en tillståndsmaskin byggd av grindar och vippor, simulerad i samma CircuitVerse.

Sex kapitel plus bilagor, och varje kapitel motsvarar något den här boken tar för givet:

Kap.	Ämne	Här i boken
1	Logiska grindar, och transistorerna och fördröjningen i dem	avsnitt 1.1, avsnitt 5.4
2	Boolesk algebra, De Morgan, talsystem och koder	avsnitt 1.2
3	Karnaughdiagram och minimering	avsnitt 2.1, avsnitt 8.2
4	Kombinatoriska byggblock: multiplexer, avkodare, komparator, adderare	avsnitt 2.4
5	Vippor, register, metastabilitet och synkronisering	kapitel 3 och 4
6	Räknare, timers och tillståndsmaskiner	kapitel 6, 7 och 8
A	Svar på övningarna	—
B	CircuitVerse	avsnitt 1.3

Bilagorna efter B är två skrivningar med lösningsförslag. De är det snabbaste sättet att pröva om förkunskaperna sitter innan kapitel 1, och att skriva en av dem på tid visar tydligare än en genomläsning vilka av kapitlen ovan som är värda en repetition.

E.2 Protokollreferensen

CAN: bussen, ramen och kontrollern är den bok den här boken lutar sig mot varje gång CAN som *protokoll* kommer på tal. Den är kort och skriven för båda halvorna av ett CAN-system: den som bygger kontrollern i hårdvara och den som skriver drivrutinen i mjukvara. Den nämner därför varken kurs eller klass, och den är avsiktligt oberoende av vilken av de två kurserna som läser den.

Den finns i två utgåvor med samma kapitel- och avsnittsnumrering, så varje hänvisning i den här boken gäller båda: på svenska på <https://github.com/Yrgo-26/can-book/blob/main/sv/can-sv.pdf>, och på engelska, som *CAN: the bus, the frame and the controller*, på <https://github.com/Yrgo-26/can-book/blob/main/en/can-en.pdf>.

Sju kapitel plus ett appendix, och kapitelindelningen är den här bokens del II sedd från protokollhållet:

Kap.	Ämne	Här i boken
1	Bussen: topologi, dominant och recessiv, wired-AND, terminering	avsnitt 10.2
2	Framen: fälten, identifieraren, RTR, DLC, kvittensen	avsnitt 10.4
3	Bitstoppning och CRC-15	avsnitt 10.5, kapitel 13, 14 och 15
4	Arbitrering, bit för bit	avsnitt 10.6, avsnitt 17.6
5	Bittajming och synkronisering	avsnitt 10.7, avsnitt 12.2
6	Felhantering: felramar, felräknare, bus-off	avsnitt 19.7, som lucka
7	Den förenklade kontrollern, och de tre begränsningar som når ända upp	avsnitt 19.7, avsnitt C.6.1, bilaga D
A	Svar på samtliga övningar	—

Tre saker gör den värd att ha uppslagen bredvid den här boken snarare än läst en gång:

- **Kapitel 1–6 är kravspecifikationen, bit för bit**, för den kontroller del II bygger. Där den här boken säger "regeln är fem lika bitar i rad", säger den boken varför regeln finns och vad som går sönder utan den.
- **Kapitel 7 beskriver exakt den här konstruktionen**, utifrån: blocken, vad den implementerar av standarden, vad den medvetet låter bli, varför ett registerlager behövs, och de tre begränsningar som når ända upp i drivrutinens kod. Den som vill ha helheten på fem sidor innan kapitel 11:s arkitektur läser det kapitlet först.
- **Fyra övningar per kapitel, alla med papper och penna, alla besvarade i appendix A.** De prövar protokollförståelsen utan att en rad VHDL är inblandad, vilket gör dem till den bästa förberedelsen som finns inför kapitel 20:s läsuppgifter.

Böckerna delar författare och sätts på samma sätt, så en referens ser likadan ut i båda. De numrerar däremot sina kapitel oberoende av varandra: "kapitel 3" utan vidare bestämning betyder alltid ett kapitel i *den här* boken, och en hänvisning till den andra skrivs alltid ut i sin helhet.

E.3 Standarder och specifikationer

Boken förhåller sig hela tiden till riktig CAN utan att vara den, och *CAN: bussen, ramen och kontrollern* är mellansteget: den beskriver standarden i den detaljnivå en konstruktör behöver. Vill du därefter se vad som faktiskt står i originalet:

- **Bosch CAN Specification 2.0** (del A och B) är grunddokumentet: ramformaten, bit-stopppningen, CRC-polynomet och arbitreringen. Del A är standardramen med 11-bitars identifierare, alltså den här bokens ram; del B lägger till den utökade 29-bitars formen.
- **ISO 11898-1** är standardiseringen av samma sak, med datalänklagret och det fysiska signalgränssnittet. ISO 11898-2 är höghastighetstransceivern, alltså den krets som skulle sitta mellan tx_bus/bus_en och en verklig differentiell buss.
- **IEEE 1076-1993** är VHDL-93, språkstandarderna --std=93 väljer. Den är läsvärd som uppslagsverk snarare än pärm till pärm; paketen `ieee.std_logic_1164` och `ieee.numeric_std` är de två delar du använt genom hela boken.

De fyra punkter där bokens kontroller medvetet avviker från standarden är samlade i avsnitt 19.7, och avsnitt C.6.1 säger vilka tre av dem som syns ända ut i registerkartan.

E.4 Verktyn

- **GHDL:s dokumentation** täcker betydligt mer än de tre kommandon bilaga A behöver: vågformsutmatning till VCD eller GHW, kodtäckning, och de andra språkstandardsflaggorna. **GTKWave** är den vanliga vågformsvisaren att para ihop den med, och första gången du felsöker en tajmingbugg genom att titta på signalerna i stället för att läsa report-rader är den värd sin installationstid.
- **Quartus Prime Lites egen dokumentation**, i synnerhet tidsanalysen: bilaga B nöjer sig med ett `create_clock`, medan en riktig konstruktion också behöver ingångs- och utgångskrav och undantag för korsande klockdomäner.
- **DE0-CV:s användarmanual** från korttillverkaren har den fullständiga pinntabellen, kringkretsarna och kopplingsschemat. Den är facit när `DE0_CV.qsf` inte täcker något du vill använda.
- **CircuitVerse** är gratis och körs i webbläsaren. Det är fortfarande det snabbaste sättet att pröva en liten kombinatorisk idé, och kapitel 9 och 20 tillåter det av just det skälet.

E.5 Vidare i digital konstruktion

Det här är en bok om ett system, inte om hela ämnet. Fyra riktningar som ligger nära och som material i övrigt inte saknas för:

- **Tidsanalys och klockdomäner på allvar.** Kapitel 4 ger dubbelvippsynkroniseraren och avsnitt 5.4 en första Fmax. Nästa steg är setup- och holdanalys, FIFO:er mellan klockdo-

mäner, och gray-kodade pekare — det vill säga vad man gör när en hel buss, och inte en ensam bit, ska passera mellan två klockor.

- **Verifiering bortom en handskriven testbänk:** självkontrollerande modeller, slumpstimuli och täckningsmått. VHDL:s OSVVM och UVVM är de två vanliga ramverken, och steget därifrån till SystemVerilog och UVM är det industrin tar.
- **Andra gränssnitt, med samma metod.** Boken bygger CAN; UART, SPI, I²C och Modbus har alla samma form — en bittimer, ett skiftregister, en sekvenserare — och att bygga en till av dem med bokens mönster är den snabbaste vägen att märka hur mycket av kapitel 11:s uppdelning som var allmängods.
- **Processorkonstruktion.** En liten RISC-kärna är nästa naturliga steg för den som tyckt att tillståndsmaskinen i kapitel 16 var det intressantaste: samma byggstenar, en instruktionsavkodare i stället för en ramavkodare, och ett registerfilsgränssnitt i stället för en buss.

Gemensamt för alla fyra är att de vilar på samma fem mönster kapitel 20 räknar upp. Det är den egentliga behållningen av den här boken: inte CAN, utan att du kan bygga nästa protokoll också.